MAYANK SHRIVASTAVA · PREP 2026–27

SDE2 Java Interview Study Guide

Core Java · Concurrency · JVM · Spring Boot · SQL & Databases · LLD · HLD · Behavioral

Everything in an SDE2 loop except DSA. Every topic: Why → Internals → How → Example → When → What can go wrong → Interview question.
Edition: October 2026 · Java 21 · Spring Boot 3.x / 4.x

Contents

Part 0 — Start Here

0. How to Use This Study Guide

What this guide is
How every topic is explained
The honest starting point
Daily rhythm
The 16-week plan
How to study a section (and remember it)
Free resources

Part 1 — Java, Deeply

1. Core Java

1.1 How Java runs: JDK, JRE, JVM, bytecode and JIT
1.2 Primitives, wrappers, autoboxing and the Integer cache
1.3 Integer overflow and numeric conversions
1.4 Strings, the String pool and immutability
1.5 StringBuilder and string concatenation
1.6 Stack vs heap, and why Java is always pass-by-value
1.7 Classes, objects, constructors, static, final and initialisation order
1.8 Access modifiers and encapsulation
1.9 Inheritance and polymorphism (overloading vs overriding)
1.10 Interfaces vs abstract classes
1.11 Composition vs inheritance
1.12 equals() and hashCode()
1.13 Comparable and Comparator
1.14 Immutable objects
1.15 Exceptions: checked vs unchecked, try-with-resources
1.16 Generics, type erasure and wildcards (PECS)
1.17 Nested, inner, local and anonymous classes
1.18 Enums
1.19 Annotations and reflection

2. Collections Framework Internals

2.1 The collections hierarchy and choosing the right one
2.2 ArrayList
2.3 LinkedList
2.4 HashMap internals
2.5 LinkedHashMap and building an LRU cache
2.6 TreeMap and TreeSet
2.7 HashSet, LinkedHashSet and EnumSet
2.8 ArrayDeque — your stack and your queue
2.9 PriorityQueue — a binary heap
2.10 Iterators: fail-fast vs fail-safe
2.11 Immutable collections, utilities and sorting

3. Modern Java: 8 to 21

3.1 Lambdas and functional interfaces
3.2 Streams: the pipeline model
3.3 Collectors and groupingBy
3.4 Parallel streams
3.5 Optional
3.6 The java.time API
3.7 Records
3.8 Sealed classes and pattern matching
3.9 Switch expressions, var and text blocks
3.10 What else is new in Java 21

4. Concurrency and Multithreading

4.1 Threads and the thread lifecycle
4.2 Race conditions: atomicity, visibility and ordering
4.3 Executors and ThreadPoolExecutor
4.4 Callable, Future and their limits
4.5 synchronized, monitors, and wait/notify
4.6 The Java Memory Model, volatile and happens-before
4.7 Explicit locks: ReentrantLock, ReadWriteLock, StampedLock, Condition
4.8 Atomics and compare-and-swap (CAS)
4.9 ConcurrentHashMap internals
4.10 Synchronizers: CountDownLatch, CyclicBarrier, Semaphore
4.11 CompletableFuture: composing asynchronous work
4.12 Virtual threads (Java 21)
4.13 Deadlock, livelock and starvation
4.14 Producer–consumer, BlockingQueue and classic problems
4.15 ThreadLocal

5. JVM Internals, Garbage Collection and Production Troubleshooting

5.1 Class loading
5.2 Runtime memory areas
5.3 Object allocation and the generational heap
5.4 How garbage collection finds and frees garbage
5.5 The garbage collectors: Serial, Parallel, G1, ZGC
5.6 Strong, soft, weak and phantom references
5.7 Memory leaks in a garbage-collected language
5.8 Diagnosing high CPU, hangs and slow requests
5.9 Diagnosing memory problems: GC logs, heap dumps, JFR
5.10 JIT compilation in depth
5.11 The JVM in containers and the flags that matter

Part 2 — Spring Boot

6. Spring Core and Spring Boot Fundamentals

6.1 What Spring and Spring Boot solve
6.2 Inversion of Control and Dependency Injection
6.3 Defining beans: stereotypes, @Bean and @Configuration
6.4 Multiple implementations: @Primary, @Qualifier, and injecting collections
6.5 Bean scopes
6.6 Bean lifecycle and post-processors
6.7 Proxies and AOP: how @Transactional, @Async and @Cacheable really work
6.8 Configuration, @ConfigurationProperties and profiles
6.9 Auto-configuration and starters
6.10 Startup sequence and application events

7. Spring Web and Data: REST, Validation, JPA, Transactions

7.1 How a request flows through Spring MVC
7.2 REST controllers and HTTP semantics
7.3 Validation
7.4 Centralised exception handling
7.5 Filters vs interceptors
7.6 Calling other services: RestClient, timeouts and connection pools
7.7 JPA, Hibernate and Spring Data repositories
7.8 Relationships and fetch types
7.9 The persistence context, entity states and dirty checking
7.10 The N+1 query problem and fetching strategies
7.11 Lazy loading, LazyInitializationException and Open-Session-in-View
7.12 @Transactional: how it works and its rollback rules
7.13 Transaction propagation
7.14 Isolation levels and concurrency anomalies
7.15 Optimistic locking, pessimistic locking and atomic updates
7.16 Connection pools, migrations and JPA performance settings

8. Spring in Production: Security, Testing, Messaging, Caching, Resilience, Observability

8.1 Spring Security architecture
8.2 JWT, OAuth2 and OpenID Connect
8.3 Unit testing with JUnit 5 and Mockito
8.4 Slice tests and integration tests with Testcontainers
8.5 Kafka with Spring: producers, consumers, idempotency and dead letters
8.6 Caching with Spring Cache and Redis
8.7 Resilience: timeouts, retries, circuit breakers and bulkheads (Resilience4j)
8.8 Async execution and scheduled jobs
8.9 Observability: health, metrics, logs and traces
8.10 Microservices patterns in the Spring ecosystem
8.11 The Java project — your proof of the switch

Part 3 — Low-Level Design

9. LLD Foundations: OOP, SOLID and Design Patterns

9.1 OOP pillars as design tools
9.2 Class relationships and UML in interviews
9.3 Single Responsibility Principle (SRP)
9.4 Open/Closed Principle (OCP)
9.5 Liskov Substitution Principle (LSP)
9.6 Interface Segregation Principle (ISP)
9.7 Dependency Inversion Principle (DIP)
9.8 Singleton
9.9 Factory patterns: simple factory, factory method, abstract factory
9.10 Builder
9.11 Prototype
9.12 Adapter
9.13 Decorator
9.14 Proxy
9.15 Facade
9.16 Composite
9.17 Strategy
9.18 Observer
9.19 State
9.20 Command
9.21 Chain of Responsibility

9.22 Template Method
9.23 Specification, and patterns to recognise
9.24 Supporting principles interviewers listen for
9.25 How to run an LLD interview (60–90 minutes)

10. LLD Problems, Solved in Java

10.1 Tic-Tac-Toe
10.2 Parking Lot
10.3 LRU Cache
10.4 Logger Framework
10.5 Rate Limiter
10.6 Bounded Blocking Queue and a Job Processing System
10.7 Vending Machine
10.8 Elevator System
10.9 Splitwise (Expense Sharing)
10.10 BookMyShow (Movie Ticket Booking)
10.11 Loyalty Points Ledger (your domain)
10.12 Rules Engine for Incentives (your domain)
10.13 Snake and Ladder
10.14 In-Memory Pub-Sub (a mini Kafka)
10.15 Library Management System
10.16 Coffee Machine
10.17 Cab Booking (Uber/Ola)
10.18 More problems to practise

Part 4 — High-Level Design

11. HLD Fundamentals

11.1 The system design interview framework
11.2 Back-of-envelope estimation
11.3 Scaling basics: vertical, horizontal and stateless services
11.4 Load balancing
11.5 Caching
11.6 CDNs and the edge
11.7 Choosing a database
11.8 Replication
11.9 Partitioning and sharding
11.10 CAP, PACELC and consistency models
11.11 Consistent hashing
11.12 Message queues and asynchronous processing (RabbitMQ and friends)
11.13 Kafka in depth
11.14 Idempotency and delivery guarantees in practice
11.15 The transactional outbox and sagas
11.16 Rate limiting and API gateways
11.17 API design: REST, gRPC, GraphQL and real-time
11.18 Storage for files, search and analytics
11.19 Reliability patterns
11.20 Observability, SLIs and SLOs
11.21 Security in system design
11.22 Generating unique IDs at scale

12. Databases and SQL Internals

12.1 Joins and how the engine executes them
12.2 Window functions
12.3 CTEs and recursive queries
12.4 Classic SQL interview problems and evaluation order
12.5 How indexes work: B+ trees, clustered and covering indexes
12.6 Query plans and query optimisation
12.7 Transactions, MVCC, locks and database deadlocks
12.8 Data modelling: normalisation, denormalisation and table partitioning
12.9 OLTP vs OLAP: where ClickHouse and Trino fit

13. System Design Case Studies

13.1 URL Shortener (TinyURL / bit.ly)
13.2 Distributed Rate Limiter
13.3 Notification System
13.4 News Feed (Twitter/LinkedIn/Instagram home feed)
13.5 Chat System (WhatsApp / Slack)
13.6 Distributed Cache (design Redis/Memcached)
13.7 Loyalty and Rewards Platform (your system — rehearse it three levels deep)
13.8 Payment System
13.9 Leaderboard (gaming / sales incentives)
13.10 Typeahead / Search Autocomplete
13.11 Metrics and Logging Pipeline
13.12 Distributed Job Scheduler (cron at scale)
13.13 Ride Matching and Nearby Search (Uber, delivery partner assignment)
13.14 Incentive Engine at Scale (your system)
13.15 Instagram (photo sharing + feed)
13.16 YouTube / Netflix (video upload and streaming)
13.17 Twitter / X — what's specific beyond the generic feed
13.18 Food Delivery (Swiggy / Zomato)
13.19 E-commerce Checkout and Flash Sales (Flipkart Big Billion Days)
13.20 More systems to practise

Part 5 — Interviews: Stories, Scripts and Revision

14. Behavioral Rounds and Your Resume

14.1 STAR, done properly
14.2 Your story bank — build it from your own work
14.3 Amazon Leadership Principles (used, in spirit, by most product companies)
14.4 The questions you will definitely get
14.5 Resume rules for SDE2 product-company applications
14.6 Getting interviews: referrals and recruiters

15. The Interview Playbook: Never Go Blank

15.1 Why people blank, and the fixes
15.2 Coding round script (45 minutes)
15.3 Coding round: what this guide adds to your DSA prep
15.4 LLD round script (60–90 minutes)
15.5 HLD round script (45–60 minutes)
15.6 Java/Spring theory round
15.7 Mock interviews: how to run them and score them
15.8 Online assessments (OA)
15.9 The day before and the day of
15.10 The weekly review (Sunday, 1 hour)

16. Rapid Revision Bank

16.1 Core Java
16.2 Collections
16.3 Java 8–21
16.4 Concurrency
16.5 JVM
16.6 Spring Boot and JPA
16.7 LLD
16.8 HLD
16.9 Databases and SQL

Part 0 — Start Here

This part has the plan, the daily routine and the rules. Read it fully once. Then come back every Sunday and tick off what you finished that week.

0. How to Use This Study Guide

What this guide is

This guide covers everything in an SDE2 / Senior Software Engineer interview loop **except DSA**. You have a separate resource for DSA.

It is written for a .NET engineer who is moving to Java. It assumes only general programming experience — nothing more.

Area	Where
Core Java, how collections work inside, Java 8–21	Part 1: Chapters 1–3
Concurrency, JVM, GC, troubleshooting	Part 1: Chapters 4–5
Spring core, web, data/JPA/transactions, production Spring (security, testing, Kafka, Redis, resilience, observability) + the portfolio project	Part 2: Chapters 6–8
Low-level design: OOP, SOLID, 25 patterns, 17 machine-coding problems (all code compiled and run)	Part 3: Chapters 9–10
High-level design basics, SQL and how databases work inside, 19 case studies	Part 4: Chapters 11–13
Behavioral, resume, interview scripts, rapid revision	Part 5: Chapters 14–16

How every topic is explained

Every topic follows the same seven steps. So you always know where you are. You also know what an interviewer can dig into next.

1. **Why?** — the problem that made this thing exist. If you know the problem, you can rebuild the solution even under pressure.
2. **What is happening internally?** — what sits underneath: the data structures, how memory is laid out, the threads, or the network hops.
3. **How does it work?** — the mechanism, step by step.
4. **Example** — runnable Java (or SQL / config). Type it out yourself and run it.
5. **When would I use it?** — the rule for deciding, and what you would pick instead.
6. **What can go wrong?** — the bugs you see in production and the traps interviewers set.
7. **Interview question** — asked the way interviewers ask it, with a model answer.

Use the same order when you answer in an interview: problem → internals → mechanism → example → trade-off → what can fail. This structure is what makes an answer sound like SDE2 level. A plain definition does not.

The honest starting point

Your resume says "Solved 500+ DSA problems". Remove that line now.

Here is why. Your LeetCode and GFG links sit right above that line. Interviewers at product companies open those links. If the profile shows far fewer solved problems, they stop trusting you in the first five minutes. After that, they doubt every other line on the resume too.

Put a true number back once you have one. Also, before you apply, make sure the "Java, Spring Boot" skills line is backed by the project in Chapter 8.11.

> **Rule:** Put nothing on the resume that you cannot defend for 10 minutes of follow-up questions. Interviewers are trained to pick one claim and keep digging until it either holds or breaks.

Daily rhythm

This plan is built for someone with a full-time Senior role: **~2 hours on weekdays, ~5 hours on weekend days**.

DSA time is shown only so the week adds up. In that slot, follow your own DSA resource.

Slot	Mon–Fri	Saturday	Sunday
DSA (your own resource)	1 h	2 h	1 h
Java / Spring / DB section of the week (this guide)	1 h	—	—
LLD (Chapters 9–10)	—	3 h	—
HLD (Chapters 11, 13)	—	—	2 h
Java project (Weeks 8–12), then mocks (Weeks 13–16)	—	—	2 h
Flashcard review (Anki deck that comes with this guide)	15 min daily	15 min	15 min

Weekly metric — 1 new + 2 revised. In each study session, do one new section. Then answer again the interview questions from two sections you studied earlier. Answering an old section aloud, without notes, is worth more than reading a new one.

Stuck rule: Sometimes a section won't make sense after one careful read. Then run its code, change one thing, and guess the output before you run it again. If it still doesn't make sense, mark it and move on. Come back to it on Saturday.

The 16-week plan

Week 1 starts on Monday, Oct 5, 2026. You start applying in Week 13. Interviews start from late January 2027.

Wk	Dates	Weekdays: Java / Spring / DB	Sat: LLD	Sun: HLD	End-of-week output
1	Oct 5–11	1.1–1.8 Java basics, JVM view, String, equals/hashCode	9.1–9.7 OOP pillars, UML, SOLID	11.1–11.3 Approach, estimation	Resume line removed
2	Oct 12–18	1.9–1.19 Generics, exceptions, immutability, I/O	9.8–9.11 Creational patterns	11.4–11.6	Explain equals/hashCode + erasure aloud

Wk	Dates	Weekdays: Java / Spring / DB	Sat: LLD	Sun: HLD	End-of-week output
3	Oct 19–25	2.1–2.11 Collections internals	9.12–9.16 Structural patterns	11.7–11.9	Explain HashMap aloud in 3 min
4	Oct 26–Nov 1	3.1–3.10 Java 8–21, streams, records, virtual threads	9.17–9.25 Behavioral patterns, LLD method · 10.1–10.2	11.10–11.11	**Gate 1**
5	Nov 2–8	4.1–4.8 Threads, executors, JMM, locks, AQS	10.3–10.5	11.12–11.15	Thread-pool params from memory
6	Nov 9–15	4.9–4.15 CHM, CompletableFuture, deadlock, producer–consumer, ThreadLocal	10.6–10.8	11.16–11.19	Bounded blocking queue coded alone
7	Nov 16–22	5.1–5.11 JVM, GC, troubleshooting	10.9–10.11	11.20–11.22	Read a GC log + thread dump
8	Nov 23–29	6.1–6.10 Spring core, proxies, AOP, auto-config	10.12–10.14	13.1–13.3	**Gate 2**; project repo created (8.11)
9	Nov 30–Dec 6	7.1–7.6 Web, validation, errors · 12.1–12.4 SQL joins, window functions, CTEs	10.15–10.16	13.4–13.6	Project: entities + APIs
10	Dec 7–13	7.7–7.16 JPA, transactions, locking, Flyway · 12.5–12.9 indexes, EXPLAIN, MVCC, DB locks	10.17–10.18	13.7–13.9	Project: transactional ledger
11	Dec 14–20	8.1–8.5 Security, testing, Kafka	Timed redo: Parking Lot (90 min)	13.10–13.13	Project: tests + Kafka
12	Dec 21–27	8.6–8.10 Redis, resilience, scheduling, observability	Timed redo: BookMyShow	13.14–13.16	**Gate 3**; project live
13	Dec 28–Jan 3	Ch 16 Java + concurrency bank	Timed redo: Elevator	13.17–13.20	Resume final; 15 applications/referrals out
14	Jan 4–10	Ch 16 Spring + DB bank · Ch 14 story bank written	Timed redo: Rules engine	Redo 2 case studies on a blank page	First mock
15	Jan 11–17	Ch 15 playbook drills	Timed redo: your weakest problem	Redo 2 case studies	2 mocks
16	Jan 18–24	Ch 16 full pass, flashcards	Redo 2 favourites	Redo 2 case studies	2 mocks; interviews begin

Gates — do not move on until they pass

A gate is a self-test at the end of a phase. If you can't pass it, spend more time on that phase before moving ahead.

- **Gate 1 (end of Week 4):** Without notes, explain these aloud:
- HashMap internals (hashing, treeify, resize);

- the equals/hashCode contract;
- why String is immutable, and the String pool;
- checked vs unchecked exceptions;
- why generics are erased, and what PECS means;
- how a stream pipeline runs lazily (only when a final operation asks for results).
- **Gate 2 (end of Week 8):**
- Code a thread-safe bounded blocking queue and a producer–consumer from scratch in 30 minutes.
- Explain how ThreadPoolExecutor behaves with core size, max size, queue and rejection.
- Explain `volatile` vs `synchronized`.
- Explain why `@Transactional` does nothing when a method calls another method in the same class (self-invocation).
- Finish six LLD problems in Java.
- **Gate 3 (end of Week 12):**
- Java project deployed, with a README, tests and a load-test number.
- From memory, explain the Spring bean lifecycle, `@Transactional` propagation and the N+1 fix.
- Write a window-function query that returns the top 3 per group.
- Explain why a given index is or isn't used.
- Present one HLD case study end to end in 40 minutes.

How to study a section (and remember it)

1. Read the section once, slowly. Run every code block (IntelliJ Community is free).
2. Close the guide. Say the seven steps for the topic aloud, in about two minutes.
3. Answer the section's interview question aloud in ~90 seconds. Then compare it with the model answer. Mark the ones where you stumble.
4. Review the flashcards for 15 minutes every day. This is spaced repetition — seeing a card again just before you would forget it. It is what prevents the "I knew this yesterday" blank.

> **Remember:** Recognising something is not the same as recalling it. Re-reading feels productive, but it leaves you blank in interviews. Closing the guide and pulling the answer out of your own head is what builds recall.

Free resources

- **Official docs** for deep dives: docs.oracle.com (Java), docs.spring.io (Spring Boot), postgresql.org/docs, kafka.apache.org.
- **Peers** for mocks — from Week 13, trade a 1-hour mock with a peer every week.
- **Anki (free)** — import the flashcard CSV that comes with this guide.

You don't need any paid course for anything in this plan.

Part 1 — Java, Deeply

Every topic in this part follows the same seven steps:

1. **Why?** — the problem it solves.
2. **What is happening internally?** — the machinery underneath.
3. **How does it work?** — step by step.
4. **Example** — code, plus a walk-through of what happens when it runs.
5. **When would I use it?**
6. **What can go wrong?**
7. **Interview questions** — with answers.

The first time, read the steps in order. When revising, jump straight to *What can go wrong* and the questions.

1. Core Java

1.1 How Java runs: JDK, JRE, JVM, bytecode and JIT

Why?

> **In simple words:** Java compiles your code into a common "middle language" called bytecode. A program called the JVM runs that bytecode on any machine, and it speeds up the busy parts while the program runs.

A C or C++ program is compiled straight into machine code. That machine code is for one CPU and one operating system. So a binary built for Linux x86 won't run on Windows or on ARM.

Java was designed for "write once, run anywhere". You compile once, and the same file runs on any machine.

Java also wanted two more things: - **Memory safety** — no pointer arithmetic, and no manual `free`. - **Optimisation while the program runs.**

Both need a layer between your code and the hardware. That layer is the **Java Virtual Machine** (JVM).

Think of a film with subtitles. The film (bytecode) is the same in every country. Only the subtitle track (the JVM) changes for each local audience.

What is happening internally?

```
 Hello.java ──javac──► Hello.class (bytecode) ──► JVM
                                                  ├─ Class loader: finds and loads .class files
                                                  ├─ Bytecode verifier: rejects unsafe/corrupt code
                                                  ├─ Interpreter: executes bytecode instruction by
instruction
                                                  ├─ JIT compiler (C1, C2): turns HOT methods into native
machine code
                                                  ├─ Garbage collector: frees unreachable objects
                                                  └─ Runtime data areas: heap, stacks, metaspace
```

- **Bytecode** is a set of instructions for an imaginary *stack machine*. A stack machine does its work by pushing values onto a stack and popping them off.
- For example, `iload_1` pushes local variable 1 onto the operand stack.
- `iadd` pops two ints and pushes their sum.
- Bytecode doesn't depend on any real CPU.
- **JVM** is a program that understands bytecode. A separate JVM is written for each OS and CPU. This is where portability comes from: the bytecode is the same everywhere, and only the JVM differs.
- **JRE** = JVM + the standard library (`java.lang`, `java.util`, ...).
- **JDK** = JRE + developer tools (`javac`, `jshell`, `jar`, `jcmd`, `jstack`, `jmap`, `jfr`). Since Java 11, you install only the JDK.
- **JIT (Just-In-Time) compiler** — it compiles code to machine code *while the program runs*: 1. The JVM counts how often each method and loop runs. 2. When a method crosses a threshold, **C1** compiles it to native code. C1 compiles fast but optimises lightly. 3. If the method stays busy ("hot"), **C2** compiles it again. C2 is slower to compile but optimises heavily: inlining (copying a called method's body into the caller), loop unrolling, and escape analysis (spotting objects that never leave a method). 4. This two-level setup is called **tiered compilation**.

How does it work?

1. `javac Hello.java` reads the source, checks the types, and writes `Hello.class`. No machine code is made yet.
2. `java Hello` starts the JVM. The class loader loads `Hello.class`. It also loads what it needs, such as `java.lang.Object` and `String`.
3. The verifier checks that the bytecode is type-safe. For example, no jumping into the middle of an instruction, and no treating an int as a reference.
4. `main` starts running in the **interpreter**. The interpreter runs bytecode one instruction at a time.
5. Counters go up as code runs. Hot methods are queued for JIT compilation on background threads. When compilation finishes, later calls jump to the native code.
6. Sometimes an optimisation assumption breaks. For example, the JIT turned a call into a direct call because there was only one possible target. Then a new subclass is loaded, and now there are two targets. When this happens, the JVM **deoptimises** — it throws away that compiled code and goes back to the interpreter. It may recompile later.

Example

```
public class Hello {
    static int square(int x) { return x * x; }
    public static void main(String[] args) {
        long sum = 0;
        for (int i = 0; i < 50_000_000; i++) sum += square(i % 1000);  // square() becomes hot -> JIT-
compiled, then inlined
        System.out.println(sum);
    }
}
// javac Hello.java && java Hello
// javap -c Hello      -> prints the bytecode of square():  iload_0, iload_0, imul, ireturn
// java -XX:+PrintCompilation Hello   -> shows square() being compiled by tier 3 (C1) then tier 4 (C2)
```

What happens when it runs: 1. The first few thousand loop iterations run in the interpreter. 2. Once `square` crosses the C1 threshold, it is compiled. 3. Once the loop is hot, C2 recompiles `main` with `square` **inlined**. Now there is no method call at all, so no call overhead.

This is why a "warmed-up" Java service is faster than one that just started.

When would I use it?

You use this every day without thinking about it. It matters in interviews and in production when you: - explain why the first requests after a deploy are slow (the JIT and caches are still cold), and why load tests need a warm-up period; - use `javap -c` to see what the compiler really produced (for string concatenation, autoboxing, lambdas); - choose startup optimisations (Class Data Sharing, GraalVM native image) for short-lived jobs or serverless functions.

What can go wrong?

- **Micro-benchmarks lie.** Say you time a loop with `System.nanoTime()`. You may be measuring the interpreter, or JIT compilation itself, or dead-code elimination. (Dead-code elimination means the JIT removes code whose result is never used — so you end up timing nothing.) Use **JMH** for benchmarks.
- **"Java is slow"** is usually a cold-start problem or a badly configured GC. It is rarely a JIT problem.
- **Version mismatch.** Code compiled for Java 21 (class file version 65) fails on a Java 17 JVM with `UnsupportedClassVersionError`. The build target and the runtime image must match.

Interview questions

Q1. Is Java compiled or interpreted? Both. `javac` compiles source code ahead of time into bytecode, which is the same on every platform. At runtime, the JVM interprets that bytecode, and it JIT-compiles hot methods into native machine code using profiling data (counts of what actually runs).

Q2. JDK vs JRE vs JVM? The JVM runs bytecode. The JRE is the JVM plus the standard libraries. The JDK is the JRE plus development tools such as `javac`, `jcmd` and `jstack`.

Q3. What makes Java platform-independent? Bytecode is the same on every platform. Only the JVM implementation is different for each platform.

Q4. What is tiered compilation? Code starts in the interpreter. Hot methods are compiled quickly by C1, which also collects profiling data. The hottest methods are compiled again by C2 with aggressive optimisations. If an assumption becomes invalid, the code can be deoptimised back to the interpreter.

1.2 Primitives, wrappers, autoboxing and the Integer cache

Why?

> **In simple words:** Java has fast plain numbers (`int`) and slower "boxed" numbers (`Integer`) that are real objects. Collections need the boxed kind, and Java converts between the two for you — which sometimes causes surprising bugs.

Collections and generics work only with **objects**. But doing arithmetic on objects is slow. Every number would need its own memory on the heap, and the CPU would have to follow a pointer to read it.

So Java keeps **8 primitive types** for fast plain values. Each one also has a **wrapper class**, so the value can live inside a collection. **Autoboxing** does the conversion for you, so you don't have to write it by hand.

Think of a tiffin box. The roti (the `int`) is the actual food. The tiffin box (the `Integer`) lets you pack it and put it in a bag (a collection). But it takes extra space, and you must open it before eating.

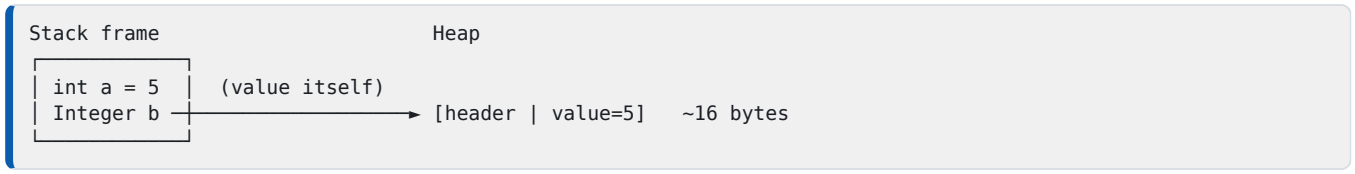
What is happening internally?

Primitive	Size	Wrapper	Default value
`byte` / `short` / `int` / `long`	8 / 16 / 32 / 64 bit	`Byte` / `Short` / `Integer` / `Long`	0
`float` / `double`	32 / 64 bit IEEE-754	`Float` / `Double`	0.0
`char`	16 bit unsigned UTF-16 unit	`Character`	`'\u0000'`
`boolean`	Depends on the JVM	`Boolean`	false

A primitive local variable sits **directly in the stack frame**. `int x = 5` is just 4 bytes that hold 5.

An `Integer` is an **object on the heap**. It takes about 16 bytes: - an object header (~12 bytes), - a 4-byte `int value` field, - plus padding.

Your variable holds only a *reference* (a pointer) to that object.



Autoboxing is a shortcut the compiler writes for you: - `Integer b = 5;` compiles to `Integer b = Integer.valueOf(5);` - `int c = b;` compiles to `int c = b.intValue();`

The Integer cache: `Integer.valueOf(int)` keeps a static array of ready-made `Integer` objects for **−128 to 127**. - For a value in that range, it returns the shared cached object. - Outside that range, it creates a new object every time. - You can raise the upper limit with `-XX:AutoBoxCacheMax`. - `Short`, `Byte`, `Long` and `Character` (0–127) cache in a similar way. `Double` and `Float` don't cache.

How does it work?

1. You write `Integer x = 100;`. The compiler turns this into `Integer.valueOf(100)`. The cache returns the shared object for 100.
2. You write `Integer y = 100;`. The **same** cached object comes back. So `x == y` is `true`, because `==` compares references (addresses).
3. Now use 1000 instead. Each `valueOf(1000)` call creates a **new object**. So you get two different objects, and `==` is `false` even though the values are equal.
4. Arithmetic on wrappers works in three steps: unbox, compute on primitives, box again. Wrappers are immutable (they can't change), so `x++` on an `Integer` creates a new object.

Example

```
Integer a = 127, b = 127;
System.out.println(a == b);        // true   — both point to the cached object
Integer c = 128, d = 128;
System.out.println(c == d);        // false  — two different objects
System.out.println(c.equals(d));   // true   — compares the int values

Map<String, Integer> stock = new HashMap<>();
stock.put("sku1", 500); stock.put("sku2", 500);
System.out.println(stock.get("sku1") == stock.get("sku2"));            // false! (500 > 127)
System.out.println(stock.get("sku1").equals(stock.get("sku2")));      // true

Integer missing = stock.get("sku3");    // null
int qty = missing;                      // NullPointerException: unboxing null
```

When would I use it?

- **Primitives** wherever you calculate: loops, counters, arrays (`int[]`), DSA code.
- **Wrappers** when an API needs objects: `List<Integer>`, `Map<String, Long>`, generics. Also when you need `null` to mean "no value" — for example, a nullable DB column mapped in JPA.
- **Primitive streams** (`IntStream`, `mapToInt`) in large calculations, so you avoid boxing.

What can go wrong?

- **`==` on wrappers.** Tests with small numbers pass, because those come from the cache. In production, large numbers fail. Always use `equals`, or unbox first.
- **NullPointerException on unboxing.** This often happens when `map.get()` returns `null` and you assign it to an `int`. Use `getOrDefault(key, 0)` instead.
- **Performance.** A `List<Integer>` of a million elements takes ~16 MB of objects plus 4–8 MB of references. An `int[]` of the same size takes only 4 MB. Heavy boxing in hot loops also creates lots

of garbage, which puts pressure on the GC.

- **Ternary unboxing surprise.** In `Integer r = flag ? map.get(k) : 0;` the `0` is an `int`. Because of that, Java unboxes `map.get(k)` too. If it is null, you get an NPE.

Interview questions

Q1. Why does `Integer a = 128, b = 128; a == b` print false? `Integer.valueOf` caches only −128 to 127. So 128 creates two separate objects, and `==` compares references, not values.

Q2. What is autoboxing and what does it cost? The compiler adds `valueOf()` and `xxxValue()` calls to convert between primitives and wrappers for you. The costs are: a new object for values outside the cache, extra memory, more GC pressure, and a possible NullPointerException when unboxing `null`.

Q3. When would you use `Integer` instead of `int` in an entity or DTO? When the value can genuinely be missing, like a nullable column or an optional field. An `int` would quietly default to 0, and 0 may be a valid business value, so you could not tell "missing" from "zero".

Q4. Why can't you write `List<int>`? Generics work only with reference types because of type erasure (1.16). After compilation, the element type becomes `Object`, and an `Object` can't hold a primitive.

1.3 Integer overflow and numeric conversions

Why?

> **In simple words:** An `int` has a fixed size. If a result gets too big, Java does not complain — the number silently wraps around to a negative value and your answer is wrong.

Primitive integers have fixed sizes. When a calculation goes past the range, Java doesn't throw an error. The value just **wraps around** silently.

This hits money, counters, IDs, DSA sums and midpoints. The wrong answer often passes small tests and then fails the hidden tests.

Think of a car's odometer that shows only 6 digits. After 999999 km it goes back to 000000. The car really drove more, but the meter shows a small number.

What is happening internally?

`int` is 32-bit **two's complement** — the standard way CPUs store signed numbers. Its range is −2,147,483,648 to 2,147,483,647.

Add 1 to the maximum, and the bit pattern `0111…1` becomes `1000…0`. That pattern is the minimum value. The CPU does set an overflow flag, but Java ignores it for `+ - *`.

```
Integer.MAX_VALUE = 0111 1111 ... 1111  (2,147,483,647)
                + 1
                   = 1000 0000 ... 0000  (-2,147,483,648)   ← wraps, no error
```

Type rules: - **Widening** (`int → long → float → double`) happens automatically. It is safe for integers. But `long → double` can lose precision above 2^{53}. - **Narrowing** (`double → int` , `long → int`) needs a cast. It cuts off the decimal part (towards zero) or drops the high bits. - Inside an expression, `byte` , `short` and `char` are first promoted to `int` . If any operand is `long` , the whole expression is calculated in `long` .

How does it work?

Take `long total = a * b;` where `a` and `b` are `int`.

1. Both operands are `int`, so Java calculates `a * b` **in int**.
2. The result overflows.
3. *Only then* is the already-wrong result widened to `long`.

The type of the variable on the left does not change how the right-hand side is calculated.

Example

```
int a = 50_000, b = 50_000;
long wrong = a * b;            // 2,500,000,000 doesn't fit in int -> -1,794,967,296, then widened
long right = (long) a * b;     // cast ONE operand first -> computed in long -> 2,500,000,000
long alsoRight = 1L * a * b;

int lo = 2_000_000_000, hi = 2_100_000_000;
int badMid = (lo + hi) / 2;          // lo + hi overflows -> negative midpoint
int goodMid = lo + (hi - lo) / 2;    // 2,050,000,000

Math.addExact(Integer.MAX_VALUE, 1); // throws ArithmeticException instead of wrapping
Math.abs(Integer.MIN_VALUE);         // still -2,147,483,648 (no positive counterpart)
(int) 3.99;                          // 3   (truncates)
(int) -3.99;                         // -3
'a' + 1;                             // 98 (int);  (char) ('a' + 1) -> 'b'
0.1 + 0.2 == 0.3;                    // false — binary floating point can't represent 0.1 exactly
```

When would I use it?

- In DSA, use `long` for sums of many ints, for products, and for prefix sums. Write midpoints as `lo + (hi - lo) / 2`.
- For **money and points**, use `long` in the smallest unit (paise), or use `BigDecimal`. Never use `double`.
- Use the `Math.*Exact` methods where you must detect overflow, such as balances and quotas. They throw an exception instead of wrapping.

What can go wrong?

- **`double` for currency.** `0.1 + 0.2` is not `0.3`. Small rounding errors add up in totals, and reconciliations fail.
- **`BigDecimal` traps:**
- `new BigDecimal(0.1)` copies the binary error from the double. Use `BigDecimal.valueOf(0.1)` or the String constructor.
- `equals` also compares scale (the number of decimal places). So `2.0` ≠ `2.00`. Use `compareTo` instead.
- **Comparators written as `a - b`** overflow for large or negative values (see 1.13).

Interview questions

Q1. What happens on integer overflow in Java? The value wraps around silently, using two's complement — no error is thrown. When overflow is possible, use `long`, `Math.addExact` / `multiplyExact` (which throw instead), or `BigInteger`.

Q2. Why should you never use `double` for money? Binary floating point can't store most decimal fractions exactly, so every calculation adds a small rounding error, and these add up. Use `long` in minor units (paise) or `BigDecimal` with an explicit rounding mode.

Q3. Why is `(lo + hi) / 2` dangerous? When both are large, `lo + hi` can overflow and give a negative number. `lo + (hi - lo) / 2` can't overflow.

1.4 Strings, the String pool and immutability

Why?

> **In simple words:** A Java String can never change after it is created. Because of that, Java can safely share one copy of the same text everywhere, remember its hash, and hand it to many threads without locks.

Strings are the most used objects in any program: keys, URLs, SQL, JSON, log lines.

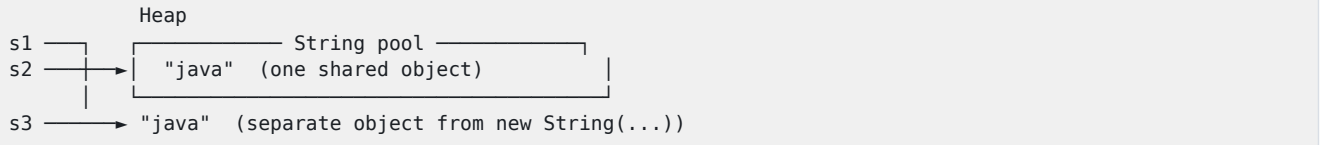
Strings are **immutable** — once created, they can't change. This gives four benefits: 1. **Memory saving.** The JVM can share one copy of identical literals. 2. **Speed as map keys.** The hash code can be calculated once and cached. 3. **Thread safety.** Threads can pass strings around without locks. 4. **Security.** Say a file path is checked and found safe. Nobody holding a reference can change it afterwards.

Think of a printed railway ticket. Once printed, nobody can change the seat number. If you need a different seat, a new ticket is printed. So the ticket checker can trust whatever he sees.

What is happening internally?

```
public final class String {            // final: nobody can subclass and add mutability
    private final byte[] value;        // the characters (Java 9+ "compact strings")
    private final byte coder;          // 0 = LATIN1 (1 byte/char), 1 = UTF16 (2 bytes/char)
    private int hash;                  // cached hashCode, computed lazily on first call
}
```

- **Compact strings (Java 9+).** ASCII and Latin-1 text uses 1 byte per character instead of 2. In most applications, this roughly halves the memory used by Strings.
- **String pool (also called the intern table).** It is a hash table in the heap. (It moved out of PermGen in Java 7.) It holds exactly one shared copy of each distinct literal. Every literal in your code, like `"tenant_id"`, goes into the pool when its class loads.
- **`hashCode`** is `s[0]*31^(n-1) + … + s[n-1]`. It is calculated once and stored in the `hash` field.



How does it work?

1. **A literal `"java"`.** The JVM looks it up in the pool. If it is not there, it is added. The variable gets the pooled reference.
2. **`new String("java")`.** This always creates a **new** object on the heap. Its contents are copied from the pooled literal.
3. **`s.intern()`.** This returns the pooled copy for that content. If there is no pooled copy, it adds one first.
4. **"Modifying" methods** (`toUpperCase`, `replace`, `substring`, `concat`) **return a new String**. The original never changes.

5. **Compile-time constants are joined early.** `"ja" + "va"` becomes `"java"` inside the class file, so it is pooled.

Example

```
String s1 = "java";
String s2 = "java";
String s3 = new String("java");
String s4 = s3.intern();

System.out.println(s1 == s2);       // true  (same pooled object)
System.out.println(s1 == s3);       // false (s3 is a separate heap object)
System.out.println(s1 == s4);       // true  (intern returned the pooled one)
System.out.println(s1.equals(s3));  // true  (same characters)

final String ja = "ja";
String part = "ja";
System.out.println(("ja" + "va") == s1);   // true  — folded at compile time
System.out.println((ja + "va") == s1);     // true  — ja is a compile-time constant (final + literal)
System.out.println((part + "va") == s1);   // false — computed at runtime, new object

String up = s1.toUpperCase();
System.out.println(s1 + " " + up);         // java JAVA — s1 unchanged
```

When would I use it?

- Compare string contents with `equals` / `equalsIgnoreCase`. Never use `==`.
- Use Strings freely as `HashMap` keys. They are immutable, and their hash is cached.
- You rarely need `intern()`. It helps only when parsing creates millions of duplicate strings (country codes, status values) that waste heap. Even then, the G1 flag `-XX:+UseStringDeduplication` is usually better.

What can go wrong?

- `==` **comparison** works by luck with literals. It fails with strings that come from input, from the DB, or from concatenation.
- **Passwords in Strings** stay in memory until the GC removes them, and they may show up in heap dumps. That is why security APIs use `char[]` — you can wipe it after use.
- `substring` **makes a copy** (since Java 7u6). So it costs O(length) — time grows with the length. Calling it in a loop can become $O(n^2)$.
- `split(".")` **takes a regex**, and `.` in a regex means "any character". So `"a.b".split(".")` returns an empty array. Use `split("\\.")` or `Pattern.quote(".")`.

Interview questions

Q1. Why is String immutable? Four reasons. It makes the string pool possible, because shared copies can't be changed. It allows a cached hash code, which makes Strings fast map keys. It gives thread safety without synchronisation. And it helps security, because a value can't be changed after it has been validated.

Q2. Where is the String pool and how do strings get into it? It is a table in the heap (since Java 7). Literals and compile-time constants are added when the class loads. Strings created at runtime are added only if you call `intern()`.

Q3. How many objects does `new String("abc")` create? Up to two. One is the pooled literal `"abc"`, if it isn't pooled already. The other is the new heap object.

Q4. What changed in Java 9 for Strings? Compact strings: a `byte[]` plus a coder flag stores Latin-1 text in 1 byte per character. Also, concatenation now compiles to `invokedynamic` (StringConcatFactory), which performs better.

1.5 StringBuilder and string concatenation

Why?

> **In simple words:** Adding to a String in a loop makes a brand-new copy every time, which gets very slow. A `StringBuilder` is a changeable buffer — you keep adding to the same buffer, and make one String at the end.

Strings are immutable. So when you build a long string with `+=` in a loop, each step creates a new String. Each step also copies all the earlier characters again.

That costs O(n²) time — doubling the input makes it about four times slower. It also creates lots of garbage.

`StringBuilder` is a **mutable** (changeable) character buffer. You append to it in place.

Think of writing a long list on a whiteboard. With `+=`, you rewrite the whole list on a fresh board every time you add one item. With `StringBuilder`, you just add the next line to the same board.

What is happening internally?

`StringBuilder` holds two things: - a growable `byte[]` (like an `ArrayList`, but for characters); - a `count` of how many slots are used.

When the array is full, it grows to about `2 × old + 2` and copies the old contents once. So appends are **amortised O(1)** — usually instant, sometimes slow when it grows, but fast on average.

`toString()` makes one final copy into a new immutable String.

`StringBuffer` is the same class, but every method is `synchronized` (locked so only one thread can use it at a time). It is a leftover from Java 1.0. It is slower, and you almost never need it.

How does it work?

```
s = ""; for 4 iterations: s += "ab"
  iter1: new String "ab"        (copy 2 chars)
  iter2: new String "abab"      (copy 4)
  iter3: new String "ababab"    (copy 6)
  iter4: new String "abababab"  (copy 8)   → total copies 2+4+6+8 = O(n²)

StringBuilder sb (capacity 16): append "ab" ×4 → writes into the same buffer, no copies until full → O(n)
```

The compiler already optimises `a + b + c` when it is **a single expression**. Since Java 9 it does this through `invokedynamic`. So the problem is only repeated concatenation across loop iterations.

Example

```java
// O(n^2): avoid
String csv = "";
for (String id : outletIds) csv += id + ",";

// O(n): correct
StringBuilder sb = new StringBuilder(outletIds.size() * 12);   // pre-size if you know roughly
for (String id : outletIds) sb.append(id).append(',');
```

```
if (sb.length() > 0) sb.setLength(sb.length() - 1);          // drop trailing comma
String result = sb.toString();

// Cleanest for joining:
String joined = String.join(",", outletIds);
String viaStream = outletIds.stream().collect(Collectors.joining(",", "[", "]"));

// DSA uses: reverse, insert, deleteCharAt, setCharAt
String rev = new StringBuilder("level").reverse().toString();
```

When would I use it?

- When building strings in loops: CSV export, dynamic SQL fragments, log messages built with conditions, DSA output.
- `String.join` / `Collectors.joining` for simple joins with a delimiter.
- `StringBuffer` practically never. If several threads really do build one string, put the synchronisation at a higher level.

What can go wrong?

- **Sharing one `StringBuilder` across threads** corrupts it. It is not thread-safe.
- **Comparing builders with `sb.equals(otherSb)`.** `StringBuilder` doesn't override `equals`, so this compares references, not text. Use `sb.toString().equals(...)`, or `sb.compareTo(other) == 0` (Java 11+).
- **Logging with `+`.** Even when the log level is off, the string still gets built. Use parameterised logging instead: `log.debug("user {} points {}", id, pts)`.

Interview questions

Q1. String vs StringBuilder vs StringBuffer? String is immutable. StringBuilder is mutable and not synchronised — it is fast and the default choice for building strings. StringBuffer is mutable and synchronised — it is a legacy class and slower.

Q2. Is `s1 + s2 + s3` inefficient? Not inside one expression, because the compiler optimises it. The problem is repeated `+=` across loop iterations, which is O(n^2).

Q3. How does StringBuilder grow? When its internal array is full, it grows to about double the old capacity plus 2 and copies once. This gives amortised O(1) appends — fast on average.

1.6 Stack vs heap, and why Java is always pass-by-value

Why?

> **In simple words:** Each thread has its own small "stack" for local variables, and all objects live in one shared "heap". When you call a method, Java always passes a copy — for an object, it copies the address, not the object.

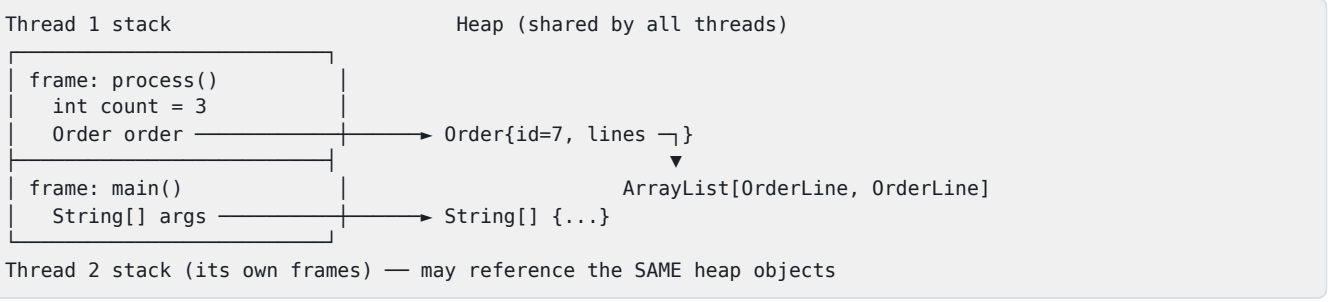
You need a clear picture of where data lives. Without it, you can't reason about performance, thread safety, memory leaks or method calls.

Two common bugs come from this: - "Why did my change not show up?" - "Why did this change show up when I didn't expect it?"

Both are explained by this model.

Think of your house address written on a slip of paper. If you give a friend a photocopy of the slip, he can visit your house and rearrange the furniture — you will see the change. But if he scribbles a new address on his copy, your slip still has your address.

What is happening internally?



- **Stack** — one per thread.
- Each method call pushes a **frame** onto the stack.
- The frame holds local variables and an operand stack (scratch space for calculations).
- A primitive local stores the value itself. An object local stores a reference.
- When the method returns, its frame is popped. Memory cleanup is automatic and instant.
- **Heap** — one per JVM, shared by all threads.
- Every object and every array lives here.
- The garbage collector frees an object when nothing can reach it any more.
- Because each thread has its own stack, local variables are **thread-confined** — only one thread sees them, so they are safe. Heap objects that several threads can reach need synchronisation (Chapter 4).

Pass-by-value: when you call a method, Java **copies** each argument's value into the new frame. - For a primitive, the value is the number. - For an object, the value is the **reference** (the address). The object itself is not copied.

How does it work?

1. The caller has `List<Integer> list`. This is a reference, say `0xA1`, pointing to an ArrayList on the heap.
2. Calling `mutate(list)` copies `0xA1` into the parameter `l` in the new frame. Now both variables point to the same heap object.
3. `l.add(1)` follows `0xA1` and changes the shared object. So the caller sees the change.
4. `l = new ArrayList<>()` changes only the **copy** in the called method's frame, to `0xB2`. The caller's variable still holds `0xA1`.

Example

```
static void mutate(List<Integer> l)   { l.add(1); }                      // changes shared object
static void reassign(List<Integer> l) { l = new ArrayList<>(); l.add(2); } // changes local copy only
static void bump(int x)               { x++; }                            // copy of primitive

List<Integer> list = new ArrayList<>();
mutate(list);     // list = [1]
reassign(list);   // list = [1]  — the new list was attached to the callee's copy and is now garbage
int n = 5; bump(n);   // n = 5

// "Return two values" in Java: return an object instead of out/ref parameters
record MinMax(int min, int max) {}
```

```
static MinMax minMax(int[] a) { return new MinMax(Arrays.stream(a).min().orElseThrow(),
Arrays.stream(a).max().orElseThrow()); }
```

When would I use it?

- To explain why a method can change a collection you pass in. This is why defensive copies matter (1.14).
- To reason about thread safety. Locals are safe; shared heap state needs protection.
- To tell apart `StackOverflowError` and `OutOfMemoryError`. The first means too many frames, usually from deep recursion. The second means the heap is full.

What can go wrong?

- **Expecting a method to "swap" two variables** or to point the caller's variable at a new object. This is impossible in Java. Java has no C# `ref` / `out`.
- **Exposing internal mutable state.** If you return your private `List` directly, callers can change it.
- **Deep recursion.** For example, DFS on 10^5 nodes can overflow the default thread stack of ~512 KB–1 MB. Use an explicit stack instead.

Interview questions

Q1. Is Java pass-by-value or pass-by-reference? Always pass-by-value. For objects, the value that gets copied is the reference. So the called method can change the object, but it can't make the caller's variable point to a different object.

Q2. What lives on the stack and what on the heap? The stack holds per-thread frames with local variables, primitive values and references. The heap holds all objects and arrays; it is shared across threads and managed by the GC.

Q3. Why are local variables thread-safe? Each thread has its own stack, so local variables are never shared between threads. But the objects they point to can still be shared.

1.7 Classes, objects, constructors, `static`, `final` and initialisation order

Why?

> **In simple words:** A class bundles data with the methods that protect it, and a constructor makes sure every new object starts valid. `static` means "belongs to the class, one copy"; `final` means "can't be changed later". The order in which Java sets things up explains some strange `null` bugs.

A class keeps state together with the behaviour that protects it. A constructor guarantees an object starts in a valid state.

- `static` gives you data and utility methods at class level (one shared copy).
- `final` gives you guarantees:
- a value that is set once;
- a method that can't be overridden;
- a class that can't be extended.

Interviewers test the initialisation order. It explains surprising `null` values and bugs in real code.

Think of building a house. The foundation (parent class) must be laid before the upper floor (child class). If someone tries to use the upper floor's furniture while only the foundation exists, the furniture

isn't there yet — that is the `null`.

What is happening internally?

- A **class** is loaded once per class loader. Its **static fields** live with the class, so there is one copy. They are set up when the class is first actively used.
- An **object** is a block of memory on the heap. It has:
- a header — a mark word (used for locking, hash code and GC age) and a pointer to its class;
- then the instance fields.
- `new Account("A1")` runs in this order: 1. Allocate memory. All fields start at zero values: 0, false, null. 2. Run field initialisers and instance blocks. 3. Run the constructor body.
- A subclass constructor first calls `super(...)`. If you don't write it, Java adds `super()` for you. So the parent part is always set up first.
- `final` fields get a special memory-model guarantee. Once the constructor finishes, other threads see the final fields correctly set, without any synchronisation. The condition: `this` must not have escaped (been handed to other code) during construction.

How does it work?

The order when `new Child()` runs for the first time:

```
1. Parent static fields/blocks   (once, at class init, in source order)
2. Child static fields/blocks    (once)
3. Parent instance fields/blocks, then Parent constructor body
4. Child instance fields/blocks, then Child constructor body
```

Example

```
class Parent {
    static { System.out.println("1 parent static"); }
    { System.out.println("3 parent instance block"); }
    Parent() { System.out.println("4 parent ctor"); init(); }   // calls an overridable method: dangerous
    void init() {}
}
class Child extends Parent {
    static { System.out.println("2 child static"); }
    private final String name = "child";
    { System.out.println("5 child instance block"); }
    Child() { System.out.println("6 child ctor"); }
    @Override void init() { System.out.println("init sees name = " + name); }   // prints null!
}
// new Child();  ->  1, 2, 3, 4, "init sees name = null", 5, 6
// new Child();  ->  3, 4, "init sees name = null", 5, 6   (statics don't run again)

public class Account {
    private static int created = 0;            // one per class
    private final String id;                   // must be set exactly once, in every constructor path
    private long balancePaise;

    public Account(String id) { this(id, 0); }                      // constructor chaining
    public Account(String id, long opening) {
        if (opening < 0) throw new IllegalArgumentException("negative opening balance");
        this.id = Objects.requireNonNull(id);
        this.balancePaise = opening;
        created++;
    }
    public static int created() { return created; }                 // static: no 'this'
}
```

Why does it print `null`? 1. `Parent()` calls `init()`. 2. Because of dynamic dispatch (Java picks the method by the object's real type), `Child.init()` runs. 3. But Child's field initialisers (step 4 of the order) haven't run yet. 4. So `name` is still `null`.

When would I use it?

- `static` for constants (`static final int MAX_RETRY = 3`), factory methods (`Money.inr(100)`) and stateless utilities.
- `final` **fields** by default for everything set in the constructor. This gives immutability and thread safety.
- `final` **classes** for value types.
- `private` **constructors** for singletons, utility classes and static factories.
- **Validation in the constructor**, so an invalid object can never exist.

What can go wrong?

- **Calling overridable methods from constructors** — this causes the `null` shown above. Only call `private` or `final` methods there.
- **Mutable static state** (like `static Map cache`). It is global, shared across all threads and requests, and never garbage-collected. It is a classic cause of memory leaks and race conditions.
- **Leaking `this` from a constructor** — for example, registering a listener or starting a thread. Other threads may then see a half-built object.
- **Exceptions in a static initialiser.** The first failure throws `ExceptionInInitializerError`. After that, every later use throws `NoClassDefFoundError`. This is a confusing error in production, because the real cause was earlier.

Interview questions

Q1. What is the order of initialisation in Java? First the static parts — parent then child, once, when the class is initialised. Then, for each new object: the parent's instance initialisers and constructor, then the child's instance initialisers and constructor.

Q2. Can a constructor be private, and why would it be? Yes. It is used for singletons, utility classes, static factory methods like `List.of`, and builders that force you to create the object through `build()`.

Q3. What does `final` mean on a field, method, class? On a field: it is assigned only once, and it also gets safe-publication guarantees (other threads see it correctly set). On a method: it can't be overridden. On a class: it can't be extended.

Q4. Why shouldn't constructors call overridable methods? The subclass's override runs before the subclass's fields are initialised. So it sees default values (null or 0) instead of the real ones.

1.8 Access modifiers and encapsulation

Why?

> **In simple words:** Keep an object's data private and only let people change it through methods that check the rules. Then the object can never get into a wrong state, like a negative balance.

An *invariant* is a rule that must always be true for an object, like "balance is never negative". Encapsulation keeps invariants true. It makes the state reachable only through methods that check the rules.

Access modifiers are the language tool that enforces this. They also define a module's public API. Everything behind that API can be changed freely.

Think of an ATM. You can't open the cash box and take notes yourself. You must go through the machine, and the machine checks your PIN and balance first.

What is happening internally?

Access is checked twice: by the compiler and by the JVM verifier. - Private members aren't visible outside the class. - Package-private members (no modifier) are visible only inside the same package. - Nested classes may access each other's private members. Since Java 11, the JVM supports this directly through "nestmates".

Modifier	Same class	Same package	Subclass in another package	Everyone
`private`	✔			
(none — package-private)	✔	✔		
`protected`	✔	✔	✔	
`public`	✔	✔	✔	✔

C# comparison: - Java has no `internal`. Package-private is the closest thing. - Java's `protected` also opens access to the whole package, not only to subclasses.

How does it work?

Design rule: 1. Start with the most restrictive access (`private`). 2. Widen it only when a real caller needs it. 3. Expose **behaviour** (`debit`, `credit`) instead of raw setters. Then the class itself enforces the rules.

Example

```
// Anaemic (bad): anyone can break the invariant
public class Wallet { public long balance; }
wallet.balance -= 1_000_000;          // negative balance, no audit, no rule

// Encapsulated (good)
public final class Wallet {
    private long balancePaise;
    private final List<String> audit = new ArrayList<>();

    public void debit(long paise, String reason) {
        if (paise <= 0) throw new IllegalArgumentException("amount must be positive");
        if (paise > balancePaise) throw new IllegalStateException("insufficient balance");
        balancePaise -= paise;
        audit.add("DEBIT " + paise + " " + reason);
    }
    public long balance() { return balancePaise; }
    public List<String> audit() { return List.copyOf(audit); }   // don't leak the mutable list
}
```

When would I use it?

- **`private`** for fields, always.
- **`public`** methods form the API.
- **Package-private** for helpers used only inside one feature package. This works well with a "package by feature" layout.

- `protected` rarely — mainly for template-method hooks (steps a subclass is meant to fill in).
- In Spring, `@Transactional` and `@Async` work only on `public` methods called through the proxy (Chapter 6.7).

What can go wrong?

- **Getters that return internal mutable collections.** Callers can change the collection and skip your rules.
- **A public setter for every field** ("JavaBean style"). Objects become plain data bags, and the rules end up scattered across many services.
- **Reflection with `setAccessible(true)`** (used by frameworks and tests) can get around `private`. So don't rely on access modifiers for security.

Interview questions

Q1. What is encapsulation and why does it matter? It means hiding state behind methods that enforce the invariants (rules that must always hold). So the object is always valid, and you can change the internals without breaking callers.

Q2. Difference between `protected` and package-private? Package-private is visible only inside the same package. `protected` is visible in the package too, and also to subclasses in other packages.

Q3. Are getters and setters encapsulation? Not by themselves. A setter for every field exposes state just like a public field does. Real encapsulation exposes meaningful operations that keep the invariants true.

1.9 Inheritance and polymorphism (overloading vs overriding)

Why?

> **In simple words:** You write code that talks to a general type, like "any payment gateway". At runtime, Java runs the correct version for the real object. So you can add new types later without touching the old code.

Polymorphism means "one call, many behaviours". You write code against a general type such as `PaymentGateway` or `Shape`. When the program runs, each object runs its own specific version of the method.

Why does this help? Because you can add a new type (say, a new payment gateway) without editing the code that uses it.

Inheritance is one way to get polymorphism. A subclass *is a* kind of its parent. It reuses the parent's code. It replaces (overrides) only the parts that are different.

What is happening internally?

- **Overriding (runtime polymorphism)** — the method is chosen while the program runs: 1. Every class has a **virtual method table (vtable)**. This is an array of pointers, one for each method, pointing to that class's code. 2. A subclass's vtable starts as a copy of its parent's. Then the entries it overrides are replaced with its own methods. 3. A call like `animal.speak()` compiles to the bytecode instruction `invokevirtual Animal.speak`. 4. At runtime, the JVM reads the object's *actual* class (every object stores a pointer to its class). 5. It looks up the `speak` slot in that class's vtable and

jumps there. 6. This is called **dynamic dispatch** — the decision is made at runtime, not compile time.

- Interface calls use `invokeinterface`, which works with similar tables.
- The JIT (the just-in-time compiler that turns hot bytecode into machine code) often sees only one implementation at a call site. Such a call is called *monomorphic*. The JIT then turns it into a direct call and inlines it (copies the method body into the caller).
- **Overloading (compile-time polymorphism)** — the method is chosen while compiling:
- Several methods share one name but have different parameter lists.
- The **compiler** picks one. It looks only at the **static (declared) types** of the arguments — the types written in the code.
- That choice is fixed in the bytecode. It never changes at runtime.
- In Java, **every non-static, non-private, non-final method is virtual by default**. "Virtual" means it can be overridden and is chosen at runtime. In C#, you must opt in with the `virtual` keyword.
- `@Override` does not change behaviour. It only asks the compiler to check that you really are overriding something.

The picture below shows the two types involved in one call. The reference type decides what you are allowed to call. The object type decides which code actually runs.

```
Animal a = new Dog();     a.speak()
   reference type: Animal — decides WHICH METHODS are callable (compile time)
   object type:    Dog    — decides WHICH IMPLEMENTATION runs (runtime, via Dog's vtable)
```

How does it work?

The table compares the two side by side. "Covariant" means the overriding method may return a more specific type (a subtype).

	Overloading	Overriding
Where	Same class (or inherited)	Subclass
Signature	Same name, different parameters	Same name and same parameters
Return type	Anything	Same or a subtype (covariant)
Access	Anything	Same or wider (e.g. protected → public)
Checked exceptions	Anything	Same, narrower or none
Resolved	At compile time, by declared argument types	At runtime, by the actual object type
`static` methods	Can be overloaded	Can't be overridden; they are **hidden** instead

Example

```java
class Animal {
    void speak() { System.out.println("..."); }
    static void kind() { System.out.println("animal"); }
}
class Dog extends Animal {
    @Override void speak() { System.out.println("Woof"); }
    static void kind() { System.out.println("dog"); }       // hides, doesn't override
}

Animal a = new Dog();
a.speak();          // Woof   -> runtime dispatch on the actual object
```

```
a.kind();          // animal -> static: chosen by the reference type at compile time

// Overloading is resolved by DECLARED types, not runtime types
class Printer {
    void print(Object o) { System.out.println("object"); }
    void print(String s) { System.out.println("string"); }
}
Object o = "hello";
new Printer().print(o);   // "object" — the declared type of o is Object
```

Step-by-step trace of `a.speak()`:

1. The compiler sees that `a` is declared as `Animal`.
2. It checks that `Animal` has a `speak()` method. It does, so the call is allowed.
3. It writes `invokevirtual Animal.speak` into the bytecode.
4. At runtime, the object's class pointer says `Dog`.
5. Dog's vtable slot for `speak` points to `Dog.speak`.
6. So "Woof" prints.

`a.kind()` is different. `kind()` is static, so there is no vtable lookup. The compiler picks `Animal.kind()` from the declared type, and "animal" prints.

When would I use it?

- **Overriding**: Strategy implementations (swappable behaviours behind one interface), template-method hooks (a base class calls a method that subclasses fill in), `equals` / `hashCode` / `toString`, and Spring beans that implement interfaces.
- **Overloading**: convenience APIs such as `log(String)` and `log(String, Throwable)`. Also constructors with optional parameters. But if you have more than 3–4 parameters, consider a builder instead.
- **Inheritance**: only for true *is-a* relationships where the shared behaviour is stable. Otherwise prefer composition (1.11).

What can go wrong?

- **A silent non-override.** You forget `@Override`. Then you misspell the method name or change a parameter type. You have now created an *overload*, not an override. Your "override" is silently never called. `@Override` would have caught this at compile time.
- **Confusing overloads with boxing and varargs.** Think of `m(long)` vs `m(Integer)` vs `m(int...)`. *Boxing* means wrapping a primitive `int` into an `Integer` object. *Varargs* means "any number of arguments". The compiler's order of preference is: first widening (`int` → `long`), then boxing, then varargs.
- **Deep inheritance trees.** You change a base class, and subclasses break in unexpected ways. This is called the *fragile base class problem*.
- **Calling static methods through an object reference** (like `a.kind()`). It looks like runtime dispatch, but it is not. This misleads readers about which method runs.

Interview questions

Q1. Difference between overloading and overriding? Overloading means the same method name with different parameters; the compiler picks one at compile time using the declared types of the arguments. Overriding means a subclass redefines a method with the same signature; the JVM picks one at runtime using the actual object type, through dynamic dispatch.

Q2. Can you override static or private methods? No. Static methods are hidden, not overridden — the compiler picks them by the reference type. Private methods are not inherited at all, so a method with the same name in a subclass is simply a new, separate method.

Q3. How does the JVM decide which overridden method to call? It looks at the object's runtime class and finds the method in that class's virtual method table (via the `invokevirtual` instruction). If the JIT sees only one implementation at that call site, it may inline the call for speed.

Q4. What is covariant return type? An overriding method is allowed to return a subtype of the parent method's return type. For example, `Dog clone()` can override `Animal clone()`.

1.10 Interfaces vs abstract classes

Why?

> **In simple words:** An interface is a promise — "I can do this". Any class can make that promise, even unrelated ones. An abstract class is a half-built parent class that shares real code and fields with its children.

Both let you write code against an *abstraction* (a general type) instead of a concrete class.

- **Interfaces** describe a *capability* or *contract*. Unrelated classes can share it, like `Comparable` or `PaymentGateway`. One class can implement many interfaces.
- **Abstract classes** share *state (fields) and code* among closely related classes. They give subclasses a partly built skeleton to complete.

Think of a driving licence (interface): a truck driver, a taxi driver and a doctor can all hold one. A family recipe book (abstract class) is passed only within one family, with half the recipes already written.

What is happening internally?

- **An interface** compiles to a class file with the `ACC_INTERFACE` flag set.
- Its fields are automatically `public static final`. So they can only be constants.
- Since Java 8, it can have **default** methods (instance methods with a body) and **static** methods.
- Since Java 9, it can also have **private** helper methods.
- It still has no instance fields and no constructor.
- **An abstract class** is a normal class that you can't create directly with `new`.
- It can have fields.
- It can have constructors. Subclasses run them through `super(...)`.
- It can mix abstract methods (no body) and concrete methods (with body) freely.
- **Inheritance limits.** A Java class can extend only one class. But it can implement many interfaces.
- **The diamond problem.** Two interfaces may provide the same default method. Then the class must override that method itself. Inside, it can pick one version with `A.super.method()`.

How does it work?

	Interface	Abstract class
A class can extend/implement	Many	Only one
Instance state	No (constants only)	Yes

	Interface	Abstract class
Constructors	No	Yes
Method kinds	abstract, default, static, private	Any
Access of members	public (private helpers allowed)	Any
Good for	Capabilities, ports, strategies, APIs	Template methods, a shared base implementation

A *port* here means an interface your core code depends on, which an adapter implements (for example a repository). A *template method* is a fixed algorithm in the base class that calls "hook" methods subclasses fill in.

Example

The example below shows three things:

1. `RewardRule` is an interface: a contract with a shared default method and a static factory method.
2. `MinimumAmountRule` is an abstract class: it holds shared state (`minAmountPaise`) and fixed logic, with a hook (`extraCheck`) for subclasses.
3. `Duck` shows the diamond: two interfaces give `move()`, so `Duck` must decide.

```
// Interface: a contract many unrelated classes can satisfy
public interface RewardRule {
    boolean applies(Order o);
    long points(Order o);
    default String describe() { return getClass().getSimpleName() + " rule"; }   // shared default
    static RewardRule none() { return new RewardRule() {                          // factory on the interface
        public boolean applies(Order o) { return false; }
        public long points(Order o) { return 0; } }; }
}

// Abstract class: shared state + skeleton for related rules
public abstract class MinimumAmountRule implements RewardRule {
    protected final long minAmountPaise;                          // shared state
    protected MinimumAmountRule(long min) { this.minAmountPaise = min; }
    @Override public final boolean applies(Order o) {             // shared, fixed logic
        return o.amountPaise() >= minAmountPaise && extraCheck(o);
    }
    protected boolean extraCheck(Order o) { return true; }        // hook for subclasses
}

public final class FlatBonusRule extends MinimumAmountRule {
    private final long bonus;
    public FlatBonusRule(long min, long bonus) { super(min); this.bonus = bonus; }
    @Override public long points(Order o) { return bonus; }
}
record Order(long amountPaise) {}

// Diamond with defaults: must resolve explicitly
interface Walker { default String move() { return "walk"; } }
interface Swimmer { default String move() { return "swim"; } }
class Duck implements Walker, Swimmer {
    @Override public String move() { return Walker.super.move() + "+" + Swimmer.super.move(); }
}
```

When would I use it?

- **Start with an interface** for anything a caller depends on: repositories, gateways, strategies, Spring service ports.

- It keeps dependencies inverted. This is the "D" in SOLID: high-level code depends on abstractions, not on concrete classes.
- It also makes mocking easy in tests.
- **Add an abstract class underneath** when several implementations share real code or state. Examples: `AbstractList` in the JDK, or a base `MinimumAmountRule`.
- **Use default methods to evolve a published interface.** You can add a new method without breaking every class that already implements it. That is exactly how `Collection.stream()` was added in Java 8.

What can go wrong?

- **Abstract class just to share a few helpers.** A class can extend only one class. So this locks subclasses out of extending anything else. Prefer composition or static helper methods.
- **State-like behaviour in default methods.** Default methods can't hold fields. Complex defaults also make interfaces hard to understand.
- **"Constant interfaces".** These are interfaces holding only constants, which classes implement just to use short names. This is an anti-pattern (a common bad practice). Use a final class or an enum instead.

Interview questions

Q1. Interface vs abstract class, and how do you choose? Use an interface for a contract that many classes — possibly unrelated — implement, and when you need multiple inheritance of type. Use an abstract class when related classes share state and implementation, especially with a template method.

Q2. Why were default methods added? To let interfaces grow — for example, adding `stream()` to `Collection` — without breaking every existing implementation.

Q3. Can an interface have a constructor or instance fields? No. It can have only constants, plus abstract, default, static and private methods.

Q4. What happens if a class inherits the same default method from two interfaces? The code does not compile unless the class overrides that method. Inside the override, it can call one of them with `InterfaceName.super.method()`.

1.11 Composition vs inheritance

Why?

> **In simple words:** Instead of *being* something (inheritance), your class *has* something and asks it to do the work (composition). This keeps classes loosely connected, and you can swap the helper at runtime.

Inheritance ties a subclass to its parent's *implementation* — the inner details of how the parent works.

- When the parent changes, subclasses can break.
- The relationship is fixed. You can't change it while the program runs.

Composition means an object *has* a collaborator and passes work to it. This is called *delegation*. You get the same code reuse, but with looser coupling (less dependence between classes) and flexibility at runtime.

"Favour composition over inheritance" is a core principle of good LLD (low-level design).

Think of a tiffin service. A restaurant doesn't need to *be* a delivery company. It *has* a delivery partner. If the partner is bad, it switches to another one tomorrow.

What is happening internally?

- **Inheritance**:
- The subclass object physically contains all the parent's fields.
- Calls go through the vtable (the per-class table of method pointers).
- The relationship is fixed at compile time by `extends`.
- **Composition**:
- The object holds a **reference** to another object. That reference is usually typed as an interface.
- It forwards calls to that object.
- The collaborator can be swapped, for example through constructor injection (passing it in the constructor). So the behaviour is chosen at runtime.

How does it work?

Ask yourself two questions:

1. "Is X truly a kind of Y, in every situation, forever?" If not, use composition.
2. "Do I want *all* of the parent's public methods in my public API?" With inheritance, you get all of them, whether you want them or not.

Example

The first class below shows how inheritance can go wrong:

1. You call `addAll` with 3 items. Your override adds 3 to the counter.
2. Then `super.addAll` runs. Inside `HashSet`, `addAll` calls `add()` once per item.
3. But `add()` is your overridden version, so each call adds 1 more.
4. Result: 6 instead of 3.

The composition version wraps a `Set` and doesn't care how the set works inside. The last part shows swapping behaviour at runtime with the Strategy pattern.

```
// Inheritance gone wrong: a counting set that double-counts
class CountingSet<E> extends HashSet<E> {
    int added = 0;
    @Override public boolean add(E e) { added++; return super.add(e); }
    @Override public boolean addAll(Collection<? extends E> c) { added += c.size(); return super.addAll(c); }
}
CountingSet<String> s = new CountingSet<>();
s.addAll(List.of("a", "b", "c"));
System.out.println(s.added);   // 6, not 3 — HashSet.addAll internally calls add(), which we also counted

// Composition: wrap and delegate — immune to the parent's internal calls
class CountingSet2<E> {
    private final Set<E> inner;                       // has-a
    private int added = 0;
    CountingSet2(Set<E> inner) { this.inner = inner; }
    boolean add(E e) { added++; return inner.add(e); }
    boolean addAll(Collection<? extends E> c) { added += c.size(); return inner.addAll(c); }
    int added() { return added; }
}

// Runtime flexibility: swap pricing behaviour without subclassing Ride
interface PricingStrategy { long fare(double km); }
```

```java
final class Ride {
    private final PricingStrategy pricing;            // composition + Strategy pattern
    Ride(PricingStrategy pricing) { this.pricing = pricing; }
    long fare(double km) { return pricing.fare(km); }
}
```

When would I use it?

- **Composition by default.** Services built from repositories and clients. Strategy and Decorator patterns. Spring DI (dependency injection) — every injected dependency is composition.
- **Inheritance** for:
- Genuine type hierarchies with stable shared behaviour (`Shape → Circle`).
- Framework extension points designed for it (`AbstractList`, `OncePerRequestFilter`).
- Sealed hierarchies of value types (a fixed, closed set of subclasses).

What can go wrong?

- **Inheriting only to reuse code.** This leads to deep, fragile hierarchies. You also get subclasses that override methods just to throw `UnsupportedOperationException`. That breaks the Liskov substitution principle (9.5) — a subclass should work anywhere its parent works.
- **Composition done badly.** You end up writing endless one-line forwarding methods. If you forward everything, maybe the type really is a subtype, or the abstraction is wrong.

Interview questions

Q1. Why favour composition over inheritance? Less coupling to the parent's implementation details. Flexibility at runtime, because you can swap collaborators. No fragile base class problems. And you control exactly which API you expose.

Q2. Give an example where inheritance broke something. A `HashSet` subclass that counts additions double-counts, because `addAll` calls `add` internally and both are overridden. Composition avoids depending on the parent's internal self-calls.

Q3. When is inheritance the right choice? When there is a true, stable is-a relationship with shared behaviour, or when a framework class is explicitly designed to be extended.

1.12 `equals()` and `hashCode()`

Why?

> **In simple words:** `equals` decides when two objects mean "the same thing". `hashCode` gives each object a number so that hash collections can jump to the right place. If two objects are equal, their numbers must also be equal.

`==` asks: "Is this the very same object in memory?"

Business logic usually asks something else: "Do these two represent the same thing?" For example, two `Sku("ABC", 6)` objects created separately are the same SKU.

- `equals` defines this *logical* equality.
- `hashCode` helps hash-based collections (`HashMap`, `HashSet`) find equal objects fast. It narrows the search to one *bucket* (one slot in the internal array).

What is happening internally?

- **Default behaviour from `Object`:**
- `Object.equals` is just `this == o`.
- `Object.hashCode` returns an *identity hash* — a number tied to that object. In HotSpot it comes from a thread-local random generator and is stored in the object header on the first call. It is **not** the memory address.
- **How a `HashMap` uses them** (full details in 2.3): 1. **First**, it calls `hashCode()` to pick a bucket. 2. **Then**, it calls `equals()` to find the exact key inside that bucket.

The trace below shows what happens when `hashCode` is and isn't overridden:

```
put(sku1) → hash(sku1) = 37 → bucket 5 → stored
get(sku2) where sku2.equals(sku1):
    if sku2.hashCode() == 37 → bucket 5 → equals() true → FOUND
    if hashCode not overridden → identity hash 9182 → bucket 14 → empty → NOT FOUND (bug)
```

The contract (the rules every `equals` / `hashCode` pair must follow):

1. `equals` must be: - *reflexive* — `x.equals(x)` is true; - *symmetric* — if `x.equals(y)` then `y.equals(x)`; - *transitive* — if `x` equals `y` and `y` equals `z`, then `x` equals `z`; - *consistent* — the same answer every time, if nothing changed; - and `x.equals(null)` is `false`.
2. **If `a.equals(b)`, then `a.hashCode() == b.hashCode()`.** (This is mandatory.)
3. Equal hash codes do **not** mean the objects are equal. Two different objects may share a hash code. This is a *collision*. It is allowed; it just makes lookups slower.

How does it work?

To implement `equals` and `hashCode`:

1. If `this == o`, return true. This is a fast shortcut.
2. Check the type with `instanceof`. (Or use `getClass()` if you want strict equality, where a subclass object is never equal to a parent object.)
3. Compare the **significant fields** — the ones that define identity.
4. Build `hashCode` from **exactly the same fields**. Use `Objects.hash(...)` or the "multiply by 31" pattern.

Example

The second half shows the *mutable-key disaster*:

1. A `Tag` named "gold" is added to the set. It goes into the bucket for "gold"'s hash.
2. Then the name is changed to "platinum". Its hash changes too.
3. `contains(t)` now looks in the "platinum" bucket. That bucket is empty.
4. The entry is stuck in the old bucket — unreachable but still in memory. That is a leak.

```java
public final class Sku {
    private final String code;
    private final int packSize;
    public Sku(String code, int packSize) { this.code = code; this.packSize = packSize; }

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Sku other)) return false;          // also handles null
        return packSize == other.packSize && code.equals(other.code);
    }
```

```
    @Override public int hashCode() { return 31 * code.hashCode() + packSize; }   // same fields as equals
}

Set<Sku> set = new HashSet<>();
set.add(new Sku("ABC", 6));
System.out.println(set.contains(new Sku("ABC", 6)));   // true. Without hashCode(): false

// Records generate correct equals/hashCode/toString automatically
record SkuKey(String code, int packSize) {}

// The mutable-key disaster
class Tag { String name; Tag(String n) { name = n; }
    public boolean equals(Object o) { return o instanceof Tag t && t.name.equals(name); }
    public int hashCode() { return name.hashCode(); } }
Set<Tag> tags = new HashSet<>();
Tag t = new Tag("gold"); tags.add(t);
t.name = "platinum";                      // hash changes after insertion
System.out.println(tags.contains(t));     // false — searching the wrong bucket
System.out.println(tags.size());          // 1 — the entry is stuck: unreachable but still referenced (leak)
```

When would I use it?

- **Value objects and keys**: `Money`, `Sku`, composite map keys (keys made of several fields), and comparing DTOs in tests. Prefer **records** — they generate a correct `equals` and `hashCode` for you.
- **JPA entities are special.**
- Base equality on a stable business key, or on the ID with care. The ID is null until the entity is saved (persisted).
- Never include lazy collections (collections loaded from the database only when first used).
- A common safe pattern: `equals` compares a non-null ID, and `hashCode` returns a constant (`getClass().hashCode()`).

What can go wrong?

- **Overriding `equals` without `hashCode`.** Hash collections can't find equal objects. Sets end up holding "duplicates".
- **Mutable fields in `hashCode`.** You change a field after inserting the object. The entry becomes lost.
- **`instanceof` with subclasses that add fields.** This can break symmetry: `point.equals(colorPoint)` is true, but the reverse is false. Make value classes `final`, or use `getClass()`.
- **`double` fields compared with `==`.** Use `Double.compare` instead. For arrays, use `Arrays.equals` and `Arrays.hashCode`.

Interview questions

Q1. What is the equals/hashCode contract? Equal objects must have equal hash codes. Equal hash codes do not require the objects to be equal. `equals` must be reflexive, symmetric, transitive and consistent, and must return false for null.

Q2. What happens if you override `equals` but not `hashCode`? Two equal objects usually get different hash codes and land in different buckets. So `HashMap.get` and `HashSet.contains` fail, and sets accept logical duplicates.

Q3. What happens if two unequal objects have the same hashCode? That is a collision. They share a bucket, and `equals` tells them apart. Everything stays correct, but lookups slow down if collisions happen often.

Q4. Why should map keys be immutable? If a field used in `hashCode` changes after insertion, the entry stays in its old bucket. Lookups now compute a different bucket, so the entry can't be found — and it

leaks.

1.13 Comparable and Comparator

Why?

> **In simple words:** `Comparable` is the one "default" way a class sorts itself, like dates in time order. `Comparator` is any extra sorting rule you write from outside, like "by rating, then by name".

Sorting, `TreeMap`, `TreeSet`, `PriorityQueue` and binary search all need to know how to order objects.

- `Comparable` gives a class its single **natural order**. This is written inside the class.
- `Comparator` gives any number of **external orders**, such as by rating, then by name. You don't need to touch the class at all.

Think of a merit list. The default order is by marks (Comparable). But the admission office may also sort the same list by category, then by date of birth (a Comparator).

What is happening internally?

- `Comparable<T>` has one method: `int compareTo(T o)`.
- It returns a negative number if `this` comes first, zero if equal, and positive if `this` comes after.
- `Comparator<T>` is a *functional interface* (an interface with one abstract method, so you can use a lambda). Its method is `int compare(T a, T b)`.
- It comes with handy helper methods to build and combine comparators: `comparing`, `comparingInt`, `thenComparing`, `reversed`, `nullsFirst`.
- **Who calls these methods:**
- `Collections.sort` and `List.sort` use **TimSort**. It is *stable* — equal elements keep their original order. It is O(n log n) and very fast on data that is already partly sorted.
- `TreeMap` calls `compare` at every node of its red-black tree (a self-balancing binary search tree).
- `PriorityQueue` calls it when moving elements up or down the heap (sift-up/sift-down).

How does it work?

Look at `Comparator.comparingInt(Employee::level).thenComparing(Employee::name)`. It works like this:

1. Compare the two employees by level.
2. If the levels differ, that's the answer.
3. Only if the levels are equal, compare by name.

Each `thenComparing` wraps the previous comparator. So you can chain as many tie-breakers as you need.

Example

The last part shows the *overflow bug*. `a - b` looks fine, but `Integer.MIN_VALUE - 1` is too small for an `int`. It wraps around to a large positive number. So the comparator gives the wrong sign.

```
record Employee(String name, int level, double rating) implements Comparable<Employee> {
    @Override public int compareTo(Employee o) { return Integer.compare(level, o.level); }   // natural order
}

List<Employee> list = new ArrayList<>(List.of(
        new Employee("Asha", 2, 4.1), new Employee("Bharat", 3, 3.9), new Employee("Chirag", 2, 4.8)));
```

```
Collections.sort(list);                                          // by level: Asha(2), Chirag(2), Bharat(3) — stable
list.sort(Comparator.comparingInt(Employee::level)
        .thenComparing(Employee::rating, Comparator.reverseOrder())
        .thenComparing(Employee::name));                         // level asc, rating desc, name asc
                                                                 // -> Chirag(2,4.8), Asha(2,4.1), Bharat(3,3.9)

PriorityQueue<Employee> best = new PriorityQueue<>(Comparator.comparingDouble(Employee::rating).reversed());
TreeMap<String, Integer> ci = new TreeMap<>(String.CASE_INSENSITIVE_ORDER);

// The overflow bug
Comparator<Integer> broken = (a, b) -> a - b;
System.out.println(broken.compare(Integer.MIN_VALUE, 1));   // positive (overflow!) -> MIN_VALUE sorts after 1
Comparator<Integer> correct = Integer::compare;
```

When would I use it?

- `Comparable` when a type has one obvious order: dates, money, version numbers.
- `Comparator` for everything else:
- sorting DTOs for the UI;
- heaps in DSA, like `PriorityQueue<int[]>` ordered by distance;
- leaderboards (score descending, then time ascending);
- `TreeMap` with custom keys.

What can go wrong?

- `a - b` **comparators overflow.** Always use `Integer.compare`, `Long.compare` or `comparingInt`.
- `compareTo` **inconsistent with** `equals` **in a** `TreeSet` **/** `TreeMap`**.** These collections use `compare`, not `equals`, to detect duplicates. So two different objects that compare as 0 are treated as duplicates. One is silently dropped. Example: ordering employees only by level.
- **Comparators that aren't transitive** (they give random or inconsistent results). TimSort can detect this and throws "Comparison method violates its general contract!".
- **Null elements.** `compareTo(null)` throws a `NullPointerException` (NPE). Use `Comparator.nullsFirst` or `nullsLast`.

Interview questions

Q1. Comparable vs Comparator? Comparable defines a class's single natural ordering, written inside the class. Comparator defines external orderings; you can combine them, and the class itself does not need to change.

Q2. Why not write `return a - b`**?** Because it can overflow for large values or values with mixed signs. Then the result has the wrong sign and the sort order is wrong.

Q3. What does "consistent with equals" mean and why does it matter? It means `compareTo` returns 0 exactly when `equals` returns true. Sorted collections use compare to decide uniqueness, so if the two disagree, distinct elements get dropped or merged.

Q4. Which algorithm does `List.sort` **use, and is it stable?** TimSort — a stable mix of merge sort and insertion sort, with O(n log n) worst case.

1.14 Immutable objects

Why?

> **In simple words:** An immutable object can never change after it is created. So many threads can share it safely, you can use it as a map key, and you never have to ask "who changed this?".

Once an immutable object is built, its state never changes. Because of that, it is:

- **Automatically thread-safe.** There is nothing to synchronise.
- **A safe map key.** Its hash code can never change.
- **Safe to cache and share.** Nobody can modify it behind your back.
- **Easy to reason about.** No "who changed this?" debugging.

Money, IDs, configuration snapshots and events should be immutable.

Think of a printed train ticket. Once printed, nobody edits it. If you change your journey, you get a new ticket.

What is happening internally?

- `final` **fields get safe-publication guarantees from the JMM** (Java Memory Model, the rules for what one thread sees of another thread's writes).
- Any thread that sees the object reference also sees the final fields fully set up. No locks are needed.
- This holds as long as `this` doesn't *escape* during construction — that is, the constructor doesn't hand `this` to another thread before it finishes.
- **Shallow vs deep immutability.**
- `final List<String> skus` stops you from pointing the field at a different list.
- But it does not stop `skus.add(...)`.
- True immutability needs the contents to be immutable too. You get this with defensive copies (your own private copy) or unmodifiable collections.
- **New objects are cheap.** The JVM and GC are tuned for many short-lived objects.
- Allocation is cheap because each thread has its own buffer to allocate from.
- Young-generation garbage collection (cleaning up new, short-lived objects) is cheap too.
- So "create a new object instead of changing the old one" is usually cheap.

How does it work?

Recipe:

1. Make the class `final` (or give it only private constructors). Then no subclass can add mutable state.
2. Make all fields `private final`.
3. Provide no setters.
4. Copy mutable inputs in the constructor (for example with `List.copyOf`). Never return your internal mutable objects.
5. For "changes", return a new instance. This method is often called a "wither", like `withSku(...)`.

Example

```
public final class Campaign {
    private final String id;
    private final List<String> skus;
    private final Instant start;           // java.time types are already immutable

    public Campaign(String id, List<String> skus, Instant start) {
        this.id = Objects.requireNonNull(id);
        this.skus = List.copyOf(skus);       // defensive copy + unmodifiable; caller's later edits can't
affect us
        this.start = Objects.requireNonNull(start);
    }
    public List<String> skus() { return skus; }           // safe: unmodifiable
    public Campaign withSku(String sku) {                  // "modification" = new object
        List<String> next = new ArrayList<>(skus); next.add(sku);
        return new Campaign(id, next, start);
    }
}

List<String> input = new ArrayList<>(List.of("A", "B"));
Campaign c = new Campaign("C1", input, Instant.now());
input.add("HACK");                         // no effect on c
// c.skus().add("X");                      // UnsupportedOperationException

// Record version (shallowly immutable — copy collections in the compact constructor)
record CampaignR(String id, List<String> skus) {
    CampaignR { skus = List.copyOf(skus); }
}
```

When would I use it?

Use immutable objects for:

- value objects (`Money`, `Sku`, `DateRange`);
- DTOs and API responses (records);
- events published to Kafka;
- configuration loaded at startup;
- cache values and map keys.

Keep mutable objects for entities with a lifecycle (JPA entities) and for builders.

What can go wrong?

- **Forgetting the defensive copy.** The caller still holds the original list and can change your "immutable" list.
- **Returning `Date` or arrays directly.** Both are mutable. Use `Instant` and return copies.
- **Too much copying of huge collections in hot loops.** Use persistent collections or builder patterns, or keep immutability only at API boundaries.
- **Records are only *shallowly* immutable.** A record holding an `ArrayList` can still be changed through that list.

Interview questions

Q1. How do you make a class immutable? Make the class final and all fields private final. Provide no setters. Make defensive copies of mutable inputs and outputs. And return new instances for any change.

Q2. Why are immutable objects thread-safe? Their state never changes after safe construction. So threads reading at the same time can never see half-updated state, and no locks are needed.

Q3. Is a class with only `final` fields immutable? Not necessarily. A final reference can still point to a mutable object — a list, an array, a `Date` — and that object must be protected too.

1.15 Exceptions: checked vs unchecked, try-with-resources

Why?

> **In simple words:** When something fails, Java throws an exception that jumps straight to the code that can handle it. Checked exceptions are ones the compiler forces you to deal with, like file and network errors.

Error codes have two problems. People forget to check them. And they mix error handling into the normal logic.

Exceptions fix both:

- They keep the "happy path" (normal flow) separate from failure handling.
- They carry context: a message, a stack trace (the list of method calls that led here), and a cause.

Java also has **checked exceptions**. The compiler forces you to handle or declare them. This is a reminder for recoverable failures like I/O and network errors. C# does not have this.

What is happening internally?

Here is the exception class tree:

```
Throwable
├── Error                   JVM-level problems: OutOfMemoryError, StackOverflowError — don't catch
└── Exception               checked (must be declared or caught): IOException, SQLException,
InterruptedException
    └── RuntimeException    unchecked: NullPointerException, IllegalArgumentException, IllegalStateException,
                            ArithmeticException, IndexOutOfBounds, ConcurrentModificationException
```

What happens when an exception is thrown:

1. The JVM creates the exception object.
2. It **captures the stack trace** by walking all the stack frames. This is the most expensive part.
3. Then it **unwinds** the stack, one frame at a time: - Each method has an **exception table** in its bytecode. It maps ranges of instructions to their handlers. - If a matching `catch` covers the current instruction, the JVM jumps there. - If not, it pops the frame (exits that method) and repeats in the caller.
4. If nobody catches it, the thread ends. The default handler prints the stack trace.

Other points:

- The **checked** rule exists only at compile time. The JVM itself treats checked and unchecked exceptions the same.
- **try-with-resources** compiles into a `finally`-like block. It calls `close()` on each resource, in reverse order of opening.
- If both the body and `close()` throw, the close exception is attached to the main one as a **suppressed** exception. So nothing is lost.

How does it work?

1. Code inside `try` throws an exception.

2. The first `catch` whose type matches runs. A subclass counts as a match. So order your catches from most specific to least specific.
3. `finally` always runs after `try` / `catch`, even if there was a `return`. It does not run on `System.exit` or a JVM crash.
4. To re-throw with more context, wrap it: `throw new LedgerException("posting failed for " + id, e)`. This keeps the original exception as the **cause**.

Example

```
// Domain exception: unchecked, carries context
public class InsufficientPointsException extends RuntimeException {
    private final String accountId; private final long requested, available;
    public InsufficientPointsException(String accountId, long requested, long available) {
        super("Account %s requested %d but has %d".formatted(accountId, requested, available));
        this.accountId = accountId; this.requested = requested; this.available = available;
    }
}

// try-with-resources + wrapping a checked exception at a boundary
public List<String> readSkus(Path file) {
    try (BufferedReader in = Files.newBufferedReader(file)) {
        return in.lines().map(String::strip).filter(s -> !s.isEmpty()).toList();
    } catch (IOException e) {
        throw new UncheckedIOException("could not read " + file, e);   // keep the cause
    }
}

// finally beats return — and a return in finally swallows exceptions (never do this)
static int f() { try { return 1; } finally { System.out.println("finally runs"); } }
static int g() { try { throw new RuntimeException("lost"); } finally { return 2; } }   // exception silently discarded

// InterruptedException: restore the flag if you can't rethrow
try { Thread.sleep(1000); } catch (InterruptedException e) { Thread.currentThread().interrupt(); return; }
```

Notes on the example:

- `f()` returns 1, but "finally runs" prints first.
- `g()` is a trap. The `return 2` inside `finally` throws away the exception. Never return from `finally`.
- For `InterruptedException`, if you can't re-throw, call `Thread.currentThread().interrupt()`. This sets the interrupt flag again so code higher up knows the thread was asked to stop.

When would I use it?

- **Unchecked exceptions** for programming errors and broken business rules (`IllegalArgumentException`, `InsufficientPointsException`).
- Spring and most modern code prefer unchecked exceptions.
- Map them to HTTP responses in one place, a `@ControllerAdvice` (Chapter 7.4).
- **Checked exceptions** in APIs where the caller can really recover (retry a file read, use a fallback). Or when a library forces them on you (I/O, JDBC). Wrap them at layer boundaries.
- **`try-with-resources`** for everything that is `AutoCloseable`: streams, readers, JDBC connections and statements, and locks wrapped in closeable helpers.

What can go wrong?

- **Swallowing.** `catch (Exception e) {}` hides failures. At the very least, log and re-throw — or handle it on purpose.

- **Losing the cause.** `throw new MyException(e.getMessage())` drops the original stack trace. Pass `e` as the cause.
- **Logging and re-throwing at every layer.** The same stack trace fills your logs five times. Log once, at the boundary.
- **Exceptions for control flow.** For example, using `NumberFormatException` to test whether input is numeric in a hot loop. This is slow, because each exception captures a stack trace.
- **Transactions.** Spring rolls back on unchecked exceptions by default, but **not on checked ones** (Chapter 7.12).

Interview questions

Q1. Checked vs unchecked exceptions? Checked exceptions extend `Exception` (but not `RuntimeException`), and you must declare or handle them. Unchecked exceptions extend `RuntimeException` and represent programming errors or conditions you can't recover from.

Q2. Does `finally` always execute? Yes, after try/catch — even when there is a `return`. The exceptions are `System.exit`, a JVM crash or an infinite loop. Also, a `return` inside `finally` overrides any exception or return value from the try block.

Q3. What are suppressed exceptions? In try-with-resources, if `close()` throws after the body already threw, the close exception does not replace the main one. Instead it is attached to the main exception using `addSuppressed`.

Q4. Should you catch `Exception` or `Throwable`? Catch the most specific type you can actually handle. Use a catch-all only at top-level boundaries, for logging and translating errors. Never swallow `Error`s like `OutOfMemoryError` (OOM).

1.16 Generics, type erasure and wildcards (PECS)

Why?

> **In simple words:** Generics let you say "this list holds only Strings", and the compiler checks it for you. After checking, Java removes the type details from the compiled code — this is called type erasure.

Before Java 5, collections held plain `Object`.

- Every read needed a cast.
- Wrong types were found only at runtime, as a `ClassCastException`.

Generics move these checks to **compile time**. A `List<String>` can't accept an `Integer`. And reading from it needs no cast.

What is happening internally?

Type erasure works in three steps:

1. The compiler checks all the generic types.
2. Then it **erases** them. Each type parameter is replaced by its bound. The bound is `Object` by default, or `Comparable` for `<T extends Comparable<T>>`.
3. Where a value comes out, the compiler inserts a cast.

So at runtime, `List<String>` and `List<Integer>` are the same class: `ArrayList`.

C# is different. C# generics are *reified* — the runtime knows the real type, such as `List<int>`. That is why C# allows `new T()` and `List<int>`.

```
Source:    List<String> l = new ArrayList<>();  String s = l.get(0);
Bytecode:  List l = new ArrayList();            String s = (String) l.get(0);   ← cast inserted
```

What erasure stops you from doing:

- **No primitive type arguments** like `List<int>`. The erased type is `Object`, and an `Object` can't hold a raw `int`.
- **No `new T()`, no `new T[n]`, no `obj instanceof List<String>`.** The type isn't known at runtime.
- **No overloading `f(List<String>)` with `f(List<Integer>)`.** Both erase to `f(List)`, so they clash.
- **Bridge methods.** The compiler generates small extra methods so that overriding still works after erasure.

How does it work?

Variance — whether `List<Child>` counts as a `List<Parent>`.

Generics are **invariant**: `List<String>` is **not** a `List<Object>`. Why? If it were, you could add an `Integer` to a list of strings through the `List<Object>` view. That would corrupt the list.

Wildcards (`?`) bring back flexibility, safely:

- **`List<? extends Number>`** means "a list of some subtype of Number".
- You can **read** items as `Number`.
- You can't add anything (except `null`).
- It is a **producer** — it gives you values.
- **`List<? super Integer>`** means "a list of `Integer` or some parent type of it".
- You can **add** `Integer`s.
- Reads give you only `Object`.
- It is a **consumer** — it takes your values.
- **PECS: Producer Extends, Consumer Super.**

Think of a tiffin box. If someone only *gives* you a box of "some kind of Indian food" (producer), you can eat from it, but you can't put random food in. If someone only *collects* boxes for "any food" (consumer), you can put your paneer in, but you don't know what you'll get out.

Example

```
// Bounded generic method: works for any comparable type
static <T extends Comparable<T>> T max(List<T> items) {
    T best = items.get(0);
    for (T t : items) if (t.compareTo(best) > 0) best = t;
    return best;
}

// PECS in practice
static double total(List<? extends Number> nums) {              // producer: we only read
    double s = 0; for (Number n : nums) s += n.doubleValue(); return s;
}
static void addDefaults(List<? super Integer> sink) {           // consumer: we only write Integers
    sink.add(0); sink.add(1);
}
total(List.of(1, 2, 3)); total(List.of(1.5, 2.5));             // List<Integer>, List<Double> both OK
addDefaults(new ArrayList<Number>()); addDefaults(new ArrayList<Object>());
```

```java
// Working around erasure: pass a type token or a factory
static <T> T create(Supplier<T> factory) { return factory.get(); }
static <T> T[] newArray(Class<T> type, int n) {
    @SuppressWarnings("unchecked") T[] a = (T[]) java.lang.reflect.Array.newInstance(type, n);
    return a;
}

// Arrays are covariant — and that's unsafe at runtime
Object[] objs = new String[1];
objs[0] = 42;            // compiles, throws ArrayStoreException at runtime
// List<Object> lo = new ArrayList<String>();   // doesn't compile — generics caught it at compile time
```

The last part shows why Java made generics invariant. Arrays are *covariant* (`String[]` counts as `Object[]`). So the bad store compiles and fails only at runtime with `ArrayStoreException`. With generics, the same mistake is caught at compile time.

When would I use it?

- **Every collection and every reusable component**: repositories (`Repository<T, ID>`), API wrappers (`ApiResponse<T>`), caches (`Cache<K, V>`), result types.
- **Wildcards in method *parameters*** of library-style utilities, like `Collections.copy(List<? super T> dst, List<? extends T> src)`. Avoid wildcards in return types — they make life hard for callers.

What can go wrong?

- **Raw types** like `List list = new ArrayList()`. They switch off type checking. This causes "heap pollution" — a wrong type gets into a collection, and the `ClassCastException` appears far away from the real bug.
- **Ignoring unchecked warnings.** Each warning marks a place where type safety can break.
- **Deserialising generic types with Jackson/Spring.** To read a `List<Order>`, they need type info that erasure removed. Use `TypeReference<List<Order>>` or `ParameterizedTypeReference`.
- **Overcomplicated bounds** like `<T extends Comparable<? super T>>` in business code. Keep generics simple outside libraries.

Interview questions

Q1. What is type erasure? The compiler checks generic types, then replaces type parameters with their bounds and inserts casts. So generic type information is not available at runtime.

Q2. What can't you do because of erasure? You can't use primitives as type arguments, create a `T` or a `T[]`, use `instanceof` with a parameterised type, or overload methods that differ only in their type arguments.

Q3. Explain PECS. Producer Extends, Consumer Super. Use `? extends T` when the structure produces values that you read. Use `? super T` when it consumes values that you write.

Q4. Why isn't `List<String>` a subtype of `List<Object>`? Because generics are invariant. Otherwise you could insert non-String objects through the `List<Object>` reference and corrupt the list.

1.17 Nested, inner, local and anonymous classes

Why?

> **In simple words:** Some small classes are only useful inside one bigger class, so you put them inside it. Be careful: a non-static inner class secretly holds on to its outer object, which can cause

> memory leaks.

Some classes make sense only inside another class:

- a `Node` inside a linked list;
- a `Builder` inside the class it builds;
- an `Iterator` that walks over a collection's internals.

Nesting keeps them close to their only user. It also lets them access the outer class's private members.

What is happening internally?

Kind	Declared	Holds a hidden reference to the outer instance?	Typical use
Static nested	`static class Node` inside the class	**No**	Builders, nodes, DTOs, helper types
Inner (non-static)	`class Itr` inside the class	**Yes** (`Outer.this`)	Iterators that need the outer object's state
Local	Inside a method	Yes (if inside an instance method), plus copies of captured effectively-final locals	Rare
Anonymous	`new Interface() { ... }`	Yes (if inside an instance method)	Callbacks, before lambdas existed

"Effectively final" means a local variable that is never changed after it is set, even if it isn't marked `final`.

How the hidden reference works:

1. An inner class object stores a hidden, compiler-generated field called `this$0`.
2. This field points to its outer object.
3. So as long as the inner object is alive, the outer object can't be garbage-collected (freed from memory).

Lambdas are smarter. They capture `this` only if they actually use instance members.

Think of a child holding a parent's hand in a crowd. As long as the child (inner object) stays, the parent (outer object) can't leave either.

How does it work?

```
public class Outer {
    private int secret = 42;
    static class StaticNested { int read(Outer o) { return o.secret; } }   // needs an explicit Outer
    class Inner { int read() { return secret; } }                          // uses the hidden Outer.this
}
Outer.StaticNested sn = new Outer.StaticNested();
Outer outer = new Outer();
Outer.Inner in = outer.new Inner();      // must be created from an outer instance
```

Key points from the code:

- A static nested class is created on its own. If it needs an outer object, you must pass one explicitly.
- An inner class must be created from an outer object, using the special `outer.new Inner()` syntax.

Example

Here is the leak, step by step:

1. Each `ReportService` holds a 50 MB buffer.
2. Each `Task` (an inner class) holds a hidden reference to its `ReportService`.
3. A long-lived static `QUEUE` holds the tasks.
4. So every task in the queue keeps a full 50 MB `ReportService` alive — even though `run()` never uses the buffer.

The fix makes `Task` a static nested class that stores only what it needs.

```
// The leak: a long-lived cache holding inner-class objects keeps every outer ReportService alive
class ReportService {
    private final byte[] bigBuffer = new byte[50_000_000];
    class Task implements Runnable { public void run() { /* doesn't even use bigBuffer */ } }
    Runnable task() { return new Task(); }      // each Task keeps a hidden ref to its ReportService (+ 50
MB)
}
static final List<Runnable> QUEUE = new ArrayList<>();
// QUEUE.add(new ReportService().task());  -> 50 MB retained per entry

// Fix: static nested class with only what it needs
class ReportService2 {
    static class Task implements Runnable { private final String reportId; Task(String id) { reportId = id; }
public void run() { } }
}
```

When would I use it?

- **Static nested by default**: `Node` in DSA linked lists and tries, `Builder`, internal value types.
- **Inner classes** only when the nested object truly needs the outer instance (for example, collection iterators).
- **Lambdas instead of anonymous classes** for single-method interfaces.

What can go wrong?

- **The memory leak shown above.** Inner classes or anonymous listeners live longer than their outer object should, and keep it in memory.
- **Serialisation.** Serialising an inner class object also serialises the outer object — or fails if the outer one can't be serialised.
- **Capturing loop variables in lambdas.** Captured variables must be effectively final. A loop counter changes, so copy it to a local variable first.

Interview questions

Q1. Static nested vs inner class? A static nested class has no reference to an outer instance, so you can create it on its own. An inner class silently holds a reference to its outer instance and needs one to be created.

Q2. How can an inner class cause a memory leak? Its hidden reference to the outer object keeps the outer object — and everything it references — reachable for as long as the inner object lives.

Q3. Why must captured local variables be effectively final? Lambdas and inner classes capture a *copy* of the value. If later changes were allowed, the copy and the original would drift apart, which is confusing — and across threads it would cause race conditions.

1.18 Enums

Why?

In simple words: An enum is a fixed list of allowed values, like SILVER, GOLD and PLATINUM. The compiler blocks anything outside the list. In Java, each value is a real object that can hold data and methods.

Suppose you store a fixed set of values — order status, tier, channel — as strings or ints. That invites typos and invalid values, like `"DELIVRED"` or `7`.

Enums fix this:

- The set of values is closed. Nothing else is allowed.
- The compiler checks the type.
- In Java, enums are full classes. They can carry data and behaviour.

Think of the coach classes on Indian Railways: SL, 3A, 2A, 1A. There is a fixed list. You can't book a "4A" ticket.

What is happening internally?

1. Each enum compiles to `final class Tier extends java.lang.Enum<Tier>`.
2. Each constant becomes one `public static final` instance of that class.
3. These instances are created once, when the class is first initialised.

That makes each constant a **singleton** (only one instance ever exists). So comparing with `==` is safe and correct.

Useful built-in methods:

- `values()` returns a copy of the array of all constants.
- `valueOf("GOLD")` finds a constant by its name.
- `ordinal()` gives the constant's position (0, 1, 2...).

Special collections:

- `EnumMap` is backed by an array indexed by ordinal.
- `EnumSet` is backed by a bit vector (one bit per constant).
- Both are extremely fast and compact.

How does it work?

- Constants can pass arguments to the enum's constructor.
- The enum can have fields and methods.
- Each constant can override an abstract method with its own version. This is *constant-specific behaviour* — a compact form of the Strategy pattern.

Example

```
public enum Tier {
    SILVER(1.0) { Tier next() { return GOLD; } },
    GOLD(1.5)   { Tier next() { return PLATINUM; } },
    PLATINUM(2.0) { Tier next() { return this; } };

    private final double multiplier;
```

```
    Tier(double m) { this.multiplier = m; }
    public long apply(long basePoints) { return Math.round(basePoints * multiplier); }
    abstract Tier next();                                    // constant-specific behaviour
}

Tier t = Tier.valueOf("GOLD");
System.out.println(t.apply(100) + " " + t.next());          // 150 PLATINUM

String label = switch (t) {                                  // exhaustive switch, no default needed
    case SILVER -> "Silver member";
    case GOLD -> "Gold member";
    case PLATINUM -> "Platinum member";
};

Map<Tier, Long> members = new EnumMap<>(Tier.class);         // array-backed, ordered by declaration
Set<Tier> premium = EnumSet.of(Tier.GOLD, Tier.PLATINUM);    // bit vector
```

The `switch` covers every constant, so it is *exhaustive* — the compiler knows nothing is missing and no `default` is needed.

When would I use it?

- Statuses and state machines (`OrderStatus` with allowed transitions).
- Types and categories.
- Configuration flags.
- Simple strategies, one per constant.
- Singletons: `enum IdGenerator { INSTANCE; }` is the safest singleton in Java.

What can go wrong?

- **Persisting `ordinal()`.** JPA uses `EnumType.ORDINAL` by default. If you insert a new constant in the middle, every number after it shifts. Every stored row is now wrong. Always use `@Enumerated(EnumType.STRING)`.
- **`valueOf` on bad input.** It throws `IllegalArgumentException` for unknown names, and it is case-sensitive. Validate external input first.
- **Adding a constant.** It breaks clients that switch over every constant. For public APIs, plan for unknown values.

Interview questions

Q1. Why are enums good singletons? The JVM guarantees exactly one instance per constant and thread-safe initialisation. It also blocks reflection and serialisation from creating extra copies.

Q2. Can enums have constructors, fields and methods? Yes. Constructors are automatically private, and each constant can override methods with its own behaviour.

Q3. Why store enums as strings in the database? Ordinals change when constants are reordered or inserted, and that silently corrupts stored data. Names are stable and readable.

1.19 Annotations and reflection

Why?

> **In simple words:** Annotations are labels you stick on code, like `@Service`. Reflection is how a framework reads those labels while the program runs and then acts on them — creating objects, filling fields, calling methods.

Frameworks need to know things about your code that the Java language itself doesn't say. For example:

- "This class is a Spring service."
- "This field maps to the column `tenant_id`."
- "This method is a test."

Annotations attach this extra information (metadata) to your code.

Reflection lets a framework read that metadata at runtime and act on your classes. It can create objects, inject fields and call methods.

Spring, Hibernate, Jackson and JUnit are all built on this pair.

Think of luggage tags at an airport. The tag (annotation) does nothing by itself. The baggage system (framework) scans the tag (reflection) and sends the bag to the right plane.

What is happening internally?

- **An annotation** is a special kind of interface, declared with `@interface`.
- Its **retention** decides how long it survives:
  - `SOURCE` — thrown away by the compiler (like `@Override`);
  - `CLASS` — kept in the bytecode but not visible at runtime (this is the default);
  - `RUNTIME` — readable through reflection. This is what frameworks need.
- `@Target` limits where you can put the annotation (class, field, method...).
- **Reflection** uses the classes `Class`, `Method`, `Field` and `Constructor`.
- It reads class metadata at runtime.
- It can call methods and read fields — even private ones, using `setAccessible(true)`. Since Java 9, module rules can block this.
- **Reflection is slower than a direct call.** It does access checks, boxes the arguments (wraps primitives into objects), and is not inlined at first. So frameworks speed things up by:
- caching metadata;
- using `MethodHandle`s (a faster way to call methods dynamically);
- generating bytecode or proxies at runtime (CGLIB, ByteBuddy);
- or doing the work at **compile time** with annotation processors (Lombok, MapStruct).

How does it work?

At startup, Spring does roughly this:

1. Scans the classpath for classes.
2. Reads `@Component` / `@Service` on them through reflection.
3. Finds their constructors.
4. Works out the dependencies each one needs.
5. Creates the beans (instantiates them).
6. Wraps some of them in proxies (for things like transactions).

Jackson works similarly. It uses reflection over getters, fields and record components to turn objects into JSON.

Example

This example defines a `@Masked` annotation. A small reflection method reads every field. If a field has `@Masked`, it prints `****` instead of the value.

```
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Masked {}                          // our own annotation: mask this field in logs

record Customer(String name, @Masked String phone) {}

static String toSafeString(Object o) throws IllegalAccessException {
    StringBuilder sb = new StringBuilder(o.getClass().getSimpleName()).append('{');
    for (Field f : o.getClass().getDeclaredFields()) {
        f.setAccessible(true);
        Object v = f.get(o);
        sb.append(f.getName()).append('=').append(f.isAnnotationPresent(Masked.class) ? "****" : v).append('
');
    }
    return sb.append('}').toString();
}
// toSafeString(new Customer("Asha", "9876543210")) -> Customer{name=Asha phone=**** }
```

When would I use it?

- Mostly you are a **user** of annotation-driven frameworks: Spring, JPA, validation, Jackson, JUnit.
- Write your own annotations for *cross-cutting concerns* — things needed in many places, like timing, auditing or masking (`@Timed`, `@Audited`, `@Masked`). Something must process them: an aspect or a validator.
- Reflection inside business code is rarely justified.

What can go wrong?

- **Annotations do nothing on their own.** Something must process them. A custom `@Timed` with no aspect behind it is just a comment.
- **Reflection breaks encapsulation and refactoring safety.** Method names live in strings, so renaming a method won't update them. It also fails with the module system or GraalVM native images unless you configure it.
- **Performance.** Reflective calls in hot paths, without caching, are slow.

Interview questions

Q1. What is annotation retention? It defines how long an annotation lives: `SOURCE` (only for the compiler), `CLASS` (in the bytecode, but not at runtime), or `RUNTIME` (available through reflection, which frameworks need).

Q2. How does Spring use reflection? It scans classes, reads their annotations, finds constructors and fields, creates and injects beans, and builds proxies. It caches this metadata for performance.

Q3. What are the downsides of reflection? Slower calls, broken encapsulation, loss of compile-time safety, and problems with the module system and native images.

2. Collections Framework Internals

This is the Java topic that Indian product companies ask about the most. At SDE2 level, saying "HashMap is O(1)" is not enough. You must explain *why* it is fast. You must also say what makes it slow, and how each structure is laid out in memory.

2.1 The collections hierarchy and choosing the right one

Why?

> **In simple words:** Java gives you a few standard "containers" for groups of data. Each one is good at some jobs and bad at others. Pick the one that matches how you will read and write the data.

Every program keeps groups of things: orders, users, IDs. If you pick the wrong structure, two bad things can happen:

- A fast operation becomes slow. For example, an O(1) lookup (constant time — same speed for any size) turns into O(n) (time grows with the number of items).
- You waste memory.

The Collections Framework gives you a small set of interfaces: `List`, `Set`, `Queue`, `Deque` and `Map`. Each interface has several implementations that you can swap. So you write your code against the interface. Then you pick the implementation based on your *access pattern* — how you will add, find and remove items.

What is happening internally?

Here is the family tree of the main types:

```
Iterable
└── Collection
    ├── List   : ArrayList (resizable array) · LinkedList (doubly linked nodes) · CopyOnWriteArrayList
    ├── Set    : HashSet (HashMap inside) · LinkedHashSet · TreeSet (TreeMap inside) · EnumSet (bit vector)
    └── Queue  : PriorityQueue (binary heap) · ArrayDeque · BlockingQueues
        └── Deque : ArrayDeque (circular array) · LinkedList
Map (separate root): HashMap · LinkedHashMap · TreeMap (red-black tree) · EnumMap · ConcurrentHashMap ·
WeakHashMap
```

Under the hood, almost every class uses one of only four memory layouts:

1. A **contiguous array** — items sit side by side in one block of memory.
2. **Linked nodes** — each item is in its own small object that points to the next one.
3. A **hash table** — the key is turned into a number that picks a slot.
4. A **balanced tree** — items are kept sorted in a tree that never gets too tall.

If you know which layout sits under a class, you can work out its speed for each operation. You do not need to memorise tables.

How does it work?

Choose by asking yourself a few simple questions:

- Do I need the items kept in order?
- Must every item be unique?

- Do I look things up by a key?
- Do I need sorted order, or ranges like "between X and Y"?
- Do I need the smallest or largest item quickly?
- Will many threads use it at the same time?

Then match your answer to this table. ("Amortised O(1)" means usually instant, sometimes slow, but fast on average.)

Requirement	Pick	Layout	Key operations
Indexed list, append	`ArrayList`	Array	get O(1), add O(1) amortised
Stack or queue	`ArrayDeque`	Circular array	push/pop/offer/poll O(1)
Key → value, fast	`HashMap`	Hash table	get/put O(1) average
Key → value in insertion/access order	`LinkedHashMap`	Hash table + linked list	O(1) + ordered iteration
Sorted keys, floor/ceiling/ranges	`TreeMap`	Red-black tree	O(log n)
Unique elements	`HashSet` / `TreeSet`	Same as maps	O(1) / O(log n)
Repeatedly take min/max	`PriorityQueue`	Binary heap	offer/poll O(log n), peek O(1)
Enum keys or flags	`EnumMap` / `EnumSet`	Array / bit vector	O(1), tiny memory
Shared across threads	`ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue`	Chapter 4	

Coming from C#? Here is the mapping:

- `List<T>` → `ArrayList`
- `Dictionary` → `HashMap`
- `SortedDictionary` → `TreeMap`
- `Queue` / `Stack` → `ArrayDeque`
- `PriorityQueue<T,P>` → `PriorityQueue<T>` with a `Comparator`
- `ConcurrentDictionary` → `ConcurrentHashMap`

Example

Declare the variable with the interface type. Create the object with the implementation class.

```
List<Order> recent = new ArrayList<>();                // declare by interface, construct the implementation
Map<String, Long> pointsByOutlet = new HashMap<>();
NavigableMap<Instant, Price> priceHistory = new TreeMap<>();
Deque<Integer> stack = new ArrayDeque<>();
Queue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));
```

When would I use it?

On every task. In an interview, say your choice and the reason in one sentence. For example: "I'll use a TreeMap, because I need the nearest earlier timestamp (`floorKey`) in O(log n)."

What can go wrong?

- **Using legacy classes.** `Vector`, `Stack` and `Hashtable` lock (synchronise) on every single call, so they are slow. Use `ArrayList`, `ArrayDeque`, and `HashMap` or `ConcurrentHashMap` instead.
- **Choosing `LinkedList` "because inserts are O(1)".** This is usually wrong in practice (see 2.3).
- **Using a `List` for "is it present?" checks inside a loop.** `list.contains` is O(n) — it checks items one by one. Inside a loop, the whole thing becomes $O(n^2)$. Use a `HashSet` instead.

Interview questions

Q1. Which collection would you use for X? Decide by how the data will be accessed: random access by index → ArrayList; lookup by key → HashMap; sorted order or ranges → TreeMap; take the min or max again and again → PriorityQueue; first-in-first-out (FIFO) or last-in-first-out (LIFO) → ArrayDeque.

Q2. Why is Map not part of the Collection interface? A map holds key-value pairs and works through keys. It does not fit the `Collection` contract, which is about single elements added with `add(E)`. Instead, a map gives you collection *views* of itself: `keySet`, `values` and `entrySet`.

2.2 ArrayList

Why?

> **In simple words:** An ArrayList is a normal array that grows by itself. When it is full, Java makes a bigger array and copies everything across.

Arrays are the fastest structure when you read by index. But an array has a fixed size. `ArrayList` is a *growable* array. You get array speed, plus good *locality* (items sit close together in memory, so the CPU reads them fast). And it resizes itself automatically.

What is happening internally?

Inside, there are just two fields:

```
transient Object[] elementData;   // backing array (references to elements)
private int size;                 // how many slots are used
```

`size` is how many items you have stored. *Capacity* is the length of the backing array — how many items fit before it must grow.

```
size = 5, capacity = 10
elementData: [ A | B | C | D | E | _ | _ | _ | _ | _ ]
```

Key facts:

- `new ArrayList<>()` starts with a shared empty array. The **first add** creates the real array with capacity **10**.
- When the array is full, `grow()` makes a new array **1.5× the old capacity** (`old + (old >> 1)`). It copies everything with `Arrays.copyOf`, which is a fast native memory copy.

- The array holds **references** (pointers) to your objects, side by side. So looping over it is friendly to the CPU cache. The objects themselves live elsewhere on the heap.
- A counter called `modCount` goes up on every structural change (add or remove). Iterators use it to detect changes and fail fast (see 2.10).

Think of a tiffin box with 10 compartments. When all are full, you buy a box with 15 compartments and move the food across.

How does it work?

- `get(i)` : check that `i` is in range, then return `elementData[i]` → **O(1)**.
- `add(e)` : if `size == capacity`, grow first (an O(n) copy). Then store at position `size` and increase `size`. Growth is rare and the array grows by a multiple each time. So across n adds, the total copying is only O(n). That makes each add **amortised O(1)** — usually instant, occasionally slow, fast on average.
- `add(i, e)` / `remove(i)` : `System.arraycopy` shifts all items after `i` right or left → **O(n – i)**.
- `remove(Object)` : search item by item using `equals`, then shift → O(n).
- `contains` / `indexOf` : check item by item → O(n).

Why is the average add O(1)? Start at 10 and grow ×1.5 each time. The copies are 10 + 15 + 22 + … and this total stays below 3n. Spread over n adds, each add costs O(1) on average.

Example

```
List<Integer> ids = new ArrayList<>(expectedCount);   // pre-size: avoids ~log1.5(n) reallocations
for (int i = 0; i < 1_000_000; i++) ids.add(i);

ids.remove(5);                      // removes INDEX 5
ids.remove(Integer.valueOf(5));     // removes VALUE 5  — the classic trap
ids.removeIf(x -> x % 2 == 0);      // single pass, O(n) — better than removing in a loop
List<Integer> page = ids.subList(0, 20);   // VIEW backed by ids: modifying ids structurally invalidates it
ids.trimToSize();                   // shrink capacity to size after bulk loading
```

When would I use it?

It is the default list for almost everything: query results, DTO lists, DSA arrays when you don't know the size up front, building responses. It is best at two things: adding at the end, and looping over items.

What can go wrong?

- **`remove(int)` vs `remove(Object)` confusion** with a `List<Integer>`. `remove(5)` removes the item at index 5, not the value 5.
- **Removing items one by one from the front, in a loop, is O(n²).** Each removal shifts every item after it. Use `removeIf`, loop from the back, or use an `ArrayDeque`.
- **`subList` is a view, not a copy.** If you then add to or remove from the parent list, using the view throws `ConcurrentModificationException`.
- **Not thread-safe.** If two threads add at the same time, items can get lost. You may also see `ArrayIndexOutOfBoundsException` during a resize.
- **Huge lists of boxed numbers waste memory.** Each `Integer` is a full object. For millions of numbers, prefer `int[]` or a primitive collection library.

Interview questions

Q1. How does ArrayList grow? When the array is full, it creates a new array 1.5× larger and copies the elements over. Because it grows by a multiple each time (geometric growth), adding at the end is amortised O(1).

Q2. What is the default capacity? 10. It is created lazily, only on the first add.

Q3. Time complexity of get, add, add(index), remove(index)? O(1), amortised O(1), O(n), O(n).

2.3 LinkedList

Why?

> **In simple words:** A LinkedList is a chain of small boxes, each holding one item and pointing to the box before and after it. Adding or removing in the middle is cheap only if you are already standing at that spot.

In a doubly linked list, each element knows its previous and next neighbour. So you can insert or remove an element in O(1), without shifting anything. But there is a condition: **you must already hold a reference to the neighbour** (for example, an iterator's current position). `LinkedList` implements both `List` and `Deque` (a double-ended queue — you can add or remove at both ends).

What is happening internally?

```
private static class Node<E> { E item; Node<E> next; Node<E> prev; }
transient Node<E> first, last;
```

```
first → [prev|A|next] ⇄ [prev|B|next] ⇄ [prev|C|next] ← last
```

Every element needs its own `Node` object. A node has an object header (~12 bytes) plus 3 references. That is roughly **24–32 bytes per element**, on top of the element itself.

These nodes are scattered across the heap. So the CPU cannot read them in one smooth run. This is called *poor cache locality*, and it makes the list slow to walk through.

Think of a treasure hunt. Each clue tells you where the next clue is. To reach clue 50, you must visit all 49 before it.

How does it work?

- `addFirst` / `addLast` / `removeFirst` / `removeLast`: change a few pointers → **O(1)**.
- `get(i)`: walk from whichever end is closer → **O(n)**.
- `add(i, e)`: O(n) to walk to the position, then O(1) to relink.
- `ListIterator.add/remove` at the iterator's current position: O(1), because you are already there.

Example

```
LinkedList<String> ll = new LinkedList<>(List.of("a", "b", "c"));
ll.addFirst("start"); ll.addLast("end");          // O(1)
ListIterator<String> it = ll.listIterator();
while (it.hasNext()) if (it.next().equals("b")) it.add("b2");   // O(1) insert at the cursor

// The trap: index loops are O(n^2)
for (int i = 0; i < ll.size(); i++) ll.get(i);     // each get walks the list
```

When would I use it?

Rarely. Even for queues and stacks, `ArrayDeque` is faster and uses less memory.

Real uses are few:

- You hold iterators and keep inserting or removing in the middle all the time.
- You need a list that is also a deque, and you need to store `null`.

In DSA, you usually write your own node class instead (LRU cache, LeetCode `ListNode`). You don't use `LinkedList` there.

What can go wrong?

- **Calling `get(i)` inside a loop.** Each `get` walks the list, so the loop takes quadratic time — O(n²).
- **Memory.** It uses 3–5× more memory than an `ArrayList` with the same elements.
- **It loses in practice.** In benchmarks, `ArrayList` usually beats `LinkedList` even for middle inserts on lists of moderate size. Why? `System.arraycopy` moves contiguous memory extremely fast. Following pointers, on the other hand, keeps missing the CPU cache.

Interview questions

Q1. ArrayList vs LinkedList? ArrayList is a contiguous array: O(1) random access, amortised O(1) append, cache-friendly, and low memory. LinkedList is a chain of nodes: O(1) insert/remove at the ends or at a position you already hold, but O(n) access by index and high memory use. Choose ArrayList by default.

Q2. When is LinkedList actually better? When you insert or remove often at an iterator position inside a large list, or when you need a Deque that allows nulls. In other cases, ArrayDeque or ArrayList wins.

2.4 HashMap internals

Why?

> **In simple words:** A HashMap is a set of numbered boxes. From the key itself, Java calculates which box the value goes in — so to find it again, it jumps straight to that box instead of searching everywhere.

You often need to find something by a key, fast. For example: find a user by ID, count how many times each word appears, or look up a product by SKU.

If you keep the data in a list, you must check items one by one. With a million items, that can be a million checks. This is O(n) — the work grows with the size of the data.

A hash table avoids this. It uses the key to calculate **where** the value should be stored. Finding it later means doing the same calculation and going straight there. So a lookup takes about the same time whether you have 10 items or 10 million. This is called O(1) on average.

Think of a railway parcel office with 16 numbered racks. The clerk does a small sum on your receipt number to pick a rack. When you come back, he does the same sum and walks straight to the right rack.

What is happening internally?

The main fields:

```
transient Node<K,V>[] table;        // the bucket array; length is ALWAYS a power of two
static class Node<K,V> { final int hash; final K key; V value; Node<K,V> next; }
int size;  int threshold;           // threshold = capacity × loadFactor
final float loadFactor;             // default 0.75
// defaults: capacity 16 (allocated lazily on first put), threshold 12
```

A few terms:

- **Bucket** — one slot in the `table` array.
- **Capacity** — the number of buckets (the array length).
- **Load factor** — how full the table may get before it grows. Default 0.75, meaning 75% full.
- **Threshold** — the size at which the table grows: capacity × load factor. For the default, 16 × 0.75 = 12.

Here is what the table can look like:

```
table (capacity 16)
 [0] → null
 [1] → (hash=17,"apple",3) → (hash=33,"mango",9) → null      ← collision: chain (linked list)
 [2] → null
 ...
 [5] → TreeNode (red-black tree)                              ← chain grew ≥ 8 → treeified
 ...
[15] → (hash=31,"kiwi",1) → null
```

A *collision* is when two different keys land in the same bucket. They are then kept in a small linked list (a *chain*) in that bucket.

Key constants:

- `TREEIFY_THRESHOLD = 8` — a chain this long may turn into a tree.
- `UNTREEIFY_THRESHOLD = 6` — a tree this small turns back into a chain.
- `MIN_TREEIFY_CAPACITY = 64` — the table needs at least this many buckets before any tree is made.

How does it work?

`put(key, value)` in steps:

1. **Hash spreading.** Java computes `h = key.hashCode(); hash = h ^ (h >>> 16)`. This mixes (XORs) the top 16 bits into the bottom 16 bits. Why? Only the low bits are used to choose the bucket. Without mixing, keys whose hashCodes differ only in the high bits would all land in the same bucket.
2. **Index.** Java computes `i = (n - 1) & hash`. Here `n` is always a power of two, so `n - 1` is all ones in binary (16 → `1111`). So this `&` is a fast way to do `hash % n`.
3. **Empty bucket?** If `table[i]` is empty, put a new Node there. Done.
4. **Look for the same key.** Otherwise, walk through the bucket. If a node has the **same hash and `key.equals()` is true**, replace its value and return the old value.
5. **Add a new node.** If no node matched, **append** the new node at the end of the chain. (Java 8+ adds at the tail; Java 7 added at the head.)
6. **Chain too long?** If the chain length reaches **8**: - If the table has at least **64** buckets, convert this bucket into a **red-black tree** (a self-balancing sorted tree). Now the worst case in this bucket is O(log n). - Otherwise, **resize** the table instead. In a small table, too few buckets is the more likely reason for collisions.
7. **Too full?** If `++size > threshold`, **resize** the table.

`get(key)` in steps:

1. Compute the same hash and the same index.
2. Check the first node in that bucket.
3. If it is not a match, walk the list or search the tree.
4. At each node, compare the hash first (cheap). Call `equals` only if the hashes match.

Resize in steps:

1. Make a new table with double the capacity.
2. For each node, check just **one bit**: `(hash & oldCap)`.
3. If that bit is `0`, the node stays at index `i`.
4. If that bit is `1`, the node moves to `i + oldCap`.

No hash is recomputed. Each old chain simply splits into a "low" chain and a "high" chain, and the order inside each chain is kept.

```
oldCap = 16 (10000b). Node hash ...0 0101 → bit 16 is 0 → stays at 5
                      Node hash ...1 0101 → bit 16 is 1 → moves to 5 + 16 = 21
```

Why 0.75 and 8?

- 0.75 is a balance. A lower value wastes many empty buckets. A higher value makes chains longer.
- With a decent hash function, chain lengths follow roughly a *Poisson distribution* (a standard pattern for random events). At this load the average chain length is about 0.5.
- So the chance of one chain reaching 8 is about 0.00000006 — practically never.
- That means treeification is a safety net. It protects against bad hashCodes or deliberate collision attacks. It is not the normal path.

Example

```java
Map<String, Integer> freq = new HashMap<>();
for (String w : "to be or not to be".split(" ")) freq.merge(w, 1, Integer::sum);
System.out.println(freq);    // {not=1, or=1, be=2, to=2} — bucket order, not insertion order; never rely on it

// Trace of put("to", 1) into an empty default map:
//  1. table allocated with capacity 16
//  2. "to".hashCode() = 3707; spread: 3707 ^ (3707 >>> 16) = 3707
//  3. index = 15 & 3707 = 11  (3707 = ...1110 0111 1011; low 4 bits 1011 = 11)
//  4. table[11] empty → store Node(3707, "to", 1)
//  later put("to", ...) → same index 11 → hash equal & "to".equals("to") → value updated

// Java 8 API that makes DSA code shorter and atomic-per-call
Map<Integer, List<String>> byLength = new HashMap<>();
for (String w : words) byLength.computeIfAbsent(w.length(), k -> new ArrayList<>()).add(w);
freq.getOrDefault("xyz", 0);
freq.computeIfPresent("be", (k, v) -> v > 1 ? v - 1 : null);   // returning null REMOVES the entry

// Pre-size to avoid resizes when you know n
Map<String, Order> byId = HashMap.newHashMap(100_000);          // Java 19+; else new HashMap<>((int)(n/0.75f)+1)
```

When would I use it?

All the time:

- Caches and lookups by ID.
- Counting and grouping.
- Memoisation in DSA — saving answers you already computed (`Map<String, Integer> memo`).

- Removing duplicates.
- Two-sum style problems, where you look up the "complement" you need.
- Adjacency lists for sparse graphs (graphs with few edges).

What can go wrong?

- **Bad or mutable keys.** If a key class has no proper `hashCode` / `equals`, or you change a key after putting it in (1.12), the entry can no longer be found.
- **All keys colliding.** For example, a hashCode that returns a constant. Then every key lands in one bucket, and lookups slow down to O(n). After treeification it is O(log n), but the tree works well only if the keys are `Comparable`.
- **Iteration order is not stable.** It can change after a resize, or between Java versions. Never depend on it. Use `LinkedHashMap` or `TreeMap` when order matters.
- **Not thread-safe.** Two threads calling `put` together can lose updates and corrupt the size. In Java 7, a concurrent resize could even create a loop in a chain, so `get` ran forever. Use `ConcurrentHashMap` (4.9).
- **Memory.** Each entry is a Node object (~32–48 bytes), plus the key and value objects. A map of a million `Long → Long` takes about 80+ MB.
- **Double lookups.** Looping over `keySet()` and calling `get(k)` for each key does two lookups per key. Loop over `entrySet()` instead.

Interview questions

Q1. Explain HashMap internals. It is an array of buckets whose length is always a power of two. The key's hashCode is spread (`h ^ h>>>16`) and masked to get an index. Keys that collide form a linked list in the bucket; that list becomes a red-black tree when a chain reaches 8 and the table has 64+ buckets. A lookup compares the hash first, then `equals`. When size goes above capacity × 0.75, the table doubles, and each node either stays at `i` or moves to `i + oldCap`, decided by one bit.

Q2. Why is HashMap O(1) and when isn't it? A good hash spreads keys evenly, so each bucket holds about one entry; a lookup is just index arithmetic plus one or two comparisons. It gets slow when many keys collide because of a poor hashCode — O(n) per bucket, or O(log n) once the bucket becomes a tree.

Q3. Why must the capacity be a power of two? So that `(n-1) & hash` can replace the slower modulo (`%`), and so that a resize can split each bucket by checking a single bit.

Q4. What changed in Java 8? Long chains are turned into trees (treeification); new nodes are added at the tail instead of the head, which fixed Java 7's infinite loop during concurrent resize; and the hash spreading was simplified.

Q5. Can HashMap have a null key? Yes, one. Its hash is treated as 0, so it is stored in bucket 0. Values can be null too. `ConcurrentHashMap` allows neither null keys nor null values.

Q6. What happens when two keys have the same hashCode? Both go into the same bucket, and `equals` tells them apart. Both are stored and found correctly — it just costs one extra comparison.

2.5 LinkedHashMap and building an LRU cache

Why?

> **In simple words:** A LinkedHashMap is a HashMap that also remembers order. It can remember the order you added keys, or the order you last used them — and the second one gives you an LRU cache almost for free.

`HashMap` gives no promise about order. Sometimes you need fast lookup **and** a predictable order. Two common cases:

- **Insertion order** — the order in which keys were first added. Useful to keep JSON fields in order, or to get the same output every run.
- **Access order** — the order in which keys were last used. Useful to throw out the *least recently used* entry. This is an **LRU cache** (Least Recently Used).

What is happening internally?

`LinkedHashMap` extends `HashMap`. Each entry gets two extra pointers, `before` and `after`. These pointers join **all entries into one doubly linked list**. The map also keeps `head` and `tail` pointers to the two ends of that list.

```
hash table buckets: normal HashMap structure (for O(1) lookup)
linked list:  head ⇄ [k1] ⇄ [k2] ⇄ [k3] ⇄ tail     (insertion or access order)
```

When `accessOrder = true`:

- Every `get` or `put` of an existing key **moves that entry to the tail**. The tail is the "most recently used" end.
- After each insertion, the map calls `removeEldestEntry(head)`. If your method returns true, the head is removed. The head is the least recently used entry.

Think of a stack of newspapers on a table. Whenever you read one, you put it back on top. When the table is full, you throw away the one at the bottom — the one nobody has touched for longest.

How does it work?

1. `get(k)`: a normal hash lookup, O(1). Then the node is unlinked and relinked at the tail, also O(1).
2. `put(k, v)` for a new key: the entry goes into its bucket and is appended at the tail. Then `removeEldestEntry` is asked whether to remove the head.
3. Iteration walks the linked list. So it costs O(size). A plain HashMap must scan every bucket, which costs O(capacity + size).

Example

```java
public class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;
    public LruCache(int capacity) {
        super(capacity, 0.75f, true);            // true = access order
        this.capacity = capacity;
    }
    @Override protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}

LruCache<String, String> c = new LruCache<>(2);
```

```
c.put("a", "1"); c.put("b", "2");     // order: a, b
c.get("a");                           // order: b, a   (a moved to most-recent end)
c.put("c", "3");                      // insert c → size 3 > 2 → evict eldest = b
System.out.println(c.keySet());       // [a, c]
```

When would I use it?

- Small caches inside one process (per request or per instance).
- When output must always come in the same order.
- When you need LRU behaviour quickly.

In LeetCode 146 (LRU Cache), interviewers usually want the manual HashMap + doubly linked list version (10.3). Mention this one as the production shortcut. For real production caches, use **Caffeine** — it has better hit rates, handles concurrency, and supports TTL (time-to-live, i.e. entries expire after a set time).

What can go wrong?

- **Not thread-safe.** In access-order mode, even `get` changes the structure (it moves the entry). So two threads that only read can still corrupt it. Guard it with a lock, or use Caffeine.
- **ConcurrentModificationException with access order.** If you call `get` while looping over the map, it throws. This is because `get` counts as a structural change in this mode.
- **No TTL.** It cannot expire entries by time. It also cannot evict by weight (for example, by how many bytes an entry uses).

Interview questions

Q1. How does LinkedHashMap maintain order? A doubly linked list runs through all the entries. It keeps insertion order by default, or access order when `accessOrder` is true.

Q2. How do you build an LRU cache in Java quickly? Extend LinkedHashMap with `accessOrder=true`, and override `removeEldestEntry` to return `size() > capacity`.

Q3. Is that LRU cache thread-safe? No — even reads change the order. Synchronise access to it, or use Caffeine or `ConcurrentLinkedHashMap`-style libraries.

2.6 TreeMap and TreeSet

Why?

> **In simple words:** A TreeMap keeps its keys sorted all the time. So it can answer "what is just before or just after this key?" and "give me everything between A and B" quickly.

A hash map cannot answer questions about order. For example:

- "What was the price at 14:05?" — that is, the latest price change at or before that time.
- "Give me all orders between two timestamps."
- "What is the smallest key greater than x?"
- "Give me the top or bottom N."

A **sorted** structure answers all of these in O(log n) — the work grows very slowly, roughly with the number of times you can halve n.

What is happening internally?

`TreeMap` is a **red-black tree**. That is a *binary search tree* (smaller keys go left, larger keys go right) that balances itself.

It follows four rules:

1. Every node is red or black.
2. The root is black.
3. A red node has only black children.
4. Every path from the root down to a leaf has the same number of black nodes.

Because of these rules, the tree's height stays at most $2 \cdot \log_2(n+1)$. So search, insert and delete are **O(log n)**, guaranteed — even in the worst case.

After an insert or a delete, the tree may break the rules for a moment. It fixes them with **rotations** (small re-arrangements of a few nodes) and by recolouring nodes. Insert needs at most 2 rotations; delete needs at most 3.

```
           (20,B)
          /      \
      (10,R)    (30,R)
      /   \
   (5,B) (15,B)
in-order traversal = 5, 10, 15, 20, 30  → sorted iteration
```

How are keys ordered? By the key's `compareTo` method (if the key is `Comparable`), or by a `Comparator` you pass in. `equals` is not used at all.

`TreeSet` is simply a `TreeMap` where every value is the same dummy object.

Think of a dictionary book. Words are always in alphabetical order. So you can open it in the middle and quickly narrow down to the word — or to the word just before it.

How does it work?

Example: `floorKey(25)` — find the largest key that is ≤ 25, using the tree above.

1. Start at the root, 20. Since 20 ≤ 25, remember 20 as a possible answer and go right.
2. Now at 30. Since 30 > 25, go left.
3. There is nothing on the left. Stop.
4. The answer is the last remembered key: 20.

Each step throws away about half of the remaining tree. So the search is O(log n).

Example

```java
TreeMap<LocalTime, Long> priceChanges = new TreeMap<>();     // effective time -> price
priceChanges.put(LocalTime.of(9, 0), 100L);
priceChanges.put(LocalTime.of(13, 30), 110L);
priceChanges.put(LocalTime.of(17, 0), 95L);

priceChanges.floorEntry(LocalTime.of(14, 5)).getValue();     // 110 — latest change at or before 14:05
priceChanges.ceilingKey(LocalTime.of(14, 5));                // 17:00
priceChanges.headMap(LocalTime.of(13, 30));                  // keys < 13:30 (a live VIEW)
priceChanges.subMap(LocalTime.of(9, 0), true, LocalTime.of(17, 0), false);
priceChanges.firstKey(); priceChanges.lastEntry(); priceChanges.pollFirstEntry();
priceChanges.descendingMap();

// DSA: sliding window with duplicates -> multiset via TreeMap<value, count>
```

```java
TreeMap<Integer, Integer> window = new TreeMap<>();
window.merge(5, 1, Integer::sum);
window.computeIfPresent(5, (k, v) -> v == 1 ? null : v - 1);  // remove one occurrence
int maxInWindow = window.lastKey();
```

When would I use it?

- **Time-series lookups**: rates, prices or configs that were effective at a given time.
- **Calendar and booking conflicts**: find the neighbouring bookings of a new interval.
- **Leaderboards** where you loop in rank order.
- **Range queries**: everything between A and B.
- **DSA problems that need a sorted multiset** (a sorted collection that allows duplicates): sliding window median, "contains nearby almost duplicate", "my calendar".

What can go wrong?

- **Comparator inconsistent with equals.** If two different keys compare as 0, TreeMap treats them as the **same key**. One silently overwrites the other.
- `null` **keys throw NPE** (NullPointerException), because null cannot be compared. The only exception is a comparator that handles null.
- **Slower than HashMap for plain lookups.** It is O(log n) and must follow pointers down the tree. So use it only when you need order.
- `headMap` / `subMap` **are views.** Changing them changes the original map too.

Interview questions

Q1. TreeMap vs HashMap? TreeMap keeps keys sorted, is O(log n), has navigation methods (floor, ceiling, ranges), needs Comparable keys or a Comparator, and allows no null keys. HashMap has no order, is O(1) on average, and uses hashCode/equals.

Q2. What data structure backs TreeMap? A red-black tree — a self-balancing binary search tree (BST) with height ≤ $2 \cdot \log_2(n+1)$. So every operation is O(log n) even in the worst case.

Q3. How would you find the latest price effective at a given time? Store the price changes in a TreeMap keyed by effective time, and call `floorEntry(time)`.

2.7 HashSet, LinkedHashSet and EnumSet

Why?

> **In simple words:** A set is a bag that never holds the same thing twice. Use it to ask "have I already seen this?" in one quick step.

You use a set to:

- Answer "have I seen this before?" in O(1).
- Remove duplicates.
- Do set operations like union (everything in either) and intersection (only what is in both), without writing nested loops.

What is happening internally?

```
public class HashSet<E> { private transient HashMap<E, Object> map;
    private static final Object PRESENT = new Object();
    public boolean add(E e) { return map.put(e, PRESENT) == null; } }
```

A `HashSet` **is really a HashMap**. Your elements are the keys. Every value is the same shared dummy object. So everything in 2.4 applies here too, including the hashCode/equals rules.

The other sets follow the same idea:

- `LinkedHashSet` uses a LinkedHashMap inside, so it keeps insertion order.
- `TreeSet` uses a TreeMap inside, so it keeps elements sorted.
- `EnumSet` is different. It stores one bit per enum constant inside a `long` (or a `long[]` if there are more than 64 constants). So checking membership is a single bit operation — extremely fast and tiny in memory.

Think of a guest register at a wedding gate. Before writing a name, the guard checks whether it is already there. If it is, he does not write it again.

How does it work?

`add(e)` returns `false` if `e` was already in the set. So "detect a duplicate while adding" takes just one line.

For set algebra, use the bulk methods:

- `retainAll` — intersection (keep only common items).
- `addAll` — union (add everything).
- `removeAll` — difference (remove the other set's items).

Example

```
Set<String> seen = new HashSet<>();
for (String eventId : incomingEventIds)
    if (!seen.add(eventId)) System.out.println("duplicate " + eventId);    // dedupe in one pass

Set<String> eligible = new HashSet<>(outletsInScheme);
eligible.retainAll(activeOutlets);                // intersection
Set<String> ordered = new LinkedHashSet<>(List.of("c", "a", "b", "a"));   // [c, a, b]
EnumSet<DayOfWeek> weekend = EnumSet.of(DayOfWeek.SATURDAY, DayOfWeek.SUNDAY);
```

When would I use it?

- Removing duplicates.
- "Visited" sets in BFS/DFS (graph searches).
- Membership checks inside loops.
- Set algebra across lists of IDs.
- "First unique element" problems (use LinkedHashSet, since it keeps order).
- Flags (use EnumSet).

What can go wrong?

The same problems as HashMap keys: a missing `hashCode` / `equals`, or elements that change after being added.

Also, `retainAll` with a `List` as the argument is O(n·m). For each of your n elements, it searches the m-item list one by one. Pass a `Set` instead.

Interview questions

Q1. How does HashSet work internally? It wraps a HashMap. Elements are stored as keys with one constant dummy value. Uniqueness comes from how the map treats keys — the same key cannot be stored twice.

Q2. HashSet vs TreeSet vs LinkedHashSet? HashSet: no order, O(1). TreeSet: sorted, O(log n). LinkedHashSet: insertion order, O(1).

2.8 ArrayDeque — your stack and your queue

Why?

> **In simple words:** ArrayDeque is one class that works as both a stack and a queue. It is an array whose ends wrap around in a circle, so adding or removing at either end is instant.

Stacks and queues need O(1) operations at their ends:

- **Stacks** (last in, first out): DFS, undo, matching brackets, monotonic stack.
- **Queues** (first in, first out): BFS, task buffers, sliding windows.

The older options are poor. `Stack` is a legacy class and locks (synchronises) on every call. `LinkedList` creates a new node object for every element. `ArrayDeque` is a **circular array** that does both jobs faster.

What is happening internally?

```
elements: [ _ | _ | C | D | E | _ | A | B ]    (capacity 8)
               ↑ tail          ↑ head
logical order: A, B, C, D, E  (wraps around the end of the array)
```

Here is how it works inside:

- Two indices, `head` and `tail`, mark the two ends.
- They move using modular arithmetic. When an index passes the end of the array, it wraps back to the start.
- When the array fills up, it doubles in size and "unwraps" the items into the new array in order.
- Every operation at either end is O(1) amortised (usually instant; occasionally a resize, but fast on average). No node objects are created per element.
- `null` elements are **not allowed**. Inside, null means "empty slot". Also, `poll` returns null to tell you the deque is empty.

Think of a round table in a canteen with numbered seats. People join at one end and leave from the other. When you reach the last seat number, you simply continue from seat 1.

How does it work?

Use as	Methods
Stack (LIFO, at the head)	`push`, `pop`, `peek`
Queue (FIFO)	`offer` (tail), `poll` (head), `peek`

Use as	Methods
Deque (both ends)	`offerFirst/Last`, `pollFirst/Last`, `peekFirst/Last`

The methods come in two families:

- `add` / `remove` / `element` **throw an exception** when they fail.
- `offer` / `poll` / `peek` **return a special value** instead (false or null).

Prefer the second family in DSA loops.

Example

```
Deque<Character> stack = new ArrayDeque<>();
for (char c : "{[()]}".toCharArray()) {
    if ("({[".indexOf(c) >= 0) stack.push(c);
    else if (stack.isEmpty() || "({[".indexOf(stack.pop()) != ")}]".indexOf(c)) {
System.out.println("invalid"); break; }
}

Deque<int[]> queue = new ArrayDeque<>();     // BFS over a grid
queue.offer(new int[]{0, 0});
while (!queue.isEmpty()) { int[] cell = queue.poll(); /* expand neighbours */ }
```

When would I use it?

- Every stack and queue in DSA: BFS, DFS, monotonic stack, sliding-window deque.
- Task buffers in single-threaded code.
- Undo history.

If several threads share the queue, use a `BlockingQueue` instead (4.14).

What can go wrong?

- **`push` and `offer` add to different ends.** `push` adds to the **head**, but `offer` adds to the **tail**. If you mix them on the same deque, the order becomes confusing.
- **Adding `null` throws NPE.**
- **Not thread-safe.**

Interview questions

Q1. Why prefer ArrayDeque over Stack and LinkedList? Stack is old, synchronised code that extends Vector. LinkedList creates a node object for every element. ArrayDeque is a cache-friendly circular array with O(1) amortised operations at both ends.

Q2. Why can't ArrayDeque hold null? Inside, null is used to mark empty slots. Also, `poll` / `peek` return null to say "the deque is empty".

2.9 PriorityQueue — a binary heap

Why?

> **In simple words:** A PriorityQueue always keeps the smallest item at the front. You can peek at it instantly and take it out quickly, without sorting everything.

Many problems keep asking for "the smallest (or largest) item so far". For example:

- The next job by deadline.
- The K most frequent elements.
- The nearest unvisited node in Dijkstra's shortest-path algorithm.
- Merging K sorted lists.

Sorting everything for each question costs O(n log n) every time. A *heap* is much cheaper. It gives you the minimum in O(1) and removes it in O(log n).

Think of a hospital emergency ward. The most serious patient is always seen next. But the waiting patients are not kept in a perfectly sorted line.

What is happening internally?

PriorityQueue is a **binary min-heap** stored in an array. A binary min-heap is a *complete* binary tree (every level is full except maybe the last, which fills from the left). Its one rule: every parent is ≤ its children.

```
array: [1, 3, 2, 7, 4, 5]          tree:          1
index:  0  1  2  3  4  5                        /     \
                                               3       2
children of i: 2i+1, 2i+2                     / \     /
parent of i:   (i-1)/2                       7   4   5
```

Only the root is guaranteed to be the minimum. Siblings are **not** ordered between themselves. So looping over a PriorityQueue does **not** give you sorted order.

How does it work?

- `offer(x)` : put x at the end of the array. Then **sift up**: while x is smaller than its parent, swap them. This is O(log n).
- `poll()` : take the root. Move the last element into the root's place. Then **sift down**: while it is larger than its smaller child, swap with that child. This is O(log n).
- `peek()` : just read the root → O(1).
- `remove(Object)` **and** `contains` : O(n), because they search item by item.
- **Building from a collection** (`new PriorityQueue<>(list)`): this "heapifies" all items at once in O(n).

Trace of `offer(0)` into `[1,3,2,7,4,5]` :

1. Append 0 at index 6. Its parent is at index 2, with value 2.
2. 0 < 2, so swap. Now 0 is at index 2. Its parent is at index 0, with value 1.
3. 0 < 1, so swap. Now 0 is at the root.
4. The result is `[0,3,1,7,4,5,2]` , with 0 at the root.

Example

```java
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());

// Top-K frequent SKUs: keep a MIN-heap of size k by count; the root is the weakest of the current top k
Map<String, Integer> freq = Map.of("A", 9, "B", 3, "C", 7, "D", 1);
PriorityQueue<Map.Entry<String, Integer>> heap = new PriorityQueue<>(Map.Entry.comparingByValue());
for (var e : freq.entrySet()) { heap.offer(e); if (heap.size() > 2) heap.poll(); }
// heap now holds A=9 and C=7 → O(n log k)

// Scheduler: jobs by run time, then priority
record Job(String id, Instant runAt, int priority) {}
```

```
PriorityQueue<Job> jobs = new PriorityQueue<>
(Comparator.comparing(Job::runAt).thenComparingInt(Job::priority));
```

When would I use it?

- Top-K problems.
- K-way merges (merging K sorted lists).
- Dijkstra and Prim (graph algorithms).
- Scheduling by time or priority (timer wheels, cron schedulers).
- Running medians (using two heaps).
- Event simulation.

What can go wrong?

- **Expecting sorted iteration or a sorted `toString()`.** You won't get it. To get sorted output, call `poll` again and again.
- **Changing an element's priority field after adding it.** The heap does not notice, so its order breaks. Remove the element and add it again instead. (In Dijkstra, the usual trick is to add a new entry and skip the old, stale ones when you poll them.)
- **Comparator overflow.** Writing `a[0] - b[0]` can overflow with large values and give the wrong sign. Use `Integer.compare`.
- **Not thread-safe.** Across threads, use `PriorityBlockingQueue`.

Interview questions

Q1. How is PriorityQueue implemented? As an array-based binary min-heap. `offer` and `poll` are O(log n) using sift-up and sift-down, and `peek` is O(1).

Q2. Why use a min-heap of size k to find the k largest elements? The root is the smallest of the current top k. Each new element either replaces the root in O(log k) or is thrown away. So the total time is O(n log k), and memory is only O(k).

Q3. Is PriorityQueue iteration sorted? No. Only the head is guaranteed to be the minimum.

2.10 Iterators: fail-fast vs fail-safe

Why?

> **In simple words:** If you change a list while you are looping over it, things go wrong. Normal Java collections notice this and throw an error at once; concurrent collections instead let you loop safely over a stable picture of the data.

Changing a collection while looping over it causes trouble. The loop can skip elements, see some twice, or read half-updated data.

Java's standard collections **fail fast**. They throw an exception immediately, instead of quietly giving you wrong results.

Concurrent collections make a different choice. They let you loop safely, either over a *snapshot* (a frozen copy) or over a *weakly consistent view* (the live data, which may or may not show recent changes).

Think of a teacher taking attendance from a register. If someone tears out a page in the middle, she stops and complains at once. She does not quietly mark the wrong students.

What is happening internally?

- Non-concurrent collections keep a counter, `int modCount`. It goes up on every **structural** change — an add or a remove. (`set` does not count, because it doesn't change the size.)
- When an iterator is created, it saves a copy: `expectedModCount = modCount`.
- On each `next()`, the iterator checks that both numbers still match. If they don't, it throws `ConcurrentModificationException` (CME).
- The iterator's own `remove()` updates both counters. So removing through the iterator is allowed.
- **Fail-safe / weakly consistent** iterators work differently:
- `CopyOnWriteArrayList` loops over the array snapshot taken when the iterator was created. Later writes are not seen.
- `ConcurrentHashMap` iterators walk the live buckets (bins). They may or may not see changes made at the same time, but they never throw.

How does it work?

The enhanced `for` loop is just a shortcut for an iterator. So `for (x : list) list.remove(x)` changes the list behind the iterator's back. The next call to `next()` then throws.

Example

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4));
for (Integer n : nums) if (n % 2 == 0) nums.remove(n);    // throws ConcurrentModificationException

Iterator<Integer> it = nums.iterator();
while (it.hasNext()) if (it.next() % 2 == 0) it.remove(); // correct
nums.removeIf(n -> n % 2 == 0);                           // simplest and O(n)

Map<String, Integer> m = new HashMap<>(Map.of("a", 1, "b", 0));
m.entrySet().removeIf(e -> e.getValue() == 0);            // remove while iterating a map
m.replaceAll((k, v) -> v * 10);                           // modify values in place (not structural)

// Quirk: removing the second-to-last element may NOT throw (hasNext() returns false first)
// -> CME is "best effort", never rely on it for correctness
```

When would I use it?

To filter a collection in place, use `removeIf`, `iterator.remove()`, `replaceAll`, or "collect first, then remove". When other threads change the collection while you loop, use a concurrent collection.

What can go wrong?

- **Treating CME as a thread-safety tool.** It is only best-effort detection — it may not fire every time. And it is not only about threads: single-threaded code can trigger it too.
- **Using `CopyOnWriteArrayList` for data that changes often.** Every write copies the whole array (O(n)). Use it only for data that is read much more than written, such as listener lists.

Interview questions

Q1. What is ConcurrentModificationException and when is it thrown? A fail-fast iterator throws it when the collection was structurally changed after the iterator was created, by anything other than the iterator itself. It detects this using `modCount`.

Q2. Fail-fast vs fail-safe iterators? Fail-fast iterators throw CME when the collection is changed during the loop (ArrayList, HashMap). Fail-safe or weakly consistent iterators work on a snapshot, or put up with changes, without throwing (CopyOnWriteArrayList, ConcurrentHashMap).

Q3. How do you remove elements while iterating? Use `Iterator.remove()` or `Collection.removeIf`, or collect the items to remove first and remove them after the loop.

2.11 Immutable collections, utilities and sorting

Why?

> **In simple words:** An immutable collection cannot be changed after it is made, so nobody can break your data by mistake. Java also gives you ready-made helpers in `Collections` and `Arrays`, and its sort methods — you use these daily, so know their traps.

If your class returns its internal list as-is, callers can change it and corrupt your object. Immutable collections fix this: nobody can change them. They are also thread-safe by nature, because there is nothing to change.

The `Collections` and `Arrays` utility classes, and the JDK's sort methods, are used constantly. They have a few sharp edges worth knowing.

Think of a printed railway timetable versus a whiteboard. Anyone can scribble on the whiteboard. The printed copy stays the same, no matter who reads it.

What is happening internally?

Two words matter in this table:

- A **view** is a window onto another collection. If the original changes, the view shows the change.
- A **copy** has its own data. Later changes to the original do not affect it.

Factory	Mutability	Nulls	Notes
`List.of(...)`, `Set.of`, `Map.of` (Java 9)	Immutable	Rejected (NPE)	Small, specially built classes; `Set.of` / `Map.of` reject duplicates; the iteration order of `Set.of` / `Map.of` is deliberately left unspecified
`List.copyOf(c)` (Java 10)	Immutable **copy**	Rejected	Independent of the source
`Collections.unmodifiableList(l)`	Read-only **view**	Allowed	Changes to `l` show through
`Arrays.asList(arr)`	Fixed size, backed by the array	Allowed	`set` works (it writes into the array), `add` / `remove` throw
`stream.toList()` (Java 16)	Unmodifiable	Allowed	

Sorting:

- `Arrays.sort(int[])` (primitives) uses **dual-pivot quicksort**. It is O(n log n) on average. It is not *stable* — stable means equal items keep their original order. For primitives this doesn't matter, because two equal numbers look exactly the same.

- `Arrays.sort(Object[])`, `List.sort` and `Collections.sort` use **TimSort**. It is a stable mix of merge sort and insertion sort. It spots runs of data that are already sorted and reuses them. It is O(n log n) in the worst case, and O(n) on input that is already sorted.

How does it work?

Immutability is enforced simply: every method that would change the collection throws `UnsupportedOperationException`.

A view passes every read through to the collection it wraps. A copy owns its own data.

Example

```
List<String> view = Collections.unmodifiableList(internal);
internal.add("x");            // view now shows "x" too
List<String> copy = List.copyOf(internal);   // snapshot, unaffected by later changes

int[] arr = {3, 1, 2};
List<int[]> wrong = Arrays.asList(arr);       // List with ONE element: the int[] itself
List<Integer> right = Arrays.stream(arr).boxed().toList();

Integer[] boxed = {3, 1, 2};
Arrays.sort(boxed, Collections.reverseOrder());   // descending needs objects
Arrays.fill(dp, -1); Arrays.copyOfRange(a, 2, 5); Arrays.equals(a, b); Arrays.toString(a);
Collections.frequency(list, "x"); Collections.nCopies(3, 0); Collections.swap(list, 0, 1);
int pos = Collections.binarySearch(sortedList, key);   // negative = -(insertionPoint) - 1
```

When would I use it?

- From getters, return `List.copyOf(...)` or an unmodifiable view.
- Use `List.of` / `Map.of` for constants and test data.
- Use the immutable factories for DTOs and records.
- Use the `Arrays` utilities all the time in DSA.

What can go wrong?

- **`List.of` with a null element throws NPE.** This surprises code that used to use `Arrays.asList`, which allows nulls.
- **`Arrays.asList(...).add()` throws `UnsupportedOperationException`**, because the list has a fixed size.
- **`Map.of` iteration order changes between runs.** Don't rely on it in tests.
- **Sorting a huge boxed `Integer[]` is much slower** than sorting an `int[]`.

Interview questions

Q1. `Collections.unmodifiableList` vs `List.copyOf`? The first is a read-only view: the original list can still change, and the changes show through the view. The second is an independent immutable copy.

Q2. What sorting algorithms does Java use? Dual-pivot quicksort for arrays of primitives, and TimSort (which is stable) for objects and lists.

Q3. What is the gotcha with `Arrays.asList`? The list has a fixed size and is backed by the array, so add/remove don't work. And if you pass a primitive array, you get a list with one element: the array itself.

3. Modern Java: 8 to 21

Interviewers assume you know Java 17 or 21. So you must be able to:

- write streams without thinking,
- explain what a lambda turns into after compiling,
- explain records, sealed types and pattern matching.

Virtual threads are covered in depth in 4.12.

3.1 Lambdas and functional interfaces

Why?

> **In simple words:** A lambda is a small piece of code that you pass around like a value. You hand it to a method, and the method runs it when it needs to.

Sometimes you need to pass *behaviour* to a method. For example: a comparator for sorting, a callback, or a filter rule.

Before Java 8, the only way was an *anonymous inner class* — a class with no name, written inline. That meant five lines of boilerplate for one line of real logic. Each one also created an extra `.class` file.

Lambdas fix this. You can pass behaviour as a short value, like `o -> o.amount() > 100`. This is what made streams, `CompletableFuture` and Spring's functional configuration practical to use.

If you know C#, a Java lambda is like a C# lambda assigned to a `Func` or `Action` delegate.

What is happening internally?

Functional interface. This is an interface with exactly one abstract method. Default methods and static methods don't count.

- A lambda is an implementation of that one method.
- The *target type* — the variable or parameter type the lambda is assigned to — decides which interface it becomes.

Lambdas are not anonymous inner classes. The compiler does something smarter:

1. It moves the lambda body into a private hidden method. The compiler creates this method itself, so it is called a *synthetic* method (for example `lambda$main$0`).
2. At the place where the lambda is created, it writes an **`invokedynamic`** instruction. This instruction means "work out what to do the first time this line runs".
3. The first time the line runs, the JVM calls a helper called the `LambdaMetafactory` *bootstrap method*.
4. The `LambdaMetafactory` generates a tiny class at runtime. This class implements the interface and simply calls the synthetic method.
5. The JVM then links that call site permanently. Later runs skip the setup.

Capturing vs non-capturing.

- A **non-capturing** lambda uses no outside variables. It is created once and reused as a singleton (one shared object).

- A **capturing** lambda uses outside variables. Each time, it creates a small object that holds the captured values. The JVM's *escape analysis* (a check that sees the object never leaves the method) can often remove even this allocation.

Effectively final. A local variable used inside a lambda must be *effectively final*. This means you never change it after you assign it. The reason: the lambda gets a *copy* of the value, not the variable itself.

```
source:    list.forEach(x -> System.out.println(x));
bytecode:  invokedynamic accept() → LambdaMetafactory → generated Consumer class → calls lambda$0(x)
```

How does it work?

Java gives you ready-made functional interfaces in `java.util.function`. Learn the shape of each one — what goes in and what comes out.

Interface	Method	Shape	C# equivalent
`Function<T,R>`	`apply`	T → R	`Func<T,R>`
`BiFunction<T,U,R>`	`apply`	(T,U) → R	`Func<T,U,R>`
`Supplier<T>`	`get`	() → T	`Func<T>`
`Consumer<T>`	`accept`	T → void	`Action<T>`
`Predicate<T>`	`test`	T → boolean	`Predicate<T>`
`UnaryOperator<T>` / `BinaryOperator<T>`	`apply`	T → T / (T,T) → T	
`Runnable` / `Callable<V>`	`run` / `call`	() → void / () → V (may throw)	`Action` / `Func<V>`

There are also primitive versions, like `IntPredicate`, `IntFunction`, `ToLongFunction` and `IntBinaryOperator`. They work on `int` or `long` directly. So Java does not need to *box* each number (wrap it in an `Integer` or `Long` object). This matters in hot paths — code that runs millions of times.

Method references. When a lambda only calls one existing method, you can write a shorter form called a *method reference*:

Kind	Reference	Equivalent lambda
Static	`Integer::parseInt`	`s -> Integer.parseInt(s)`
Bound instance	`System.out::println`	`x -> System.out.println(x)`
Unbound instance	`String::toUpperCase`	`s -> s.toUpperCase()`
Constructor	`ArrayList::new`	`() -> new ArrayList<>()`

"Bound" means the object is already fixed (`System.out`). "Unbound" means the object comes in as the lambda's argument.

Example

The example below shows three things:

- **Composition:** joining small predicates and functions with `and`, `negate`, `compose` and `andThen`.

- **Your own functional interface:** marked with `@FunctionalInterface`.
- **Effectively final:** what happens if you change a captured variable.

```
Predicate<Order> highValue = o -> o.amount() > 10_000;
Predicate<Order> delivered = o -> o.status() == Status.DELIVERED;
Predicate<Order> rule = highValue.and(delivered).negate();          // composition

Function<Long, Long> gst = p -> p * 118 / 100;
Function<Long, Long> discount = p -> p * 90 / 100;
gst.compose(discount).apply(1000L);     // discount first, then gst -> 1062
gst.andThen(discount).apply(1000L);     // gst first, then discount -> 1062 (same here, order matters
generally)

@FunctionalInterface
interface PointsCalculator { long points(Order o); }                // your own functional interface
PointsCalculator flat = o -> 100;

int threshold = 500;                    // effectively final: captured by copy
Predicate<Order> big = o -> o.amount() > threshold;
// threshold = 600;                     // compile error if uncommented: no longer effectively final
```

Remember the order: `f.compose(g)` runs `g` first, then `f`. `f.andThen(g)` runs `f` first, then `g`.

When would I use it?

- Comparators for sorting.
- Stream operations.
- Callbacks, such as `CompletableFuture.thenApply`.
- Strategy objects that are really just one function — pricing rules, validators.
- Spring beans of type `Function` or `Supplier`.
- Event handlers.

What can go wrong?

- **Changing a local counter inside a lambda.** It won't compile. People then hack around it with `int[] count = {0}` or an `AtomicInteger`. This is usually a sign you should use a stream reduction instead, like `count()` or `sum()`.
- **Checked exceptions.** `Function` cannot throw checked exceptions. So a lambda that does I/O must wrap the exception (for example in `UncheckedIOException`). Or you write your own functional interface that declares `throws`.
- **Long multi-line lambdas.** They are hard to read. Stack traces also show ugly names like `lambda$process$7`. Move the logic into a named method and use a method reference.
- **`this` means something different.** Inside a lambda, `this` is the enclosing object (the class the lambda is written in). Inside an anonymous class, `this` is the anonymous object itself.

Interview questions

Q1. What is a functional interface? It is an interface with exactly one abstract method. You can use it as the target of a lambda or a method reference. The `@FunctionalInterface` annotation makes the compiler check this rule.

Q2. How are lambdas implemented — are they anonymous classes? No. The compiler moves the body into a private synthetic method. The lambda object is created through `invokedynamic` and `LambdaMetafactory` the first time that line runs. Non-capturing lambdas are cached and reused as singletons.

Q3. Why must captured variables be effectively final? The lambda captures a copy of the value, not the variable. If you could change the variable later, the copy would be out of date. It would also invite data races when the lambda runs on another thread.

Q4. Function vs Supplier vs Consumer vs Predicate? `Function` takes T and returns R (T→R). `Supplier` takes nothing and returns T (()→T). `Consumer` takes T and returns nothing (T→void). `Predicate` takes T and returns a boolean (T→boolean).

3.2 Streams: the pipeline model

Why?

> **In simple words:** A stream is a factory conveyor belt for your data. You describe the stations — filter, transform, collect — and each item travels down the belt one at a time.

A normal loop mixes two things:

- *What* you want — filter, transform, add up.
- *How* to loop — indexes, temporary lists, running totals.

Streams let you write only the *what*. You describe the data transformation as a pipeline. This gives you code that:

- is easier to read,
- joins together cleanly,
- can stop early (*short-circuit*) when it has enough,
- can run in parallel if needed.

C# developers: streams are Java's version of LINQ.

What is happening internally?

A stream pipeline always has this shape: **source → zero or more intermediate operations → one terminal operation**.

Laziness. An intermediate operation (`filter`, `map`, ...) **does not run anything**. It only returns a new pipeline stage, linked to the previous one. Nothing happens until you call the terminal operation. This is called *lazy evaluation*.

One element at a time. When the terminal operation runs:

1. Java joins all the stages into one chain of small objects called **Sinks**.
2. The source's **Spliterator** (the object that walks over the source data) pushes elements into this chain **one at a time**.
3. Element 1 goes through filter → map → collect. Then element 2 does the same. And so on.
4. So no temporary collection is created between stages.

Stateless vs stateful operations.

- **Stateless** operations (`filter`, `map`, `flatMap`, `peek`) look at each element on its own.
- **Stateful** operations (`sorted`, `distinct`, and `limit` in parallel) must remember or hold back elements.
- `sorted` is a *barrier*. It must collect every element before it can pass even one forward.

Short-circuiting. Some operations (`limit`, `findFirst`, `anyMatch`, `takeWhile`) stop pulling from the source as soon as they have their answer. This is why a stream can even work on an infinite source like `Stream.iterate`.

One use only. You can consume a stream only **once**. Using it again throws `IllegalStateException`.

The diagram shows each element flowing down the whole pipe before the next one starts:

```
orders.stream().filter(paid).map(Order::amount).limit(3).toList()

element 1 ──► filter ✓ ──► map ──► limit(1/3) ──► list
element 2 ──► filter ✗ (stops here)
element 3 ──► filter ✓ ──► map ──► limit(2/3) ──► list
element 4 ──► filter ✓ ──► map ──► limit(3/3) ──► list  → limit signals "done": source stops, element 5 is never read
```

How does it work?

Intermediate (lazy)	Terminal (triggers execution)
`filter`, `map`, `mapToInt/Long/Obj`, `flatMap`, `distinct`, `sorted`, `peek`, `limit`, `skip`, `takeWhile`, `dropWhile`, `boxed`	`forEach`, `collect`, `toList`, `reduce`, `count`, `min`, `max`, `sum`, `average`, `anyMatch/allMatch/noneMatch`, `findFirst/findAny`, `toArray`

`map` vs `flatMap`:

- `map` turns each element into exactly one element. If your function returns a list, you get a `Stream<List<X>>` — a stream of lists.
- `flatMap` turns each element into a *stream*. Then it flattens all those streams into one `Stream<X>`.
- Think of tiffin boxes. `map` gives you a row of tiffin boxes. `flatMap` opens every box and puts all the food on one plate.

`reduce`: It combines all elements into one value. You give it a starting value (the *identity*) and a combining function. The function must be *associative* — grouping must not change the answer, like `(a+b)+c == a+(b+c)`. Example: `reduce(0, Integer::sum)`.

Example

```java
record Txn(String outletId, String type, long points, LocalDate date) {}

// Top 5 outlets by points accrued this month
List<Map.Entry<String, Long>> top5 = txns.stream()
        .filter(t -> t.type().equals("ACCRUAL"))
        .filter(t -> !t.date().isBefore(LocalDate.now().withDayOfMonth(1)))
        .collect(Collectors.groupingBy(Txn::outletId, Collectors.summingLong(Txn::points)))
        .entrySet().stream()
        .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
        .limit(5)
        .toList();

// flatMap: all distinct SKUs across orders
Set<String> skus = orders.stream().flatMap(o ->
o.lines().stream()).map(Line::sku).collect(Collectors.toSet());

// Laziness proof: peek prints only for elements actually pulled
Stream.of("a", "bb", "ccc", "dddd")
      .peek(s -> System.out.println("saw " + s))
      .filter(s -> s.length() > 1)
      .findFirst();                       // prints "saw a", "saw bb" — then stops

// Primitive streams avoid boxing
```

```
long total = txns.stream().mapToLong(Txn::points).sum();
IntSummaryStatistics stats = IntStream.rangeClosed(1, 100).summaryStatistics();
```

Look at the "laziness proof". `findFirst` gets its answer at `"bb"`. So `"ccc"` and `"dddd"` are never even read.

When would I use it?

Use streams for business data transformation:

- filtering,
- mapping entities to DTOs,
- grouping and adding up results,
- building lookup maps from lists,
- generating reports,
- validating collections.

In DSA problems, a simple `for` loop is usually clearer and faster.

What can go wrong?

- **Side effects in lambdas.** For example `forEach(x -> externalList.add(x))`, or changing shared state inside `map`. These break in parallel and hide what you mean. Use collectors instead.
- **Using `peek` for real logic.** `peek` is meant for debugging. It may not run at all if the terminal operation can skip elements.
- **Reusing a stream**, or storing a `Stream` in a field.
- **Performance.** For small collections inside hot loops, streams add extra object creation and extra method calls. Measure with a profiler before you "optimise" in either direction.
- **Debugging.** Stack traces through stream internals are long. Keep pipelines short and move logic into named methods.
- **Checked exceptions** inside lambdas must be wrapped.

Interview questions

Q1. What is lazy evaluation in streams? Intermediate operations only build the pipeline; nothing runs until a terminal operation is called. Then each element flows through all the stages, one element at a time. This allows short-circuiting and avoids temporary collections between stages.

Q2. Intermediate vs terminal operations? Intermediate operations return a stream and are lazy. Terminal operations produce a result or a side effect, and they start the execution.

Q3. map vs flatMap? `map` is one-to-one: one element in, one element out. `flatMap` maps each element to a stream and then flattens all those streams into one stream.

Q4. Can you reuse a stream? No. After a terminal operation the stream is consumed, and using it again throws `IllegalStateException`. Create a new stream from the source instead.

3.3 Collectors and groupingBy

Why?

> **In simple words:** A collector is the last station on the stream's belt. It decides what box the items

> land in — a list, a set, a map, a count or a joined string.

Most real stream code ends with collecting the results. For example:

- a list,
- a map by ID,
- a count per status,
- a total per outlet,
- a comma-separated string.

`Collectors` gives you these patterns ready-made. The official name is *mutable reduction*: you build up a result by adding into a changeable container. Because `Collectors` does this for you, you don't hand-write accumulation loops.

What is happening internally?

A `Collector<T, A, R>` is just four functions working together:

1. **supplier** — creates an empty mutable container of type `A` (for example a new `ArrayList`).
2. **accumulator** — adds one element into that container.
3. **combiner** — merges two containers into one. This is used in parallel streams, where each thread fills its own container.
4. **finisher** — converts the container `A` into the final result `R`.

How `groupingBy(classifier, downstream)` works:

1. It keeps a `Map<K, A>` — one container per key.
2. For each element, it calculates the key using the classifier.
3. It gets that key's container, or creates it if it is new.
4. It adds the element into that container using the *downstream* collector.

The downstream collector can itself be another `groupingBy`. You can nest them as deep as you like.

Think of a school teacher sorting answer sheets into piles by section. The classifier is "which section?". The downstream collector is what she does with each pile — count it, total the marks, or just keep the sheets.

How does it work?

```
// Collectors you'll use weekly
.toList()                                                  // Java 16, unmodifiable
.collect(Collectors.toSet())
.collect(Collectors.toMap(User::id, Function.identity()))   // throws IllegalStateException on duplicate
keys!
.collect(Collectors.toMap(User::city, u -> 1, Integer::sum)) // merge function handles duplicates
.collect(Collectors.toMap(k, v, merge, LinkedHashMap::new))  // choose the map type
.collect(Collectors.groupingBy(Emp::dept))                                          // Map<Dept, List<Emp>>
.collect(Collectors.groupingBy(Emp::dept, Collectors.counting()))                   // Map<Dept, Long>
.collect(Collectors.groupingBy(Emp::dept, Collectors.summingDouble(Emp::salary)))
.collect(Collectors.groupingBy(Emp::dept, Collectors.mapping(Emp::name, Collectors.toList())))
.collect(Collectors.groupingBy(Emp::dept, TreeMap::new, Collectors.toList()))      // sorted keys
.collect(Collectors.partitioningBy(e -> e.salary() > 100_000))                     // Map<Boolean, List<Emp>>
.collect(Collectors.joining(", ", "[", "]"))
.collect(Collectors.collectingAndThen(Collectors.toList(), Collections::unmodifiableList))
.collect(Collectors.teeing(Collectors.counting(), Collectors.summingLong(Emp::age), (c, s) -> s / c))  //
Java 12
```

A few notes on the list above:

- `toMap` with a *merge function* (the third argument) tells Java what to do when two elements have the same key.
- `teeing` (Java 12) sends every element to two collectors at once, then combines their two results.

Example

```
// Second-highest distinct salary
Optional<Double> second = emps.stream().map(Employee::salary).distinct()
        .sorted(Comparator.reverseOrder()).skip(1).findFirst();

// Highest-paid employee per department
Map<String, Optional<Employee>> top = emps.stream().collect(Collectors.groupingBy(Employee::dept,
        Collectors.maxBy(Comparator.comparingDouble(Employee::salary))));

// First non-repeating character (LinkedHashMap keeps first-seen order)
Optional<Character> firstUnique = "swiss".chars().mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()))
        .entrySet().stream().filter(e -> e.getValue() == 1).map(Map.Entry::getKey).findFirst();   // 'w'

// Duplicates in a list
Set<Integer> seen = new HashSet<>();
Set<Integer> dups = nums.stream().filter(n -> !seen.add(n)).collect(Collectors.toSet());   // sequential
streams only!

// Orders -> Map<customerId, totalAmount>
Map<String, Long> totals = orders.stream().collect(Collectors.groupingBy(Order::customerId,
Collectors.summingLong(Order::amount)));
```

Ten exercises to do in Week 3. Four of them are answered above. Do all ten by hand:

1. Words longer than 3 letters, uppercased, distinct, sorted.
2. Second-highest salary.
3. Highest-paid employee per department.
4. Character frequencies.
5. First non-repeating character.
6. Partition numbers into even and odd.
7. Flatten a `List<List<Integer>>` and sum it.
8. Orders → total per customer.
9. Duplicates in a list.
10. Names per department, joined with commas, sorted.

When would I use it?

- Building lookup maps from database results: `toMap(Entity::getId, identity())`.
- Dashboard numbers: counts and sums per status, per tenant.
- CSV export: `joining`.
- Splitting records into valid and invalid: `partitioningBy`.

What can go wrong?

- **`toMap` throws on bad input.** Duplicate keys throw `IllegalStateException`. Null values throw `NullPointerException` (NPE). So give a merge function and avoid nulls.
- **`groupingBy` order is random.** It returns a `HashMap`, so key order is not fixed. Pass `TreeMap::new` (sorted keys) or `LinkedHashMap::new` (insertion order) when order matters.

- **The `seen.add` trick is wrong in parallel streams.** It is a *stateful lambda* — it changes shared state, and a plain `HashSet` is not thread-safe.
- **Grouping huge datasets in memory.** When the data is large, do the aggregation in the database with `GROUP BY` instead.

Interview questions

Q1. Why does `Collectors.toMap` throw? There are two cases. Duplicate keys without a merge function throw `IllegalStateException`. Null values throw an NPE.

Q2. How do you count elements per category? Use `groupingBy(classifier, counting())`.

Q3. groupingBy vs partitioningBy? `partitioningBy` always splits into exactly two groups, `true` and `false`, using a predicate. `groupingBy` creates as many groups as there are distinct keys.

3.4 Parallel streams

Why?

> **In simple words:** A parallel stream splits your data into pieces and gives each piece to a different CPU core. It is like opening more billing counters when the queue is long.

On a machine with many cores, a big CPU-heavy job can be split across those cores. `parallelStream()` does this for you. You don't write any thread code yourself.

What is happening internally?

1. **Splitting.** The source's **Spliterator** keeps cutting the data into smaller chunks. This works well for arrays and `ArrayList`. It works badly for `LinkedList` and I/O sources, because they can't be cut in the middle cheaply.
2. **Running.** Each chunk becomes a task on the **common ForkJoinPool**. This is one thread pool shared by the whole JVM. It has (cores − 1) worker threads, plus the thread that called the stream.
3. **Work stealing.** When a thread finishes its own tasks, it takes ("steals") tasks from the queue of a busy thread. So no core sits idle.
4. **Merging.** The partial results are combined using the collector's combiner.

How does it work?

Splitting and merging have a cost. A parallel stream is faster only when all of these are true:

- There is a **lot of data** — roughly 10,000+ elements.
- The work per element is **CPU-heavy**.
- The source is **easy to split**.
- There is **no shared mutable state** (no shared variable that threads change).
- The operations are **associative** — grouping the work differently gives the same answer.

Example

```
long primes = LongStream.rangeClosed(2, 5_000_000).parallel().filter(Primes::isPrime).count();   // good fit

// Bad fit: blocking I/O on the shared common pool starves everyone else in the JVM
orderIds.parallelStream().map(id -> restClient.fetch(id)).toList();   // don't

// Wrong result: shared mutable state
```

```
List<Integer> out = new ArrayList<>();
IntStream.range(0, 10_000).parallel().forEach(out::add);   // lost elements or ArrayIndexOutOfBoundsException
```

When would I use it?

Rarely in server code. It fits:

- offline batch calculations,
- CPU-heavy transformations over big in-memory arrays,
- one-off data processing jobs.

For I/O fan-out — calling many services or URLs at once — use your own executor with `CompletableFuture`, or virtual threads (Chapter 4).

What can go wrong?

- **The common pool is shared by the whole JVM.** This includes `CompletableFuture`s that were not given an executor. So one blocking task in it can stall unrelated code elsewhere.
- **Order-dependent operations lose most of the benefit.** For example `findFirst`, `forEachOrdered` and `limit`. They force threads to wait for each other.
- **Thread-unsafe collectors or lambdas corrupt the results.**
- **Busy servers don't gain anything.** Every request already uses a thread. So parallel streams just add fighting for CPU (*contention*) without adding speed.

Interview questions

Q1. How do parallel streams work? A Spliterator splits the source into chunks. Each chunk runs as a ForkJoin task on the common pool, and idle threads use work stealing to pick up extra tasks. Finally the partial results are combined.

Q2. When are parallel streams a bad idea? With small data; with I/O or other blocking work; with shared mutable state; with order-dependent operations; with sources that split poorly; and inside servers, where the common pool is shared by everything.

3.5 Optional

Why?

> **In simple words:** `Optional` is a box that may be empty. The method's return type tells the caller "check the box first", so nobody forgets.

When a method returns `null`, you can't see that in its signature. Every caller must *remember* to check for null. Someone forgets, and you get a `NullPointerException`.

`Optional<T>` puts "this may be missing" right in the **return type**. The caller can't ignore it. They must decide what to do when the value is absent.

C# developers: this is similar in spirit to nullable reference types (`User?`), but as a real wrapper object.

What is happening internally?

`Optional` is a small `final` wrapper class. It holds either one value or nothing. `Optional.empty()` is one shared object (a singleton).

It is a **value-based class**. This means its identity has no meaning. So never use `synchronized` on it, and never compare two Optionals with `==`. Use `equals` instead.

How does it work?

```
Optional.of(x)            // x must be non-null (NPE otherwise)
Optional.ofNullable(x)    // empty if x is null
opt.map(f)                // transform if present
opt.flatMap(f)            // f returns Optional — avoids Optional<Optional<T>>
opt.filter(p)
opt.orElse(def)           // def evaluated ALWAYS (eagerly)
opt.orElseGet(() -> def)  // supplier evaluated only when empty (lazy)
opt.orElseThrow()         // NoSuchElementException; or orElseThrow(() -> new NotFound(...))
opt.ifPresentOrElse(consumer, runnable)
opt.stream()              // 0 or 1 element — handy in flatMap
```

The most important pair to remember:

- `orElse(def)` — the default is calculated **every time** (eagerly), even when a value is present.
- `orElseGet(() -> def)` — the default is calculated **only when the box is empty** (lazily).

Example

```
Optional<User> u = userRepo.findById(id);      // Spring Data returns Optional

String city = u.map(User::address).map(Address::city).orElse("UNKNOWN");
User mustExist = u.orElseThrow(() -> new NotFoundException("user " + id));

// orElse vs orElseGet
User a = cache.get(id).orElse(loadFromDb(id));           // loadFromDb runs EVERY time, even on a cache hit
User b = cache.get(id).orElseGet(() -> loadFromDb(id));  // runs only on a miss

// Flatten a list of optionals
List<String> emails = users.stream().map(User::email).flatMap(Optional::stream).toList();
```

When would I use it?

Use it as the **return type** of a lookup that may find nothing. For example `findById` or `findFirstActiveScheme`.

Don't use it for:

- fields,
- method parameters,
- collections. Return an empty collection instead of `Optional<List<T>>`.

What can go wrong?

- **Calling `opt.get()` without checking.** This just swaps the NPE for a `NoSuchElementException`. Use `orElseThrow()` so the intent is clear.
- **`orElse(expensiveCall())`.** The expensive call runs every time, even when the value is present. Use `orElseGet`.
- **`Optional` fields.** `Optional` isn't `Serializable`. It also adds one extra heap object per field.
- **Returning `null` from a method declared to return `Optional`.** This defeats the whole purpose.

Interview questions

Q1. Why use Optional? To make "value may be absent" visible in the API signature. It forces callers to handle the empty case, so you get fewer NullPointerExceptions.

Q2. orElse vs orElseGet? `orElse` evaluates its argument eagerly, even when a value is present. `orElseGet` calls the supplier only when the Optional is empty.

Q3. Where shouldn't you use Optional? In fields, method parameters and collections, and in performance-critical inner loops.

3.6 The java.time API

Why?

> **In simple words:** The old `Date` and `Calendar` classes were buggy and confusing. `java.time` gives you clean, safe types — one for each idea of time — so date bugs become much rarer.

The old classes, `java.util.Date` and `Calendar`, had many problems:

- **Mutable.** Anyone holding a shared date object could change it by accident.
- **Not thread-safe.** `SimpleDateFormat` gives wrong results when many threads use it at once.
- **Months start from zero.** January is 0, which causes off-by-one bugs.
- **Mixed concepts.** They mix an exact moment in time with a calendar date.

Our platform runs in 32+ countries. There, a date bug means wrong expiry dates, wrong reports and missed schedules. `java.time` (Java 8) fixes all of these problems.

What is happening internally?

Every class in `java.time` is **immutable and thread-safe**. A "change" like `plusDays(1)` never edits the object. It returns a new object instead.

Each type models one clear idea:

Type	Represents	Example use
`Instant`	A point on the UTC timeline (seconds + nanos since epoch)	Event timestamps, DB audit columns, logs
`LocalDate` / `LocalTime` / `LocalDateTime`	Wall-clock values with **no zone**	Birthdays, business dates, "store opens at 09:00"
`ZonedDateTime` / `OffsetDateTime`	Date-time with a zone / offset	Scheduling in Asia/Kolkata, API payloads
`ZoneId`	A region with DST rules (`Asia/Kolkata`, `Europe/London`)	Conversions
`Duration` / `Period`	Time-based amount (hours, seconds) / date-based (days, months)	Timeouts / validity periods
`DateTimeFormatter`	Thread-safe formatting/parsing	APIs, reports
`Clock`	Source of "now" — injectable	Testable time

A few terms from the table:

- **Epoch** — the fixed starting point, 1 January 1970 UTC.
- **Wall-clock value** — what a clock on the wall shows, with no idea which country it is in.

- **DST** (Daylight Saving Time) — some countries move clocks one hour forward in summer and back in winter.

How does it work?

Follow one simple rule:

1. **Store and calculate in UTC**, using `Instant`.
2. **Convert to a time zone only at the edges** — in the UI, in reports, or for a user's schedule.

For amounts of time, pick the right type:

- Use `Period` for calendar maths. Adding 1 month to 31 Jan gives 28 Feb (or 29 Feb in a leap year).
- Use `Duration` for exact elapsed time, like "2 seconds".

Example

```
Instant now = Instant.now();
ZonedDateTime ist = now.atZone(ZoneId.of("Asia/Kolkata"));
LocalDate today = LocalDate.now(ZoneId.of("Asia/Kolkata"));
LocalDate expiry = today.plusMonths(12);                        // points validity
boolean expired = LocalDate.now().isAfter(expiry);
long daysLeft = ChronoUnit.DAYS.between(today, expiry);
Duration timeout = Duration.ofSeconds(2);
String s = DateTimeFormatter.ISO_LOCAL_DATE.format(today);     // "2026-10-05"

// Testable time: inject a Clock instead of calling now() directly
class ExpiryService {
    private final Clock clock;
    ExpiryService(Clock clock) { this.clock = clock; }
    boolean isExpired(Instant expiresAt) { return Instant.now(clock).isAfter(expiresAt); }
}
new ExpiryService(Clock.fixed(Instant.parse("2027-01-01T00:00:00Z"), ZoneOffset.UTC));   // deterministic tests
```

The `Clock` trick matters for tests. If code calls `now()` directly, the answer changes every second. If you pass in a `Clock`, a test can give a fixed time. So the test gives the same result every run — it is *deterministic*.

When would I use it?

Everywhere time appears:

- expiry of points and coupons,
- scheduling jobs in a tenant's time zone,
- audit timestamps,
- report periods,
- SLAs and timeouts.

What can go wrong?

- **Using `LocalDateTime` for event timestamps.** It has no zone. So the same value means different moments on servers in different countries. Use `Instant`.
- **Calling `LocalDate.now()` with the server's default zone.** A server running in UTC will think "today" is still yesterday until 05:30 IST.
- **DST.** In zones with DST, some local times don't exist, and some happen twice. India has no DST, but European tenants do.

- **Mixing with old JDBC code that uses `Date` / `Timestamp`.** Convert at the boundary, for example `Timestamp.from(instant)`.

Interview questions

Q1. Why is java.time better than Date/Calendar? It is immutable and thread-safe. It keeps exact instants separate from local and zoned values. Its API is clear, and its formatter is thread-safe.

Q2. Instant vs LocalDateTime? `Instant` is one exact moment on the UTC timeline. `LocalDateTime` is a wall-clock value with no zone, so it can mean different moments in different places.

Q3. How do you make time-dependent code testable? Inject a `Clock` and call `now(clock)`. In tests, pass `Clock.fixed(...)` so time does not move.

3.7 Records

Why?

> **In simple words:** A record is a class whose only job is to carry data. You write one line, and Java writes the constructor, getters, `equals`, `hashCode` and `toString` for you.

Classes that only carry data are everywhere: DTOs, events, value objects, composite keys. Each one needed:

- a constructor,
- getters,
- `equals`,
- `hashCode`,
- `toString`.

That is about 50 lines of boilerplate, and it is easy to get wrong (1.12). Records (Java 16) generate all of it from a one-line declaration. They are also immutable by design — the values can't change after creation.

C# developers: this is the same idea as a C# `record`.

What is happening internally?

When you write `record Money(long paise, String currency) {}`, the compiler turns it into a `final class extends java.lang.Record` with:

- a `private final` field for each component,
- a **canonical constructor** — the one constructor that takes all components,
- accessor methods named exactly after the components (`paise()`, not `getPaise()`),
- `equals`, `hashCode` and `toString` that use all components. These are built through `invokedynamic` and the `ObjectMethods` helper.

What records **can** do:

- implement interfaces,
- have static fields and static methods,
- have extra instance methods.

What records **cannot** do:

- extend another class (they already extend `Record`),
- declare extra instance fields.

How does it work?

You can add a **compact constructor**. It has no parameter list. It runs *before* the fields are assigned. So you use it to check inputs (validation) or to clean them up (normalisation), for example converting `"inr"` to `"INR"`.

Example

```
public record Money(long paise, String currency) implements Comparable<Money> {
    public Money {                                          // compact constructor
        if (paise < 0) throw new IllegalArgumentException("negative amount");
        currency = Objects.requireNonNull(currency).toUpperCase();   // normalise before assignment
    }
    public static Money inr(long paise) { return new Money(paise, "INR"); }
    public Money plus(Money o) {
        if (!currency.equals(o.currency)) throw new IllegalArgumentException("currency mismatch");
        return new Money(paise + o.paise, currency);
    }
    @Override public int compareTo(Money o) { return Long.compare(paise, o.paise); }
}

record OrderPlaced(String eventId, String tenantId, List<String> skus, Instant at) {
    OrderPlaced { skus = List.copyOf(skus); }                // records are only SHALLOWLY immutable
}

Money a = Money.inr(500), b = new Money(500, "inr");
System.out.println(a.equals(b) + " " + a);                   // true Money[paise=500, currency=INR]
```

Note `OrderPlaced`. A record's fields are final, but a `List` inside it can still be changed by whoever holds it. That is what "shallowly immutable" means. `List.copyOf` makes a read-only copy, so the record is truly safe.

When would I use it?

- API request and response DTOs.
- Kafka event payloads.
- Value objects (like `Money`).
- Composite map keys — a key made of two or more values.
- Query projections — Spring Data supports records as projections.
- Configuration properties — `@ConfigurationProperties` records.
- Returning more than one value from a method.

What can go wrong?

- **JPA entities can't be records.** Hibernate needs a no-arg constructor, mutable state, and the ability to create proxies (subclasses). Use records only for DTOs and projections.
- **Mutable components leak.** A `List` or array inside a record can still be changed, unless you copy it in the compact constructor. Arrays have another problem: the generated `equals` compares them by reference, not by contents.
- **Every component is part of `equals`.** This includes ones you may not want, like timestamps. So choose components with care.
- **Changing a record's components breaks its JSON.** Its serialised form changes, which is a breaking change for clients.

Interview questions

Q1. What does a record generate? Private final fields, a canonical constructor, accessor methods, and `equals` / `hashCode` / `toString` that use all components.

Q2. Records vs Lombok `@Data`? Records are part of the language, they are immutable, and they need no annotation processing. `@Data` generates mutable getters and setters, and it needs the Lombok library. Use records for DTOs and value objects.

Q3. Can a JPA entity be a record? No. JPA needs a no-arg constructor, mutable fields, and classes it can subclass to create proxies.

3.8 Sealed classes and pattern matching

Why?

> **In simple words:** A sealed type says "these are the only possible kinds, no others". Then when you `switch` over it, the compiler can check you handled every kind — and shouts if you forget one.

Some domains have a **closed** set of variants. For example, a payment result is Success, Declined or Pending — nothing else.

With a normal open interface, the compiler can't know all the variants. So it can't check that your `switch` handles every case. When someone adds a new variant, it silently falls into `default`, and the bug ships.

Two features fix this:

- **Sealed types** make the hierarchy closed. The compiler knows the full list of subtypes.
- **Pattern matching** lets you check a type and pull out its data in one step. So the compiler can verify the switch is *exhaustive* — it covers every case.

Together they give Java *algebraic data types* — a type that is "exactly one of these fixed shapes".

Think of a railway ticket status: Confirmed, RAC or Waitlisted. There is no fourth option. A sealed type lets you say that in code.

What is happening internally?

Sealed declaration.

- `sealed interface PaymentResult permits Success, Declined, Pending` stores the allowed subclasses in the class file, in the `PermittedSubclasses` attribute.
- The JVM refuses to load any other subclass.
- Each permitted subclass must itself be `final`, `sealed` or `non-sealed`.

Pattern matching for `instanceof` (Java 16). `if (o instanceof Order ord)` checks the type and creates the variable `ord` in one go. No cast is needed.

Pattern matching for `switch` and record patterns (Java 21).

- `case Declined(String reason) when …` does three things at once: 1. checks the type, 2. *deconstructs* the record — pulls its fields out into variables, 3. applies a *guard* — the extra `when` condition.
- Pattern switches compile to `invokedynamic` with `SwitchBootstraps.typeSwitch`.

- Because the hierarchy is sealed, the compiler knows all the cases. So no `default` is needed.
- If you add a new variant, every switch that doesn't handle it becomes a **compile error**.

How does it work?

1. Declare the sealed hierarchy. Records make ideal leaves (the final concrete types).
2. Switch over the base type, with one `case` per variant.
3. Later, add a new variant. Every switch that doesn't cover it now fails to compile, until you handle it.

Example

```
public sealed interface PaymentResult permits Success, Declined, Pending {}
public record Success(String txnId, long paise) implements PaymentResult {}
public record Declined(String reason) implements PaymentResult {}
public record Pending(Instant retryAt) implements PaymentResult {}

static String describe(PaymentResult r) {
    return switch (r) {
        case Success(String txnId, long paise) -> "paid %d paise, txn %s".formatted(paise, txnId);
        case Declined(String reason) when reason.contains("limit") -> "over limit — ask for another method";
        case Declined d -> "declined: " + d.reason();
        case Pending(Instant at) -> "retry at " + at;
    };                                  // exhaustive: no default needed
}

// instanceof pattern
Object payload = readMessage();
if (payload instanceof OrderPlaced op && !op.skus().isEmpty()) handle(op);
```

When would I use it?

- Results and errors: `Result = Ok | Err`.
- Domain events: `OrderEvent = Placed | Paid | Shipped | Cancelled`.
- State machines.
- Command types in a CQRS handler (CQRS = keeping write commands separate from read queries).
- Parsed syntax-tree (AST) nodes. The rules engine in 11.12 uses a sealed `Condition`.

What can go wrong?

- **Adding `default` to a sealed switch.** It silently switches off the exhaustiveness check. A new variant will then fall into `default` without any compile error.
- **Placement rules across modules or packages.** Permitted subclasses must be in the same module. If you are not using modules (the *unnamed module*), they must be in the same package.
- **Case order matters.** Guards (`when`) are checked from top to bottom. Put specific cases before general ones. Otherwise the compiler reports a *dominated pattern* — a case that can never be reached.

Interview questions

Q1. What are sealed classes for? To restrict which classes can extend a type. This makes the hierarchy closed and known to the compiler, so pattern-matching switches can be checked for exhaustiveness.

Q2. What does pattern matching for switch add? It allows type patterns, record deconstruction and guards (`when`) in `case` labels. The compiler also checks exhaustiveness over sealed types.

Q3. How does this compare to the Visitor pattern? It reaches the same goal — one operation per variant — without the boilerplate of double dispatch (the `accept` / `visit` back-and-forth calls). A new

operation is just a new method with a switch.

3.9 Switch expressions, `var` and text blocks

Why?

> **In simple words:** Three small features that make everyday code shorter and safer: a `switch` that returns a value and never falls through, `var` to skip repeating long types, and `"""` for clean multi-line strings.

Each feature removes a common bug or a lot of noise:

- Old `switch` statements *fell through* to the next case when you forgot a `break`. They also couldn't return a value.
- Long generic types made variable declarations hard to read.
- Multi-line SQL or JSON strings needed `+` and `\n` everywhere.

What is happening internally?

Switch expressions (Java 14).

- `case X ->` arms never fall through.
- The whole switch **yields a value** — you can assign it to a variable.
- Inside a block arm `{ ... }`, use `yield` to return the value.
- The compiler checks exhaustiveness for enums and sealed types.

`var` (Java 10).

- The compiler works out the type of a local variable from its right-hand side. This is *type inference*, and it happens at compile time.
- The variable still has one fixed type. This is **not** dynamic typing like JavaScript.
- It only works for local variables that have an initial value. It doesn't work for fields, parameters or return types.
- C# developers: it is the same as C# `var`.

Text blocks (Java 15).

- A `"""` literal can span many lines.
- The compiler removes the common indentation and makes line endings consistent.
- The result is an ordinary `String`.

How does it work?

```
int bonus = switch (tier) {
    case SILVER -> 100;
    case GOLD, PLATINUM -> 250;          // multiple labels, no fall-through
};
String label = switch (code) {
    case 1 -> "one";
    default -> { log.debug("unknown {}", code); yield "other"; }
};

var byOutlet = new HashMap<String, List<Order>>();   // inferred HashMap<String, List<Order>>
for (var e : byOutlet.entrySet()) { /* e is Map.Entry<String, List<Order>> */ }

String sql = """
```

```
    SELECT outlet_id, SUM(points) AS total
    FROM ledger_entry
    WHERE tenant_id = ?
    GROUP BY outlet_id
    """;
```

Example

See the code above. In interviews:

- write switch expressions for state machines and enum mappings,
- write text blocks for SQL inside `@Query` annotations.

When would I use it?

- **Switch expressions** — for any mapping over an enum or a sealed type.
- `var` — when the type is obvious from the right-hand side, like `var list = new ArrayList<Order>()`.
- **Text blocks** — for SQL, JSON test fixtures (sample data for tests) and templates.

What can go wrong?

- `var x = getThing();` hides the type when it isn't obvious. This hurts readability.
- `var list = new ArrayList<>();` infers `ArrayList<Object>`, because there is nothing to infer the element type from.
- **Text block whitespace.** Trailing spaces at line ends are removed unless you escape them with `\s`. A closing `"""` on its own line adds a final newline.

Interview questions

Q1. Is `var` dynamic typing? No. The type is inferred once, at compile time, and then it is fixed. `var` only removes the repeated type name from the declaration.

Q2. What does `yield` do? It returns a value from a block-style arm (`{ ... }`) of a switch expression.

3.10 What else is new in Java 21

- **Virtual threads** — very light threads managed by the JVM, not the OS. They let you run a huge number of blocking I/O tasks at once (full section: 4.12).
- **Sequenced collections** — new interfaces `SequencedCollection` and `SequencedMap`. They give the same methods everywhere: `getFirst()`, `getLast()`, `reversed()`, `firstEntry()`, `pollLastEntry()`. They work on lists, deques, `LinkedHashMap` and `TreeMap`.
- **Record patterns and pattern-matching switch** — now final (no longer preview) in 21 (3.8).
- **Generational ZGC** — the low-pause garbage collector, now with separate young and old generations (5.5).
- **Mention-only previews** (features you can try but which may still change):
- *structured concurrency* — start a group of tasks together and treat them as one unit;
- *scoped values* — an immutable replacement for `ThreadLocal`;
- *unnamed variables* `_` — final in Java 22.
- **String templates** were previewed and then withdrawn. So don't mention them as a feature.

Interview questions

Q1. What are the headline features of Java 21? Virtual threads; record patterns and pattern matching for switch (now final); sequenced collections; and generational ZGC. There are also previews of structured concurrency and scoped values.

4. Concurrency and Multithreading

This topic clearly separates SDE1 answers from SDE2 answers. You need three things:

- **The words**: atomicity, visibility, happens-before.
- **The tools**: executors, locks, atomics, concurrent collections, CompletableFuture and virtual threads.
- **The hands-on skill**: coding the classic problems on a whiteboard.

4.1 Threads and the thread lifecycle

Why?

> **In simple words:** A thread is one worker inside your program. With many workers, one can wait for the database while another uses the CPU, so nothing sits idle.

A server handles thousands of requests. Most of the time, each request is just *waiting* — for the database, the network or the disk.

If one thread did all the work one after another, the CPU would sit idle during every wait. Threads let independent pieces of work run at the same time. This is called *concurrency*.

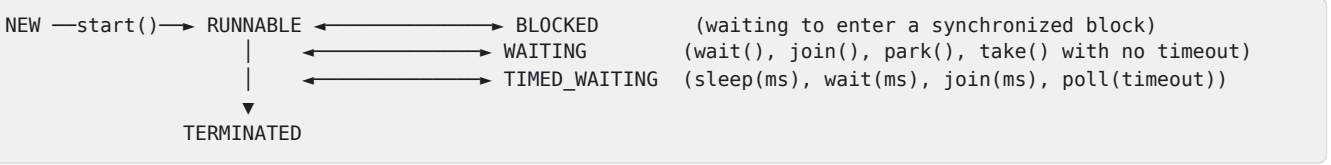
With threads you get three benefits:

- Waits overlap, so total time drops.
- All CPU cores get used.
- The system stays responsive.

Think of a bank with one counter versus five counters. With five counters, while one clerk waits for a printout, the others keep serving customers.

What is happening internally?

- A **process** is a running program. It owns memory: the heap (where objects live) and the code.
- **Threads** live inside a process. All threads share the same heap. But each thread has its own **stack** (its method calls and local variables) and its own **program counter** (which line it is on).
- A Java **platform thread** is a thin wrapper around an **OS thread**. The OS scheduler decides which thread runs on which core. It also pauses threads and switches between them using small time slices. This is called *preemption*.
- Creating a platform thread is not free. It reserves stack memory (default ~512 KB–1 MB). It also costs tens of microseconds. So you can have thousands of them, not millions.
- Switching from one thread to another is called a **context switch**. The CPU saves and restores registers and stack pointers. It also spoils the CPU caches (the CPU's small, fast memory). Each switch typically costs a few microseconds.



`RUNNABLE` means "running or ready to run". Java does not separate these two.

One surprise: a thread that is blocked reading from a socket also shows as RUNNABLE in thread dumps. So RUNNABLE does not always mean "using CPU".

How does it work?

1. `new Thread(task)` creates the object. The state is NEW. No OS thread exists yet.
2. `start()` makes the JVM create the OS thread. That thread calls `run()`. The state becomes RUNNABLE.
3. The thread may block on a lock (BLOCKED), wait for a condition (WAITING), or sleep (TIMED_WAITING). After that it returns to RUNNABLE.
4. When `run()` returns or throws, the state becomes TERMINATED. A thread can't be restarted.

Interruption is the polite way to ask a thread to stop. The thread must cooperate. Here is how it works:

1. Another thread calls `t.interrupt()`. This only sets a flag on `t`.
2. If `t` is inside a blocking method (`sleep`, `wait`, `join`, `BlockingQueue.take`), that method notices the flag. It throws `InterruptedException` and **clears** the flag.
3. If `t` is running normal non-blocking code, nothing happens by itself. The code must check `Thread.currentThread().isInterrupted()` regularly.

Example

```
Runnable task = () -> {
    while (!Thread.currentThread().isInterrupted()) {
        try {
            processNextBatch();
            Thread.sleep(1000);                          // TIMED_WAITING
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();          // restore the flag so the loop condition sees it
        }
    }
    System.out.println("stopped cleanly");
};
Thread worker = new Thread(task, "batch-worker");      // always name threads: readable thread dumps
worker.start();                                        // NOT run(): run() would execute on the current
thread
Thread.sleep(3500);
worker.interrupt();                                    // ask it to stop
worker.join();                                         // wait until TERMINATED

Thread daemon = new Thread(this::heartbeat);
daemon.setDaemon(true);                                // daemon threads don't keep the JVM alive
```

When would I use it?

In application code, you should almost never create raw threads. Instead, submit tasks to an **executor** (4.3) or use virtual threads (4.12).

But you must understand the lifecycle for two jobs:

- **Reading thread dumps.** A thread dump is a snapshot of every thread and its state. Many threads BLOCKED on one lock means contention — too many threads fighting for one lock. Many threads WAITING in `getConnection` means the connection pool is exhausted.
- **Writing correct shutdown logic.**

What can go wrong?

- **Calling `run()` instead of `start()`.** No new thread is created. The code runs on the current thread, one step after another.
- **Swallowing `InterruptedException`** with an empty catch block. Now the thread can no longer be stopped, and shutdowns hang.
- **Unbounded thread creation** — for example, one new thread per request without a pool. Memory runs out and you get `OutOfMemoryError: unable to create native thread`.
- **Uncaught exceptions kill the thread silently.** Nobody sees the error unless you set an `UncaughtExceptionHandler`, or use executors and check the `Future`s.
- **`Thread.stop()`** is deprecated and dangerous. It releases locks in the middle of an update. This leaves data in a corrupted, half-changed state.

Interview questions

Q1. Process vs thread? A process has its own memory space. Threads live inside a process: they share its heap, but each has its own stack and program counter. So threads are cheaper to create and to switch between. The cost is that shared data needs synchronisation.

Q2. `start()` vs `run()`? `start()` creates a new thread, and that new thread executes `run()`. If you call `run()` directly, it simply executes on the current thread, like a normal method call.

Q3. Explain the thread states. NEW (created, not started). RUNNABLE (running or ready to run). BLOCKED (waiting to get a monitor lock). WAITING / TIMED_WAITING (waiting for a signal, or for a signal or a timeout). TERMINATED (finished).

Q4. How do you stop a thread correctly? Cooperatively. You interrupt it (or set a volatile flag), and the task checks the flag and exits by itself. When the task catches `InterruptedException`, it must either restore the flag or exit.

4.2 Race conditions: atomicity, visibility and ordering

Why?

> **In simple words:** When two threads change the same data, three different things can break: an update can get lost, a change can stay hidden from the other thread, or steps can happen in a surprising order.

As soon as two threads touch the same mutable (changeable) data, three separate problems can appear. They are independent of each other.

SDE2 candidates name these three problems precisely. That is how they explain a concurrency bug and choose the right fix.

What is happening internally?

1. **Atomicity** — an operation should happen fully or not at all. But `count++` is actually three machine steps: read, add, write. Two threads can mix (interleave) these steps:

```
count = 5
T1: read 5          T2: read 5
T1: add → 6         T2: add → 6
T1: write 6         T2: write 6       → final 6, one increment LOST
```

Both threads read 5, both write 6. One increment is lost.

2. **Visibility** — a write by one thread should be seen by others. But each CPU core has its own caches (L1/L2). Also, the JIT compiler may keep a variable in a CPU register. So a write by thread A may stay inside A's core. Thread B may never see it. Without synchronisation, B may read an old (stale) value **forever**.
3. **Ordering** — steps should happen in the order you wrote them. But compilers and CPUs **reorder** instructions to run faster. They only do this when the result looks the same *to the current thread*. Another thread, however, can see the changed order. For example, it may see the "ready" flag set before the data that the flag protects was written.

Two more terms:

- A **critical section** is any code that reads or writes shared mutable state.
- A **data race** is when two threads access the same variable without synchronisation, and at least one of them writes.

How does it work?

You fix a race with one of these strategies. They are listed best first:

1. **Don't share.** Keep the data inside one thread only — local variables, per-request objects. This is called *thread confinement*.
2. **Don't mutate.** Share only immutable objects, which can never change (1.14).
3. **Synchronise.** Pick the tool that matches the problem: - Locks (`synchronized` , `ReentrantLock`) give atomicity + visibility. - `volatile` gives visibility only. - Atomics give atomic updates to one single variable. - Concurrent collections give safe shared data structures.

Example

```
class UnsafeCounter { int count; void inc() { count++; } }

UnsafeCounter c = new UnsafeCounter();
Thread[] ts = new Thread[4];
for (int i = 0; i < 4; i++) { ts[i] = new Thread(() -> { for (int j = 0; j < 100_000; j++) c.inc(); });
ts[i].start(); }
for (Thread t : ts) t.join();
System.out.println(c.count);       // expected 400000, typically prints something like 231874 — lost updates

// Check-then-act race (just as common in business code)
if (!cache.containsKey(key)) {     // T1 and T2 both see "absent"
    cache.put(key, load(key));     // both load and both put → duplicate work or overwritten state
}
```

When would I use it?

Use these three ideas as a checklist for diagnosis. When a bug "only happens under load", ask which of the three is broken:

- Lost updates → atomicity is broken.
- A flag that never seems to change → visibility is broken.
- Objects that look half-initialised → ordering is broken.

What can go wrong?

- **Races don't happen every time.** Tests pass, but production fails under load. So test with many threads and many repetitions. For low-level code, use tools like jcstress.
- **Only writes synchronised.** If writes are synchronised but *reads* are not, visibility is still broken. The reader may see stale values.
- **Compound operations on thread-safe classes are still races.** Each single call on a `ConcurrentHashMap` is safe. But `if (!map.containsKey(k)) map.put(k, v)` is two calls, and another thread can act between them. Use `putIfAbsent` / `computeIfAbsent` instead — they do both steps as one.

Interview questions

Q1. What is a race condition? It is a correctness bug where the result depends on the timing, or interleaving, of threads that use shared mutable state. A classic example is lost updates when many threads run `count++` at the same time.

Q2. Atomicity vs visibility? Atomicity means other threads see an operation either fully done or not done at all — never half-done. Visibility means a write by one thread can actually be seen by other threads. Locks give both. `volatile` gives visibility only.

Q3. Why is `count++` not thread-safe? It is a read-modify-write made of three steps. Two threads can interleave those steps, so updates get lost.

4.3 Executors and ThreadPoolExecutor

Why?

> **In simple words:** A thread pool is a fixed team of workers with a waiting line. Tasks join the line, and free workers pick them up — so you never hire thousands of new workers when traffic jumps.

Creating one thread per task has two problems:

- **It is expensive.** Each one needs an OS thread and stack memory.
- **It has no limit.** A traffic spike creates thousands of threads and crashes the JVM.

A **thread pool** solves both. It reuses a fixed set of threads. Extra work waits in a queue.

It also gives you one single place to control three things: how many tasks run at once (concurrency), how to slow down callers when overloaded (*back-pressure*), and shutdown.

Think of a railway reservation office. It has a fixed number of counters and one queue. When the queue gets too long, the guard stops letting new people in.

What is happening internally?

`ThreadPoolExecutor` has three parts:

- a set of worker threads,
- a `BlockingQueue` of waiting tasks,
- policies that decide what to do in each situation.

When you submit a task, it follows this decision path:

```
submit(task)
   │
   ├─ running threads < corePoolSize?  ──yes──► start a NEW worker with this task
   │ no
   ├─ queue.offer(task) succeeds?        ──yes──► task waits in the queue
   │ no (queue full)
   ├─ running threads < maximumPoolSize? ──yes──► start a NEW (non-core) worker
   │ no
   └─ RejectedExecutionHandler           ──► Abort / CallerRuns / Discard / DiscardOldest
Workers loop: take() a task from the queue → run it → repeat. Non-core workers idle longer than keepAliveTime
exit.
```

Note the order: the queue is used **before** the pool grows to max. This is the most misunderstood detail.

The result is surprising. If the queue is unbounded (has no size limit), it never becomes full. So the pool never grows, and `maximumPoolSize` is never used.

How does it work?

Here is a properly configured pool:

```
ThreadPoolExecutor exec = new ThreadPoolExecutor(
    8,                                      // corePoolSize
    32,                                     // maximumPoolSize
    60, TimeUnit.SECONDS,                   // keepAliveTime for non-core threads
    new ArrayBlockingQueue<>(1_000),        // BOUNDED work queue
    Thread.ofPlatform().name("reward-", 0).factory(),   // named threads (Java 21 builder)
    new ThreadPoolExecutor.CallerRunsPolicy()); // when saturated, the submitting thread runs the task →
natural back-pressure
```

When the pool and the queue are both full, the **rejection policy** decides what happens:

Rejection policy	Behaviour	Use
`AbortPolicy` (default)	Throws `RejectedExecutionException`	Fail fast; the caller handles it
`CallerRunsPolicy`	The submitting thread runs the task itself	Slows down producers: back-pressure
`DiscardPolicy`	Drops the task silently	Only for truly optional work (metrics samples)
`DiscardOldestPolicy`	Drops the oldest queued task, then retries	"Latest wins" workloads

The `Executors` factory methods and their traps:

Factory	Configuration	Trap
`newFixedThreadPool(n)`	core = max = n, **unbounded** `LinkedBlockingQueue`	Under overload the queue grows without limit → OutOfMemoryError
`newCachedThreadPool()`	core 0, max `Integer.MAX_VALUE`, `SynchronousQueue`	No limit on threads → thread explosion
`newSingleThreadExecutor()`	1 thread, unbounded queue	Same queue risk
`newScheduledThreadPool(n)`	Delayed or repeating tasks	

Factory	Configuration	Trap
`newVirtualThreadPerTaskExecutor()`	One virtual thread per task (Java 21)	Not for CPU-bound work

Sizing — how many threads?

- **CPU-bound work** (the task mostly calculates): threads ≈ number of cores. Get it with `Runtime.getRuntime().availableProcessors()`.
- **I/O-bound work** (the task mostly waits): threads ≈ cores × (1 + wait time / compute time). Example: 8 cores, and a task that waits 90 ms on a DB and computes for 10 ms. That gives 8 × (1 + 9) = 80 threads.
- **Cross-check with Little's Law**: concurrency = throughput × latency. In words: work in progress = arrival rate × time each item takes. Serving 400 req/s at 200 ms each needs ~80 workers busy at the same time.
- **The downstream system limits you too.** If 80 threads share a DB pool of 20 connections, 60 of them just wait in line for the pool.

Shutdown:

```
exec.shutdown();                                        // stop accepting; finish queued tasks
if (!exec.awaitTermination(30, TimeUnit.SECONDS)) {
    exec.shutdownNow();                                 // interrupt running tasks, return queued ones
}
```

Example

```
// Fan out reward calculation for a batch of outlets with bounded concurrency and back-pressure
List<Future<Long>> futures = new ArrayList<>();
for (String outlet : outlets) futures.add(exec.submit(() -> calculatePoints(outlet)));
long total = 0;
for (Future<Long> f : futures) total += f.get(5, TimeUnit.SECONDS);   // timeout per result

// Trace with core=2, max=3, queue=2 and 6 quick submissions while all tasks are still running:
// t1 → new core worker #1; t2 → new core worker #2; t3 → queue [t3]; t4 → queue [t3,t4];
// t5 → queue full → new non-core worker #3; t6 → queue full & max reached → REJECTED
```

When would I use it?

- Background processing, such as notifications and report generation.
- Parallel fan-out — sending many calls at once to services or DB shards.
- Scheduled jobs, with `ScheduledExecutorService`.
- Moving slow work off request threads.

In Spring, define pools as beans (`ThreadPoolTaskExecutor`) and inject them where needed.

What can go wrong?

- **Unbounded queues hide overload.** Nothing looks wrong until the heap fills up. This is the reason Alibaba's Java guidelines ban `Executors.newFixedThreadPool`.
- **Thread starvation deadlock.** Tasks in a pool submit subtasks to the *same* pool and then wait for them. Once every thread is waiting, no thread is free to run the subtasks. Everything stops.
- **Silent failures.** If a task given to `submit()` throws, the exception is stored inside the `Future`. If nobody calls `get()`, the error simply disappears. So log inside the task, or use `execute()` with an uncaught exception handler.

- **ThreadLocal leakage** — data from one pooled task stays on the thread and leaks into the next task (4.15).
- **Forgetting shutdown.** Pool threads are non-daemon threads, so they keep the JVM alive and the app never exits.

Interview questions

Q1. Explain how ThreadPoolExecutor handles a new task. If fewer than core-size threads are running, it starts a new core thread. Otherwise it puts the task in the queue. If the queue is full, it starts a non-core thread, up to the max size. If it is already at max, it rejects the task through the rejection handler.

Q2. Why is `Executors.newFixedThreadPool` risky? It uses an unbounded LinkedBlockingQueue. Under overload, tasks keep piling up until you hit OutOfMemoryError. They are never rejected and the callers are never slowed down.

Q3. How do you size a thread pool? For CPU-bound work, about the number of cores. For I/O-bound work, about cores × (1 + wait/compute). Then check it with Little's Law and load tests. Finally, keep it within what downstream systems can handle, such as the DB pool size.

Q4. What is CallerRunsPolicy and why is it useful? When the pool is full, the thread that submitted the task runs it itself. That keeps the producer busy, so it slows down. You get simple back-pressure without dropping any work.

4.4 Callable, Future and their limits

Why?

> **In simple words:** A Future is a receipt for work happening in the background. You keep the receipt, and later you use it to collect the result — or give up if it takes too long.

A `Runnable` cannot return a result. It also cannot throw a checked exception.

But background work usually produces a value, such as a score or a price. So you need a handle to collect the result later. You also want a timeout and a way to cancel.

Think of a restaurant token. You order, get a token number, and sit down. When the food is ready, you collect it with the token.

What is happening internally?

1. `submit(Callable<V>)` wraps your task in a `FutureTask`.
2. A `FutureTask` plays two roles. It is the `Runnable` that the worker thread runs. It is also the `Future<V>` that you hold.
3. It stores either the result or the exception.
4. It also tracks a state: NEW → COMPLETING → NORMAL/EXCEPTIONAL/CANCELLED.
5. Threads that call `get()` are **parked** — put to sleep — until the state is final.

How does it work?

- `get()` blocks until the task is done. `get(timeout, unit)` throws `TimeoutException` if the time runs out first.
- If the task throws, `get()` rethrows the exception wrapped in `ExecutionException`. Call `getCause()` to get the original exception.

- `cancel(true)` interrupts the running task. This only works if the task responds to interrupts.
- `invokeAll(tasks)` waits for all tasks. `invokeAny(tasks)` returns the first successful result and cancels the rest.

Example

```
ExecutorService pool = Executors.newFixedThreadPool(4);
Future<Price> price = pool.submit(() -> pricingClient.price(sku));      // Callable<Price>
try {
    Price p = price.get(800, TimeUnit.MILLISECONDS);
} catch (TimeoutException e) {
    price.cancel(true);                                                  // stop waiting and interrupt the
task
    useCachedPrice();
} catch (ExecutionException e) {
    log.error("pricing failed", e.getCause());                           // the real exception
}
```

When would I use it?

Use it for the simple case: "run this in the background and collect the result, with a timeout".

If you need to combine several async steps — do this, then that; combine these two; recover from a failure — use `CompletableFuture` (4.11).

What can go wrong?

- **`get()` without a timeout** can block a request thread forever.
- **No chaining.** You can't chain or combine Futures without blocking. That is exactly the gap CompletableFuture fills.
- **Ignored Futures swallow exceptions.** If nobody calls `get()`, nobody sees the error.

Interview questions

Q1. Runnable vs Callable? Callable returns a value and can throw checked exceptions. Runnable returns nothing and cannot throw checked exceptions.

Q2. What are Future's limitations? `get()` blocks. There are no callbacks. You can't chain or combine Futures. You can't complete one manually. CompletableFuture solves all of these.

4.5 `synchronized`, monitors, and wait/notify

Why?

> **In simple words:** `synchronized` is a lock on a door: only one thread can be inside at a time, and whoever enters next sees everything the last person changed. `wait` / `notify` let a thread sleep until someone tells it "the thing you need is ready".

You often need two guarantees for a block of code:

- **Mutual exclusion** — only one thread can run it at a time.
- **Visibility** — changes made inside are seen by the next thread that enters.

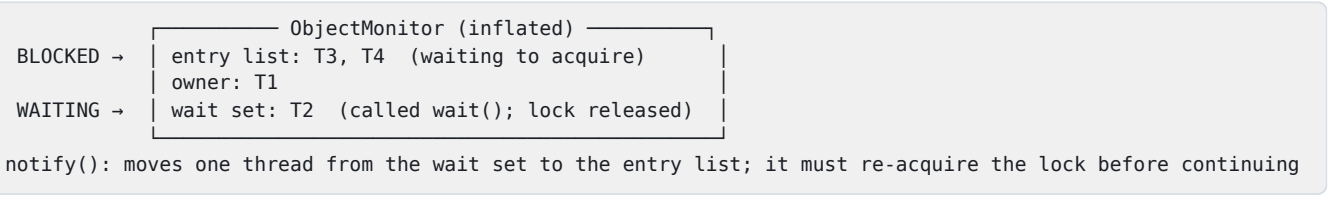
`synchronized` is Java's built-in tool for both.

`wait` / `notify` add one more ability. A thread can sleep until a condition becomes true, such as "the queue is not empty".

Think of a single-key washroom at an office. Whoever has the key goes in. Others wait outside. When the key comes back, the next person goes in.

What is happening internally?

- **Every Java object can act as a monitor** — a lock plus a waiting room. The object header has a field called the *mark word*. It stores the object's lock state.
- **Uncontended locking is cheap.** "Uncontended" means no other thread wants the lock right now. In that case the JVM uses a **lightweight (thin) lock**. It does one CAS (an atomic compare-and-swap, see 4.8) on the mark word. The mark word then points to a lock record on the owning thread's stack. (Biased locking, an older trick, was removed in Java 15/18.)
- **Under contention, the lock inflates.** When threads start fighting for the lock, it becomes a heavyweight monitor — an `ObjectMonitor`. This has an entry list of BLOCKED threads and a wait set of WAITING threads. It uses OS-level parking to put threads to sleep.
- **Bytecode.** A synchronized block compiles to `monitorenter` / `monitorexit`. There is also an exception path that runs `monitorexit`, so the lock is released even on errors. A synchronized method instead sets the `ACC_SYNCHRONIZED` flag.
- **Memory effects.** Releasing a monitor flushes your writes to memory. Acquiring it throws away stale cached reads. In JMM terms, an unlock *happens-before* every later lock of the same monitor (4.6).
- **Reentrant.** A thread that already holds the lock can enter other synchronized blocks on the same object. A counter tracks how deep it is.



How does it work?

1. A thread holding the monitor calls `wait()`. It is only allowed while holding the monitor.
2. `wait()` **releases the lock** and moves the thread into the wait set.
3. Another thread changes the state and calls `notify()` or `notifyAll()`. This moves one waiter (or all waiters) back to compete for the lock.
4. The woken thread reacquires the lock.
5. It **must re-check the condition** in a `while` loop. Why? Because another thread may have changed the state in between. Also, the JVM allows **spurious wakeups** — a thread can wake up even when nobody called notify.

Example

```java
public class Inventory {
    private final Object lock = new Object();                // private lock object: outsiders can't
interfere
    private final Map<String, Integer> stock = new HashMap<>();

    public boolean reserve(String sku, int qty) {
        synchronized (lock) {                                // check-then-act made atomic
            int have = stock.getOrDefault(sku, 0);
            if (have < qty) return false;
            stock.put(sku, have - qty);
            return true;
        }
    }
```

```
    public void awaitStock(String sku, int qty) throws InterruptedException {
        synchronized (lock) {
            while (stock.getOrDefault(sku, 0) < qty) lock.wait();   // ALWAYS a while loop
        }
    }
    public void restock(String sku, int qty) {
        synchronized (lock) {
            stock.merge(sku, qty, Integer::sum);
            lock.notifyAll();                                  // wake waiters to re-check their conditions
        }
    }
}
```

When would I use it?

- Simple mutual exclusion around small critical sections.
- Classic interview problems, such as odd/even printing and producer–consumer.

For production producer–consumer code, use a `BlockingQueue` instead. If you need timeouts, fairness or several separate conditions, use `ReentrantLock` (4.7).

What can go wrong?

- **Locking on the wrong object:**
- On `this` in library classes — callers can lock the same object too.
- On String literals or boxed Integers — these are shared, pooled instances used elsewhere.
- On a non-final field — the field can change, so threads lock different objects.
- **Holding a lock during slow I/O.** This kills throughput and invites deadlocks.
- **`wait` in an `if` instead of `while`.** This causes bugs on spurious wakeups, or when another thread already used up the condition.
- **`notify` instead of `notifyAll`** when there are several kinds of waiters. It can wake the wrong thread. The right thread never wakes, and the system hangs. This is called a *lost wakeup*.
- **`wait` / `notify` outside a synchronized block** throws `IllegalMonitorStateException`.
- **Virtual threads (Java 21).** Blocking inside `synchronized` *pins* the carrier thread — the real OS thread under the virtual thread cannot be freed (4.12).

Interview questions

Q1. What does `synchronized` guarantee? Two things. Mutual exclusion on the monitor: only one thread at a time. And visibility: everything written before releasing the lock is seen by the next thread that acquires it.

Q2. `sleep()` vs `wait()`? `sleep` is a static Thread method. It pauses the thread but does not release any locks. `wait` must be called while holding the monitor. It releases the monitor and waits for a notify or a timeout.

Q3. Why must `wait()` be called in a loop? Spurious wakeups are allowed — a thread can wake without a notify. Also, the condition may have changed between the notify and the moment the thread gets the lock back. So it must check again.

Q4. Is `synchronized` reentrant? Why does it matter? Yes. A thread can re-enter locks it already holds. So synchronized methods on the same object can call each other without deadlocking on themselves.

4.6 The Java Memory Model, `volatile` and happens-before

Why?

> **In simple words:** The CPU and compiler shuffle and cache memory to go faster, so one thread may not see another thread's changes. The Java Memory Model is the rulebook that says when a change *must* be seen, and `volatile` is the simplest way to force it.

Modern CPUs and the JIT compiler aggressively reorder and cache memory operations. Without clear rules, you could not reason about what one thread sees of another thread's writes.

The **Java Memory Model (JMM)** gives these rules. It defines exactly when a write is guaranteed to be visible. It does this through a relation called **happens-before**. It also gives you `volatile`, the lightest tool to create a happens-before link.

What is happening internally?

Happens-before (HB): if action A happens-before action B, then B sees all of A's memory changes. B also sees everything that happened before A.

The main ways to get an HB link are:

- **Program order** inside one thread.
- **Unlock → later lock** of the same monitor.
- **Volatile write → later volatile read** of the same variable.
- **Thread start and join.** `Thread.start()` → everything in the started thread. Everything in a thread → a successful `join()` on it.
- **Hand-offs between threads.** Submitting a task to an executor → the task running. A `Future` completing → `get()` returning. Putting into a concurrent collection → taking it out.
- **HB is transitive.** If A → B and B → C, then A → C.

How `volatile` works:

1. The JIT cannot cache a volatile variable in a register. Every read and write goes to memory.
2. The CPU adds **memory barriers** (also called fences) around it. These stop other reads and writes from being reordered across the volatile access.
3. So a volatile write *publishes* everything written before it. A thread that reads the volatile then sees all those earlier writes too.

`volatile` gives **visibility and ordering, but not atomicity**. With `volatile int c; c++`, updates are still lost — `c++` is still three steps.

```
Thread A                                Thread B
data = 42;            (1)
ready = true;  (volatile write) (2) ──HB──►  if (ready)  (volatile read) (3)
                                               print(data)                (4) → guaranteed 42
Without volatile: B might loop forever on a stale `ready`, or see ready=true but data=0 (reordered).
```

How does it work?

Use `volatile` for a single variable that one thread writes and others read. Good examples: flags, and configuration references that you swap in one go. It works only when you don't need a compound read-modify-write. For anything else, use locks or atomics.

Double-checked locking (a lazy singleton) needs `volatile`. Here is why. `instance = new X()` is really three steps:

1. Allocate memory.
2. Run the constructor.
3. Publish the reference into `instance`.

Without `volatile`, step 3 can be reordered before step 2 finishes. Then another thread sees a non-null reference to a half-built object, and uses it.

Example

```
class Poller implements Runnable {
    private volatile boolean running = true;          // without volatile the loop may never see 'false'
    public void stop() { running = false; }
    public void run() { while (running) poll(); }
}

class RuleRegistry {                                  // hot-swappable immutable config
    private volatile Map<String, Rule> rules = Map.of();
    void reload(Map<String, Rule> fresh) { rules = Map.copyOf(fresh); }   // one volatile write publishes the
whole map
    Rule get(String id) { return rules.get(id); }                          // lock-free reads
}

public final class ConfigHolder {                     // double-checked locking, done right
    private static volatile ConfigHolder instance;
    public static ConfigHolder get() {
        ConfigHolder local = instance;                // one volatile read on the fast path
        if (local == null) {
            synchronized (ConfigHolder.class) {
                local = instance;
                if (local == null) instance = local = new ConfigHolder();
            }
        }
        return local;
    }
    private ConfigHolder() {}
}
// Simpler and equally lazy: the holder idiom (class init is thread-safe by JVM spec)
final class Config2 { private static class H { static final Config2 I = new Config2(); } static Config2 get()
{ return H.I; } }
```

When would I use it?

- Stop flags and status flags.
- Publishing immutable snapshots that many threads read — config, rule tables, feature flags.
- Lazy initialisation patterns.

Most of the time, you get happens-before "for free" from higher-level tools: executors, concurrent collections and CompletableFuture.

What can go wrong?

- **Using `volatile` for counters or check-then-act.** It is not atomic, so updates get lost.
- **Volatile reference, mutable object.** Making the *reference* volatile does not protect the object behind it. For example, `volatile Map` plus `map.put` from many threads is not safe. Instead, build a new immutable object and swap the whole reference.
- **Synchronising only the writer.** Readers must also synchronise (or the field must be volatile). Otherwise they may never see the write.

Interview questions

Q1. What does `volatile` guarantee? Visibility: writes are seen by all threads. And ordering: no reordering across the volatile access. It does not make compound operations atomic.

Q2. What is happens-before? It is the JMM's ordering guarantee: if A happens-before B, then B sees A's effects. It is created by locks, volatile, thread start/join, executor hand-offs and concurrent collections. It is also transitive.

Q3. Why does double-checked locking need volatile? Without it, the reference can be published before the constructor finishes. Another thread may then use a partly constructed object.

Q4. volatile vs synchronized vs AtomicInteger? volatile gives visibility only. synchronized gives mutual exclusion plus visibility for a whole block. AtomicInteger gives lock-free atomic operations on a single variable, with visibility.

4.7 Explicit locks: ReentrantLock, ReadWriteLock, StampedLock, Condition

Why?

> **In simple words:** `synchronized` is a basic lock with no options. Explicit locks add the options real systems need: "give up if you can't get the lock in 500 ms", "let many readers in together", and "separate waiting rooms for different reasons".

`synchronized` has four limits:

- It can't time out.
- A thread waiting for it can't be interrupted.
- It isn't fair — it doesn't serve threads in arrival order.
- It has only one wait set.

Real systems need more:

- "Try to get the lock for 500 ms, else fail fast." This avoids hanging requests and deadlocks.
- "Many readers or one writer" for caches that are read much more than written.
- Several separate wait conditions, such as "not full" and "not empty".

What is happening internally?

Most `java.util.concurrent` locks are built on **AbstractQueuedSynchronizer (AQS)**. AQS has these parts:

- **An `int state`.** For ReentrantLock, 0 means free, and n means the owner holds it n times. It is changed with **CAS** (atomic compare-and-swap, 4.8).
- **A FIFO wait queue** of parked (sleeping) threads. It is built from CLH-style linked nodes.
- **Acquire steps:** 1. Try a CAS on the state. 2. If it fails, join the queue and **park** (`LockSupport.park`). 3. Sleep until the releasing thread calls **unpark** on you.

Other details:

- **Fair mode** hands the lock out in queue order. Nobody can jump the line (no *barging*). So no thread starves, but throughput is lower.

- **Non-fair mode** is the default. A newly arriving thread may grab a lock that was just freed. Throughput is higher.
- `Condition` objects each keep their own wait queue. `await()` releases the lock, just like `wait()`.
- `ReentrantReadWriteLock` splits the AQS state into two parts: a read count (high 16 bits) and a write count (low 16 bits).
- `StampedLock` (Java 8) adds **optimistic reads**: 1. Read without taking any lock. 2. Call `validate(stamp)` to check that no write happened meanwhile. 3. If a write did happen, fall back to a real read lock.

This is very fast when reads dominate. But it is not reentrant.

How does it work?

```
lock.lock();
try { /* critical section */ }
finally { lock.unlock(); }          // ALWAYS in finally — unlike synchronized, nothing releases it for you
```

Feature	`synchronized`	`ReentrantLock`
Timed / try acquisition	No	`tryLock()`, `tryLock(t, unit)`
Interruptible waiting	No	`lockInterruptibly()`
Fairness	No	Optional
Multiple conditions	One	Many, via `newCondition()`
Auto release	Yes	Manual (in `finally`)
Virtual-thread friendly (Java 21)	Pins when blocking	Doesn't pin

Example

```
public class TransferService {
    private final ReentrantLock lock = new ReentrantLock();
    public boolean transfer(Account from, Account to, long amt) throws InterruptedException {
        if (!lock.tryLock(500, TimeUnit.MILLISECONDS)) return false;     // fail fast instead of hanging
        try {
            if (from.balance() < amt) return false;
            from.debit(amt); to.credit(amt);
            return true;
        } finally { lock.unlock(); }
    }
}

public class RuleCache {                                    // read-heavy: many concurrent readers
    private final ReadWriteLock rw = new ReentrantReadWriteLock();
    private final Map<String, Rule> rules = new HashMap<>();
    public Rule get(String id) { rw.readLock().lock();  try { return rules.get(id); } finally {
rw.readLock().unlock(); } }
    public void put(String id, Rule r) { rw.writeLock().lock(); try { rules.put(id, r); } finally {
rw.writeLock().unlock(); } }
}

class Point {                                               // StampedLock optimistic read
    private double x, y; private final StampedLock sl = new StampedLock();
    double distanceFromOrigin() {
        long stamp = sl.tryOptimisticRead();
        double cx = x, cy = y;
        if (!sl.validate(stamp)) {                          // a write happened: retry with a real read lock
            stamp = sl.readLock();
            try { cx = x; cy = y; } finally { sl.unlockRead(stamp); }
```

```
        }
        return Math.hypot(cx, cy);
    }
}
```

When would I use it?

- **`tryLock` with timeouts** around resources that may be contended. Examples: account transfers, seat holds kept in memory.
- **Read-write locks** for caches that are mostly read, when a `ConcurrentHashMap` doesn't fit.
- **Conditions** for custom blocking structures, such as the bounded buffer in 4.14.
- **`ReentrantLock` instead of `synchronized`** around blocking calls on virtual threads, because it doesn't pin.

What can go wrong?

- **Forgetting `unlock()` in `finally`.** If an exception happens, the lock stays held forever. Everyone else hangs.
- **Trying to upgrade a read lock.** `ReadWriteLock` doesn't let you upgrade a read lock to a write lock. If you try, you deadlock.
- **Writer starvation.** In non-fair mode, under heavy reads, writers may wait for a very long time.
- **Fair locks are slow.** They cut throughput a lot. Use them only when starvation is a real problem.
- **`StampedLock` traps.** It isn't reentrant. And with optimistic reads, you must copy fields into local variables before you validate.

Interview questions

Q1. ReentrantLock vs synchronized? ReentrantLock adds tryLock with timeouts, interruptible acquisition, optional fairness and multiple Conditions. But you must unlock it yourself in finally. synchronized is simpler and is released automatically.

Q2. What is AQS? AbstractQueuedSynchronizer. It is the framework behind most j.u.c locks and synchronisers. It uses an int state changed by CAS, and a FIFO queue of parked threads.

Q3. When would you use a ReadWriteLock? For shared data that is read a lot and written rarely. Reads can then run at the same time, and only writes are exclusive.

Q4. Fair vs non-fair locks? A fair lock serves threads in arrival order: no starvation, but slower. A non-fair lock lets new threads jump in (barging) for higher throughput. Non-fair is the default.

4.8 Atomics and compare-and-swap (CAS)

Why?

> **In simple words:** Instead of locking, a thread says "change this value from 5 to 6, but only if it is still 5". If someone else changed it first, the thread just tries again. Nobody ever sleeps or waits.

Sometimes you share just one variable — a counter, a sequence number, or a reference to the latest config. A lock is heavy for this. Threads get parked and unparked, and each context switch costs microseconds.

Modern CPUs offer an atomic instruction called **compare-and-swap (CAS)**. It lets you update a variable safely **without blocking** any thread.

What is happening internally?

- **CAS(address, expected, new)** does this in one atomic step: "If the value at this address equals `expected`, set it to `new` and report success. Otherwise, report failure." On x86 it is the `LOCK CMPXCHG` instruction.
- **`AtomicInteger.incrementAndGet()` is a loop:** 1. Read the current value. 2. Try CAS(current, current+1). 3. If it fails, another thread changed the value. Go back to step 1.

This is called **lock-free**: at any moment, at least some thread is making progress. - **Visibility.** The value field is `volatile`, so atomics also give visibility. - **Cost under contention.** When many threads hit the same atomic, many CAS attempts fail and retry. The cache line also bounces between cores. **`LongAdder`** fixes this. It spreads the count across an array of cells — roughly one per contending thread, chosen by hash. On read, it adds up all the cells. So increments rarely collide. - **The ABA problem.** The value changes from A to B and back to A between your read and your CAS. Your CAS succeeds, even though the state did change in between. This matters for lock-free stacks and queues. The fix is a version stamp (`AtomicStampedReference`).

How does it work?

```
incrementAndGet():
  loop:
    cur = value                // volatile read
    if CAS(value, cur, cur+1)  // hardware atomic
        return cur+1
    // else someone else won the race: retry with the fresh value
```

Example

```
AtomicInteger seq = new AtomicInteger();
int id = seq.incrementAndGet();
seq.updateAndGet(x -> Math.min(x * 2, 1_000));        // arbitrary atomic update (function may run multiple
times!)
boolean won = seq.compareAndSet(5, 6);                // only one of many racing threads succeeds

LongAdder requests = new LongAdder();                 // metrics counter under heavy contention
requests.increment();
long total = requests.sum();                          // not an atomic snapshot while increments continue

AtomicReference<RateCard> card = new AtomicReference<>(RateCard.initial());
card.updateAndGet(c -> c.withSurge(1.2));             // immutable object + CAS = lock-free config updates

// Exactly-once initialisation flag
AtomicBoolean started = new AtomicBoolean(false);
if (started.compareAndSet(false, true)) startScheduler();   // only the first caller starts it
```

When would I use it?

- ID and sequence generators.
- Counters and metrics (`LongAdder`).
- "Only once" flags.
- Lock-free swaps of immutable state (`AtomicReference`).
- Simple rate-limiter state.

If several variables must change together, use a lock. Or put them into one immutable object inside an `AtomicReference`.

What can go wrong?

- **Two atomics are not atomic together.** If you update them one after the other, another thread can see one changed and the other not. For example, balance and version kept as separate atomics can drift apart.
- **Update functions can run more than once.** Under contention, the function passed to `updateAndGet` may run several times. So it must have no side effects.
- **Busy CAS loops waste CPU** under extreme contention. `LongAdder` or a lock may be better.
- **ABA** in hand-written lock-free structures.

Interview questions

Q1. What is CAS? An atomic CPU instruction. It sets a memory location to a new value only if it currently holds the expected value. It is the base of lock-free algorithms and Java atomics.

Q2. How does AtomicInteger.incrementAndGet work? With a retry loop. It reads the volatile value, tries a CAS to value+1, and retries if the CAS fails.

Q3. AtomicLong vs LongAdder? LongAdder splits (stripes) the count across several cells to reduce contention. So increments are much faster with many threads. The cost is that `sum()` is slower and is not an atomic snapshot.

Q4. What is the ABA problem? CAS can't detect that a value changed and then changed back. You solve it with versioned references, like AtomicStampedReference.

4.9 ConcurrentHashMap internals

Why?

> **In simple words:** `ConcurrentHashMap` is a map that many threads can use at the same time safely. Readers never wait, and writers only lock the one small bucket they are changing — not the whole map.

Servers use shared maps all the time. Some examples:

- an in-process cache (a cache inside the app's own memory),
- a counter per tenant,
- a set of IDs you have already seen, to skip duplicates (a de-duplication set),
- a registry of active sessions.

A plain `HashMap` breaks if two threads write to it at the same time. Its internal data can get corrupted.

`Hashtable` and `Collections.synchronizedMap` are safe. But they use **one lock for everything**. So every reader and every writer must stand in one line, one after another.

`ConcurrentHashMap` (CHM) fixes this:

- Reads take no lock at all.
- Writes to different buckets run in parallel.

Think of a big post office with many counters. `Hashtable` has one counter for everyone. CHM opens a separate counter for each bucket, and anyone who only wants to *look* at something does not need a counter at all.

What is happening internally?

Java 7: the map was split into 16 parts called `Segment`s. Each segment was a small hash table with its own `ReentrantLock`. So at most 16 threads could write at the same time.

Java 8+ (this is the version to describe in interviews): there is one `Node[] table`, just like HashMap. But it adds these tricks:

- **Reads take no lock.** The `table` and each node's `val` and `next` fields are `volatile`. Volatile means every thread always sees the latest written value. So `get` just reads; it never waits.
- **Writes to an empty bin use CAS.** CAS means *compare-and-swap*: "put my node here only if the slot is still empty", done as one atomic CPU step. The call is `casTabAt(tab, i, null, newNode)`. This common case needs no lock.
- **Writes to a non-empty bin use `synchronized` on the bin's first node.** Only that one bucket is locked. Writers to other buckets carry on in parallel.
- **Treeify at 8 nodes.** Like HashMap, a bin with 8 nodes turns into a tree. Here it is a `TreeBin`, which has its own lightweight read-write locking.
- **Resizing is shared work (cooperative).** It works like this: 1. A resizing thread moves bins to a new, bigger table. 2. After moving a bin, it puts a `ForwardingNode` in the old bin. This is a signpost saying "moved to the new table". 3. If another thread tries to *write* and hits a forwarding node, it does not wait. It **helps transfer** a range of bins. 4. If a *read* hits a forwarding node, it simply looks in the new table.
- **`size()` uses a `baseCount` plus a `CounterCell[]` array.** This is the same idea as `LongAdder`: threads add to different cells, so no single counter becomes a hotspot. The catch: while writes are happening, `size()` is only an estimate.
- **No null keys or values.** In a concurrent map, `get` returning `null` must clearly mean "not present". Why? Because you cannot run `containsKey` and `get` together as one atomic step. Another thread could change the map between the two calls. So null is reserved to mean "absent".

```
table: [ ∅ ][ A→B ][ ∅ ][ C ][ FWD ][ D→E→F ] ...
put into an empty bin  → CAS (no lock)
put into "A→B" bin      → synchronized(A) { append/replace }
bin already moved       → ForwardingNode: help resize, then retry in the new table
get(...)                → volatile reads only, never blocks
```

How does it work?

The real power of CHM is its **atomic compound operations**. A compound operation is a "check and then change" done as one step. This is what makes CHM more than just "a safe HashMap".

Each of these runs atomically **per key**. It holds that key's bin lock while it runs:

Method	Atomic behaviour
`putIfAbsent(k, v)`	Insert only if the key is not there yet
`computeIfAbsent(k, fn)`	Compute and insert only if absent; `fn` runs at most once per key
`compute(k, (k, old) -> new)`	Read, change and write back in one step; return null to remove the key
`merge(k, v, fn)`	Insert v, or combine it with the existing value using fn
`replace(k, old, new)`	CAS on the value: replace only if the current value is still `old`

Example

```java
// Per-SKU request counters (hot, many threads)
ConcurrentHashMap<String, LongAdder> hits = new ConcurrentHashMap<>();
hits.computeIfAbsent(sku, k -> new LongAdder()).increment();

// Atomic stock reservation: check-then-act inside compute, under the bin lock
ConcurrentHashMap<String, Integer> stock = new ConcurrentHashMap<>(Map.of("SKU1", 10));
boolean[] reserved = {false};
stock.compute("SKU1", (k, have) -> {
    if (have == null || have < 3) return have;    // not enough: unchanged
    reserved[0] = true;
    return have - 3;
});

// WRONG — two separate calls, a race even on CHM
if (!map.containsKey(k)) map.put(k, expensiveLoad(k));
// RIGHT
map.computeIfAbsent(k, this::expensiveLoad);

// Concurrent set
Set<String> seenEventIds = ConcurrentHashMap.newKeySet();
if (!seenEventIds.add(eventId)) return;           // duplicate delivery
```

When would I use it?

- In-process caches and registries shared by request threads.
- Counters and metrics.
- De-duplication sets (via `ConcurrentHashMap.newKeySet()`).
- Per-key memoisation, which means "compute once per key and remember the result" (`computeIfAbsent`).
- Connection or session registries (as in the chat design, 13.5).

If your cache needs a size limit or TTLs (time-to-live — entries expire after some time), use Caffeine instead. It is built on similar ideas.

What can go wrong?

- **Compound actions built from separate calls are races.** For example, `get` then `put`, or `containsKey` then `put`. Another thread can slip in between the two calls. Use the atomic methods instead.
- **Slow or blocking functions inside `compute*`.** The function runs while holding the bin lock. So other keys in the same bin are blocked until it finishes. Also, if you update the same map from inside `computeIfAbsent` (a recursive update), it can throw `IllegalStateException` ("Recursive update") or deadlock.
- **Iterators are weakly consistent.** They never throw `ConcurrentModificationException`. But they may or may not show changes made while you iterate. So don't use them when you need an exact snapshot.
- **`size()` is approximate** while other threads are writing.
- **Unbounded growth.** A CHM used as a cache with no eviction (nothing ever removed) is a memory leak (5.7).

Interview questions

Q1. How does ConcurrentHashMap achieve thread safety in Java 8? Reads are lock-free because the fields are volatile. Inserting into an empty bin uses CAS. Writing to a non-empty bin uses `synchronized`

on that bin's first node only. Resizing is cooperative — threads that hit a forwarding node help move bins. And the size is kept in striped counters, so counting is not a bottleneck.

Q2. Why doesn't it allow null keys or values? In a concurrent map, if `get` returned null, you could not tell "key absent" from "key mapped to null". You also cannot check both with `containsKey` and `get` atomically, because another thread may change the map in between. So null is banned and always means "absent".

Q3. HashMap vs Hashtable vs synchronizedMap vs ConcurrentHashMap? HashMap has no synchronisation, so it is unsafe for concurrent writes. Hashtable and synchronizedMap use one lock for every operation, so all access goes one at a time. ConcurrentHashMap allows reads in parallel with no lock, and writes in parallel with fine-grained, per-bucket locking.

Q4. Is `map.get(k) + map.put(k, v+1)` safe on ConcurrentHashMap? No. Each call on its own is atomic, but the pair is not — another thread can update between them and you lose its update. Use `merge(k, 1, Integer::sum)` or `compute` to do it in one atomic step.

4.10 Synchronizers: CountDownLatch, CyclicBarrier, Semaphore

Why?

> **In simple words:** These are ready-made tools for making threads wait for each other. A latch says "wait until N things are done", a barrier says "wait until all N friends arrive", and a semaphore says "only N people inside at a time".

Many coordination problems are not about protecting data. They are about timing, like:

- "wait until N things happen",
- "let N threads start together",
- "allow at most N calls at a time to this slow partner service".

You could build these yourself with `wait` / `notify`. But that is easy to get wrong. The JDK gives you tested building blocks instead. They are built on AQS (`AbstractQueuedSynchronizer` — the JDK's common base for locks and waiting queues).

What is happening internally?

Tool	AQS state means	Behaviour	Reusable
`CountDownLatch(n)`	How many counts are left	`countDown()` reduces the count by one; `await()` blocks until it reaches 0	No (one-shot)
`CyclicBarrier(n, action)`	Threads still to arrive (built on a lock + condition, not AQS directly)	Each `await()` blocks until n threads arrive; then the optional action runs and all continue	Yes (resets for each round, called a "generation")
`Semaphore(permits)`	Permits available	`acquire()` takes one (blocks when 0 are left); `release()` gives one back	Yes

Tool	AQS state means	Behaviour	Reusable
`Phaser`	Parties and phases that can change at runtime	A flexible barrier where threads can join (register) and leave (deregister)	Yes

How does it work?

- **Latch**: a one-way gate. Worker threads count down. The coordinator thread waits for the count to reach zero. Once open, it never closes again.
- **Barrier**: threads wait for *each other* at a meeting point (a rendezvous). It is used for work done in phases — everyone finishes phase 1, then everyone starts phase 2.
- **Semaphore**: a counter of permits. It is the classic way to **limit how many threads use a resource at once**. A semaphore has no owner thread, so any thread can call `release()`. A lock is different: only the thread that locked it can unlock it.

Think of a restaurant with 10 tables. The manager hands out 10 table tokens (permits). When all tokens are out, new guests wait. When a guest leaves, the token goes back, and the next guest gets in.

Example

```
// Fan out to 3 services; wait at most 2 s for all
ExecutorService io = Executors.newFixedThreadPool(3);
CountDownLatch done = new CountDownLatch(3);
Map<String, Object> results = new ConcurrentHashMap<>();
for (String svc : List.of("pricing", "stock", "schemes")) {
    io.submit(() -> {
        try { results.put(svc, call(svc)); }
        finally { done.countDown(); }              // ALWAYS count down, even on failure
    });
}
boolean allArrived = done.await(2, TimeUnit.SECONDS);

// Limit concurrent calls to a partner API that allows 10 parallel requests
Semaphore partnerSlots = new Semaphore(10);
Voucher issueVoucher(Req r) throws InterruptedException {
    if (!partnerSlots.tryAcquire(500, TimeUnit.MILLISECONDS)) throw new BusyException("partner saturated");
    try { return partner.issue(r); }
    finally { partnerSlots.release(); }
}

// Barrier: 4 workers compute a phase, then merge, then continue
CyclicBarrier barrier = new CyclicBarrier(4, () -> System.out.println("phase merged"));
```

When would I use it?

- **Latch**:
- start-up readiness, e.g. wait for caches to warm up before taking traffic;
- tests that wait for async work to finish;
- fan-out-and-wait (start many tasks, wait for all) — though CompletableFuture's `allOf` is usually nicer.
- **Semaphore**:
- bulkheads around a slow dependency (a bulkhead caps how many threads one dependency can take, so it can't sink the whole app);
- limiting how many virtual threads hit the DB at the same time;
- simple resource pools.
- **Barrier**: parallel simulations and batch work done in phases.

What can go wrong?

- **Forgetting `countDown()` on the error path.** If a task throws before counting down, `await()` hangs forever. Put `countDown()` in `finally`, and give `await` a timeout.
- **`release()` without a matching `acquire()`.** This adds an extra permit, so more threads get in than you wanted. A semaphore does not track who took which permit.
- **One bad thread breaks a `CyclicBarrier` for everyone.** If a thread fails or times out at the barrier, all waiting threads get `BrokenBarrierException`.

Interview questions

Q1. CountDownLatch vs CyclicBarrier? A latch lets one or more threads wait for N events, and it is one-shot — it cannot be reset. A barrier makes N threads wait for each other. It is reusable for the next round, and it can run an action once all threads arrive.

Q2. What is a Semaphore used for? It limits how many threads can use a resource at the same time. For example, at most N parallel calls to a partner API, or at most N DB-heavy jobs running together.

Q3. Semaphore vs lock? A lock has one owner and gives mutual exclusion — only one thread inside at a time. A semaphore has N permits and no owner, so any thread may release a permit.

4.11 CompletableFuture: composing asynchronous work

Why?

> **In simple words:** `CompletableFuture` lets you start several slow calls at once and say "when they finish, do this next" — without making a thread sit and wait. It is Java's version of C#'s `Task`.

Say a recommendation API needs three things from three services: price, stock and eligible schemes.

- If you call them one after another, the waits add up: 300 + 250 + 400 ms = 950 ms.
- If you call them in parallel, you only wait for the slowest one: about 400 ms.

Plain `Future` can run them in parallel. But the only way to get the result is `get()`, which blocks the thread until the value is ready.

`CompletableFuture` (CF) is better. Without blocking any thread, it lets you:

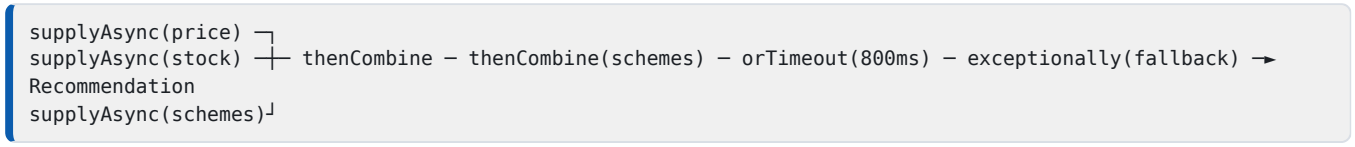
- **chain** steps ("then do this"),
- **combine** results,
- **time out** slow work,
- **recover** from errors.

If you know C#, this is like `Task` with `ContinueWith` and `WhenAll`.

What is happening internally?

- A CF has a `result` field. It holds the value, or a wrapper around an exception.
- It also has a **stack of dependent actions**. Each one is a `Completion` object — "work to do when I finish".
- `thenApply(fn)` adds a dependent stage to that stack. When the CF completes, the thread that completed it **pops and runs the dependents**. For the `*Async` versions, it instead hands them to an executor.

- If the CF is **already complete** when you call `thenApply`, the function runs **right away, on your calling thread**.
- `supplyAsync(supplier)` with no executor uses the **common ForkJoinPool**. This one pool is shared by the whole JVM, and its size is (CPU cores − 1).
- Exceptions travel down the chain, wrapped in a `CompletionException`, until some stage handles them (`exceptionally` or `handle`).



How does it work?

The main methods, with the C#/functional idea each one matches:

Method	Analogy	Use
`supplyAsync(s, executor)`	`Task.Run`	Start async work that returns a value
`thenApply(fn)`	`map`	Change the result with a normal (non-async) function
`thenCompose(fn)`	`flatMap`	Chain another async call that itself returns a CF (avoids ending up with `CF<CF<T>>`)
`thenCombine(other, fn)`	zip	Join two independent results into one
`thenAccept` / `thenRun`		Use the result / just run something after; returns no result
`allOf(...)` / `anyOf(...)`	`WhenAll` / `WhenAny`	Fan-in: wait for all / for the first one
`exceptionally(fn)`	catch	Recover from an error by returning a value
`handle((v, ex) -> ...)`		Look at success or failure, and return a value either way
`whenComplete((v, ex) -> ...)`	finally	Do a side effect (like logging); the original outcome passes through unchanged
`orTimeout(t)` / `completeOnTimeout(v, t)` (Java 9)		After the timeout, fail / use a default value
`*Async` variants		Run the stage on an executor instead of on the thread that completed the previous stage

Example

```
ExecutorService io = new ThreadPoolExecutor(16, 64, 60, TimeUnit.SECONDS,
        new ArrayBlockingQueue<>(500), Thread.ofPlatform().name("io-", 0).factory(),
        new ThreadPoolExecutor.CallerRunsPolicy());

CompletableFuture<Price> price   = CompletableFuture.supplyAsync(() -> pricing.get(sku), io);
CompletableFuture<Stock> stock   = CompletableFuture.supplyAsync(() -> inventory.get(sku), io);
CompletableFuture<List<Scheme>> schemes = CompletableFuture.supplyAsync(() -> schemeSvc.eligible(outlet,
sku), io)
        .completeOnTimeout(List.of(), 300, TimeUnit.MILLISECONDS);     // optional data: default if slow

CompletableFuture<Recommendation> rec = price
```

```
        .thenCombine(stock, Partial::new)                          // both needed
        .thenCombine(schemes, (p, s) -> recommend(p, s))
        .orTimeout(800, TimeUnit.MILLISECONDS)                     // overall budget
        .exceptionally(ex -> {
            log.warn("recommendation degraded for {}", sku, ex);
            return Recommendation.fallback(sku);
        });

// thenCompose: dependent async calls (need the user before fetching their tier)
CompletableFuture<Tier> tier = CompletableFuture.supplyAsync(() -> users.find(id), io)
        .thenCompose(user -> CompletableFuture.supplyAsync(() -> loyalty.tier(user), io));

// Fan out over a list and collect all results
List<CompletableFuture<Long>> fs = outlets.stream()
        .map(o -> CompletableFuture.supplyAsync(() -> points(o), io)).toList();
List<Long> all = CompletableFuture.allOf(fs.toArray(new CompletableFuture[0]))
        .thenApply(v -> fs.stream().map(CompletableFuture::join).toList())
        .join();
```

When would I use it?

- Aggregator endpoints that call several services in parallel. NOVA's recommendation flow is exactly this.
- Async pipelines that need fallbacks.
- Spring `@Async` methods that return `CompletableFuture`.
- Non-blocking HTTP clients (`HttpClient.sendAsync`).

With Java 21, there is often a simpler option for the same fan-out: plain blocking code on virtual threads (4.12).

What can go wrong?

- **No executor means the common pool.** If you do blocking I/O there, you starve parallel streams and every other CF in the whole JVM. Always pass your own I/O executor.
- **Lost exceptions.** If a chain has no `exceptionally` / `handle`, and nobody calls `join()`, the error disappears silently.
- **Mixing up `thenApply` and `thenCompose`.** Using `thenApply` with a function that returns a CF gives you `CompletableFuture<CompletableFuture<T>>`.
- **Calling `join()` / `get()` inside a stage.** This blocks a pool thread while it waits. With a small pool, that can deadlock.
- **No timeout.** A dependency that hangs will hold resources forever. Use `orTimeout`, plus timeouts on the HTTP/DB client itself.
- **ThreadLocal context does not follow.** Things like MDC (the logging context) and the security context are stored per thread. They are not copied to executor threads automatically. Wrap the executor, or pass the context explicitly.

Interview questions

Q1. Future vs CompletableFuture? Future only gives you a blocking `get` and `cancel`. CompletableFuture adds non-blocking callbacks, chaining, combining results, timeouts, completing it yourself from code, and proper exception handling.

Q2. `thenApply` vs `thenCompose` vs `thenCombine`? `thenApply` transforms the value, like `map`. `thenCompose` chains a function that returns another CF and flattens the result, like `flatMap`. `thenCombine` merges the results of two independent CFs.

Q3. Which thread runs `thenApply`? The thread that completes the previous stage runs it. If the stage is already complete, your calling thread runs it immediately. `thenApplyAsync` instead runs it on the executor you pass (or the common pool if you pass none).

Q4. How do you handle errors in a CF chain? Use `exceptionally` to recover with a value, `handle` to map both success and failure, and `whenComplete` for side effects like logging. Remember that exceptions arrive wrapped in `CompletionException`.

4.12 Virtual threads (Java 21)

Why?

> **In simple words:** Virtual threads are very cheap threads managed by the JVM. When one is waiting for the network or DB, it steps aside so the real OS thread can run someone else. So you can write simple blocking code and still handle a huge number of requests.

The thread-per-request model is easy to write and easy to debug: each request gets its own thread. The problem is that normal threads (called **platform threads**) are expensive:

- each one reserves about 1 MB of stack memory,
- each one is backed by a real OS thread.

So one server tops out at a few thousand requests at the same time.

Most requests spend most of their time **waiting on I/O** — for the DB, for another service. During that wait, the thread sits idle but still holds those scarce resources.

The old fix was reactive/async code (WebFlux, CompletableFuture chains). It scales well. But it is hard to write, hard to read and hard to debug.

Virtual threads give you the scalability of async code, while you keep writing plain blocking code.

What is happening internally?

- **What it is.** A virtual thread is a normal `Thread` object, but the JVM manages it, not the OS. Its stack is stored on the heap as small chunks that can grow (starting at a few hundred bytes). That is why you can have **millions** of them.
- **Carrier threads.** Virtual threads actually run on a small pool of real platform threads called **carriers**. This pool is a ForkJoinPool with one thread per CPU core.
- **Mounting and unmounting.** Here is what happens when a virtual thread blocks: 1. The virtual thread calls a blocking operation that the JDK has made virtual-thread-aware. Examples: socket I/O, `Thread.sleep`, `BlockingQueue.take`, `ReentrantLock`, and JDBC drivers that use java.net sockets. 2. The JVM **copies its stack frames to the heap and unmounts it** (takes it off the carrier). 3. The carrier is now free and runs another virtual thread. 4. When the I/O finishes, the virtual thread is put back in the queue. 5. It resumes on whichever carrier is free.
- **Pinning** (Java 21). Sometimes a virtual thread **cannot unmount**. This is called pinning. It happens:
- inside a `synchronized` block or method,
- inside a native frame (native code).

If the thread blocks while pinned, it keeps its carrier busy the whole time. There are only a few carriers, so a handful of pinned threads can stall everything. - Java 24 (JEP 491) removed pinning for

`synchronized`. - On Java 21, prefer `ReentrantLock` around blocking calls. - Detect pinning with `-Djdk.tracePinnedThreads=full` (Java 21) or with JFR (Java Flight Recorder) events.

Think of a bank with 4 cashiers (carriers) and 1,000 customers (virtual threads). When a customer has to go fetch a missing document (I/O), they step out of the line. The cashier serves the next person. The customer comes back later to any free cashier. Pinning is a customer who goes to fetch the document but stays standing at the counter, blocking that cashier.

```
Carrier threads (= cores):   C1           C2
Virtual threads:          VT1 running   VT7 running
                          VT2..VT6, VT8..VT100000 parked (stacks on heap, waiting for I/O)
VT1 calls db.query() → unmount VT1 → C1 picks VT3 → DB responds → VT1 queued → resumes on C1 or C2
```

How does it work?

```
Thread.ofVirtual().name("vt-", 0).start(() -> handle(request));
try (ExecutorService exec = Executors.newVirtualThreadPerTaskExecutor()) {   // one new virtual thread per task
    ...
}                                                                            // close() waits for tasks
// Spring Boot 3.2+: spring.threads.virtual.enabled=true  → Tomcat request handling and @Async on virtual threads
```

Don't pool them. Virtual threads are cheap, so create a new one for each task. To limit how many tasks hit a scarce resource at once, use a `Semaphore` or the resource's own pool (for example the DB connection pool).

Example

```
// Fetch 10,000 outlet profiles from a partner API, at most 200 in flight (partner limit)
Semaphore inFlight = new Semaphore(200);
try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<Profile>> fs = outletIds.stream().map(id -> exec.submit(() -> {
        inFlight.acquire();
        try { return partnerClient.fetch(id); }        // blocking HTTP call — cheap on a virtual thread
        finally { inFlight.release(); }
    })).toList();
    for (Future<Profile> f : fs) save(f.get());
}
// The same with platform threads would need a 200-thread pool; with reactive code, a much harder program.
```

When would I use it?

Use them for services that handle **lots of requests at once** and are **I/O-bound** (mostly waiting, not computing):

- REST APIs that call DBs and other services,
- fan-out to many HTTP calls,
- message consumers that do I/O for each message.

They give little benefit for:

- CPU-bound work — it is still limited by the number of cores;
- apps that only have a few requests at a time.

What can go wrong?

- **Pinning in Java 21.** `synchronized` around I/O pins the thread. This can be in your code or inside libraries (older JDBC drivers and connection pools). Check with pinning tracing and upgrade

libraries.

- **Overwhelming downstream systems.** A million virtual threads will happily fire a million DB queries. Now the DB connection pool and partner rate limits are your real bottleneck. Limit concurrency with semaphores.
- **Code that relies heavily on ThreadLocal.** If each of a million threads holds big ThreadLocal values, memory use multiplies. Prefer passing context explicitly, or scoped values.
- **Pooling virtual threads.** This is pointless and harmful. Create one per task.
- **CPU-heavy tasks.** They can hog the carriers. There is no time-slicing between virtual threads, so one busy thread does not get paused to let others run.

Interview questions

Q1. What are virtual threads and how do they differ from platform threads? They are lightweight threads managed by the JVM. Their stacks live on the heap, and many of them share a few carrier OS threads. When blocked on I/O they unmount and free the carrier, so millions can exist. Platform threads, in contrast, map one-to-one to expensive OS threads.

Q2. What is pinning? Pinning is when a virtual thread cannot unmount from its carrier. In Java 21 this happens inside `synchronized` or native code. If it blocks there, it holds the carrier thread for the whole wait.

Q3. Do virtual threads make code faster? They increase throughput — total requests handled — for I/O-bound concurrency, because OS threads are not tied up during waits. They do not make a single request faster, and they do not speed up CPU-bound work.

Q4. Should you pool virtual threads? No. They are cheap to create, so use one per task. Limit access to scarce resources with semaphores instead.

4.13 Deadlock, livelock and starvation

Why?

> **In simple words:** A deadlock is when two threads each hold something the other needs, so both wait forever. Once your code takes more than one lock, this can happen — so you must design it away and know how to spot it.

As soon as your code takes more than one lock (or waits on more than one resource), it can freeze forever. This usually happens under production load, and almost never in tests.

Interviewers expect you to:

- name the conditions that cause deadlock,
- prevent it by design,
- diagnose it from a thread dump (a snapshot of what every thread is doing).

What is happening internally?

A **deadlock** needs all four **Coffman conditions** to be true at the same time: 1. **Mutual exclusion**: only one thread can hold the resource at a time. 2. **Hold and wait**: a thread holds one resource while waiting for another. 3. **No preemption**: nobody can forcibly take a resource away from a thread. 4. **Circular wait**: T1 waits for T2's resource, and T2 waits for T1's.

Picture two cars meeting on a narrow one-lane bridge from opposite sides. Each holds its half of the bridge and waits for the other to back up. Neither does, so both wait forever.

```
T1: lock(A) ─── wants B ──►┐
                           │   circular wait → both BLOCKED forever
T2: lock(B) ─── wants A ──►┘
```

Two related problems:

- **Livelock**: the threads are *not* blocked. They keep reacting to each other, but no one makes progress. Example: two people in a corridor who both step aside the same way, again and again, forever. Or two transactions that keep retrying at exactly the same moments.
- **Starvation**: one thread never gets the CPU or the lock, because others always win. Causes include unfair locks, thread priorities, or a flood of readers that never lets a writer in.

How does it work?

To prevent deadlock, you break **one** of the four conditions. Any one is enough:

Break	Technique
Circular wait	**Global lock ordering**: every thread takes locks in the same fixed order (for example, by account ID)
Hold and wait	Take all locks at once, or use `tryLock` with a timeout and, on failure, release what you hold and back off
No preemption	Timeouts that make a thread give up its attempt
Mutual exclusion	Lock-free designs, immutable data, or single-writer models (actor/partition: only one thread ever owns a piece of data)

Example

```java
// Deadlock-prone: transfer(A,B) and transfer(B,A) running together
void transferBad(Account from, Account to, long amt) {
    synchronized (from) { synchronized (to) { from.debit(amt); to.credit(amt); } }
}

// Fixed with lock ordering: both threads lock the lower ID first
void transfer(Account from, Account to, long amt) {
    Account first = from.id().compareTo(to.id()) < 0 ? from : to;
    Account second = first == from ? to : from;
    synchronized (first) { synchronized (second) { from.debit(amt); to.credit(amt); } }
}

// Or tryLock with timeout + randomised backoff (avoids livelock)
boolean transferTry(Account from, Account to, long amt) throws InterruptedException {
    for (int attempt = 0; attempt < 5; attempt++) {
        if (from.lock.tryLock(50, TimeUnit.MILLISECONDS)) {
            try {
                if (to.lock.tryLock(50, TimeUnit.MILLISECONDS)) {
                    try { from.debit(amt); to.credit(amt); return true; } finally { to.lock.unlock(); }
                }
            } finally { from.lock.unlock(); }
        }
        Thread.sleep(ThreadLocalRandom.current().nextInt(5, 50));   // jitter breaks lockstep retries
    }
    return false;
}
```

Diagnosing in production: take a thread dump with `jcmd <pid> Thread.print` (or `jstack`). The JVM finds lock cycles itself — for both monitors (`synchronized`) and `ReentrantLock` — and prints them like this:

```
Found one Java-level deadlock:
"pool-1-thread-2": waiting to lock monitor 0x... (object 0x... Account), which is held by "pool-1-thread-1"
"pool-1-thread-1": waiting to lock monitor 0x... (object 0x... Account), which is held by "pool-1-thread-2"
```

When would I use it?

Use these rules in any design that takes several locks or resources:

- money or points transfers between two accounts,
- seat booking with several seats (lock the seats in sorted order),
- batch updates of many rows.

The same rules apply to **database** deadlocks too (12.7).

What can go wrong?

- **Hidden lock taking.** You call an outside method (a listener, a callback) while holding a lock. That method may need the same lock, or another lock in the opposite order. This is called an "alien method" deadlock.
- **Nested locks across classes** with no documented order.
- **Thread-pool starvation deadlock (4.3).** Tasks in a pool wait for other tasks that can't get a thread. It is not a lock cycle, but the system freezes in the same way.
- **Retrying without jitter.** If threads retry at exactly the same intervals, they collide again and again. A deadlock just becomes a livelock. Jitter means adding a small random delay.

Interview questions

Q1. What are the four conditions for deadlock? Mutual exclusion, hold and wait, no preemption, and circular wait. All four must be true at the same time for a deadlock to happen.

Q2. How do you prevent deadlocks? Break one of the conditions. The usual ways: take locks in one consistent global order, use tryLock with timeouts and backoff, take all resources at once, or use lock-free designs.

Q3. How do you detect a deadlock in a running JVM? Take a thread dump with `jcmd Thread.print` or `jstack`. The JVM reports Java-level deadlocks and names the threads and locks involved. To check from code, use JMX: `ThreadMXBean.findDeadlockedThreads()`.

Q4. Deadlock vs livelock vs starvation? Deadlock: threads are blocked forever, each waiting on the other. Livelock: threads are active and busy, but make no progress. Starvation: one thread is always denied the resource while the others keep making progress.

4.14 Producer–consumer, BlockingQueue and classic problems

Why?

> **In simple words:** One side makes work and puts it in a queue; the other side takes work out and does it. The queue has a fixed size, so if workers fall behind, the makers are forced to slow down instead of filling up memory.

The producer–consumer pattern separates *creating* work from *doing* it. Some examples:

- an HTTP handler puts notifications in a queue, and worker threads send them;
- a Kafka listener hands records to processor threads.

A **bounded buffer** (a queue with a maximum size) sits in the middle. It does two jobs:

- it absorbs short bursts of work;
- it applies **back-pressure** — when consumers fall behind and the queue is full, producers are made to wait. Without this, the queue keeps growing until you run out of memory.

Think of a tiffin service with a shelf that holds 20 boxes. Cooks (producers) put boxes on the shelf. Delivery boys (consumers) pick them up. If the shelf is full, cooks wait. If it is empty, delivery boys wait.

Interviewers often ask you to build one from scratch. It tests whether you understand locks and conditions.

What is happening internally?

The JDK queues work like this:

- **`ArrayBlockingQueue`** = a circular array + one `ReentrantLock` + two `Condition`s (`notFull` and `notEmpty`). A `Condition` is a waiting room tied to a lock.
- `put` waits on `notFull` when the queue is full.
- `take` waits on `notEmpty` when the queue is empty.
- After each put or take, it signals the other side.
- **`LinkedBlockingQueue`** uses two separate locks, one for put and one for take. So producers and consumers don't block each other, which gives higher throughput. A size limit is optional.
- **`SynchronousQueue`** has no capacity at all. It is a direct hand-off from producer to consumer.
- **`PriorityBlockingQueue`** has no size limit and returns elements in priority order.
- **`DelayQueue`** releases each element only after its delay has expired.

The four styles of method on a BlockingQueue:

Throws	Returns special value	Blocks	Times out
`add` / `remove`	`offer` / `poll`	`put` / `take`	`offer(e, t)` / `poll(t)`

How does it work?

This is the bounded buffer you must be able to write in 15 minutes:

```
import java.util.concurrent.locks.*;

public class BoundedBlockingQueue<T> {
    private final Object[] items;
    private int head, tail, count;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();

    public BoundedBlockingQueue(int capacity) { items = new Object[capacity]; }

    public void put(T item) throws InterruptedException {
        lock.lockInterruptibly();
        try {
            while (count == items.length) notFull.await();     // while: re-check after wakeup
            items[tail] = item;
            tail = (tail + 1) % items.length;
```

```
            count++;
            notEmpty.signal();                          // wake one consumer
        } finally { lock.unlock(); }
    }

    @SuppressWarnings("unchecked")
    public T take() throws InterruptedException {
        lock.lockInterruptibly();
        try {
            while (count == 0) notEmpty.await();
            T item = (T) items[head];
            items[head] = null;                         // let GC reclaim it
            head = (head + 1) % items.length;
            count--;
            notFull.signal();                           // wake one producer
            return item;
        } finally { lock.unlock(); }
    }
}
```

Why two conditions instead of one shared wait set? Because producers only wake consumers, and consumers only wake producers. No thread gets woken up just to find it still can't proceed. So there are no wasted wakeups.

Example

```
// Production version: BlockingQueue + poison pill shutdown
record Job(int id) {}
final Job POISON = new Job(-1);
BlockingQueue<Job> q = new ArrayBlockingQueue<>(100);
int consumers = 4;

Runnable producer = () -> {
    try {
        for (int i = 0; i < 1_000; i++) q.put(new Job(i));    // blocks when full → back-pressure
        for (int i = 0; i < consumers; i++) q.put(POISON);    // one pill per consumer
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
};
Runnable consumer = () -> {
    try {
        for (Job j = q.take(); j != POISON; j = q.take()) process(j);
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
};

// Classic: two threads print 1..N alternately (odd/even)
class OddEven {
    private int n = 1; private final int max;
    OddEven(int max) { this.max = max; }
    synchronized void print(boolean odd) throws InterruptedException {
        while (n <= max) {
            while (n <= max && (n % 2 == 1) != odd) wait();
            if (n <= max) System.out.println(Thread.currentThread().getName() + " " + n++);
            notifyAll();
        }
    }
}
```

How the poison pill works: the producer puts one special "stop" job (the poison pill) per consumer at the end. When a consumer takes the pill, it exits its loop.

Other classic problems to practise (LeetCode's free concurrency set):

- Print in Order,
- FooBar Alternately,
- Zero Even Odd,

- Building H_2O,
- Dining Philosophers (fix it with lock ordering, or with a semaphore of N−1 seats),
- a thread-safe rate limiter (10.5).

When would I use it?

- In-process work queues: notification dispatch, audit log writers, async enrichment.
- Batching: take up to N items at once with `drainTo`.
- Smoothing out producers that send work in bursts.

Across separate processes, the same idea becomes a message broker such as Kafka or RabbitMQ (11.12).

What can go wrong?

- **Unbounded queues.** `new LinkedBlockingQueue<>()` with no size limit removes back-pressure. The queue grows until you hit OOM (OutOfMemoryError).
- **Losing in-memory work on crash or deploy.** An in-process queue is not durable — if the app dies, the queued work is gone. For work that must survive, use a broker or an outbox table.
- **Shutdown.** Consumers blocked in `take()` never exit on their own. You must interrupt them or send poison pills.
- **Exceptions in consumers.** An uncaught exception kills the worker loop silently. Catch errors per item, log them, and send the failed item to a retry or dead-letter queue (a queue for items that keep failing).

Interview questions

Q1. Implement a bounded blocking queue. Use a circular array, one ReentrantLock and two Conditions (`notFull` and `notEmpty`). Wrap each `await` in a `while` loop to re-check after waking up. After each put, `signal` notEmpty; after each take, `signal` notFull (see the code above).

Q2. Why use two Conditions instead of wait/notifyAll? Producers wait on notFull and consumers wait on notEmpty. So each signal wakes only the right kind of thread, and no time is wasted waking threads that can't proceed.

Q3. How do you shut down consumers cleanly? Send poison-pill messages, one per consumer. Or interrupt the threads, and have them exit when they catch `InterruptedException`.

Q4. ArrayBlockingQueue vs LinkedBlockingQueue? ArrayBlockingQueue is always bounded, uses one lock, and allocates its array up front, so memory use is predictable. LinkedBlockingQueue is optionally bounded and uses separate put and take locks for higher throughput, but it allocates a new node for every element.

4.15 ThreadLocal

Why?

> **In simple words:** A `ThreadLocal` is a private pocket on each thread. Each thread puts its own value in the pocket and can read it from anywhere in its code, without passing it through every method.

Some data belongs to one request or one thread. For example:

- the current tenant ID,

- the logged-in user,
- a correlation ID for logs (one ID that ties together all log lines of one request),
- the current transaction or DB connection,
- a formatter object that is not thread-safe.

Passing these through every method signature makes the code noisy. `ThreadLocal` gives each thread its own copy, and any code running on that thread can reach it.

What is happening internally?

Here is how it is stored:

1. Every `Thread` object has a field called `threadLocals`.
2. That field is a `ThreadLocalMap` — an open-addressing hash table (on a collision it uses the next free slot, not a chain).
3. The **keys are weak references to the `ThreadLocal` object**. A weak reference does not stop GC from collecting the object.
4. The **values are strong references** to your data.
5. `tl.get()` really does `Thread.currentThread().threadLocals.get(tl)`.

The important consequence: the value lives as long as the **thread** lives. Server threads are pooled and live forever. So values stay alive from one request to the next, unless you remove them.

```
Thread "http-nio-8080-exec-7"
  └── threadLocals: { [weak] TENANT → "tenant-42", [weak] MDC → {requestId=...} }
       (pooled thread reused by the NEXT request → stale tenant-42 is still there unless removed)
```

Spring uses ThreadLocals a lot:

- `SecurityContextHolder` — the current user;
- `TransactionSynchronizationManager` — this is how `@Transactional` ties a DB connection to the current thread;
- `RequestContextHolder` — the current HTTP request;
- SLF4J's `MDC` — extra fields added to log lines.

How does it work?

The pattern has three steps:

1. Set the value at the start of a unit of work (for example, at the start of a request).
2. Read it anywhere on that thread.
3. **Always remove it in `finally`.**

Example

```java
public final class TenantContext {
    private static final ThreadLocal<String> CURRENT = new ThreadLocal<>();
    public static void set(String tenantId) { CURRENT.set(tenantId); }
    public static String current() {
        String t = CURRENT.get();
        if (t == null) throw new IllegalStateException("no tenant bound to this thread");
        return t;
    }
    public static void clear() { CURRENT.remove(); }
}
```

```
// Servlet filter: bind per request, ALWAYS clear (threads are pooled)
public class TenantFilter extends OncePerRequestFilter {
    @Override protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain
chain)
            throws ServletException, IOException {
        TenantContext.set(req.getHeader("X-Tenant-Id"));
        try { chain.doFilter(req, res); }
        finally { TenantContext.clear(); }
    }
}
```

When would I use it?

- Request-scoped context in thread-per-request servers: tenant, user, correlation ID.
- Framework plumbing: transactions, security.
- Per-thread caches of expensive objects that are not thread-safe. This is rarely needed now — for example, `DateTimeFormatter` is already thread-safe.

What can go wrong?

- **Leaking data between requests.** Threads are pooled. If you forget to clear, the next request on that thread sees the previous request's tenant or user. In a multi-tenant app this is a serious **security bug** — one customer could see another's data. Always call `remove()` in `finally`.
- **Memory leaks.** Big values stay alive as long as the long-lived pool threads. In app servers there is a worse case: if a ThreadLocal value holds classes from an app that has been redeployed, it keeps that app's whole ClassLoader alive. This is a Metaspace leak.
- **Async boundaries.** Values do not follow your work to other threads — executors, `CompletableFuture`, `@Async` or reactive code. You must pass them on explicitly, using task decorators or context-propagation libraries.
- **Virtual threads.** Millions of threads × ThreadLocal values means a lot of memory. Java's newer **ScopedValue** (a preview feature in Java 21) is meant to replace ThreadLocal here.
- **`InheritableThreadLocal` surprises.** It copies values into a child thread when the child is *created*. With pools, threads are created once and then reused, so the copied values are often stale.

Interview questions

Q1. How does ThreadLocal work internally? Each Thread holds a ThreadLocalMap. Its keys are weak references to the ThreadLocal objects, and its values are strong references to the data. `get` simply looks up the value in the current thread's map.

Q2. Why can ThreadLocal cause memory leaks or data leaks? Pooled threads live forever. So a value that is set and never removed stays there across tasks. This leaks memory, and can even leak one request's data into the next request.

Q3. How does Spring's `@Transactional` know which connection to use? `TransactionSynchronizationManager` binds the connection or EntityManager to the current thread using a ThreadLocal. The repositories then look it up there. That is also why a transaction does not follow your work onto other threads.

Q4. How do you pass context to async tasks? ThreadLocals don't move to other threads automatically. So capture the value before you submit the task, then set it at the start of the task and clear it at the end. A TaskDecorator or a context-propagation library can do this for you.

5. JVM Internals, Garbage Collection and Production Troubleshooting

At SDE2 level, JVM questions change. Juniors get "what is GC?". You will get something like: "p99 latency doubled and memory keeps climbing — walk me through what you'd do". (p99 latency means: 99% of requests finish faster than this time; it shows the slowest 1%.) This chapter gives you two things. First, a clear picture of how the JVM works inside. Second, the tools and steps to debug it in production.

5.1 Class loading

Why?

> **In simple words:** Java does not load all your classes at startup. It loads each class from disk only when your code first needs it, and it always asks the "senior" loaders first so nobody can sneak in a fake `String` class.

Java loads classes **lazily, at runtime**. "Lazily" means only when they are first needed. It does not link everything into one big binary at build time.

This gives you useful powers:

- **Plugins** can be added without rebuilding the app.
- **Frameworks like Spring** can scan the classpath and discover your classes on their own.
- **Dynamic proxies** (classes generated in memory while the app runs) become possible.
- **Hot reload** (swapping code without a full restart) becomes possible.
- **Startup memory stays small.** You only pay for the classes you actually use.

But this flexibility needs safety. Nobody should be able to replace `java.lang.String` with their own version. The loading design below protects against that.

What is happening internally?

A *class loader* is the part of the JVM that finds a class's bytes and turns them into a usable class. There are three built-in loaders. They are arranged like a chain of seniors and juniors. This arrangement is called the **parent-delegation** hierarchy:

```
Bootstrap loader (native)     → java.base: java.lang.*, java.util.* …
   ▲ parent
Platform loader               → other Java SE / JDK modules (java.sql, …)
   ▲ parent
Application (system) loader   → your classpath / module path: your code + dependency JARs
   ▲ parent
Custom loaders                → app servers, plugin systems, Spring Boot's LaunchedURLClassLoader (nested
JARs)
```

Delegation works like this. When a loader is asked for a class, it first asks its parent. Only if the parent cannot find the class does it load the class itself. Because of this rule, core classes like `String` always come from the bootstrap loader. Your fake `String` is never even looked at.

Think of a government office. A clerk gets a file. He first sends it up to his senior. Only if the senior says "not mine" does the clerk handle it himself.

Every class goes through three stages:

1. **Loading**: The JVM reads the class bytes. They can come from a JAR, a file, or be generated in memory. It then creates a `Class` object in Metaspace (the memory area for class information, see 5.2).
2. **Linking**: This has three small steps. - *Verification*: check that the bytecode is well-formed and type-safe, so it cannot corrupt memory. - *Preparation*: create the static fields and fill them with default values (0, null, false). - *Resolution*: replace symbolic names (like "the method `foo` in class `Bar`") with direct memory references. This is often done lazily, only when first used.
3. **Initialisation**: The JVM runs static initialiser blocks and static field assignments. This happens: - exactly once; - thread-safely (the JVM holds an init lock, so two threads can't both run it); - on the first **active use** of the class. Active use means: creating a new instance, calling a static method, reading a static field, using reflection on it, or initialising a subclass.

One more key point. The JVM identifies a class at runtime by two things together: **(fully qualified name, defining class loader)**. So if two different loaders load the same class, the JVM sees two *different* types. They are not interchangeable.

How does it work?

Here is what happens the first time your code runs `new OrderService()`:

1. The application loader is asked for `OrderService`.
2. It delegates up to the platform loader.
3. The platform loader delegates up to the bootstrap loader.
4. The bootstrap loader does not find it. The platform loader does not find it either.
5. The request comes back down to the application loader.
6. The application loader finds `OrderService.class` in your JAR.
7. The class is loaded, then linked, then its statics are initialised.
8. Finally, the object itself is allocated.

Example

```
public class LoadDemo {
    static class Lazy { static { System.out.println("Lazy initialised"); } static final int CONST = 42;
static int value = 7; }
    public static void main(String[] args) throws Exception {
        System.out.println(Lazy.CONST);        // 42 — compile-time constant is INLINED: does NOT initialise
Lazy
        System.out.println(Lazy.value);        // prints "Lazy initialised" then 7 — first active use
        System.out.println(String.class.getClassLoader());                // null (bootstrap)
        System.out.println(LoadDemo.class.getClassLoader());              // app class loader
    }
}
// java -verbose:class LoadDemo   → watch every class being loaded and from where
```

Note the first line of output. `Lazy.CONST` is a compile-time constant. The compiler copies the value 42 directly into the calling code. So reading it never touches the `Lazy` class, and `Lazy` is not initialised. Reading `Lazy.value` is a real active use, so initialisation runs first.

When would I use it?

- Diagnosing `ClassNotFoundException` and `NoClassDefFoundError`.

- Fixing dependency conflicts. If two versions of a library are on the classpath, the first one found wins.
- Understanding Spring Boot fat JARs (one JAR that contains other JARs inside it).
- Explaining when static initialisation runs and why it is thread-safe. The holder-idiom singleton depends on this (4.6).

What can go wrong?

- `ClassNotFoundException`: You asked for a class by its name (for example with `Class.forName`, or a framework using reflection). It is not on the classpath.
- `NoClassDefFoundError`: The class was there at compile time but is missing at runtime. *Or* — a very common case — its static initialiser already failed earlier. Search the logs for the first `ExceptionInInitializerError`; that is the real cause.
- **Dependency hell**: A `NoSuchMethodError` at runtime usually means a different version of a library won on the classpath. That version does not have the method your code expects. Check with `mvn dependency:tree`, and use the Spring Boot BOM (bill of materials — a managed list of compatible versions).
- **ClassCastException between "identical" classes**: The same class was loaded by two different loaders. This happens in plugin systems and app servers. To the JVM they are two different types, so casting fails.
- **ClassLoader leaks**: In app servers, you redeploy the app. But something still holds a reference to an old class — for example a ThreadLocal, a static registry, or a JDBC driver. That one reference keeps the whole old ClassLoader alive, along with every class it loaded. After many redeploys, Metaspace keeps growing until you get `OutOfMemoryError: Metaspace`.

Interview questions

Q1. Explain parent delegation. When a class loader is asked for a class, it first passes the request to its parent. It loads the class itself only if the parent cannot find it. Because of this, core classes always come from the bootstrap loader, so nobody can spoof them and there are no duplicate copies.

Q2. ClassNotFoundException vs NoClassDefFoundError? CNFE: a class that was requested dynamically by name (for example `Class.forName`) is not found. NCDFE: a class that was available at compile time cannot be loaded at runtime — either it is missing, or its static initialisation failed earlier.

Q3. When is a class initialised? On its first active use: creating an instance, calling a static method, reading a static field that is not a compile-time constant, using reflection, or initialising a subclass. Reading a compile-time constant does not trigger it, because the value is copied into the caller at compile time.

5.2 Runtime memory areas

Why?

> **In simple words:** The JVM's memory is not one big bucket. It is split into several areas, each storing a different kind of data and each with its own "full" error — and the heap is only one of them.

You need this map to do three practical things:

- size containers correctly,

- read GC logs, and
- understand what an `OutOfMemoryError` message is telling you.

For each kind of data, you should know which memory area it lives in. You should also know what happens when that area fills up.

What is happening internally?

```
┌──────────────────────── JVM process memory ────────────────────────┐
│ HEAP (-Xmx)          objects & arrays; shared; GC-managed          │
│   Young gen: Eden | S0 | S1      Old gen (tenured)                 │
├────────────────────────────────────────────────────────────────────┤
│ METASPACE (native)   class metadata, method bytecode, constant pools  │
│ CODE CACHE (native)  JIT-compiled machine code                     │
│ THREAD STACKS        one per platform thread (-Xss, ~512KB–1MB)    │
│ DIRECT BUFFERS       ByteBuffer.allocateDirect, Netty, NIO         │
│ GC / JIT internals, symbol tables, native libraries                │
└────────────────────────────────────────────────────────────────────┘
Per thread: JVM stack (frames: locals + operand stack), PC register, native method stack
```

Quick meaning of each area:

- **Heap**: where all your objects and arrays live. All threads share it. The garbage collector manages it. `-Xmx` sets its maximum size.
- The **young generation** (Eden plus two survivor spaces, S0 and S1) holds new objects.
- The **old generation** (also called *tenured*) holds objects that have lived a long time.
- **Metaspace**: information about classes — their structure, method bytecode and constant pools. It lives in *native memory*, meaning memory outside the heap that the JVM gets directly from the operating system.
- **Code cache**: the machine code produced by the JIT compiler (5.10).
- **Thread stacks**: one per platform thread. Each holds the method calls currently running in that thread. Size is set by `-Xss`, usually about 512 KB–1 MB.
- **Direct buffers**: memory outside the heap used for fast I/O, for example by `ByteBuffer.allocateDirect`, Netty and NIO.
- **Other internals**: the GC's and JIT's own data, symbol tables and native libraries.

Each thread also has its own small private areas: its JVM stack (made of *frames* — one frame per method call, holding local variables and the operand stack), a PC register (which instruction is next), and a native method stack.

Area	Contains	Error when exhausted
Heap	All objects and arrays	`OutOfMemoryError: Java heap space` / `GC overhead limit exceeded`
Metaspace	Class metadata (replaced PermGen in Java 8; grows on its own as needed)	`OutOfMemoryError: Metaspace`
Thread stack	One frame per method call	`StackOverflowError`
Native memory for threads	The OS's own structures for each thread	`OutOfMemoryError: unable to create native thread`
Direct memory	Buffers outside the heap	`OutOfMemoryError: Direct buffer memory`

Area	Contains	Error when exhausted
Code cache	JIT-compiled code	Warning "CodeCache is full"; the JIT stops compiling → sudden slowdown

Why PermGen was removed: Before Java 8, class metadata and interned strings lived in *PermGen*. PermGen was a fixed-size part of the heap. Apps that loaded many classes kept running out of it and failed with `PermGen space` errors. Examples are Groovy apps, app servers and apps with many proxies.

Java 8 fixed this in two ways:

1. Class metadata moved to **Metaspace**. It lives in native memory and grows as needed. You can still put a cap on it with `-XX:MaxMetaspaceSize`.
2. Interned strings moved to the normal heap.

How does it work?

Here is the key formula:

Total container memory ≈ heap + metaspace + code cache + (threads × stack) + direct buffers + GC overhead.

`-Xmx` limits only the first item, the heap. Everything else comes on top of it. So say you set `-Xmx` equal to the container's memory limit. The total process will go over the limit. Kubernetes (the Linux kernel, really) then kills the pod — this is called being *OOM-killed*. Java itself never reports an `OutOfMemoryError`, because the heap was never full.

Think of a 2 BHK flat. `-Xmx` is the size of the hall only. If you rent a flat exactly the size of the hall, the kitchen and bedrooms don't fit.

Example

```
Container limit 2 GiB:
  -XX:MaxRAMPercentage=70     → heap ≈ 1.4 GiB
  metaspace ~150 MB, code cache ~50 MB, 200 threads × 1 MB = 200 MB, direct buffers ~64 MB, GC structures ~50
MB
  → ~1.9 GiB: fits. With -Xmx2g the process would exceed the limit and be killed (exit code 137, OOMKilled).
```

You can check the real numbers on a running JVM:

```
jcmd <pid> VM.native_memory summary     # requires -XX:NativeMemoryTracking=summary
jcmd <pid> GC.heap_info
```

When would I use it?

- Sizing pods and choosing JVM flags.
- Reading an OOM message and knowing which area ran out.
- Explaining in an incident why "the JVM uses more memory than -Xmx".

What can go wrong?

- **Pods OOM-killed by the kernel with no Java error.** This happens when you forget non-heap memory in your sizing.
- **Thread-per-request with thousands of threads.** Each thread needs native memory for its stack and OS structures. Native memory runs out.

- **Deep recursion.** Each call adds a frame to the stack until you hit `StackOverflowError`.
- **Metaspace growth from classes generated at runtime.** Sources are proxies, lambdas in dynamically generated code, and redeploy leaks (5.1).

Interview questions

Q1. What are the JVM memory areas? The heap (young generation with Eden and two survivor spaces, plus the old generation), Metaspace, the code cache, per-thread stacks and PC registers, native method stacks, and direct memory.

Q2. What replaced PermGen and why? Metaspace, in Java 8. It lives in native memory and grows automatically, so the old fixed-size PermGen out-of-memory errors went away. Interned strings moved to the heap.

Q3. Why does a Java process use more memory than -Xmx? `-Xmx` limits only the heap. Metaspace, thread stacks, the code cache, direct buffers and GC data structures all come on top of it.

5.3 Object allocation and the generational heap

Why?

> **In simple words:** Most objects are like paper tea cups — used once and thrown away within seconds. A few are like steel utensils that stay for years. So the JVM keeps the new objects in a separate small area and cleans that area very often and very cheaply.

In a server, most objects die almost immediately. Examples: request DTOs, iterators, temporary strings. A few objects live for the whole run of the app. Examples: caches, Spring beans, connection pools.

This pattern has a name: the **weak generational hypothesis** ("most objects die young"). Because it is true, the JVM can:

- collect the young area — which is mostly garbage — cheaply and often, and
- collect the old area only rarely.

What is happening internally?

1. Allocation is just a pointer bump. Each thread gets a **TLAB** (Thread-Local Allocation Buffer). A TLAB is a private chunk of Eden that only that thread uses. To create an object, the thread just moves a pointer forward inside its TLAB. No locking is needed, because no other thread uses that chunk. It typically takes a few nanoseconds — faster than `malloc` in C.

2. New objects go to Eden. When Eden fills up, a **minor GC** (also called a young GC) runs:

1. Live objects in Eden and in the "from" survivor space are **copied** into the "to" survivor space.
2. Each copied object's **age** goes up by one.
3. Everything left behind in Eden is garbage. The JVM frees all of it at once just by resetting the pointer.

So the cost depends only on the number of *live* objects, not on the garbage. Garbage is free to remove.

3. Survivors get promoted. An object that survives enough minor GCs is **promoted** (moved) to the **old generation**. The limit is called the *tenuring threshold*. It is at most 15, and the JVM adjusts it on its own. Very large objects may go straight to old gen. In G1 they go to special *humongous regions* instead (5.5).

4. Old gen is cleaned rarely. The old generation is collected less often, by *major* or *mixed* collections. These are more expensive.

5. Card table / remembered sets. Sometimes an old object points to a young object. The JVM keeps a record of such links. So during a minor GC, it doesn't have to scan the whole old generation to find which young objects are still used. It just checks this record.

6. Escape analysis. Sometimes the JIT compiler can prove that an object never leaves the method that created it. Then it doesn't put the object on the heap at all. Instead, it turns the object's fields into plain local variables. This is called *scalar replacement*.

```
new Order() → TLAB in Eden
minor GC #1: Order still referenced → copied to S0 (age 1)
minor GC #2: → copied to S1 (age 2)  ... age N → promoted to Old
request DTOs: dead before the first minor GC → reclaimed for free
```

How does it work?

A healthy service in a steady state looks like this:

1. Eden fills every few seconds.
2. A minor GC runs and takes a few milliseconds.
3. Most objects are gone after it.
4. Old gen grows slowly with long-lived data, and gets cleaned occasionally.

Trouble starts with **medium-lived** objects. Examples: a response cache with a short TTL (time-to-live), or objects held while waiting on a slow downstream call. They live just long enough to be promoted to old gen. Then they die there. Now old gen fills with garbage quickly, so old-gen collections become frequent and costly. This problem is called **premature promotion**.

Example

```
GC log (G1), healthy:
[12.345s] GC(41) Pause Young (Normal) (G1 Evacuation Pause) 612M->98M(1024M) 7.812ms
             young collections frequent and short; heap after GC (98M) stable over time

GC log, leak or overload:
[901.2s] GC(880) Pause Young (Normal) 1010M->940M(1024M) 45ms
[901.9s] GC(881) Pause Full (G1 Compaction Pause) 1020M->985M(1024M) 2950ms
             heap after GC near the max and rising, full GCs taking seconds → leak or undersized heap
```

How to read a log line like `612M->98M(1024M) 7.812ms`: heap used before GC → heap used after GC (total heap size), and the pause time. The number to watch is the "after GC" value over time.

When would I use it?

- Reading GC logs.
- Explaining latency spikes.
- Tuning heap size.
- Deciding whether to reduce allocation in hot code paths, for example by reusing objects or using primitive arrays.

What can go wrong?

- **Young gen too small.** Minor GCs happen too often, and objects get promoted too early (premature promotion).

- **Huge object arrays.** In G1 these become humongous allocations, which fragment the regions.
- **Allocation storms.** Logging, boxing (turning `int` into `Integer`) or JSON parsing inside hot loops create huge numbers of short-lived objects.

Interview questions

Q1. Why is the heap divided into generations? Because most objects die young. Collecting a small young generation often is cheap, since only the few survivors are copied. Long-lived objects sit in the old generation, which is collected rarely.

Q2. How does an object move to old gen? By surviving a number of minor GCs — it ages each time it is copied between survivor spaces. It can also go there directly if it is too large, or if the survivor spaces overflow.

Q3. What is a TLAB? A Thread-Local Allocation Buffer: a private chunk of Eden for one thread. The thread allocates objects by just moving a pointer forward, with no synchronisation.

5.4 How garbage collection finds and frees garbage

Why?

> **In simple words:** The garbage collector starts from a few fixed starting points, follows every reference it can reach, and treats everything it could not reach as rubbish — while your app keeps running and changing things.

In C, you manage memory by hand with `malloc` and `free`. That causes three classic bugs:

- **use-after-free** — using memory that was already freed, which crashes or corrupts data;
- **double free** — freeing the same memory twice;
- **leaks** — forgetting to free memory.

Java frees memory automatically. But the collector has a hard job. It must decide *safely* which objects are dead. And it must do this while the application keeps changing references.

What is happening internally?

Reachability, not reference counting. An object is *live* if you can reach it by following references from **GC roots**. GC roots are the fixed starting points:

- local variables and operands on thread stacks,
- static fields,
- JNI references (references held by native code),
- live threads,
- objects used as synchronisation monitors (locks).

Everything that cannot be reached is garbage. This includes **cycles** — objects pointing to each other but not reachable from any root. Reference counting (counting how many references point to an object) cannot free such cycles, because each object in the cycle still has a count above zero. Reachability handles them correctly.

Core algorithms:

- **Mark**: start from the roots, walk all references, and mark every reachable object.

- **Sweep**: free the memory of unmarked objects in place. This leaves gaps, called *fragmentation*.
- **Compact**: slide live objects together and update all references to them. No fragmentation, but extra work.
- **Copy/evacuate**: copy live objects into an empty area, then treat the old area as empty. Cost depends on the amount of live data. Used for the young gen and for G1 regions.

Stop-the-world (STW) pause. Some GC phases need all application threads to be stopped. Threads can only stop at **safepoints**. A safepoint is a place where the JVM knows exactly what the thread's state is — for example a method return or the jump back to the start of a loop.

Modern collectors do most of their work **concurrently**, meaning while the application keeps running. To do this safely they use **barriers**. A barrier is a small piece of code the JIT inserts wherever your code reads or writes a reference. It lets the GC track changes made during concurrent marking.

Tri-colour marking. During marking, each object has one of three colours:

- **White**: not seen yet.
- **Grey**: seen, but its children (the objects it points to) are not scanned yet.
- **Black**: fully scanned.

During concurrent marking, the app could move a reference to a white object into a black object. The GC will never rescan the black object, so it would miss the white one and wrongly free it. Write barriers prevent this by telling the GC about such changes.

How does it work?

```
GC roots → A → B → C            D ⇄ E (cycle, unreachable)
mark: A, B, C marked            D, E unmarked → garbage, freed despite referencing each other
```

Here A, B and C are reachable from a root, so they are marked and kept. D and E point to each other, but nothing from the roots reaches them. So they stay unmarked and are freed, even though each one references the other.

Example

```
void handle(Request r) {
    Order o = new Order(r);       // reachable from this stack frame while handle() runs
    cache.put(o.id(), o);         // now ALSO reachable from a static cache → survives after return
}                                  // frame popped; if the cache weren't there, o would be garbage now
```

When would I use it?

- To reason about leaks. The question "which path from a GC root keeps this object alive?" is exactly what heap-dump tools answer.
- To reason about latency, because of STW pauses.
- To explain why calling `System.gc()` is pointless. It is only a hint to the JVM — and a harmful one, because it can trigger an expensive full GC.

What can go wrong?

- **Long STW pauses.** These come from a large live set (lots of live data to process) or from full GCs.
- **Time-to-safepoint delays.** A GC pause must wait until *every* thread reaches a safepoint. A long counted loop with no safepoint check inside can make everyone wait for that one thread.

- **Finalizers (deprecated).** A finalizer can bring an object back to life ("resurrect" it) and delays the freeing of its memory.

Interview questions

Q1. How does the JVM decide an object is garbage? If it cannot be reached from any GC root — thread stacks, static fields, JNI references, active threads — it is garbage. Unreachable cycles are collected too.

Q2. What is a stop-the-world pause and a safepoint? A stop-the-world pause is a phase where all application threads are halted so the GC can work safely. Threads stop at safepoints, which are points where the JVM knows the exact state of their stacks.

Q3. Mark-sweep vs mark-compact vs copying? Mark-sweep frees dead objects in place, which leaves fragmentation. Mark-compact moves live objects together, so there are no gaps. Copying evacuates live objects to a fresh area; its cost is proportional to the amount of live data.

5.5 The garbage collectors: Serial, Parallel, G1, ZGC

Why?

> **In simple words:** Different apps want different things from the GC — maximum work done, shortest pauses, or least memory used. No collector wins on all three, so Java gives you several to choose from.

Each type of application cares about something different:

- A **batch job** wants maximum **throughput** — the most total work done per hour. Pauses don't matter much.
- A trading or ordering **API** wants low **latency** — short pauses, so no request waits long.
- A **small container** wants a low **footprint** — little memory and CPU used by the GC itself.

No single collector is best at all three. So the JVM offers several.

What is happening internally?

Collector	Flag	How it works	Pauses	Best for
Serial	`-XX:+UseSerialGC`	One thread. Stops the app and does mark-copy for young gen and mark-compact for old gen	Long	Tiny heaps, 1-CPU containers
Parallel	`-XX:+UseParallelGC`	Same as Serial, but uses many threads	Long but efficient	Batch jobs, max throughput

Collector	Flag	How it works	Pauses	Best for
G1 (default since Java 9)	`-XX:+UseG1GC`	Splits the heap into ~2048 equal **regions** (1–32 MB each). Young collections evacuate the young regions. **Concurrent marking** finds old regions that are mostly garbage. **Mixed collections** then evacuate the regions with the most garbage first ("Garbage First"), staying within a **pause-time target** (`MaxGCPauseMillis` , default 200 ms). Huge objects get humongous regions.	Typically 10–200 ms	General-purpose services, 2–32+ GB heaps
ZGC	`-XX:+UseZGC` (+ `-XX:+ZGenerational` on 21; generational by default from 23)	Does almost everything while the app runs, **including compaction**. It uses coloured pointers (extra metadata bits inside each reference) and **load barriers** (checks on every reference read), so objects can be moved while the app runs. If the app reads an outdated reference, it is fixed on the spot	**< 1 ms**, no matter how big the heap is	Latency-sensitive APIs, very large heaps
Shenandoah	`-XX:+UseShenandoahGC`	Compacts while the app runs, using forwarding pointers (built by Red Hat)	< 10 ms	Similar to ZGC
CMS	removed in Java 14	Concurrent mark-sweep with no compaction → fragmentation		Legacy only

How does it work?

A G1 cycle, step by step:

1. Young pauses happen all the time, cleaning the young regions.
2. When heap usage crosses a threshold, concurrent marking starts and runs alongside the app. This threshold is called IHOP (Initiating Heap Occupancy Percent), about 45%.
3. Short *remark* and *cleanup* pauses finish the marking.
4. A series of **mixed** pauses follows. Each one evacuates the young regions *plus* the old regions with the most garbage.
5. The cycle repeats.

A **full GC** compacts the entire heap in one long pause. G1 does this only as a last resort, when evacuation can't keep up with the app. This is the multi-second pause you see in incidents.

ZGC's trade-off: It gives up some throughput and CPU — for the barrier checks and its concurrent GC threads. In return, pauses stay below a millisecond even with heaps of hundreds of GB.

Example

```
# Typical service on Java 21 in a 4 GiB pod
-XX:+UseG1GC -XX:MaxRAMPercentage=70 -XX:MaxGCPauseMillis=200
# Latency-critical API (p99 < 50 ms SLO) with large caches
-XX:+UseZGC -XX:+ZGenerational -Xmx8g
```

When would I use it?

- **Keep G1** unless measurements show GC pauses breaking your SLO (service level objective — your latency promise). Then try ZGC.
- Use **Parallel** for offline batch jobs that need maximum throughput.
- **Tune heap size and allocation rate first.** Only then touch collector flags.

What can go wrong?

- `MaxGCPauseMillis` **set too low.** G1 then does less work per pause. It falls behind the app and ends up triggering full GCs — the opposite of what you wanted.
- **Humongous allocations.** In G1, an object of at least half a region (huge arrays or Strings) gets its own humongous region. Many of these fragment the heap.
- **ZGC without enough headroom.** ZGC needs extra free heap. If the app allocates faster than the concurrent collector can free memory, application threads are forced to wait. This is called an *allocation stall*.

Interview questions

Q1. G1 vs ZGC? G1 is region-based. It evacuates mostly during stop-the-world pauses and aims for a pause target of tens of milliseconds, giving a good balance with throughput. ZGC compacts concurrently using load barriers. Its pauses stay under a millisecond regardless of heap size, but it costs some throughput and CPU.

Q2. Why is G1 called "Garbage First"? After concurrent marking, it evacuates first the old regions that contain the most garbage. This gets back the most space for each pause.

Q3. Which collector for a batch job vs a low-latency API? Parallel for batch throughput. G1 by default for services. ZGC when pause times threaten the latency SLO.

5.6 Strong, soft, weak and phantom references

Why?

> **In simple words:** Sometimes you want to point to an object but still let the GC throw it away. Special reference types let you say "keep this only while someone else needs it" or "tell me when it is gone".

Normally, if you hold a reference to an object, the object stays alive. But sometimes you want to refer to an object **without keeping it alive**. For example:

- a cache that should give memory back when memory is short;
- extra data attached to objects, which should disappear together with them;
- cleanup work that should run when an object is collected.

What is happening internally?

Type	Cleared by GC when	Typical use
Strong (normal)	Never, while it is reachable	Everything
`SoftReference`	Memory is running low (before an OOM)	Memory-sensitive caches (rare now, because behaviour is unpredictable)

Type	Cleared by GC when	Typical use
`WeakReference`	At the next GC, if only weak references point to it	`WeakHashMap` keys, listener registries, canonicalising maps (maps that keep one shared copy per value)
`PhantomReference`	After the object has been finalised; `get()` always returns null	Cleanup after the object is gone, via `ReferenceQueue` / `Cleaner` (for off-heap resources)

When the GC clears a reference, it can put it on a `ReferenceQueue`. Your code reads that queue to learn that the object is gone.

How does it work?

`WeakHashMap` holds its keys through weak references. Once nothing else points to a key, the GC clears it. The entry then disappears during the next operation on the map.

There is a classic gotcha. The *values* are held strongly. So if a value references its own key, the key stays reachable through the value. The entry is never removed.

Example

```
Map<Session, Metadata> meta = new WeakHashMap<>();   // metadata disappears when the Session is unreachable
Cleaner cleaner = Cleaner.create();
class OffHeapBuffer implements AutoCloseable {
    private final Cleaner.Cleanable cleanable;
    OffHeapBuffer(long address) { cleanable = cleaner.register(this, () -> freeNative(address)); }   //
safety net
    public void close() { cleanable.clean(); }                                                        //
preferred: explicit
}
```

The `Cleaner` here is only a safety net. The preferred way is to call `close()` yourself (for example with try-with-resources).

When would I use it?

Rarely in business code. You mostly need to recognise these in interviews and in library code. For caches, use Caffeine with size and time limits instead of soft references.

What can go wrong?

- **Soft-reference caches** cause GC thrashing (the GC runs again and again) and unpredictable latency.
- `WeakHashMap` **entries never cleared**, because their values reference their keys.
- **Relying on GC timing for cleanup.** You don't know when the GC will run. Always provide an explicit `close()`.

Interview questions

Q1. Weak vs soft references? A weak reference is cleared at the next GC once the object is reachable only through weak references. A soft reference is cleared only when memory is under pressure.

Q2. How does WeakHashMap work? It holds keys through weak references. When a key is no longer reachable from anywhere else, the GC clears it and the map removes that entry.

5.7 Memory leaks in a garbage-collected language

Why?

> **In simple words:** The GC only frees objects nobody points to. If your code keeps pointing to objects it will never use again — like a register where entries are added but never struck off — memory keeps filling up. That is a leak, even in Java.

"Java has a GC, so it can't leak" is a classic wrong answer.

A Java leak is an object that is still **reachable** but no longer **needed**. The GC cannot know you won't use it again. It only knows something still points to it.

How a leak shows up, in order:

1. The heap grows slowly.
2. GCs become longer and more frequent.
3. Latency spikes appear.
4. Finally, `OutOfMemoryError`.

What is happening internally?

Some long-lived root holds a reference graph that keeps growing. The root can be a static field, a singleton bean, a long-lived thread or a cache. On every GC, the collector marks all of it as live. It copies or keeps it, and can never free it.

The usual suspects:

1. **Unbounded caches and static collections**: for example, a `static Map<String, Result> CACHE` that only ever grows.
2. **Listeners and callbacks** that are registered but never removed.
3. **ThreadLocals in pooled threads** that are never removed. Pool threads live forever, so their ThreadLocal values do too (4.15).
4. **Inner classes and lambdas** that capture a large outer object and are stored long-term (1.17).
5. **Mutable map keys** whose hash changed after insertion. The map can't find the entry any more, so it can never be removed (1.12).
6. **Unclosed resources**: streams, result sets, HTTP responses. These also use up connection pools and file descriptors.
7. **Unbounded queues** in executors that keep growing under overload (4.3).
8. **ClassLoader leaks** on redeploy, which fill Metaspace (5.1).
9. **Session or per-tenant maps** with no expiry.

How does it work?

The signature in monitoring is the "sawtooth" heap graph. The heap rises as objects are created, then drops at each GC. Look at the **floor** — the heap level right after each GC.

- In a healthy app, the floor stays flat.
- In a leak, the floor rises steadily over hours or days.

Along with this, old-gen GCs become frequent, and the percentage of time spent in GC goes up.

```
heap used
 |      /\    /\     /\      /\
```

```
  |  /\ /  \  /  \   /  \    /  \     ← floor after each GC keeps rising = leak
  | /  V    \/    \_/    \__/
  └──────────────────────────────── time
```

Example

```
// Leak: request-scoped results cached forever, keyed by unique request IDs
@Component
class RuleResultCache {
    private static final Map<String, RuleResult> CACHE = new ConcurrentHashMap<>();
    RuleResult get(String requestId, Supplier<RuleResult> calc) { return CACHE.computeIfAbsent(requestId, k -
> calc.get()); }
}

// Fix: bounded, time-limited cache (Caffeine), keyed by something with bounded cardinality
Cache<String, RuleResult> cache = Caffeine.newBuilder()
        .maximumSize(50_000)
        .expireAfterWrite(Duration.ofMinutes(10))
        .recordStats()
        .build();
```

Why the first version leaks: every request has a new, unique ID. So every request adds a new entry, and nothing ever removes one. The fix limits both the size (50,000 entries) and the age (10 minutes) of entries.

When would I use it?

Every time you add a static or long-lived collection, ask two questions:

1. What limits its size?
2. What removes entries?

In incident reviews, use the investigation flow in 5.9.

What can go wrong?

- **Restarting pods "fixes" the symptom.** It hides the leak until the next traffic peak.
- **Increasing `-Xmx` only delays the OOM.** The leak is still there.

Interview questions

Q1. Can Java have memory leaks? Yes. A leak is an object that stays reachable but is no longer needed. Common causes: unbounded caches, static collections, listeners and ThreadLocals that are never removed, lambdas or inner classes holding outer references, and unclosed resources.

Q2. How do you recognise a leak from metrics? Heap usage right after GC rises steadily over time. GC frequency and duration grow. Eventually you get an OOM. A restart resets it, but only for a while.

5.8 Diagnosing high CPU, hangs and slow requests

Why?

> **In simple words:** When production is slow or stuck, you don't guess. You take snapshots of what every thread is doing, find the busy or stuck ones, and follow the evidence.

Production problems rarely tell you what is wrong. You see "CPU at 100%", "requests hanging" or "p99 doubled". To handle these, you need a repeatable approach driven by evidence.

Interviewers ask "walk me through debugging this" to check exactly that: do you have a method?

What is happening internally?

Two main tools:

- A **thread dump** is a snapshot of every thread: its state (running, blocked, waiting) and its stack trace (which methods it is inside right now).
- A **CPU profile** samples which methods are running, many times over a period, and adds up where the time goes.

Combine them with OS tools. Together they tell you *which* thread is busy and *what* it is doing.

How does it work?

High CPU:

1. Run `top -H -p <pid>`. This shows CPU use per thread. Note the thread IDs (TIDs) using the most CPU.
2. Convert the TID to hex: `printf '%x\n' 12345` → `3039`. Thread dumps show IDs in hex.
3. Run `jcmd <pid> Thread.print > dump.txt`. Search for `nid=0x3039`. That thread's stack shows the hot code.
4. Take 3 dumps 5–10 s apart. If you see the same stack every time, you have confirmed it.
5. Check the common culprits: - **GC threads** — if these are the busy ones, the real problem is memory; go to memory analysis (5.9) instead; - an infinite loop; - regex catastrophic backtracking (a badly written regex that tries millions of combinations); - huge JSON serialisation or deserialisation; - a `HashMap` infinite loop after unsafe use from many threads (old Java versions); - busy spin loops (a loop that keeps checking a condition without sleeping).
6. For a full picture, use **async-profiler** flame graphs, or **JFR** (`jcmd <pid> JFR.start duration=60s filename=cpu.jfr`) and open the recording in JDK Mission Control.

Hangs and slow requests:

1. Take thread dumps again. Look for these patterns: - Many threads `BLOCKED` and "waiting to lock" the same monitor → **lock contention** (threads queueing for one lock). - Threads `WAITING` in `HikariPool.getConnection` → the **DB connection pool is exhausted**. Causes are slow queries or leaked connections. - Threads `RUNNABLE` in `SocketInputStream.read` for minutes → a downstream call with **no read timeout**. The thread waits forever for a reply. - A section saying "Found one Java-level deadlock" → a **deadlock** (4.13).
2. Match what you see with APM traces and metrics: pool usage, GC pauses, downstream latency.

Think of a railway reservation hall where the queue has stopped moving. You don't guess — you walk up and see what each counter clerk is doing. A thread dump is that walk.

Example

```
"http-nio-8080-exec-41" #87 daemon prio=5 tid=0x... nid=0x3039 waiting on condition
   java.lang.Thread.State: TIMED_WAITING (parking)
      at jdk.internal.misc.Unsafe.park(Native Method)
      at com.zaxxer.hikari.util.ConcurrentBag.borrow(ConcurrentBag.java:...)
      at com.zaxxer.hikari.pool.HikariPool.getConnection(HikariPool.java:...)
      at ...LedgerRepository.balance(...)
 → 150 of 200 Tomcat threads look like this: the DB pool (size 20) is exhausted.
   Next step: which queries hold connections? (slow query log / pg_stat_activity) — often a missing index or
```

```
a
   transaction that calls an external API while holding the connection.
```

When would I use it?

- On-call incidents.
- Performance tuning.
- Telling an interview story with a concrete diagnosis. Your ~30 s → ~10 s API improvement can be told this way: measure, find the bottleneck, fix it, verify the result.

What can go wrong?

- **Trusting a single dump.** One snapshot can mislead. Take several.
- **Heavy instrumenting profilers in production.** They add code to every method and change how the app behaves. Prefer sampling profilers such as JFR or async-profiler.
- **Fixing symptoms instead of causes.** Bigger pools or more pods without finding the real cause just move the problem.

Interview questions

Q1. CPU is at 100% on a Java service. What do you do? Find the hot threads with `top -H`. Convert their TIDs to hex and match them to the `nid` in thread dumps taken a few seconds apart. Read their stacks. Check whether the busy threads are GC threads, which would point to a memory problem. Profile with JFR or async-profiler. Then fix the hot code path.

Q2. Requests are hanging. How do you investigate? Take thread dumps to see what the threads are waiting on — locks, connection pools, sockets without timeouts, or deadlocks. Then correlate that with traces and pool metrics.

5.9 Diagnosing memory problems: GC logs, heap dumps, JFR

Why?

> **In simple words:** Memory problems are the most common JVM incidents. You must switch on evidence collection in advance, because when the process dies, the evidence dies with it — and then you need a clear method to read that evidence.

Leaks, GC thrashing and OOMs are the most common JVM incidents. To handle them you need two things:

1. The right artefacts (logs, dumps, recordings), captured *before* the process dies.
2. A method for reading them.

What is happening internally?

The four sources of evidence:

- **GC logs** record every collection: its type, its cause, heap size before and after, and the pause time.
- **Class histogram**: for each class, how many live objects exist and how many bytes they use. It is cheap and needs no full dump.
- **Heap dump** (`.hprof` file): a copy of every object and every reference in the heap. Analysis tools compute two key things from it:

- The **dominator tree**: for each object, how much memory would be freed if that object disappeared. This is its *retained size*.
- The **path to GC roots**: the chain of references that keeps an object alive.
- **JFR** (Java Flight Recorder): a recording that runs all the time with low overhead (~1%). It captures allocations, GC, locks, I/O and CPU samples.

How does it work?

```
# Always-on in production
-Xlog:gc*:file=/logs/gc.log:time,uptime,level,tags:filecount=5,filesize=20m
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps

# During an incident
jcmd <pid> GC.class_histogram | head -20          # take twice, 10 minutes apart: which classes grow?
jcmd <pid> GC.heap_dump /dumps/heap.hprof         # pauses the app; heap-sized file — mind disk and PII
jcmd <pid> JFR.start duration=120s filename=/dumps/rec.jfr settings=profile
```

Note: a heap dump pauses the app and writes a file as big as the heap. Check disk space first, and remember it contains user data (PII — personally identifiable information).

Investigation flow:

1. **Which OOM is it?** Heap, Metaspace, direct buffers or native threads. Each type has a different cause (5.2).
2. **Read the GC logs.** Is heap-after-GC rising over time? That is a leak. Or is it just high under load? That means the heap is too small, or there was a traffic spike.
3. **Take two class histograms some time apart.** Which classes keep growing?
4. **Open the heap dump in Eclipse MAT.** Run the "Leak Suspects" report. Then open the dominator tree. Follow the path to GC roots of the biggest retainers. Find the code that owns that root.
5. **Fix it, then verify** using the same metrics.

Example

```
MAT Leak Suspects:
  One instance of "java.util.concurrent.ConcurrentHashMap" loaded by "app" occupies 1,284,331,112 (82.4%)
bytes.
  Path to GC root: RuleResultCache.CACHE (static) → ConcurrentHashMap → Node[] → RuleResult …
  → The static request-keyed cache from 5.7. Fix: bounded Caffeine cache. Heap after GC drops from 85% to
40%.
```

> **Say this:** "p99 latency of the reward API jumped from 300 ms to 4 s every few minutes. GC logs showed full GCs of about 3 s, and old gen never dropped below 85%. I took two class histograms ten minutes apart, and `RuleResult` objects were growing. The heap dump's dominator tree pointed to a static ConcurrentHashMap keyed by request ID, with no eviction. We replaced it with a Caffeine cache limited by size and TTL. Old gen settled around 40%, p99 went back under 300 ms, and we added an alert for old-gen-after-GC above 70%." Build your own version of this from a real incident you handled.

When would I use it?

- For any OOM or memory alert.
- Proactively before launches: run a load test and check the heap.

What can go wrong?

- **Heap dumps contain customer data (PII).** Handle and store them securely.
- **Dumping a big heap hurts.** A 16 GB heap dump pauses the service for a long time and needs a lot of disk. So first remove that instance from the load balancer, then take the dump.
- **No `HeapDumpOnOutOfMemoryError` flag.** Then the evidence dies with the process.

Interview questions

Q1. How do you investigate an OutOfMemoryError? First identify the OOM type. Check the GC logs for the heap-after-GC trend. Compare class histograms taken some time apart. Analyse the heap dump's dominator tree and the paths to GC roots. Fix the reference that is holding the memory. Then verify with metrics.

Q2. What is retained size? The memory that would be freed if an object were collected. It includes the object itself and everything that is reachable only through it.

Q3. What is JFR? Java Flight Recorder. It is a built-in event recorder with low overhead. It captures CPU samples, allocations, GC, locks and I/O, and you analyse the recording in JDK Mission Control. It is safe to run in production.

5.10 JIT compilation in depth

Why?

> **In simple words:** Java first runs your code slowly, by interpreting it, while watching what it actually does. Then it compiles only the busy parts into fast machine code, tuned to how your app really behaves.

There are two simple options, and both have problems:

- **Interpreting bytecode** (running it instruction by instruction) is slow — 10–50× slower than native machine code.
- **Compiling everything ahead of time** loses runtime information. For example: which branches are actually taken, and which subclass actually shows up at a call.

The JIT (just-in-time compiler) takes a middle path:

- It compiles **only hot code** (code that runs a lot).
- It uses **live profiling data** collected while the app runs.
- So it can make optimistic optimisations that a normal ahead-of-time compiler cannot safely make.

What is happening internally?

Tiered compilation. Code moves up through levels:

- Level 0: the interpreter.
- Level 3: the C1 compiler, with profiling. It compiles quickly and collects statistics.
- Level 4: the C2 compiler, with full optimisation. It compiles slowly but produces very fast code.
- (Levels 1–2 are C1 variants.)

Key C2 optimisations:

- **Inlining**: copying the body of a small, hot method directly into its caller. This removes the cost of the call. More importantly, it enables every other optimisation. It is the most important one.
- **Devirtualisation**: An interface or virtual call normally needs a lookup to find the right implementation. If profiling sees only one implementation at a call site (*monomorphic*), the JIT calls it directly and can inline it. If it sees two types (*bimorphic*), it adds a quick type check. With more types (*megamorphic*), it falls back to a normal vtable call (a table lookup).
- **Escape analysis**: If an object never leaves the method, it is scalar-replaced — its fields become local variables and no allocation happens. Any locks on it are removed too (*lock elision*).
- **Loop optimisations**: unrolling (doing several iterations per loop pass), range-check elimination (removing repeated array-bounds checks), and vectorisation (SIMD — one CPU instruction working on several values at once).
- **Dead-code elimination** (removing code that can never run) and **constant folding** (calculating constant expressions at compile time).

Deoptimisation. The JIT's optimisations rely on assumptions. Sometimes an assumption becomes false — for example, a new subclass gets loaded, or a branch that was never taken is suddenly hit. Then:

1. The JVM throws away that compiled code at a safepoint.
2. Execution goes back to the interpreter.
3. Later, the JIT may compile it again with the new information.

On-stack replacement (OSR). Suppose a method is still interpreted but is stuck in a very long loop. The JVM can switch that running loop to compiled code in the middle, without waiting for the method to be called again.

How does it work?

```
call count / loop iterations cross thresholds → C1 compiles with profiling counters
→ profile shows hot path, receiver types, branch frequencies → C2 compiles optimised code for those
→ assumptions guarded by cheap checks; guard fails → deoptimise → interpreter → recompile
```

In steps:

1. Call counts or loop iterations cross a threshold. C1 compiles the method, with profiling counters.
2. The profile shows the hot path, the receiver types and how often each branch is taken. C2 compiles optimised code for exactly those cases.
3. The assumptions are protected by cheap checks called *guards*.
4. If a guard fails → deoptimise → back to the interpreter → recompile later.

Example

```
interface Fee { long of(long amt); }
final class Flat implements Fee { public long of(long a) { return 10; } }
long total(List<Long> amounts, Fee fee) {
    long s = 0;
    for (long a : amounts) s += fee.of(a);   // only Flat seen → devirtualised + inlined → s += 10
    return s;
}
// If later a PercentFee is passed here, the guard fails → deoptimisation → recompiled as a bimorphic call.
```

When would I use it?

- Explaining **warm-up**, and how to design benchmarks correctly.
- Explaining why hot paths should be small, monomorphic methods.

- Explaining that `final` and sealed classes help readability — the JIT does not need them for speed, because profiling already tells it the real types.
- Discussing startup options: CDS archives (pre-loaded class data), CRaC (restoring a saved, warmed-up JVM), and GraalVM native image for fast-starting CLIs and serverless functions.

What can go wrong?

- **Benchmarks without warm-up.** You end up measuring the interpreter, not the real performance.
- **Megamorphic call sites in hot loops.** For example, many different lambda implementations called through one interface. The JIT can't inline them.
- **A full code cache.** The JIT stops compiling, and new hot code stays slow.
- **Deoptimisation storms.** Repeated deoptimise-and-recompile cycles cause latency jitter.

Interview questions

Q1. What does the JIT do? It profiles running code and compiles hot methods into optimised native code, first with C1 and then C2. It uses inlining, devirtualisation, escape analysis and loop optimisations. If its assumptions break, it deoptimises.

Q2. Why do Java services need warm-up? Code starts in the interpreter. The JIT needs profiling data and time to compile the hot paths. Caches and connection pools also need time to fill up.

Q3. What is escape analysis? A JIT analysis that proves an object does not escape its method or thread. This allows the JIT to skip the allocation entirely (scalar replacement) and to remove locks on it (lock elision).

5.11 The JVM in containers and the flags that matter

Why?

> **In simple words:** Inside a container, the JVM must size itself from the container's limits, not the whole machine's. A handful of flags control this, and getting them wrong causes OOM kills and slow, throttled services.

Your services run in Kubernetes on Azure. Imagine a JVM that ignores container limits. It looks at the *host machine's* RAM and CPUs instead. It then sizes its heap and thread pools for that much bigger machine. The results are OOM kills and CPU throttling (the container being slowed down for using more CPU than allowed).

Knowing a handful of flags lets you configure services sensibly and explain incidents.

What is happening internally?

Since Java 10 (also backported to 8u191), the JVM reads **cgroup** limits. Cgroups are the Linux feature that enforces a container's CPU and memory limits. This means:

- `availableProcessors()` returns the container's CPU limit, rounded up.
- The default maximum heap is a percentage of the container's memory. The default is 25%, which is too low for most services.
- The CPU count also decides the number of GC threads, the size of the common ForkJoinPool, and the size of many library thread pools.

How does it work?

```
-XX:MaxRAMPercentage=70.0            # heap = 70% of the container limit (leave room for non-heap, 5.2)
-XX:InitialRAMPercentage=70.0        # equal initial/max avoids resize pauses (or -Xms = -Xmx)
-XX:+UseG1GC  (default)  | -XX:+UseZGC -XX:+ZGenerational (Java 21)
-XX:MaxGCPauseMillis=200
-Xss512k                             # smaller thread stacks if you run many platform threads
-XX:MaxMetaspaceSize=256m            # cap runaway class generation
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps
-XX:+ExitOnOutOfMemoryError          # die fast and let Kubernetes restart instead of limping
-Xlog:gc*:file=/logs/gc.log:time,uptime:filecount=5,filesize=20m
-XX:ActiveProcessorCount=2           # override if the CPU limit is fractional and the JVM misjudges it
```

Example

Take a Kubernetes pod with `limits.memory: 2Gi` and `limits.cpu: 2`:

- `MaxRAMPercentage=70` → about 1.4 GiB of heap.
- G1 runs with 2 parallel GC threads.
- Tomcat has max 200 threads, which reserve about 200 MB for stacks.
- A readiness probe points at `/actuator/health/readiness`. So traffic arrives only after the app has warmed up.

When would I use it?

- Writing Dockerfiles and Helm values.
- Sizing pods.
- Investigating OOMKilled restarts (exit code 137) and CPU throttling.

What can go wrong?

- **`-Xmx` equal to the container limit.** Non-heap memory pushes the process over the limit → OOMKilled.
- **A very low CPU limit** (e.g., 500m, meaning half a CPU). The JVM thinks it has only 1 CPU. So it picks Serial GC and tiny thread pools.
- **Requests and limits set far apart.** This causes noisy, unpredictable throttling.
- **No volume for heap dumps.** The dump has nowhere to go, so the evidence is lost.

Interview questions

Q1. How does the JVM behave in containers? It is container-aware. It reads the cgroup CPU and memory limits and uses them to size the heap, the GC threads and thread pools. You should set `MaxRAMPercentage` to about 70–75%, to leave room for non-heap memory.

Q2. A pod keeps restarting with exit code 137 but no Java OOM in the logs. Why? The kernel OOM-killed it. The total process memory — heap plus metaspace, threads and direct buffers — went over the container limit. Fix it by lowering the heap percentage or raising the limit, and check native memory usage with NMT (Native Memory Tracking).

Part 2 — Spring Boot

This part has three chapters:

- **The container.** How Spring really works underneath the annotations.
- **The web and data layers.** This is where most production bugs and interview questions live.
- **The production toolkit.** Security, testing, messaging, caching, resilience and observability (logs, metrics and traces that show what the app is doing). It ends with the Java project that proves your switch from .NET.

6. Spring Core and Spring Boot Fundamentals

6.1 What Spring and Spring Boot solve

Why?

> **In simple words:** Spring builds all your objects and connects them to each other, so you don't write `new` everywhere. Spring Boot does the remaining setup for you, so a working web service needs almost no configuration.

A real backend has hundreds of objects that depend on each other:

- Controllers need services.
- Services need repositories, clients, caches and config.
- Repositories need a `DataSource` (the object that gives out database connections).

You could create and connect all of them by hand with `new`. But that causes three problems:

1. **The code becomes rigid.** Each class hard-codes which implementation it uses.
2. **The code is hard to test.** You can't swap in a fake object for a test.
3. **The same plumbing repeats everywhere.** Transactions, security checks and logging get copy-pasted into many methods.

Spring fixes this. It creates these objects and connects them for you. It can also add shared behaviour (like transactions) around them. Behaviour that many classes need is called *cross-cutting* behaviour.

Spring Boot removes the setup that is still left. It gives you:

- sensible defaults,
- infrastructure that configures itself automatically,
- a web server built into your app,
- and a single JAR file that you can run directly.

What is happening internally?

- **Spring Framework** = an **IoC container** (`ApplicationContext`) + modules built on top of it. The container is the object that creates and holds all your objects. The modules include Web MVC, Data/JDBC/ORM integration, transactions, AOP, events, Security (a separate project), and more.
- **Spring Boot** adds these things on top:
- **Starters**: ready-made bundles of dependencies. For example, `spring-boot-starter-web` brings Spring MVC, Jackson (for JSON), embedded Tomcat and validation hooks — all in one line.
- **Auto-configuration**: configuration classes guarded by `@Conditional` checks. They create beans (DataSource, ObjectMapper, transaction manager...) only when two things are true: the needed classes and properties are present, and you haven't defined your own bean (6.9). A *bean* is simply an object that Spring creates and manages.
- **Externalised configuration**: settings live outside the code — in `application.yml`, profiles and environment variables (6.8).
- **An embedded server**: Tomcat runs *inside* your app. So you don't deploy your app onto a separate server; the app carries its own server.

- **Executable fat JAR**: one JAR that holds your classes, all dependency JARs nested inside, and a special launcher class loader that can read the nested JARs.
- **Actuator**: ready-made endpoints for health, metrics and info (8.9).

Here is how the ideas map from ASP.NET Core, which you already know:

ASP.NET Core you know	Spring Boot
`Program.cs`, `WebApplication.CreateBuilder`	`@SpringBootApplication` + `SpringApplication.run`
`builder.Services.AddScoped<IFoo, Foo>()`	`@Service class Foo implements IFoo` (found by component scan) or a `@Bean` method
Singleton / Scoped / Transient	`singleton` (default) / `request` / `prototype`
Middleware	Servlet filters, `HandlerInterceptor`s, AOP
`[ApiController]`	`@RestController`
`appsettings.json` + `IOptions<T>`	`application.yml` + `@ConfigurationProperties`
`ASPNETCORE_ENVIRONMENT`	`spring.profiles.active`
EF Core	Spring Data JPA + Hibernate
`IHostedService`	`@Scheduled`, `ApplicationRunner`, `SmartLifecycle`
Health checks	Actuator `/actuator/health`

How does it work?

When you run the JAR, these steps happen in order:

```
java -jar loyalty.jar
 → JarLauncher sets up a class loader over BOOT-INF/classes + BOOT-INF/lib/*.jar
 → your main(): SpringApplication.run(App.class)
 → creates the ApplicationContext, scans components, applies auto-configuration
 → instantiates and wires beans, wraps some in proxies
 → starts embedded Tomcat on port 8080 → ready
```

In plain words:

1. `JarLauncher` sets up a class loader. It can read your classes and every library JAR nested inside the fat JAR.
2. Your `main()` method calls `SpringApplication.run`.
3. Spring creates the `ApplicationContext` (the container).
4. It scans your packages for components and applies auto-configuration.
5. It creates all the beans, connects them, and wraps some of them in proxies (wrapper objects — see 6.7).
6. It starts the embedded Tomcat server on port 8080. The app is now ready.

Example

```java
@SpringBootApplication              // = @Configuration + @EnableAutoConfiguration + @ComponentScan
public class LoyaltyApplication {
    public static void main(String[] args) { SpringApplication.run(LoyaltyApplication.class, args); }
}
```

```
@RestController
@RequestMapping("/api/v1/accounts")
class AccountController {
    private final AccountService service;
    AccountController(AccountService service) { this.service = service; }   // injected by Spring

    @GetMapping("/{id}/balance")
    BalanceDto balance(@PathVariable String id) { return service.balance(id); }
}
```

```
<!-- pom.xml essentials -->
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.5.6</version>   <!-- take the current version from start.spring.io; concepts here also apply to
4.x -->
</parent>
<dependencies>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</artifactId>
</dependency>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-
jpa</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
validation</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
actuator</artifactId></dependency>
  <dependency><groupId>org.postgresql</groupId><artifactId>postgresql</artifactId><scope>runtime</scope>
</dependency>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope></dependency>
</dependencies>
```

The parent POM imports the Spring Boot **BOM** (Bill of Materials). A BOM is a list that fixes compatible versions of hundreds of libraries. Because of it, your starters need no version numbers — Boot has already chosen versions that work together.

When would I use it?

For any Java backend service. It is the default stack at most Indian product companies (Flipkart, PhonePe, Swiggy, Walmart Global Tech). It is also widely used at FAANG for internal services.

What can go wrong?

- **"Magic" you don't understand.** A bean you never defined behaves in an unexpected way. Learn to read the auto-configuration report, which tells you what Boot set up and why (6.9).
- **Pulling in starters you don't need.** This gives you a larger JAR, slower startup and surprise auto-configuration. For example, adding `spring-boot-starter-data-jpa` makes the app fail to start if there is no DataSource configured.
- **Overriding managed dependency versions by hand.** If you change versions that the BOM picked, you can break compatibility between libraries.

Interview questions

Q1. Spring vs Spring Boot? Spring is the IoC container plus its family of modules. Boot is an opinionated layer on top of it — it makes default choices for you. It adds starters, auto-configuration, externalised config, an embedded server, executable JARs and Actuator, so an app needs almost no setup.

Q2. What does `@SpringBootApplication` do? It combines three annotations: `@Configuration`, `@EnableAutoConfiguration` and `@ComponentScan`. The component scan starts from the annotated class's

package and goes downward into its sub-packages.

Q3. What is a starter? It is a dependency bundle that brings everything needed for one capability (web, JPA, security), with versions that work together. Having it on the classpath also triggers the matching auto-configuration.

6.2 Inversion of Control and Dependency Injection

Why?

> **In simple words:** A class should not build the things it needs. It should just ask for them, and someone else (the Spring container) hands them over. This makes the class easy to test and easy to change.

Suppose `RedemptionService` creates its own dependencies:

- `new PostgresLedgerRepository()`
- `new TwilioSmsClient()`

Now it is welded to those exact implementations. Two problems follow:

1. You can't unit test it without a real database and a real SMS account.
2. Switching to another SMS vendor means editing your business code.

Inversion of Control (IoC) flips this. The class only declares what it needs. An outside **container** creates those dependencies and hands them in. This handing-in is called **Dependency Injection** (DI).

Think of a restaurant kitchen. The chef does not go to the market for vegetables. He writes what he needs on a list, and the supplier delivers it. If the supplier changes, the chef's recipes don't change.

What is happening internally?

The `ApplicationContext` works in two phases.

1. **Phase 1 — Bean definitions (the recipes).** - Spring collects a *recipe* for each bean. A recipe says: which class, which scope, which constructor arguments, which dependencies, and which init/destroy methods. - Recipes come from component scanning, `@Bean` methods and auto-configuration. - Each recipe is stored as a `BeanDefinition` object in a registry. - No real objects exist yet.
2. **Phase 2 — Instantiation (cooking the recipes).** For each singleton that is not marked lazy, Spring:
3. works out the dependency graph (who needs whom),
4. creates the dependencies first,
5. calls the constructor with them,
6. injects fields and setters,
7. runs lifecycle callbacks,
8. applies post-processors (these may wrap the bean in a proxy),
9. stores the result in the **singleton cache** (a `ConcurrentHashMap<String, Object>`).

Spring finds a dependency **by type** first. If several beans match that type, it narrows the choice by qualifier or by name (6.4).

```
Bean definitions:  accountController → needs AccountService
                   accountService    → needs AccountRepository, Clock
```

```
                  accountRepository (Spring Data proxy) → needs EntityManager
Creation order:   EntityManager → AccountRepository → Clock → AccountService → AccountController
```

In the diagram, nothing can be created before the things it needs. So the `EntityManager` comes first and the controller comes last.

How does it work?

There are three ways to inject a dependency:

```
@Service
public class RedemptionService {
    private final LedgerRepository ledger;          // 1. Constructor injection — use this
    private final NotificationSender notifier;

    public RedemptionService(LedgerRepository ledger, NotificationSender notifier) {   // single constructor: @Autowired optional
        this.ledger = ledger;
        this.notifier = notifier;
    }
}

@Service
class LegacyService {
    @Autowired private AuditClient audit;           // 2. Field injection — avoid (hidden deps, no final, needs reflection in tests)
    private Notifier notifier;
    @Autowired void setNotifier(Notifier n) { this.notifier = n; }   // 3. Setter — only for optional deps
}
```

1. **Constructor injection** — use this by default.
2. **Field injection** — avoid it. The dependencies are hidden, the field can't be `final`, and tests need reflection to set it.
3. **Setter injection** — use it only for optional dependencies.

Why constructor injection wins:

- **Dependencies are explicit.** You can see them all in the constructor signature.
- **Fields are `final`.** So they are immutable (can't change), and other threads always see them correctly set. This is called *safe publication*.
- **The object is never half-built.** It can't exist without its dependencies.
- **Unit tests are simple.** You just call `new RedemptionService(fakeLedger, fakeNotifier)`.
- **It warns you about bad design.** If the constructor has too many parameters, the class is clearly doing too much.

Example

```
// Unit test with no Spring at all — the payoff of DI
class RedemptionServiceTest {
    @Test void notifiesAfterRedemption() {
        var ledger = new InMemoryLedgerRepository();
        var notifier = new RecordingNotifier();
        var service = new RedemptionService(ledger, notifier);
        service.redeem("acc1", 100);
        assertThat(notifier.sent()).hasSize(1);
    }
}
```

This test needs no Spring at all. You pass in fake objects by hand. This is the real payoff of DI.

When would I use it?

For every service, repository, client and component.

Program to interfaces (also called *ports*) for anything you might swap or fake in tests:

- external clients,
- notification channels,
- clocks,
- ID generators.

What can go wrong?

- **Circular dependencies.** A needs B, and B needs A.
- With constructor injection, Spring can't create either one first. So startup fails with `BeanCurrentlyInCreationException`.
- Since Boot 2.6, circular references are blocked by default, even with field injection.
- The fix is to change the design. Extract the shared logic into a third bean, or use events.
- Putting `@Lazy` on one parameter is a last resort. It injects a proxy (a stand-in object) instead of the real bean.
- **Calling `new` yourself on a Spring-managed class.** The object you get has no injection, no proxies and no `@Transactional`. Spring never touched it.
- **Static access to beans.** For example, a static holder that keeps the ApplicationContext. This hides the dependencies again, which is exactly what DI was meant to fix.

Interview questions

Q1. What are IoC and DI? IoC means the container, not your code, controls creating objects and connecting them. DI means dependencies are given to the object through the constructor, a setter or a field, instead of the object creating them itself. This gives loose coupling, easy testing and one central place for configuration.

Q2. Why prefer constructor injection? Dependencies are explicit and immutable, objects are never partly initialised, unit tests are easy without Spring, and circular dependencies fail early at startup.

Q3. How does Spring resolve which bean to inject? First by type. If several beans match, it uses `@Primary`, then `@Qualifier`, then the parameter name matching a bean name. If none of these decides it, startup fails with `NoUniqueBeanDefinitionException`.

Q4. How do you fix a circular dependency? Refactor the design: move the shared responsibility into a new bean, or reverse one dependency using an interface or events. Only as a last resort, use `@Lazy` injection.

6.3 Defining beans: stereotypes, `@Bean` and `@Configuration`

Why?

> **In simple words:** You must tell Spring which objects to create. For your own classes, put an annotation on the class. For classes you can't edit, write a small method that builds the object and mark it `@Bean`.

The container needs to know two things: which classes to manage, and how to build them.

- **Your own classes:** an annotation on the class is enough.
- **Third-party classes you can't annotate** (an `ObjectMapper`, an `ExecutorService`, an SDK client), or objects that need some building logic: you write a *factory method* — a method whose job is to create and return the object.

What is happening internally?

- **Component scanning** walks through the packages under your `@SpringBootApplication` class.
- It reads the class files with a library called ASM. So it does not need to load every class into memory.
- For each class marked with `@Component`, or with an annotation built on `@Component` (a *stereotype*), it registers a bean definition.

Annotation	Meaning
`@Component`	A general managed component
`@Service`	Business logic (just a label; no extra behaviour)
`@Repository`	Data access; Spring also converts database exceptions into its own `DataAccessException` family
`@Controller` / `@RestController`	Web handler; `@RestController` = `@Controller` + `@ResponseBody`
`@Configuration`	A class that holds `@Bean` factory methods

- **`@Configuration` classes are wrapped in a CGLIB proxy.** CGLIB is a library that creates a subclass of your class at runtime.
- Because of this proxy, when one `@Bean` method calls another `@Bean` method in the same class, the call goes through the proxy.
- The proxy returns the **existing singleton** instead of creating a new object.
- **"Lite mode":** **`@Configuration(proxyBeanMethods = false)`.** There is no proxy, so startup is a bit faster.
- But now the methods are plain Java calls. Calling one `@Bean` method from another creates a **new** instance each time.
- So in lite mode, take other beans as method parameters instead of calling their methods.

How does it work?

```
@Configuration
class InfraConfig {
    @Bean
    Clock clock() { return Clock.systemUTC(); }                    // third-party type → @Bean

    @Bean(destroyMethod = "shutdown")
    ExecutorService rewardExecutor() {
        return new ThreadPoolExecutor(4, 16, 60, TimeUnit.SECONDS, new ArrayBlockingQueue<>(500),
                Thread.ofPlatform().name("reward-", 0).factory(), new ThreadPoolExecutor.CallerRunsPolicy());
    }

    @Bean
    PartnerClient partnerClient(RestClient.Builder builder, PartnerProps props) {   // dependencies as
parameters
        return new PartnerClient(builder.baseUrl(props.baseUrl()).build());
    }
}
```

Three patterns are shown here:

1. `Clock` is a third-party type, so you create it with a `@Bean` method.
2. The thread pool uses `destroyMethod = "shutdown"`. So Spring calls `shutdown()` on it when the app stops.
3. `PartnerClient` needs other beans. It receives them as method parameters, and Spring fills them in.

Example

```
@Configuration
class Demo {
    @Bean A a() { return new A(b()); }
    @Bean B b() { return new B(); }
}
// With full @Configuration (default): b() inside a() returns the SAME singleton B as the "b" bean.
// With proxyBeanMethods = false: b() creates a SECOND B — two instances, subtle bugs (two caches, two pools).
```

With the default full `@Configuration`, both places get the same `B`. In lite mode you silently get two `B` objects. That can mean two caches or two connection pools — bugs that are hard to spot.

When would I use it?

- **Stereotype annotations** for your own classes.
- `@Bean` for third-party objects, clients that need configuration, thread pools, and variants that depend on a condition or a profile.
- Group your infrastructure beans into a few `@Configuration` classes, one per concern. For example: `PersistenceConfig`, `KafkaConfig`, `SecurityConfig`.

What can go wrong?

- **A class outside the scanned packages is silently ignored.** If it sits above the main class's package, scanning never sees it. You get `NoSuchBeanDefinitionException`.
- **`final` `@Configuration` classes or `final` `@Bean` methods can't be proxied by CGLIB.** CGLIB works by subclassing and overriding, and `final` blocks both.
- **Two beans of the same type defined in different places** (yours plus an auto-configured one).
- Usually yours wins, because the auto-configured one has `@ConditionalOnMissingBean` and steps back.
- But if the two beans have the same name, it is a clash. Bean overriding is disabled by default, so startup fails.

Interview questions

Q1. `@Component` vs `@Bean`? `@Component` marks your own class so component scanning picks it up. `@Bean` is a factory method inside a `@Configuration` class; you use it for third-party types or when building the object needs custom logic.

Q2. `@Component` vs `@Service` vs `@Repository`? All three register beans. `@Service` is only a label that shows meaning. `@Repository` also adds persistence exception translation — it converts database exceptions into Spring's `DataAccessException` types.

Q3. Why are `@Configuration` classes proxied? So that when one `@Bean` method calls another, it gets back the container's singleton instead of creating a new instance.

6.4 Multiple implementations: `@Primary`, `@Qualifier`, and injecting collections

Why?

> **In simple words:** When one interface has several implementations, you must tell Spring which one to use. You can mark a default, ask for a specific one by name, or take all of them and choose at runtime.

Real systems often have several implementations of one interface. For example:

- payment gateways: Razorpay, UPI, card,
- notification channels: SMS, WhatsApp, email,
- reward rules.

You need one of three things:

1. a **default** one,
2. a **specific** one,
3. **all** of them, so you can pick one at runtime. This is the Strategy pattern — choose the algorithm while the program runs.

What is happening internally?

Say a constructor parameter has type `T`. The container collects every bean that can be assigned to `T`. Then:

1. If there is exactly one, it injects that one.
2. If there are several, a bean marked `@Primary` wins.
3. If there is no `@Primary`, a `@Qualifier` on the parameter filters the candidates.
4. If there is no qualifier, Spring matches the parameter name to a bean name.
5. If nothing decides it, startup fails with `NoUniqueBeanDefinitionException`.

Collections work differently:

- For `List<T>`, Spring injects **all** candidates. They are ordered by `@Order` or the `Ordered` interface.
- For `Map<String, T>`, Spring injects a map of bean name → bean.

Think of an ATM that accepts several bank cards. It has a default network, but it can also route to a specific bank when the card says so.

How does it work?

```
public interface PaymentGateway { Receipt pay(Money amount, String ref); String code(); }

@Component("razorpay") @Primary class RazorpayGateway implements PaymentGateway { /* ... */ }
@Component("upi")                class UpiGateway implements PaymentGateway { /* ... */ }
@Component("card")               class CardGateway implements PaymentGateway { /* ... */ }
```

Each gateway is a bean with its own name. `RazorpayGateway` is marked `@Primary`, so it is the default.

Example

```
@Service
class CheckoutService {
```

```
    private final PaymentGateway defaultGateway;                    // the @Primary one
    private final Map<String, PaymentGateway> gateways;              // "razorpay" -> ..., "upi" -> ...,
"card" -> ...

    CheckoutService(PaymentGateway defaultGateway, Map<String, PaymentGateway> gateways) {
        this.defaultGateway = defaultGateway;
        this.gateways = gateways;
    }

    Receipt pay(String method, Money amount, String ref) {
        PaymentGateway gw = Optional.ofNullable(gateways.get(method))
                .orElseThrow(() -> new IllegalArgumentException("unsupported method " + method));
        return gw.pay(amount, ref);                                  // Strategy chosen at runtime
    }
}

@Service
class Refunds {
    Refunds(@Qualifier("upi") PaymentGateway upi) { /* explicitly the UPI one */ }
}
```

- `CheckoutService` gets the `@Primary` gateway as its default.
- It also gets a map of all gateways, keyed by bean name. It picks one at runtime from the `method` the user chose.
- `Refunds` asks for the UPI gateway by name with `@Qualifier("upi")`.

Now, adding a new gateway is just one new `@Component` class. You don't edit any existing code. This is the open/closed principle (9.4) for free — open for extension, closed for modification.

When would I use it?

- **Strategy registries:** payment methods, notification channels, incentive rule evaluators keyed by rule type.
- **Plugin-style extension points:** new behaviour is added by dropping in a new bean.
- **Environment-specific defaults:** `@Primary` on the real client, and a stub (fake) under a test profile.

What can go wrong?

- **Two `@Primary` beans of the same type.** Spring can't choose, so you get an ambiguity error.
- **Qualifier strings scattered across the code.** If someone renames a bean, these strings break silently. Prefer a custom qualifier annotation, or a map keyed by a method on the interface (like `code()`).
- **Relying on parameter-name matching.** It breaks easily when someone renames a parameter during refactoring.

Interview questions

Q1. Two beans implement the same interface. How does Spring choose? First `@Primary`, then `@Qualifier`, then the parameter name. If none of these decides it, Spring throws NoUniqueBeanDefinitionException. If you want all of them, inject `List<T>` or `Map<String, T>`.

Q2. How do you implement the Strategy pattern with Spring? Make each strategy a bean that implements a common interface. Then inject `Map<String, Strategy>` or a `List`, and build a registry keyed by type to pick the right strategy at runtime.

6.5 Bean scopes

Why?

> **In simple words:** Scope decides how many copies of a bean exist and how long each one lives. Most beans are one shared copy for the whole app; a few need a new copy per request or per use.

- Most beans are stateless services — they hold no per-user data. So everyone can share one instance.
- Some objects must exist once per request (for example, a request context).
- Some must be brand new each time (stateful helpers that collect data).

Scope controls **how many instances exist and how long they live**.

What is happening internally?

Scope	Instances	Lifecycle managed by container
`singleton` (default)	One per ApplicationContext, cached	Full (created at startup, destroyed at shutdown)
`prototype`	A new instance on **every lookup or injection**	Creation only — no destroy callbacks
`request`	One per HTTP request (stored in request attributes)	Web only
`session`	One per HTTP session	Web only
`application`	One per ServletContext	Web only

A Spring singleton is **one per container, not one per JVM**. This is different from the classic GoF Singleton pattern (from the "Gang of Four" design patterns book).

A singleton is shared by **every request thread** at the same time. So it must be **stateless or thread-safe**.

Think of a railway enquiry counter with one clerk. Everyone talks to the same clerk. If he writes your coach number on a single shared slate, the next person can overwrite it before you read it back.

How does it work?

Suppose you inject a **shorter-lived** bean (request or prototype) into a singleton. The injection happens only **once** — when the singleton is created. After that, the singleton keeps the same object forever.

To get a fresh instance each time, you need a middle step:

- **`ObjectProvider<T>`**: call `getObject()` whenever you need a new one.
- **`@Lookup` method injection**: Spring overrides the method so that it fetches a fresh bean from the container.
- **A scoped proxy** (`@Scope(value = "request", proxyMode = ScopedProxyMode.TARGET_CLASS)`): the singleton holds a proxy. On every call, the proxy finds the instance for the current request.

Example

```
@Component @Scope("prototype")
class ReportBuilder { private final List<String> rows = new ArrayList<>(); void add(String r) { rows.add(r);
} }
```

```
@Service
class ReportService {
    private final ObjectProvider<ReportBuilder> builders;
    ReportService(ObjectProvider<ReportBuilder> builders) { this.builders = builders; }
    String build(List<Order> orders) {
        ReportBuilder b = builders.getObject();            // fresh, unshared instance per call
        orders.forEach(o -> b.add(o.toString()));
        return String.join("\n", b.rows);
    }
}

// The bug: request state in a singleton
@Service
class BadPricingService {
    private String currentTenant;                          // shared by ALL threads!
    Price price(String tenant, String sku) {
        currentTenant = tenant;                            // thread A sets "t1", thread B sets "t2"…
        return lookup(currentTenant, sku);                 // …thread A may now read "t2" → wrong tenant's
price
    }
}
```

- `ReportService` gets a new `ReportBuilder` on each call through `ObjectProvider`. So two requests never share the same rows.
- `BadPricingService` stores request data in a field of a singleton. Thread A sets the tenant to "t1". Then thread B sets it to "t2". Thread A now reads "t2" and returns the wrong tenant's price.

When would I use it?

- **Singleton** for almost all services, repositories, clients and controllers.
- **Request scope** for request context objects. (Or use a ThreadLocal holder — a per-thread variable — that a filter sets.)
- **Prototype** for stateful, short-lived helpers built by a factory.

What can go wrong?

- **Mutable fields in singletons.** They cause race conditions (threads overwriting each other) and **cross-tenant data leaks**, as shown above.
- **Prototype beans holding resources.** The container never destroys them, so you must clean them up yourself.
- **Request-scoped beans used outside a request.** In async threads or scheduled jobs there is no request. So they fail with "No thread-bound request found".

Interview questions

Q1. What bean scopes exist? singleton (the default), prototype, request, session and application — plus websocket.

Q2. Are singleton beans thread-safe? Only if you make them so. Spring does not synchronise them. Keep them stateless, or use thread-safe fields.

Q3. What happens when a prototype is injected into a singleton? It is injected only once, so the singleton always uses the same instance. To get a fresh one, use ObjectProvider, @Lookup, or a scoped proxy.

Q4. Spring singleton vs GoF Singleton? Spring's singleton is one instance per container, so several contexts mean several instances. The GoF Singleton is one per class loader, and the class itself enforces it.

6.6 Bean lifecycle and post-processors

Why?

> **In simple words:** Every bean goes through fixed stages: it is created, filled with its dependencies, set up, used, and finally cleaned up. Knowing these stages tells you where to put your setup and cleanup code — and when Spring adds its proxies.

Beans often need two kinds of extra work:

- **Setup after dependencies are injected:** load a rule cache, check the config, open a connection.
- **Cleanup at shutdown:** flush buffers, close pools.

Spring itself also needs hooks to add behaviour to beans. For example, it adds proxies for transactions, security and async.

The lifecycle defines exactly when each of these happens.

What is happening internally?

```
1. Instantiate              constructor (constructor injection happens here)
2. Populate properties      field / setter injection
3. Aware callbacks          BeanNameAware, BeanFactoryAware, ApplicationContextAware
4. BeanPostProcessor.postProcessBeforeInitialization   (all BPPs, in order)
5. Initialisation           @PostConstruct → InitializingBean.afterPropertiesSet() → custom init-method
6. BeanPostProcessor.postProcessAfterInitialization    ← AOP auto-proxy creator WRAPS the bean in a proxy
here
7. Ready — the PROXY (if any) is what gets stored and injected everywhere
   ...
8. Shutdown (singletons)    @PreDestroy → DisposableBean.destroy() → custom destroy-method
```

Step by step:

1. **Instantiate.** Spring calls the constructor. Constructor injection happens here.
2. **Populate properties.** Field and setter injection happen.
3. **Aware callbacks.** If the bean asks for it, Spring gives it its own name, the BeanFactory, or the ApplicationContext.
4. **Before-init post-processing.** Every `BeanPostProcessor` runs `postProcessBeforeInitialization`, in order.
5. **Initialisation.** First `@PostConstruct`, then `InitializingBean.afterPropertiesSet()`, then any custom init-method.
6. **After-init post-processing.** Every `BeanPostProcessor` runs `postProcessAfterInitialization`. **Here the AOP auto-proxy creator wraps the bean in a proxy.**
7. **Ready.** If a proxy was made, the **proxy** is stored and injected everywhere — not the raw object.
8. **Shutdown (singletons only).** First `@PreDestroy`, then `DisposableBean.destroy()`, then any custom destroy-method.

There are two extension points. Their names are similar, so keep them apart:

- **BeanFactoryPostProcessor** works on the **recipes**.
- It runs **before any bean is created**, and it changes **bean definitions**.
- Examples: `PropertySourcesPlaceholderConfigurer` fills in `${...}` placeholders. Configuration-class processing registers more bean definitions.
- **BeanPostProcessor** works on the **finished objects**.

- It runs **for each bean instance**, and it can change the bean or **replace** it completely.
- Examples: `AutowiredAnnotationBeanPostProcessor` does the `@Autowired` injection. `AnnotationAwareAspectJAutoProxyCreator` creates AOP proxies for `@Transactional`, `@Async`, `@Cacheable` and aspects.

Think of a tiffin service. The BeanFactoryPostProcessor edits the menu card before anything is cooked. The BeanPostProcessor works on each packed tiffin — it can add a label, or put the box inside a bigger insulated bag (the proxy).

How does it work?

Order matters here.

- Inside a constructor or `@PostConstruct`, `this` is the raw object. It is **not the proxy**, because the proxy is created later, in step 6.
- So if you call your own `@Transactional` method from `@PostConstruct`, it runs **without** a transaction.
- Work that needs the full context and the proxies should run after startup. Use `ApplicationRunner` or `@EventListener(ApplicationReadyEvent.class)`.

Example

```
@Component
class RuleCache {
    private final RuleRepository repo;
    private volatile Map<String, Rule> rules = Map.of();
    RuleCache(RuleRepository repo) { this.repo = repo; }        // 1: dependencies available

    @PostConstruct
    void load() { rules = repo.findAllActive(); }               // 5: safe to use injected deps

    @PreDestroy
    void close() { rules = Map.of(); }                          // 8: on graceful shutdown
}

@Component
class WarmUp {
    private final RecommendationService recs;
    WarmUp(RecommendationService recs) { this.recs = recs; }
    @EventListener(ApplicationReadyEvent.class)                 // after everything (proxies, server) is
ready
    void warm() { recs.preloadPopularSkus(); }
}

// A custom BeanPostProcessor (how frameworks add behaviour)
@Component
class TimingPostProcessor implements BeanPostProcessor {
    @Override public Object postProcessAfterInitialization(Object bean, String name) {
        // could return a proxy wrapping 'bean' — exactly how AOP works
        return bean;
    }
}
```

- `RuleCache` loads its rules in `@PostConstruct` (step 5), when the repository is already injected. It clears them in `@PreDestroy` (step 8).
- `WarmUp` waits for `ApplicationReadyEvent`. At that point all proxies exist and the server is running.
- `TimingPostProcessor` shows the hook frameworks use. It could return a proxy around `bean` instead of `bean` itself — that is exactly how AOP works.

When would I use it?

- `@PostConstruct`: cache warm-up and validation that use injected dependencies.
- `@PreDestroy`: flushing and closing things.
- `ApplicationRunner`: jobs that should run once at startup.
- **BeanPostProcessors**: only when you write infrastructure or library code.

What can go wrong?

- **Heavy or slow work in constructors or `@PostConstruct`.** It delays startup and readiness. And if it fails, the whole startup is aborted.
- **Expecting `@Transactional` or `@Async` to work inside `@PostConstruct`.** There is no proxy yet, so they do nothing.
- **`@PreDestroy` not being called.** It is never called for prototype beans, or when the process is killed with `kill -9`. Enable graceful shutdown (`server.shutdown=graceful`) and handle SIGTERM (the polite stop signal) in Kubernetes.

Interview questions

Q1. Describe the Spring bean lifecycle. Instantiate → inject dependencies → Aware callbacks → BPP before-init → @PostConstruct / afterPropertiesSet / init-method → BPP after-init (this is where proxies are created) → in use → @PreDestroy / destroy on shutdown.

Q2. BeanFactoryPostProcessor vs BeanPostProcessor? A BFPP changes bean definitions (the recipes) before any bean is created. A BPP changes or wraps each bean instance, around the time it is initialised.

Q3. When are AOP proxies created? In `postProcessAfterInitialization`, by the auto-proxy creator BeanPostProcessor. The proxy then replaces the raw bean in the container.

6.7 Proxies and AOP: how `@Transactional`, `@Async` and `@Cacheable` really work

Why?

> **In simple words:** Spring puts a wrapper object (a proxy) in front of your bean. Every outside call passes through the wrapper, which adds extra work like starting a transaction. But a call from inside the bean to itself skips the wrapper — and that is the source of many Spring bugs.

Some concerns cut across hundreds of methods: transactions, security checks, caching, retries, metrics and logging.

If you copy-paste `begin/commit/rollback` or timing code into each method, you will make mistakes. **Aspect-Oriented Programming** (AOP) solves this. You write the behaviour once, and it is applied around every matching method.

Spring builds AOP using **proxies**. Once you understand proxies, you can explain the most common Spring bugs.

Think of a bank branch. You don't walk straight to the cashier. The security guard at the door checks you first, and logs you on the way out. The cashier (your real method) only does the money work.

What is happening internally?

A **proxy** is an object that:

1. has the same type as your bean,
2. catches each call made to it,
3. runs **advice** — extra logic before, after or around the call,
4. and passes the call on to the real object.

There are two kinds of proxy:

- **JDK dynamic proxy**: it implements the bean's interfaces (using `java.lang.reflect.Proxy`). Because it only implements interfaces, you can't cast it to the concrete class.
- **CGLIB proxy**: it creates a **subclass** of your class at runtime and overrides its methods. Spring Boot uses CGLIB by default.
- Result: `final` classes, and `final` or `private` methods, can't be intercepted. A subclass can't override them.

```
caller ──► [CGLIB proxy of RedemptionService]
              TransactionInterceptor: begin tx
              ──► real RedemptionService.redeem()
                     └── this.audit()   ← internal call on 'this' — NOT through the proxy
              TransactionInterceptor: commit (or rollback on RuntimeException)
```

In the diagram, the outside call goes through the proxy, so the transaction starts. But `this.audit()` is called on the real object directly. It never passes through the proxy.

Key words you should know:

- **Aspect**: the module that holds the cross-cutting logic.
- **Advice**: the logic itself, and when it runs — `@Before`, `@After`, `@AfterReturning`, `@AfterThrowing`, `@Around`.
- **Pointcut**: an expression that selects which methods get the advice.
- **Join point**: one single method execution.
- **Weaving**: applying aspects to code. Spring does it at runtime with proxies. AspectJ can also weave at compile time or when classes are loaded.

How does it work?

Only calls that **come in through the proxy** are intercepted.

A call from inside the object (`this.method()`) goes straight to the real object and bypasses the proxy. This is the **self-invocation problem**.

Because of it, these annotations silently stop working on the called method: `@Transactional`, `@Async`, `@Cacheable`, `@Retryable`, `@PreAuthorize`. There is no error — they just do nothing.

Example

```java
@Service
public class LedgerService {
    public void processBatch(List<Txn> txns) {
        for (Txn t : txns) post(t);          // this.post(t): bypasses proxy → NO new transaction per txn
    }
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void post(Txn t) { /* ... */ }
}

// Fix 1 (cleanest): move the transactional method to another bean
@Service class LedgerPoster { @Transactional(propagation = Propagation.REQUIRES_NEW) public void post(Txn t)
{ } }
```

```
@Service class LedgerBatch {
    private final LedgerPoster poster;
    LedgerBatch(LedgerPoster poster) { this.poster = poster; }
    public void processBatch(List<Txn> txns) { txns.forEach(poster::post); }   // through the proxy
}

// Fix 2: programmatic transactions
@Service class LedgerBatch2 {
    private final TransactionTemplate tx;
    LedgerBatch2(PlatformTransactionManager tm) { this.tx = new TransactionTemplate(tm); }
    public void processBatch(List<Txn> txns) { for (Txn t : txns) tx.executeWithoutResult(s -> post(t)); }
    private void post(Txn t) { /* ... */ }
}

// Writing your own aspect: time any method annotated @Timed
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.METHOD) @interface Timed {}

@Aspect @Component
class TimingAspect {
    private static final Logger log = LoggerFactory.getLogger(TimingAspect.class);
    @Around("@annotation(com.fa.loyalty.Timed)")
    public Object time(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.nanoTime();
        try { return pjp.proceed(); }
        finally { log.info("{} took {} ms", pjp.getSignature().toShortString(), (System.nanoTime() - start) /
1_000_000); }
    }
}
```

What the code shows:

- **The bug.** `processBatch` calls `post(t)` on `this`. The proxy is skipped, so there is **no** new transaction per transaction record.
- **Fix 1 (cleanest).** Move the transactional method into another bean (`LedgerPoster`). Now the call goes through that bean's proxy.
- **Fix 2.** Use programmatic transactions with `TransactionTemplate`. You start each transaction in code, so no proxy is needed.
- **Your own aspect.** `TimingAspect` times any method marked `@Timed`. `@Around` wraps the call, and `pjp.proceed()` runs the real method.

When would I use it?

- Use the built-in proxied annotations every day: `@Transactional`, `@Async`, `@Cacheable`, `@PreAuthorize`, `@Retryable`.
- Write custom aspects only for truly cross-cutting concerns: timing and metrics, audit logging, tenant checks, and idempotency guards (checks that make a repeated request safe — doing it twice has the same effect as doing it once).
- Keep business logic out of aspects.

What can go wrong?

- **Self-invocation** (explained above). This is the #1 Spring interview trap.
- **Annotations on `private` or `final` methods.** They are ignored, with no error at all.
- **Calling the bean before it is proxied.** For example, in the constructor or in `@PostConstruct`.
- **Objects created with `new`.** Spring never sees them, so they are not proxied.
- **Aspect ordering surprises.** For example, is the retry inside the transaction or outside it? Set the order explicitly with `@Order`.

- **Over-broad pointcuts** like `execution(* com..*(..))`. They slow everything down and make the code hard to reason about.

Interview questions

Q1. How does Spring AOP work? Spring creates proxies — CGLIB subclasses or JDK interface proxies — inside a BeanPostProcessor. The proxy runs the advice around each call and then passes the call to the target bean.

Q2. Why doesn't `@Transactional` work when I call the method from the same class? The internal call goes through `this`, not through the proxy, so the transaction interceptor never runs. Move the method to another bean, or use TransactionTemplate.

Q3. JDK proxy vs CGLIB? A JDK proxy can only implement interfaces. CGLIB creates a subclass of the class, and it is Boot's default. Because it subclasses, final classes and final methods can't be advised.

Q4. Name the advice types. @Before, @AfterReturning, @AfterThrowing, @After (runs always, like `finally`), and @Around (wraps the call and decides whether and when to proceed).

6.8 Configuration, `@ConfigurationProperties` and profiles

Why?

> **In simple words:** Build the app once, and change its settings from outside — through files, environment variables and secrets. The same JAR then runs on your laptop, in QA and in production.

The same JAR must run in different places: on a laptop, in QA and in production. Each place has different databases, URLs, credentials and feature settings.

If you hard-code these values, you must rebuild the app for every environment.

Externalised configuration fixes this. You keep one artefact (one built JAR). Only the files, environment variables and secrets change.

What is happening internally?

Spring builds an `Environment` object from an ordered list of **property sources** (places where settings come from). If two sources set the same key, the higher one wins.

Simplified order, highest first:

1. Command-line arguments (`--server.port=9090`)
2. `SPRING_APPLICATION_JSON`
3. OS environment variables (`SPRING_DATASOURCE_URL`, `LOYALTY_REDEMPTION_MAXPOINTSPERDAY`). **Relaxed binding** maps these to normal keys — so `SPRING_DATASOURCE_URL` becomes `spring.datasource.url`.
4. Profile-specific files: `application-prod.yml`
5. `application.yml` / `application.properties`
6. `@PropertySource`, then defaults

`@ConfigurationProperties` takes all properties under one prefix and loads them into a **typed object** at startup. A record works well for this. It also:

- **converts values** — durations like `10s`, data sizes like `10MB`,

- **validates values** — with `@Validated`.

So bad config fails at startup, not at 2 a.m. in production.

Profiles (`spring.profiles.active=prod`) switch on profile-specific files, and beans marked `@Profile("prod")`.

Think of a mobile phone. The phone is the same everywhere. When you change the SIM card, it picks up a different network and number. The JAR is the phone; the config is the SIM.

How does it work?

```yaml
# application.yml
spring:
  application.name: loyalty-service
  datasource:
    url: jdbc:postgresql://localhost:5432/loyalty
    username: ${DB_USER:loyalty}            # env var with a default
    password: ${DB_PASSWORD}                # must come from the environment/secret store
  jpa.open-in-view: false
loyalty:
  redemption:
    max-points-per-day: 50000
    hold-timeout: 10m
---
spring.config.activate.on-profile: prod
logging.level.root: WARN
```

Key points in this file:

- `${DB_USER:loyalty}` reads an environment variable, and uses `loyalty` if it is not set.
- `${DB_PASSWORD}` has no default. It must come from the environment or a secret store.
- The part after `---` applies only when the `prod` profile is active.

Example

```java
@ConfigurationProperties(prefix = "loyalty.redemption")
@Validated
public record RedemptionProps(@Positive long maxPointsPerDay, @NotNull Duration holdTimeout) {}

@SpringBootApplication
@ConfigurationPropertiesScan
public class LoyaltyApplication { }

@Service
class RedemptionLimits {
    private final RedemptionProps props;
    RedemptionLimits(RedemptionProps props) { this.props = props; }
    boolean allowed(long todaysTotal, long requested) { return todaysTotal + requested <=
props.maxPointsPerDay(); }
}

@Configuration
class SmsConfig {
    @Bean @Profile("!prod") SmsSender fakeSms() { return (to, msg) -> System.out.println("SMS to " + to + ":
" + msg); }
    @Bean @Profile("prod")  SmsSender realSms(GupshupProps p) { return new GupshupSmsSender(p); }
}
```

- `RedemptionProps` is a typed, validated record for everything under `loyalty.redemption`. A negative limit or a missing timeout stops startup.
- `@ConfigurationPropertiesScan` tells Spring to find such records.

- `SmsConfig` gives a fake SMS sender in every profile except `prod`, and the real Gupshup sender in `prod`.

In Kubernetes:

- **Non-secret config** comes through ConfigMaps — as mounted files or environment variables.
- **Secrets** come through Kubernetes Secrets or Azure Key Vault. Spring Cloud Azure can load Key Vault secrets as a property source.

When would I use it?

For every value that changes per environment: URLs, timeouts, limits, feature flags, pool sizes and credentials.

Prefer typed `@ConfigurationProperties` over many scattered `@Value("${...}")` strings.

What can go wrong?

- **Secrets committed** to `application.yml` in Git. Use environment variables or a vault instead.
- **A typo in a property name is silently ignored.** This happens with `@Value` plus a default, or with properties that nothing binds to. Validated properties records catch type errors. Add the configuration processor to get IDE hints for property names.
- **Too many profiles that combine in surprising ways** (`dev,local,kafka-off`). Keep only a few profiles, one per environment.
- **Different values in different pods** because of a stale ConfigMap. A config change needs a rollout or a refresh mechanism to reach every pod.

Interview questions

Q1. How does Spring Boot resolve a property that's defined in several places? By property-source precedence. Command-line arguments beat environment variables. Environment variables beat profile-specific files. Profile-specific files beat application.yml.

Q2. `@Value` vs `@ConfigurationProperties`? `@Value` injects one value at a time by its string key. `@ConfigurationProperties` loads a whole prefix into a typed object that can be validated. It is better for groups of related settings.

Q3. How do you manage secrets? Never in source control. Inject them through environment variables from Kubernetes Secrets, or from a vault (Azure Key Vault, HashiCorp Vault), and rotate them regularly.

6.9 Auto-configuration and starters

Why?

> **In simple words:** Boot looks at which libraries you added and sets up the matching objects for you. If you define your own version of an object, Boot sees it and steps back.

Without Boot, every app wrote the same 200 lines of setup:

- a DataSource and a connection pool,
- an EntityManagerFactory,
- a transaction manager,
- Jackson's ObjectMapper,

- the DispatcherServlet,
- Tomcat...

Auto-configuration does this for you when it sees that you need it. And it **steps aside** the moment you define your own.

Think of a hotel room. The basic things — bed, towel, kettle — are already set up when you arrive. If you bring your own kettle, housekeeping doesn't add a second one.

What is happening internally?

1. `@EnableAutoConfiguration` reads a file inside each JAR on the classpath: `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`. It lists the candidate configuration classes. The spring-boot-autoconfigure JAR alone lists about 150.
2. Each candidate is an `@AutoConfiguration` class. It is guarded by **conditions** that are checked at startup:

Condition	The config applies when...
`@ConditionalOnClass(DataSource.class)`	that class is on the classpath (a driver or library is present)
`@ConditionalOnMissingBean(DataSource.class)`	you haven't defined your own bean of that type
`@ConditionalOnProperty("spring.kafka.bootstrap-servers")`	a property is set (or has a given value)
`@ConditionalOnBean`	some other bean exists
`@ConditionalOnWebApplication`	it's a web app

3. Auto-configurations are processed **after** your own configuration. So `@ConditionalOnMissingBean` can already see your beans, and it backs off.
4. Properties classes feed values into the auto-configured beans. For example, `DataSourceProperties` is bound to `spring.datasource.*`.

How does it work?

Here is a real chain. You add `spring-boot-starter-data-jpa` and the Postgres driver. Then:

1. `DataSourceAutoConfiguration` creates a HikariCP connection pool from `spring.datasource.*`.
2. `HibernateJpaAutoConfiguration` creates the EntityManagerFactory and a `JpaTransactionManager`.
3. `JpaRepositoriesAutoConfiguration` scans for repository interfaces and creates proxies for them.

If you define your own `DataSource` bean, step 1 backs off.

To debug what was configured:

- Run with `--debug`. This prints the **condition evaluation report**. It lists positive matches (applied) and negative matches (skipped), each with a reason.
- Or call the Actuator endpoint `/actuator/conditions`.

To switch something off explicitly: `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)`.

Example

```java
// Your own starter: shared multi-tenancy support for all services in the platform
@AutoConfiguration
@ConditionalOnClass(TenantResolver.class)
@EnableConfigurationProperties(TenantProps.class)
public class TenantAutoConfiguration {
    @Bean @ConditionalOnMissingBean
    TenantResolver tenantResolver(TenantProps props) { return new HeaderTenantResolver(props.header()); }

    @Bean @ConditionalOnMissingBean
    FilterRegistrationBean<TenantFilter> tenantFilter(TenantResolver resolver) {
        var reg = new FilterRegistrationBean<>(new TenantFilter(resolver));
        reg.setOrder(Ordered.HIGHEST_PRECEDENCE + 10);
        return reg;
    }
}
// + one line in META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports:
//   com.fa.platform.tenant.TenantAutoConfiguration
```

This is your own starter. It gives every service on the platform the same multi-tenancy support:

- It only applies when `TenantResolver` is on the classpath.
- Each bean has `@ConditionalOnMissingBean`, so a service can still replace it.
- You register it with one line in the `AutoConfiguration.imports` file.

A platform team publishing a "tenant starter" is a great senior-level example to mention in interviews. Every service then gets the same tenant header handling, MDC tagging (adding the tenant to every log line) and Hibernate filter.

When would I use it?

- **Every day, as a user.** Read the condition report when something is configured (or not configured) unexpectedly.
- **Write your own starter** for conventions shared across services: tenancy, auth, log format, standard error responses.

What can go wrong?

- **Surprise auto-config from a transitive dependency** (a library pulled in by another library). For example, Spring Security on the classpath suddenly protects every endpoint with a generated password. Or a Kafka client starts trying to connect.
- **Defining a bean with the "wrong" type or name.** The auto-config doesn't recognise it, so it doesn't back off. You end up with two beans.
- **Startup failure "Failed to configure a DataSource".** JPA is on the classpath, but no database URL is set.

Interview questions

Q1. How does Spring Boot auto-configuration work? Starters put auto-configuration classes on the classpath, listed in `AutoConfiguration.imports`. Each class applies only if its @Conditional checks pass. It backs off if you have already defined that bean yourself.

Q2. How do you see why something was or wasn't auto-configured? Run with `--debug` to get the condition evaluation report, or use Actuator's `/conditions` endpoint.

Q3. How do you disable a specific auto-configuration? Use `@SpringBootApplication(exclude = …)` or `spring.autoconfigure.exclude` in properties. Or simply define your own bean of that type.

6.10 Startup sequence and application events

Why?

> **In simple words:** Know what Spring does, step by step, from `java -jar` until the app is ready. And learn Spring events: one part of your app announces "this happened", and other parts react, without the two knowing about each other.

Knowing what happens between `java -jar` and "Started in 4.2 seconds" helps you:

- debug startup failures,
- put initialisation code in the right place,
- make startup faster,
- and decouple parts of your app with events (inside one process).

What is happening internally?

1. `SpringApplication.run` decides the app type (servlet, reactive, or none). It loads initializers and listeners.
2. It prepares the `Environment` (property sources and profiles) and prints the banner.
3. It creates the `ApplicationContext`.
4. **Bean definition loading.** Component scan, `@Configuration` processing and auto-configuration all run. These are done by BeanFactoryPostProcessors.
5. **Singleton instantiation.** Spring resolves the dependency graph. Beans are created, injected, initialised and proxied (by BeanPostProcessors).
6. The embedded web server starts. The port opens.
7. `ApplicationStartedEvent` fires. Then the `ApplicationRunner` / `CommandLineRunner` beans run. Then `ApplicationReadyEvent` fires, and readiness becomes UP.
8. On shutdown: stop accepting new traffic (graceful), destroy beans, close pools.

Application events are a publish–subscribe system inside one process.

- You call `ApplicationEventPublisher.publishEvent(obj)`.
- Spring calls every matching `@EventListener` method.
- By default this is **synchronous, on the same thread**. If a transaction is open, the listener runs inside that same transaction.

Think of the announcement system at a railway station. The station master announces once, and every passenger waiting for that train reacts. He doesn't need to know who they are.

How does it work?

`@TransactionalEventListener(phase = AFTER_COMMIT)` waits until the surrounding transaction commits. If the transaction rolls back, the listener is skipped.

That makes it the correct place to send notifications about database changes.

Example

```
public record PointsRedeemed(String accountId, long points, String redemptionId) {}

@Service
class RedemptionService {
    private final ApplicationEventPublisher events;
```

```
    RedemptionService(ApplicationEventPublisher events) { this.events = events; }
    @Transactional
    public void redeem(String acc, long pts) {
        // ... debit ledger, save redemption ...
        events.publishEvent(new PointsRedeemed(acc, pts, "R-123"));   // listener runs only if we commit
    }
}

@Component
class RedemptionNotifier {
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    @Async("notificationExecutor")                                    // don't block the request thread
    void on(PointsRedeemed e) { smsSender.send(e.accountId(), "Redeemed " + e.points() + " points"); }
}
```

What happens here:

1. `redeem` runs inside a transaction and publishes `PointsRedeemed`.
2. The listener does not run yet. It waits for the commit.
3. If the transaction commits, the listener runs. If it rolls back, the listener never runs.
4. `@Async("notificationExecutor")` runs the SMS on another thread pool, so the request thread is not blocked.

When would I use it?

To separate side effects from core logic inside one service: notifications, audit, cache eviction (removing stale cache entries) and metrics.

For **cross-service** reliability, in-process events are not enough. If the process crashes right after the commit, the in-process event is lost. When the event must reach Kafka no matter what, use the transactional **outbox** pattern (11.15).

What can go wrong?

- **Synchronous listeners hurting the publisher.** A slow listener slows the publisher's request. An exception in a plain `@EventListener` goes back up to the publisher and can roll back its transaction.
- **`@TransactionalEventListener` with no active transaction.** By default the event is silently dropped. Set `fallbackExecution = true` if you need it to run anyway.
- **Slow startup from eagerly creating heavy beans.** Measure it with the `/actuator/startup` endpoint (with `BufferingApplicationStartup`). Consider lazy init for beans that aren't critical, and keep `@PostConstruct` light.

Interview questions

Q1. What happens when a Spring Boot application starts? Spring prepares the environment, creates the context, and loads bean definitions (scan, config, auto-config). Then it creates the singletons, running post-processors and creating proxies. It starts the embedded server, runs the runners, and finally fires ApplicationReadyEvent.

Q2. Are Spring events synchronous? Yes, by default — they run on the publisher's thread. Add `@Async` to handle them asynchronously. `@TransactionalEventListener` ties delivery to a transaction phase, such as after commit.

Q3. Why use `@TransactionalEventListener(AFTER_COMMIT)` for notifications? So users are never told about changes that later roll back. For guaranteed delivery across services, use an outbox instead.

7. Spring Web and Data: REST, Validation, JPA, Transactions

7.1 How a request flows through Spring MVC

Why?

In simple words: A request does not jump straight into your controller. It passes through a fixed line of "counters" — Tomcat, filters, the DispatcherServlet, interceptors — and if you know that line, you can find where any problem happens.

Sometimes a request is slow. Sometimes it returns the wrong status code. Sometimes it skips your validation, or it never reaches your controller at all. To debug any of these, you must know every component the request passes through, and in which order.

Interviewers ask "What happens when a request hits a Spring Boot app?" to check exactly this.

What is happening internally?

```
HTTP request
 → Tomcat connector: accepts the socket; a worker thread from Tomcat's pool (default max 200) takes the
request
 → Servlet FILTER chain (in order): CharacterEncoding, Spring Security's FilterChainProxy, your
tenant/correlation filters, CORS
 → DispatcherServlet (the "front controller", one servlet for the whole app)
     1. HandlerMapping: find the handler for METHOD + PATH (RequestMappingHandlerMapping matches @GetMapping
etc.)
     2. HandlerInterceptor.preHandle (each interceptor; can short-circuit)
     3. HandlerAdapter invokes the controller method:
          argument resolvers: @PathVariable, @RequestParam, @RequestHeader, @RequestBody
             (@RequestBody → HttpMessageConverter → Jackson ObjectMapper deserialises JSON)
          validation: @Valid / @Validated → Bean Validation
     4. Controller → service → repository … returns a value
     5. Return value handler: HttpMessageConverter serialises it to JSON (content negotiation via Accept)
     6. HandlerInterceptor.postHandle / afterCompletion
     7. Exceptions anywhere in 3–5 → HandlerExceptionResolvers → @ControllerAdvice @ExceptionHandler
 → back through filters → response bytes on the socket
```

Here is the same flow in plain steps:

1. **Tomcat** (the web server inside Spring Boot) accepts the connection. It gives the request to one worker thread from its thread pool. The default maximum is 200 threads.
2. **Filters** run one after another. A *filter* is code that runs for every request before Spring MVC sees it — for example, security checks or reading the tenant ID.
3. The **DispatcherServlet** receives the request. It is the *front controller* — one single entry point for the whole app that sends each request to the right place.
4. Inside it: find the controller method, run interceptors, convert JSON into Java objects, validate them, call your code, convert the result back to JSON.
5. If anything throws an exception, your `@ControllerAdvice` handler turns it into an error response.
6. The response goes back out through the filters and onto the network.

One request = one Tomcat thread for its whole life. This is true unless you use async MVC, WebFlux or virtual threads.

So every blocking call — a DB query, an HTTP call to another service — keeps that thread busy while it waits. Because of this, timeouts on outbound calls and pool sizes matter a lot. Think of a bank with 200 counters: if every clerk is stuck waiting on a phone call, new customers stand in the queue.

How does it work?

- `HandlerMapping` builds its lookup table once, at startup, from all `@RequestMapping` methods. So if two methods map to the same URL and method, the app fails at startup — not later.
- *Content negotiation* means choosing the format of the data. Spring looks at the `Accept` header (what the client wants back) and the `Content-Type` header (what the client sent). Based on these, it picks a converter. For JSON, Jackson is used by default.
- The HTTP status code comes from one of three places:
- the return type (`ResponseEntity`),
- the `@ResponseStatus` annotation,
- or an exception handler.

Example

```
POST /api/v1/accounts/A1/redemptions  {"points": 500, "rewardCode": "AMZ100"}
1. Tomcat thread http-nio-8080-exec-12 picks it up
2. SecurityFilterChain validates the JWT → SecurityContext set
3. TenantFilter reads X-Tenant-Id → TenantContext (ThreadLocal)
4. HandlerMapping → RedemptionController.redeem(String, String, RedeemRequest)
5. Jackson builds RedeemRequest; @Valid checks @Positive points → OK
6. Service runs in a @Transactional proxy → DB → returns RedemptionResponse
7. Jackson writes JSON, ResponseEntity sets 201 + Location
8. Filters' finally blocks clear TenantContext and MDC
```

Note step 8. Tomcat threads are reused for the next request. So the filters must clear `TenantContext` and `MDC` in a `finally` block. Otherwise the next request on that thread could see the previous tenant's data.

When would I use it?

Use this knowledge when you are:

- Debugging a **404** (no controller matched the URL).
- Debugging a **415** (the client sent the wrong `Content-Type`).
- Debugging a **406** (the client's `Accept` header asks for a format you can't produce).
- Finding out why validation did not run.
- Fixing filter order problems — for example, auth must run before tenant resolution.
- Planning capacity. The number of requests you can handle depends on threads × time per request.

What can go wrong?

- **Slow downstream calls use up all Tomcat threads.** New requests then wait in a queue (the size is set by *accept count*). When that queue is full too, new connections are refused.
- **A filter forgets to call `chain.doFilter`**, so the request stops there. Or a filter does not clean up its ThreadLocals in `finally`, so data leaks to the next request on the same thread.
- **Large request bodies are read fully into memory**, which can cause memory pressure.

Interview questions

Q1. What happens when a request hits a Spring Boot app? Tomcat gives the request a thread from its pool. Then servlet filters run, such as security. Next the DispatcherServlet takes over: HandlerMapping finds the controller method, interceptors run, and the HandlerAdapter prepares the method arguments — Jackson turns the JSON into objects and validation runs. Then the controller runs, and a message converter writes the response as JSON. If an exception is thrown anywhere, @ControllerAdvice handles it.

Q2. What is the DispatcherServlet? It is Spring MVC's front controller — one single servlet that receives every request. It sends each request to the right handler using mappings, adapters, resolvers and converters.

7.2 REST controllers and HTTP semantics

Why?

> **In simple words:** Your API is a promise to everyone who calls it. If you use the right URLs, HTTP methods and status codes, callers know what to expect and can safely retry. If you use the wrong ones, you get duplicate orders and confused clients.

An API is a contract. Mobile apps, partner systems and other services all depend on it.

When resources, methods and status codes are correct, the API is:

- **predictable** — callers know what each call does,
- **cacheable** — GET responses can be cached,
- **safe to retry** — a retry does not cause damage.

When they are wrong, you get duplicate orders, broken clients and confusing support tickets.

What is happening internally?

- Methods in a `@RestController` return normal Java objects. Spring converts these to JSON for you. This conversion is called *serialisation*.
- `ResponseEntity<T>` gives you full control of the response: the status, the headers and the body.
- Method parameters get their values from different parts of the request:
- the URL path → `@PathVariable`,
- the query string (`?page=2`) → `@RequestParam`,
- the headers → `@RequestHeader`,
- the JSON body → `@RequestBody`.
- For paging, Spring can fill a `Pageable` object from the `page`, `size` and `sort` parameters.

How does it work?

First, one important word. *Idempotent* means: doing the same call two times has the same effect as doing it once. Deleting order 5 twice still leaves order 5 deleted. *Safe* means the call changes nothing at all.

Method	Meaning	Idempotent?	Typical success status
GET	Read	Yes (and safe — changes nothing)	200

Method	Meaning	Idempotent?	Typical success status
POST	Create / action	**No** (make it idempotent with an idempotency key)	201 + Location, or 202 for async
PUT	Replace the whole resource	Yes	200 / 204
PATCH	Update only some fields	Not guaranteed	200 / 204
DELETE	Remove	Yes	204

Error status codes:

- **400** — the request is malformed or fails validation.
- **401** — the caller is not logged in (unauthenticated).
- **403** — the caller is logged in but not allowed (forbidden).
- **404** — not found.
- **409** — conflict, such as a duplicate or a version clash.
- **422** — the input is valid, but a business rule says no.
- **429** — too many requests (rate limited).
- **500** — a bug on the server.
- **502 / 503 / 504** — a service you depend on has a problem.

Example

```
@RestController
@RequestMapping("/api/v1/accounts/{accountId}/redemptions")
class RedemptionController {
    private final RedemptionService service;
    RedemptionController(RedemptionService service) { this.service = service; }

    @PostMapping
    ResponseEntity<RedemptionResponse> redeem(@PathVariable String accountId,
                                              @RequestHeader("Idempotency-Key") String idemKey,
                                              @Valid @RequestBody RedeemRequest req) {
        RedemptionResponse r = service.redeem(TenantContext.current(), accountId, idemKey, req.points());
        return ResponseEntity.created(URI.create("/api/v1/accounts/%s/redemptions/%s".formatted(accountId,
r.id())))
                             .body(r);
    }

    @GetMapping
    List<RedemptionResponse> list(@PathVariable String accountId,
                                  @RequestParam(required = false) Long afterId,          // keyset cursor
                                  @RequestParam(defaultValue = "20") @Max(100) int size) {
        return service.list(accountId, afterId, size);
    }
}

public record RedeemRequest(@Positive long points, @NotBlank String rewardCode) {}
public record RedemptionResponse(String id, long points, String status, Instant createdAt) {}
```

Points to notice:

- The POST returns **201 Created** with a `Location` header that points to the new redemption.
- The POST requires an `Idempotency-Key` header, so a retry is safe.
- The GET uses a cursor (`afterId`) for paging and limits `size` to at most 100.
- The request and response are small `record` DTOs.

When would I use it?

In every API.

Always return **DTO records** — small classes made only for the API. Never return JPA entities. Entities cause three problems:

1. They leak internal fields to the outside world.
2. They trigger lazy loading while JSON is being written.
3. They tie your API to your database schema, so a schema change breaks clients.

What can go wrong?

- **Returning entities.** While Jackson writes JSON, it touches lazy fields. This causes a `LazyInitializationException`, or N+1 queries (one query for the list, plus one more for each item). Two-way relationships cause infinite recursion. Internal columns get exposed.
- **Returning 200 for everything**, with an `"error"` field in the body. Clients and monitoring tools then can't tell a failure from a success.
- **Non-idempotent POSTs retried by mobile apps.** On a flaky network the app retries, and you get duplicate redemptions. Require an `Idempotency-Key` header (11.14).
- **Offset paging on huge tables.** `OFFSET 1000000` makes the DB walk past a million rows — slow. Results also shift when new rows arrive. Use keyset/cursor paging instead ("give me rows after ID X").

Interview questions

Q1. PUT vs PATCH vs POST? PUT replaces the whole resource and is idempotent — repeating it gives the same result. PATCH updates only some fields. POST creates something or triggers an action, and it is not idempotent unless you add an idempotency key.

Q2. Which status code for a business-rule failure such as insufficient points? 422 Unprocessable Entity — the input is fine but the rule says no. Use 409 if it is a conflict with the current state. 400 is for input that is malformed or invalid.

Q3. Why not return JPA entities from controllers? Because of lazy-loading exceptions, accidental N+1 queries, infinite recursion on two-way mappings, leaking internal fields, and tying the API to the database schema.

7.3 Validation

Why?

> **In simple words:** Never trust input from outside. Put simple annotations like `@NotBlank` on your request class, and Spring checks every request for you — and sends back a clear 400 error if something is wrong.

Every input from outside must be checked before it reaches business logic or the database. "Outside" means mobile apps, partners, admin panels — anything.

Declarative validation means you declare the rules as annotations instead of writing `if` checks by hand. This has three benefits:

- The rules live next to the DTO, so they are easy to find.

- They are consistent and easy to test.
- Bad input gets a clear 400 error. Without this, the bad data goes deep into the code and fails later with a confusing 500.

What is happening internally?

Spring does not validate by itself. It hands the job to **Bean Validation** — the standard Java validation API (Jakarta Validation). The library that actually does the work is Hibernate Validator.

The steps for a request body:

1. Jackson converts the JSON into your object.
2. Because the parameter has `@Valid`, Spring now validates the object.
3. Each constraint annotation (like `@NotBlank`) has a matching `ConstraintValidator` class that does the check.
4. If any check fails, Spring throws `MethodArgumentNotValidException`.

For `@PathVariable` and `@RequestParam` constraints, it is different. You need `@Validated` on the class. Failures then throw `HandlerMethodValidationException` (Spring 6.1+) or `ConstraintViolationException`.

If you put `@Valid` on a nested field, validation also goes inside that nested object. This is called *cascading*.

How does it work?

```
public record CreateCampaignRequest(
        @NotBlank @Size(max = 80) String name,
        @NotNull @FutureOrPresent LocalDate startDate,
        @NotNull LocalDate endDate,
        @NotEmpty List<@NotBlank String> skuCodes,          // container element constraints
        @Valid @NotNull RewardSpec reward) {                // cascade into the nested object

    @AssertTrue(message = "endDate must be after startDate")
    boolean isRangeValid() { return startDate == null || endDate == null || endDate.isAfter(startDate); }
}
public record RewardSpec(@NotNull RewardType type, @Positive long value) {}
```

What this shows:

- Simple field rules: `@NotBlank`, `@Size`, `@NotNull`, `@FutureOrPresent`.
- Rules on list items: `List<@NotBlank String>` checks every SKU code in the list.
- `@Valid` on `reward` means the rules inside `RewardSpec` are also checked.
- `@AssertTrue` on a method checks a rule that needs two fields together — the end date must be after the start date.

Example

```
// Custom constraint: Indian mobile number
@Target({ElementType.FIELD, ElementType.PARAMETER, ElementType.RECORD_COMPONENT})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = IndianMobileValidator.class)
public @interface IndianMobile {
    String message() default "must be a valid Indian mobile number";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
public class IndianMobileValidator implements ConstraintValidator<IndianMobile, String> {
    private static final Pattern P = Pattern.compile("^(\\+91)?[6-9]\\d{9}$");
    @Override public boolean isValid(String v, ConstraintValidatorContext ctx) { return v == null ||
```

```
P.matcher(v).matches(); }
}
public record RetailerSignup(@NotBlank String shopName, @NotNull @IndianMobile String mobile) {}
```

To make a custom rule, you need two pieces:

1. An annotation (`@IndianMobile`) marked with `@Constraint(validatedBy = …)`.
2. A validator class that implements `isValid`.

Note that `isValid` returns `true` for `null`. This is the usual habit: let `@NotNull` handle "missing", and let the custom rule handle only "wrong format".

Validation groups (`@Validated(OnCreate.class)`) let one DTO have different rules for create and update. But usually it is clearer to write a separate request record for each operation.

When would I use it?

On all request DTOs, and on `@ConfigurationProperties` classes (so bad config fails at startup).

Keep **business rules that need data** in the service layer, not in annotations. For example, "points ≤ balance" needs the account's balance from the DB, so it belongs in the service.

What can go wrong?

- **Forgetting `@Valid`.** The constraints are then silently ignored — no error, no check.
- **Validating only on the client** (the app or web page). Anyone can call the API directly with curl or Postman and skip those checks.
- **Putting DB lookups inside validators.** They become slow and hard to test, and they hide business logic in a place nobody looks.
- **Sending the raw validation exception text to clients.** Section 7.4 shows how to shape it into a clean response.

Interview questions

Q1. How does `@Valid` work in Spring MVC? After Jackson turns the body into an object, Spring calls Bean Validation on it. Any broken rule throws MethodArgumentNotValidException, which you map to a 400 response.

Q2. How do you write a custom validator? Create a constraint annotation marked with `@Constraint(validatedBy = …)`, and write a class that implements `ConstraintValidator<A, T>.isValid`.

Q3. Where should business-rule validation live? In the service or domain layer, because that layer has the data and context to decide. Annotations are only for checking the shape of the input.

7.4 Centralised exception handling

Why?

> **In simple words:** Instead of every controller catching and formatting errors in its own way, you write one class that catches all exceptions. It decides the status code, builds a clean error body, and logs real bugs once.

Without one central handler, three bad things happen:

1. Every controller catches and formats errors differently.

2. Stack traces leak to clients.
3. Unexpected exceptions become plain 500s with no useful information.

A single `@RestControllerAdvice` class fixes all of this. It maps each exception type to the right status code and a consistent body. It also logs unexpected errors exactly once.

What is happening internally?

When a controller method throws an exception, this happens:

1. `DispatcherServlet` asks its `HandlerExceptionResolver`s to deal with it.
2. One of them, `ExceptionHandlerExceptionResolver`, looks for the best matching `@ExceptionHandler` for that exception type.
3. It looks first inside the controller itself. Then it looks in `@ControllerAdvice` beans, in the order set by `@Order`. It picks the most specific match.
4. It calls that handler method.
5. Whatever the handler returns is written out like a normal response.

Spring 6 gives you **`ProblemDetail`**. It is the standard error body format from RFC 9457. It has the fields `type`, `title`, `status`, `detail` and `instance`, and you can add your own extra properties.

How does it work?

```
@RestControllerAdvice
class GlobalExceptionHandler {
    private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ProblemDetail validation(MethodArgumentNotValidException ex) {
        ProblemDetail pd = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
        pd.setTitle("Validation failed");
        pd.setProperty("errors", ex.getBindingResult().getFieldErrors().stream()
                .map(f -> Map.of("field", f.getField(), "message",
String.valueOf(f.getDefaultMessage()))).toList());
        return pd;
    }

    @ExceptionHandler(InsufficientPointsException.class)
    ProblemDetail insufficient(InsufficientPointsException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.UNPROCESSABLE_ENTITY,
ex.getMessage());
        pd.setTitle("Insufficient points");
        pd.setType(URI.create("https://errors.example.com/insufficient-points"));
        return pd;
    }

    @ExceptionHandler({ObjectOptimisticLockingFailureException.class, DataIntegrityViolationException.class})
    ProblemDetail conflict(Exception ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, "Concurrent or duplicate update, please
retry");
    }

    @ExceptionHandler(Exception.class)
    ProblemDetail unexpected(Exception ex, HttpServletRequest req) {
        log.error("Unhandled error on {} {}", req.getMethod(), req.getRequestURI(), ex);   // log the stack
ONCE, here
        return ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR, "Unexpected error");
    }
}
```

What each handler does:

- **Validation failure** → 400, with a list of which field failed and why.

- `InsufficientPointsException` (a business rule) → 422, with a clear title and a `type` link.
- **Optimistic lock or unique-key clash** → 409, "please retry".
- **Anything else** → 500 with a generic message. The full stack trace is logged here, once — and never sent to the client.

Example

```
HTTP/1.1 422
{ "type": "https://errors.example.com/insufficient-points", "title": "Insufficient points",
  "status": 422, "detail": "Account A1 requested 500 but has 120", "instance":
"/api/v1/accounts/A1/redemptions" }
```

When would I use it?

In every service.

Pair it with domain exceptions — your own unchecked exceptions (1.15) such as `InsufficientPointsException`. Services throw them, and the advice turns them into responses. So controllers stay free of try/catch.

What can go wrong?

- **Wrong order of handlers.** A catch-all handler wins over specific ones because of class order, or you have several advice classes without `@Order`. Then the same error gets different responses.
- **Logging and rethrowing in every layer.** The same stack trace then appears five times in the logs.
- **Leaking internals in `detail`** — SQL, class names or stack traces.
- **Exceptions thrown inside filters.** Filters run before the DispatcherServlet. So `@ControllerAdvice` never sees these exceptions. Handle them inside the filter itself, or in Spring Security's entry points.

Interview questions

Q1. How do you handle exceptions globally in Spring? With a `@RestControllerAdvice` class that has `@ExceptionHandler` methods. Each one maps an exception type to a status code and a ProblemDetail body. Unexpected errors are logged once, there.

Q2. Why doesn't my @ControllerAdvice catch an exception thrown in a filter? Filters run before the DispatcherServlet, so Spring's exception resolvers are not involved yet. Handle it in the filter itself or in the security entry point.

7.5 Filters vs interceptors

Why?

> **In simple words:** Both let you run code for every request. A filter sits at the building's main gate and sees everyone. An interceptor sits at the office door, after Spring already knows which controller method will handle the request.

Some logic must run for every request. Examples: authentication, finding the tenant, adding correlation IDs, request logging, rate limiting.

You can put such logic in a Servlet **filter** or a Spring MVC **interceptor**. The right choice depends on two things: what the logic needs to see, and when it must run.

What is happening internally?

- A **Filter** (`jakarta.servlet.Filter`, usually written by extending Spring's `OncePerRequestFilter`) runs in the servlet container, **around** the DispatcherServlet.
- It runs for every request — including static files and error pages.
- It can wrap or replace the request and response objects.
- It can stop the chain, so the request goes no further.
- A **HandlerInterceptor** runs **inside** the DispatcherServlet, after the controller method has been chosen.
- It knows which handler method will run, and can read its annotations.
- It has three hooks: `preHandle` (before the controller), `postHandle` (after the controller) and `afterCompletion` (after the response is done).

```
Filter A → Filter B → DispatcherServlet → [Interceptor.preHandle] → Controller → [postHandle] → view/JSON → [afterCompletion] → Filter B → Filter A
```

How does it work?

	Filter	Interceptor
Level	Servlet container	Spring MVC
Sees	Raw request/response (can wrap them)	The handler method and its annotations
Runs for	Everything	Only requests that map to a handler
Typical use	Security, tenant, correlation ID, CORS, compression, request/response logging	Auth checks per endpoint based on annotations, metrics per handler, locale

Example

```
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
class CorrelationIdFilter extends OncePerRequestFilter {
    @Override protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String id = Optional.ofNullable(req.getHeader("X-Request-Id")).orElse(UUID.randomUUID().toString());
        MDC.put("requestId", id);
        res.setHeader("X-Request-Id", id);
        try { chain.doFilter(req, res); }
        finally { MDC.remove("requestId"); }                  // pooled threads: always clean up
    }
}

@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.METHOD) @interface RequiresFeature { String value(); }

@Component
class FeatureFlagInterceptor implements HandlerInterceptor {
    private final FeatureFlags flags;
    FeatureFlagInterceptor(FeatureFlags flags) { this.flags = flags; }
    @Override public boolean preHandle(HttpServletRequest req, HttpServletResponse res, Object handler)
throws IOException {
        if (handler instanceof HandlerMethod hm) {
            RequiresFeature rf = hm.getMethodAnnotation(RequiresFeature.class);
            if (rf != null && !flags.enabled(TenantContext.current(), rf.value())) { res.sendError(404);
return false; }
        }
        return true;
    }
}
```

```
@Configuration
class WebConfig implements WebMvcConfigurer {
    private final FeatureFlagInterceptor ffi;
    WebConfig(FeatureFlagInterceptor ffi) { this.ffi = ffi; }
    @Override public void addInterceptors(InterceptorRegistry r) {
r.addInterceptor(ffi).addPathPatterns("/api/**"); }
}
```

What the code does:

- `CorrelationIdFilter` runs first for every request. It takes the `X-Request-Id` header, or makes a new UUID. It puts the ID in the logging context (MDC) and in the response header. In `finally`, it removes the ID — because Tomcat threads are reused, and the next request must not see the old ID.
- `FeatureFlagInterceptor` reads the `@RequiresFeature` annotation on the chosen controller method. If the feature is off for this tenant, it returns 404 and stops. A filter could not do this, because a filter doesn't know which method will run.
- `WebConfig` registers the interceptor for paths under `/api/**`.

When would I use it?

- Use a **filter** for anything that must happen before Spring MVC, or for all traffic — security, tenancy, tracing.
- Use an **interceptor** for logic that depends on which handler was chosen.

What can go wrong?

- **Wrong filter order.** For example, tenant resolution runs before security has checked who the caller is. Or it runs after another filter that already needed the tenant.
- **Reading the request body in a filter.** The body is a stream, and reading it uses it up. Then the controller gets an empty body. Wrap the request with `ContentCachingRequestWrapper` to fix this.
- **Expecting interceptors to run for errors or static files.** They don't run for those, though you may expect them to.

Interview questions

Q1. Filter vs interceptor? A filter works at the servlet level. It runs for every request and can wrap the request and response. An interceptor works at the Spring MVC level. It runs around the chosen handler and can see its method and annotations.

Q2. Where would you put tenant resolution and why? In a filter, early in the chain. Then every later component — security rules, controllers, repositories — has the tenant context. Clear it in `finally` so the next request on that thread starts clean.

7.6 Calling other services: RestClient, timeouts and connection pools

Why?

> **In simple words:** Every time your service calls another service, it can get stuck waiting. Set timeouts, reuse connections, and keep one configured client per downstream — otherwise one slow service can bring yours down too.

Microservices call each other and partners all the time — pricing, inventory, SMS, payment gateways.

Every outbound call is a risk. Your service can hang there, leak connections, or spread someone else's outage to your own callers. So configuring the HTTP client correctly is basic production hygiene. It also comes up often in SDE2 interviews.

What is happening internally?

The client options in Spring:

- `RestClient` (Spring 6.1+) — the modern, synchronous client with a fluent (chained-method) style.
- `WebClient` — the reactive, non-blocking client.
- `RestTemplate` — the old client; it is in maintenance mode (only bug fixes).
- **HTTP interfaces** (`@HttpExchange`) — you write an annotated interface, and Spring generates the client for you.

Connection pools: Under the hood, an HTTP library does the real work — JDK `HttpClient`, Apache HttpClient 5 or Reactor Netty. It keeps a *connection pool* for each host — a set of open connections ready to reuse.

- Reusing a connection skips the TCP and TLS handshakes, which saves time.
- The pool's limit caps how many calls can go to one host at the same time.

Timeouts — there are three:

1. **Connect timeout** — how long to wait to open the TCP connection.
2. **Read/response timeout** — how long to wait for data after connecting.
3. **Pool-acquire timeout** — how long to wait for a free connection from the pool.

Without a read timeout, a stuck downstream service holds your thread forever.

How does it work?

Follow these rules:

1. Set timeouts **shorter than your own SLA** (the response time you promise).
2. Also keep them shorter than the timeout your caller uses for you. This is called a *timeout budget* — each hop gets a share of the total time.
3. Add retries only for idempotent calls — calls that are safe to repeat.
4. Add a circuit breaker for unhealthy dependencies (8.7). A circuit breaker stops calling a service that keeps failing, for a short while, like a fuse in your house.

Example

```
@Configuration
class ClientsConfig {
    @Bean
    RestClient pricingClient(RestClient.Builder builder) {
        var http = HttpClient.newBuilder().connectTimeout(Duration.ofMillis(500)).build();   // JDK client
        var factory = new JdkClientHttpRequestFactory(http);
        factory.setReadTimeout(Duration.ofSeconds(2));
        return builder.baseUrl("http://pricing-service")
                      .requestFactory(factory)
                      .defaultHeader("X-Caller", "loyalty-service")
                      .build();
    }
}

@Component
class PricingClient {
```

```
    private final RestClient client;
    PricingClient(RestClient pricingClient) { this.client = pricingClient; }

    Price price(String tenant, String sku) {
        return client.get().uri("/v1/prices/{sku}", sku)
                .header("X-Tenant-Id", tenant)
                .retrieve()
                .onStatus(HttpStatusCode::is4xxClientError, (req, res) -> { throw new
PriceNotFoundException(sku); })
                .body(Price.class);
    }
}

// Declarative HTTP interface (Spring 6)
interface InventoryApi {
    @GetExchange("/v1/stock/{sku}") Stock stock(@PathVariable String sku);
}
```

What the code does:

- One `RestClient` bean for the pricing service, with a 500 ms connect timeout and a 2 s read timeout.
- `PricingClient` wraps it. A 4xx response from pricing becomes your own `PriceNotFoundException`.
- `InventoryApi` shows the declarative style: Spring generates the client from the interface.

When would I use it?

For every outbound HTTP call.

Make one client bean per downstream service. Each one gets:

- its own timeouts and pool limits — this is a *bulkhead*, so one slow service can't use up the connections meant for others;
- observability — metrics, and tracing headers that pass along automatically when you build from the injected `RestClient.Builder`;
- resilience policies such as retries and circuit breakers.

What can go wrong?

- **No timeouts.** Your threads fill up with requests stuck on a slow dependency. Then the outage spreads to everyone who calls you.
- **Creating a new client for every request.** There is no connection reuse. You pay for a TLS handshake on every call, and you can run out of ports (sockets stuck in TIME_WAIT).
- **Retrying non-idempotent POSTs.** This causes duplicate payments or orders.
- **Not reading or closing response bodies.** The connection never goes back to the pool, so the pool slowly leaks.
- **Calling remote services inside a DB transaction.** The DB connection and its locks stay held while you wait on the network (7.12).

Interview questions

Q1. What timeouts do you configure on an HTTP client? Connect timeout, read/response timeout, and pool-acquisition timeout. All of them must fit inside the caller's latency budget.

Q2. RestTemplate vs RestClient vs WebClient? RestTemplate is the old synchronous client. RestClient is the modern synchronous client with a fluent style. WebClient is reactive and non-blocking, used with WebFlux or for high-concurrency async calls.

Q3. Why reuse HTTP clients? To reuse pooled TCP/TLS connections, avoid handshake costs and port exhaustion, and keep timeouts and limits in one central place.

7.7 JPA, Hibernate and Spring Data repositories

Why?

> **In simple words:** You write Java classes, and the library turns them into database tables and SQL for you. With Spring Data, you only write an interface; Spring writes the code behind it.

With plain JDBC (Java's low-level database API), you do everything by hand:

- You copy every column into every field, one by one.
- You open and close connections and statements yourself.
- You write the same insert, select, update and delete SQL for every table. This basic set is called *CRUD* (Create, Read, Update, Delete).

This is boring and easy to get wrong.

A tool that maps Java objects to database tables is called an *ORM* (Object-Relational Mapper). In Java:

- **JPA** is the specification — a set of rules and annotations. It is like an interface.
- **Hibernate** is the implementation of JPA — the real engine. It is like EF Core in .NET.

Hibernate maps Java objects to tables, generates the SQL, tracks what you changed and manages links between objects.

Spring Data JPA goes one step further. You only declare a repository *interface*. Spring writes the class that implements it for you.

Think of a tiffin service. You just say "veg thali, no onion" on the order slip. The kitchen knows how to cook it. You never enter the kitchen.

What is happening internally?

- **Entity**: a class marked with `@Entity`. It maps to one table.
- Hibernate needs a constructor with no arguments, so it can create the object itself.
- The class and its methods must not be `final`. Why? Hibernate creates a subclass at runtime (a *proxy*) to support lazy loading. A final class cannot be subclassed.
- **EntityManager** (called `Session` in Hibernate): the object that does the actual work with the database.
- It holds the **persistence context** (7.9) — a list of the objects loaded in this transaction.
- This pattern is called a *unit of work*: collect all changes, then write them together.
- Spring does not give you a raw EntityManager. It injects a shared proxy that finds the EntityManager of the current transaction.
- **Spring Data repositories**: at startup, Spring does these steps: 1. It scans for interfaces that extend `Repository`. 2. For each one, it creates a **JDK proxy** — a fake object generated at runtime that implements your interface. 3. Basic CRUD calls go to a ready-made class, `SimpleJpaRepository`. 4. Your own query methods get generated code. They come in a few kinds:
- **Derived queries**: Spring reads the method name and builds the query. For example, `findByTenantIdAndStatusOrderByCreatedAtDesc` is parsed into JPQL at startup. (JPQL is SQL-like,

but written against classes and fields, not tables and columns.) If you make a typo in the name, the app fails at startup. This is good — you find the bug early, not in production.

- `@Query`: you write the JPQL or native SQL yourself.
- **Projections**: an interface or record that selects only some columns, not the full entity.
- **Specifications**: build filters at runtime, for search screens where the user picks the filters.
- **ID generation** — how a new row gets its primary key:
- `IDENTITY`: the database auto-increments the ID. Hibernate does not know the ID until the row is inserted. So it must insert each row immediately. Because of this, **JDBC batch inserts are disabled** (no sending 50 inserts in one go).
- `SEQUENCE`: Hibernate fetches a block of IDs from a database sequence in advance. The block size is `allocationSize`. Since Hibernate already knows the IDs, it can batch inserts. This is the preferred choice on Postgres.
- `UUID`: the app makes the ID itself, with no need to ask the database. Use time-ordered UUIDv7. Its values grow over time, so new rows land next to each other in the index. This is called better index locality, and it makes inserts faster.

How does it work?

```
@Entity
@Table(name = "ledger_entry",
       indexes = @Index(name = "ix_ledger_account_created", columnList = "account_id, created_at"),
       uniqueConstraints = @UniqueConstraint(name = "uq_ledger_idem", columnNames = {"tenant_id",
"idempotency_key"}))
public class LedgerEntry {
    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "ledger_seq")
    @SequenceGenerator(name = "ledger_seq", sequenceName = "ledger_seq", allocationSize = 50)
    private Long id;

    @Column(name = "tenant_id", nullable = false)  private String tenantId;
    @Column(name = "account_id", nullable = false) private String accountId;
    @Enumerated(EnumType.STRING) @Column(nullable = false) private EntryType type;   // STRING, never ORDINAL
    @Column(nullable = false) private long points;
    @Column(name = "idempotency_key", nullable = false) private String idempotencyKey;
    @Column(name = "created_at", nullable = false) private Instant createdAt;

    protected LedgerEntry() {}                       // for JPA
    public LedgerEntry(String tenantId, String accountId, EntryType type, long points, String key, Instant
at) {
        this.tenantId = tenantId; this.accountId = accountId; this.type = type;
        this.points = points; this.idempotencyKey = key; this.createdAt = at;
    }
    // getters only: ledger rows are append-only
}
```

Example

```
public interface LedgerRepository extends JpaRepository<LedgerEntry, Long> {
    Optional<LedgerEntry> findByTenantIdAndIdempotencyKey(String tenantId, String key);          // derived

    @Query("select coalesce(sum(e.points), 0) from LedgerEntry e where e.accountId = :acc")
    long balance(@Param("acc") String accountId);                                                // JPQL

    List<LedgerEntry> findTop20ByAccountIdAndIdLessThanOrderByIdDesc(String accountId, long afterId);  //
keyset page

    @Query(value = """
        SELECT account_id AS accountId, SUM(points) AS total FROM ledger_entry
        WHERE tenant_id = :tenant AND created_at >= :since
        GROUP BY account_id ORDER BY total DESC LIMIT 10
        """, nativeQuery = true)
```

```
    List<TopAccount> top10(@Param("tenant") String tenant, @Param("since") Instant since);       // native +
projection

    @Modifying
    @Query("update Account a set a.status = 'DORMANT' where a.lastActivity < :cutoff")
    int markDormant(@Param("cutoff") Instant cutoff);                                            // bulk
update
}
interface TopAccount { String getAccountId(); long getTotal(); }
```

The repository interfaces build on each other. Each level adds more methods:

`Repository` → `CrudRepository` / `ListCrudRepository` → `PagingAndSortingRepository` → `JpaRepository`.

`JpaRepository` is the top level. It adds flush, batch delete and `getReferenceById`.

When would I use it?

Use it for domain services that do a lot of CRUD and have clear *aggregates* — a main object with its child objects, saved and loaded together. Examples: accounts, orders, campaigns.

Use something else for:

- complex reporting queries,
- bulk operations,
- anything where performance is critical.

For these, use native SQL, `JdbcTemplate` (Spring's thin JDBC helper) or jOOQ (a type-safe SQL builder). Mixing both approaches in one service is normal.

What can go wrong?

- **`ORDINAL` enums**: the enum is saved as its position number (0, 1, 2). If someone reorders the enum values later, old rows now point to the wrong value. Your data is silently corrupted. Always use `STRING`.
- **`IDENTITY` IDs**: they disable batch inserts, so bulk inserts become slow.
- **`ddl-auto=update` in production**: Hibernate changes the schema on its own at startup. Schema changes must go through versioned migrations instead (7.16).
- **Bulk `@Modifying` updates**: they run directly in the database and skip the persistence context. So entities you already loaded in memory now hold old values (they are *stale*). Fix: use `clearAutomatically = true`, or reload the entities.
- **Derived query names that grow silly**, like `findByAAndBOrCAndDOrderBy…`. Nobody can read them. Switch to `@Query`.
- **`findAll()` on a big table**: it loads every row into memory.

Interview questions

Q1. JPA vs Hibernate vs Spring Data JPA? JPA is the ORM specification — the rules and annotations. Hibernate is the implementation that actually does the work. Spring Data JPA sits on top of JPA and generates repository implementations from your interfaces.

Q2. How does Spring Data create a repository implementation from an interface? At startup, it creates a proxy for the interface, backed by SimpleJpaRepository for CRUD. It parses method names, or reads @Query annotations, and turns them into queries.

Q3. IDENTITY vs SEQUENCE generation? IDENTITY uses the database's auto-increment. Hibernate must insert each row immediately to learn its ID, so batching is disabled. SEQUENCE fetches IDs in advance in blocks, so Hibernate can batch the inserts.

7.8 Relationships and fetch types

Why?

> **In simple words:** You tell JPA how your objects are linked — an order has lines, a line belongs to an order. Then you can just call `order.getLines()`. But each link can secretly fire extra queries, so you must control how and when it loads.

Domain objects point to each other. For example:

- An order has many lines.
- An order belongs to an outlet.
- A scheme has many rules.

If you map these links, you can move from one object to another in plain Java code. This web of linked objects is called the *object graph*.

But every link has a cost. It can:

- fire a hidden query,
- cause a surprise *cascade* (an action on the parent spreading to children),
- or become a performance trap.

What is happening internally?

- **Owning side vs inverse side**:
- The **owning side** is the side that holds the foreign key. It uses `@ManyToOne` with `@JoinColumn`. Only this side decides what gets written to the database.
- The **inverse side** uses `@OneToMany(mappedBy = …)`. It is only a mirror, so you can navigate in the other direction. Changes made only here are not saved.
- **Fetch types** — when the linked data is loaded:
- `EAGER`: load the link immediately, together with the parent. Hibernate uses a join or an extra query.
- `LAZY`: do not load it yet. Hibernate puts a placeholder in the field instead. For a single object, the placeholder is a **proxy**. For a collection, it is a `PersistentCollection` wrapper. The data is loaded the first time you touch it.
- **Defaults**: `@ManyToOne` and `@OneToOne` are **EAGER**. `@OneToMany` and `@ManyToMany` are **LAZY**.
- **Cascade**: an operation on the parent (persist, merge, remove) is passed on to its children automatically.
- `orphanRemoval = true`: if you remove a child from the parent's collection, Hibernate deletes that child row.

Think of a lazy proxy like a restaurant token. You get the token now. The food (the data) comes only when you go to the counter and ask for it.

How does it work?

```
@Entity
public class Order {
    @Id @GeneratedValue Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)       // override the EAGER default
    @JoinColumn(name = "outlet_id")
    private Outlet outlet;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<OrderLine> lines = new ArrayList<>();

    public void addLine(OrderLine l) { lines.add(l); l.setOrder(this); }      // keep both sides in sync
    public void removeLine(OrderLine l) { lines.remove(l); l.setOrder(null); }
}

@Entity
public class OrderLine {
    @Id @GeneratedValue Long id;
    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "order_id") private Order order;   // owning side
    private String sku; private int qty;
    void setOrder(Order o) { this.order = o; }
}
```

Example

```
Order o = orderRepo.findById(7L).orElseThrow();   // SELECT order only
o.getOutlet().getId();                              // no query: the proxy already knows the FK id
o.getOutlet().getName();                            // SELECT outlet (proxy initialised)
o.getLines().size();                                // SELECT order_line WHERE order_id = 7
```

Sometimes a many-to-many link has its own data. For example, an outlet joins a scheme on a certain date. That joining date belongs to the link itself. In this case, do not use `@ManyToMany`. Make the join table its own entity instead.

When would I use it?

Map relationships **inside an aggregate** — for example, order → lines.

Across aggregates or across services, store only the ID (like `outletId`), not the object. Because of this:

- loading stays predictable (no surprise queries), and
- the boundaries between services stay clean.

What can go wrong?

- **EAGER `@ManyToOne` everywhere**: every query drags in a chain of joined entities.
- **Updating only one side of a two-way link**: the objects in memory disagree with each other until you reload them.
- **`CascadeType.REMOVE` on `@ManyToMany`**: deleting one parent deletes children that other parents still share.
- **`toString`, `equals`, `hashCode` or JSON serialisation touching lazy collections**: this causes extra queries, endless recursion (A prints B, B prints A ...), or `LazyInitializationException`.

Interview questions

Q1. What are the default fetch types? ToOne links (`@ManyToOne`, `@OneToOne`) are EAGER. ToMany links are LAZY. You should set ToOne links to LAZY explicitly.

Q2. What is the owning side of a relationship? It is the side that holds the foreign key (`@JoinColumn`). JPA writes the relationship to the database from this side. `mappedBy` marks the other side, the inverse side.

Q3. When would you avoid `@ManyToMany`? When the link has its own data or its own lifecycle. Then model the join table as an entity, with two ManyToOne relations.

7.9 The persistence context, entity states and dirty checking

Why?

> **In simple words:** Inside a transaction, Hibernate keeps a notebook of every object it loaded and what it looked like at load time. At the end, it compares, finds what you changed, and writes the UPDATE for you — even if you never call `save()`.

Hibernate does more than run SQL. It keeps a **unit of work** — it collects changes and writes them together. In detail, it:

- tracks every entity it loaded,
- makes sure one database row = one Java object within a transaction,
- groups writes into batches,
- writes your changes to the database automatically. Sending pending changes to the database is called *flushing*.

If you do not understand this, you will see puzzles like:

- "Why did this UPDATE happen? I never called save."
- "Why is my change not saved?"
- "Why does my batch job run out of memory?"

What is happening internally?

The **persistence context** is also called the *first-level cache*. It holds two things:

1. A map: `(EntityClass, id) → entity instance`.
2. A **snapshot** of each entity — a copy of its values as they were when loaded.

In Spring, the persistence context lives as long as the transaction.

Think of a library register. Every book issued is written down with its condition. When it comes back, the clerk compares it with the register and notes any damage.

An entity moves through these states:



- **Transient**: a new object that Hibernate does not know about yet.
- **Managed**: Hibernate is tracking it.
- **Detached**: Hibernate tracked it earlier, but the context is closed or cleared now.

- **Removed**: marked for delete; the DELETE runs at flush.

Key behaviour:

- `find` / `findById` first checks the persistence context. Within one transaction, the same ID always gives you the **same Java object**.
- **Dirty checking**: an entity is *dirty* when its values differ from its snapshot. At **flush**, Hibernate compares each managed entity with its snapshot. It issues `UPDATE`s for the changed ones. So you do not call `save()` to update a managed entity.
- When does flush happen? Before commit. Also, in the default AUTO mode, before a query that touches the affected tables — so the query sees your changes.
- Writes wait in a queue (the *action queue*). At flush they run in a fixed order: inserts, then updates, then collection changes, then deletes.

How does it work?

```
@Transactional
public void rename(Long outletId, String newName) {
    Outlet o = outletRepo.findById(outletId).orElseThrow();   // SELECT; entity MANAGED, snapshot taken
    o.setName(newName);                                       // only changes the Java object
}                                                             // commit → flush → dirty check → UPDATE outlet
SET name=? …
```

What `save()` does in Spring Data:

- For a **new** entity, it calls `persist`.
- For an entity with an existing ID, it calls `merge`. `merge` first runs a `SELECT` for that row, then copies your values onto it. That is an extra query you often do not need.

Example

```
// Batch job that runs out of memory: 1M entities accumulate in the persistence context
@Transactional
public void importOutlets(Stream<OutletRow> rows) {
    int i = 0;
    for (OutletRow r : (Iterable<OutletRow>) rows::iterator) {
        em.persist(new Outlet(r));
        if (++i % 500 == 0) { em.flush(); em.clear(); }     // write the batch, then forget it
    }
}
// + spring.jpa.properties.hibernate.jdbc.batch_size=500, order_inserts=true, SEQUENCE ids
```

Without the `flush` + `clear`, all 1 million objects (and their snapshots) stay in memory. With it, every 500 rows are written and then dropped from the context.

When would I use it?

- **Everyday updates**: load the entity, change it, and let the commit flush it.
- **Read-only queries**: use `@Transactional(readOnly = true)`. Hibernate then skips the snapshots and the dirty checking, which saves memory and CPU.
- **Bulk jobs**: flush and clear every few hundred rows. Or use JDBC batching or plain SQL directly.

What can go wrong?

- **Unexpected UPDATEs**: some code changed a managed entity — even a setter called only "for display". Hibernate sees the change and saves it.

- **Running out of memory**: a long transaction loads too many entities, and they all stay in the context.
- **Stale state**: a bulk JPQL or SQL update skips the context. Entities already in memory now hold old values.
- **Detached entity confusion**:
- Changing an entity after the transaction has ended changes nothing in the database.
- `merge` returns a *different* managed object. Keep using the returned one, not the one you passed in.
- **Flush order surprises**: an error such as a unique-constraint violation may appear later, on some other query (when AUTO flush runs). It does not appear on the line where you called `save`.

Interview questions

Q1. What is the persistence context? It is Hibernate's first-level cache and unit of work, one per transaction. It maps IDs to managed entity objects, keeps a snapshot of each, and flushes changes at commit.

Q2. What is dirty checking? At flush, Hibernate compares each managed entity with its snapshot. For every changed entity, it generates an UPDATE automatically.

Q3. Do you need to call `save()` after modifying an entity? Not for a managed entity inside a transaction — the change is flushed at commit. `save` matters only for new or detached entities.

Q4. persist vs merge? persist makes a new object managed. merge copies a detached object's values onto a managed object (loading it first if needed) and returns that managed object.

7.10 The N+1 query problem and fetching strategies

Why?

> **In simple words:** You load 500 orders with one query. Then, for each order, you read its outlet name — and Hibernate quietly fires one more query each time. So 1 query becomes 501.

This is the most common ORM performance bug. The code looks like one query, but it runs hundreds.

A typical story:

- A list endpoint is fast in dev, with 10 rows.
- In production, with 500 rows, it takes 8 seconds.
- Why? Each row triggers more queries.

What is happening internally?

Step by step:

1. Loading N parent rows runs **1 query**.
2. Your code touches a lazy link on each parent (for example, `order.getOutlet().getName()`).
3. Each touch loads that proxy or collection with **1 more query**.
4. Total: **N+1 round trips** to the database.

Each query alone is cheap. But the network delay adds up: 500 × 2 ms = 1 second. And the whole time, the request holds a connection from the pool, so other requests may have to wait.

Think of a shopkeeper who walks to the godown separately for each item on your list, instead of picking all items in one trip.

```
SELECT * FROM orders WHERE status='OPEN'          -- 1 query, returns 500 orders
SELECT * FROM outlet WHERE id=?   (×500)           -- one per order when the DTO mapper reads
order.getOutlet().getName()
```

How does it work?

Fixes, from most targeted to least targeted:

1. **Fetch join** (JPQL): load the linked data in the same query, using `join fetch`.
2. `@EntityGraph`: on a repository method, list which links should be loaded together with the parent.
3. **Batch fetching**: set `hibernate.default_batch_fetch_size=50`. Hibernate then loads lazy links for many parents at once, using `IN (?, ?, …)`. So N+1 queries become 1 + N/50.
4. **DTO projection**: select exactly the columns the response needs, straight into a DTO (a plain data object). This is often the best choice for read endpoints.

Example

```
public interface OrderRepository extends JpaRepository<Order, Long> {
    @Query("select o from Order o join fetch o.outlet where o.status = :s")              // 1. fetch join
    List<Order> findOpenWithOutlet(@Param("s") OrderStatus s);

    @EntityGraph(attributePaths = {"outlet", "lines"})                                    // 2. entity graph
    List<Order> findByStatus(OrderStatus status);

    @Query("""
        select new com.fa.order.OrderSummary(o.id, ou.name, o.totalPaise, o.createdAt)
        from Order o join o.outlet ou where o.tenantId = :t and o.createdAt >= :since
        """)
    List<OrderSummary> summaries(@Param("t") String tenant, @Param("since") Instant since);   // 4. DTO
projection
}
public record OrderSummary(Long id, String outletName, long totalPaise, Instant createdAt) {}
```

How to detect it:

- In dev and tests, log the SQL: `logging.level.org.hibernate.SQL=DEBUG`.
- Turn on `hibernate.generate_statistics` to see query counts.
- In integration tests, assert the number of queries (for example, with datasource-proxy).
- In production, look at APM traces (application performance monitoring). Hundreds of identical SELECTs in one request is the sign.

When would I use it?

On every list or report endpoint.

Decide what to load **for each use case** — a list view needs less than a detail view. Do not decide it once for the whole app by making links EAGER.

What can go wrong?

- **Fetch-joining a collection while paginating**: Hibernate cannot apply the page limit in SQL. So it loads **all** rows and pages them **in memory**. You will see warning HHH90003004. Fix: first fetch only the parent IDs for the page, then fetch the links for those IDs. Or use batch fetching.
- **Fetch-joining two collections** (Hibernate calls a `List` mapping a "bag"): you get `MultipleBagFetchException`, or a Cartesian product — every line row multiplied by every other row. Fix: fetch one collection per query, or use `Set`s, or batch fetching.

- **Making links EAGER to "fix" N+1**: this does not fix it. It just moves the problem to every other place that loads the entity.

Interview questions

Q1. What is the N+1 problem? One query loads N parent rows. Then reading a lazy link on each parent fires one more query per parent. That is N+1 queries in total.

Q2. How do you fix it? Use fetch joins, @EntityGraph, batch fetching (`default_batch_fetch_size`), or DTO projections that select exactly what is needed. Detect it by logging and counting the SQL.

Q3. Why is `join fetch` with pagination dangerous? When you fetch a collection, Hibernate cannot apply LIMIT in SQL. So it loads all the rows and does the paging in memory.

7.11 Lazy loading, LazyInitializationException and Open-Session-in-View

Why?

> **In simple words:** A lazy link can load its data only while the database session is open. If you touch it after the transaction has ended, it crashes. Spring Boot's default "fix" keeps the session open for the whole web request — which creates bigger problems.

Lazy loading is good: you do not load data you do not need. But there is a catch:

- A lazy proxy can load only while its session (the persistence context) is still open.
- If you touch it after the transaction ends, you get `LazyInitializationException`.
- This usually happens in the controller, or while converting the object to JSON (*serialisation*).

By default, Spring Boot "solves" this with **Open-Session-in-View (OSIV)**. But OSIV causes worse problems, as you will see below.

What is happening internally?

- A lazy link is a Hibernate proxy or `PersistentCollection`. It keeps a reference to the session that created it.
- On first access, it asks that session to load the data.
- If the session is already closed, it throws `LazyInitializationException: could not initialize proxy - no Session`.

Think of a railway counter that closes at 5 pm. Your token is valid, but if you come at 6 pm, nobody is there to serve you.

OSIV is controlled by `spring.jpa.open-in-view=true`. It is on by default, and Spring prints a warning about it at startup. Here is what it does:

1. An interceptor opens an `EntityManager` at the **start of the web request**.
2. It closes it only after the response has been written.

So lazy loading now works in controllers and in JSON serialisers. But:

- Queries now run in the web layer, **outside your service transaction**. N+1 queries can hide inside the JSON writing, where nobody looks.

- **A database connection is held for the whole request.** That includes slow calls to other services and the serialisation time. Under load, this uses up the whole connection pool.

How does it work?

The correct approach has two parts:

1. **Turn OSIV off.**
2. Load everything the response needs **inside the service transaction**. Then convert it to DTOs before you return.

```
spring.jpa.open-in-view: false
```

Example

```
@Service
public class OrderQueryService {
    private final OrderRepository repo;
    OrderQueryService(OrderRepository repo) { this.repo = repo; }

    @Transactional(readOnly = true)
    public OrderDetailDto detail(Long id) {
        Order o = repo.findWithLinesAndOutletById(id).orElseThrow();   // @EntityGraph loads what we need
        return OrderDetailDto.from(o);                                  // mapping happens INSIDE the
transaction
    }
}
// Controller returns OrderDetailDto (a record) — no entities, no lazy proxies cross the boundary.
```

When would I use it?

Always, in services that give data to controllers. Use clear fetch plans (what to load, decided per use case), and map to DTOs inside the transactional service.

What can go wrong?

- **Leaving OSIV on**: the pool runs out under load, and queries hide in the web layer.
- **"Fixing" LazyInitializationException the wrong way**: making links EAGER, or calling `Hibernate.initialize` everywhere.
- **Serialising entities directly with Jackson**: lazy proxies fire queries or throw errors. Return DTOs instead.

Interview questions

Q1. What causes LazyInitializationException? Touching a lazy link that is not loaded yet, after its persistence context (session) has closed. This usually happens outside the transaction.

Q2. What is Open-Session-in-View and why disable it? It keeps the persistence context open for the whole web request, so lazy loading works in the view layer. But it hides queries outside transactions and holds a DB connection for the full request. Disable it and load data explicitly in services.

7.12 `@Transactional`: how it works and its rollback rules

Why?

> **In simple words:** You put `@Transactional` on a method. Spring then starts a transaction before the method, and commits it after — or rolls everything back if a runtime exception escapes. You write

> the boundary once, not in every method.

A business operation must be **atomic** — all steps happen, or none do. For example:

- debit the account,
- *and* write the ledger entry,
- *and* write the outbox event.

Either all three are saved, or none are.

Think of an ATM. Either the cash comes out *and* your account is debited, or neither happens.

Doing this by hand in every method is repetitive and easy to get wrong. You would call `connection.setAutoCommit(false)`, then commit, then roll back on errors — every single time. `@Transactional` lets you declare the boundary once.

What is happening internally?

1. Spring wraps your bean in a **proxy** (6.7) — a wrapper object that sits in front of it. The proxy has a `TransactionInterceptor`.
2. When a call comes through the proxy, the interceptor asks the `PlatformTransactionManager` to begin a transaction. For JPA, this is the `JpaTransactionManager`. It uses the settings you gave: propagation, isolation, timeout and readOnly.
3. The manager gets a DB connection and turns autocommit off. Then it **binds** the connection and the EntityManager to the current thread. It does this through `TransactionSynchronizationManager`, which uses a ThreadLocal (4.15) — a variable that each thread has its own copy of. Any repository called in the same thread finds this connection and reuses it.
4. Your method runs.
5. If the method returns normally → flush + **commit**. If it throws an exception → **rollback**, but only if the rollback rules match. Otherwise, it still commits.
6. The connection is unbound and returned to the pool. Then after-commit callbacks (`@TransactionalEventListener`) run.

Default rollback rules:

- **Roll back** on **unchecked exceptions (RuntimeException) and Errors**.
- **Commit** on checked exceptions (like `IOException`).
- You can change this with `rollbackFor` / `noRollbackFor`.

How does it work?

```
@Service
public class RedemptionService {
    private final AccountRepository accounts;
    private final LedgerRepository ledger;
    private final OutboxRepository outbox;
    // constructor omitted

    @Transactional(timeout = 5)
    public RedemptionResponse redeem(String tenant, String accountId, String idemKey, long points) {
        var prior = ledger.findByTenantIdAndIdempotencyKey(tenant, idemKey);
        if (prior.isPresent()) return RedemptionResponse.from(prior.get());           // idempotent replay

        int updated = accounts.debitIfSufficient(accountId, points);                   // atomic conditional
UPDATE (7.15)
        if (updated == 0) throw new InsufficientPointsException(accountId, points,
accounts.balance(accountId));
```

```
        LedgerEntry e = ledger.save(new LedgerEntry(tenant, accountId, EntryType.REDEMPTION, -points,
idemKey, Instant.now()));
        outbox.save(OutboxEvent.pointsRedeemed(e));                                    // same transaction
(11.15)
        return RedemptionResponse.from(e);
    }                                                                                   // commit: all three
rows or none
}
```

A note on the "idempotent replay" line: *idempotent* means doing the same request twice gives the same result as doing it once. If the same idempotency key comes again, we return the old result instead of debiting again.

Example

```
// Checked exception → COMMITS by default (surprise!)
@Transactional
public void importFile(Path p) throws IOException {
    repo.save(new Batch(p));
    parse(p);                       // throws IOException → the Batch row is still committed
}
@Transactional(rollbackFor = Exception.class)     // roll back for checked exceptions too
public void importFileSafely(Path p) throws IOException { /* ... */ }

// Swallowed exception → COMMITS
@Transactional
public void bad() {
    try { repo.save(x); risky(); } catch (RuntimeException e) { log.warn("ignored", e); }   // no rollback
}
```

When would I use it?

- Put it on **service-layer** methods. Each such method should be one business operation.
- Use `@Transactional(readOnly = true)` on query services. It turns on read optimisations, and it can send the reads to a replica database.
- Keep transaction boundaries out of controllers and repositories.

What can go wrong? (where `@Transactional` silently doesn't work)

- **Self-invocation** (6.7): a method calls another method in the same class. The call does not go through the proxy, so the annotation is ignored. The same is true for **private or final methods**, and for objects you create with `new`.
- **Checked exceptions** commit by default. **Swallowed exceptions** (caught and not rethrown) also commit.
- **Work on another thread** (`@Async`, executors, `CompletableFuture`) runs outside the transaction. Why? The transaction is bound to the original thread only.
- **Multiple data sources** without saying which `transactionManager` to use.
- **Long transactions with remote calls inside**: they hold a connection and row locks while waiting on the network. Also, the remote side effect cannot be rolled back. Better: do the DB work, commit, then call out. Or use the outbox pattern.
- `UnexpectedRollbackException`: this happens in three steps: 1. An inner `REQUIRED` method throws. This marks the shared transaction as *rollback-only*. 2. The outer method catches the exception. 3. The outer method tries to commit — and Spring throws this exception, because the transaction can only roll back.

Interview questions

Q1. How does @Transactional work? A proxy's TransactionInterceptor begins a transaction through the PlatformTransactionManager and binds the connection to the current thread. Then it calls the method. At the end, it commits, or rolls back according to the rollback rules.

Q2. What are the default rollback rules? Roll back on RuntimeException and Error. Commit on checked exceptions, unless `rollbackFor` says otherwise.

Q3. List cases where @Transactional is silently ignored. Self-invocation, private or final methods, objects not managed by Spring, work on other threads, and the wrong transaction manager. Also, it still commits when a checked exception is thrown or an exception is swallowed.

Q4. Why shouldn't you call external APIs inside a transaction? The DB connection and locks stay held while you wait on the network. This hurts throughput and risks pool exhaustion and deadlocks. Also, if the DB transaction fails, you cannot roll back the external effect.

7.13 Transaction propagation

Why?

> **In simple words:** When one transactional method calls another, propagation decides: does the inner one join the outer transaction, start its own separate one, or insist that one already exists?

Transactional methods often call other transactional methods. So a question comes up. Should the inner method:

- join the caller's transaction,
- start its own transaction, or
- require that a transaction already exists?

The *propagation* setting decides this. A wrong choice causes one of two problems:

- You lose data you wanted to keep. For example, an audit record is rolled back together with the failed operation.
- You commit half-done work that you wanted to undo.

What is happening internally?

Propagation	If a transaction exists	If none exists
`REQUIRED` (default)	Join it	Start a new one
`REQUIRES_NEW`	**Pause** it; start a separate one (on a separate connection)	Start a new one
`NESTED`	Create a **savepoint** inside it; if the inner part fails, roll back only to the savepoint	Start a new one
`SUPPORTS`	Join	Run without a transaction
`NOT_SUPPORTED`	Pause it; run without a transaction	Run without a transaction
`MANDATORY`	Join	Throw an error

Propagation	If a transaction exists	If none exists
`NEVER`	Throw an error	Run without a transaction

(Pausing a transaction is officially called *suspending* it. A *savepoint* is a bookmark inside a transaction that you can roll back to.)

With `REQUIRED`, the inner and outer methods share **one physical transaction**. So if the inner method throws an unchecked exception, the whole transaction is marked rollback-only. This happens even if the outer method catches the exception.

How does it work?

```
outer() @Transactional(REQUIRED)          tx#1 begins
   ├─ ledger.save(...)                    tx#1
   ├─ audit.record(...) @REQUIRES_NEW     tx#1 suspended → tx#2 begins … commits → tx#1 resumes
   └─ throws RuntimeException             tx#1 rolls back — the audit row (tx#2) STAYS committed
```

Example

```java
@Service
class AuditService {
    @Transactional(propagation = Propagation.REQUIRES_NEW)     // survives the caller's rollback
    public void recordAttempt(String accountId, String action, String outcome) {
        auditRepo.save(new AuditEntry(accountId, action, outcome, Instant.now()));
    }
}

@Service
class RedemptionFacade {
    private final RedemptionService redemption; private final AuditService audit;
    // constructor omitted
    @Transactional
    public void redeemWithAudit(String acc, long pts) {
        try {
            redemption.redeem(acc, pts);
            audit.recordAttempt(acc, "REDEEM", "OK");
        } catch (InsufficientPointsException e) {
            audit.recordAttempt(acc, "REDEEM", "INSUFFICIENT");  // committed independently
            throw e;                                              // outer transaction rolls back
        }
    }
}
```

`NESTED` uses JDBC savepoints. `DataSourceTransactionManager` supports it. With JPA's transaction manager, it works only in limited setups. It is useful for "try this sub-step; if it fails, undo only that step and carry on".

When would I use it?

- **`REQUIRED`**: for almost everything.
- **`REQUIRES_NEW`**: for work that must be saved whatever happens to the outer transaction. Examples: audit logs, failure records, idempotency-key reservations, or sequence allocation.
- **`MANDATORY`**: on repository-like methods that must never run outside a transaction.
- **`SUPPORTS`**: for `readOnly` queries that work fine with or without a transaction.

What can go wrong?

- **Pool deadlock with `REQUIRES_NEW`**: it needs a **second connection** while the first one is still held. Under load, with a small pool, every thread holds one connection and waits for another. Nobody can move forward.
- **`REQUIRES_NEW` cannot see the outer transaction's writes**: the paused outer transaction has written rows but not committed them. The new transaction **won't see them**. It may even block, waiting on their locks.
- **Expecting a caught inner exception to be harmless**: with `REQUIRED`, the outer transaction is already marked rollback-only. You get `UnexpectedRollbackException` at commit.

Interview questions

Q1. REQUIRED vs REQUIRES_NEW vs NESTED? REQUIRED joins the current transaction, or creates one; everything shares it. REQUIRES_NEW pauses the current one and runs a separate transaction that commits or rolls back on its own. NESTED creates a savepoint inside the current transaction.

Q2. When would you use REQUIRES_NEW? For work that must be saved no matter what happens to the caller, such as audit or failure logs. But remember it needs an extra connection.

Q3. Why might REQUIRES_NEW cause a deadlock? Each request holds one connection and needs a second. If such requests fill the whole pool, they all wait for each other forever.

7.14 Isolation levels and concurrency anomalies

Why?

> **In simple words:** Many transactions touch the same rows at the same time. The isolation level decides how much one transaction can see of another's unfinished work. Too loose gives wrong data; too strict makes everything slow.

Many transactions run at the same time on the same rows. *Isolation* decides how much they see of each other's changes — changes that are not committed yet, or that happen in parallel.

- **Too weak**: you get double redemptions and wrong balances.
- **Too strong**: you get blocking, retries and lower throughput.

Choosing the level on purpose, not by accident, is a senior skill.

What is happening internally?

These are the problems (*anomalies*) that can happen. T1 and T2 are two transactions running at the same time.

Anomaly	What happens
Dirty read	T2 reads T1's change before T1 commits it; then T1 rolls back, so T2 used data that never existed
Non-repeatable read	T1 reads the same row twice and gets different values, because T2 committed an update in between
Phantom read	T1 runs the same range query twice and sees new or missing rows, because T2 inserted or deleted rows in between

Anomaly	What happens
Lost update	T1 and T2 both read balance 100, and each writes its own result; one of the updates is lost
Write skew	T1 and T2 read the same data, then each writes a *different* row based on it, and together they break a rule (two doctors both go off call, leaving nobody)

Isolation level	Dirty	Non-repeatable	Phantom	Default in
READ UNCOMMITTED	possible	possible	possible	—
READ COMMITTED	prevented	possible	possible	**PostgreSQL, SQL Server, Oracle**
REPEATABLE READ	prevented	prevented	possible (prevented in Postgres's snapshot implementation; mostly prevented in InnoDB using gap locks)	**MySQL InnoDB**
SERIALIZABLE	prevented	prevented	prevented	—

How do databases build these levels? Two main ways:

- **MVCC snapshots** (multi-version concurrency control): each transaction reads a consistent snapshot of the data, as of a point in time. Used by PostgreSQL, Oracle, InnoDB, and SQL Server with snapshot isolation.
- **Locks**: readers and writers block each other. SQL Server's default READ COMMITTED works this way.

More details are in 12.7.

> **Remember: Lost updates and write skew are not prevented by READ COMMITTED.** To stop them, you need row locks, optimistic versions or atomic statements (7.15), or SERIALIZABLE.

How does it work?

```
@Transactional(isolation = Isolation.REPEATABLE_READ)
public Report monthlyStatement(String accountId) { /* several reads that must be consistent with each other */ }

@Transactional(isolation = Isolation.SERIALIZABLE)
public void allocateLimitedVouchers(...) { /* rare: invariant across rows; be ready to RETRY serialization failures */ }
```

(An *invariant* is a rule that must always stay true — for example, "balance never goes below zero".)

Example

```
Lost update under READ COMMITTED (balance = 100):
T1: SELECT balance → 100              T2: SELECT balance → 100
T1: UPDATE balance = 100 - 80 = 20    T2: UPDATE balance = 100 - 50 = 50
T1: COMMIT                            T2: COMMIT  → final 50; 130 points spent from a 100 balance
Fix: UPDATE account SET balance = balance - :p WHERE id = :id AND balance >= :p   (7.15)
```

When would I use it?

- **Most OLTP work** (normal day-to-day transactions): keep the default, READ COMMITTED. Protect specific rules with locking or atomic statements.
- **Several reads that must agree with each other** (reports, statements): use REPEATABLE READ or a snapshot.
- **Complex rules across many rows**: use SERIALIZABLE, but only rarely, and with automatic retry.

What can go wrong?

- **Raising isolation for the whole app "to be safe"**: you get more blocking, more deadlocks and more serialisation failures.
- **Assuming a level behaves the same in every database**: it does not. Postgres REPEATABLE READ is snapshot isolation; MySQL's REPEATABLE READ is not the same thing.
- **Forgetting to retry serialisation failures** (SQLSTATE 40001). At SERIALIZABLE, the database cancels some transactions on purpose. You must retry them.

Interview questions

Q1. Name the isolation levels and the anomalies each prevents. Read uncommitted prevents none. Read committed prevents dirty reads. Repeatable read also prevents non-repeatable reads. Serializable also prevents phantoms and write skew.

Q2. What is a lost update and how do you prevent it? Two read-modify-write transactions overwrite each other's result. Prevent it with an atomic UPDATE statement, `SELECT FOR UPDATE`, or an optimistic version check.

Q3. What is the default isolation in PostgreSQL and MySQL? PostgreSQL uses READ COMMITTED. MySQL InnoDB uses REPEATABLE READ.

7.15 Optimistic locking, pessimistic locking and atomic updates

Why?

> **In simple words:** When two requests change the same row at once, you have three choices. Check for a clash at save time (optimistic). Lock the row while reading (pessimistic). Or do the check and the change in one SQL statement (atomic update).

Many requests change the same rows at the same time. For example:

- two redemptions from the same account,
- two admins editing the same campaign,
- many users booking the last seat.

You need a strategy that keeps the data correct, without killing throughput.

Think of IRCTC tatkal booking. Many people want the last seat. Only one should get it, and nobody should be charged for a seat they did not get.

What is happening internally?

1. Optimistic locking (`@Version`) — "assume no clash, but check at save time":

1. No lock is taken while reading.

2. On update, Hibernate adds `WHERE id = ? AND version = ?` and increases the version by one.
3. If another transaction changed the row first, the version no longer matches. The update matches **0 rows**.
4. Hibernate throws `OptimisticLockException`. (Spring wraps it as `ObjectOptimisticLockingFailureException`.)
5. You retry, or report a conflict to the user.

No locks are held between the read and the write.

2. Pessimistic locking — "lock first, then work":

- `SELECT … FOR UPDATE` takes a **row lock** at read time.
- Other writers, and other `FOR UPDATE` readers, **wait** until you commit.
- The lock holder is sure to succeed. The cost is that others are blocked.

3. Atomic conditional update — "let the database do it in one step":

- One SQL statement does the read, the check and the write, all inside the database: `UPDATE … SET balance = balance - ? WHERE id = ? AND balance >= ?`.
- The row lock is held only for that one statement.
- The number of affected rows tells you whether it worked: 1 = success, 0 = failed.

How does it work?

Strategy	Best when	Cost
Optimistic `@Version`	Clashes are rare (editing campaigns, profiles); the user takes a long time between reading and saving	Retries or 409 errors when a clash happens
Pessimistic `FOR UPDATE`	Clashes are frequent and retries are costly; a rule across many rows inside one transaction	Blocking, risk of deadlock, lower throughput
Atomic UPDATE	A numeric rule on a single row (balance, stock, counters)	The logic must fit in one SQL statement

Example

```
@Entity
class Campaign {
    @Id Long id;
    String name;
    @Version long version;                      // optimistic locking
}
// Two admins load version 3; first save → version 4; second save → UPDATE ... WHERE version=3 → 0 rows → 409
Conflict

public interface AccountRepository extends JpaRepository<Account, String> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)                                          // SELECT ... FOR UPDATE
    @QueryHints(@QueryHint(name = "jakarta.persistence.lock.timeout", value = "3000"))
    @Query("select a from Account a where a.id = :id")
    Optional<Account> findByIdForUpdate(@Param("id") String id);

    @Modifying
    @Query("update Account a set a.balance = a.balance - :p where a.id = :id and a.balance >= :p")
    int debitIfSufficient(@Param("id") String id, @Param("p") long points);       // 1 = success, 0 =
insufficient
}

// Optimistic retry with Spring Retry
@Retryable(retryFor = ObjectOptimisticLockingFailureException.class, maxAttempts = 3, backoff =
```

```
@Backoff(delay = 50))
@Transactional
public void rename(Long id, String name) { campaignRepo.findById(id).orElseThrow().setName(name); }
```

Here is what happens when two redemptions run at the same time with the atomic update. The balance is 100, and both requests ask for 80:

```
T1: UPDATE … WHERE id='A1' AND balance >= 80 → row locked, balance 100 ≥ 80 → balance = 20, 1 row
T2: same UPDATE → waits for T1's row lock → T1 commits → re-evaluates: 20 ≥ 80 false → 0 rows → insufficient
Result: exactly one succeeds; no lost update, no application-level lock.
```

When would I use it?

- **Atomic updates**: for balances, stock and counters — like your loyalty ledger.
- **Optimistic versions**: for entities and aggregates that admins edit.
- **Pessimistic locks**: for operations on many rows that must stay consistent within one transaction. Always lock rows in the same order everywhere, to avoid deadlocks (4.13 / 12.7).

What can go wrong?

- **Holding pessimistic locks while calling external services**: others stay blocked for a long time, and the pool runs out.
- **Locking rows in different orders in different code paths**: T1 locks A then B, T2 locks B then A. Each waits for the other — a database deadlock.
- **Optimistic retries without backoff under heavy contention**: everyone retries at the same moment and clashes again. This is a *retry storm*. Also, retrying side effects that are not idempotent (not safe to repeat) can do them twice.
- `@Version` **with bulk JPQL updates**: bulk updates do not increase the version, so the check is skipped.

Interview questions

Q1. Optimistic vs pessimistic locking? Optimistic finds clashes at write time with a version check. It takes no locks and retries on conflict, so it suits low contention. Pessimistic locks rows at read time (`FOR UPDATE`) and blocks other writers, so it suits high contention.

Q2. How would you prevent double-spending of points under concurrency? Use an atomic conditional UPDATE (`balance >= amount`) and check the affected row count — or use a row lock. Add an idempotency key so duplicate requests are not processed twice.

Q3. What happens when an optimistic lock check fails? The update affects zero rows, and Hibernate throws OptimisticLockException. The application then retries, or returns 409.

7.16 Connection pools, migrations and JPA performance settings

Why?

> **In simple words:** Keep a set of ready database connections and reuse them (a pool). Change the schema only through numbered SQL scripts (migrations). And switch on a few Hibernate settings so inserts go in batches.

Three needs, three tools:

- **Pool**: opening a DB connection takes milliseconds — TCP setup, TLS, login. So every app keeps a **pool** of open connections and reuses them.
- **Migrations**: schema changes must be versioned and must run the same way in every environment. So you need **migrations**.
- **Settings**: a few Hibernate settings make the difference between 50 and 5,000 inserts per second.

Think of an auto stand. Autos wait in a line, ready to go. You take one, finish your trip, and it comes back to the line. Nobody builds a new auto for each ride.

What is happening internally?

- **HikariCP** is Spring Boot's default pool. It keeps a set of open connections. 1. A thread borrows one (`getConnection`). 2. It uses it for the transaction. 3. It gives it back. 4. If no connection is free, the thread waits up to `connection-timeout`, and then fails.
- **Pool size — more is not better.** The database has limited CPU cores and disk. Beyond a point, more parallel queries just fight each other for them.
- A common starting point is about 10–20 connections per app instance.
- Multiply by the number of instances. The total must stay within the DB's `max_connections`.
- **Flyway** runs numbered SQL scripts (`V1__init.sql`, `V2__add_index.sql`) in order at startup. It records which ones it has run in the `flyway_schema_history` table. Liquibase is the alternative, based on XML/YAML.

How does it work?

```yaml
spring:
  datasource:
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5
      connection-timeout: 3000        # ms to wait for a free connection before failing
      max-lifetime: 1500000           # below DB/firewall idle limits (25 min here)
      leak-detection-threshold: 20000 # log a stack trace if a connection is held > 20 s
  jpa:
    open-in-view: false
    hibernate.ddl-auto: validate      # never 'update' in production
    properties:
      hibernate.jdbc.batch_size: 50
      hibernate.order_inserts: true
      hibernate.order_updates: true
      hibernate.default_batch_fetch_size: 50
  flyway.enabled: true
```

Example

```sql
-- src/main/resources/db/migration/V3__ledger_account_index.sql
CREATE INDEX CONCURRENTLY IF NOT EXISTS ix_ledger_account_created ON ledger_entry (account_id, created_at
DESC);
-- (PostgreSQL CONCURRENTLY can't run inside a transaction: mark this migration non-transactional in Flyway
config)
```

Zero-downtime schema change — the **expand → migrate → contract** pattern. You change the schema while the app keeps running:

1. **Expand**: add the new column as nullable (V4).
2. Deploy code that writes to both the old and the new column.
3. **Migrate**: fill in old rows (backfill) in small batches.

4. Deploy code that reads from the new column.
5. **Contract**: drop the old column in a later release (V5).

When would I use it?

In every service:

- Tune the pool using its metrics (`hikaricp.connections.active`, `pending`).
- Put every schema change in a versioned migration.
- Turn on batching for jobs that write a lot.

What can go wrong?

- **Pool exhaustion**: slow queries, long transactions, OSIV or leaked connections (borrowed and never returned). Threads get stuck in `getConnection`. A thread dump shows this (5.8).
- **Too many connections during autoscaling**: pool size × number of pods goes above the DB's connection limit. Use PgBouncer (a connection pooler in front of Postgres), or size the pools correctly.
- **Migrations that lock big tables**: for example, adding a column with a volatile default (a value computed per row, like `random()`), creating an index without `CONCURRENTLY`, or running a long backfill in one transaction.
- **Editing a migration file that has already run**: Flyway stores a checksum of each file. The checksum no longer matches, so the app fails at startup.

Interview questions

Q1. How do you size a connection pool? Start small, about 10–20 per instance. Keep the total under what the DB can handle. Then tune from pool metrics and DB load. The DB's cores and I/O limit useful parallel work — not the number of app threads.

Q2. Why not use `ddl-auto=update` in production? It is not versioned, and it cannot do safe data migrations. It may make destructive or locking changes, and it can behave differently in each environment. Use Flyway/Liquibase migrations, and set `validate`.

Q3. How do you make a schema change with zero downtime? Use expand-migrate-contract. Make additive changes first, write to both old and new, backfill in batches, switch reads to the new structure, and remove the old structure in a later release.

8. Spring in Production: Security, Testing, Messaging, Caching, Resilience, Observability

8.1 Spring Security architecture

Why?

In simple words: Spring Security is a security guard that stands in front of your code. It checks who the caller is and what they may do, before your controller ever runs.

Every endpoint must answer two questions:

- **Who is calling?** This is called *authentication*.
- **Are they allowed to do this?** This is called *authorisation*.

In a multi-tenant SaaS there is a third question. A *tenant* is one customer company that shares the platform with others. So you also ask: **can this caller touch this tenant's data?**

If you write these checks by hand in every controller, someone will forget one. One missed check can leak data. Spring Security puts all these checks in one place. It is a configurable chain of filters that runs before your code.

Think of a company office. The guard at the gate checks your ID card (authentication). The lift only takes you to the floors your card allows (authorisation). You do not ask each desk to check your ID again.

What is happening internally?

```
Request → DelegatingFilterProxy (a servlet filter registered with Tomcat)
        → FilterChainProxy → picks the first SecurityFilterChain whose matcher fits the URL
            ├─ CorsFilter
            ├─ CsrfFilter (for browser/cookie sessions)
            ├─ authentication filters: BearerTokenAuthenticationFilter (JWT),
UsernamePasswordAuthenticationFilter, BasicAuthenticationFilter …
            │     → AuthenticationManager → AuthenticationProvider(s) → Authentication object
            │     → stored in SecurityContextHolder (ThreadLocal) for this request
            ├─ ExceptionTranslationFilter: AuthenticationException → 401 (entry point); AccessDeniedException
→ 403
            └─ AuthorizationFilter: checks URL rules (requestMatchers … hasAuthority …)
        → DispatcherServlet → controller → method security (@PreAuthorize) via AOP proxy
```

Here is the same flow as steps:

1. Tomcat receives the request. It passes it to `DelegatingFilterProxy`, a normal servlet filter.
2. That filter hands over to `FilterChainProxy`. It picks the first `SecurityFilterChain` whose URL matcher fits the request.
3. The chain runs its filters in order: CORS, CSRF, then the authentication filters.
4. An authentication filter (for example, the JWT one) asks the `AuthenticationManager` to check the credentials. The manager uses one or more `AuthenticationProvider`s. If the check passes, you get an `Authentication` object.
5. This object is stored in `SecurityContextHolder`. That is a `ThreadLocal` — a per-thread storage box — so it lives only for this request.

6. `ExceptionTranslationFilter` catches security errors. An `AuthenticationException` becomes 401. An `AccessDeniedException` becomes 403.
7. `AuthorizationFilter` checks your URL rules (`requestMatchers … hasAuthority …`).
8. The request reaches `DispatcherServlet` and then your controller.
9. If a service method has `@PreAuthorize`, an AOP proxy (a wrapper object Spring puts around your bean) checks it before the method runs.

Key objects you should know:

- `Authentication` — holds the principal (who the user is), the credentials and the authorities (what they may do).
- `SecurityContextHolder` — where the current `Authentication` is kept for this request.
- `AuthenticationManager` / `AuthenticationProvider` — the classes that actually check credentials.
- `UserDetailsService` — loads a user from your store for username/password login.
- `PasswordEncoder` — hashes passwords with BCrypt or Argon2. Never store plain text or MD5.
- `GrantedAuthority` — one permission. It can be a role like `ROLE_ADMIN` or a scope like `SCOPE_points.redeem`.

401 vs 403:

- 401 Unauthorized means "I don't know who you are". Credentials are missing or invalid. The name is confusing — it really means *not authenticated*.
- 403 Forbidden means "I know who you are, but you are not allowed to do this".

How does it work?

You define one or more `SecurityFilterChain` beans. You write them with the lambda DSL — the `http.csrf(...).authorizeHttpRequests(...)` style shown below. Spring Boot auto-configures everything else.

You protect at two levels:

- **URL rules** give coarse protection. For example, "everything under `/admin` needs ADMIN".
- **Method security** (`@PreAuthorize`) gives fine-grained checks. It sits close to the business logic, so it can look at method arguments like `tenantId`.

Example

```
@Configuration
@EnableMethodSecurity
class SecurityConfig {
    @Bean
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())                                    // stateless bearer-token API
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health/**").permitAll()
                .requestMatchers(HttpMethod.POST,
"/api/v1/accounts/*/redemptions").hasAuthority("SCOPE_points.redeem")
                .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))      // validate JWTs (8.2)
            .build();
    }
}

@Service
```

```
class CampaignService {
    @PreAuthorize("hasRole('ADMIN') and #tenantId == authentication.token.claims['tenant_id']")
    public Campaign create(String tenantId, CreateCampaignRequest req) { /* ... */ return null; }
}
```

When would I use it?

In every service that exposes HTTP. Use three layers together:

1. **URL rules** — coarse checks.
2. **Method security** — fine checks near the business logic.
3. **Data-level tenant filtering** — every query is scoped by tenant. You can use a Hibernate filter or an explicit `WHERE tenant_id = ?` condition.

Why all three? Because in a multi-tenant system, one authorisation bug can show one customer's data to another customer. That is the worst kind of bug in SaaS.

What can go wrong?

- When you add the security starter, every endpoint suddenly needs login. Spring prints a generated password at startup. This surprises people, but it is the safe default.
- Rule order matters. Rules are matched from top to bottom. If you put `anyRequest().permitAll()` before the specific rules, everything is open.
- Trusting a tenant ID from a request header without checking it against the token's tenant claim. A user can just change the header and read another tenant's data. This is called an **IDOR**: insecure direct object reference.
- Disabling CSRF for a browser app that uses cookie sessions. CSRF protection is needed there. (It is only safe to disable for stateless bearer-token APIs.)
- The SecurityContext does not move to `@Async` threads on its own, because it is stored in a ThreadLocal. Use `DelegatingSecurityContextExecutor` to carry it over.

Interview questions

Q1. How does Spring Security process a request? A FilterChainProxy runs the matching SecurityFilterChain. First, authentication filters check the credentials, build an Authentication object and store it in the SecurityContextHolder. Then authorisation filters check the URL rules. Security exceptions are turned into 401 or 403 responses. Finally, method security checks like @PreAuthorize run at the service proxy.

Q2. Authentication vs authorisation? Authentication checks who you are; if it fails, you get 401. Authorisation decides what an already-authenticated user may do; if it fails, you get 403.

Q3. How do you enforce tenant isolation? Take the tenant from the verified token, not only from a header the client sends. Check it in method security. And apply a mandatory tenant filter to every data query, so even a missed check cannot return another tenant's rows.

8.2 JWT, OAuth2 and OpenID Connect

Why?

> **In simple words:** A JWT is a signed pass that the caller carries with every request. Any service can check the signature by itself, so nobody needs to call a central session store. OAuth2 is the set of rules for how you get that pass.

In microservices, you don't want every service to call a central session store on every request. That store becomes slow and a single point of failure.

- A **JWT** (JSON Web Token) is a self-contained, signed token. Any service can verify it locally, without a network call.
- **OAuth2** is the standard for how clients get tokens.
- **OpenID Connect (OIDC)** sits on top of OAuth2 and adds the user's identity.

Different callers use different OAuth2 flows. Your mobile apps use one flow, partner integrations another, and service-to-service calls another.

Think of a railway ticket with an official stamp. The TT on any coach can check the stamp himself. He does not phone the booking office for every passenger.

What is happening internally?

A JWT has three parts joined by dots: `base64url(header).base64url(payload).signature`. Base64url is just a text encoding, not encryption.

```
header:  {"alg":"RS256","kid":"2026-key-1"}
payload: {"iss":"https://login.example.com","sub":"user-123","aud":"loyalty-api","exp":1790000000,
          "iat":1789999100,"scope":"points.read points.redeem","tenant_id":"t-42","roles":["RETAILER"]}
signature: RSA-SHA256(header.payload, issuer's PRIVATE key)
```

What this means:

- **Signed, not encrypted.** Anyone can decode and read the payload. So never put secrets or sensitive PII (personal data like phone numbers) inside it. The signature only proves that nobody changed it.
- **Asymmetric signing (RS256/ES256).** The identity provider (Keycloak, Azure AD/Entra ID, Auth0, Cognito) signs with its **private** key. Your services verify with the matching **public** keys. These public keys are published at the provider's **JWKS** endpoint (JSON Web Key Set). Services cache them. When keys rotate, the `kid` (key ID) in the header tells which key to use.
- **What a resource server checks.** A *resource server* is your API that receives the token. It checks: 1. the signature; 2. `exp` (expiry) and `nbf` (not-before), allowing a small clock skew; 3. `iss` — who issued it; 4. `aud` — which API it was made for.

Then it maps `scope` and roles to Spring authorities. - **Stateless — but revocation is hard.** There is no server-side session to look up. The downside: you cannot easily cancel a token before it expires. So keep **access tokens** short-lived (5–15 min). Pair them with **refresh tokens**. Refresh tokens live longer, are rotated on use, can be revoked, and must be stored securely.

OAuth2 flows:

Flow	Who uses it
Authorization Code + **PKCE**	Mobile and web apps acting for a user (no client secret is stored on the device)
Client Credentials	Service-to-service calls (a machine identity, no user)
Refresh Token	Getting new access tokens without logging in again
Device Code	TVs and CLIs
(Implicit, Password — deprecated)	Don't use them

PKCE (Proof Key for Code Exchange) is a one-time secret the app creates for each login. It stops a stolen authorisation code from being used by another app.

OIDC adds two things: an **ID token** (it says who the user is) and a `userinfo` endpoint. The **access token** is still the one your APIs check.

How does it work?

You point Spring at the identity provider. From the `issuer-uri`, Spring discovers the JWKS URL and also validates `iss`. You list the expected audience.

```
spring.security.oauth2.resourceserver.jwt:
  issuer-uri: https://login.example.com/realms/fieldassist     # discovers JWKS, validates iss
  audiences: loyalty-api
```

Often your provider puts roles in a custom claim, like `roles`. Spring does not know about it by default. This converter maps that claim to `ROLE_` authorities:

```
@Bean
JwtAuthenticationConverter jwtAuthConverter() {                  // map a custom "roles" claim to ROLE_
authorities
    var roles = new JwtGrantedAuthoritiesConverter();
    roles.setAuthoritiesClaimName("roles");
    roles.setAuthorityPrefix("ROLE_");
    var conv = new JwtAuthenticationConverter();
    conv.setJwtGrantedAuthoritiesConverter(roles);
    return conv;
}
```

Example

A service-to-service call using client credentials (loyalty service → voucher partner gateway). The interceptor gets a token, attaches it to each request and refreshes it when it expires:

```
@Bean
RestClient voucherClient(RestClient.Builder b, OAuth2AuthorizedClientManager mgr) {
    var interceptor = new OAuth2ClientHttpRequestInterceptor(mgr);       // Spring Security 6.4+: attaches
and refreshes tokens
    interceptor.setClientRegistrationIdResolver(req -> "voucher-gateway");
    return b.baseUrl("https://vouchers.internal").requestInterceptor(interceptor).build();
}
// application.yml: spring.security.oauth2.client.registration.voucher-gateway.authorization-grant-
type=client_credentials …
```

When would I use it?

- **JWT resource server** — for every API behind an identity provider.
- **Client credentials** — for calls between services.
- **Authorization Code + PKCE** — for mobile app login.
- **mTLS or service-mesh identity** — for internal calls in high-security setups. mTLS means both sides show certificates to each other.

What can go wrong?

- Accepting `alg: none` (no signature at all), or HS256 with a secret that can be guessed. Good libraries block this if you configure them correctly.
- Not checking `aud`. Then a token made for some other API is accepted by yours.
- Long-lived access tokens that cannot be revoked.

- Storing tokens in browser `localStorage`. An XSS attack (injected script) can steal them. Use secure cookies or keep them in memory instead.
- Huge tokens with too many claims. They can hit HTTP header size limits.
- Clocks on servers that differ slightly (clock skew). This causes random "token expired" errors.

Interview questions

Q1. How does JWT authentication work? The client sends a signed token with the request. The service checks the signature using the issuer's public key from the JWKS endpoint. It then checks expiry, issuer and audience, and builds the security context from the claims. There is no session lookup.

Q2. How do you revoke a JWT? You cannot revoke it directly. So you keep access tokens short-lived and revoke the refresh tokens instead. For sensitive actions, use token introspection (ask the provider if the token is still valid) or keep a deny-list of token IDs (`jti`).

Q3. OAuth2 vs OpenID Connect? OAuth2 is delegated authorisation — it gives access tokens for calling APIs. OIDC is an identity layer on top of it — it adds ID tokens and user info, so you know who the user is.

Q4. Which flow for a mobile app, and which for service-to-service? Authorization Code with PKCE for a mobile app. Client Credentials for service-to-service.

8.3 Unit testing with JUnit 5 and Mockito

Why?

> **In simple words:** Unit tests are small, fast checks of one class at a time. You replace its neighbours with fakes, so you test only your logic. They let you change code later without fear.

Tests let you change code without fear. Unit tests give the fastest feedback — they run in milliseconds. They are also the cheapest way to lock down business rules, such as redemption limits, rule evaluation and idempotency behaviour. (*Idempotent* means doing the same thing twice has the same effect as doing it once.)

Interviewers also judge your code quality by asking: can you test what you design?

What is happening internally?

- **JUnit 5 (Jupiter)** finds methods marked `@Test` and runs them. It gives you:
- lifecycle hooks such as `@BeforeEach` and `@AfterEach` (run before/after each test);
- **parameterised tests** (`@ParameterizedTest` + `@CsvSource` / `@MethodSource`) — one test method run with many inputs;
- extensions (`@ExtendWith`) that plug in extra behaviour.
- **Mockito** creates fake objects:
- **Mocks** are generated subclasses or proxies. They record the calls they get and return the answers you stubbed.
- **Spies** wrap a real object.
- `@InjectMocks` builds the class under test and passes the mocks into it.
- **AssertJ** gives fluent, readable assertions, like `assertThat(x).isEqualTo(…)` and `assertThatThrownBy(…)` .

The test pyramid tells you the mix: many fast unit tests at the bottom, fewer slice and integration tests in the middle (8.4), and only a handful of end-to-end tests at the top.

Think of a car-mechanic testing an engine on a stand. He does not fit the whole car. He feeds fake fuel and fake signals and checks what the engine does.

How does it work?

Every test has three parts:

1. **Arrange** — set up data and stubs.
2. **Act** — call the method.
3. **Assert** — check the result and the important interactions.

Mock only **collaborators at the boundaries**: repositories, HTTP clients, clocks. Never mock the class under test, and never mock value objects (simple data holders).

Example

```
@ExtendWith(MockitoExtension.class)
class RedemptionServiceTest {
    @Mock AccountRepository accounts;
    @Mock LedgerRepository ledger;
    @Mock OutboxRepository outbox;
    @InjectMocks RedemptionService service;

    @Test
    void rejectsWhenBalanceIsInsufficient() {
        when(ledger.findByTenantIdAndIdempotencyKey("t1", "k1")).thenReturn(Optional.empty());
        when(accounts.debitIfSufficient("a1", 500)).thenReturn(0);           // conditional update failed
        when(accounts.balance("a1")).thenReturn(120L);

        assertThatThrownBy(() -> service.redeem("t1", "a1", "k1", 500))
                .isInstanceOf(InsufficientPointsException.class)
                .hasMessageContaining("has 120");
        verify(ledger, never()).save(any());
        verifyNoInteractions(outbox);
    }

    @Test
    void replayingSameIdempotencyKeyDoesNotDebitAgain() {
        var prior = new LedgerEntry("t1", "a1", EntryType.REDEMPTION, -50, "k1", Instant.EPOCH);
        when(ledger.findByTenantIdAndIdempotencyKey("t1", "k1")).thenReturn(Optional.of(prior));

        service.redeem("t1", "a1", "k1", 50);

        verify(accounts, never()).debitIfSufficient(anyString(), anyLong());
    }

    @ParameterizedTest
    @CsvSource({"SILVER,100,100", "GOLD,100,150", "PLATINUM,100,200"})
    void tierMultiplier(Tier tier, long base, long expected) {
        assertThat(tier.apply(base)).isEqualTo(expected);
    }

    @Test
    void capturesPublishedEvent() {
        when(ledger.findByTenantIdAndIdempotencyKey(any(), any())).thenReturn(Optional.empty());
        when(accounts.debitIfSufficient("a1", 50)).thenReturn(1);
        when(ledger.save(any())).thenAnswer(inv -> inv.getArgument(0));       // return what was saved
        service.redeem("t1", "a1", "k2", 50);
        ArgumentCaptor<OutboxEvent> captor = ArgumentCaptor.forClass(OutboxEvent.class);
        verify(outbox).save(captor.capture());
        assertThat(captor.getValue().type()).isEqualTo("PointsRedeemed");
    }
}
```

What each test shows:

- `rejectsWhenBalanceIsInsufficient` — a failed debit throws the right exception, and nothing is saved to the ledger or outbox.
- `replayingSameIdempotencyKeyDoesNotDebitAgain` — the same key sent twice does not debit twice.
- `tierMultiplier` — one parameterised test covers three tiers.
- `capturesPublishedEvent` — an `ArgumentCaptor` grabs the object passed to `outbox.save`, so you can check its contents.

When would I use it?

For all domain logic and service orchestration (services that call other services in order).

Inject `Clock`, ID generators and random sources instead of calling them directly. Then tests can control them, and results are deterministic — the same every run.

What can go wrong?

- **Over-mocking.** Tests copy the implementation line by line. They break on every refactor and prove nothing. Check behaviour — outputs and important interactions — not every single call.
- Mocking value objects or the class under test.
- Unused stubs. Mockito's strict stubs fail the test when a stub is never used. This is good: it catches outdated tests.
- Tests that depend on time, randomness, HashMap ordering or the state left by other tests. They pass sometimes and fail sometimes (flaky tests).

Interview questions

Q1. Mock vs stub vs spy? A stub just returns canned answers. A mock also lets you verify which calls were made. A spy wraps a real object and calls the real methods unless you stub them.

Q2. What should you mock? External boundaries and collaborators — repositories, HTTP clients, clocks and message publishers. Do not mock value objects or the logic you are testing.

Q3. How do you test time-dependent code? Inject a `Clock` into the class. In tests, pass `Clock.fixed` so "now" is always the same.

8.4 Slice tests and integration tests with Testcontainers

Why?

> **In simple words:** Mocks can't tell you if your SQL is wrong or your JSON mapping is broken. So you also run tests with real Spring wiring and a real database in Docker. Testcontainers starts that database for you inside the test.

Unit tests with mocks cannot catch:

- a wrong JPQL query;
- a broken JSON mapping;
- a missing index;
- a Postgres-specific SQL feature;
- a misconfigured Kafka listener.

You need tests that run the real Spring wiring against **real infrastructure**. These tests must still be fast and give the same result on every laptop and every CI run.

What is happening internally?

- **Slice tests** start only *part* of the application context — one layer:
- `@WebMvcTest(Controller.class)` loads the MVC setup, the controller, its advice and Jackson. Services are replaced by `@MockitoBean`s. `MockMvc` sends fake requests without starting a real server.
- `@DataJpaTest` loads JPA and your repositories. Each test runs in a transaction that is rolled back at the end.
- There are others too, like `@JsonTest` and `@RestClientTest`.
- `@SpringBootTest` starts the full context. It can also start a real server on a random port.
- **Testcontainers** starts real Docker containers (PostgreSQL, Kafka, Redis) from inside the test. With `@ServiceConnection` (Boot 3.1+), Spring automatically picks up their dynamic URLs and ports. Containers can be reused across test classes.
- The **context cache**: Spring keeps an application context and reuses it for other test classes with the same configuration. But each different set of `@MockitoBean`s creates a new context. That slows the suite.

Think of a dress rehearsal for a wedding. Practising each dance alone is a unit test. The rehearsal on the real stage, with the real sound system, is the integration test.

How does it work?

A web-slice test. It checks that bad input gives a 400 with two field errors, without any real service:

```
@WebMvcTest(RedemptionController.class)
class RedemptionControllerTest {
    @Autowired MockMvc mvc;
    @MockitoBean RedemptionService service;                    // Spring Framework 6.2+ (older: @MockBean)

    @Test
    void returns400WithFieldErrors() throws Exception {
        mvc.perform(post("/api/v1/accounts/a1/redemptions")
                .header("Idempotency-Key", "k1")
                .contentType(MediaType.APPLICATION_JSON)
                .content("{\"points\": -5, \"rewardCode\": \"\"}"))
           .andExpect(status().isBadRequest())
           .andExpect(jsonPath("$.errors.length()").value(2));
    }
}
```

Example

A full integration test against real Postgres. Twenty threads try to redeem 100 points each from an account holding 1,000:

```
@SpringBootTest
@Testcontainers
class RedemptionConcurrencyIT {
    @Container @ServiceConnection
    static PostgreSQLContainer<?> pg = new PostgreSQLContainer<>("postgres:16-alpine");

    @Autowired RedemptionService service;
    @Autowired AccountRepository accounts;

    @Test
    void twentyConcurrentRedemptionsNeverOverdraw() throws Exception {
```

```
        accounts.save(new Account("a1", "t1", 1_000));
        try (ExecutorService pool = Executors.newFixedThreadPool(20)) {
            List<Future<?>> fs = new ArrayList<>();
            for (int i = 0; i < 20; i++) {
                String key = "k" + i;
                fs.add(pool.submit(() -> {
                    try { service.redeem("t1", "a1", key, 100); } catch (InsufficientPointsException ignored)
{ }
                }));
            }
            for (Future<?> f : fs) f.get();
        }
        assertThat(accounts.findById("a1").orElseThrow().balance()).isZero();   // exactly 10 succeeded,
never negative
    }
}
```

Exactly 10 redemptions should succeed and the balance must end at zero — never negative. This concurrency test against real Postgres is the single most convincing thing to show in your Java project (8.11).

When would I use it?

- `@WebMvcTest` — for request mapping, validation, error format and security rules.
- `@DataJpaTest` + Testcontainers — for custom queries and constraints.
- `@SpringBootTest` + Testcontainers — for end-to-end flows (API → DB → Kafka) and concurrency guarantees.

Run all of them in CI on every pull request.

What can go wrong?

- Using H2 (an in-memory database) instead of your real database. H2 has a different SQL dialect, different locking, different JSONB and constraint behaviour. Tests pass and production breaks.
- Slow suites because of many unique contexts. Each different mock set or property set builds a new context. Standardise your test configurations so they share one.
- Tests that share data and don't clean up. Then a test passes or fails depending on run order.
- `@Transactional` on integration tests can hide bugs. The data never really commits. Lazy loading works only because the test's transaction is still open — in production it would fail.

Interview questions

Q1. @WebMvcTest vs @DataJpaTest vs @SpringBootTest? @WebMvcTest is a web-layer slice with mocked services. @DataJpaTest is a JPA slice with repositories, each test in a rolled-back transaction. @SpringBootTest loads the full application context.

Q2. Why use Testcontainers instead of H2? Your tests run against the same database engine and version as production. So they catch differences in SQL dialect, locking, constraints and features that H2 would hide.

Q3. How would you test that concurrent redemptions never overdraw? Write an integration test against real Postgres. Fire N concurrent redemptions, each with a different idempotency key. Then assert the final balance and the exact number of successes.

8.5 Kafka with Spring: producers, consumers, idempotency and dead letters

Why?

> **In simple words:** Kafka is a durable notice board of events. One service pins up "order completed", and other services read it whenever they are ready — even if they were down for an hour. Spring Kafka makes writing and reading these events easy.

Services must react to each other's events. For example: order completed → accrue points → notify the retailer → update analytics.

But they should not depend on each other synchronously (waiting for a direct reply):

- If the loyalty service is down, ordering must still work.
- When loyalty comes back, it must process everything it missed.

Kafka gives you an event log that is durable (survives crashes), ordered and replayable (you can read it again). Spring Kafka makes producing and consuming events simple. The Kafka concepts are in 11.12–11.13; this section covers the Spring side.

What is happening internally?

- **Producer side.** `KafkaTemplate` wraps a thread-safe `KafkaProducer`. The producer groups records per partition into batches and sends them asynchronously. With `acks=all` plus producer idempotence (on by default since Kafka 3.0), producer retries cause neither lost nor duplicate records.
- **Consumer side.** `@KafkaListener` creates a *listener container*: 1. It starts one or more consumer threads (set by `concurrency`). 2. Each thread owns a `KafkaConsumer` in the consumer group. 3. Each consumer polls records from the partitions assigned to it and calls your method. 4. After your method returns, the container commits the offset (the "read up to here" bookmark). The default is `AckMode.BATCH`.
- **So delivery is at-least-once.** If the app crashes after processing but before the commit, the same record comes again. Your code must handle duplicates.
- **Error handling.** `DefaultErrorHandler` retries a failed record with a backoff (wait) by seeking back to it. After the retries run out, it gives the record to a *recoverer*. Usually that is `DeadLetterPublishingRecoverer`. It publishes the record to `<topic>.DLT` (dead-letter topic) with headers explaining the error.
- **Ordering is only per partition.** If you use `accountId` as the key, all events of one account go to the same partition, so they stay in order.

Think of the railway parcel office register. Each parcel is noted down. Every godown clerk marks the last entry he handled. If he goes on leave, he restarts from his mark — and may handle one parcel twice, so he must check before acting.

How does it work?

```
spring.kafka:
  bootstrap-servers: kafka:9092
  producer:
    acks: all
    key-serializer: org.apache.kafka.common.serialization.StringSerializer
    value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
```

```
  consumer:
    group-id: loyalty-accrual
    auto-offset-reset: earliest
    key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
    value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
    properties.spring.json.trusted.packages: "com.fa.events"
```

Example

```
public record OrderCompleted(String eventId, String tenantId, String outletId, String orderId, long
amountPaise, Instant at) {}

@Component
class AccrualConsumer {
    private final ProcessedEventRepository processed;
    private final AccrualService accrual;
    AccrualConsumer(ProcessedEventRepository processed, AccrualService accrual) { this.processed = processed;
this.accrual = accrual; }

    @KafkaListener(topics = "order-completed", concurrency = "3")
    @Transactional
    public void on(OrderCompleted evt) {
        if (processed.existsById(evt.eventId())) return;                        // duplicate delivery → no-op
        accrual.accrue(evt.tenantId(), evt.outletId(), evt.orderId(), evt.amountPaise());
        processed.save(new ProcessedEvent(evt.eventId(), Instant.now()));      // same DB transaction as the
effect
        // PK on event_id also protects against two consumers racing on a redelivered record
    }
}

@Configuration
class KafkaErrorConfig {
    @Bean
    DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
        var backoff = new ExponentialBackOffWithMaxRetries(3);
        backoff.setInitialInterval(500); backoff.setMultiplier(2.0);
        var handler = new DefaultErrorHandler(new DeadLetterPublishingRecoverer(template), backoff);
        handler.addNotRetryableExceptions(ValidationException.class, DeserializationException.class);   //
poison → DLT now
        return handler;
    }
}

@Service
class PointsEventPublisher {
    private final KafkaTemplate<String, Object> kafka;
    PointsEventPublisher(KafkaTemplate<String, Object> kafka) { this.kafka = kafka; }
    void publish(PointsEvent e) {
        kafka.send("points-events", e.accountId(), e)                           // key → partition → per-
account order
             .whenComplete((res, ex) -> { if (ex != null) log.error("publish failed {}", e.eventId(), ex);
});
    }
}
```

How the consumer stays safe against duplicates:

1. It checks if this `eventId` was already processed. If yes, it returns and does nothing.
2. It applies the accrual.
3. It saves the `eventId` in `processed_event` — in the **same** database transaction as the accrual.
4. The primary key on `event_id` blocks two consumers that race on the same redelivered record.

The error handler retries 3 times with exponential backoff (500 ms, then doubling). Validation and deserialisation errors skip the retries — those records are *poison* (they will never succeed), so they go to the DLT at once.

For **non-blocking retries**, use `@RetryableTopic`. It sends failed records through separate retry topics with delays, and finally to the DLT. This way one failing record does not stall its whole partition.

When would I use it?

- Event-driven integration between services.
- Asynchronous processing of high-volume events: accruals, notifications, analytics.
- CDC pipelines (Change Data Capture — streaming every database change as an event).
- Audit streams.

When the event must not be lost, publish from the **outbox relay** (11.15), not directly inside the business transaction.

What can go wrong?

- **Non-idempotent consumers** apply the effect twice when a record is redelivered. This is the root cause of "duplicate reward postings under retries".
- **Dual writes**: `repo.save(); kafka.send();` — two separate systems. The DB may commit while the send fails, or the reverse. Use the outbox pattern.
- **Poison messages** retried forever, blocking the partition. Classify exceptions as retryable or not, and use a DLT.
- **Slow processing.** If one poll takes longer than `max.poll.interval.ms`, Kafka kicks the consumer out of the group. This causes rebalance storms and redeliveries. Keep handlers fast, or reduce `max.poll.records`.
- **Hot keys.** If one huge tenant is the key, all its events land in one partition. So one consumer does all the work.
- **Schema changes** that break consumers. Use a schema registry, or allow only additive JSON changes (add fields, never remove or rename).

Interview questions

Q1. How does Spring Kafka deliver messages — exactly once? No — by default it is at-least-once. Offsets are committed after the listener returns, so a failure causes redelivery. So you make consumers idempotent. Kafka transactions give exactly-once only for Kafka-to-Kafka processing.

Q2. How do you make a consumer idempotent? Store the processed event ID, with a unique constraint, in the same DB transaction as the business effect. Or make the effect itself idempotent, for example an upsert by business key.

Q3. What is a dead-letter topic and when do you send to it? It is a separate topic for records that failed after all retries, or that can never succeed (validation or deserialisation errors). Moving them out stops them from blocking the partition. The DLT is monitored; records are fixed and replayed.

Q4. How do you preserve ordering per account? Use the account ID as the record key. All of that account's events then go to one partition and are processed by one consumer thread, in order.

8.6 Caching with Spring Cache and Redis

Why?

> **In simple words:** A cache keeps a copy of often-read data in fast memory. Next time, you read the

> copy instead of asking the database again.

Some data is read far more often than it changes: scheme definitions, SKU catalogues, tenant settings, computed balances.

Reading it from the database every time wastes DB capacity and adds latency. A cache serves repeated reads from memory:

- a local in-process cache answers in microseconds;
- Redis answers in about a millisecond.

Think of a shopkeeper who keeps the 20 most-sold items on the counter. He only goes to the godown for the rest.

What is happening internally?

- `@EnableCaching` makes Spring wrap beans that use caching annotations in a **proxy** (6.7). A proxy is a wrapper object that runs extra logic before and after your method.
- `@Cacheable` works like this: 1. Build the key. By default it comes from the method arguments. You can customise it with SpEL (Spring's expression language). 2. Look up the key in the `CacheManager`. 3. **Hit** (found): return the cached value. Your method is not called. 4. **Miss** (not found): call your method and store the result.
- `@CacheEvict` removes entries. `@CachePut` always calls the method and updates the cache.
- The `CacheManager` implementation decides where data is stored:
- **Caffeine** — in-process, one copy per instance, nanosecond access, bounded by size and TTL (time to live).
- **Redis** — shared by all instances, one network hop away, values are serialised, TTL per cache.
- `sync = true`: if many requests miss the same key at once on one instance, only one loads it; the others wait. This is local protection against a *stampede* (many requests hitting the DB together).

How does it work?

The pattern is called **cache-aside**:

1. On a read, look in the cache first. On a miss, load from the DB and store it.
2. On a write, update the DB, then **evict** (delete) the key.

Why delete instead of writing the new value into the cache? Because writing can race with a concurrent reader. That reader may put the old, stale value back in after your update. Deleting forces the next read to load fresh data.

Example

```
@Configuration
@EnableCaching
class CacheConfig {
    @Bean
    RedisCacheManager cacheManager(RedisConnectionFactory cf) {
        var defaults = RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(10))
                .serializeValuesWith(RedisSerializationContext.SerializationPair
                        .fromSerializer(new GenericJackson2JsonRedisSerializer()));
        return RedisCacheManager.builder(cf)
                .cacheDefaults(defaults)
                .withCacheConfiguration("balances", defaults.entryTtl(Duration.ofSeconds(60)))
                .build();
```

```
    }
}

@Service
class SchemeService {
    @Cacheable(cacheNames = "schemes", key = "#tenantId + ':' + #outletId", sync = true)
    public List<SchemeDto> eligible(String tenantId, String outletId) { return repo.findEligible(tenantId,
outletId); }

    @CacheEvict(cacheNames = "schemes", allEntries = true)
    public void onSchemesChanged(String tenantId) { }
}

@Service
class BalanceService {
    @Cacheable(cacheNames = "balances", key = "#accountId")
    public long balance(String accountId) { return ledger.balance(accountId); }

    @CacheEvict(cacheNames = "balances", key = "#accountId")
    @Transactional
    public void redeem(String accountId, long pts) { /* ... */ }      // evict after the change
}
```

Redis is useful for much more than caching (11.5, 13.x):

- atomic counters (`INCR`);
- rate limiting (a token bucket written in Lua);
- distributed locks (`SET key value NX PX 30000` , or Redisson);
- leaderboards (sorted sets);
- idempotency keys with a TTL;
- pub/sub and streams.

When would I use it?

For read-heavy, change-light data, expensive computations and per-tenant configuration.

- **Caffeine** — for small reference data where brief per-instance staleness is fine.
- **Redis** — when instances must share results, or the data set is large.
- **Both together** — a two-level cache: Caffeine first, then Redis, then the DB.

What can go wrong?

- **Stale data.** Two common causes:
- You evict before the transaction commits. Another request then caches the old value again.
- You evict on only one instance, while others still hold local copies.

Fix: evict after commit (`@TransactionalEventListener`), and keep TTLs as a safety net. - **Self-invocation.** `@Cacheable` on a method called from the same class does not cache. The call skips the proxy. - **Stampede.** A hot key expires and thousands of requests hit the DB at once. See 11.5: single-flight, jitter, early refresh. - **Unbounded caches.** The default `ConcurrentMapCacheManager` has no limits, so memory keeps growing — a memory leak. Always set a size limit and a TTL. - **Caching mutable objects locally.** Callers change the shared cached object, and everyone sees the change. Cache immutable DTOs or records. - **Serialisation changes.** After a deploy, old Redis entries may no longer be readable. Version your cache keys, or clear the cache on deploy.

Interview questions

Q1. How does @Cacheable work? A proxy builds a key from the method arguments. On a hit, it returns the cached value without calling the method. On a miss, it calls the method and stores the result.

Q2. Why evict on write instead of updating the cache? Updating can race with a concurrent read that reloads old data and writes it to the cache last, leaving stale data. Deleting forces the next read to load fresh data.

Q3. Local cache vs Redis? A local cache (Caffeine) is fastest, but each instance has its own copy and they can drift apart. Redis is shared and consistent across instances, but costs a network hop and serialisation.

8.7 Resilience: timeouts, retries, circuit breakers and bulkheads (Resilience4j)

Why?

> **In simple words:** Other services will sometimes be slow or down. These patterns make sure one sick dependency doesn't drag your whole service down with it — you stop waiting, stop calling it for a while, and show a sensible fallback.

In a distributed system, dependencies **will** be slow or down. Without protection, this happens:

1. One partner API becomes slow.
2. Your threads wait on it, and your thread pool fills up.
3. Your service stops responding to anyone.
4. Your callers' thread pools fill up too.
5. The outage **cascades** across the whole platform.

Resilience patterns contain the failure:

- fail fast instead of waiting;
- retry only what is safe to retry;
- isolate what is slow;
- degrade gracefully (give a reduced but useful answer).

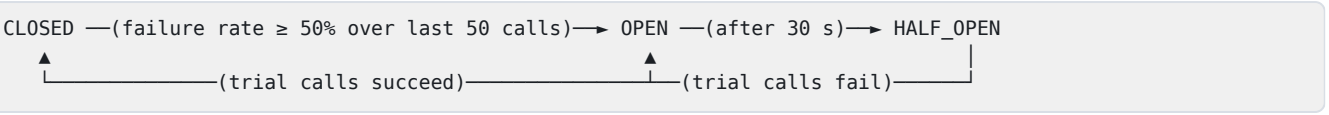
Think of the electrical circuit breaker (MCB) in your house. When there is a short circuit, it trips and cuts power to that line, so the whole house doesn't catch fire. After the fault is fixed, you switch it back on.

What is happening internally?

Resilience4j wraps (decorates) your calls. You use it with annotations and Spring AOP, or in functional style.

- **TimeLimiter / client timeouts** — limit how long you wait. Always set this first.
- **Retry** — calls again on the transient exceptions you list. It waits longer each time (exponential backoff), plus some randomness (jitter) so clients don't retry together.
- **CircuitBreaker** — keeps a *sliding window* of recent results (by call count or by time).
- If the failure rate or slow-call rate goes above a threshold, the breaker **opens**.
- While open, calls fail at once with `CallNotPermittedException`. The dependency is not touched.
- After a wait, it becomes **half-open** and lets a few trial calls through.

- If they succeed, it closes. If they fail, it opens again.
- **Bulkhead** — caps how many calls can run at once (with a semaphore), or gives the dependency its own thread pool. So one dependency cannot use up all your threads. (The name comes from the walls inside a ship that stop one leak from sinking the whole ship.)
- **RateLimiter** — caps the number of calls per time unit, to respect a partner's quota.



Order of decorators with annotations, outermost first: Retry → CircuitBreaker → RateLimiter → TimeLimiter → Bulkhead → your call.

Because Retry is outside the CircuitBreaker:

- every retry attempt is recorded by the breaker; and
- once the breaker opens, the retries fail fast too, so they stop quickly.

How does it work?

With this config, the breaker looks at the last 50 calls (and needs at least 20 before it decides). It opens if 50% fail, or if 60% take longer than 2 s. It stays open for 30 s, then allows 5 trial calls. Retry tries up to 3 times, starting at 200 ms and doubling, only for I/O and timeout errors. The bulkhead allows 20 calls at once and waits at most 50 ms for a free slot.

```yaml
resilience4j:
  circuitbreaker.instances.voucherPartner:
    sliding-window-type: COUNT_BASED
    sliding-window-size: 50
    minimum-number-of-calls: 20
    failure-rate-threshold: 50
    slow-call-duration-threshold: 2s
    slow-call-rate-threshold: 60
    wait-duration-in-open-state: 30s
    permitted-number-of-calls-in-half-open-state: 5
  retry.instances.voucherPartner:
    max-attempts: 3
    wait-duration: 200ms
    enable-exponential-backoff: true
    exponential-backoff-multiplier: 2
    retry-exceptions: [java.io.IOException, java.net.SocketTimeoutException]
    ignore-exceptions: [com.fa.voucher.InvalidVoucherRequestException]
  bulkhead.instances.voucherPartner:
    max-concurrent-calls: 20
    max-wait-duration: 50ms
```

Example

```java
@Service
class VoucherClient {
    private final RestClient client;
    VoucherClient(RestClient voucherClient) { this.client = voucherClient; }

    @Retry(name = "voucherPartner")
    @CircuitBreaker(name = "voucherPartner", fallbackMethod = "queueForLater")
    @Bulkhead(name = "voucherPartner")
    public VoucherResult issue(IssueVoucher cmd) {
        return client.post().uri("/v1/vouchers")
                .header("Idempotency-Key", cmd.redemptionId())          // makes the retried POST safe
                .body(cmd).retrieve().body(VoucherResult.class);
    }
```

```
    VoucherResult queueForLater(IssueVoucher cmd, Throwable t) {          // graceful degradation
        pendingVouchers.enqueue(cmd);                                     // a job retries later; user sees
"processing"
        return VoucherResult.pending(cmd.redemptionId());
    }
}
```

Two points to notice:

- The `Idempotency-Key` header makes the retried POST safe. The partner sees the same key and does not issue a second voucher.
- `queueForLater` is the fallback. When the breaker is open, the request goes into a queue, a background job retries it later, and the user sees "processing". This is graceful degradation.

When would I use it?

Around every remote dependency: partner APIs, other microservices, even the database for non-critical reads.

Combine it with:

- client timeouts (7.6);
- idempotency keys on any write that may be retried;
- fallbacks that keep core flows working — cached prices, "processing" states, hiding optional widgets.

What can go wrong?

- **Retrying non-idempotent operations** creates duplicates — double vouchers, double payments.
- **Retry storms.** If every layer retries 3 times, three layers multiply the load 27 times (3 × 3 × 3) on a dependency that is already struggling. Retry at one layer only, with backoff, jitter and a retry budget (a cap on total retries).
- **No timeout.** Calls just hang, so the circuit breaker never sees a failure.
- Fallbacks that hide real outages and raise no alert.
- Breaker windows that are too small, so the breaker keeps flipping open and closed (flapping). Or thresholds so high that it never opens.
- Self-invocation again: the annotations only work when the call goes through the proxy.

Interview questions

Q1. Explain the circuit breaker states. Closed: calls flow normally and results are recorded. It moves to Open when the failure or slow-call rate crosses a threshold; now calls fail fast. After a wait, it moves to Half-open and allows a few trial calls. If they succeed, it goes back to Closed; if they fail, it goes back to Open.

Q2. Which calls should you retry? Only idempotent operations that fail with transient errors — timeouts, 503s, connection resets. Use exponential backoff with jitter, a limited number of attempts, and ideally retry at one layer only.

Q3. What is a bulkhead? It isolates resources per dependency, using a semaphore or a thread-pool limit. So one slow dependency cannot use up all your threads.

Q4. How do these patterns prevent cascading failures? Timeouts limit how long you wait. Circuit breakers stop calling a failing dependency. Bulkheads limit how many resources one dependency can use. Fallbacks keep the caller responsive.

8.8 Async execution and scheduled jobs

Why?

> **In simple words:** `@Async` runs a task in the background so the user doesn't wait for it. `@Scheduled` runs a task at fixed times. The big catch: with many pods, every pod runs the scheduled job unless you add a lock.

Some work should not make the request wait: sending an SMS after a redemption, generating a report, warming a cache.

Other work runs on a timetable: daily point expiry at 02:00, hourly reconciliation, retrying pending vouchers.

Spring gives you `@Async` and `@Scheduled` for these. Both have pitfalls that matter when you run many instances.

What is happening internally?

- **`@Async`.** Add `@EnableAsync`. Then a proxy intercepts every `@Async` method. It hands the call to a `TaskExecutor` (a thread pool) and returns at once. The method returns `void` or a `CompletableFuture`. If you configure nothing, Boot gives you a `ThreadPoolTaskExecutor` — or virtual threads when `spring.threads.virtual.enabled=true`.
- **`@Scheduled`.** Add `@EnableScheduling`. A `TaskScheduler` then triggers `@Scheduled` methods by `fixedRate`, `fixedDelay` or `cron`. Note: by default it has only **one thread**!
- **Many instances.** If you run several pods, **every** pod runs every `@Scheduled` method. So a daily job runs N times unless you coordinate.

Think of a society with three watchmen and one rule: "lock the main gate at 10 pm". If nobody coordinates, all three walk to the gate. ShedLock is the register at the gate — the first one to sign does the job.

How does it work?

This config sets up a dedicated pool for notifications and a DB-based lock for jobs:

- 4 core threads, up to 8, and a queue of 500 tasks.
- When the queue is full, `CallerRunsPolicy` makes the caller run the task itself. This slows the caller down instead of dropping work.
- A `TaskDecorator` copies MDC and tenant context into the worker thread (4.15).
- ShedLock stores its locks in a `shedlock` table in your DB, so only one instance runs each job.

```
@Configuration
@EnableAsync
@EnableScheduling
@EnableSchedulerLock(defaultLockAtMostFor = "PT30M")         // ShedLock: one instance per job run
class AsyncConfig {
    @Bean("notificationExecutor")
    ThreadPoolTaskExecutor notificationExecutor() {
        var ex = new ThreadPoolTaskExecutor();
        ex.setCorePoolSize(4); ex.setMaxPoolSize(8); ex.setQueueCapacity(500);
        ex.setThreadNamePrefix("notify-");
        ex.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
        ex.setTaskDecorator(new MdcTaskDecorator());           // your TaskDecorator copying MDC/tenant
context (4.15)
        ex.initialize();
        return ex;
```

```
    }
    @Bean
    LockProvider lockProvider(DataSource ds) { return new JdbcTemplateLockProvider(ds); }   // shedlock table
in the DB
}
```

Example

```
@Service
class Notifier {
    @Async("notificationExecutor")
    public CompletableFuture<Void> sms(String phone, String text) {
        gateway.send(phone, text);
        return CompletableFuture.completedFuture(null);
    }
}

@Component
class ExpiryJob {
    @Scheduled(cron = "0 0 2 * * *", zone = "Asia/Kolkata")          // 02:00 IST daily
    @SchedulerLock(name = "pointsExpiry", lockAtMostFor = "PT2H", lockAtLeastFor = "PT5M")
    public void expirePoints() {
        for (String tenant : tenants.active()) {
            expiryService.expireDueLots(tenant);                     // chunked, resumable, idempotent per
lot
        }
    }
}
```

The expiry job runs at 02:00 IST every day. ShedLock holds the lock for at least 5 minutes and at most 2 hours. The work is chunked, resumable and idempotent per lot, so a crash in the middle is safe to restart.

When would I use it?

- `@Async` — for fire-and-forget side effects where losing one on a crash is acceptable (or a later job will fix it).
- **Durable queue (Kafka/outbox)** — for work that must not be lost.
- `@Scheduled` **+ ShedLock** — for periodic maintenance.
- **Kubernetes CronJobs or a workflow engine** — for heavy or long batch jobs.

What can go wrong?

- Self-invocation and private methods. Proxy rules again — these calls skip the proxy, so they run synchronously.
- An exception in a `void @Async` method disappears silently, unless you configure an `AsyncUncaughtExceptionHandler`.
- ThreadLocal context (tenant, security, MDC) is lost in async threads. Use a TaskDecorator to copy it.
- The default single scheduler thread. One slow job delays all the others. Configure a pool (`spring.task.scheduling.pool.size`).
- Jobs running on every pod — duplicate expiry or duplicate payouts — when there is no ShedLock or leader election. Also, jobs that are not idempotent and cannot resume after a crash.
- Time zones and DST (daylight-saving time changes) in cron expressions. Always set the `zone`.

Interview questions

Q1. How does @Async work and what are its limits? A proxy hands the call to an executor and returns immediately. Limits: it does not work on self-invocation or private methods; it loses ThreadLocal context unless you propagate it; and the work is lost if the app crashes.

Q2. How do you ensure a scheduled job runs once across multiple instances? Use a distributed lock (ShedLock on a DB or Redis), or leader election. Or run the job outside the app — a Kubernetes CronJob, or Quartz in clustered mode.

8.9 Observability: health, metrics, logs and traces

Why?

> **In simple words:** In production you can't debug line by line. So the service must report on itself: numbers that say something is wrong, traces that show where, and logs that explain why.

In production you cannot attach a debugger. You find problems only through what the system emits:

- **Metrics** tell you *that* something is wrong — for example, the error rate went up.
- **Traces** tell you *where* — which hop in the request is slow.
- **Logs** tell you *why* — the exception and its context.

Health endpoints let Kubernetes send traffic only to healthy instances.

Think of a car dashboard. The warning light (metric) says something is wrong. The mechanic's diagnostic scan (trace) finds which part. The service record (logs) explains what happened.

What is happening internally?

- **Actuator** adds management endpoints:
- `/actuator/health`, with `liveness` and `readiness` groups for Kubernetes probes;
- `/info`, `/metrics`, `/prometheus`;
- `/loggers` — change log levels at runtime;
- `/threaddump`, `/heapdump`, `/conditions`, `/startup`.
- **Micrometer** is the metrics facade — like SLF4J, but for metrics. You write counters, gauges, timers and distribution summaries, each with **tags** (labels). It exports them to Prometheus, Azure Monitor or Datadog. Boot measures many things for you automatically: HTTP server and client requests, JVM memory/GC/threads, Hikari connection pools, Kafka clients and caches.
- **Micrometer Tracing + OpenTelemetry** work like this: 1. Each request gets a trace ID. 2. A *span* (one timed step) is created for each HTTP call, DB query and Kafka message. 3. The trace context travels to the next service in W3C `traceparent` HTTP headers and in Kafka headers. 4. Spans are exported to Jaeger, Tempo, Zipkin or Application Insights.
- **Structured logging** means logs written as JSON. It is built into Boot 3.4+. Put the trace ID and tenant ID in every line using MDC (a per-thread map of log fields). Then logs are searchable and you can join them with traces.

How does it work?

What to measure:

- **RED** for each endpoint and dependency: **R**ate (requests per second), **E**rrors, **D**uration (p50/p95/p99). p95 means 95% of requests were faster than this.
- **USE** for each resource: **U**tilisation, **S**aturation (queue lengths, waits for a pool), **E**rrors. Apply it to thread pools, connection pools, Kafka consumer lag and GC.
- **Business KPIs**: redemptions per minute, accrual lag, failed vouchers.

Alert on **symptoms measured against SLOs**, not on every CPU blip. An SLO (service level objective) is a target such as "99.9% of redemptions succeed in < 500 ms". The *error budget* is the 0.1% you are allowed to fail. Alert when you are burning that budget too fast (the burn rate).

Example

```java
@Component
class RedemptionMetrics {
    private final MeterRegistry registry;
    private final Timer latency;
    RedemptionMetrics(MeterRegistry registry) {
        this.registry = registry;
        this.latency = Timer.builder("loyalty.redemption.latency")
                .publishPercentileHistogram()                    // enables p95/p99 in Prometheus
                .register(registry);
    }
    <T> T timed(Supplier<T> op) { return latency.record(op); }
    void outcome(String tenant, String result) {
        registry.counter("loyalty.redemptions", "result", result).increment();   // low-cardinality tags only
    }
}
```

```yaml
management:
  endpoints.web.exposure.include: health,info,prometheus
  endpoint.health.probes.enabled: true          # /actuator/health/liveness and /readiness
  tracing.sampling.probability: 0.1             # sample 10% of traces in prod
logging.structured.format.console: ecs          # Boot 3.4+: JSON logs
```

This config exposes only health, info and prometheus. It turns on the liveness and readiness probes, samples 10% of traces in production, and writes JSON logs.

When would I use it?

From day one, in every service:

- a dashboard per service (RED + USE);
- alerts on SLO burn;
- traces to investigate slow requests;
- logs linked to traces by trace ID.

In interviews, the question "how would you know it's working in production?" is answered with exactly these.

What can go wrong?

- **High-cardinality tags.** *Cardinality* means how many different values a tag can have. User IDs, order IDs or raw URLs as tags create millions of series. This blows up the metrics backend's memory and cost. Use tags with a small, fixed set of values: status, endpoint template, tenant tier.
- Exposing `/heapdump` or `/env` to the public. They can leak data. Keep management endpoints on an internal port or network.

- Liveness probes that check the database. If the DB has a short blip, Kubernetes restarts every pod. Liveness should reflect only the app process itself. Put dependency checks in readiness — and carefully.
- Logging PII or secrets. Logging at DEBUG in production. Logging the same exception in many layers.

Interview questions

Q1. Metrics vs logs vs traces? Metrics are aggregated numbers, used to detect problems and alert. Traces follow one request across services to find where the latency or error is. Logs are detailed events that explain what happened.

Q2. Liveness vs readiness probes? Liveness asks: is the process healthy, or should Kubernetes restart it? Readiness asks: can it serve traffic right now — is it warmed up and are its dependencies reachable? A failing readiness probe takes the pod out of load balancing without restarting it.

Q3. What are RED and USE? RED (rate, errors, duration) is for request-driven services. USE (utilisation, saturation, errors) is for resources such as pools, queues and CPU.

8.10 Microservices patterns in the Spring ecosystem

Why?

> **In simple words:** Breaking an app into many small services gives teams freedom, but brings new problems — network calls fail, data is spread out. Each common problem has a well-known pattern and a well-known tool.

Splitting a system into services gives you:

- independent deployment;
- independent scaling;
- clear team ownership.

The price is network calls, partial failures, data spread across services, and more operational work.

A set of well-known patterns solves the recurring problems. Interviewers expect you to:

1. name the patterns;
2. know the Spring or Kubernetes tool for each;
3. say when *not* to use microservices.

What is happening internally?

Concern	Pattern	Typical implementation
One entry point for all calls: routing, auth, rate limits	API Gateway / BFF (Backend for Frontend)	Spring Cloud Gateway, Azure API Management, Kong, Envoy
Finding where service instances are running	Service discovery	Kubernetes Services + DNS; Eureka outside Kubernetes
Keeping configuration in one central place	Externalised config	ConfigMaps/Secrets, Azure App Configuration, Spring Cloud Config

Concern	Pattern	Typical implementation
One failure spreading to others (cascading failures)	Timeouts, retries, circuit breakers, bulkheads	Resilience4j (8.7)
Who owns which data	Database per service	Each service owns its schema; no shared tables
Keeping data consistent across services	Saga (choreography/orchestration) + outbox	Kafka, Temporal/Camunda, outbox relay or Debezium CDC (11.15)
Queries that need data from many services	API composition, CQRS read models	An aggregator service; a read model fed by events
Moving away from a monolith	Strangler fig	Gateway sends new paths to new services, step by step
Seeing what happens across services	Correlation IDs, distributed tracing, central logs	Micrometer Tracing + OpenTelemetry (8.9)
Deployment	Containers, orchestration, progressive delivery	Docker, Kubernetes, Helm/Kustomize, Argo/Flagger canaries
Security between services	mTLS, OAuth2 client credentials	Istio/Linkerd service mesh, Spring Security (8.2)

A few terms from the table:

- **Saga** — a long business process split into local steps. Each step has a *compensation* (an undo step) if a later step fails. In *choreography*, services react to each other's events. In *orchestration*, one coordinator tells each service what to do.
- **CQRS** — keep separate models for writing and for reading. The read model is built from events.
- **Strangler fig** — named after a plant that slowly grows around a tree. You move features out of the monolith one route at a time.
- **Canary** — send a small share of traffic to the new version first.

How does it work?

Draw service boundaries around **business capabilities**, also called *bounded contexts*: Ordering, Loyalty, Incentives, Notifications, Catalogue.

- Each service owns its own data.
- It publishes events when its data changes.
- Other services keep a local copy of what they need, instead of calling synchronously for everything. The copy may be a little behind for a short time. This is called *eventual consistency*.

Think of the departments in a big hospital. Billing, pharmacy and lab each keep their own records. When the lab finishes a test, it sends a slip to billing. Billing does not walk into the lab and read its register.

Example

```
Mobile app ─► API Gateway (JWT validation, rate limits, routing)
                ├─► Order service ──(OrderCompleted via outbox→Kafka)──► Loyalty service ──► Notification
service
                ├─► Loyalty service (owns ledger DB) ──► Voucher partner (Resilience4j, idempotency key)
```

```
                └→ Catalogue service (read-heavy, cached, CDN for images)
Each service: own DB, own deploy pipeline, health probes, metrics, traces with a shared trace ID
```

When would I use it?

When teams and domains are large enough that independent deployment and scaling pay for the extra complexity.

For a small team or a new product, a **modular monolith** is often the better senior answer. That means one deployable app with clear internal modules. Extract a module into a service only when its scaling or ownership needs become different from the rest.

What can go wrong?

- **Distributed monolith.** Services must be deployed together, and they call each other synchronously in long chains. Latency adds up. Availability multiplies down: 5 services at 99.9% each, called in series, give about 99.5% (0.999^5).
- **Shared database between services.** This is hidden coupling. One team's schema change breaks another team's service.
- Chatty APIs (N calls to render one page), missing timeouts, no idempotency, and data that drifts apart with no reconciliation job.
- Operational load without platform support — no proper CI/CD, observability or on-call setup for each service.

Interview questions

Q1. Monolith vs microservices — how do you decide? Weigh the benefits — team size, clear domain boundaries, need for independent scaling and deployment — against the costs: operational complexity, network failure modes and data consistency. Start with a modular monolith, and split along bounded contexts when it is justified.

Q2. How do services stay consistent without distributed transactions? Use local transactions plus events (the outbox pattern), sagas with compensation steps, idempotent consumers and reconciliation jobs. You accept eventual consistency.

Q3. What does an API gateway do? Routing, authentication and token validation, rate limiting, TLS termination, request shaping, CORS, observability and canary routing. It should contain no business logic.

8.11 The Java project — your proof of the switch

> **In simple words:** Build one real Spring Boot service from your own loyalty domain. It proves you can write production Java, and it gives you strong stories for every interview round.

Build this in Weeks 8–12. Use the Sunday 2-hour slots, plus whatever spare time you find.

It is deliberately taken from your own domain. So every interview question about it lands on ground you already know well.

Loyalty Ledger Service — spec

Purpose: a multi-tenant points ledger. It supports:

- accrual (from order events);

- redemption (through the API);
- reversal;
- expiry;
- balance queries.

It must stay correct under retries and concurrency — no double postings, no negative balances.

Stack: Java 21, Spring Boot 3.x, PostgreSQL, Spring Data JPA, Flyway, Kafka, Redis, Resilience4j, Spring Security (JWT resource server), Testcontainers, Docker Compose, GitHub Actions.

Endpoints

Method	Path	Notes
POST	`/api/v1/accounts`	Creates an account for a tenant + outlet
GET	`/api/v1/accounts/{id}/balance`	Cached in Redis; evicted when the balance changes
POST	`/api/v1/accounts/{id}/redemptions`	`Idempotency-Key` header is required; returns 201 / 422 / 409
POST	`/api/v1/accounts/{id}/reversals`	Reverses a ledger entry by its id
GET	`/api/v1/accounts/{id}/ledger?after=<cursor>`	Keyset pagination (page by "after this id", not by offset)
—	Kafka `order-completed`	Idempotent consumer → accrual
—	Kafka `points-events`	Published through the outbox
—	Daily 02:00 IST job	Expires points older than N months (ShedLock)

Schema (Flyway V1)

Key points of the schema:

- `CHECK (balance >= 0)` — the database itself refuses a negative balance.
- `UNIQUE (tenant_id, idempotency_key)` — the same request can never create two ledger entries.
- The index on `(account_id, created_at DESC, id DESC)` serves the keyset-paginated ledger query.
- The partial index on `outbox` covers only unpublished rows, so the relay's poll stays fast.

```
CREATE TABLE account (
  id            VARCHAR(40) PRIMARY KEY,
  tenant_id     VARCHAR(40) NOT NULL,
  outlet_id     VARCHAR(40) NOT NULL,
  balance       BIGINT NOT NULL DEFAULT 0 CHECK (balance >= 0),
  version       BIGINT NOT NULL DEFAULT 0,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, outlet_id)
);
CREATE TABLE ledger_entry (
  id              BIGSERIAL PRIMARY KEY,
  tenant_id       VARCHAR(40) NOT NULL,
  account_id      VARCHAR(40) NOT NULL REFERENCES account(id),
  type            VARCHAR(20) NOT NULL,          -- ACCRUAL, REDEMPTION, REVERSAL, EXPIRY
  points          BIGINT NOT NULL,               -- signed
  idempotency_key VARCHAR(80) NOT NULL,
  ref_entry_id    BIGINT,                         -- for reversals
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, idempotency_key)
);
```

```
CREATE INDEX ix_ledger_account_created ON ledger_entry (account_id, created_at DESC, id DESC);
CREATE TABLE outbox (
  id           UUID PRIMARY KEY,
  aggregate_id VARCHAR(40) NOT NULL,
  topic        VARCHAR(80) NOT NULL,
  payload      JSONB NOT NULL,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  published_at TIMESTAMPTZ
);
CREATE INDEX ix_outbox_unpublished ON outbox (created_at) WHERE published_at IS NULL;
CREATE TABLE processed_event (event_id VARCHAR(80) PRIMARY KEY, processed_at TIMESTAMPTZ NOT NULL);
```

Package layout

```
com.mayank.ledger
├── api            controllers, request/response records, GlobalExceptionHandler
├── application    RedemptionService, AccrualService, ReversalService, ExpiryJob
├── domain         Account, LedgerEntry, EntryType, domain exceptions, PointsPolicy
├── infrastructure repositories, Kafka consumer, OutboxRelay, Redis config, security config
└── config         properties records, executors
```

Milestones

1. Week 8: create the repo; Docker Compose with Postgres, Kafka and Redis; Flyway V1; the `Account` + `LedgerEntry` entities; create/get endpoints; CI running the tests.
2. Week 9: redemption with an idempotency key + an atomic conditional debit (one SQL update that subtracts only if the balance is enough); `ProblemDetail` errors; validation.
3. Week 10: reversal; keyset pagination; the concurrency integration test (20 threads); a note in the README comparing optimistic locking with the atomic update.
4. Week 11: the Kafka idempotent accrual consumer; the outbox table + relay (a `@Scheduled` poller that publishes unpublished rows); the DLT.
5. Week 12: Redis balance cache; Resilience4j around a fake "rewards catalogue" client; JWT security; Actuator + Prometheus metrics; a k6 load test (record p95 and throughput); a README with an architecture diagram and trade-offs.

README must contain:

- an architecture diagram;
- how idempotency works, in both the API and the consumer;
- why you chose an atomic SQL update over `@Version` for the balance, and when you would choose each;
- the concurrency test;
- the load-test numbers;
- what you would do next at 10× scale — partition the ledger by tenant, add read replicas, use CDC instead of polling the outbox.

> **Say this:** "At FieldAssist the loyalty workflows run on .NET. To make my Java switch concrete I rebuilt the core of that ledger in Spring Boot — idempotent redemption, an idempotent Kafka consumer, transactional outbox, and a test that fires 20 concurrent redemptions and proves we never overdraw."

Part 3 — Low-Level Design

An LLD round is also called a *machine coding* round or an *object-oriented design* round. The interviewer gives you a loose, unclear problem. In 60–90 minutes, you must turn it into clean Java code that works and is easy to extend.

This part has two chapters:

- **Chapter 9 is your toolkit.** It covers OOP, the SOLID principles and the main design patterns. Each one is explained in the same seven steps.
- **Chapter 10 uses the toolkit.** It solves 17 complete LLD problems.

9. LLD Foundations: OOP, SOLID and Design Patterns

9.1 OOP pillars as design tools

Why?

> **In simple words:** The four OOP pillars are not definitions to recite. They are questions you ask while designing. If your design skips one, the interviewer will notice and ask about it.

Interviewers will not ask you to *define* encapsulation. Instead, they watch whether your design actually *uses* it.

Each pillar answers one design question. If you forget a pillar, the design shows a clear warning sign. This warning sign is called a *smell*. Interviewers know these smells well, and they will dig into them.

What is happening internally?

The table shows the question each pillar answers, and the smell you see when it is missing.

Pillar	Design question it answers	Smell when missing
Encapsulation	Who owns this data, and who checks its rules?	Public setters everywhere; logic sits in "manager" classes that change other objects' fields directly
Abstraction	What does the caller really need to know?	Callers depend on concrete classes and their inner details
Inheritance	Is this truly an "is-a" relationship, with shared behaviour that rarely changes?	Very deep class trees; subclasses override methods just to throw `UnsupportedOperationException`
Polymorphism	How do I add a new type without editing old code?	`if/else` or `switch` on type codes copied all over the codebase

Rich vs anaemic domain model

- In a **rich model**, behaviour lives next to the data it protects. For example, `account.debit(points)` itself refuses to let the balance go negative.
- In an **anaemic model**, entities are just bags of data. Services read the fields, do the calculation and write the values back. So every service must remember every rule. If one service forgets, the rule is broken.

Think of a bank locker. In a rich model, the locker has its own lock and checks your key. In an anaemic model, the locker is an open box, and every staff member must remember to check your key.

How does it work?

While designing, ask three questions for each class:

1. What rules (*invariants* — things that must always be true, like "balance is never negative") does this class own?
2. What is the smallest public API that still lets callers do their job?

3. Where will change come from? New types, new rules, new channels?

Put an interface exactly at the place where change will come from — and nowhere else.

Example

```
// Anaemic: the rule "can't redeem more than balance" lives in some service — or nowhere
class Wallet { public long balance; }

// Rich: the object protects its own invariant
final class Wallet {
    private long balance;
    void credit(long pts) { if (pts <= 0) throw new IllegalArgumentException(); balance += pts; }
    void debit(long pts)  { if (pts > balance) throw new InsufficientPointsException(pts, balance); balance -
= pts; }
    long balance() { return balance; }
}

// Polymorphism instead of type switches
interface RewardRule { long points(Order o); }          // a new rule type = a new class, no edits elsewhere
```

When would I use it?

In every LLD answer. Say these things out loud:

- which class owns which rule,
- where the abstraction boundary is (what callers can and cannot see), and
- how a new variant plugs in without editing old code.

What can go wrong?

- **Over-abstraction:** making an interface for every class "just in case".
- **Fake encapsulation:** thinking that private fields with getters and setters is encapsulation. It is not, if anyone can still set any value.
- **Inheritance only to reuse code:** using `extends` just to borrow methods, when there is no real "is-a" relationship.

Interview questions

Q1. How do OOP principles show up in your design? Encapsulation: each entity checks and protects its own rules (invariants). Abstraction: callers depend on interfaces such as `PaymentGateway`, not on concrete classes. Polymorphism: a new variant is just a new class. Inheritance: I use it only for a real is-a relationship; otherwise I use composition (one object holding another).

Q2. What's an anaemic domain model and why is it a smell? It is a model where entities have only data plus getters and setters, and all the logic sits in services. It is a smell because no class owns the rules. So rules get copied into many services, or someone forgets them.

9.2 Class relationships and UML in interviews

Why?

> **In simple words:** Draw a quick box-and-arrow picture of your classes before you code. It gets you and the interviewer on the same page, and the arrows show who owns what.

A quick class diagram takes about five minutes. It makes sure you and the interviewer agree on the design before you write code.

The relationship types are association, aggregation, composition, inheritance and dependency. Using them correctly tells the interviewer you understand **ownership and lifecycles**. Ownership means "who is in charge of this object". Lifecycle means "when it is created and when it is deleted". These two things decide which class creates and deletes which object.

What is happening internally?

Relationship	Meaning	Java shape	UML
Dependency	Uses another class for a short time	A method parameter or a local variable	dashed arrow
Association	Knows about another object	A field that points to another object	plain line
Aggregation	Has-a; the part can live on its own	A field holding objects created somewhere else (`Team` → `Player`)	hollow diamond at the whole
Composition	Owns; the part dies when the whole dies	The whole creates and manages its parts (`Order` → `OrderLine`)	filled diamond at the whole
Inheritance	Is-a	`extends`	hollow triangle
Realisation	Implements an interface	`implements`	dashed line + hollow triangle

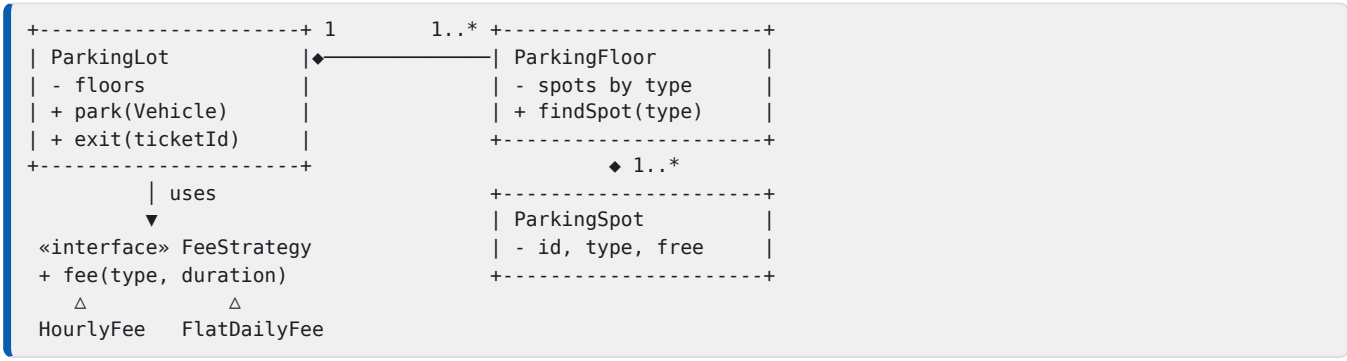
Multiplicity means "how many objects are on each side of the link". You write it as `1`, `0..1`, `*` (any number) or `1..*` (one or more).

A simple way to remember aggregation vs composition: a cricket team *has* players, but if the team is dissolved, the players still exist (aggregation). A tiffin box *has* compartments, and if you throw away the box, the compartments go with it (composition).

How does it work?

1. Draw a box for each class.
2. In the box, write the class name, 2–4 key fields and the key methods.
3. Draw the arrows between boxes.
4. Mark interfaces with «interface».
5. Do not draw getters, setters or every field.
6. Spend five to eight minutes at most.

Example



When would I use it?

- At the start of every LLD round, right after you clarify the requirements.
- In behavioural rounds, when you explain the design of a system you built.

What can go wrong?

- Spending 20 minutes making a perfect diagram.
- Drawing implementation details instead of the main structure.
- Forgetting multiplicities. These matter because they decide your data structures. For example, "1..* spots" means you need a collection of spots, keyed by spot type.

Interview questions

Q1. Aggregation vs composition? Both are has-a relationships. In composition, the whole owns the part's lifecycle — an order's lines are deleted with the order. In aggregation, the part lives independently — players still exist after a team is broken up.

Q2. Association vs dependency? Association is a lasting link in the structure, usually a field. Dependency is short-term use, usually a method parameter or a local variable.

9.3 Single Responsibility Principle (SRP)

Why?

> **In simple words:** A class should do one kind of job, for one kind of person. Then a change asked by one team touches only that team's class.

Suppose one class does many jobs. Then it changes for many reasons. A tax rule change, a database change and an email template change all edit the same file.

Because of this:

- Different people's changes clash in code reviews.
- A change for one reason can break something unrelated.

SRP keeps each class focused on one job. So changes stay local, and testing is simple.

What is happening internally?

The rule is: "A class should have one reason to change."

"One reason" means one **actor** — one person, team or concern that asks for changes. Examples: finance rules, saving to the database (*persistence*), or how things look on screen (*presentation*).

So you split responsibilities by *who asks for changes*. You do not split them by counting methods.

How does it work?

1. Look at a class.
2. List everyone who could ask for a change to it.
3. If there is more than one such stakeholder, split the class along those lines.
4. Add a small *coordinator* class that calls the pieces in the right order.

Example

```
// Before: calculation + persistence + rendering + email in one class
class InvoiceService {
    Invoice create(Order o) { /* tax + discount rules */ return null; }
    void save(Invoice i)    { /* SQL */ }
    byte[] pdf(Invoice i)   { /* layout */ return null; }
    void email(Invoice i)   { /* SMTP */ }
}

// After: each piece changes for one reason
class InvoiceCalculator { Invoice create(Order o) { return null; } }        // finance rules
interface InvoiceRepository { void save(Invoice i); }                       // persistence
class InvoicePdfRenderer { byte[] render(Invoice i) { return null; } }      // presentation
class InvoiceMailer { void send(Invoice i, byte[] pdf) { } }                // delivery
class InvoiceWorkflow {                                                     // coordinator
    private final InvoiceCalculator calc; private final InvoiceRepository repo;
    private final InvoicePdfRenderer pdf; private final InvoiceMailer mailer;
    InvoiceWorkflow(InvoiceCalculator c, InvoiceRepository r, InvoicePdfRenderer p, InvoiceMailer m) { calc =
c; repo = r; pdf = p; mailer = m; }
    void issue(Order o) { Invoice i = calc.create(o); repo.save(i); mailer.send(i, pdf.render(i)); }
}
record Order() {} record Invoice() {}
```

When would I use it?

Split a class when you see any of these signs:

- It has grown beyond about 200–300 lines.
- It has unrelated dependencies — for example a repository *and* an SMTP (email) client *and* a PDF library.
- Its tests need many unrelated mocks.

What can go wrong?

You can over-split. Dozens of one-method classes are hard to find your way around.

Group by *cohesion* instead: things that change together should stay together.

Interview questions

Q1. Explain SRP with an example. Each class should have one reason to change. For invoices, I keep calculation (finance rules), persistence (database), PDF rendering and delivery (email) in separate classes. A workflow class coordinates them.

9.4 Open/Closed Principle (OCP)

Why?

> **In simple words:** When you need a new type of something, you should add a new class — not edit an old, tested `switch`. Old code stays safe; new code plugs in.

Imagine every new reward type, payment method or notification channel means editing one central `switch`. Each edit can break the cases that already work. So you must retest all of them every time.

OCP says code should be **open for extension, closed for modification**. In plain words: add new behaviour by *adding* new code, not by *editing* code that is already tested.

What is happening internally?

1. Each place where behaviour varies (a *variation point*) becomes an **abstraction** — an interface.
2. Each variant becomes one class that implements this interface.
3. The core logic depends only on the interface.
4. The core logic receives the variants from outside — through dependency injection or a registry (a lookup map).

So polymorphism replaces the `if/else` chains.

Think of a mobile charger socket. The socket stays the same. To support a new phone, you buy a new cable that fits the socket — you do not rewire the wall.

How does it work?

1. Find a `switch` or `if-else` on a type code that keeps growing.
2. Pull out an interface with the operation that varies.
3. Move each branch into its own class.
4. Pick the right class at runtime from a map or a factory.

Example

```
// Before: every new reward type edits this method
long points(Order o, String type) {
    switch (type) {
        case "FLAT": return 100;
        case "PERCENT": return o.amount() / 10;
        default: throw new IllegalArgumentException(type);
    }
}

// After: new types are new classes; the engine never changes
interface RewardStrategy { String type(); long points(Order o); }
class FlatReward implements RewardStrategy { public String type() { return "FLAT"; } public long points(Order
o) { return 100; } }
class PercentReward implements RewardStrategy { public String type() { return "PERCENT"; } public long
points(Order o) { return o.amount() / 10; } }

class RewardEngine {
    private final Map<String, RewardStrategy> byType;
    RewardEngine(List<RewardStrategy> strategies) {                  // Spring injects all implementations
(6.4)
        byType = strategies.stream().collect(Collectors.toMap(RewardStrategy::type, s -> s));
    }
    long points(Order o, String type) { return Optional.ofNullable(byType.get(type)).orElseThrow().points(o);
}
}
record Order(long amount) {}
```

When would I use it?

Wherever you expect new variants to keep coming: pricing, fees, rule types, channels, export formats, payment providers.

What can go wrong?

You may predict change that never comes. This is *speculative abstraction*. It breaks YAGNI — "You Aren't Gonna Need It".

So apply OCP where change has already happened, or is clearly planned.

Interview questions

Q1. Explain OCP with an example from your work. In our incentive and reward system, each new rule type is a new Strategy class, registered by its type name. The evaluation engine does not change when we add a type.

9.5 Liskov Substitution Principle (LSP)

Why?

> **In simple words:** A child class must behave the way its parent promised. If code works with the parent, it must still work when you pass in the child.

Polymorphism only works if every subtype can replace its base type without surprising the caller.

Suppose a subclass breaks the base class's promise (its *contract*). For example, it throws where the base does not, ignores an input, or gives a weaker guarantee. Then bugs appear in other code — code that was correct — far away from the subclass. These bugs are hard to trace.

What is happening internally?

A subtype must keep the base type's **contract**. This means four things:

1. **No stronger preconditions.** A *precondition* is what the method needs before it runs. The subtype must accept at least everything the base accepts.
2. **No weaker postconditions.** A *postcondition* is what the method promises after it runs. The subtype must promise at least what the base promises.
3. **Keep the invariants.** Rules that are always true for the base must stay true.
4. **No new surprise exceptions.** Do not throw exceptions that callers of the base would not expect.

Think of a substitute driver for your office cab. You don't care who drives, as long as he follows the same route and reaches on time. If the substitute refuses to take the highway, the "substitute" has broken the deal.

How does it work?

Test it with this question: "Can every caller of the base type use this subtype correctly, without knowing it is a subtype?"

If callers need `instanceof` checks or special cases, LSP is broken.

To fix it, redesign the hierarchy. Use separate interfaces for separate capabilities.

Example

```
// Violation 1: the classic Square/Rectangle
class Rectangle { protected int w, h; void setWidth(int w) { this.w = w; } void setHeight(int h) { this.h =
h; } int area() { return w * h; } }
class Square extends Rectangle {
    @Override void setWidth(int w) { this.w = w; this.h = w; }      // breaks the caller's expectation
    @Override void setHeight(int h) { this.w = h; this.h = h; }
}
// Caller: r.setWidth(5); r.setHeight(4); assert r.area() == 20  → fails for Square (16)

// Violation 2: capability not supported
class ReadOnlyAccount extends Account { @Override void debit(long p) { throw new
UnsupportedOperationException(); } }
```

```java
// Fix: model capabilities separately
interface Shape { int area(); }
record Rect(int w, int h) implements Shape { public int area() { return w * h; } }
record Sq(int side) implements Shape { public int area() { return side * side; } }

interface Readable { long balance(); }
interface Debitable extends Readable { void debit(long p); }
class Account { void debit(long p) {} }
```

When would I use it?

- When you design class hierarchies.
- When you review subclasses that override a method only to throw or to do nothing.

What can go wrong?

Using inheritance because something is "is-a" in English. A square *is* a rectangle in maths class. But in code, a `Square` cannot behave like a `Rectangle`, because its width and height cannot change separately. What matters is whether one can safely *replace* the other in code — this is called *behavioural substitutability*.

Interview questions

Q1. What is LSP and give a violation? You must be able to use a subtype anywhere the base type is expected, without breaking correctness. Two classic violations: `Square` overriding `Rectangle`'s setters so width and height change together, and a subclass that throws `UnsupportedOperationException` for a base-class method.

9.6 Interface Segregation Principle (ISP)

Why?

> **In simple words:** Don't make one giant interface for everything. Make small interfaces, so each class implements only what it can really do, and each caller sees only what it really uses.

A "fat" interface has too many methods. This causes two problems:

1. Every class that implements it must provide methods it cannot support. Usually it just throws for them.
2. Every client depends on methods it never calls. So a change to an unrelated method still affects it.

What is happening internally?

Split a large interface into small **role interfaces**. Each role interface covers one need of one kind of client.

- One class can implement several roles.
- A client depends only on the role it uses.

Think of a restaurant menu. A tea stall does not need to print the full five-star menu and then write "not available" next to most items. It prints only what it serves.

How does it work?

1. Group the methods by which client calls them.
2. Turn each group into its own interface.

Example

```
// Fat interface: a simple printer must "implement" fax
interface Machine { void print(Doc d); void scan(Doc d); void fax(Doc d); }

// Segregated
interface Printer { void print(Doc d); }
interface Scanner { void scan(Doc d); }
interface Fax { void fax(Doc d); }
class SimplePrinter implements Printer { public void print(Doc d) { } }
class OfficeMachine implements Printer, Scanner, Fax { public void print(Doc d) { } public void scan(Doc d) {
} public void fax(Doc d) { } }
record Doc() {}

// Domain example: readers of a ledger shouldn't depend on posting
interface LedgerReader { long balance(String accountId); List<String> statement(String accountId); }
interface LedgerWriter { void post(String accountId, long points, String idemKey); }
```

When would I use it?

- When an interface has many methods, and some implementations leave methods empty or make them throw.
- When read clients and write clients are different. This read/write split is also the basis of CQRS (Command Query Responsibility Segregation — separate models for writes and reads).

What can go wrong?

Splitting into one method per interface with no reason. Keep each interface *cohesive* — its methods should belong together.

Interview questions

Q1. What is ISP? Clients should not depend on methods they don't use. So I prefer small, role-specific interfaces over one fat interface.

9.7 Dependency Inversion Principle (DIP)

Why?

> **In simple words:** Your business logic should not know which SMS company or which database you use. It talks to an interface it owns; the SMS or database code plugs into that interface from outside.

Suppose business logic directly creates infrastructure objects, like `new TwilioClient()` or `new PostgresRepo()`. Then two problems follow:

- Every infrastructure change ripples into business code.
- You cannot test the business logic without the real infrastructure.

DIP fixes this. It makes **both** sides depend on an abstraction (an interface), and the business side owns that interface.

What is happening internally?

1. The high-level policy (for example, the redemption rules) defines **ports**. A port is an interface the business needs, such as `NotificationSender` or `LedgerRepository`.

2. The low-level details (Twilio, Postgres) implement these ports. These implementations are called **adapters**.
3. Now the source-code dependency points *from* the detail *to* the abstraction. Normally the business code would point to the detail. This direction is now flipped — that is the "inversion".

This idea is the core of hexagonal architecture (also called *ports and adapters*) and of clean architecture. DI containers like Spring connect the adapters to the ports for you (6.2).

Think of a wall plug socket. The socket's shape is fixed by the house, not by the appliance. Any appliance — a fan, a mixer, a phone charger — makes its plug fit the socket.

How does it work?

The diagram shows the dependency before and after.

```
Before:  RedemptionService ──► TwilioClient        (policy depends on detail)
After:   RedemptionService ──► «interface» NotificationSender ◄── TwilioSender
                                                              ◄── WhatsAppSender
                                                              ◄── FakeSender (tests)
```

Example

```
// Port owned by the domain
public interface NotificationSender { void send(String to, String message); }

// Policy depends only on the port
public class RedemptionService {
    private final NotificationSender notifier;
    public RedemptionService(NotificationSender notifier) { this.notifier = notifier; }
    public void redeem(String phone, long pts) { /* rules */ notifier.send(phone, "Redeemed " + pts + "
points"); }
}

// Adapters at the edge
class TwilioSender implements NotificationSender { public void send(String to, String msg) { /* Twilio SDK */
} }
class RecordingSender implements NotificationSender {                    // for tests
    final List<String> sent = new ArrayList<>();
    public void send(String to, String msg) { sent.add(to + ":" + msg); }
}
```

When would I use it?

At every boundary with infrastructure or outside systems: the database, messaging, HTTP partners, clocks, file storage.

What can go wrong?

- **Leaky interfaces:** the interface copies one vendor's API exactly. Then the abstraction leaks that vendor's concepts into your business code.
- **Needless interfaces:** making interfaces for internal classes that will never have a second version.

Interview questions

Q1. Explain DIP. Both high-level and low-level modules depend on abstractions. The high-level (business) side owns these abstractions. The low-level details implement them and are injected from outside.

Q2. DIP vs DI? DIP is a design principle — it is about which direction dependencies point. DI (dependency injection) is a technique — you pass dependencies in from outside instead of creating

them inside. Containers like Spring automate DI.

9.8 Singleton

Why?

> **In simple words:** Some objects should exist only once in the whole program. A Singleton makes sure only one is ever created, and everyone shares it.

Some things must exist exactly once per process. Examples: a configuration registry, an ID generator, a metrics registry.

If you create many of them, you waste resources. Worse, each copy may hold different data, so the state becomes inconsistent. For example, two ID generators could hand out the same ID.

What is happening internally?

The class hides its constructor, so nobody else can call `new`. It gives out one shared instance instead.

In Java, the safe versions depend on guarantees from the JVM:

- **Holder idiom:** the JVM loads a class only when it is first used (lazy), and it does this in a thread-safe way. So a static field inside a nested holder class is created once, safely, on first use.
- **Enum:** each enum constant is created exactly once. Java also protects enums from the two common attacks on singletons — creating a second copy using reflection, or by serialising and deserialising.

How does it work?

```
public enum IdGenerator {                          // 1. Enum: simplest, safest
    INSTANCE;
    private final AtomicLong seq = new AtomicLong();
    public long next() { return seq.incrementAndGet(); }
}

public final class AppConfig {                     // 2. Holder idiom: lazy, thread-safe, no locks
    private AppConfig() { }
    private static final class Holder { static final AppConfig INSTANCE = new AppConfig(); }
    public static AppConfig get() { return Holder.INSTANCE; }
}
// 3. Double-checked locking with volatile — see 4.6
```

Example

`long id = IdGenerator.INSTANCE.next();`.

In Spring, you rarely write singletons yourself. Every bean is already a singleton per container by default. Spring injects it wherever it is needed, and you can replace it with a fake in tests.

When would I use it?

- In plain Java (LLD rounds without Spring): for stateless utilities or truly global resources.
- In Spring apps: prefer singleton-scoped beans instead.

What can go wrong?

- **Hidden global state:** tests share the same instance, so one test can affect another.
- **Tight coupling:** calls like `X.getInstance()` all over the code cannot be mocked easily.

- **Thread safety:** if the singleton holds data that changes (mutable), you must make it thread-safe.
- **Bad lazy initialisation:** if you create it lazily without proper synchronisation, two threads can create two instances. Or one thread can see a half-built object.

Interview questions

Q1. How do you implement a thread-safe singleton? Use an enum singleton, or the holder idiom. Another option is double-checked locking with a `volatile` field.

Q2. Why are singletons criticised? They create global state that can change, hide dependencies, make testing hard and couple code tightly. A single instance supplied through dependency injection avoids these problems.

9.9 Factory patterns: simple factory, factory method, abstract factory

Why?

> **In simple words:** A factory is one place that decides which class to create and how to set it up. The rest of the code just asks the factory and gets an object.

Suppose `new ConcreteClass()` is written all over the code. Two problems follow:

- Callers are tied to concrete types.
- The creation logic is copied everywhere — which class to pick for which input, and how to configure it.

A factory puts this in one place. It decides *which* class to create and *how*.

Think of a railway reservation counter. You say "Sleeper, Delhi to Varanasi". The clerk decides which coach and berth to give you. You don't walk into the train and pick a seat yourself.

What is happening internally?

There are three kinds of factory:

- **Simple factory.** It is not an official GoF ("Gang of Four" design patterns book) pattern, but it is the most common. A static method picks the class based on a parameter.
- **Factory Method (GoF).** An abstract creator class declares a `create()` method. Subclasses decide which concrete product to return. The creator's other logic works only with the product interface.
- **Abstract Factory (GoF).** An interface that produces a **family** of related products that must be used together. For example, all the components for one payment provider.

How does it work?

```
// Simple factory
enum VehicleType { BIKE, CAR, TRUCK }
sealed interface Vehicle permits Bike, Car, Truck { VehicleType type(); }
record Bike(String plate) implements Vehicle { public VehicleType type() { return VehicleType.BIKE; } }
record Car(String plate) implements Vehicle { public VehicleType type() { return VehicleType.CAR; } }
record Truck(String plate) implements Vehicle { public VehicleType type() { return VehicleType.TRUCK; } }
final class VehicleFactory {
    static Vehicle create(VehicleType t, String plate) {
        return switch (t) { case BIKE -> new Bike(plate); case CAR -> new Car(plate); case TRUCK -> new
Truck(plate); };
    }
```

```
}

// Factory Method: the report job's skeleton is fixed; subclasses choose the exporter
abstract class ReportJob {
    final void run(List<String> rows) { Exporter e = createExporter(); e.write(rows); }
    protected abstract Exporter createExporter();               // the factory method
}
interface Exporter { void write(List<String> rows); }
class CsvReportJob extends ReportJob { protected Exporter createExporter() { return rows ->
System.out.println(String.join(",", rows)); } }
```

Example

```
// Abstract Factory: one provider's components must match each other
interface PaymentProviderFactory {
    PaymentGateway gateway();
    RefundClient refunds();
    WebhookVerifier webhookVerifier();
}
class RazorpayFactory implements PaymentProviderFactory {
    public PaymentGateway gateway() { return new RazorpayGateway(); }
    public RefundClient refunds() { return new RazorpayRefunds(); }
    public WebhookVerifier webhookVerifier() { return new RazorpayWebhookVerifier(); }
}
// Checkout code receives a PaymentProviderFactory and never mixes Razorpay's verifier with Stripe's gateway.
```

When would I use it?

- When the concrete type depends on input or configuration — vehicle type, file format, provider.
- When creating the object is complex.
- When a family of objects must match each other.

In Spring, a `Map<String, Bean>` registry often replaces a hand-written factory (6.4).

What can go wrong?

- Factories that grow a giant `switch`. Combine the factory with a registry (a map from key to implementation) instead.
- Abstract factories for families that never actually change.

Interview questions

Q1. Factory Method vs Abstract Factory? Factory Method lets subclasses decide one product type. Abstract Factory creates a whole family of related products through one interface.

Q2. Factory vs Builder? A factory decides *which* class to create, usually in one step. A builder constructs *one* complex object step by step.

9.10 Builder

Why?

> **In simple words:** When an object has many optional settings, a Builder lets you set them one by one, by name, and then gives you a finished object that cannot be changed.

Objects with many optional parameters usually end up in one of two bad shapes:

1. **Telescoping constructors** — long constructors like `new Notification(to, null, body, null, 5, true, null)`. You cannot tell which argument is which.

2. **Setter-based objects** that can change. These can be left half-configured.

A builder fixes both. It gives readable, named, step-by-step construction, with validation. The end result is an **immutable** object — one that cannot change after it is built.

Think of ordering a dosa at a counter. You say "masala, extra butter, no onion". The cook checks the order makes sense, then hands you the finished plate. You don't get a half-made dosa.

What is happening internally?

1. A static nested `Builder` class holds the settings while you configure them. These settings can change.
2. Each setter-like method returns `this`. So you can chain calls one after another. This is called a *fluent interface*.
3. `build()` checks rules that involve more than one field (cross-field rules).
4. Then `build()` calls the private constructor of the immutable target class.

How does it work?

```
public final class Notification {
    enum Channel { SMS, EMAIL, PUSH }
    private final String to, subject, body; private final Channel channel; private final int priority;
private final List<String> tags;

    private Notification(Builder b) {
        to = b.to; subject = b.subject; body = b.body; channel = b.channel; priority = b.priority; tags =
List.copyOf(b.tags);
    }
    public static Builder to(String to) { return new Builder(to); }          // required field up front

    public static final class Builder {
        private final String to; private String subject = "", body = ""; private Channel channel =
Channel.SMS;
        private int priority = 5; private final List<String> tags = new ArrayList<>();
        private Builder(String to) { this.to = Objects.requireNonNull(to); }
        public Builder subject(String s) { subject = s; return this; }
        public Builder body(String b) { body = b; return this; }
        public Builder channel(Channel c) { channel = c; return this; }
        public Builder priority(int p) { if (p < 1 || p > 10) throw new IllegalArgumentException("priority 1-
10"); priority = p; return this; }
        public Builder tag(String t) { tags.add(t); return this; }
        public Notification build() {
            if (channel == Channel.EMAIL && subject.isBlank()) throw new IllegalStateException("email needs a
subject");
            return new Notification(this);
        }
    }
}
```

Example

```
Notification n = Notification.to("+919876543210")
        .body("500 points credited")
        .channel(Notification.Channel.SMS)
        .priority(8)
        .tag("loyalty")
        .build();
```

(Lombok's `@Builder` generates this same pattern for you. Records with only a few fields usually don't need a builder.)

When would I use it?

- Objects with 4 or more parameters, where several are optional.
- Immutable configuration objects and request objects.
- Test data builders.

What can go wrong?

- No validation in `build()` — so invalid objects can be created again.
- Builders for 2-field objects — just noise.
- `build()` returning an object that can still change (mutable).

Interview questions

Q1. When do you use a Builder? For complex objects with many optional parameters. It gives readable construction, validation inside `build()`, and an immutable result.

9.11 Prototype

Why?

> **In simple words:** Instead of building a complex object from zero each time, keep one ready-made sample and make copies of it, changing only what is different.

Sometimes creating an object from scratch is costly or complicated. For example, it is loaded from config, or it has many nested settings.

And you often need many similar copies with small changes. Examples: campaign templates, document templates, game units.

Think of a wedding invitation card. The printer designs one card. For each guest, he only changes the name — he does not redesign the card each time.

What is happening internally?

1. You keep one fully configured object. This is the **prototype**.
2. You create new objects by **copying** it.
3. Then you customise the copy.

In Java, prefer **copy constructors** or `copy()` / `with…` methods. Avoid `Object.clone()`, because:

- it makes a shallow copy by default,
- it needs the `Cloneable` marker interface, and
- it skips the constructor, so your constructor's checks don't run.

How does it work?

```
final class CampaignTemplate {
    private final String name; private final List<String> skus; private final Map<String, Integer> slabs;
    CampaignTemplate(String name, List<String> skus, Map<String, Integer> slabs) {
        this.name = name; this.skus = List.copyOf(skus); this.slabs = Map.copyOf(slabs);
    }
    CampaignTemplate copyFor(String newName) { return new CampaignTemplate(newName, skus, slabs); }   // deep enough: immutables
}
```

Example

`CampaignTemplate diwali = festiveTemplate.copyFor("Diwali 2026");` — you set up one template, then stamp out a copy for each tenant or season.

When would I use it?

- Templates.
- Cloning objects that are already configured.
- Undo snapshots (together with the Memento pattern).

What can go wrong?

- **Shallow copies** share the nested objects that can change. So changing one copy also changes the other.
- The pitfalls of `clone()` listed above.

Interview questions

Q1. Shallow vs deep copy in Prototype? A shallow copy shares the nested mutable objects with the original. A deep copy makes its own copies of them. I prefer immutable components or explicit copy constructors, so sharing is safe.

9.12 Adapter

Why?

> **In simple words:** An Adapter is like a travel plug adapter. Your laptop charger has one shape, the wall socket has another, and the adapter sits in between so both work without changing either.

Sometimes you must connect to an outside API (a *third-party* API) or an old system (a *legacy* system). Its methods do not match what your code expects. For example:

- An SMS vendor's SDK has strange method names and its own number-based error codes.
- An old SOAP service sends and receives data in a different format.

You do not want the vendor's classes spread all over your business code. If they spread, changing the vendor later means touching many files. You want to swap vendors later with one small change.

What is happening internally?

An **adapter** is a small class that sits between your code and the vendor:

1. It implements *your* interface. This is called the *target*.
2. Inside, it holds the vendor class. This is called the *adaptee*.
3. When your code calls the adapter, it translates the call for the vendor. It converts method names, parameters, data formats and errors.

This is *composition*: the adapter wraps (holds) the adaptee. It does not extend it.

How does it work?

`RedemptionService` only knows about the `SmsSender` interface. The adapter implements that interface and forwards each call to the vendor client.

```
RedemptionService ──► «interface» SmsSender ◄── LegacyGatewayAdapter ──► LegacyGatewayClient (vendor)
```

Example

In the code below, the adapter does two translations. First, it changes the phone format (removes the `+`). Second, it turns the vendor's number error code into a proper Java exception.

```
public interface SmsSender { void send(String phone, String text); }                // your port

class LegacyGatewayClient {                                                          // vendor API you can't
change
    int push(String msisdn, byte[] payload, int flags) { return 0; }                 // 0 = OK, non-zero =
error code
}

class LegacyGatewayAdapter implements SmsSender {
    private final LegacyGatewayClient client;
    LegacyGatewayAdapter(LegacyGatewayClient client) { this.client = client; }
    @Override public void send(String phone, String text) {
        String msisdn = phone.replace("+", "");                                      // format translation
        int rc = client.push(msisdn, text.getBytes(StandardCharsets.UTF_8), 0);
        if (rc != 0) throw new SmsDeliveryException("gateway returned " + rc);       // error translation
    }
}
class SmsDeliveryException extends RuntimeException { SmsDeliveryException(String m) { super(m); } }
```

When would I use it?

- Every third-party integration: payment gateways, SMS/WhatsApp providers, partner APIs.
- Connecting to legacy (old) systems.
- Making several different providers fit behind one common interface.

What can go wrong?

- **Leaking vendor details.** If vendor types or vendor exceptions come out of the adapter, your business code depends on the vendor again.
- **Business logic creeping in.** Adapters slowly collect business rules. Keep them thin — they should only translate.

Interview questions

Q1. What does an Adapter do? It converts one interface into a different interface that the client expects. It wraps the incompatible class and translates the calls, the data and the errors.

9.13 Decorator

Why?

> **In simple words:** A Decorator wraps an object to add one extra feature, like putting a gift wrap on a box. You can wrap the same box many times, and each layer adds something.

You want to add behaviour to an object. For example:

- Add tax on top of a price.
- Add logging or caching around a client.
- Add compression around a stream.

You also want to choose the combination at runtime. Without Decorator, you would need a subclass for every combination: `TaxedDiscountedPrice`, `LoggedCachedClient`, and so on. The number of classes explodes.

What is happening internally?

A decorator has three properties:

1. It **implements the same interface** as the object it wraps.
2. It holds a reference to that wrapped object.
3. It adds its own behaviour before or after passing the call on (*delegating*) to the wrapped object.

Because the decorator has the same interface, you can wrap a decorator inside another decorator. So decorators can be **stacked**.

Java I/O is the classic example: `new BufferedReader(new InputStreamReader(new FileInputStream(f)))`. Each layer adds one feature.

How does it work?

The outer layer calls the inner layer, gets its result, and changes it. Here, the base price goes in, then the discount is applied, then GST is added.

```
GstTax ──► SchemeDiscount ──► BasePrice
price(o) = GstTax( SchemeDiscount( BasePrice(o) ) )
```

Example

The order is ₹100 (10,000 paise). The 10% discount makes it 9,000. Then 18% GST makes it 10,620.

```
interface PriceCalculator { long price(Order o); }
record Order(long amountPaise) {}

class BasePrice implements PriceCalculator { public long price(Order o) { return o.amountPaise(); } }

abstract class PriceDecorator implements PriceCalculator {
    protected final PriceCalculator inner;
    PriceDecorator(PriceCalculator inner) { this.inner = inner; }
}
class SchemeDiscount extends PriceDecorator {
    private final int pct;
    SchemeDiscount(PriceCalculator inner, int pct) { super(inner); this.pct = pct; }
    public long price(Order o) { return inner.price(o) * (100 - pct) / 100; }
}
class GstTax extends PriceDecorator {
    GstTax(PriceCalculator inner) { super(inner); }
    public long price(Order o) { return inner.price(o) * 118 / 100; }
}

PriceCalculator calc = new GstTax(new SchemeDiscount(new BasePrice(), 10));
calc.price(new Order(10_000));     // 10000 → 9000 → 10620
```

When would I use it?

- Pricing pipelines (base price → discount → tax).
- Common behaviour around clients, such as retry, cache and metrics. These are called *cross-cutting* concerns. Spring often does this with AOP proxies.
- Stream processing.
- Add-ons, such as an extra shot on a coffee (10.16).

What can go wrong?

- **Order matters.** Tax-then-discount does not give the same result as discount-then-tax.
- **Long chains are hard to debug.** You must step through many layers to find the bug.
- **Broken contract.** A decorator might behave differently from what the interface promises.

Interview questions

Q1. Decorator vs inheritance? Decorators add behaviour at runtime by wrapping objects (composition), and you can combine them freely. Inheritance fixes behaviour at compile time, and you need a new subclass for every combination, so the number of classes explodes.

9.14 Proxy

Why?

> **In simple words:** A Proxy is a stand-in that sits in front of the real object. Like a receptionist, it decides who gets through, may answer some questions itself, and only calls the real person when needed.

Sometimes you need to control access to an object without changing the object itself. You may want to:

- Create it only when it is first needed (*lazy loading*).
- Cache its results.
- Check permissions before allowing a call.
- Count or log calls.
- Represent a remote object as if it were local.

A proxy stands in for the real object and does these jobs.

What is happening internally?

A proxy implements the **same interface** as the real object (the *real subject*). For every call, it decides whether, when and how to pass the call to the real object.

Common types of proxy:

- **Virtual proxy**: creates the real object lazily, only on first use.
- **Caching proxy**: remembers results and returns them without calling the real object again.
- **Protection proxy**: checks access before forwarding the call.
- **Remote proxy**: a local stub that hides a network call.
- **Smart reference**: adds counting or logging.

You use proxies daily without noticing. Spring AOP, Hibernate lazy-loaded entities and Spring Data repositories are all proxies (6.7, 7.8).

How does it work?

The structure looks the same as Decorator. The difference is the purpose (*intent*):

- A decorator *adds features*. The client builds the chain of decorators itself.
- A proxy *controls access*. It usually hides the fact that it exists — the client thinks it is talking to the real object.

Example

`CachingCatalogProxy` saves each product after the first slow network call. `TenantGuardProxy` blocks SKUs that are not in this tenant's catalogue.

```
interface Catalog { Product get(String sku); }
record Product(String sku, String name) {}

class RemoteCatalog implements Catalog {                              // slow: network call
    public Product get(String sku) { return new Product(sku, "loaded-" + sku); }
}
class CachingCatalogProxy implements Catalog {
    private final Catalog real;
    private final Map<String, Product> cache = new ConcurrentHashMap<>();
    CachingCatalogProxy(Catalog real) { this.real = real; }
    public Product get(String sku) { return cache.computeIfAbsent(sku, real::get); }
}
class TenantGuardProxy implements Catalog {                           // protection proxy
    private final Catalog real; private final Set<String> allowedSkus;
    TenantGuardProxy(Catalog real, Set<String> allowed) { this.real = real; this.allowedSkus = allowed; }
    public Product get(String sku) {
        if (!allowedSkus.contains(sku)) throw new SecurityException("sku not in tenant catalogue");
        return real.get(sku);
    }
}
```

When would I use it?

- Caching layers.
- Lazy loading of heavy objects.
- Access control.
- Stubs for remote services.
- Wrappers for metrics and tracing.

What can go wrong?

- **Unbounded caching.** A caching proxy with no size limit keeps growing. This is a memory leak.
- **Hidden cost.** A simple-looking method call may actually be an expensive remote call behind the proxy.
- **Self-invocation.** When an object calls its own method (`this.method()`), the call does not go through the proxy. So the proxy's behaviour (for example, a transaction) is skipped.

Interview questions

Q1. Proxy vs Decorator vs Adapter? A Proxy keeps the same interface and controls access. A Decorator keeps the same interface and adds behaviour. An Adapter changes the call to a different interface.

9.15 Facade

Why?

> **In simple words:** A Facade is one simple front door to a complicated system. Like a hotel reception desk, you make one request and it coordinates housekeeping, kitchen and billing for you.

Placing an order touches five subsystems: inventory reservation, pricing, payment, loyalty accrual and notification.

Now imagine every client does this coordination itself — the mobile API, the partner API and the admin tool. Two problems follow:

1. The same coordination code is copied in three places.
2. Every client depends on the details of all five subsystems. This is called tight *coupling*.

What is happening internally?

A **facade** gives one simple, high-level method over a complex subsystem. (*Coarse-grained* means one big call instead of many small ones.) The facade makes all the inner calls in the right order.

The subsystems are still there. Code that really needs them can still call them directly.

How does it work?

All three clients call one method. The facade talks to the five subsystems.

```
Mobile API ─┐
Partner API ─┼─► CheckoutFacade.placeOrder() ─► Inventory · Pricing · Payment · Loyalty · Notification
Admin tool  ─┘
```

Example

The steps are:

1. Reserve the stock.
2. Calculate the total price.
3. Charge the payment.
4. Confirm (commit) the stock reservation.
5. Add loyalty points.
6. Send the notification.

If anything fails, the `catch` block releases the reserved stock. This undo step is called a *compensation*.

```
class CheckoutFacade {
    private final InventoryService inventory; private final PricingService pricing;
    private final PaymentService payment; private final LoyaltyService loyalty; private final Notifier
notifier;
    CheckoutFacade(InventoryService i, PricingService p, PaymentService pay, LoyaltyService l, Notifier n) {
        inventory = i; pricing = p; payment = pay; loyalty = l; notifier = n;
    }
    public OrderResult placeOrder(Cart cart, String paymentMethod) {
        Reservation r = inventory.reserve(cart.items());
        try {
            long total = pricing.total(cart);
            Receipt receipt = payment.charge(cart.customerId(), total, paymentMethod);
            inventory.commit(r);
            loyalty.accrue(cart.customerId(), total);
            notifier.orderPlaced(cart.customerId(), receipt);
            return OrderResult.ok(receipt);
        } catch (RuntimeException e) {
            inventory.release(r);                       // compensate
            throw e;
        }
    }
}
```

When would I use it?

- Application services that coordinate several domain services.
- A simple SDK on top of a complex library.

- BFF layers (*Backend for Frontend* — one backend API shaped for one specific app).

What can go wrong?

- **God class.** The facade slowly collects all the business logic and becomes one huge class.
- **Hidden distributed-transaction problems.** If the subsystems are separate microservices, one `try/catch` is not enough. You need a *saga* — a series of local steps, each with an undo step (11.15).

Interview questions

Q1. What problem does a Facade solve? It hides a complex subsystem behind one simple interface. This reduces coupling, and clients no longer copy the same coordination code.

9.16 Composite

Why?

> **In simple words:** Composite lets you treat one item and a group of items in the same way. A folder and a file both answer "what is your size?" — the folder just asks its children and adds up.

Some data is a tree, where groups and single items must be handled the same way. Examples:

- Folders and files.
- Organisation charts.
- Menus.
- Product bundles.
- Rule conditions joined with AND/OR.

Without Composite, the client code is full of checks like "if it is a group, loop over it; else handle the single item".

What is happening internally?

There is one common interface, called the *component*. Two kinds of classes implement it:

- **Leaves**: single items with no children.
- **Composites**: containers that hold child components.

When you call an operation on a composite, it calls the same operation on each child. Then it combines the results. Because a child can itself be a composite, this happens *recursively* down the tree.

How does it work?

The tree below means: sales must be at least 50,000, AND (visits at least 20 OR at least 5 new outlets).

```
AllOf ─┬─ MinAmount(50000)
       └─ AnyOf ─┬─ MinVisits(20)
                 └─ MinNewOutlets(5)
test(order) = MinAmount.test && (MinVisits.test || MinNewOutlets.test)
```

Example

`AllOf` and `AnyOf` are composites. `MinSales` and `MinVisits` are leaves. The client just calls `scheme.test(...)` and does not care which is which.

```
interface Condition { boolean test(Metrics m); }
record Metrics(double sales, int visits, int newOutlets) {}
record MinSales(double min) implements Condition { public boolean test(Metrics m) { return m.sales() >= min;
} }
record MinVisits(int min) implements Condition { public boolean test(Metrics m) { return m.visits() >= min; }
}
record AllOf(List<Condition> parts) implements Condition { public boolean test(Metrics m) { return
parts.stream().allMatch(c -> c.test(m)); } }
record AnyOf(List<Condition> parts) implements Condition { public boolean test(Metrics m) { return
parts.stream().anyMatch(c -> c.test(m)); } }

Condition scheme = new AllOf(List.of(new MinSales(50_000), new AnyOf(List.of(new MinVisits(20), new
MinVisits(30)))));
scheme.test(new Metrics(81_000, 22, 1));      // true
```

When would I use it?

- Rules engines (10.12).
- File systems.
- Permission hierarchies.
- Bundle pricing.
- UI trees (a panel holding buttons and other panels).

What can go wrong?

- **Deep recursion.** A very deep tree can cause a stack overflow.
- **Type checks leaking into clients.** If clients ask "is this a composite?", the pattern is not doing its job.
- **Cycles.** If a node ends up as its own child, recursion never ends.

Interview questions

Q1. Where have you seen Composite? In rule trees that combine conditions with AND/OR, such as incentive schemes. Also in file systems and organisation charts.

9.17 Strategy

Why?

> **In simple words:** Strategy means "put each way of doing a job in its own class, and plug in the one you need". Like choosing Ola, Uber or auto for the same trip — the trip stays the same, only the method changes.

Often one step of the logic changes depending on the situation. Examples:

- Parking fee depends on vehicle type.
- Split rules in Splitwise (equal, exact, percentage).
- Eviction policy in a cache (LRU, LFU).
- Rate-limiting algorithm.
- Pricing per tenant.

If you hard-code all of these with `if/else` or `switch`, every new variant means editing old code. This breaks OCP (the *Open/Closed Principle*: open for adding, closed for editing).

Strategy makes each algorithm a separate, pluggable object. It is the most-used pattern in LLD rounds.

What is happening internally?

There are three parts:

1. A **strategy interface** declares the operation.
2. **Concrete strategies** each implement it in their own way.
3. The **context** holds a strategy and passes the work to it. The strategy is either injected or picked from a registry (a map of strategies).

It is simply polymorphism plus composition.

How does it work?

The parking lot does not know how fees are calculated. It just asks its `FeeStrategy`.

```
ParkingLot (context) ──► «interface» FeeStrategy ◄── HourlyFee · FlatDailyFee · WeekendFee
```

Example

`HourlyFee` rounds up to whole hours, with a minimum of one hour. `ParkingTicketing` keeps a map from vehicle type to fee strategy.

```
interface FeeStrategy { long fee(Duration parked); }
class HourlyFee implements FeeStrategy {
    private final long perHour;
    HourlyFee(long perHour) { this.perHour = perHour; }
    public long fee(Duration d) { return Math.max(1, (d.toMinutes() + 59) / 60) * perHour; }   // ceil hours,
min 1
}
class FlatDailyFee implements FeeStrategy {
    public long fee(Duration d) { return Math.max(1, d.toDays() + 1) * 500; }
}

class ParkingTicketing {
    private final Map<VehicleType, FeeStrategy> feeByType;
    ParkingTicketing(Map<VehicleType, FeeStrategy> feeByType) { this.feeByType = feeByType; }
    long charge(VehicleType t, Duration d) { return feeByType.get(t).fee(d); }
}
enum VehicleType { BIKE, CAR }
// new ParkingTicketing(Map.of(VehicleType.BIKE, new HourlyFee(10), VehicleType.CAR, new HourlyFee(40)))
```

How to justify it in an interview: *"I used Strategy for fees because pricing varies independently of the parking flow, and I want to add weekend or event pricing without modifying ticketing logic."*

When would I use it?

Use it for any behaviour that has several swappable versions:

- Pricing, discounts, payouts.
- Routing, ranking.
- Retry policies.
- Serialisation formats (JSON, XML, CSV).

Simple strategies can just be lambdas, for example `Function<Duration, Long>`.

What can go wrong?

- **Too many strategies.** Making a new class for a tiny difference is overkill. A parameter would do.
- **The giant switch moves.** The client still picks the strategy with a big `switch`. Use a registry or a factory instead.

Interview questions

Q1. Strategy vs State? They have the same structure. In Strategy, the client chooses the strategy, and it stays fixed for the whole operation. In State, the state objects themselves move the context to other states as events happen.

Q2. How does Strategy support OCP? A new algorithm is just a new class, registered with the context. The context and the existing strategies do not change.

9.18 Observer

Why?

> **In simple words:** Observer is a subscription system. Interested parties sign up once, and when something happens, everyone on the list is told — like a WhatsApp group getting a message.

When an order is placed, many things must happen:

- Loyalty must add points.
- Inventory must update.
- Notifications must go out.
- Analytics must record it.

If `OrderService` calls each of these directly, it depends on all of them. Every new reaction means editing `OrderService` again.

Observer fixes this. Interested parties **subscribe** to events, and `OrderService` just announces what happened.

What is happening internally?

1. The **subject** keeps a list of **observers** (also called *listeners*).
2. When an event happens, the subject notifies each observer in the list.

The direction of dependency is important. Observers depend on the subject's event type. The subject does not depend on the observers.

- **Inside one service (in-process):** Spring gives you `ApplicationEventPublisher` and `@EventListener` (6.10).
- **Across services:** the same idea becomes **pub/sub messaging**, for example Kafka topics (11.12).

How does it work?

```
OrderService ──publish(OrderPlaced)──► [LoyaltyListener, InventoryListener, NotificationListener,
AnalyticsListener]
```

Example

Note two details in the code. First, `CopyOnWriteArrayList` lets you loop safely even if someone subscribes or unsubscribes at the same time. Second, each listener call is inside its own `try/catch`, so one bad listener cannot break the others or the order.

```
record OrderPlaced(String orderId, String outletId, long amountPaise) {}
interface OrderListener { void onOrderPlaced(OrderPlaced e); }
```

```
class OrderService {
    private final List<OrderListener> listeners = new CopyOnWriteArrayList<>();   // safe iteration while
(un)subscribing
    void subscribe(OrderListener l) { listeners.add(l); }
    void unsubscribe(OrderListener l) { listeners.remove(l); }
    void place(String orderId, String outletId, long amount) {
        // ... persist order ...
        OrderPlaced e = new OrderPlaced(orderId, outletId, amount);
        for (OrderListener l : listeners) {
            try { l.onOrderPlaced(e); }
            catch (RuntimeException ex) { /* log; one bad listener must not break others or the order */ }
        }
    }
}
```

When would I use it?

- Separating side effects from the main action.
- UI updates.
- Domain events inside one service.
- Low-stock alerts (10.16).

Use durable messaging (such as Kafka) when the observers live in other services, or when they must never miss an event.

What can go wrong?

- **Listener exceptions** break the subject if not caught.
- **Slow listeners** that run synchronously block the caller.
- **Memory leaks** from listeners that are never unsubscribed.
- **Unclear ordering** — you cannot be sure which listener runs first.
- **Lost events on crash** — in-memory events disappear if the process dies, when they should have been stored durably.

Interview questions

Q1. Where is Observer used in Java/Spring? Spring application events and listeners, GUI listeners, and reactive streams. Across services, it becomes pub/sub messaging.

9.19 State

Why?

> **In simple words:** The State pattern gives each status its own class, and that class decides what is allowed next. Like a railway ticket: a "confirmed" ticket can be cancelled with refund, but a "journey done" ticket cannot.

Some objects behave differently depending on their current state. Examples: orders, vending machines, elevators, tickets and trips.

For example, you can cancel a paid order (with a refund), but you cannot cancel a shipped one.

If you write `if (status == …)` inside every method, two problems follow:

1. The rules for moving between states (*transitions*) are scattered everywhere.
2. It is easy to make an illegal transition by mistake.

What is happening internally?

1. Each state is its own class. All states implement one common interface, with one method per event (pay, ship, cancel).
2. The object (the *context*) holds the current state object.
3. When an event arrives, the context passes it to the current state.
4. The state handles only the events it allows. It returns the next state — or rejects the event.

For simple machines, you do not need classes. An `enum` with a transition table is enough.

How does it work?

```
CREATED ──pay──► PAID ──ship──► SHIPPED ──deliver──► DELIVERED
   │               │
   └──cancel──► CANCELLED ◄──cancel (refund)
```

Example

The interface has default methods that throw an error. Each state class overrides only the events it allows. So any event not allowed fails automatically.

```
interface OrderState {
    default OrderState pay(OrderCtx c)    { throw new IllegalStateException("cannot pay in " + name()); }
    default OrderState ship(OrderCtx c)   { throw new IllegalStateException("cannot ship in " + name()); }
    default OrderState cancel(OrderCtx c) { throw new IllegalStateException("cannot cancel in " + name()); }
    String name();
}
interface OrderCtx { void charge(); void refund(); }

final class Created implements OrderState {
    public OrderState pay(OrderCtx c) { c.charge(); return new Paid(); }
    public OrderState cancel(OrderCtx c) { return new Cancelled(); }
    public String name() { return "CREATED"; }
}
final class Paid implements OrderState {
    public OrderState ship(OrderCtx c) { return new Shipped(); }
    public OrderState cancel(OrderCtx c) { c.refund(); return new Cancelled(); }
    public String name() { return "PAID"; }
}
final class Shipped implements OrderState { public String name() { return "SHIPPED"; } }
final class Cancelled implements OrderState { public String name() { return "CANCELLED"; } }

// Enum alternative for simple machines
enum Status {
    CREATED, PAID, SHIPPED, CANCELLED;
    private static final Map<Status, Set<Status>> ALLOWED = Map.of(
            CREATED, Set.of(PAID, CANCELLED), PAID, Set.of(SHIPPED, CANCELLED), SHIPPED, Set.of(), CANCELLED,
Set.of());
    Status to(Status next) {
        if (!ALLOWED.get(this).contains(next)) throw new IllegalStateException(this + " → " + next);
        return next;
    }
}
```

In a database, enforce transitions with a conditional update: `UPDATE orders SET status='PAID' WHERE id=? AND status='CREATED'`. The `WHERE status='CREATED'` part makes it safe:

- It is *atomic* — the check and the change happen as one step.
- It is *idempotent* — running it twice has the same effect as running it once. The second run simply updates zero rows.

When would I use it?

Use it for lifecycles where each state has its own rules:

- Orders and payments.
- Trips (10.17).
- Vending machines (10.7).
- Elevators (10.8).
- Approval workflows.
- Ticket holds (10.10).

What can go wrong?

- **Huge state classes.** One state class grows too big.
- **Scattered transitions.** Transition rules spread across many states and services.
- **Concurrent transitions without atomicity.** Two threads both move PAID → CANCELLED at the same time, and the customer is refunded twice.

Interview questions

Q1. When do you use the State pattern instead of an enum? When each state has its own distinct behaviour and transitions involve real logic. For a simple status machine, an enum with a transition table is enough.

9.20 Command

Why?

> **In simple words:** Command turns an action into an object — like a written order slip in a restaurant. Because it is written down, you can queue it, keep a record of it, repeat it, or cancel it.

Sometimes you want to treat an action as **data**. Then you can:

- Put it in a queue.
- Log it.
- Retry it.
- Schedule it for later.
- Undo it.

Many systems need this:

- Text editors (undo/redo).
- Job queues.
- Macro recording.
- Ledgers, where accrue, redeem and reverse are stored as recorded operations.

What is happening internally?

A command object packs one request into an object (*encapsulates* it). It holds:

- What to do.
- On which object (the *receiver*).

- With which parameters.
- Optionally, how to **undo** it.

There are two other roles:

- The **invoker** runs commands and keeps the history.
- The **receiver** does the real work.

How does it work?

The editor keeps a stack of commands it has run. Undo takes the last one off the stack and calls its `undo()`.

```
Editor (invoker) — history stack: [AddText("Hello"), AddText(" World")]
undo() → pop → AddText(" World").undo()
```

Example

The editor uses two stacks:

1. Running a command pushes it on the undo stack and clears the redo stack.
2. Undo pops from the undo stack, reverses the command, and pushes it on the redo stack.
3. Redo pops from the redo stack, runs it again, and pushes it back on the undo stack.

```
interface Command { void execute(); void undo(); }

class AddText implements Command {
    private final StringBuilder doc; private final String text;
    AddText(StringBuilder doc, String text) { this.doc = doc; this.text = text; }
    public void execute() { doc.append(text); }
    public void undo() { doc.setLength(doc.length() - text.length()); }
}
class Editor {
    private final StringBuilder doc = new StringBuilder();
    private final Deque<Command> undo = new ArrayDeque<>(), redo = new ArrayDeque<>();
    void run(Command c) { c.execute(); undo.push(c); redo.clear(); }
    void undo() { if (!undo.isEmpty()) { Command c = undo.pop(); c.undo(); redo.push(c); } }
    void redo() { if (!redo.isEmpty()) { Command c = redo.pop(); c.execute(); undo.push(c); } }
    String text() { return doc.toString(); }
    AddText add(String t) { return new AddText(doc, t); }
}
// Editor e = new Editor(); e.run(e.add("Hello")); e.run(e.add(" World")); e.undo(); → "Hello"
```

When would I use it?

- Undo/redo.
- Job and task queues.
- Scheduled operations.
- Operations that must be recorded in an audit log.
- Compensating actions in sagas — each step has an "execute" and a "compensate" (undo).
- Command handlers in CQRS (a design that separates writes/commands from reads/queries).

What can go wrong?

- **Undo is not a true reverse.** If other changes happened since, simply undoing may give the wrong result.
- **Unbounded history.** The history stack grows forever and uses up memory.
- **Heavy commands.** Commands that hold large state use a lot of memory.

Interview questions

Q1. How would you implement undo/redo? Make each action a command with `execute` and `undo`. Keep an undo stack and a redo stack. When a new command is executed, clear the redo stack.

9.21 Chain of Responsibility

Why?

> **In simple words:** Chain of Responsibility passes a request through a line of checkers, one after another. Like airport security: ID check, then baggage scan, then body scan — anyone can stop you, and you can add or reorder checks easily.

A request must pass through a series of checks or handlers. For example:

- Authentication.
- Validation.
- Fraud check.
- Geofence check (is the user inside the allowed area?).
- Photo-evidence check.

The set of steps and their order change over time. If you hard-code all of them in one method, the method becomes long and hard to change.

What is happening internally?

1. All handlers implement one common interface.
2. They are linked in a chain.
3. Each handler does one of two things. Either it handles the request and stops the chain, or it does its part and passes the request to the next handler.

Servlet filters and Spring Security's filter chain both use this pattern.

How does it work?

```
claim → GeoFenceCheck → PhotoCheck → DuplicateClaimCheck → accepted
            └ reject          └ reject        └ reject
```

Example

Each check throws an exception to reject the claim. If every check passes, the claim is accepted. The list order is the chain order, and you can configure it per tenant.

```
record Claim(String outletId, boolean insideGeofence, List<String> photos) {}

interface ClaimCheck { void check(Claim c); }
class GeoFenceCheck implements ClaimCheck {
    public void check(Claim c) { if (!c.insideGeofence()) throw new IllegalArgumentException("outside
geofence"); }
}
class PhotoCheck implements ClaimCheck {
    public void check(Claim c) { if (c.photos().isEmpty()) throw new IllegalArgumentException("photo evidence
required"); }
}
class ClaimValidator {
    private final List<ClaimCheck> chain;                    // order matters; configurable per tenant
    ClaimValidator(List<ClaimCheck> chain) { this.chain = List.copyOf(chain); }
```

```
    void validate(Claim c) { for (ClaimCheck check : chain) check.check(c); }
}
```

(The classic version links each handler to the next one directly. A simple list of handlers, as above, is the modern and simpler form.)

When would I use it?

- Validation pipelines, such as Perfect Store evidence checks.
- Request filters.
- Approval workflows (escalate to the next approver).
- Logging levels.
- Middleware.

What can go wrong?

- **Unhandled requests.** A request reaches the end of the chain and nobody handled it — silently.
- **Hidden order rules.** One handler depends on another running first, but nobody wrote that down.
- **Hard debugging.** Long chains are hard to trace. Log which handler made the decision.

Interview questions

Q1. Where is Chain of Responsibility used in Spring? Servlet filters and the Spring Security filter chain. HandlerInterceptors work in a similar way.

9.22 Template Method

Why?

> **In simple words:** Template Method fixes the order of steps in a parent class and lets child classes fill in only the steps that differ. Like a standard form where the layout is printed and you only fill in the blanks.

Several processes often share the same skeleton (the same overall steps) but differ in a few steps. For example:

- Every report job fetches data, transforms it, and publishes it.
- Every import validates, parses, saves and audits.

If you copy the skeleton into each job, the fixed parts are duplicated. Over time the copies drift apart and behave differently.

What is happening internally?

1. An abstract base class defines the algorithm in one `final` method. This is the *template method*. Because it is `final`, subclasses cannot change the order.
2. The template method calls **abstract** steps. Subclasses must implement these.
3. It may also call **hooks**. These are optional steps with a default behaviour that subclasses can override.

This version is based on inheritance. There is also a callback-based version, where you pass in lambdas instead of subclassing. Spring's `JdbcTemplate` and `TransactionTemplate` use this callback version.

How does it work?

`run()` is the fixed skeleton. `fetch` and `publish` are required steps. `transform` has a default. `shouldPublish` is a hook.

```
abstract class ReportJob {
    public final void run() {                                  // fixed skeleton
        var data = fetch();
        var rows = transform(data);
        if (shouldPublish(rows)) publish(rows);
    }
    protected abstract List<Map<String, Object>> fetch();     // required steps
    protected abstract void publish(List<String> rows);
    protected List<String> transform(List<Map<String, Object>> d) { return
d.stream().map(Object::toString).toList(); }  // default
    protected boolean shouldPublish(List<String> rows) { return !rows.isEmpty(); }
// hook
}
```

Example

`class DailyRedemptionReport extends ReportJob { … fetch from ClickHouse, publish to S3 … }` — you write only the steps that change.

When would I use it?

- Batch jobs.
- Importers.
- Framework extension points, such as `OncePerRequestFilter.doFilterInternal` and `AbstractList`.

What can go wrong?

- **Deep class hierarchies.** Many levels of inheritance become hard to follow.
- **Rigid skeleton.** A subclass needs to change the skeleton itself. In that case, prefer composition — use Strategy for the steps instead.

Interview questions

Q1. Template Method vs Strategy? Template Method keeps a fixed skeleton and lets subclasses change individual steps (inheritance). Strategy swaps the whole algorithm by plugging in a different object (composition).

9.23 Specification, and patterns to recognise

Why?

> **In simple words:** Specification turns each business rule into a small object that answers "yes or no". You can then join rules with AND, OR and NOT, like building blocks.

Consider this business rule: "eligible outlet = active AND in region North AND (sales > X OR new outlet)".

Rules like this need to be:

- Combined with other rules.
- Reused in many places.
- Tested on their own.

- Sometimes turned into database queries.

Specification makes each rule an object that you can combine (*compose*).

What is happening internally?

A `Specification<T>` has:

- A method `isSatisfiedBy(T)` that returns true or false.
- Combinators `and`, `or` and `not` that join it with other specifications.

Spring Data JPA's `Specification<T>` works a little differently. Instead of checking objects in memory, it builds a JPA Criteria *predicate* (a condition). So the combined rules become a SQL `WHERE` clause and run in the database.

How does it work?

Each small rule is one line. Then `eligible` joins them together.

```
interface Spec<T> {
    boolean test(T t);
    default Spec<T> and(Spec<T> o) { return t -> test(t) && o.test(t); }
    default Spec<T> or(Spec<T> o)  { return t -> test(t) || o.test(t); }
    default Spec<T> not()          { return t -> !test(t); }
}
record Outlet(boolean active, String region, double sales, boolean isNew) {}
Spec<Outlet> active = Outlet::active;
Spec<Outlet> north = o -> o.region().equals("NORTH");
Spec<Outlet> bigOrNew = ((Spec<Outlet>) o -> o.sales() > 50_000).or(Outlet::isNew);
Spec<Outlet> eligible = active.and(north).and(bigOrNew);
```

Example

The same idea in Spring Data JPA. Here the filter runs inside the database, not in Java memory.

```
// Spring Data JPA: same idea, executed in the database
static Specification<OutletEntity> inRegion(String r) { return (root, q, cb) -> cb.equal(root.get("region"),
r); }
static Specification<OutletEntity> isActive() { return (root, q, cb) -> cb.isTrue(root.get("active")); }
// outletRepo.findAll(inRegion("NORTH").and(isActive()))   // repo extends JpaSpecificationExecutor
```

Other patterns to recognise (one sentence each in interviews): - **Iterator**: walk through a collection without showing how it is stored inside (`Iterator<T>`). - **Mediator**: objects talk through one central coordinator instead of talking to each other directly (an elevator dispatcher, a chat room). - **Visitor**: add new operations over a fixed set of types without changing those types. In modern Java, sealed types plus a pattern-matching switch do this more simply (3.8). - **Memento**: save an object's state and restore it later (undo snapshots). - **Flyweight**: share common, unchanging (immutable) data among many objects to save memory (the `Integer` cache). - **Bridge**: separate an abstraction from its implementation, so both can change independently (notification type × delivery channel).

When would I use it?

- Eligibility rules.
- Dynamic search filters.
- Policy checks.
- Building queries.

What can go wrong?

- **Filtering in memory instead of SQL.** Running in-memory specifications on large datasets is slow. Large data should be filtered in the database.
- **Too clever.** Combinator code can become so clever that nobody can read it.

Interview questions

Q1. What is the Specification pattern? It turns business rules into predicate objects (yes/no checks) that you can combine with `and`, `or` and `not`. You can reuse them in memory or turn them into database queries, as Spring Data JPA Specifications do.

9.24 Supporting principles interviewers listen for

Why?

> **In simple words:** Patterns are tools; principles tell you when to use a tool and when to leave it in the box. Knowing when to stop adding layers is as much a senior skill as knowing how to add them.

If you name these principles at the right moment, the interviewer sees good judgement. Knowing when to stop abstracting is as senior a skill as knowing how to abstract.

What is happening internally?

- **DRY** (Don't Repeat Yourself): every piece of *knowledge* should live in one official place. If a rule is copied, the copies drift apart over time.
- **KISS** (Keep It Simple): choose the simplest design that meets the requirements.
- **YAGNI** (You Aren't Gonna Need It): do not build for future needs that may never come.
- **Law of Demeter**: talk only to your direct collaborators. `order.customer().address().city()` ties you to three classes. Instead, ask `order.shippingCity()`.
- **Tell, don't ask**: tell an object to do something. Do not pull out its data and make the decision outside it.
- **High cohesion, low coupling**: keep related code together inside a class; keep classes loosely connected to each other.
- **Composition over inheritance** (1.11): build behaviour by holding other objects, rather than by extending classes.
- **Program to interfaces, not implementations**: depend on the interface type, not on a concrete class.
- **Fail fast**: check inputs early, and throw clear exceptions.
- **Immutability by default** for values and events (1.14): once created, they do not change.

How does it work?

Use these principles to explain your decisions out loud. For example: "I'm not adding a plugin system for payment types yet — YAGNI; but the gateway is behind an interface, so adding one later is cheap."

Example

In the "Ask" version, outside code reads the balance and decides. In the "Tell" version, the account checks its own rule.

```
// Ask (violates tell-don't-ask and Demeter)
if (account.getBalance() >= points) account.setBalance(account.getBalance() - points);
```

```
// Tell
account.debit(points);          // the account enforces its own rule
```

When would I use it?

In every design discussion and every code review.

What can go wrong?

- **Wrong DRY.** Two pieces of code only *look* similar but represent different knowledge. Merging them couples unrelated things.
- **YAGNI as an excuse.** Someone uses YAGNI to skip extension points that are clearly needed.

Interview questions

Q1. How do you avoid over-engineering in an LLD round? Build the stated requirements simply (KISS/YAGNI). But put abstractions at the obvious places of variation that the interviewer mentioned. Then explain how you would extend the design.

9.25 How to run an LLD interview (60–90 minutes)

Why?

> **In simple words:** Follow a fixed timetable in the LLD round — requirements, classes, patterns, code, demo, extensions — so you never panic and always end with working code.

Most candidates fail LLD rounds because of poor process, not because they lack knowledge. They start coding too early, or design for too long, or run out of time with nothing working.

A fixed routine keeps you calm. It also keeps your thinking visible to the interviewer, and keeps you on time.

What is happening internally?

The interviewer scores you on:

- Clarifying requirements.
- Modelling the entities.
- Using OOP and SOLID.
- Choosing suitable patterns.
- **Working code.**
- Handling edge cases and concurrency.
- Extensibility (how easily new features fit in).
- Communication.

Machine-coding rounds (Flipkart, Swiggy, PhonePe, Uber style) give heavy weight to **runnable, modular code**.

How does it work?

Minutes	Step	What you do and say
0–7	Requirements	Say 4–6 core use cases out loud and write them down. Confirm what is out of scope. Ask about concurrency, persistence (in-memory is usually fine) and scale

Minutes	Step	What you do and say
7–15	Entities and relationships	Nouns become classes, enums and records; verbs become methods. Draw a quick class diagram (9.2)
15–20	Interfaces and patterns	Define the public API of the main service. Point out where things vary and which pattern you will use there, with a one-line reason
20–70	Code	Models → interfaces → in-memory repositories → core service → one or two strategies → a `main` demo. Get the happy path working end to end first
70–80	Walkthrough	Run a scenario. Handle invalid input and edge cases
80–90	Extensions	Concurrency (what is shared, how you protect it), new features (where they plug in), persistence, tests

Package skeleton you can reuse:

```
model/       entities, enums, value objects (records)
strategy/    interfaces + implementations of what varies
repository/  in-memory repositories (ConcurrentHashMap)
service/     orchestration
exception/   domain exceptions
Main.java    demo driver
```

Example

Chapter 10 solves 17 problems in exactly this format, with complete code that compiles.

When would I use it?

In every LLD and machine-coding round. Also when you practise with a 90-minute timer.

What can go wrong?

- Coding before you have agreed on the requirements.
- A perfect design but nothing that runs at minute 90.
- Ignoring concurrency when the problem clearly has shared state (parking spots, seats, inventory).
- Silence. Keep talking through your decisions the whole time.

Interview questions

Q1. How do you approach an LLD problem? First, clarify the requirements and scope. Then identify the entities and their relationships. Define the interfaces, and choose patterns where things vary. Code the core flow end to end. Finally, handle edge cases, concurrency and extensions, and explain the trade-offs as you go.

10. LLD Problems, Solved in Java

This chapter solves seventeen LLD (low-level design) problems, in the order interviewers usually ask them. LLD means designing the classes, data structures and methods for one small system.

Every problem follows the same seven steps:

- **Why?** — what the interviewer asks, and what skill it tests.
- **What is happening internally?** — the classes (entities), data structures and design patterns.
- **How does it work?** — the main flows, step by step, and how we handle many threads at once.
- **Example** — complete code. It compiles and runs with `java File.java`. A short trace of the output follows it.
- **When would I use it?** — where the same design shows up in real systems.
- **What can go wrong?** — edge cases and common mistakes.
- **Interview questions** — the follow-up questions you will most likely get.

> **Tip:** Set a 90-minute timer and type each solution yourself first. Read the given solution only after that.

10.1 Tic-Tac-Toe

Why?

> **In simple words:** Build a noughts-and-crosses game. The clever part is checking for a winner with a few counters, instead of looking at the whole board after every move.

This is a 45-minute warm-up. Many companies ask it in the first LLD round.

It tests four things:

- Basic modelling — splitting the problem into a board, players and a game.
- Turn management — whose move is it now?
- Input validation — rejecting bad moves.
- One algorithm idea — checking a win in **O(1)** per move. O(1) means the work does not grow with the board size. The slow way is to scan the whole board every time.

What is happening internally?

- **Entities** (the main classes):
- `Player` — a name and a symbol.
- `Board` — an N×N grid of cells, plus some counters.
- `Game` — the turn order and the current status.
- **Enums** (fixed lists of values): `Symbol {X, O}` and `Status {IN_PROGRESS, X_WON, O_WON, DRAW}`.
- **Key data structure**: one counter for every row, one for every column, and two for the diagonals.
- When X plays, add +1 to the counters of that cell's lines. When O plays, add −1.
- If X fills a whole line, its counter becomes +N. If O fills it, the counter becomes −N.
- If both players have marks on a line, the +1s and −1s cancel out, so it can never reach N.
- So: if the absolute value of a counter reaches N, one player owns that full line.

- **Turn order**: a `Deque<Player>` (a double-ended queue). After each move, take the front player and put them at the back. This works for more than two players without any change.

Think of it like a scoreboard per line. X adds a run, O takes one away. Only a line where one side scored every time reaches the full score.

How does it work?

1. `move(r, c)` is called. First, check the move is valid: - the game is not already over, - the cell is on the board, - the cell is empty.
2. Place the current player's symbol in the cell.
3. Update `rows[r]` and `cols[c]`.
4. If `r == c`, the cell is on the main diagonal, so update that counter too.
5. If `r + c == n - 1`, the cell is on the anti-diagonal (top-right to bottom-left), so update that counter.
6. Check only the counters you just touched: - If any has absolute value N → this player wins. - Else, if all cells are filled → it is a draw. - Else → rotate the turn to the next player.

Example

```
// FILE: TicTacToe.java
import java.util.*;

public class TicTacToe {
    enum Symbol { X, O }
    enum Status { IN_PROGRESS, X_WON, O_WON, DRAW }
    record Player(String name, Symbol symbol) {}

    static class Board {
        private final int n;
        private final Symbol[][] cells;
        private final int[] rows, cols;
        private int diag, anti, filled;
        Board(int n) { this.n = n; cells = new Symbol[n][n]; rows = new int[n]; cols = new int[n]; }

        /** @return true if this move wins */
        boolean place(int r, int c, Symbol s) {
            if (r < 0 || c < 0 || r >= n || c >= n) throw new IllegalArgumentException("off board");
            if (cells[r][c] != null) throw new IllegalArgumentException("cell taken");
            cells[r][c] = s; filled++;
            int d = s == Symbol.X ? 1 : -1;
            rows[r] += d; cols[c] += d;
            if (r == c) diag += d;
            if (r + c == n - 1) anti += d;
            return Math.abs(rows[r]) == n || Math.abs(cols[c]) == n || Math.abs(diag) == n || Math.abs(anti)
== n;
        }
        boolean full() { return filled == n * n; }
    }

    static class Game {
        private final Board board;
        private final Deque<Player> turn = new ArrayDeque<>();
        private Status status = Status.IN_PROGRESS;
        Game(int n, Player p1, Player p2) { board = new Board(n); turn.add(p1); turn.add(p2); }

        Status move(int r, int c) {
            if (status != Status.IN_PROGRESS) throw new IllegalStateException("game over: " + status);
            Player p = turn.peekFirst();
            if (board.place(r, c, p.symbol())) status = p.symbol() == Symbol.X ? Status.X_WON : Status.O_WON;
            else if (board.full()) status = Status.DRAW;
            else turn.addLast(turn.pollFirst());
            return status;
        }
    }
```

```
    public static void main(String[] args) {
        Game g = new Game(3, new Player("Asha", Symbol.X), new Player("Ravi", Symbol.O));
        int[][] moves = {{0,0},{1,1},{0,1},{2,2},{0,2}};
        Status s = null;
        for (int[] m : moves) s = g.move(m[0], m[1]);
        System.out.println(s);   // X_WON
    }
}
```

Trace, move by move (N = 3):

1. X plays (0,0) → rows[0]=1, cols[0]=1, diag=1.
2. O plays (1,1) → rows[1]=−1, cols[1]=−1, diag=0, anti=−1. (The centre is on both diagonals.)
3. X plays (0,1) → rows[0]=2.
4. O plays (2,2) → diag=−1.
5. X plays (0,2) → rows[0]=3. That equals N, so the result is **X_WON**.

When would I use it?

The counter trick works for any "N in a line" check.

The split into Game, Board and Player, plus the turn queue, comes back again and again. You will see it in Snake & Ladder (10.13), chess and card games.

What can go wrong?

- Scanning the whole board after every move. That is O(N²) work — it grows with the square of the board size — when O(1) is possible.
- Allowing a move after the game has ended.
- Allowing a move on a cell that is already taken.
- Hard-coding two players or a 3×3 board. Keep N and the player list flexible.

Interview questions

Q1. How do you check a win in O(1)? Keep one counter per row, per column and for both diagonals. Add +1 for X and −1 for O on the lines that pass through the played cell. If a counter reaches +N or −N, that player has won.

Q2. How would you support K-in-a-row on an N×N board (K < N)? The full-line counters no longer work. Instead, after each move, start from the placed cell and count matching symbols in a row along the 4 directions (row, column, two diagonals). That costs O(K) per move — the work depends only on K.

Q3. How would you add undo or a bot? For undo, use the Command pattern: push each move onto a stack, and to undo, pop it and reverse its counter changes. For a bot, add a `MoveStrategy` interface. Plug in different versions behind it — random, greedy, or minimax (a search that tries all future moves).

10.2 Parking Lot

Why?

> **In simple words:** Cars, bikes and trucks come in, get a suitable free spot and a ticket, and pay by the hour when they leave. The key point is that two entry gates must never hand out the same spot.

This is the most common LLD question of all.

It tests:

- Modelling with several kinds of entities (lot, spot, vehicle, ticket).
- The Strategy pattern — putting a changeable rule (like pricing or spot choice) behind an interface.
- Clean separation of responsibilities — each class does one job.
- **Concurrency** — many threads running at the same time. Here, two entry gates working at the same moment must never assign the same spot.

What is happening internally?

- **Entities**:
- `ParkingLot` — the whole system.
- `Spot` — id, floor and type.
- `Vehicle` — number plate and type.
- `Ticket` — id, vehicle, spot and entry time.
- `Receipt` — what you get on exit, with the fee.
- **Enums**: `VehicleType {BIKE, CAR, TRUCK}` and `SpotType {SMALL, MEDIUM, LARGE}`.
- A mapping says which spot types each vehicle may use. A bike can use any spot. A truck can use only large ones.
- **Data structures**:
- Free spots are kept per spot type in a `ConcurrentLinkedDeque`. It is lock-free — threads do not wait on a lock — and its `pollFirst` is atomic. Atomic means it happens as one single step that no other thread can break into.
- Active tickets are kept in a `ConcurrentHashMap`. Its `remove` on exit is also atomic.
- **Patterns**:
- Strategy: `FeeStrategy` decides the price. Spot allocation could be a second strategy.
- An injected `Clock`: the lot does not read the system time directly; we pass a clock in. So tests can control time.

How does it work?

1. **Park**:
2. Get the list of spot types this vehicle may use, best fit first.
3. For each type, call `pollFirst()` to take a free spot.
4. The first spot you get creates a ticket.
5. If every type is empty → "lot full".
6. **Exit**:
7. Atomically `remove` the ticket from the active map. If two exits race, only one gets the ticket, so a ticket can never be closed twice.
8. Work out how long the vehicle stayed, and ask the fee strategy for the price.
9. Put the spot back into its free deque.
10. **Availability**: just the size of each free deque.

Concurrency: there is no global lock. `pollFirst` on a concurrent deque is atomic. So if two gates call it at the very same moment, they always get different spots.

Think of a token counter at a busy restaurant. Each token can be torn off the roll by only one waiter. Two waiters grabbing at the same time still get two different tokens.

Example

```java
// FILE: ParkingLotDemo.java
import java.time.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicLong;

public class ParkingLotDemo {
    enum VehicleType { BIKE, CAR, TRUCK }
    enum SpotType { SMALL, MEDIUM, LARGE }

    record Vehicle(String plate, VehicleType type) {}
    record Ticket(String id, Vehicle vehicle, Spot spot, Instant entry) {}
    record Receipt(Ticket ticket, Instant exit, long feeRupees) {}

    static final class Spot {
        final String id; final int floor; final SpotType type;
        Spot(String id, int floor, SpotType type) { this.id = id; this.floor = floor; this.type = type; }
        public String toString() { return id; }
    }

    interface FeeStrategy { long fee(VehicleType t, Duration d); }
    static class HourlyFee implements FeeStrategy {
        private final Map<VehicleType, Long> perHour;
        HourlyFee(Map<VehicleType, Long> perHour) { this.perHour = perHour; }
        public long fee(VehicleType t, Duration d) {
            long hours = Math.max(1, (d.toMinutes() + 59) / 60);    // ceil, minimum 1 hour
            return hours * perHour.get(t);
        }
    }

    /** Which spot types a vehicle may use, in order of preference. */
    static List<SpotType> compatible(VehicleType v) {
        return switch (v) {
            case BIKE -> List.of(SpotType.SMALL, SpotType.MEDIUM, SpotType.LARGE);
            case CAR -> List.of(SpotType.MEDIUM, SpotType.LARGE);
            case TRUCK -> List.of(SpotType.LARGE);
        };
    }

    static class ParkingLot {
        // free spots per type: a concurrent queue gives lock-free, thread-safe allocation across gates
        private final Map<SpotType, ConcurrentLinkedDeque<Spot>> free = new EnumMap<>(SpotType.class);
        private final ConcurrentHashMap<String, Ticket> active = new ConcurrentHashMap<>();
        private final FeeStrategy fees;
        private final Clock clock;
        private final AtomicLong seq = new AtomicLong();

        ParkingLot(List<Spot> spots, FeeStrategy fees, Clock clock) {
            for (SpotType t : SpotType.values()) free.put(t, new ConcurrentLinkedDeque<>());
            spots.forEach(s -> free.get(s.type).add(s));
            this.fees = fees; this.clock = clock;
        }

        Ticket park(Vehicle v) {
            for (SpotType t : compatible(v.type())) {
                Spot s = free.get(t).pollFirst();        // atomic: two gates never get the same spot
                if (s != null) {
                    Ticket ticket = new Ticket("T" + seq.incrementAndGet(), v, s, clock.instant());
                    active.put(ticket.id(), ticket);
                    return ticket;
                }
            }
            throw new IllegalStateException("No spot for " + v.type());
        }

        Receipt exit(String ticketId) {
            Ticket t = active.remove(ticketId);          // atomic: a ticket can be closed only once
            if (t == null) throw new IllegalArgumentException("Unknown or closed ticket " + ticketId);
            Instant now = clock.instant();
```

```
            long fee = fees.fee(t.vehicle().type(), Duration.between(t.entry(), now));
            free.get(t.spot().type).addFirst(t.spot());
            return new Receipt(t, now, fee);
        }

        Map<SpotType, Integer> availability() {
            Map<SpotType, Integer> m = new EnumMap<>(SpotType.class);
            free.forEach((k, v) -> m.put(k, v.size()));
            return m;
        }
    }

    /** Test clock we can move forward. */
    static class MutableClock extends Clock {
        private Instant now = Instant.parse("2026-10-05T09:00:00Z");
        void advance(Duration d) { now = now.plus(d); }
        public ZoneId getZone() { return ZoneOffset.UTC; }
        public Clock withZone(ZoneId z) { return this; }
        public Instant instant() { return now; }
    }

    public static void main(String[] args) {
        List<Spot> spots = new ArrayList<>();
        for (int f = 0; f < 2; f++) {
            for (int i = 0; i < 2; i++) spots.add(new Spot("F" + f + "-S" + i, f, SpotType.SMALL));
            for (int i = 0; i < 3; i++) spots.add(new Spot("F" + f + "-M" + i, f, SpotType.MEDIUM));
            spots.add(new Spot("F" + f + "-L0", f, SpotType.LARGE));
        }
        MutableClock clock = new MutableClock();
        ParkingLot lot = new ParkingLot(spots,
                new HourlyFee(Map.of(VehicleType.BIKE, 10L, VehicleType.CAR, 40L, VehicleType.TRUCK, 100L)),
clock);

        Ticket t1 = lot.park(new Vehicle("UP65AB1234", VehicleType.CAR));
        Ticket t2 = lot.park(new Vehicle("UP65ZZ0001", VehicleType.TRUCK));
        System.out.println(t1.spot() + " " + t2.spot() + " " + lot.availability());
        clock.advance(Duration.ofMinutes(130));
        System.out.println("Fee: " + lot.exit(t1.id()).feeRupees());   // 3 hours * 40 = 120
        System.out.println(lot.availability());
    }
}
```

Trace:

1. The car gets the first MEDIUM spot (F0-M0).
2. The truck gets the first LARGE spot (F0-L0).
3. The clock moves 130 minutes ahead.
4. The car exits. Hours are rounded up: ceil(130/60) = 3 hours. 3 × ₹40 = ₹120.
5. F0-M0 goes back to the free pool.

When would I use it?

The same shape appears whenever you have a stock of similar slots, hand them out, and bill by time. Examples:

- EV charging stations.
- Co-working desk booking.
- Warehouse bins.
- Cloud resource allocation (handing out servers or IP addresses).

What can go wrong?

- One global `synchronized` on the whole lot. It is correct, but it serialises all gates — only one gate can work at a time.

- Check-then-act races. You "find a free spot" in one step and "mark it occupied" in a second step. Between the two steps, another gate can take the same spot.
- Billing with `double`. Floating-point maths gives rounding errors with money; use `long` or `BigDecimal`.
- Forgetting the minimum charge or the rounding rule.
- Hard-coded pricing instead of a strategy, so every price change means editing the core code.

Interview questions

Q1. How do you prevent two gates assigning the same spot? Take the spot with an atomic poll from a concurrent free-spot structure, one per spot type. A lock per spot type also works. If the spots live in a database, use `SELECT … FOR UPDATE SKIP LOCKED` (lock a free row and skip rows others have locked) or a conditional update like "set occupied where it is still free".

Q2. How would you allocate the nearest spot to the entry gate? Keep, for each gate, a `PriorityQueue` / `TreeSet` of free spots sorted by distance from that gate. Hide it behind an `AllocationStrategy` interface so you can swap the rule later.

Q3. How would you add weekend pricing or a monthly pass? Write new `FeeStrategy` implementations and pick one based on the date or the vehicle. The parking flow itself does not change.

Q4. How does this change for 1,000 lots in a city? It becomes an HLD (high-level design) problem. Store the spot inventory in a database, per lot. Add a booking service that uses conditional updates so two bookings cannot take one spot. Cache availability so the "how many spots free?" reads are cheap.

10.3 LRU Cache

Why?

> **In simple words:** A fixed-size cache that, when full, throws out the item nobody has used for the longest time. A HashMap finds items fast, and a linked list keeps them in "most recently used" order.

LRU means *Least Recently Used*. This is LeetCode 146, and product companies ask it all the time.

It tests:

- Combining two data structures so that both `get` and `put` run in O(1) — constant time, no matter how big the cache is.
- Careful pointer handling in a linked list.
- For SDE2: thread safety and real-world trade-offs.

What is happening internally?

- A **HashMap** from key → node. It finds any node in O(1).
- A **doubly linked list** kept in order of use. The head side holds the most recently used item; the tail side holds the least recently used one.
- Each node knows both its previous and next node, so you can remove it from the middle in O(1).
- Moving a node to the front is O(1).
- Throwing out the oldest item is O(1) — it is always just before the tail.
- **Sentinel** head and tail nodes. These are dummy nodes that are always there. Because the list is never truly empty, you never need null checks at the ends.

- A `ReentrantLock` makes it thread-safe. Note: **even `get` mutates** the cache — it moves the node to the front. So reads need the lock too, not just writes.

Think of a small bookshelf with room for only a few books. Every time you read a book, you put it back on the far left. When the shelf is full and a new book comes, you remove the one on the far right — the one you haven't touched for the longest time.

```
map: {1→n1, 2→n2, 3→n3}
list: HEAD ⇄ n3 ⇄ n1 ⇄ n2 ⇄ TAIL     (most recent … least recent)
get(2) → move n2 to front;  put(4) at capacity 3 → evict TAIL.prev (n1)
```

How does it work?

`get(k)`:

1. Look up the key in the map.
2. If it is missing, return empty.
3. If it is there, unlink the node from where it is.
4. Add it right after the head (the "most recent" end).
5. Return the value.

`put(k, v)`:

1. If the key already exists, update its value and move it to the front. Done.
2. Otherwise, check if the cache is full.
3. If full, take `tail.prev` (the least recently used node). Unlink it, and also remove its key from the map.
4. Create a new node.
5. Add it to the map and to the front of the list.

Example

```java
// FILE: LRUCacheDemo.java
import java.util.*;
import java.util.concurrent.locks.ReentrantLock;

public class LRUCacheDemo {
    static class LRUCache<K, V> {
        private static final class Node<K, V> { K key; V val; Node<K, V> prev, next; Node(K k, V v) { key =
k; val = v; } }
        private final int capacity;
        private final Map<K, Node<K, V>> map = new HashMap<>();
        private final Node<K, V> head = new Node<>(null, null), tail = new Node<>(null, null);
        private final ReentrantLock lock = new ReentrantLock();   // get() mutates order, so reads need the
lock too

        LRUCache(int capacity) {
            if (capacity <= 0) throw new IllegalArgumentException("capacity must be > 0");
            this.capacity = capacity; head.next = tail; tail.prev = head;
        }

        public Optional<V> get(K key) {
            lock.lock();
            try {
                Node<K, V> n = map.get(key);
                if (n == null) return Optional.empty();
                moveToFront(n);
                return Optional.of(n.val);
            } finally { lock.unlock(); }
        }
```

```
        public void put(K key, V val) {
            lock.lock();
            try {
                Node<K, V> n = map.get(key);
                if (n != null) { n.val = val; moveToFront(n); return; }
                if (map.size() == capacity) {
                    Node<K, V> lru = tail.prev;
                    unlink(lru);
                    map.remove(lru.key);
                }
                n = new Node<>(key, val);
                map.put(key, n);
                addFront(n);
            } finally { lock.unlock(); }
        }

        private void addFront(Node<K, V> n) { n.next = head.next; n.prev = head; head.next.prev = n;
head.next = n; }
        private void unlink(Node<K, V> n) { n.prev.next = n.next; n.next.prev = n.prev; }
        private void moveToFront(Node<K, V> n) { unlink(n); addFront(n); }
    }

    public static void main(String[] args) {
        LRUCache<Integer, String> c = new LRUCache<>(2);
        c.put(1, "a"); c.put(2, "b");
        c.get(1);                 // 1 becomes most recent
        c.put(3, "c");            // evicts 2
        System.out.println(c.get(2).isPresent() + " " + c.get(1).orElse("-") + " " + c.get(3).orElse("-"));
// false a c
    }
}
```

Trace (capacity 2, list shown most recent first):

1. put 1, put 2 → list [2,1].
2. get 1 → 1 moves to the front → [1,2].
3. put 3 → cache is full → evict the tail (2) → [3,1].
4. get 2 → miss, because 2 was evicted.

When would I use it?

- In-process caches that must not grow beyond a fixed size.
- Page caches.
- Database buffer pools (they use approximations of LRU).
- CDN edge caches.

In real Java code, don't write your own. Use Caffeine — it uses W-TinyLFU (a smarter eviction rule with better hit rates) and also handles concurrency and TTL (time-to-live expiry). For simple cases, `LinkedHashMap` with access order is enough (2.5).

What can go wrong?

- Forgetting to remove the evicted key from the map. The list stays small, but the map grows forever.
- Updating an existing key without moving it to the front.
- No lock. Concurrent `get`s also change the list, so without a lock they can corrupt it.
- Storing only values in the nodes. Then, when you evict the tail node, you don't know which key to remove from the map.

Interview questions

Q1. Why HashMap plus a doubly linked list? The map finds the node in O(1). The doubly linked list lets you remove a node from the middle and insert it at the front in O(1). It also gives O(1) access to the least recently used item at the tail. Neither structure can do all of this alone.

Q2. How would you make it scale under heavy concurrency? Use lock striping: split the cache into N independent LRU segments, each with its own lock, and choose the segment by key hash. Threads on different segments don't block each other. The trade-off is that eviction is LRU per segment, so it only approximates a global LRU. Or use Caffeine, which buffers and samples accesses instead of locking on every read.

Q3. How would you add a TTL? Store an expiry time in each node. On access, if the entry has expired, treat it as a miss and remove it. Clean up the rest either with a periodic sweep or lazily when you touch them.

Q4. How would you implement LFU? LFU means *Least Frequently Used*. Keep two maps: key → (value, freq), and freq → `LinkedHashSet` of keys. The `LinkedHashSet` keeps insertion order, so among keys with the same frequency you evict the least recently used one. Also track the current minimum frequency, so eviction is O(1).

10.4 Logger Framework

Why?

> **In simple words:** Build a mini SLF4J/Logback. A logger drops messages below its level, sends the rest to one or more outputs, and can write them on a background thread so the app never waits for the disk.

It tests:

- Interface design — logger, appender and formatter as separate pieces.
- The Chain of Responsibility / filtering idea — each message passes a check before it goes further.
- Thread safety — many threads log at the same time.
- The **async producer–consumer** trade-off. Async means the app hands the message off and moves on. That makes logging fast (low latency), but you can lose messages if the queue is full or the app crashes (less reliable).

What is happening internally?

- **Entities**:
- `Logger` — a name, a minimum level and a list of appenders.
- `LogEvent` — one log line's data, as a Java record.
- `Appender` — *where* events go (console, file, memory...).
- `Formatter` — *how* events look as text.
- `LoggerFactory` — a registry that hands out one logger per name.
- **Patterns**:
- Strategy — the formatter can be swapped.
- Observer-like fan-out — one event is sent to every appender.
- Decorator — `AsyncAppender` wraps any appender and makes it async, without changing that appender.

- Factory/registry — `LoggerFactory` uses `computeIfAbsent`, so each name gets exactly one logger, even with many threads.
- **Async**: a bounded `BlockingQueue` (a queue with a size limit) plus one writer thread. Application threads only drop events into the queue. They never wait on I/O (disk or network).

Think of a courier drop box at an office. You drop the parcel and go back to work. One delivery person empties the box and does the slow travel. If the box is full, the office must decide: drop the parcel or wait.

How does it work?

1. `log(level, msg)` is called. First, do a cheap level check: if `ordinal < min`, return at once. So a disabled DEBUG call costs almost nothing.
2. Build an immutable `LogEvent` (it cannot change after creation) holding time, level, logger name, thread name and message.
3. Hand the event to every appender.
4. `AsyncAppender.append` only `offer`s the event to its queue. If the queue is full, the event is dropped. The app is never blocked.
5. The async worker thread takes events off the queue and passes them to the real appender.
6. `close()` stops the worker, but only after it has written all remaining events.

Example

```
// FILE: LoggerDemo.java
import java.time.*;
import java.util.*;
import java.util.concurrent.*;

public class LoggerDemo {
    enum Level { DEBUG, INFO, WARN, ERROR }
    record LogEvent(Instant at, Level level, String logger, String thread, String message) {}

    interface Formatter { String format(LogEvent e); }
    static class PlainFormatter implements Formatter {
        public String format(LogEvent e) { return "%s [%s] %-5s %s - %s".formatted(e.at(), e.thread(),
e.level(), e.logger(), e.message()); }
    }

    interface Appender { void append(LogEvent e); }
    static class ConsoleAppender implements Appender {
        private final Formatter f; ConsoleAppender(Formatter f) { this.f = f; }
        public synchronized void append(LogEvent e) { System.out.println(f.format(e)); }   // keep lines
whole
    }
    static class MemoryAppender implements Appender {
        final List<LogEvent> events = new CopyOnWriteArrayList<>();
        public void append(LogEvent e) { events.add(e); }
    }
    /** Decorator: makes any appender asynchronous via a bounded queue and one worker thread. */
    static class AsyncAppender implements Appender, AutoCloseable {
        private final BlockingQueue<LogEvent> q = new ArrayBlockingQueue<>(10_000);
        private final Thread worker;
        private volatile boolean running = true;
        AsyncAppender(Appender delegate) {
            worker = Thread.ofPlatform().name("log-writer").daemon(true).start(() -> {
                try {
                    while (running || !q.isEmpty()) {
                        LogEvent e = q.poll(100, TimeUnit.MILLISECONDS);
                        if (e != null) delegate.append(e);
                    }
                } catch (InterruptedException ie) { Thread.currentThread().interrupt(); }
            });
```

```
        }
        public void append(LogEvent e) { if (!q.offer(e)) { /* drop or count: never block the app on logging
*/ } }
        public void close() throws InterruptedException { running = false; worker.join(); }
    }

    static class Logger {
        private final String name; private volatile Level min; private final List<Appender> appenders;
        Logger(String name, Level min, List<Appender> appenders) { this.name = name; this.min = min;
this.appenders = appenders; }
        void setLevel(Level l) { this.min = l; }
        void log(Level l, String msg) {
            if (l.ordinal() < min.ordinal()) return;                    // cheap level check first
            LogEvent e = new LogEvent(Instant.now(), l, name, Thread.currentThread().getName(), msg);
            for (Appender a : appenders) a.append(e);
        }
        void debug(String m) { log(Level.DEBUG, m); } void info(String m) { log(Level.INFO, m); }
        void warn(String m) { log(Level.WARN, m); }   void error(String m) { log(Level.ERROR, m); }
    }

    static final class LoggerFactory {
        private static final ConcurrentHashMap<String, Logger> LOGGERS = new ConcurrentHashMap<>();
        private static volatile List<Appender> defaults = List.of(new ConsoleAppender(new PlainFormatter()));
        static void configure(List<Appender> appenders) { defaults = List.copyOf(appenders); }
        static Logger get(String name) { return LOGGERS.computeIfAbsent(name, n -> new Logger(n, Level.INFO,
defaults)); }
    }

    public static void main(String[] args) throws Exception {
        MemoryAppender mem = new MemoryAppender();
        try (AsyncAppender async = new AsyncAppender(new ConsoleAppender(new PlainFormatter()))) {
            LoggerFactory.configure(List.of(async, mem));
            Logger log = LoggerFactory.get("redemption");
            log.debug("hidden");
            log.info("redeemed 500 points");
            log.error("partner timeout");
        }
        System.out.println("captured=" + mem.events.size());   // 2
    }
}
```

Trace:

1. `debug("hidden")` is dropped, because the logger's level is INFO.
2. INFO and ERROR pass the check.
3. Each goes to the async console appender. The `log-writer` thread prints them.
4. Each also goes to the memory appender. So 2 events are captured.

When would I use it?

- To understand how SLF4J/Logback work inside: async appenders, MDC (per-request context) and level hierarchies.
- When designing audit logs or metrics pipelines.
- For any component that sends ("fans out") events to several outputs ("sinks").

What can go wrong?

- Doing synchronous file or network I/O on the request path. Every request then waits for the disk.
- An unbounded async queue. During a log storm it grows until you get an OOM (OutOfMemoryError).
- A queue that blocks when full. When the disk is slow, the whole app stalls.
- So choose a drop policy (drop, or block) on purpose, and document it.
- Half lines mixed together from different threads. Fix it by synchronising per appender.

- Losing buffered events at shutdown because nobody flushed the queue.

Interview questions

Q1. How do you make logging not slow the application? Check the level first, so disabled logs cost nothing. Use parameterised messages, so the string is built only if the log is enabled. Use asynchronous appenders with a bounded queue, and a clear policy for what happens when it is full — drop or block.

Q2. How would you support per-package levels like `com.fa.loyalty=DEBUG`? Use hierarchical logger names, split by dots. A logger takes the level of its nearest configured ancestor. So `com.fa.loyalty.redeem` gets DEBUG from `com.fa.loyalty`.

Q3. How would you add request IDs to every line? Use an MDC (Mapped Diagnostic Context). It is a ThreadLocal map — a map private to each thread. A request filter puts the request ID into it at the start, every LogEvent copies it, and the filter clears it after the request ends.

10.5 Rate Limiter

Why?

> **In simple words:** Allow each client only so many requests per second, and reject the extra ones. The usual way is a token bucket — each request spends a token, and tokens slowly come back over time.

Almost every product company asks this. It comes as LLD (in memory, per client) and as HLD (distributed across many servers, 13.2).

It tests:

- Algorithm knowledge — token bucket vs window-based counting.
- Clean Strategy design — different algorithms behind one interface.
- Fine-grained concurrency — locking per client, not the whole system.

What is happening internally?

The common algorithms:

Algorithm	State per client	Behaviour
Fixed window	count + window start	Simple; but up to 2× the limit can pass around a window edge
Sliding window log	deque of timestamps	Exact; but O(limit) memory per client (one timestamp per request)
Sliding window counter	two counts	Close estimate using a weighted mix of two windows; little memory
Token bucket	tokens + last refill time	Allows short bursts up to capacity, with a smooth average rate
Leaky bucket	queue	Lets requests out at a constant rate

- **Interface**: `RateLimiter.tryAcquire(clientId)` returns allow or reject. Different algorithms implement it, and you can swap them (Strategy).

- **Per-client state** lives in a `ConcurrentHashMap`.
- **Lock per client** (`synchronized (bucket)`), not one global lock. So client A never waits for client B.
- **Lazy refill**: no background thread adds tokens. Instead, on each call, you calculate how many tokens were earned since the last call.
- **Injected clock**: the limiter asks a clock object for the time. Tests use a fake clock, so results are always the same (deterministic).

Think of a bus pass with a few tickets. Each trip uses one ticket. One new ticket is added every second, but the pass holds at most 5. You can make 5 trips quickly, then only one per second.

How does it work?

Token bucket `tryAcquire`:

1. Find the time passed since the last call.
2. Add the tokens earned in that time, but never go above capacity: `tokens = min(capacity, tokens + elapsed × rate)`.
3. If `tokens ≥ 1`, take one token and allow the request.
4. Else reject it.

Sliding window log `tryAcquire`:

1. Remove timestamps older than the window from the front of the deque.
2. If fewer than `limit` timestamps remain, add "now" and allow.
3. Else reject.

Example

```
// FILE: RateLimiterDemo.java
import java.util.*;
import java.util.concurrent.*;

public class RateLimiterDemo {
    interface RateLimiter { boolean tryAcquire(String clientId); }

    /** Token bucket per client. Lazy refill on each call: no background threads. */
    static class TokenBucketLimiter implements RateLimiter {
        private static final class Bucket {
            double tokens; long lastNanos;
            Bucket(double tokens, long now) { this.tokens = tokens; this.lastNanos = now; }
        }
        private final double capacity, refillPerNano;
        private final ConcurrentHashMap<String, Bucket> buckets = new ConcurrentHashMap<>();
        private final LongSupplierClock clock;

        TokenBucketLimiter(int capacity, double tokensPerSecond, LongSupplierClock clock) {
            this.capacity = capacity; this.refillPerNano = tokensPerSecond / 1e9; this.clock = clock;
        }
        public boolean tryAcquire(String client) {
            long now = clock.nanos();
            Bucket b = buckets.computeIfAbsent(client, k -> new Bucket(capacity, now));
            synchronized (b) {                                  // lock per client, not global
                b.tokens = Math.min(capacity, b.tokens + (now - b.lastNanos) * refillPerNano);
                b.lastNanos = now;
                if (b.tokens >= 1) { b.tokens -= 1; return true; }
                return false;
            }
        }
    }

    /** Exact sliding window log. */
    static class SlidingWindowLogLimiter implements RateLimiter {
```

```
        private final int limit; private final long windowNanos; private final LongSupplierClock clock;
        private final ConcurrentHashMap<String, Deque<Long>> logs = new ConcurrentHashMap<>();
        SlidingWindowLogLimiter(int limit, long windowMillis, LongSupplierClock clock) {
            this.limit = limit; this.windowNanos = windowMillis * 1_000_000; this.clock = clock;
        }
        public boolean tryAcquire(String client) {
            long now = clock.nanos();
            Deque<Long> log = logs.computeIfAbsent(client, k -> new ArrayDeque<>());
            synchronized (log) {
                while (!log.isEmpty() && now - log.peekFirst() >= windowNanos) log.pollFirst();
                if (log.size() < limit) { log.addLast(now); return true; }
                return false;
            }
        }
    }

    interface LongSupplierClock { long nanos(); }
    static class FakeClock implements LongSupplierClock { long t; public long nanos() { return t; } void
plusMillis(long ms) { t += ms * 1_000_000; } }

    public static void main(String[] args) {
        FakeClock clock = new FakeClock();
        RateLimiter tb = new TokenBucketLimiter(5, 1.0, clock);       // burst 5, then 1/second
        int ok = 0; for (int i = 0; i < 8; i++) if (tb.tryAcquire("u1")) ok++;
        System.out.println("burst allowed=" + ok);                     // 5
        clock.plusMillis(2000);
        System.out.println("after 2s: " + tb.tryAcquire("u1") + " " + tb.tryAcquire("u1") + " " +
tb.tryAcquire("u1")); // true true false

        RateLimiter sw = new SlidingWindowLogLimiter(3, 1000, clock);
        System.out.println(sw.tryAcquire("u2") + " " + sw.tryAcquire("u2") + " " + sw.tryAcquire("u2") + " "
+ sw.tryAcquire("u2")); // t t t f
    }
}
```

Trace:

1. Token bucket, capacity 5, refill 1 token per second.
2. 8 calls at once → only 5 allowed. That is the burst.
3. Clock moves 2 seconds → 2 tokens come back → true, true, then false.
4. Sliding log, 3 per second: the first 3 calls pass; the 4th call inside the same window is rejected.

When would I use it?

- API gateways.
- Login and OTP endpoints, to stop brute-force attempts.
- Partner API quotas — when *you* call a partner, stay within *their* limits.
- Per-tenant fairness in multi-tenant platforms, so one big customer can't starve the others.
- Throttling background jobs.

What can go wrong?

- A global lock. It serialises all clients — only one client is checked at a time.
- Memory growth from clients who stopped calling. Their state stays forever unless you expire buckets after a period of no activity.
- Using wall-clock time, which can jump (for example, when the server clock is corrected). Use `System.nanoTime` or an injected monotonic clock — one that only moves forward.
- In a distributed setup, each server instance runs its own limiter. So the real limit becomes limit × number of instances (13.2).

Interview questions

Q1. Token bucket vs sliding window? A token bucket allows controlled bursts while keeping a smooth average rate, and it needs only O(1) state per client. A sliding window log is exact, but it stores one timestamp per request. A sliding window counter gives a close estimate using just two counters.

Q2. How would you make it distributed? Keep the state in Redis and update it with an atomic Lua script per key, so the read-and-update happens as one step (13.2). Another option: keep local buckets on each instance, and give each instance a share of the global quota.

Q3. How do you clean up idle clients? Store the last-access time for each bucket. Remove buckets that have been idle longer than a TTL. Or simply use a cache with expireAfterAccess, which does this for you.

10.6 Bounded Blocking Queue and a Job Processing System

Why?

> **In simple words:** Producers put jobs into a queue with a size limit, and a few worker threads take them out and do them. When the queue is full, producers wait — so the system slows down instead of running out of memory.

This tests concurrency basics inside an LLD problem:

- Producer–consumer — some threads create work, others do it.
- Back-pressure — a full queue makes producers slow down.
- Clean shutdown — every worker stops after the real work is done.
- Per-key ordering — jobs for the same key run in order.

Sometimes it is asked as "implement a blocking queue" (4.14). Sometimes as "design a job processor".

What is happening internally?

- A **bounded** `ArrayBlockingQueue` sits between the producers and a fixed pool of consumer threads. Bounded means it has a maximum size.
- **Back-pressure**: when the queue is full, `put` blocks — the producer waits. So producers slow down instead of filling all memory.
- **Poison pills**: a special "stop" job. We add one per consumer after all the real jobs. A consumer that takes a pill exits.
- (The hand-written queue built with a lock and conditions is in 4.14.)

Think of a railway reservation counter with a fixed number of chairs in the waiting area. When all chairs are full, new people must wait outside. The clerks keep serving from the front.

How does it work?

Producer:

1. `put` 50 jobs into the queue. If the queue is full, it waits.
2. Then `put` one `POISON` job per consumer.

Each consumer:

1. Loop and call `take()`. It waits if the queue is empty.

2. If the job is the pill, exit the loop.
3. Else process the job and loop again.

The main thread shuts down the pool and waits for all threads to finish.

Example

```java
// FILE: JobQueueDemo.java
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class JobQueueDemo {
    record Job(int id, String payload) {}
    static final Job POISON = new Job(-1, "STOP");

    public static void main(String[] args) throws Exception {
        BlockingQueue<Job> queue = new ArrayBlockingQueue<>(10);
        AtomicInteger processed = new AtomicInteger();
        int consumers = 3;
        ExecutorService pool = Executors.newFixedThreadPool(consumers + 1);

        pool.submit(() -> {                                          // producer
            try {
                for (int i = 1; i <= 50; i++) queue.put(new Job(i, "order-" + i));   // blocks when full
                for (int i = 0; i < consumers; i++) queue.put(POISON);              // one pill per consumer
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
            return null;
        });
        for (int c = 0; c < consumers; c++) {
            pool.submit(() -> {
                try {
                    while (true) {
                        Job j = queue.take();
                        if (j == POISON) break;
                        processed.incrementAndGet();                 // do the work
                    }
                } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
                return null;
            });
        }
        pool.shutdown();
        pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("processed=" + processed.get());          // 50
    }
}
```

Trace:

1. The queue (capacity 10) fills up quickly.
2. The producer blocks while the 3 consumers take jobs out.
3. Space opens up, so the producer continues.
4. Each consumer exits when it gets its pill.
5. All 50 jobs are processed exactly once.

When would I use it?

- In-process work queues: sending notifications, writing audit records, enriching data.
- Batching writers, which collect items and write them together.
- As the mental model for Kafka consumers and thread pools — they work the same way inside.

What can go wrong?

- Unbounded queues. There is no back-pressure, so memory can fill up.
- Fewer pills than consumers. Some consumers wait for work forever and never exit.
- A consumer thread dying on an exception. Catch errors per job, and send failed jobs to a retry list or a dead-letter list (a parking place for jobs that keep failing).
- Losing in-memory jobs on a crash. For work that must not be lost, use a durable broker (like Kafka) or an outbox table.

Interview questions

Q1. How do you preserve ordering per key with multiple consumers? Give each worker its own queue. Send each job to a fixed worker using `hash(key) % workers`. All jobs for one key then go to one worker, which handles them in order. This is exactly how Kafka partitions keep per-key order.

Q2. How would you add retries? Keep an attempt count in the job. On failure, put it back in the queue after a delay, using a `DelayQueue` or a scheduled executor. Retry up to N times. After that, move it to a dead-letter list.

Q3. Why bounded? So that overload causes back-pressure (producers wait) or rejection. Without a bound, the queue grows without limit and you get an OOM (OutOfMemoryError).

10.7 Vending Machine

Why?

> **In simple words:** A vending machine behaves differently depending on what "mode" it is in. Before you pick a product, it should not take your coins. The State pattern gives each mode its own class, so the machine always knows what is allowed right now.

This is the standard, most-asked question for the **State pattern**. (The State pattern means: put the behaviour for each mode of an object into its own class, and switch between those classes as the mode changes.)

The interviewer checks three things:

- Can you model a machine where the allowed actions depend on its current state?
- Can you handle money correctly, including giving back change? This is called *change-making*.
- Can you update stock *atomically*? Atomic means all-or-nothing: either the whole sale happens, or nothing changes.

What is happening internally?

- **Entities** (the main objects):
- `Machine` — the *context*, meaning the object that holds the current state and passes each action to it.
- `Product` — has a code, a name and a price.
- Inventory — a `Map<code, qty>` that says how many of each product are left.
- Cash box — an `EnumMap<Coin, count>` that says how many of each coin the machine holds.
- The coins the customer has inserted so far.
- **States**: 1. `Idle` — the only allowed action is *select*. 2. `HasSelection` — you can *insert coin* or *cancel*. 3. Dispensing — the product comes out. 4. Back to `Idle`.

If you try an action the current state does not allow, the default method in the `State` interface throws an exception. So invalid actions fail loudly instead of silently doing something wrong. - **Change-making**: the machine uses a *greedy* approach. Greedy means: always take the largest coin that still fits, then repeat. This gives the fewest coins for *canonical* coin systems — coin sets like Indian coins, where greedy is always the best answer. - The calculation runs on a *copy* of the cash box. - The real cash box is updated only if exact change was possible. So a failed attempt never damages the real cash box.

How does it work?

1. `select(code)` while `Idle`: check that the product exists and is in stock. Then move to `HasSelection`.
2. `insert(coin)`: add the coin to the inserted coins and to the cash box. When the total inserted is ≥ the price, start dispensing.
3. Dispense:
4. Calculate the change due.
5. If exact change is not possible, refund everything and stay in a safe state.
6. Otherwise, reduce the stock by one, give the change, and reset to `Idle`.
7. `cancel()` while in `HasSelection`: give back all inserted coins.

Example

```
// FILE: VendingMachineDemo.java
import java.util.*;

public class VendingMachineDemo {
    enum Coin { ONE(1), TWO(2), FIVE(5), TEN(10), TWENTY(20); final int value; Coin(int v) { value = v; } }
    record Product(String code, String name, int price) {}

    interface State {
        default void select(Machine m, String code) { throw new IllegalStateException("Can't select now"); }
        default void insert(Machine m, Coin c) { throw new IllegalStateException("Select a product first"); }
        default List<Coin> cancel(Machine m) { throw new IllegalStateException("Nothing to cancel"); }
    }

    static class Idle implements State {
        public void select(Machine m, String code) {
            Product p = m.products.get(code);
            if (p == null) throw new IllegalArgumentException("Unknown product");
            if (m.stock.getOrDefault(code, 0) == 0) throw new IllegalStateException("Out of stock");
            m.selected = p;
            m.state = new HasSelection();
        }
    }

    static class HasSelection implements State {
        public void insert(Machine m, Coin c) {
            m.inserted.add(c);
            m.cashbox.merge(c, 1, Integer::sum);
            if (m.insertedTotal() >= m.selected.price()) m.dispense();
        }
        public List<Coin> cancel(Machine m) {
            List<Coin> refund = new ArrayList<>(m.inserted);
            refund.forEach(c -> m.cashbox.merge(c, -1, Integer::sum));
            m.reset();
            return refund;
        }
    }

    static class Machine {
        final Map<String, Product> products = new HashMap<>();
        final Map<String, Integer> stock = new HashMap<>();
        final Map<Coin, Integer> cashbox = new EnumMap<>(Coin.class);
```

```
        final List<Coin> inserted = new ArrayList<>();
        Product selected; State state = new Idle();
        List<Coin> lastChange = List.of();

        void addProduct(Product p, int qty) { products.put(p.code(), p); stock.merge(p.code(), qty,
Integer::sum); }
        void loadCoins(Coin c, int n) { cashbox.merge(c, n, Integer::sum); }

        void select(String code) { state.select(this, code); }
        void insert(Coin c) { state.insert(this, c); }
        List<Coin> cancel() { return state.cancel(this); }

        int insertedTotal() { return inserted.stream().mapToInt(c -> c.value).sum(); }

        void dispense() {
            int changeDue = insertedTotal() - selected.price();
            List<Coin> change = makeChange(changeDue);
            if (change == null) {                                   // can't make change: refund everything
                lastChange = cancel();
                throw new IllegalStateException("Exact change only; refunded " + lastChange);
            }
            stock.merge(selected.code(), -1, Integer::sum);
            System.out.println("Dispensed " + selected.name() + ", change " + change);
            lastChange = change;
            reset();
        }

        /** Greedy change from largest coin; works for canonical coin systems like INR. */
        private List<Coin> makeChange(int amount) {
            List<Coin> out = new ArrayList<>();
            Map<Coin, Integer> temp = new EnumMap<>(cashbox);
            Coin[] desc = Coin.values(); Arrays.sort(desc, Comparator.comparingInt((Coin c) ->
c.value).reversed());
            for (Coin c : desc) {
                while (amount >= c.value && temp.getOrDefault(c, 0) > 0) { amount -= c.value; temp.merge(c,
-1, Integer::sum); out.add(c); }
            }
            if (amount != 0) return null;
            cashbox.clear(); cashbox.putAll(temp);
            return out;
        }

        void reset() { inserted.clear(); selected = null; state = new Idle(); }
    }

    public static void main(String[] args) {
        Machine m = new Machine();
        m.addProduct(new Product("A1", "Chips", 15), 2);
        m.loadCoins(Coin.FIVE, 5); m.loadCoins(Coin.ONE, 10);
        m.select("A1"); m.insert(Coin.TEN); m.insert(Coin.TEN);       // pays 20, change 5
        m.select("A1"); m.insert(Coin.TWO);
        System.out.println("Refund " + m.cancel());
        System.out.println("Stock A1=" + m.stock.get("A1"));          // 1
    }
}
```

Trace:

1. Chips cost ₹15.
2. You insert ₹10 + ₹10 = ₹20. The change is ₹5, given as one FIVE coin.
3. A second purchase is started with ₹2, then cancelled. The refund is [TWO].
4. Stock of A1 goes from 2 to 1.

When would I use it?

Use this for any device or workflow that has *modes* — situations where what you are allowed to do depends on where you are in the process. Examples:

- ATMs, kiosks and payment terminals.
- Order lifecycles (placed → packed → shipped → delivered).
- Trip lifecycles in a ride app.

The general recipe is: the State pattern plus *atomic* transitions (each move from one state to the next either fully happens or does not happen at all).

What can go wrong?

- `if (state == …)` checks spread across every method. This is hard to read and easy to break. The State pattern exists to remove these.
- Giving out the product before confirming that change can be made. Then the customer gets the product but the machine cannot pay back the balance.
- Concurrency — many users acting at the same time — if several users can use the machine at once. Fix: make the machine's operations `synchronized` (only one thread at a time), or model one session at a time.
- Using `double` for money. A `double` cannot store values like 0.1 exactly, so small rounding errors creep in. Use whole numbers (paise or rupees) instead.

Interview questions

Q1. Why the State pattern here? Each state allows a different set of actions. If you put each state's behaviour in its own class, every transition is written clearly in one place, and illegal actions simply cannot happen — the state class rejects them.

Q2. When does greedy change-making fail and what's the alternative? Greedy fails for *non-canonical* coin sets, where taking the biggest coin first is not always best. Example: coins 1, 3, 4 and amount 6 — greedy gives 4+1+1 (three coins), but 3+3 (two coins) is better. The alternative is dynamic programming (DP) — the classic "coin change" problem — extended to respect the limited number of each coin in the cash box.

Q3. How would you add UPI payment? Add a `PaymentMethod` strategy with implementations for coins, UPI and card. (A strategy is a swappable piece of behaviour behind one interface.) When any payment method completes, it moves the machine to the dispensing state.

10.8 Elevator System

Why?

> **In simple words:** Each lift keeps a list of floors to stop at above it and below it. It keeps going in one direction until no stops are left that way, then turns around. A controller decides which lift should answer each button press.

This question tests several things at once:

- **Scheduling logic** — deciding the order in which floors are served.
- **State per elevator** — each lift tracks its own floor and direction.
- **A dispatcher** that chooses which elevator answers a call. This is a Strategy (a swappable rule behind one interface).
- **Reasoning about efficiency and fairness** — fast service overall, but no floor left waiting forever.
- **Thread safety** — how you make a real-time controller safe when many requests arrive at once.

What is happening internally?

- **Entities** (the main objects):
- `Elevator` — has an id, current floor, direction, a set of stops above (up stops) and a set of stops below (down stops).
- `Controller` — holds all the elevators and the dispatch strategy.
- *Hall calls* — the button pressed on a floor (floor + direction wanted).
- *Car calls* — the destination button pressed inside the lift.
- **Data structures**: each elevator has two `TreeSet<Integer>`s — one for stops above, one for stops below. A `TreeSet` keeps numbers sorted, so finding the next stop takes O(log n) — very fast, even with many stops.
- **Algorithm**: **LOOK**. 1. Keep moving in the current direction while there are stops ahead. 2. When no stops are left that way, reverse.

This is the same idea used to move the arm of a hard disk efficiently. - **Patterns**: - Strategy — `DispatchStrategy`, so the rule for picking a lift can be swapped. - State — direction or idle, stored as an enum. - Mediator — the controller sits in the middle and coordinates all the elevators, so they don't talk to each other directly.

How does it work?

1. A hall call comes in. The strategy picks the elevator with the lowest *cost* (a score — lower is better): - An idle elevator costs its distance from the caller. - An elevator already moving toward the caller, in the same direction the caller wants, also costs just its distance. - Any other elevator gets a big penalty added.
2. The chosen elevator adds that floor to its up set or down set.
3. On each `tick()` (one step of time), every elevator moves one floor in its direction. - If the new floor is in its set, it stops there. - If it has no stops ahead, it reverses or becomes idle.

Example

```
// FILE: ElevatorDemo.java
import java.util.*;

public class ElevatorDemo {
    enum Direction { UP, DOWN, IDLE }

    static class Elevator {
        final int id; int floor = 0; Direction dir = Direction.IDLE;
        final TreeSet<Integer> upStops = new TreeSet<>(), downStops = new TreeSet<>();
        Elevator(int id) { this.id = id; }

        void addStop(int target) {
            if (target == floor && dir == Direction.IDLE) { System.out.println("E" + id + " doors open at " +
floor); return; }
            if (target > floor) upStops.add(target); else if (target < floor) downStops.add(target);
            else (dir == Direction.UP ? downStops : upStops).add(target);   // same floor while moving: next
pass
            if (dir == Direction.IDLE) dir = target >= floor ? Direction.UP : Direction.DOWN;
        }

        /** Move one floor per tick (LOOK). */
        void step() {
            if (dir == Direction.UP) {
                if (upStops.isEmpty()) { dir = downStops.isEmpty() ? Direction.IDLE : Direction.DOWN; return;
}
                floor++;
                if (upStops.remove(floor)) System.out.println("E" + id + " stops at " + floor + " (up)");
            } else if (dir == Direction.DOWN) {
```

```
                if (downStops.isEmpty()) { dir = upStops.isEmpty() ? Direction.IDLE : Direction.UP; return; }
                floor--;
                if (downStops.remove(floor)) System.out.println("E" + id + " stops at " + floor + " (down)");
            }
        }
        int pending() { return upStops.size() + downStops.size(); }
    }

    interface DispatchStrategy { Elevator choose(List<Elevator> elevators, int floor, Direction wanted); }

    /** Prefer an elevator already moving toward the caller in the same direction, else the nearest idle,
else least busy. */
    static class NearestSuitable implements DispatchStrategy {
        public Elevator choose(List<Elevator> es, int floor, Direction wanted) {
            return es.stream().min(Comparator.comparingInt(e -> cost(e, floor, wanted))).orElseThrow();
        }
        private int cost(Elevator e, int floor, Direction wanted) {
            int dist = Math.abs(e.floor - floor);
            if (e.dir == Direction.IDLE) return dist;
            boolean onTheWay = (e.dir == Direction.UP && wanted == Direction.UP && e.floor <= floor)
                            || (e.dir == Direction.DOWN && wanted == Direction.DOWN && e.floor >= floor);
            return onTheWay ? dist : dist + 2 * 100 + e.pending();   // heavily penalise wrong-way elevators
        }
    }

    static class Controller {
        final List<Elevator> elevators = new ArrayList<>();
        final DispatchStrategy strategy;
        final int floors;
        Controller(int n, int floors, DispatchStrategy s) {
            for (int i = 0; i < n; i++) elevators.add(new Elevator(i));
            this.floors = floors; this.strategy = s;
        }
        void hallCall(int floor, Direction d) {
            validate(floor);
            Elevator e = strategy.choose(elevators, floor, d);
            System.out.println("Hall call " + floor + " " + d + " -> E" + e.id);
            e.addStop(floor);
        }
        void carCall(int elevatorId, int floor) { validate(floor); elevators.get(elevatorId).addStop(floor);
}
        void tick() { elevators.forEach(Elevator::step); }
        private void validate(int f) { if (f < 0 || f >= floors) throw new IllegalArgumentException("bad
floor " + f); }
    }

    public static void main(String[] args) {
        Controller c = new Controller(2, 10, new NearestSuitable());
        c.hallCall(5, Direction.UP);
        c.hallCall(2, Direction.DOWN);
        for (int t = 0; t < 6; t++) c.tick();
        c.carCall(0, 8);
        for (int t = 0; t < 6; t++) c.tick();
    }
}
```

Trace:

1. Both elevators start at floor 0.
2. The call at floor 5 goes to E0.
3. The call at floor 2 goes to E1, because E1 is idle and nearest.
4. The ticks move E1 to floor 2 and E0 to floor 5.
5. Then someone inside E0 presses 8 (a car call). E0 keeps going up to 8.

When would I use it?

Use this for scheduling problems — deciding who serves what, and in which order:

- Banks of elevators in a building.
- Disk scheduling (moving the disk arm efficiently).
- Delivery or rider dispatch (13.13, 13.18).
- Sending jobs to a pool of workers.

What can go wrong?

- **Starvation**: a floor is never served because the lifts keep turning around near busy floors. LOOK prevents this — it puts a limit on how long any floor can wait, because the lift always finishes its current direction before reversing.
- Giving every call to the nearest elevator without checking its direction. A lift that is close but going the wrong way may take a long time to arrive.
- **Concurrency**: requests can arrive while the elevators are moving (during a tick). The cleanest fix is a single *event-loop* thread — one thread that reads requests from a queue and handles them one by one. This is simpler and safer than putting locks everywhere.

Interview questions

Q1. Which scheduling algorithm would you use and why? LOOK (or SCAN): serve all stops in the current direction, then reverse. It is efficient, and unlike "always go to the nearest request" it is *starvation-free* — no floor waits forever.

Q2. How do you choose which elevator answers a hall call? Use a cost function that looks at distance, whether the lift is going the right direction, and its current load. Put it behind a `DispatchStrategy` interface, so you can swap in other strategies — for example, one tuned for peak hours or one that saves energy.

Q3. How would you handle concurrency? Use one controller thread that processes a queue of requests and ticks, one at a time. This is *actor style* — one owner of the data that handles messages in order. Because only one thread touches the shared state, there are no race conditions (bugs where two threads change the same data at once).

10.9 Splitwise (Expense Sharing)

Why?

> **In simple words:** Friends share bills, and the app remembers who owes whom. At the end, it works out the fewest payments needed to settle everyone, instead of everyone paying everyone.

This question tests:

- **Strategy** — different ways to split a bill (equal, exact amounts, percentages), each a swappable class.
- **Money handling** — especially rounding, so no paisa is lost or created.
- **Modelling balances** between users — who owes whom, and how much.
- **Debt simplification** — the algorithm that reduces the number of payments.

What is happening internally?

- **Entities** (the main objects):
- `User`.

- `Share` — a user plus the amount they owe, stored in paise (₹1 = 100 paise).
- Expenses.
- Balances — a `Map<user, Map<user, paise>>`. A positive value means the other user owes me.
- **Strategies**: `EqualSplit`, `ExactSplit` and `PercentSplit`, all behind one `SplitStrategy` interface. The right one is picked from a map using the `SplitType`.
- **Rounding**:
- All money is in whole paise, never decimals.
- When the total does not divide evenly, the leftover paise are handed out in a fixed, predictable way: the first `remainder` users each get +1 paisa.
- So the shares always add up exactly to the total.
- **Simplification**: 1. Work out each user's **net** balance — what they are owed minus what they owe. 2. Repeatedly match the person owed the most (the largest *creditor*) with the person who owes the most (the largest *debtor*). 3. Two *max-heaps* (priority queues that always give you the biggest item first) make this fast.

How does it work?

1. `addExpense(paidBy, total, users, type, values)`:
2. The chosen strategy produces the shares.
3. For every share that is not the payer's own, increase "user owes payer" on the payer's side.
4. Decrease the same amount on the user's side (the reverse entry).
5. `net()`: add up each user's row to get their net balance.
6. `simplify()`:
7. Put creditors in one heap and debtors in another.
8. Take the top of each. Settle `min(credit, debt)` between them.
9. If either still has something left, push the remainder back into its heap.
10. Repeat until a heap is empty.

Example

```
// FILE: SplitwiseDemo.java
import java.util.*;

public class SplitwiseDemo {
    record User(String id, String name) {}
    enum SplitType { EQUAL, EXACT, PERCENT }
    record Share(User user, long paise) {}

    interface SplitStrategy { List<Share> split(long totalPaise, List<User> users, List<Double> values); }

    static class EqualSplit implements SplitStrategy {
        public List<Share> split(long total, List<User> users, List<Double> ignored) {
            long base = total / users.size(), rem = total % users.size();
            List<Share> out = new ArrayList<>();
            for (int i = 0; i < users.size(); i++) out.add(new Share(users.get(i), base + (i < rem ? 1 :
0))); // distribute leftover paise
            return out;
        }
    }
    static class ExactSplit implements SplitStrategy {
        public List<Share> split(long total, List<User> users, List<Double> amounts) {
            long sum = 0; List<Share> out = new ArrayList<>();
            for (int i = 0; i < users.size(); i++) { long p = Math.round(amounts.get(i)); sum += p;
out.add(new Share(users.get(i), p)); }
            if (sum != total) throw new IllegalArgumentException("exact amounts must add up to total");
            return out;
```

```
        }
    }
    static class PercentSplit implements SplitStrategy {
        public List<Share> split(long total, List<User> users, List<Double> pct) {
            double s = pct.stream().mapToDouble(Double::doubleValue).sum();
            if (Math.abs(s - 100.0) > 1e-9) throw new IllegalArgumentException("percentages must add to
100");
            List<Share> out = new ArrayList<>(); long assigned = 0;
            for (int i = 0; i < users.size(); i++) {
                long p = i == users.size() - 1 ? total - assigned : Math.round(total * pct.get(i) / 100.0);
                assigned += p; out.add(new Share(users.get(i), p));
            }
            return out;
        }
    }

    static class ExpenseService {
        private final Map<SplitType, SplitStrategy> strategies = Map.of(
                SplitType.EQUAL, new EqualSplit(), SplitType.EXACT, new ExactSplit(), SplitType.PERCENT, new
PercentSplit());
        /** balance[a][b] > 0 means b owes a. Stored as nested maps for sparse groups. */
        private final Map<String, Map<String, Long>> balances = new HashMap<>();

        synchronized void addExpense(User paidBy, long totalPaise, List<User> users, SplitType type,
List<Double> values) {
            for (Share s : strategies.get(type).split(totalPaise, users, values)) {
                if (s.user().equals(paidBy)) continue;
                adjust(paidBy.id(), s.user().id(), s.paise());
                adjust(s.user().id(), paidBy.id(), -s.paise());
            }
        }
        private void adjust(String a, String b, long d) {
            balances.computeIfAbsent(a, k -> new HashMap<>()).merge(b, d, Long::sum);
        }
        Map<String, Long> net() {                                  // + means others owe this user
            Map<String, Long> net = new TreeMap<>();
            balances.forEach((a, m) -> m.values().forEach(v -> net.merge(a, v, Long::sum)));
            return net;
        }
        /** Greedy settlement: largest creditor paid by largest debtor. */
        List<String> simplify() {
            PriorityQueue<long[]> cred = new PriorityQueue<>((x, y) -> Long.compare(y[1], x[1]));
            PriorityQueue<long[]> debt = new PriorityQueue<>((x, y) -> Long.compare(y[1], x[1]));
            List<String> ids = new ArrayList<>(net().keySet());
            Map<String, Long> net = net();
            for (int i = 0; i < ids.size(); i++) {
                long v = net.get(ids.get(i));
                if (v > 0) cred.add(new long[]{i, v}); else if (v < 0) debt.add(new long[]{i, -v});
            }
            List<String> tx = new ArrayList<>();
            while (!cred.isEmpty() && !debt.isEmpty()) {
                long[] c = cred.poll(), d = debt.poll();
                long amt = Math.min(c[1], d[1]);
                tx.add(ids.get((int) d[0]) + " pays " + ids.get((int) c[0]) + " " + amt / 100.0);
                if (c[1] > amt) cred.add(new long[]{c[0], c[1] - amt});
                if (d[1] > amt) debt.add(new long[]{d[0], d[1] - amt});
            }
            return tx;
        }
    }

    public static void main(String[] args) {
        User a = new User("A", "Asha"), b = new User("B", "Bharat"), c = new User("C", "Chirag");
        ExpenseService s = new ExpenseService();
        s.addExpense(a, 90000, List.of(a, b, c), SplitType.EQUAL, null);                // A paid 900 for
all
        s.addExpense(b, 30000, List.of(a, b, c), SplitType.PERCENT, List.of(50.0, 25.0, 25.0));
        System.out.println(s.net());           // {A=45000, B=-7500, C=-37500}
        System.out.println(s.simplify());      // [C pays A 375.0, B pays A 75.0]
    }
}
```

Trace:

1. A pays ₹900 for A, B and C. That is ₹300 each. So B owes A ₹300, and C owes A ₹300.
2. B pays ₹300, split 50/25/25. So A owes B ₹150, and C owes B ₹75.
3. Net balances: A +450, B −75, C −375.
4. Simplified: C pays A ₹375, and B pays A ₹75.

That is two payments instead of four.

When would I use it?

- Any *ledger-style* balance tracking — where you record every transaction and work out balances from them.
- Settlement and reconciliation systems (checking that two sets of records agree).
- Marketplace payouts to sellers.
- Inter-company netting — companies cancelling out what they owe each other so only the difference is paid.

What can go wrong?

- Problems with input and money values:
- Using `double` for money. Small rounding errors add up over time (rounding drift).
- Percentages that do not add up to 100.
- Exact amounts that do not add up to the total.
- Claiming the greedy simplification is *optimal* (the best possible). Finding the true minimum number of payments is *NP-hard* in general — no known fast method exists for large groups. Greedy is a good *heuristic* — a practical method that gives a good answer, not always the perfect one.
- Editing or deleting expenses by changing balances directly. Instead, keep expenses as *immutable* records (never changed after saving). Then either recompute balances from them, or post *reversal* entries that cancel the old expense.

Interview questions

Q1. How do you handle ₹100 split three ways? Work in paise: 3333, 3333 and 3334. Hand out the leftover paisa in a fixed, predictable way, so the shares always add up exactly to the total.

Q2. How do you minimise settlements? First work out each user's net balance. Then repeatedly pair the largest creditor with the largest debtor, using heaps — this runs in O(n log n). Mention that finding the exact minimum number of payments is NP-hard, so this greedy method is a good approximation, not a guaranteed best.

Q3. How would you support groups and expense edits? Keep balances scoped to each group. Store expenses as immutable records, and treat an edit as a reversal of the old expense plus a new expense. Then either recompute the balances or apply just the differences (deltas).

10.10 BookMyShow (Movie Ticket Booking)

Why?

> **In simple words:** When you pick seats, the system blocks them for you for a few minutes while you pay. The hard part is making sure two people clicking at the same moment can never get the same

> seat.

This question tests:

- **A realistic domain**: cities → theatres → screens → shows → seats.
- **A booking lifecycle with temporary holds** — seats are reserved for a short time while you pay.
- **Payment integration.**
- Above all, **concurrency on shared inventory** — many users touching the same seats at the same time. The rule is simple: no seat may ever be sold twice.

What is happening internally?

- **Entities** (the main objects):
- `Show` — holds its seats, the state of each seat, and a lock.
- `Seat` — id, category and price.
- `SeatState` — AVAILABLE, HELD or BOOKED; plus which booking ID holds it and when the hold expires.
- `Booking` — starts as PENDING, then becomes CONFIRMED, EXPIRED or CANCELLED.
- **Concurrency strategy**:
- One `ReentrantLock` **per show**. So users only wait for each other if they are booking the same show. Different shows run in parallel.
- Inside the lock, **first check all the requested seats, then mark all of them**. This is all-or-nothing: you get every seat you asked for, or none.
- **Holds** expire after N minutes.
- An expired hold is treated as available the next time someone tries to book. This is called *lazy expiry* — you clean up only when you need to.
- A background *sweeper* job can also clean up old holds.
- `confirm` is **idempotent** — calling it twice gives the same result as calling it once. So if the payment gateway sends the same success callback twice, the second call just returns the same booking.

How does it work?

1. `hold(user, show, seats)`:
2. Lock the show.
3. Check that every seat is AVAILABLE, or HELD with an expired hold.
4. Mark all of them HELD, with the booking ID and expiry time.
5. Create a PENDING booking.
6. Unlock.
7. Payment happens *outside* the lock, so a slow payment does not block other users.
8. `confirm(bookingId, paymentOk)`:
9. Lock the show.
10. If the booking is already final, just return it (idempotent).
11. If payment failed, or the hold expired, or the seats were taken by someone else — release the seats and mark the booking CANCELLED or EXPIRED. Refund if the user already paid.
12. Otherwise, mark the seats BOOKED and the booking CONFIRMED.

Example

```
// FILE: BookMyShowDemo.java
import java.time.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.locks.ReentrantLock;

public class BookMyShowDemo {
    enum SeatStatus { AVAILABLE, HELD, BOOKED }
    enum BookingStatus { PENDING, CONFIRMED, EXPIRED, CANCELLED }
    record Seat(String id, String category, long pricePaise) {}

    static final class SeatState { SeatStatus status = SeatStatus.AVAILABLE; String bookingId; Instant
holdUntil; }

    static final class Show {
        final String id; final Map<String, Seat> seats = new LinkedHashMap<>();
        final Map<String, SeatState> state = new HashMap<>();
        final ReentrantLock lock = new ReentrantLock();             // one lock per show: contention is per
show only
        Show(String id, List<Seat> seatList) { this.id = id; seatList.forEach(s -> { seats.put(s.id(), s);
state.put(s.id(), new SeatState()); }); }
    }

    static final class Booking {
        final String id, userId, showId; final List<String> seatIds; final long amount; final Instant
expiresAt;
        volatile BookingStatus status = BookingStatus.PENDING;
        Booking(String id, String u, String show, List<String> seats, long amt, Instant exp) {
            this.id = id; userId = u; showId = show; seatIds = List.copyOf(seats); amount = amt; expiresAt =
exp;
        }
    }

    static class BookingService {
        private final Map<String, Show> shows = new ConcurrentHashMap<>();
        private final Map<String, Booking> bookings = new ConcurrentHashMap<>();
        private final AtomicLong seq = new AtomicLong();
        private final Duration holdFor; private final Clock clock;
        BookingService(Duration holdFor, Clock clock) { this.holdFor = holdFor; this.clock = clock; }

        void addShow(Show s) { shows.put(s.id, s); }

        Booking hold(String userId, String showId, List<String> seatIds) {
            Show show = Objects.requireNonNull(shows.get(showId), "no show");
            Instant now = clock.instant();
            show.lock.lock();
            try {
                for (String sid : seatIds) {                                    // 1. check ALL seats first
                    SeatState st = show.state.get(sid);
                    if (st == null) throw new IllegalArgumentException("bad seat " + sid);
                    boolean expiredHold = st.status == SeatStatus.HELD && st.holdUntil.isBefore(now);
                    if (st.status != SeatStatus.AVAILABLE && !expiredHold)
                        throw new IllegalStateException("Seat " + sid + " not available");
                }
                String bid = "B" + seq.incrementAndGet();
                long amount = seatIds.stream().mapToLong(s -> show.seats.get(s).pricePaise()).sum();
                Instant until = now.plus(holdFor);
                for (String sid : seatIds) {                                    // 2. then mark ALL (all-or-
nothing)
                    SeatState st = show.state.get(sid);
                    st.status = SeatStatus.HELD; st.bookingId = bid; st.holdUntil = until;
                }
                Booking b = new Booking(bid, userId, showId, seatIds, amount, until);
                bookings.put(bid, b);
                return b;
            } finally { show.lock.unlock(); }
        }
```

```
        Booking confirm(String bookingId, boolean paymentOk) {
            Booking b = Objects.requireNonNull(bookings.get(bookingId));
            Show show = shows.get(b.showId);
            show.lock.lock();
            try {
                if (b.status != BookingStatus.PENDING) return b;              // idempotent
                boolean stillHeld = b.seatIds.stream().allMatch(s ->
bookingId.equals(show.state.get(s).bookingId)
                        && show.state.get(s).status == SeatStatus.HELD);
                if (!paymentOk || !stillHeld || clock.instant().isAfter(b.expiresAt)) {
                    release(show, b); b.status = paymentOk ? BookingStatus.EXPIRED : BookingStatus.CANCELLED;
                    return b;                                                  // caller refunds if payment
succeeded
                }
                b.seatIds.forEach(s -> show.state.get(s).status = SeatStatus.BOOKED);
                b.status = BookingStatus.CONFIRMED;
                return b;
            } finally { show.lock.unlock(); }
        }

        private void release(Show show, Booking b) {
            for (String s : b.seatIds) {
                SeatState st = show.state.get(s);
                if (b.id.equals(st.bookingId) && st.status == SeatStatus.HELD) { st.status =
SeatStatus.AVAILABLE; st.bookingId = null; st.holdUntil = null; }
            }
        }
    }

    public static void main(String[] args) throws Exception {
        List<Seat> seats = new ArrayList<>();
        for (int i = 1; i <= 5; i++) seats.add(new Seat("A" + i, "GOLD", 25000));
        BookingService svc = new BookingService(Duration.ofMinutes(10), Clock.systemUTC());
        svc.addShow(new Show("S1", seats));

        // 10 users race for seats A1 + A2: exactly one must win
        ExecutorService pool = Executors.newFixedThreadPool(10);
        List<Future<String>> results = new ArrayList<>();
        for (int u = 0; u < 10; u++) {
            String user = "u" + u;
            results.add(pool.submit(() -> {
                try { return svc.hold(user, "S1", List.of("A1", "A2")).id; } catch (IllegalStateException e)
{ return null; }
            }));
        }
        List<String> winners = new ArrayList<>();
        for (Future<String> f : results) if (f.get() != null) winners.add(f.get());
        pool.shutdown();
        System.out.println("winners=" + winners.size());                              // 1
        System.out.println(svc.confirm(winners.get(0), true).status);                 // CONFIRMED
    }
}
```

Trace:

1. 10 threads race for seats A1 + A2.
2. The first thread to get the show lock holds both seats.
3. The other 9 threads find the seats HELD and fail.
4. Result: exactly one winner, and its confirmation succeeds.

When would I use it?

Use this for any *limited inventory* where there is a delay between choosing and paying:

- Flight and hotel seats.
- Flash-sale stock (13.19).
- Appointment slots.

- Coupons that can only be redeemed a limited number of times.

What can go wrong?

- Checking seats one by one and marking them one by one. You can end up holding only some of the seats (a *partial hold*).
- Using one global lock for everything. Then no two shows can be booked in parallel.
- Keeping the lock held while calling the payment gateway. Payment is slow, so every other user of that show waits.
- Holds that never expire. Seats stay stuck forever.
- Payment succeeding *after* the hold expired. You need a refund path for this case.
- At scale, in-memory locks do not work across several server instances — each server has its own lock. Move the seat state to a shared store:
- The database: a conditional `UPDATE … WHERE status='AVAILABLE' OR hold_until < now()`, then check how many rows were updated. If fewer rows changed than seats requested, someone else got there first.
- Redis: `SET seat:{show}:{seat} bookingId NX PX 600000`. `NX` means "only set if it does not already exist", and `PX 600000` makes it expire after 10 minutes.

Interview questions

Q1. How do you prevent double booking? Check and hold all the requested seats *atomically* (all-or-nothing). In a single server, do this under a per-show lock. In a distributed system, use a conditional multi-row UPDATE and verify the updated row count, or use Redis `SET NX`.

Q2. How do seat holds work? Seats move to HELD, with a booking ID and an expiry time. If the user confirms within that window, the seats become BOOKED. When the hold expires — detected lazily on the next attempt, or by a sweeper job — the seats are released.

Q3. What if payment succeeds after the hold expired and someone else took the seat? Mark the booking EXPIRED and trigger a refund. The payment callback handler must be idempotent, so a repeated callback does not refund twice or book twice.

10.11 Loyalty Points Ledger (your domain)

Why?

> **In simple words:** Instead of just storing "you have 500 points", you write down every earn, spend and expiry as a separate line, like a bank passbook. The balance is just the total of those lines, so you can always explain and check it.

This is your own system. So treat it as the design you can defend three levels deep — expect the interviewer to keep asking "and then what?".

It tests:

- **Ledger modelling** — recording every change as an entry.
- **Idempotency** — the same request processed twice has the same effect as once.
- **FIFO expiry** — first in, first out: the oldest points are used first and expire first.
- **Per-account concurrency** — two operations on the same account must not clash.

It also doubles as your best behavioural story ("tell me about something you built").

What is happening internally?

- **A ledger, not just a balance**:
- Every change is an **immutable entry** (never edited after saving): ACCRUAL (points earned), REDEMPTION (points spent), REVERSAL (an undo), or EXPIRY.
- The balance is a cached sum of the entries. It is updated in the same transaction as the entry, so the two always match.
- **Idempotency**:
- Each operation carries an *idempotency key* — a unique ID from the source, such as the order ID or the redemption request ID.
- If the same key arrives again, the system returns the original entry instead of posting the points a second time.
- **Lots for FIFO expiry**:
- Each accrual creates a *lot* — a batch of points with "points still remaining" and an expiry date.
- Redemptions use up the oldest lots first.
- So at expiry time, the system knows exactly how many points from each accrual are still unused.
- **Concurrency**: handle one operation at a time per account. This is called *serialising* per account.
- In memory: `synchronized (account)`.
- In the database: a row lock or an atomic conditional update.
- **Patterns**:
- Command — each operation is stored as an entry.
- Repository — one class hides how entries are stored.
- An injected `Clock` — time is passed in, so tests can control "now".

How does it work?

1. `accrue(acc, pts, key)`:
2. Check the idempotency key.
3. Lock the account.
4. Add an ACCRUAL entry.
5. Push a new lot that expires after the validity period.
6. `redeem(acc, pts, key)`:
7. Check the idempotency key.
8. Lock the account.
9. Expire any lots that are already due.
10. Check that the balance is enough.
11. Use up lots, oldest first.
12. Add a REDEMPTION entry (with negative points).
13. `expireDue(acc)`:
14. Remove every lot whose expiry date has passed.
15. Add one EXPIRY entry for the points those lots still had.

Example

```
// FILE: LoyaltyLedgerDemo.java
import java.time.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicLong;

public class LoyaltyLedgerDemo {
    enum EntryType { ACCRUAL, REDEMPTION, REVERSAL, EXPIRY }
    record Entry(long id, String accountId, EntryType type, long points, String idemKey, Instant at, Long
refId) {}

    static final class Lot {                    // an accrual with remaining points, consumed FIFO
        final long entryId; final Instant expiresAt; long remaining;
        Lot(long entryId, long pts, Instant exp) { this.entryId = entryId; this.remaining = pts;
this.expiresAt = exp; }
    }

    static final class Account {
        final String id; long balance;
        final List<Entry> entries = new ArrayList<>();
        final Deque<Lot> lots = new ArrayDeque<>();          // ordered by accrual time
        Account(String id) { this.id = id; }
    }

    static class InsufficientPoints extends RuntimeException { InsufficientPoints(String m) { super(m); } }

    static class LedgerService {
        private final Map<String, Account> accounts = new ConcurrentHashMap<>();
        private final Map<String, Entry> byIdemKey = new ConcurrentHashMap<>();
        private final AtomicLong seq = new AtomicLong();
        private final Clock clock; private final Period validity;
        LedgerService(Clock clock, Period validity) { this.clock = clock; this.validity = validity; }

        Account account(String id) { return accounts.computeIfAbsent(id, Account::new); }

        Entry accrue(String accountId, long points, String idemKey) {
            if (points <= 0) throw new IllegalArgumentException("points must be > 0");
            return idempotent(idemKey, () -> {
                Account a = account(accountId);
                synchronized (a) {                                      // per-account serialisation
                    Entry e = append(a, EntryType.ACCRUAL, points, idemKey, null);
                    Instant exp = clock.instant().atZone(ZoneOffset.UTC).plus(validity).toInstant();
                    a.lots.addLast(new Lot(e.id(), points, exp));
                    return e;
                }
            });
        }

        Entry redeem(String accountId, long points, String idemKey) {
            return idempotent(idemKey, () -> {
                Account a = account(accountId);
                synchronized (a) {
                    expireDue(a);                                       // never redeem expired points
                    if (a.balance < points) throw new InsufficientPoints("have " + a.balance + ", need " +
points);
                    long left = points;
                    while (left > 0) {                                  // consume oldest lots first
                        Lot lot = a.lots.peekFirst();
                        long take = Math.min(left, lot.remaining);
                        lot.remaining -= take; left -= take;
                        if (lot.remaining == 0) a.lots.pollFirst();
                    }
                    return append(a, EntryType.REDEMPTION, -points, idemKey, null);
                }
            });
        }

        /** Expire lots past their date; returns points expired. Run daily by a scheduler, and lazily before
redemption. */
```

```
        long expireDue(Account a) {
            synchronized (a) {
                long expired = 0; Instant now = clock.instant();
                while (!a.lots.isEmpty() && !a.lots.peekFirst().expiresAt.isAfter(now)) expired +=
a.lots.pollFirst().remaining;
                if (expired > 0) append(a, EntryType.EXPIRY, -expired, "expiry-" + a.id + "-" + now, null);
                return expired;
            }
        }

        private Entry idempotent(String key, Callable<Entry> op) {
            Entry existing = byIdemKey.get(key);
            if (existing != null) return existing;
            synchronized (key.intern()) {                                  // demo only; in prod a DB unique
constraint does this
                existing = byIdemKey.get(key);
                if (existing != null) return existing;
                try { Entry e = op.call(); byIdemKey.put(key, e); return e; }
                catch (RuntimeException e) { throw e; }
                catch (Exception e) { throw new IllegalStateException(e); }
            }
        }

        private Entry append(Account a, EntryType t, long pts, String key, Long ref) {
            Entry e = new Entry(seq.incrementAndGet(), a.id, t, pts, key, clock.instant(), ref);
            a.entries.add(e); a.balance += pts;
            return e;
        }
    }

    static class MutableClock extends Clock {
        Instant now = Instant.parse("2026-01-01T00:00:00Z");
        public ZoneId getZone() { return ZoneOffset.UTC; } public Clock withZone(ZoneId z) { return this; }
        public Instant instant() { return now; }
    }

    public static void main(String[] args) {
        MutableClock clock = new MutableClock();
        LedgerService svc = new LedgerService(clock, Period.ofMonths(12));
        svc.accrue("acc1", 500, "order-1");
        clock.now = Instant.parse("2026-06-01T00:00:00Z");
        svc.accrue("acc1", 300, "order-2");
        svc.accrue("acc1", 300, "order-2");                  // duplicate event: ignored
        svc.redeem("acc1", 600, "redeem-1");                 // takes 500 from lot1, 100 from lot2
        System.out.println("balance=" + svc.account("acc1").balance);                  // 200
        clock.now = Instant.parse("2027-06-02T00:00:00Z");   // lot2 (expires 2027-06-01) is now due
        System.out.println("expired=" + svc.expireDue(svc.account("acc1")) + " balance=" +
svc.account("acc1").balance); // 200, 0
    }
}
```

Trace:

1. January: +500 points. This is lot 1, expiring January 2027.
2. June: +300 points. This is lot 2, expiring June 2027.
3. The duplicate `order-2` event is ignored.
4. Redeem 600: all 500 from lot 1, plus 100 from lot 2. Balance is now 200.
5. On 2 June 2027, the remaining 200 in lot 2 expire. Balance is now 0.

When would I use it?

Use this for points, wallets, credits, gift cards, prepaid balances and inventory movements. In other words, anywhere that *auditability* (being able to show the history of every change) and "never post the same thing twice" matter more than raw speed.

What can go wrong?

- Keeping only a balance that you overwrite, with no ledger. Then there is no audit trail, and nothing to *reconcile* against (cross-check the balance with).
- Checking idempotency outside the transaction or lock. Two duplicate requests can then both pass the check at the same moment and both get posted. In the database, the real guard is a unique constraint on `(tenant_id, idempotency_key)`.
- Redeeming points that should already have expired. Always run expiry before a redemption.
- Reversals that do not put the points back into the right lots.

Interview questions

Q1. How do you prevent duplicate reward postings when events are retried? Store an idempotency key for each source event. Protect it with a unique constraint, saved in the same transaction as the ledger entry. If a duplicate arrives, return the existing entry instead of posting again.

Q2. How does FIFO expiry work? Each accrual becomes a lot, with its remaining points and an expiry date. Redemptions use up the oldest lots first. Expiry then removes whatever is left in lots that are past their date.

Q3. How do balance and ledger stay consistent? Both are updated in one transaction, so they change together or not at all. On top of that, a *reconciliation* job regularly checks, for each account, that the sum of the entries equals the cached balance.

10.12 Rules Engine for Incentives (your domain)

Why?

> **In simple words:** Admins describe incentive rules as data — "if sales ≥ X and visits ≥ Y, pay Z". The engine reads that data, checks each sales rep against it, and pays out. No developer needs to change code for a new scheme.

This is also your own domain.

Product teams want admins to define schemes like this **without code changes**:

> sales of group X ≥ ₹50,000 AND (visits ≥ 20 OR new outlets ≥ 5) → slab payout

It tests several patterns:

- **Composite** — a tree where single items and groups are treated the same way.
- **Specification** — a business rule as an object that answers "does this pass?".
- **Strategy** — swappable payout calculations.
- **Factory** — building the objects from a stored definition (parsing).
- **Explainability** — telling the user *why* they did or did not qualify.

What is happening internally?

- **Conditions** form a tree:
- A sealed `Condition` interface. (*Sealed* means only the listed classes may implement it.)
- `Threshold` leaves — a single check like "visits ≥ 20".
- `All` / `Any` / `Not` composites — AND, OR and NOT over other conditions.

- Together this is Composite + Specification + Interpreter. (Interpreter means: walking a tree of objects to "run" a small language.)
- **Payouts** are strategies:
- `Flat` — a fixed amount.
- `PercentOf(metric)` — a percentage of some metric.
- `Slabs(metric, ranges)` — a fixed amount depending on which range the metric falls in.
- **Context**: the facts for one user for one period, as a `Map<metric, value>`. These are computed earlier, upstream — *pre-aggregated* (totals already calculated), not worked out from raw data during evaluation.
- **Factory**: `parse(Map)` builds the condition tree from a JSON-like definition stored per tenant (per client company).
- **Explainability**: every evaluation also returns the *why*, for example: "sales 81000 GTE 50000 ✓, visits 14 GTE 20 × ...".

How does it work?

`engine.evaluate(schemes, ctx)` does this for each scheme:

1. Evaluate the condition tree *recursively* — each node asks its children first, then combines their answers. Build the explanation along the way.
2. If the condition passes, calculate the payout using the scheme's payout strategy.
3. Return one outcome per scheme.

Example

```
// FILE: IncentiveRulesDemo.java
import java.util.*;
import java.util.function.*;

public class IncentiveRulesDemo {
    /** Facts about one user for one period. */
    record Context(String userId, Map<String, Double> metrics) {
        double get(String k) { return metrics.getOrDefault(k, 0.0); }
    }
    record Result(boolean passed, String explanation) {}

    // ---------- Conditions: Composite + Specification ----------
    sealed interface Condition permits Threshold, All, Any, Not {
        Result eval(Context c);
    }
    enum Op { GTE, LTE, EQ;
        boolean test(double a, double b) { return switch (this) { case GTE -> a >= b; case LTE -> a <= b;
case EQ -> a == b; }; }
    }
    record Threshold(String metric, Op op, double value) implements Condition {
        public Result eval(Context c) {
            double actual = c.get(metric);
            boolean ok = op.test(actual, value);
            return new Result(ok, "%s=%.0f %s %.0f %s".formatted(metric, actual, op, value, ok ? "✓" : "✗"));
        }
    }
    record All(List<Condition> parts) implements Condition {
        public Result eval(Context c) {
            List<Result> rs = parts.stream().map(p -> p.eval(c)).toList();
            return new Result(rs.stream().allMatch(Result::passed), "ALL(" + String.join(", ",
rs.stream().map(Result::explanation).toList()) + ")");
        }
    }
    record Any(List<Condition> parts) implements Condition {
        public Result eval(Context c) {
            List<Result> rs = parts.stream().map(p -> p.eval(c)).toList();
```

```
            return new Result(rs.stream().anyMatch(Result::passed), "ANY(" + String.join(", ",
rs.stream().map(Result::explanation).toList()) + ")");
        }
    }
    record Not(Condition inner) implements Condition {
        public Result eval(Context c) { Result r = inner.eval(c); return new Result(!r.passed(), "NOT(" +
r.explanation() + ")"); }
    }

    // ---------- Payouts: Strategy ----------
    interface Payout { long amountPaise(Context c); }
    record Flat(long paise) implements Payout { public long amountPaise(Context c) { return paise; } }
    record PercentOf(String metric, double pct) implements Payout {
        public long amountPaise(Context c) { return Math.round(c.get(metric) * 100 * pct / 100.0); }
    }
    record Slab(double from, double to, long paise) {}
    record Slabs(String metric, List<Slab> slabs) implements Payout {
        public long amountPaise(Context c) {
            double v = c.get(metric);
            return slabs.stream().filter(s -> v >= s.from() && v <
s.to()).mapToLong(Slab::paise).findFirst().orElse(0);
        }
    }

    record Scheme(String id, String name, Condition eligibility, Payout payout) {}
    record Outcome(String schemeId, String userId, boolean qualified, long payoutPaise, String why) {}

    static class Engine {
        List<Outcome> evaluate(List<Scheme> schemes, Context ctx) {
            List<Outcome> out = new ArrayList<>();
            for (Scheme s : schemes) {
                Result r = s.eligibility().eval(ctx);
                out.add(new Outcome(s.id(), ctx.userId(), r.passed(), r.passed() ?
s.payout().amountPaise(ctx) : 0, r.explanation()));
            }
            return out;
        }
    }

    // ---------- Factory: build conditions from a simple definition (would be JSON in production) ----------
    static Condition parse(Map<String, Object> def) {
        String type = (String) def.get("type");
        return switch (type) {
            case "threshold" -> new Threshold((String) def.get("metric"), Op.valueOf((String) def.get("op")),
((Number) def.get("value")).doubleValue());
            case "all" -> new All(children(def));
            case "any" -> new Any(children(def));
            case "not" -> new Not(parse(castMap(def.get("child"))));
            default -> throw new IllegalArgumentException("unknown condition " + type);
        };
    }
    @SuppressWarnings("unchecked") static Map<String, Object> castMap(Object o) { return (Map<String,
Object>) o; }
    @SuppressWarnings("unchecked") static List<Condition> children(Map<String, Object> def) {
        return ((List<Object>) def.get("children")).stream().map(o -> parse(castMap(o))).toList();
    }

    public static void main(String[] args) {
        Condition cond = parse(Map.of("type", "all", "children", List.of(
                Map.of("type", "threshold", "metric", "sales_groupX", "op", "GTE", "value", 50000),
                Map.of("type", "any", "children", List.of(
                        Map.of("type", "threshold", "metric", "visits", "op", "GTE", "value", 20),
                        Map.of("type", "threshold", "metric", "new_outlets", "op", "GTE", "value", 5))))));

        Scheme festive = new Scheme("S1", "Festive push", cond,
                new Slabs("sales_groupX", List.of(new Slab(50000, 75000, 200000), new Slab(75000,
Double.MAX_VALUE, 350000))));
        Engine engine = new Engine();
        Context rep1 = new Context("rep1", Map.of("sales_groupX", 81000.0, "visits", 14.0, "new_outlets",
6.0));
        Context rep2 = new Context("rep2", Map.of("sales_groupX", 52000.0, "visits", 11.0, "new_outlets",
1.0));
```

```
        engine.evaluate(List.of(festive), rep1).forEach(System.out::println);
        engine.evaluate(List.of(festive), rep2).forEach(System.out::println);
    }
}
```

Trace:

- **rep1**: 1. Sales 81,000 ✓. 2. Visits 14 ×, but new outlets 6 ✓ — so the ANY part passes. 3. So rep1 qualifies. 4. Sales fall in the 75,000+ slab, so the payout is ₹3,500.
- **rep2**: 1. Sales 52,000 ✓. 2. Visits 11 × and new outlets 1 × — so the ANY part fails. 3. So rep2 does not qualify. The explanation shows exactly why.

When would I use it?

Use this anywhere business users need to configure logic themselves:

- Incentive and commission schemes.
- Promotions and coupon eligibility.
- Fraud rules.
- Feature-flag targeting (who sees a new feature).
- Pricing rules.

What can go wrong?

- Rules changing in the middle of a period. Fix: *version* the scheme definitions, and evaluate each period with the version that was active when the period started.
- Very deep nesting or very expensive conditions. Fix: validate definitions when they are saved — limit the depth and allow only known metrics.
- Evaluating against raw transaction data for every user on every request. This is slow. Fix: pre-aggregate the metrics (13.14).
- Payouts that cannot be reproduced later. Fix: store the evaluation trace and the input values with every payout.

Interview questions

Q1. How would you let admins create rules without deployments? Store rule trees as data (JSON). When loading, parse them into Composite condition objects and payout strategies. Validate them when they are saved, and keep versions.

Q2. How do you explain to a sales rep why they didn't qualify? Each node in the tree returns pass/fail plus a short description. The engine joins these into a full explanation and stores it with the outcome.

Q3. How does this scale to hundreds of thousands of reps? Use pre-aggregated metrics per user and period. Run the evaluation in parallel — it is *stateless* (it keeps no data between calls), so it can be split by tenant across workers. Cache the scheme definitions so they are not loaded again for every rep (13.14).

10.13 Snake and Ladder

Why?

> **In simple words:** Snake and Ladder is a small game, but it shows whether you can split a problem into clean classes. The key trick: a snake and a ladder are the same thing — a jump from one cell to

> another.

Interviewers often use this as a warm-up for machine coding. (Machine coding means writing working code for a design problem in 60–90 minutes.)

It checks four things:

- **Clean entity design** — the right classes, each with one job.
- **Pluggable dice** — the dice is an interface, so you can swap it. This is the *Strategy* pattern. It also makes testing easy, because a test can use fixed dice that always roll the same numbers (deterministic dice).
- **Board validation** — rejecting a bad board before the game starts.
- **The game loop** — the turn-by-turn logic.

What is happening internally?

- **Entities** (the main classes):
- `Board` — holds the size and one `Map<Integer, Integer>` of jumps. Snakes and ladders both go into this one map, because both are just "if you land on cell X, move to cell Y".
- `Player` — just a name.
- `Game` — keeps a turn queue (whose turn is next) and each player's position.
- `Dice` — an interface with one method, `roll()`.
- **Validation** (checks when building the board):
- A snake must go down. A ladder must go up.
- Two jumps cannot start on the same cell.
- Every jump must stay inside the board.
- **Rules you must decide and state clearly:**
- You must land exactly on 100. If the roll takes you past 100 (overshoot), you stay where you are.
- Jumps can chain: a ladder may end on a snake's head. The code follows jumps in a loop until it reaches a cell with no jump.

How does it work?

Each turn goes like this:

1. Roll the dice.
2. Calculate `next = pos + roll`.
3. If `next` is past the last cell, the player stays in place.
4. Otherwise, follow any jumps (snake or ladder) from `next`.
5. If `next == size`, this player wins and the game ends.
6. Otherwise, put the player at the back of the turn queue and move to the next player.

Example

```java
// FILE: SnakeLadderDemo.java
import java.util.*;

public class SnakeLadderDemo {
    interface Dice { int roll(); }
    static class RandomDice implements Dice {
        private final Random rnd; private final int count;
        RandomDice(long seed, int count) { rnd = new Random(seed); this.count = count; }
        public int roll() { int s = 0; for (int i = 0; i < count; i++) s += rnd.nextInt(6) + 1; return s; }
```

```
    }
    record Player(String name) {}

    static class Board {
        final int size; final Map<Integer, Integer> jumps = new HashMap<>();   // snakes and ladders unified
        Board(int size) { this.size = size; }
        Board snake(int head, int tail) { check(head > tail, "snake must go down"); add(head, tail); return
this; }
        Board ladder(int bottom, int top) { check(top > bottom, "ladder must go up"); add(bottom, top);
return this; }
        private void add(int from, int to) {
            check(from > 1 && from < size && to >= 1 && to <= size, "out of board");
            check(!jumps.containsKey(from), "cell already has a jump");
            jumps.put(from, to);
        }
        int resolve(int pos) { while (jumps.containsKey(pos)) pos = jumps.get(pos); return pos; }
        private static void check(boolean ok, String msg) { if (!ok) throw new IllegalArgumentException(msg);
}
    }

    static class Game {
        private final Board board; private final Dice dice;
        private final Deque<Player> turns; private final Map<Player, Integer> pos = new HashMap<>();
        Game(Board b, Dice d, List<Player> players) { board = b; dice = d; turns = new ArrayDeque<>(players);
players.forEach(p -> pos.put(p, 0)); }

        Player play() {
            while (true) {
                Player p = turns.pollFirst();
                int roll = dice.roll(), cur = pos.get(p), next = cur + roll;
                if (next <= board.size) next = board.resolve(next); else next = cur;   // exact landing rule
                pos.put(p, next);
                System.out.printf("%s rolled %d: %d -> %d%n", p.name(), roll, cur, next);
                if (next == board.size) return p;
                turns.addLast(p);
            }
        }
    }

    public static void main(String[] args) {
        Board b = new Board(100).ladder(4, 56).ladder(12, 50).ladder(20, 98).snake(99, 7).snake(54,
19).snake(70, 55);
        Player winner = new Game(b, new RandomDice(42, 1), List.of(new Player("Asha"), new
Player("Ravi"))).play();
        System.out.println("Winner: " + winner.name());
    }
}
```

Trace: the dice use a fixed seed (`new Random(42)`). A seed is the starting number for the random generator. With the same seed, you get the same "random" numbers every time. So every run prints the same game.

- Each line shows the roll and the move.
- Ladders lift a player up: 4→56, 12→50, 20→98.
- Snakes drop a player down: 99→7, 54→19, 70→55.
- If a roll would take a player past 100, they stay in place. They win only when they roll the exact number needed.

When would I use it?

- Any turn-based game, like Ludo or other board games.
- As practice for entity modelling (choosing classes and their fields).
- As practice for making randomness testable — random in production, fixed in tests.

What can go wrong?

- **Two separate maps for snakes and ladders.** The logic gets duplicated. One `jumps` map is enough.
- **Real randomness in tests.** Tests become flaky — they pass sometimes and fail sometimes. Inject a deterministic `Dice` instead.
- **Infinite jump loops.** If jumps form a cycle (A→B, B→A), `resolve` never ends. Validate that no chain of jumps forms a cycle.
- **Game state shared across rooms.** If many games run at once, two rooms could change the same positions. Create one `Game` object per room.

Interview questions

Q1. How do you make the game testable? I pass a `Dice` strategy into the game instead of creating dice inside it. In production it is random; in tests I pass dice that return a fixed sequence of rolls, so every test gives the same result.

Q2. How would you add "roll again on a 6, three 6s forfeits the turn"? I would put this rule in the turn loop as a separate rule object, say `TurnPolicy`. The game asks the policy what happens after each roll. Then each rule variant is its own class, and you can plug in a different one without changing the game loop.

10.14 In-Memory Pub-Sub (a mini Kafka)

Why?

> **In simple words:** You build a tiny Kafka in one program. Publishers add messages to the end of a list, and each subscriber reads that list at its own speed, remembering how far it has reached.

This problem tests how well you can design a message broker. A *broker* is the middleman that takes messages from publishers and hands them to subscribers.

It checks these ideas:

- **Topics** — named channels, like "orders".
- **Ordered logs** — messages kept in the order they arrived.
- **Independent subscribers** — each one tracks its own position, called an *offset*.
- **At-least-once delivery** — every message is delivered, but sometimes more than once.
- **Back-pressure** — a slow reader must not overload or slow down the system.
- **Thread safety** — many threads reading and writing at the same time without corrupting data.

It connects directly to the Kafka discussion in 11.13.

What is happening internally?

- **Topic** = a `List<Message>` that you only add to at the end. This is called an *append-only log*. A message's offset is simply its index in the list (0, 1, 2...).
- **Subscription** = four things together: the topic, its current offset, a handler (the code that processes a message), and a worker thread.
- **Pull-based delivery.** Each subscriber pulls (reads) messages at its own pace. The broker does not push to them. So a slow subscriber never blocks the others.
- `Topic.read(offset, timeout)` waits until a message exists at that offset. It uses Java's `wait` / `notifyAll`: the reader sleeps, and a new message wakes it up.

- **Delivery rule:** the offset moves forward only after the handler succeeds. If the handler fails, the same offset is tried again. So no message is skipped — this gives **at-least-once** delivery.
- **Replay:** to process old messages again, just set a subscription's offset back to an earlier number.

Think of a railway announcement register. The clerk writes each new announcement on the next line. Each department keeps its own bookmark showing which line it has read up to. A slow department doesn't stop anyone else.

How does it work?

1. `publish(topic, key, body)`:
2. Take the topic's lock.
3. Add the message at the end of the log.
4. Call `notifyAll` to wake any readers waiting for a new message.
5. Each subscription's worker thread runs a loop:
6. Read the message at its current offset. If there is none yet, wait.
7. Run the handler on the message.
8. If the handler succeeds, move the offset forward by one.
9. If the handler throws an exception, sleep briefly and retry the same offset.
10. `close()` stops all the worker threads.

Example

```
// FILE: PubSubDemo.java
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.Consumer;

public class PubSubDemo {
    record Message(long offset, String key, String body) {}

    static final class Topic {
        final String name; private final List<Message> log = new ArrayList<>();
        Topic(String name) { this.name = name; }
        synchronized long append(String key, String body) {
            long off = log.size(); log.add(new Message(off, key, body)); notifyAll(); return off;
        }
        /** Blocks until a message at `offset` exists or timeout. */
        synchronized Message read(long offset, long timeoutMs) throws InterruptedException {
            long deadline = System.currentTimeMillis() + timeoutMs;
            while (offset >= log.size()) {
                long wait = deadline - System.currentTimeMillis();
                if (wait <= 0) return null;
                wait(wait);
            }
            return log.get((int) offset);
        }
    }

    static final class Subscription implements Runnable {
        final String id; final Topic topic; final Consumer<Message> handler;
        final AtomicLong offset; volatile boolean running = true;
        Subscription(String id, Topic t, long startOffset, Consumer<Message> h) { this.id = id; topic = t;
offset = new AtomicLong(startOffset); handler = h; }
        public void run() {
            try {
                while (running) {
                    Message m = topic.read(offset.get(), 200);
                    if (m == null) continue;
                    try { handler.accept(m); offset.incrementAndGet(); }        // advance only after success
```

```
                    catch (RuntimeException e) { Thread.sleep(50); }            // retry same offset (at-
least-once)
                }
            } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        }
        void resetTo(long off) { offset.set(off); }                             // replay
    }

    static class Broker implements AutoCloseable {
        private final Map<String, Topic> topics = new ConcurrentHashMap<>();
        private final List<Subscription> subs = new CopyOnWriteArrayList<>();
        private final ExecutorService workers = Executors.newCachedThreadPool();
        void createTopic(String name) { topics.putIfAbsent(name, new Topic(name)); }
        long publish(String topic, String key, String body) { return get(topic).append(key, body); }
        Subscription subscribe(String subId, String topic, long fromOffset, Consumer<Message> handler) {
            Subscription s = new Subscription(subId, get(topic), fromOffset, handler);
            subs.add(s); workers.submit(s); return s;
        }
        private Topic get(String t) { return Optional.ofNullable(topics.get(t)).orElseThrow(() -> new
IllegalArgumentException("no topic " + t)); }
        public void close() throws InterruptedException { subs.forEach(s -> s.running = false);
workers.shutdown(); workers.awaitTermination(2, TimeUnit.SECONDS); }
    }

    public static void main(String[] args) throws Exception {
        try (Broker b = new Broker()) {
            b.createTopic("orders");
            List<String> fast = new CopyOnWriteArrayList<>(), slow = new CopyOnWriteArrayList<>();
            b.subscribe("loyalty", "orders", 0, m -> fast.add(m.body()));
            b.subscribe("analytics", "orders", 0, m -> { slow.add(m.body()); sleep(20); });
            for (int i = 1; i <= 5; i++) b.publish("orders", "outlet-" + i, "order-" + i);
            Thread.sleep(600);
            System.out.println("loyalty=" + fast + "\nanalytics=" + slow);
        }
    }
    static void sleep(long ms) { try { Thread.sleep(ms); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); } }
}
```

Trace: there are two subscribers. "loyalty" is fast. "analytics" is slow (it sleeps 20 ms per message). Both receive all 5 orders, in the correct order. Analytics simply finishes later — and it does not slow loyalty down at all.

When would I use it?

- To understand how Kafka, Pulsar and Redis Streams work inside.
- For an event bus inside one application (in-process), where parts of the code publish and listen to events.
- When designing notification fan-out — one event sent to many listeners.

What can go wrong?

- **Push-based delivery.** If the broker pushes each message to every subscriber, a slow subscriber can block the publisher.
- **Moving the offset before processing.** If the handler then fails, that message is skipped forever. This is *at-most-once* delivery, and it loses messages on failure.
- **A log that grows forever in memory.** Eventually you run out of memory. Add retention: drop old segments (chunks of the log) older than X.
- **Poison messages retried forever.** A *poison message* is one that always fails, for example because of bad data. It blocks the subscriber in an endless retry. Add a maximum attempt count, then move the message to a dead-letter topic (a side topic for failed messages).

Interview questions

Q1. How would you add partitions and ordering per key? Keep several logs per topic — each log is a partition. Choose the partition with `hash(key) % partitions`, so the same key always goes to the same partition. Ordering is then guaranteed within a partition, which means all messages for one key stay in order.

Q2. How do consumer groups work? Subscribers in one group share the partitions between them. Each partition is read by only one member of the group. The group stores one shared offset per partition. Different groups are fully independent — each group gets every message.

Q3. At-least-once vs at-most-once here? If you move the offset after processing, you get at-least-once: nothing is lost, but a crash can cause duplicates. If you move it before processing, you get at-most-once: no duplicates, but a crash can lose a message. Because duplicates can happen, consumers should be idempotent — processing the same message twice must give the same result as processing it once.

10.15 Library Management System

Why?

> **In simple words:** A library has one *book* (title, author) but many physical *copies* of it on the shelf. Keep those as two separate classes, and the rest — borrowing, returning, fines, waiting lists — becomes simple.

This is a classic first-round LLD problem. (LLD = low-level design: classes, methods and how they work together.)

The main thing interviewers look for: do you separate **Book** from **BookCopy**?

- **Book** = catalogue details (metadata) — ISBN, title, author.
- **BookCopy** = one physical item, with its own status.

It also tests:

- **Loans** and loan **limits** per member.
- **Fines**, using the Strategy pattern so the fine rule can change.
- **Reservations** — a waiting queue, plus an Observer-style notification when a book comes back.
- **Search** by title or author.

What is happening internally?

- **Entities:**
- `Book` — ISBN, title, author.
- `BookCopy` — id, status (AVAILABLE / LOANED / ON_HOLD), and which member it is held for.
- `Member` — includes the maximum number of loans allowed.
- `Loan` — the copy, the member, the due date, and the returned date.
- **Collections** (how data is stored):
- Copies grouped by ISBN.
- A reservation queue per ISBN (`Deque<Member>`) — first come, first served.
- Active loans per member.
- **Patterns used:**

- **Strategy** — `FinePolicy`, so the fine rule can be swapped.
- **Observer-like** — a `Notifier` that tells the next member when a book is returned.
- **Facade** — one service class (`LibraryService`) that hides all the details behind simple methods.
- **Injected Clock** — the service asks a `Clock` for "today", so tests can move time forward.

How does it work?

1. **Borrow:**
2. Check the member has not hit their loan limit.
3. Find a copy that is AVAILABLE, or one ON_HOLD for this same member.
4. Mark the copy LOANED.
5. Create a loan due in 14 days.
6. **Reserve:** if no copy is available, add the member to the end of the book's waiting queue.
7. **Return:**
8. Close the loan (set the returned date).
9. If someone is waiting, hold the copy for the first person in the queue and notify them.
10. If nobody is waiting, mark the copy AVAILABLE.
11. Calculate the fine by comparing the due date with the return date.

Example

```
// FILE: LibraryDemo.java
import java.time.*;
import java.util.*;
import java.util.concurrent.atomic.AtomicLong;

public class LibraryDemo {
    record Book(String isbn, String title, String author) {}
    enum CopyStatus { AVAILABLE, LOANED, ON_HOLD }
    static final class BookCopy { final String id; final Book book; CopyStatus status = CopyStatus.AVAILABLE;
String heldFor;
        BookCopy(String id, Book b) { this.id = id; this.book = b; } }
    record Member(String id, String name, int maxLoans) {}
    static final class Loan { final long id; final BookCopy copy; final Member member; final LocalDate due;
LocalDate returned;
        Loan(long id, BookCopy c, Member m, LocalDate due) { this.id = id; copy = c; member = m; this.due =
due; } }

    interface FinePolicy { long fineRupees(LocalDate due, LocalDate returned); }
    static class PerDayFine implements FinePolicy {
        private final long perDay, cap; PerDayFine(long perDay, long cap) { this.perDay = perDay; this.cap =
cap; }
        public long fineRupees(LocalDate due, LocalDate ret) { long late = Math.max(0,
Duration.between(due.atStartOfDay(), ret.atStartOfDay()).toDays()); return Math.min(cap, late * perDay); }
    }
    interface Notifier { void notify(Member m, String msg); }

    static class LibraryService {
        private final Map<String, Book> books = new HashMap<>();
        private final Map<String, List<BookCopy>> copiesByIsbn = new HashMap<>();
        private final Map<String, Deque<Member>> reservations = new HashMap<>();
        private final Map<String, List<Loan>> activeLoans = new HashMap<>();         // memberId -> loans
        private final Map<String, Member> members = new HashMap<>();
        private final AtomicLong seq = new AtomicLong();
        private final FinePolicy fines; private final Notifier notifier; private final Clock clock;
        private final Period loanPeriod = Period.ofDays(14);
        LibraryService(FinePolicy f, Notifier n, Clock c) { fines = f; notifier = n; clock = c; }

        void addMember(Member m) { members.put(m.id(), m); }
        void addCopies(Book b, int n) {
            books.putIfAbsent(b.isbn(), b);
```

```
            for (int i = 0; i < n; i++) copiesByIsbn.computeIfAbsent(b.isbn(), k -> new ArrayList<>
()).add(new BookCopy(b.isbn() + "-" + (i + 1), b));
        }
        List<Book> search(String q) {
            String s = q.toLowerCase();
            return books.values().stream().filter(b -> b.title().toLowerCase().contains(s) ||
b.author().toLowerCase().contains(s)).toList();
        }

        synchronized Loan borrow(String memberId, String isbn) {
            Member m = Objects.requireNonNull(members.get(memberId), "no member");
            List<Loan> loans = activeLoans.computeIfAbsent(memberId, k -> new ArrayList<>());
            if (loans.size() >= m.maxLoans()) throw new IllegalStateException("loan limit reached");
            BookCopy copy = copiesByIsbn.getOrDefault(isbn, List.of()).stream()
                    .filter(c -> c.status == CopyStatus.AVAILABLE || (c.status == CopyStatus.ON_HOLD &&
memberId.equals(c.heldFor)))
                    .findFirst().orElseThrow(() -> new IllegalStateException("no copy available; reserve
instead"));
            copy.status = CopyStatus.LOANED; copy.heldFor = null;
            Loan loan = new Loan(seq.incrementAndGet(), copy, m, LocalDate.now(clock).plus(loanPeriod));
            loans.add(loan);
            return loan;
        }

        synchronized void reserve(String memberId, String isbn) {
            reservations.computeIfAbsent(isbn, k -> new ArrayDeque<>()).addLast(members.get(memberId));
        }

        /** @return fine in rupees */
        synchronized long giveBack(long loanId) {
            Loan loan = activeLoans.values().stream().flatMap(List::stream).filter(l -> l.id ==
loanId).findFirst()
                    .orElseThrow(() -> new IllegalArgumentException("unknown loan"));
            loan.returned = LocalDate.now(clock);
            activeLoans.get(loan.member.id()).remove(loan);
            BookCopy copy = loan.copy;
            Deque<Member> queue = reservations.getOrDefault(copy.book.isbn(), new ArrayDeque<>());
            Member next = queue.pollFirst();
            if (next != null) {                                    // hold for the first reserver (Observer-
style notify)
                copy.status = CopyStatus.ON_HOLD; copy.heldFor = next.id();
                notifier.notify(next, "'" + copy.book.title() + "' is ready for pickup");
            } else copy.status = CopyStatus.AVAILABLE;
            return fines.fineRupees(loan.due, loan.returned);
        }
    }

    static class MutableClock extends Clock {
        Instant now = Instant.parse("2026-10-05T10:00:00Z");
        public ZoneId getZone() { return ZoneOffset.UTC; } public Clock withZone(ZoneId z) { return this; }
public Instant instant() { return now; }
    }

    public static void main(String[] args) {
        MutableClock clock = new MutableClock();
        LibraryService lib = new LibraryService(new PerDayFine(5, 200), (m, msg) ->
System.out.println("notify " + m.name() + ": " + msg), clock);
        lib.addMember(new Member("m1", "Asha", 3)); lib.addMember(new Member("m2", "Ravi", 3));
        Book ddia = new Book("978-1449373320", "Designing Data-Intensive Applications", "Kleppmann");
        lib.addCopies(ddia, 1);
        System.out.println(lib.search("kleppmann").size());                    // 1
        Loan l = lib.borrow("m1", ddia.isbn());
        try { lib.borrow("m2", ddia.isbn()); } catch (IllegalStateException e) {
System.out.println(e.getMessage()); }
        lib.reserve("m2", ddia.isbn());
        clock.now = clock.now.plus(Duration.ofDays(17));                       // 3 days late
        System.out.println("fine=" + lib.giveBack(l.id));                      // 15
        System.out.println("m2 got loan " + lib.borrow("m2", ddia.isbn()).copy.id);
    }
}
```

Trace:

1. Asha borrows the only copy.
2. Ravi tries to borrow it and fails, so he reserves it.
3. Asha returns it 3 days late. At ₹5 per day, her fine is ₹15.
4. The copy is held for Ravi, and he is notified.
5. Ravi then borrows it.

When would I use it?

The Book/BookCopy split (shared details vs individual physical items) appears in many systems:

- Inventory — SKU (the product type) vs a serialised item (one unit with its own serial number).
- Rentals and equipment tracking.
- Ticketing — the event vs each seat.

What can go wrong?

- **One class for both the book and the copy.** Then you cannot track multiple copies of the same book.
- **Holds that never expire.** If the reserving member never comes, the copy stays ON_HOLD forever and nobody can borrow it.
- **Allowing renewals while others are waiting.** One member can keep the book forever while the queue waits.
- **A global `synchronized` service at scale.** Every borrow and return waits for one lock, so the system gets slow. Use locking per ISBN, or database constraints, instead.

Interview questions

Q1. Why separate Book from BookCopy? All copies share the same details — title, author, ISBN — so those live once in `Book`. But each physical copy has its own status, location and loan history, so that lives in `BookCopy`. With one class you could not track several copies of the same title.

Q2. How do reservations work? Each book has a FIFO (first-in, first-out) queue of members. When a copy is returned, it is held for the first member in the queue, and that member is notified. If they don't collect it before the hold expires, it passes to the next member in the queue.

Q3. How would you search by title or author at scale? I would not scan every book. I would build an inverted index — a map from each word to the books that contain it — over the tokenised (split into words) titles and authors. In production, Elasticsearch does this for you.

10.16 Coffee Machine

Why?

> **In simple words:** A drink needs several ingredients. The machine must either take all of them or take none — never use the milk and then find out there's no coffee.

This is a popular machine-coding problem. It has come up in several Indian product-company rounds.

It tests four things:

- **Data-driven recipes** — each drink is described by data (a list of ingredients), not by its own class.

- **Atomic all-or-nothing deduction** — removing several ingredients as one step. *Atomic* means it either fully happens or doesn't happen at all.
- **Parallel outlets** — several drinks made at the same time.
- **Low-stock alerts** — warnings when an ingredient runs low.

What is happening internally?

- **Recipes are data.** A recipe is `Recipe(name, Map<Ingredient, qty>)`. So adding a new drink means adding a new entry — configuration, not code.
- **Inventory.** Stock is kept in an `EnumMap<Ingredient, qty>`. Each ingredient can have a low-stock threshold. Listeners get told when it goes below that mark (Observer pattern).
- **Atomic deduction.** Inside one lock, the code does two passes: 1. Check *every* ingredient first. 2. Only if all are enough, deduct *every* ingredient.

So a drink never uses up milk and then fails because coffee is short. - **Outlets.** A thread pool with N threads, so N drinks brew in parallel. - **Add-ons** (extra shot, oat milk) use the *Decorator* pattern. A decorator wraps a base recipe and returns a new recipe with extra ingredients.

Think of a tiffin service packing a thali. The cook first checks there is enough dal, rice and sabzi. Only then does he start filling the box. He doesn't fill dal and then discover the rice is finished.

How does it work?

1. `order(recipe)` sends the order to an outlet thread.
2. That thread calls `tryConsume(needs)`, which:
3. Takes the lock.
4. Collects every ingredient that is short.
5. If anything is short, returns failure. Nothing has been deducted.
6. Otherwise, deducts all ingredients and notes which ones crossed their low mark.
7. Releases the lock.
8. Low-stock listeners are notified *outside* the lock, so a slow listener doesn't block other drinks.
9. The drink is brewed.

Example

```
// FILE: CoffeeMachineDemo.java
import java.util.*;
import java.util.concurrent.*;
import java.util.function.Consumer;

public class CoffeeMachineDemo {
    enum Ingredient { WATER, MILK, COFFEE, SUGAR, FOAM }
    record Recipe(String name, Map<Ingredient, Integer> needs) {}

    /** Decorator for add-ons: wraps a recipe and adds ingredients. */
    static Recipe withAddOn(Recipe base, String addOn, Ingredient i, int qty) {
        Map<Ingredient, Integer> m = new EnumMap<>(base.needs());
        m.merge(i, qty, Integer::sum);
        return new Recipe(base.name() + " + " + addOn, m);
    }

    static class Inventory {
        private final Map<Ingredient, Integer> stock = new EnumMap<>(Ingredient.class);
        private final Map<Ingredient, Integer> lowMark = new EnumMap<>(Ingredient.class);
        private final List<Consumer<Ingredient>> lowStockListeners = new CopyOnWriteArrayList<>();
        void refill(Ingredient i, int qty) { synchronized (this) { stock.merge(i, qty, Integer::sum); } }
        void setLowMark(Ingredient i, int qty) { lowMark.put(i, qty); }
```

```
        void onLowStock(Consumer<Ingredient> l) { lowStockListeners.add(l); }

        /** All-or-nothing deduction: check every ingredient, then deduct every ingredient, under one lock.
*/
        boolean tryConsume(Map<Ingredient, Integer> needs, List<Ingredient> missing) {
            List<Ingredient> low = new ArrayList<>();
            synchronized (this) {
                needs.forEach((i, q) -> { if (stock.getOrDefault(i, 0) < q) missing.add(i); });
                if (!missing.isEmpty()) return false;
                needs.forEach((i, q) -> {
                    int left = stock.merge(i, -q, Integer::sum);
                    if (left <= lowMark.getOrDefault(i, -1)) low.add(i);
                });
            }
            low.forEach(i -> lowStockListeners.forEach(l -> l.accept(i)));   // notify outside the lock
            return true;
        }
        synchronized int level(Ingredient i) { return stock.getOrDefault(i, 0); }
    }

    static class CoffeeMachine implements AutoCloseable {
        private final Inventory inventory; private final Map<String, Recipe> menu = new LinkedHashMap<>();
        private final ExecutorService outlets;
        CoffeeMachine(int outletCount, Inventory inv) { this.outlets =
Executors.newFixedThreadPool(outletCount); this.inventory = inv; }
        void addRecipe(Recipe r) { menu.put(r.name(), r); }

        Future<String> order(Recipe r) {
            return outlets.submit(() -> {
                List<Ingredient> missing = new ArrayList<>();
                if (!inventory.tryConsume(r.needs(), missing)) return r.name() + " cannot be prepared:
insufficient " + missing;
                Thread.sleep(50);                                          // brewing
                return r.name() + " is prepared";
            });
        }
        Future<String> order(String name) { return order(Objects.requireNonNull(menu.get(name), "unknown
beverage")); }
        public void close() { outlets.shutdown(); }
    }

    public static void main(String[] args) throws Exception {
        Inventory inv = new Inventory();
        inv.refill(Ingredient.WATER, 500); inv.refill(Ingredient.MILK, 300); inv.refill(Ingredient.COFFEE,
60);
        inv.refill(Ingredient.SUGAR, 50); inv.refill(Ingredient.FOAM, 40);
        inv.setLowMark(Ingredient.COFFEE, 20);
        inv.onLowStock(i -> System.out.println("ALERT: low " + i));

        try (CoffeeMachine m = new CoffeeMachine(3, inv)) {
            Recipe latte = new Recipe("latte", Map.of(Ingredient.WATER, 50, Ingredient.MILK, 150,
Ingredient.COFFEE, 20));
            m.addRecipe(new Recipe("espresso", Map.of(Ingredient.WATER, 30, Ingredient.COFFEE, 20)));
            m.addRecipe(latte);
            m.addRecipe(new Recipe("cappuccino", Map.of(Ingredient.WATER, 50, Ingredient.MILK, 100,
Ingredient.COFFEE, 20, Ingredient.FOAM, 30)));
            List<Future<String>> results = List.of(m.order("latte"), m.order("cappuccino"),
m.order("espresso"),
                                                   m.order(withAddOn(latte, "extra shot", Ingredient.COFFEE,
10)));
            for (Future<String> f : results) System.out.println(f.get());
            System.out.println("coffee left=" + inv.level(Ingredient.COFFEE) + ", milk left=" +
inv.level(Ingredient.MILK));
        }
    }
}
```

Trace: the machine starts with 60 units of coffee.

- Latte, cappuccino and espresso each take 20 coffee.

- The extra-shot latte needs 30 coffee and 150 milk. There isn't enough of either, so it fails cleanly with "insufficient [MILK, COFFEE]". Nothing is deducted for it.
- The low-coffee alert fires once coffee drops to the threshold of 20.

When would I use it?

Any time you must reserve several resources together, all-or-nothing:

- An order that reserves several SKUs (products) at once.
- A booking that holds several seats together.
- A job that needs several resources before it can start.

What can go wrong?

- **Checking and deducting one ingredient at a time.** If the third ingredient is short, the first two are already used up. This is partial consumption.
- **Per-ingredient locks taken in different orders.** Thread A locks milk then waits for coffee; thread B locks coffee then waits for milk. Both wait forever — a *deadlock*. If you use one lock per ingredient (fine-grained locks), always lock them in enum order.
- **Notifying listeners while holding the lock.** A slow listener then blocks all brewing.
- **A hard-coded class for every beverage** instead of recipe data. Every new drink then needs new code.

Interview questions

Q1. How do you guarantee a drink never partially consumes ingredients? I check all the ingredients and deduct all of them inside one critical section (one locked block). If any check fails, nothing is deducted. The alternative is one lock per ingredient, but then I must always take the locks in the same fixed order to avoid deadlock.

Q2. How would you support N outlets? I use a fixed thread pool of N workers, or a semaphore with N permits. Orders are processed in parallel by these workers. The inventory is the one shared resource, and it is synchronised so only one thread changes the stock at a time.

Q3. How would you add a new beverage or an add-on? A new beverage is just a new recipe entry — data, not code. An add-on is a decorator: it takes a base recipe and returns a new recipe with the extra ingredient quantities added.

10.17 Cab Booking (Uber/Ola)

Why?

> **In simple words:** When a rider books, you find nearby free drivers, offer the ride to the best one, and move to the next if they say no. The hard part is making sure two riders booking at the same moment never get the same driver.

This is a high-signal problem for SDE2 — it shows a lot about your skill in one question. It combines:

- **Spatial search** — finding the nearest drivers on a map.
- **Matching** — offering the ride to drivers, who can accept or decline.
- **A trip state machine** — a trip moves through fixed stages in a fixed order.
- **Pricing strategies** — fare rules, including surge pricing.

- **A real concurrency hazard** — the same driver being assigned to two riders at once.

What is happening internally?

- **Entities:**
- `Rider`.
- `Driver` — location, vehicle type, and status stored in an `AtomicReference<DriverStatus>`.
- `Trip` — a state machine: REQUESTED → ACCEPTED → STARTED → COMPLETED / CANCELLED.
- `Location`.
- **Spatial index** (a structure for fast "who is near here?" lookups):
- The map is cut into a uniform grid of square cells: `cell = floor(coord / cellKm)`.
- To find nearby drivers, scan the rider's cell, then the ring of cells around it, and so on, up to the radius.
- At real scale you would use geohash, H3 or quadtrees instead — see 13.13.
- **Strategies** (swappable rules):
- `MatchingStrategy` — how to rank drivers: nearest first, weighted by rating, and so on.
- `PricingStrategy` — base fare + per km + per minute, all multiplied by surge.
- **Concurrency:**
- `driver.tryReserve()` calls `compareAndSet(AVAILABLE, ON_TRIP)`. This is CAS (compare-and-set): "change the status to ON_TRIP only if it is still AVAILABLE", done as one atomic step.
- So only one rider can claim a driver, and you don't need a global lock.
- `DriverGateway.offer` stands in for the driver's app, where the driver accepts or declines.

Think of a restaurant with a token board. Two waiters may both see table 5 is free. But only the one who flips the "occupied" sign first gets it. The other sees the sign and goes to the next table.

How does it work?

1. `request(rider, from, to, type)`:
2. Try search radii of 1, 3 and 5 km, in that order.
3. For each radius, get the candidate drivers near `from`, and rank them with the matching strategy.
4. For each ranked driver, try to reserve them with CAS. If that fails, someone else got them — skip to the next.
5. If reserved, send the offer to the driver.
6. If the driver accepts, the trip becomes ACCEPTED and we return it.
7. If the driver declines, release them and try the next driver.
8. If no driver accepts in any radius, the trip becomes CANCELLED.
9. `start` and `complete` move the trip forward through guarded state transitions. *Guarded* means the move is allowed only from the correct previous state.
10. `complete` also:
11. Calculates the fare.
12. Moves the driver to the drop location in the spatial index.
13. Releases the driver so they are AVAILABLE again.

Example

```
// FILE: CabBookingDemo.java
import java.util.*;
```

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

public class CabBookingDemo {
    record Location(double x, double y) { double dist(Location o) { return Math.hypot(x - o.x, y - o.y); } }
// km on a flat grid for the demo
    enum VehicleType { AUTO, MINI, SEDAN }
    enum DriverStatus { AVAILABLE, ON_TRIP, OFFLINE }
    enum TripStatus { REQUESTED, ACCEPTED, STARTED, COMPLETED, CANCELLED }

    static final class Driver {
        final String id; final VehicleType type; volatile Location loc;
        final AtomicReference<DriverStatus> status = new AtomicReference<>(DriverStatus.AVAILABLE);
        Driver(String id, VehicleType t, Location l) { this.id = id; type = t; loc = l; }
        boolean tryReserve() { return status.compareAndSet(DriverStatus.AVAILABLE, DriverStatus.ON_TRIP); }
// atomic: no double booking
        void release() { status.set(DriverStatus.AVAILABLE); }
    }
    record Rider(String id, String name) {}

    static final class Trip {
        final String id; final Rider rider; final Location from, to; final VehicleType type;
        volatile Driver driver; volatile TripStatus status = TripStatus.REQUESTED; volatile long fareRupees;
        Trip(String id, Rider r, Location from, Location to, VehicleType t) { this.id = id; rider = r;
this.from = from; this.to = to; type = t; }
        synchronized void transition(TripStatus from, TripStatus to) {          // State machine guard
            if (status != from) throw new IllegalStateException("trip " + id + " is " + status + ", not " +
from);
            status = to;
        }
    }

    /** Uniform grid spatial index: cell = floor(coord / cellKm). Nearby search scans rings of cells. */
    static class DriverIndex {
        private final double cellKm; private final ConcurrentHashMap<Long, Set<Driver>> cells = new
ConcurrentHashMap<>();
        private final ConcurrentHashMap<String, Long> cellOf = new ConcurrentHashMap<>();
        DriverIndex(double cellKm) { this.cellKm = cellKm; }
        private long key(int cx, int cy) { return (((long) cx) << 32) ^ (cy & 0xffffffffL); }
        private int c(double v) { return (int) Math.floor(v / cellKm); }
        void update(Driver d, Location l) {
            long k = key(c(l.x()), c(l.y()));
            Long old = cellOf.put(d.id, k);
            if (old != null && old != k) cells.getOrDefault(old, Set.of()).remove(d);
            cells.computeIfAbsent(k, x -> ConcurrentHashMap.newKeySet()).add(d);
            d.loc = l;
        }
        List<Driver> nearby(Location l, double radiusKm) {
            int r = (int) Math.ceil(radiusKm / cellKm), cx = c(l.x()), cy = c(l.y());
            List<Driver> out = new ArrayList<>();
            for (int dx = -r; dx <= r; dx++) for (int dy = -r; dy <= r; dy++)
                for (Driver d : cells.getOrDefault(key(cx + dx, cy + dy), Set.of())) if (d.loc.dist(l) <=
radiusKm) out.add(d);
            return out;
        }
    }

    interface MatchingStrategy { List<Driver> rank(List<Driver> candidates, Trip t); }
    static class NearestFirst implements MatchingStrategy {
        public List<Driver> rank(List<Driver> c, Trip t) {
            return c.stream().filter(d -> d.type == t.type && d.status.get() == DriverStatus.AVAILABLE)
                    .sorted(Comparator.comparingDouble(d -> d.loc.dist(t.from))).toList();
        }
    }
    interface PricingStrategy { long fare(Trip t, double km, double minutes); }
    static class DistanceTimeSurge implements PricingStrategy {
        private final Map<VehicleType, double[]> rates = Map.of(VehicleType.AUTO, new double[]{25, 12, 1},
                VehicleType.MINI, new double[]{40, 14, 1.5}, VehicleType.SEDAN, new double[]{60, 18, 2});  //
base, per km, per min
        private final double surge; DistanceTimeSurge(double surge) { this.surge = surge; }
        public long fare(Trip t, double km, double min) { double[] r = rates.get(t.type); return
Math.round((r[0] + r[1] * km + r[2] * min) * surge); }
```

```
    }
    interface DriverGateway { boolean offer(Driver d, Trip t); }   // driver app accept/decline (with timeout
in reality)

    static class RideService {
        private final DriverIndex index; private final MatchingStrategy matcher; private final
PricingStrategy pricing; private final DriverGateway gateway;
        private final Map<String, Trip> trips = new ConcurrentHashMap<>(); private final AtomicLong seq = new
AtomicLong();
        RideService(DriverIndex i, MatchingStrategy m, PricingStrategy p, DriverGateway g) { index = i;
matcher = m; pricing = p; gateway = g; }

        Trip request(Rider r, Location from, Location to, VehicleType type) {
            Trip t = new Trip("TR" + seq.incrementAndGet(), r, from, to, type);
            trips.put(t.id, t);
            for (double radius : new double[]{1, 3, 5}) {                       // expanding search radius
                for (Driver d : matcher.rank(index.nearby(from, radius), t)) {
                    if (!d.tryReserve()) continue;                              // someone else got this
driver
                    if (gateway.offer(d, t)) { t.driver = d; t.transition(TripStatus.REQUESTED,
TripStatus.ACCEPTED); return t; }
                    d.release();                                                // declined: free the driver,
try next
                }
            }
            t.transition(TripStatus.REQUESTED, TripStatus.CANCELLED);
            return t;
        }
        void start(String tripId) { trips.get(tripId).transition(TripStatus.ACCEPTED, TripStatus.STARTED); }
        long complete(String tripId, double minutes) {
            Trip t = trips.get(tripId);
            t.transition(TripStatus.STARTED, TripStatus.COMPLETED);
            t.fareRupees = pricing.fare(t, t.from.dist(t.to), minutes);
            index.update(t.driver, t.to);
            t.driver.release();
            return t.fareRupees;
        }
    }

    public static void main(String[] args) throws Exception {
        DriverIndex idx = new DriverIndex(1.0);
        List<Driver> drivers = List.of(new Driver("d1", VehicleType.MINI, new Location(0.5, 0.5)),
                new Driver("d2", VehicleType.MINI, new Location(2.5, 0.2)), new Driver("d3",
VehicleType.SEDAN, new Location(0.2, 0.1)));
        drivers.forEach(d -> idx.update(d, d.loc));
        DriverGateway gw = (d, t) -> !d.id.equals("d1") || !t.rider.id().equals("r2");   // d1 declines r2
for the demo
        RideService svc = new RideService(idx, new NearestFirst(), new DistanceTimeSurge(1.2), gw);

        // Two riders request MINI at the same time: each must get a different driver
        ExecutorService pool = Executors.newFixedThreadPool(2);
        Future<Trip> a = pool.submit(() -> svc.request(new Rider("r1", "Asha"), new Location(0, 0), new
Location(6, 8), VehicleType.MINI));
        Future<Trip> b = pool.submit(() -> svc.request(new Rider("r2", "Ravi"), new Location(0, 0), new
Location(3, 4), VehicleType.MINI));
        Trip t1 = a.get(), t2 = b.get(); pool.shutdown();
        System.out.println(t1.rider.name() + " -> " + (t1.driver == null ? "none" : t1.driver.id) + " " +
t1.status);
        System.out.println(t2.rider.name() + " -> " + (t2.driver == null ? "none" : t2.driver.id) + " " +
t2.status);
        Trip done = t1.driver != null ? t1 : t2;
        svc.start(done.id);
        System.out.println("fare=" + svc.complete(done.id, 25));
    }
}
```

Trace:

1. Two riders request a MINI at the same instant, from the same spot.
2. Both rank driver d1 first, because d1 is closest.

3. Only one CAS on d1 succeeds. The other rider skips to d2.
4. d1 is scripted to decline rider r2. This shows the release-and-retry path.
5. The completed ride is 10 km and 25 minutes. Fare = (40 + 14×10 + 1.5×25) × 1.2 surge = ₹261.

When would I use it?

Any dispatch and matching problem:

- Ride-hailing.
- Assigning food-delivery partners (13.18).
- Assigning tasks to field-force staff.
- Technician scheduling.
- Geo-indexed search in general — any "find things near this point" problem.

What can go wrong?

- **A check-then-assign race.** Two threads both check "is d1 free?", both see yes, and both assign. Now two riders have the same driver. This happens when there's no atomic reservation.
- **Scanning every driver for every request.** This is far too slow with many drivers. Use a spatial index.
- **Offers without timeouts.** If a driver never answers, the request waits forever.
- **Forgetting to release drivers who decline.** They stay ON_TRIP and never get new rides.
- **Invalid trip transitions.** For example, completing a trip that was already cancelled. Guard every state change.

Interview questions

Q1. How do you prevent assigning one driver to two riders? Before offering the ride, I change the driver's status atomically. In memory, that is a CAS from AVAILABLE to ON_TRIP. In a distributed system, it is a conditional UPDATE in the database (update only if status is still AVAILABLE) or a Redis SET NX (set only if the key doesn't exist). Only one request can win.

Q2. How do you find nearby drivers efficiently? I use a spatial index — a grid, geohash, H3 or quadtree — keyed by map cell. I search the rider's own cell first, then the rings of neighbouring cells, until I have enough candidates.

Q3. How would surge pricing work? Every few minutes, I calculate the demand-to-supply ratio (riders requesting vs drivers free) for each area cell. Based on that ratio, I pick a multiplier and apply it through the PricingStrategy.

10.18 More problems to practise

> **In simple words:** Practise the problems below with the same seven-step method. Each one tests a different pattern, shown in brackets.

- Food delivery order assignment.
- Hotel booking.
- Chess — move validation for each piece (Strategy).
- ATM (State).
- Online auction — bids arriving at the same time (concurrency).

- File system (Composite).
- Notification service with templates and channels.
- Cron scheduler — a PriorityQueue of next-run times.
- Inventory with reservations.
- In-memory URL shortener.
- Coupon/discount engine (Specification + Strategy).
- Meeting room scheduler — a TreeMap of time intervals.
- Versioned document editor (Memento + Command).

Part 4 — High-Level Design

System design often decides your level in an interview. At SDE2, you are expected to run a 45–60 minute design discussion yourself, from start to finish. You must talk about trade-offs out loud — what you gain and what you give up with each choice. You must also go deep on two components.

This part has three chapters: - **Chapter 11** — the building blocks (load balancers, caches, databases, replication and so on). - **Chapter 12** — how databases work inside. - **Chapter 13** — using all of this to design 19 real systems.

11. HLD Fundamentals

11.1 The system design interview framework

Why?

> **In simple words:** A system design interview is like a vague order — "design a URL shortener". A fixed set of steps helps you ask the right questions, use numbers, and finish on time instead of drawing random boxes.

System design questions are kept vague on purpose. For example: "design a URL shortener".

The strong candidates are not the ones who know the most technologies. They are the ones who: - give the conversation a clear structure, - put numbers on the problem (users, requests per second, data size), and - explain why they chose one option over another (trade-offs).

Without a framework, you jump straight to drawing boxes. You miss requirements. Then you run out of time before you reach the interesting parts.

What is happening internally?

While you talk, the interviewer is checking seven things:

1. **Requirement gathering** — do you ask questions to fix the scope?
2. **Estimation** — do you reason with numbers?
3. **Design** — are your components and data flows sensible?
4. **Depth** — can you go deep on the hard parts?
5. **Trade-offs** — do you know the other options and what they cost?
6. **Operability** — do you think about failures, scaling and monitoring?
7. **Communication** — can they follow your thinking easily?

How does it work?

Use this time plan for a 60-minute round. QPS means *queries per second* — how many requests hit the system each second. DAU means *daily active users*.

Minutes	Step	What goes on the board
0–5	Functional requirements	3–5 main use cases; clearly list what is out of scope
5–8	Non-functional requirements	DAU, QPS, read/write ratio, latency targets, availability, consistency, durability (data must not be lost), data size, compliance
8–12	Estimates	Average and peak QPS, storage per year, bandwidth, cache size
12–16	API	3–6 endpoints or events with their key fields
16–20	Data model	Entities, keys, access patterns (how data is read and written) → choice of store with a one-line reason
20–30	High-level architecture	Clients → edge/gateway → services → caches/queues → stores; walk one read and one write through it

Minutes	Step	What goes on the board
30–50	Deep dives	2 hard components (you suggest them, the interviewer picks)
50–60	Bottlenecks and evolution	How it fails, what happens at 10× growth, monitoring, what you would do differently

Example

This is how you can open a URL shortener question:

> "Before designing, let me pin down scope. Users create short links and get redirected; optional custom aliases and expiry; basic click counts. Out of scope: user accounts UI and link editing. Non-functionally, redirects are latency-critical and must be highly available; creation can be slower. Reads dominate — roughly 100:1. Let me estimate..."

Notice what this does. In under a minute, you fixed the scope, said what is out, named the most important non-functional need (fast, always-up redirects) and gave a read/write ratio.

When would I use it?

In every HLD (high-level design) round. Also at work, in real design reviews and RFCs (written design proposals). The same structure makes a good design document.

What can go wrong?

- Drawing boxes before you know the requirements.
- Giving no numbers at all.
- Naming technologies without reasons — for example, just saying "use Kafka".
- Spending 30 minutes on the API.
- Going deep on something easy while ignoring the real bottleneck (the slowest, most loaded part).
- Silence — the interviewer cannot judge thinking they cannot hear.

Interview questions

Q1. How do you approach a system design question? First I clarify the functional and non-functional requirements and estimate the scale. Then I define the APIs and the data model and sketch the architecture. Next I go deep on the hardest components. Finally I discuss bottlenecks, failures and how the system grows — and I justify each choice with its trade-offs.

11.2 Back-of-envelope estimation

Why?

> **In simple words:** Quick rough maths tells you how big the problem really is. A small problem needs one server; a huge one needs many machines, caches and queues. The numbers pick the design for you.

Numbers decide the architecture. For example: - 50 QPS fits easily on one server with one Postgres database. - 500,000 QPS needs sharding (splitting data across many databases), caching and queues.

When you estimate, the interviewer sees that you are designing for *this* problem. You are not just repeating a standard architecture from memory.

What is happening internally?

First, know roughly how long common operations take. Only the order of size matters, not the exact number. (1 ms = 1,000 µs; 1 µs = 1,000 ns.)

Operation	Approximate time
L1 cache / main memory reference	1 ns / 100 ns
Read 1 MB sequentially from memory	~3 µs
SSD random read	~16–100 µs
Round trip within a datacentre	~0.5 ms
Redis GET over the network	~0.5–1 ms
Indexed DB lookup	~1–10 ms
Read 1 MB sequentially from SSD	~0.05–1 ms
HDD seek	~10 ms
India ↔ US round trip	~150–250 ms

The main lesson: memory is very fast, the network adds about a millisecond, the database adds a few milliseconds, and crossing continents costs hundreds of milliseconds.

Capacity rules of thumb (rough numbers you can use in any interview): - A day is 86,400 seconds. Round it to **10^5 s**. So **1 million requests/day ≈ 12 QPS**. - **Peak traffic is about 2–5× the average.** Use 3× unless told otherwise. - One app instance (4–8 CPU cores, running on the JVM): about 1,000–5,000 simple requests per second. - One PostgreSQL/MySQL primary (the main database server): about 5,000–20,000 simple queries per second. It is comfortable up to a few TB of data. - One Redis node: 100,000+ operations per second. Its limit is usually RAM, not speed. - One Kafka partition: about 10 MB/s of writes. A full Kafka cluster can handle millions of messages per second. - Powers of two: $2^{10} \approx 10^3$ (KB), $2^{20} \approx 10^6$ (MB), $2^{30} \approx 10^9$ (GB), $2^{40} \approx 10^{12}$ (TB).

How does it work?

Follow these four calculations:

1. **QPS:** users → actions per user per day → requests per day → ÷ 10^5 → average QPS → × 3 → peak QPS.
2. **Storage:** writes per day × bytes per record → storage per day → × 365 × number of years. Multiply again by the replication factor (how many copies you keep).
3. **Bandwidth:** read QPS × response size.
4. **Cache size:** hot set × object size. The *hot set* is the data that is read most often — often about 20% of daily reads.

Example

> NOVA-like ordering assistant: 10M daily active retailers × 5 API calls = 50M/day ÷ 10^5 = **500 QPS average, ~1,500 peak**. 2M orders/day × 1 KB = 2 GB/day ≈ **730 GB/year** → a single well-indexed Postgres primary handles it for a while; partition the orders table by month for retention. Catalogue reads are 10× higher (5,000 QPS) → cache SKU and price data in Redis; hot set ~200k SKUs × 2 KB ≈ 400 MB.

What this tells you: - 1,500 peak QPS and 730 GB/year are small enough for **one** good Postgres primary for some time. No sharding needed yet. - Splitting the orders table by month (partitioning) makes it easy to delete old data later (retention). - Catalogue reads are much heavier, so they go to a Redis cache. The hot data is only about 400 MB, so it fits easily in memory.

When would I use it?

- At the start of every design and every capacity plan.
- When deciding between "one database" and "shard it".
- When sizing caches, Kafka partitions and pods.

What can go wrong?

- **False precision.** Do not do exact maths. Spend about 3 minutes and round hard.
- **Forgetting peaks, replication and indexes.** Real storage is often 2–3× the raw data size.
- **Looking at the wrong bottleneck.** The limit is usually the database, not the app servers.

Interview questions

Q1. How many QPS is 100 million requests per day? About 1,150 on average (10^8 / 86,400). At a 3× peak, that is roughly 3,500.

Q2. Why estimate at all? Because the scale decides the architecture. It tells you whether one node is enough or you need sharding, how much caching you need, how much queue throughput you need, and what it will cost.

11.3 Scaling basics: vertical, horizontal and stateless services

Why?

> **In simple words:** When traffic grows, you either buy a bigger machine or add more machines. Adding machines works best — but only if any machine can handle any request, which means the machines must not keep user data in their own memory.

Traffic grows over time. You have two choices: - Buy a bigger machine. This is **vertical** scaling. - Add more machines. This is **horizontal** scaling.

Only horizontal scaling can keep growing without a hard limit. It also survives a machine failure, because the other machines keep working. But it needs a design where **any** instance can serve **any** request.

What is happening internally?

- **Vertical scaling** means more CPU and RAM on one box.

- Good: it is simple and needs no code changes.
- Bad: hardware has a maximum size, big machines get expensive, and that one box is still a single point of failure (if it dies, everything dies).
- **Horizontal scaling** means N identical instances behind a load balancer (a component that spreads requests across them).
- It needs **stateless** services. Stateless means the instance keeps no user session or request data in its own memory between requests.
- The state lives somewhere shared instead: a database, a cache, or a token the client carries (such as a JWT — a signed token).
- **The data tier is the hard part.** App servers are easy to scale — just add more. Databases are harder. They need replication (11.8), partitioning (11.9) and caching (11.5).

Think of a bank with many counters. If any clerk can serve you because your account is in the central system, you can open more counters any time. If only "your" clerk knows your details, you must always wait for that one clerk.

Availability maths: - Components in **series** (a request must pass through all of them) multiply their availability. Two parts at 99.9% each give 0.999 × 0.999 = 0.998. So the chain is *less* available than each part. - **Redundant** components in parallel (any one can do the job) improve availability. Two parts at 99.9% give $1 - 0.001^2 \approx 0.999999$. - 99.9% availability ≈ 8.8 hours of downtime per year. 99.99% ≈ 53 minutes per year.

How does it work?

```
Clients → Load balancer → [App 1] [App 2] [App 3]   (stateless, autoscaled)
                                ↓       ↓       ↓
                          Cache (Redis) · DB primary + replicas · Queue · Object storage
```

The app instances are all the same and hold no state. All shared data sits in the layer below them. So you can add or remove app instances freely (autoscaling).

Example

Take a shopping cart (session state):

1. The cart is stored in the memory of one app pod.
2. The user's next request lands on a different pod.
3. That pod has no cart. The cart looks empty — the design is broken.

The fix: move the cart to Redis (or into a signed token). Now every pod is the same. Kubernetes can add and remove pods freely.

When would I use it?

- Always design the app tier to be stateless and horizontally scalable.
- For databases, scale vertically first because it is simpler. Move to replicas or shards only when you need them.

What can go wrong?

- **Sticky sessions hiding state.** Sticky sessions send a user always to the same instance. This hides the fact that the service keeps state. It causes uneven load, and sessions are lost when that instance fails (failover).

- **Local file storage on instances.** Files saved on one instance are missing on the others.
- **In-memory caches that drift apart.** Each instance's local cache can hold a different version of the data.
- **The database becomes the bottleneck.** Adding more app servers just sends more load to the same database.

Interview questions

Q1. Vertical vs horizontal scaling? Vertical scaling adds resources to one machine. It is simple, but it has a limit and is a single point of failure. Horizontal scaling adds more machines. It has no hard limit and tolerates failures, but it needs stateless services and load balancing.

Q2. What makes a service stateless? The instance keeps no client-specific state in its memory between requests. All state is moved out to databases, caches or tokens.

11.4 Load balancing

Why?

> **In simple words:** A load balancer is the traffic police at the front. It spreads requests across your servers and stops sending traffic to any server that is sick.

Having many instances only helps if: - traffic is spread across them, and - dead instances stop getting requests.

A load balancer does both. It also does other jobs: - It handles TLS (HTTPS encryption), so the app servers don't have to. This is called *terminating TLS*. - It routes by path or host name. - It is the first line of defence against too much traffic.

What is happening internally?

- **L4 (transport layer)** load balancers route TCP/UDP connections using only IP address and port. They don't read the HTTP request. They are very fast, but they cannot route by URL.
- **L7 (application layer)** load balancers understand HTTP. They can route by path, header or cookie. They can terminate TLS, retry failed requests, rewrite requests and rate-limit (cap how many requests a client can send).
- **Health checks** regularly test each instance and remove the failing ones.
- **Connection draining** lets requests that are already running finish before an instance is removed, for example during a deploy.
- **Global load balancing** sends users to the nearest healthy region. It uses DNS/GeoDNS (DNS answers differently based on the user's location) or Anycast (many locations share one IP; the network picks the nearest).
- **In Kubernetes:** a `Service` gives L4 balancing (through kube-proxy). An Ingress or Gateway gives L7 routing. **In Azure:** Load Balancer is L4; Application Gateway and Front Door are L7.

Ways to pick the next instance (algorithms):

Algorithm	Use
Round robin / weighted	Identical, stateless instances (weighted gives bigger machines more traffic)

Algorithm	Use
Least connections / least response time	Requests that cost very different amounts of work
IP hash / consistent hashing	Affinity — the same user or key always goes to the same server (caches, sticky routing)

How does it work?

A request passes through these layers:

```
DNS → Front Door / CDN (global L7) → regional L7 LB (TLS, path routing) → service pods (L4 Service) → app
```

1. DNS points the user to the global edge.
2. The global L7 layer (Front Door / CDN) picks the nearest healthy region.
3. The regional L7 load balancer decrypts TLS and routes by path.
4. The Kubernetes Service (L4) picks one healthy pod.
5. The app handles the request.

Example

- `/api/orders/*` goes to the order service.
- `/api/loyalty/*` goes to the loyalty service.
- `/static/*` goes to the CDN.

Pods that fail their readiness probes (the "am I ready to serve?" check) are removed within seconds. A canary release — a new version tried on a small slice of users — gets 5% of traffic through weighted routing.

When would I use it?

In front of every horizontally scaled tier. Also between services — often through the service mesh or the Kubernetes Service.

What can go wrong?

- **The load balancer itself becomes a single point of failure.** Use managed, redundant load balancers.
- **Bad health checks.** Some only test "is the process alive?" and miss real problems. Others fail whenever any dependency has a tiny blip, which removes healthy instances.
- **Sticky sessions** make the load uneven.
- **Long-lived connections don't rebalance.** WebSockets and gRPC keep one connection open for a long time. When you add new instances, the old connections stay on the old instances, so the new ones get little traffic.

Interview questions

Q1. L4 vs L7 load balancing? L4 routes connections by IP and port. It is fast and works with any protocol. L7 understands HTTP. It routes by path, header or cookie, and it terminates TLS.

Q2. How does a load balancer know an instance is unhealthy? In two ways. Active health checks call the instance regularly — for example an HTTP readiness endpoint or a TCP check. Passive detection

watches real traffic for errors or timeouts.

11.5 Caching

Why?

> **In simple words:** A cache keeps a copy of popular data in fast memory. Most requests get their answer from this copy and never touch the slow database.

The fastest request is the one that never reaches the database.

Caching serves data that is read often and changed rarely, straight from memory. This gives two benefits: - **Lower latency** — from milliseconds down to microseconds. - **Protection for the database** — it is saved from load it could not handle.

Think of a kirana shop. The shopkeeper keeps the fast-selling items on the counter. Only for rare items does he go to the back storeroom.

What is happening internally?

Cache layers — a request can be answered at any of these levels, from nearest to farthest: browser → CDN → API gateway → in-process cache inside the app (Caffeine) → distributed cache shared by all instances (Redis/Memcached) → the database's own in-memory buffer pool.

Read and write strategies:

Strategy	How	Trade-off
Cache-aside (lazy)	Read the cache → on a miss, load from DB → fill the cache. On a write, update the DB, then **delete** the key	The default choice; data can be stale for a short time; the first read is always a miss
Read-through	The cache itself loads from the DB on a miss	Simpler app code; but the cache becomes critical
Write-through	Write the cache and the DB together, synchronously	Reads are always fresh; writes are slower
Write-behind	Write the cache; copy to the DB later, asynchronously	Fast writes; risk of losing data
Write-around	Write only to the DB; cache it when it is read	Avoids caching data that is rarely read again

Eviction — which item to throw out when the cache is full: - LRU (least recently used) — the most common. - LFU (least frequently used). - FIFO (first in, first out). - TTL (time to live) — the item expires after a fixed time.

Redis uses an approximate LRU/LFU. It checks a small random sample of keys instead of tracking all of them exactly.

How does it work?

Common failure modes and their fixes:

- **Stampede (thundering herd).** A popular key expires. Thousands of requests miss at the same moment and all hit the DB together. Fixes:

- **Single-flight** — only one request loads the key; the others wait for its result.
- Short locks.
- **Early probabilistic refresh** — refresh the key a little before it expires.
- **Stale-while-revalidate** — keep serving the old value while one request fetches the new one.
- **Penetration.** Requests ask for keys that don't exist, so they always miss and always hit the DB. Fixes: cache "not found" results for a short time, or put a Bloom filter (a small structure that quickly says "definitely not present") in front.
- **Avalanche.** Many keys expire at the same time and the DB gets flooded. Fix: add random **jitter** to TTLs so they expire at different times.
- **Hot keys.** One key gets huge traffic and overloads one cache node. Fixes: copy it under several names (`key#1..#n`), and add a local in-process cache in front.
- **Consistency.** Cache and DB can disagree. Fixes: delete on write (don't update), keep short TTLs as a safety net, and evict the key only after the DB transaction commits.

Example

```
GET /outlets/42/schemes
  → Redis GET schemes:t7:42 → hit (0.8 ms) → return
  → miss → DB query (25 ms) → SET schemes:t7:42 (TTL 10 min ± 60 s jitter) → return
Admin edits a scheme → DB commit → DEL schemes:t7:* for affected outlets (or bump a version in the key)
```

Step by step: 1. The app checks Redis. On a hit, it answers in under 1 ms. 2. On a miss, it queries the DB (25 ms) and stores the result with a 10-minute TTL plus some random jitter. 3. When an admin edits a scheme, the DB commits first. Then the related keys are deleted (or a version number in the key is increased, so old keys are simply never read again).

Cache sizing: 20% of 10M outlet profiles × 2 KB = 4 GB. A small Redis cluster is enough.

When would I use it?

- Data that is read a lot and changes little: catalogues, configuration, profiles, pre-computed totals, session data.
- Expensive computations.
- Protecting slow or rate-limited dependencies.

What can go wrong?

- **Caching data that must be strongly consistent** — for example, point balances during a redemption. A stale value can cause wrong results.
- **Unbounded caches** that grow until memory runs out.
- **Caching per-user data that is rarely reused** — it wastes memory.
- **Stale data after writes.**
- **Using the cache as the database** — data is lost when it is evicted or when the cache restarts.

Interview questions

Q1. What caching strategy would you use and why? Usually cache-aside with TTLs. It is simple, and the system still works if the cache fails. I delete the key on writes and add jitter to TTLs.

Q2. How do you prevent a cache stampede? Use single-flight loading per key, locks, early refresh, serving stale data while one request refreshes it, and TTL jitter.

Q3. Why delete instead of update the cache on writes? An update can race with a read happening at the same time. That read may put old data back into the cache after your update. If you delete instead, the next read loads fresh data from the DB.

11.6 CDNs and the edge

Why?

> **In simple words:** A CDN keeps copies of your images and files in many cities, close to users. Users download from the nearby copy instead of your far-away main server.

Users are often far from your datacentre. Images, scripts and videos are large files.

A **Content Delivery Network (CDN)** caches this content at hundreds of *edge locations* — small data centres close to users. This gives two benefits: - Lower latency, because the data travels a shorter distance. - Less load (and cost) on your *origin* servers (your own main servers), because the CDN serves most of the bandwidth.

Think of a newspaper. It is printed in many city presses, not shipped from one office to the whole country.

What is happening internally?

- **Pull CDN:** on a miss, the edge fetches the file from the origin. It then caches it, following the `Cache-Control` headers. **Push CDN:** you upload the content to the CDN ahead of time.
- **Cache keys:** the CDN identifies a cached item by its URL plus some chosen headers. Use **versioned file names** (`app.3f9a.js`). A new version gets a new name, so users get it at once. This is better than purging (deleting) old copies.
- **Other jobs of the edge:** it terminates TLS, absorbs DDoS attacks (floods of fake traffic), compresses responses, converts images (to WebP/AVIF, or resizes them), and can run small pieces of your code called **edge functions**.
- **Dynamic API responses** can also be cached at the edge for a short time (using `s-maxage` and `stale-while-revalidate`).

How does it work?

```
User in Varanasi → nearest edge (Delhi) → cache hit → 20 ms
                                        → miss → origin (Mumbai region) → cache → serve
```

1. A user in Varanasi is sent to the nearest edge, in Delhi.
2. If Delhi has the file (a hit), it answers in about 20 ms.
3. If not (a miss), Delhi fetches it from the origin in the Mumbai region, caches it, and serves it. The next user near Delhi gets a hit.

Example

Product images in a retailer app: 1. Upload the original images to object storage. 2. The CDN serves URLs like `img.cdn/sku/123?w=320&fmt=webp` . The edge resizes the image, converts it to WebP, and caches it. 3. Cheap, low-end Android phones on 4G get small images in tens of milliseconds.

When would I use it?

For all static assets, images, video segments (13.16), downloadable files, and public API responses that are safe to cache.

What can go wrong?

- **Caching personalised responses.** One user's private data can be shown to another user — a data leak.
- **Forgetting invalidation.** Users keep getting old assets.
- **Query-string explosion.** Many slightly different URLs for the same file split the cache into many small, rarely-hit entries.
- **Origin overload after a purge.** All edges miss at once and hit the origin together. A shield (origin-shield) layer — one middle cache in front of the origin — helps.

Interview questions

Q1. How does a CDN reduce latency? It serves cached content from edge locations close to users. So requests avoid the long round trip to the origin.

Q2. How do you invalidate CDN content? Use versioned URLs (the preferred way), Cache-Control TTLs, or the CDN's explicit purge APIs.

11.7 Choosing a database

Why?

> **In simple words:** Pick the database based on how your data is read and written, how correct it must be, and how big it will get — not because a database is popular. Start with PostgreSQL and add others only for a clear reason.

The data store is the hardest component to change later. It is also usually the bottleneck.

So choose it based on: - access patterns (how data is read and written), - consistency needs (how correct and up to date reads must be), and - scale.

Do not choose by fashion. Making this choice well is one of the most visible SDE2 skills.

What is happening internally?

Type	Examples	Strengths	Typical use
Relational (OLTP — everyday transactions)	PostgreSQL, MySQL, SQL Server	ACID transactions, joins, constraints, flexible queries	Orders, payments, ledgers, users — the default
Key-value	Redis, DynamoDB	O(1) access by key (same speed at any size), at huge scale	Sessions, carts, idempotency keys (to detect repeated requests), caches
Wide-column	Cassandra, ScyllaDB, Bigtable	Very high write throughput, access by partition key, multiple datacentres	Messages, activity feeds, time series, IoT

Type	Examples	Strengths	Typical use
Document	MongoDB, Cosmos DB	Flexible schema, nested documents	Catalogues, content, configuration
Search	Elasticsearch, OpenSearch	Full-text search, fuzzy (typo-tolerant) matching, facets (filters with counts), relevance ranking	Product and log search
Columnar OLAP (analytics)	ClickHouse, BigQuery, Snowflake, Trino over a lake	Fast totals and group-bys over billions of rows	Analytics, dashboards, incentive aggregates
Graph	Neo4j	Queries that follow many relationship hops	Social graphs, fraud rings
Time-series	Prometheus, TimescaleDB, InfluxDB	Retention and downsampling (keeping old data at lower detail)	Metrics
Object storage	S3, Azure Blob	Cheap, durable storage for large objects	Images, files, backups, data lake

ACID vs BASE: - **ACID** — atomic (all or nothing), consistent (rules always hold), isolated (transactions don't disturb each other), durable (committed data is not lost). Use it for data where correctness is critical. - **BASE** — basically available, soft state, eventually consistent (copies agree after some time). Many distributed NoSQL stores work this way.

How does it work?

Decision path:

1. Start with PostgreSQL.
2. Switch or add a store **only when a specific requirement forces it**. Name that requirement out loud. For example: - "100k writes/s of append-only events with partition-key reads → Cassandra" - "full-text with typo tolerance → Elasticsearch" - "aggregations over 5 billion rows → ClickHouse"
3. Using several stores in one system is normal. This is called *polyglot persistence*. A common setup: Postgres for the truth, Redis for hot paths, Elasticsearch for search, ClickHouse for analytics. The other stores are fed from Postgres by events or CDC (change data capture — streaming every change from the database's log).

How indexes, MVCC and locking work inside the database is covered in Chapter 12.

Example

A loyalty platform: - Ledger and accounts in PostgreSQL — it needs transactions and constraints. - Balances cached in Redis. - Redemption history search in Elasticsearch (optional). - Monthly incentive and analytics totals in ClickHouse, fed via CDC.

When would I use it?

At the data-model step of every design.

What can go wrong?

- Choosing NoSQL "for scale", then finding out you need transactions and joins.
- Choosing SQL for a write volume that a single primary cannot handle.

- Using the search index or the cache as the source of truth.
- Designing NoSQL tables like relational ones. In NoSQL you should design *query-first* — start from the queries you need and shape the tables around them.

Interview questions

Q1. SQL vs NoSQL — how do you decide? I default to SQL, because it gives transactional integrity and flexible queries. I choose NoSQL only for a concrete need: massive write throughput, a flexible schema, simple key access at huge scale, or multi-region availability.

Q2. Why might you use several databases in one system? Because each one serves a different access pattern — transactions, caching, search, analytics. I keep one source of truth and feed the others from it using events or CDC.

11.8 Replication

Why?

> **In simple words:** Replication means keeping copies of the database on other machines. If the main one dies, a copy takes over, and copies can also share the read load — but keeping all copies in sync is the hard part.

One database server is a single point of failure. It is also a limit on how many reads you can serve.

Copies on other machines are called **replicas**. They give you: - **Availability** — if the main server fails, a replica takes over. This is called *failover*. - **Read scaling** — reads can be spread across replicas. - **Geographic closeness** — a replica can sit near users in another region.

The cost: you must keep all the copies consistent.

What is happening internally?

- **Leader–follower (primary–replica).** All writes go to the leader. The leader streams its log of changes (the WAL in Postgres, the binlog in MySQL) to the followers. Reads can go to the followers.
- **Synchronous:** the leader waits for a follower to confirm each write. No data is lost on failover, but writes are slower.
- **Asynchronous:** the leader does not wait. Writes are fast, but followers **lag** behind. The latest writes can be lost on failover.
- **Semi-synchronous:** the leader waits for at least one follower.
- **Multi-leader.** Writes are accepted in several regions. When two regions change the same data, the conflict must be resolved — by last-write-wins, CRDTs (data types that merge automatically) or merge logic in the application.
- **Leaderless** (Dynamo-style, as in Cassandra): 1. Write to N replicas. The write succeeds after W of them confirm (acks). 2. Read from R replicas. 3. If **R + W > N**, the replicas you read from and the replicas you wrote to always overlap. So at least one read sees the latest write. 4. *Read repair* (fixing stale copies during reads) and *anti-entropy* (a background job that compares and syncs copies) fix any differences.
- **Failover** has three steps: detect that the leader failed, promote a follower to be the new leader, and point clients to it. The risks:
- Asynchronous writes that never reached the follower are lost.

- **Split brain** — two nodes both think they are the leader. This is prevented with consensus (Raft/Paxos, through etcd, ZooKeeper or Patroni) and fencing tokens (a number that lets storage reject writes from the old leader).

Think of a head office and branch offices. The head office makes every change and sends copies of the register to the branches. A branch can answer questions, but its register may be a few minutes old.

How does it work?

```
writes → Primary ──WAL stream──► Replica 1 (reads)
                  └────────────► Replica 2 (reads, other AZ)
primary dies → orchestrator (Patroni/managed service) promotes Replica 1 → DNS/endpoint switches
```

1. All writes go to the primary.
2. The primary streams its WAL to both replicas. Replica 2 is in another availability zone (AZ — a separate datacentre in the same region).
3. If the primary dies, an orchestrator (Patroni or the managed cloud service) promotes Replica 1.
4. The DNS name or endpoint switches to the new primary, so clients reconnect there.

Example

Read-your-writes problem: 1. A retailer redeems points. This write goes to the primary. 2. The app immediately reads the balance from a replica that is lagging. 3. The screen shows the old balance. The user is confused.

Fixes: - Read the user's own recent data from the primary for a short time after they write. - Track a "last write LSN/timestamp" (the position of their last write in the log) per session. Send reads only to replicas that have caught up to it. - For critical screens, always read from the primary.

When would I use it?

In every production database: - At least one synchronous or semi-synchronous standby in another availability zone, for failover. - Plus asynchronous replicas for read scaling and reporting.

What can go wrong?

- Replica lag causes stale reads and a confusing user experience.
- Asynchronous failover loses recent writes.
- Heavy reports run on the primary instead of a replica, slowing everything down.
- Split brain, when fencing is not set up properly.

Interview questions

Q1. Synchronous vs asynchronous replication? Synchronous replication waits for replicas to confirm each write. It is durable but makes writes slower. Asynchronous replication does not wait. It is fast, but replicas lag and recent writes can be lost on failover.

Q2. What is replication lag and how do you handle it? It is the delay before replicas show the leader's writes. I handle it with read-your-writes routing (send a user's reads to the leader right after they write), lag-aware routing, or by accepting eventual consistency where that is fine.

Q3. What is a quorum? In leaderless replication with N replicas, a write needs W acks and a read needs R responses, with R + W > N. Because the two sets must overlap, every read includes at least one replica with the latest successful write.

11.9 Partitioning and sharding

Why?

> **In simple words:** When one database can't hold all the data or keep up with the writes, you split the data across several databases. Each one holds only its own part.

Sometimes one database node cannot hold all the data. Or it cannot keep up with the rate of writes.

Then you split the data across nodes. Each piece is called a **shard**, and each shard holds only part of the data. So storage and write capacity grow as you add shards.

Think of a railway reservation office with separate counters by train number range. Each counter handles only its own trains, so the queues move faster.

What is happening internally?

Strategy	How	Pros	Cons
Range	Each shard gets a range of keys (A–F, G–M; or date ranges)	Range scans are efficient	Hot spots (recent dates, popular ranges get all the traffic)
Hash	`hash(key) % N`, or consistent hashing (11.11)	Data spreads evenly	Range queries must go to all shards; resharding is costly
Directory / lookup	A service keeps a map of key → shard	Flexible placement; easy to move data	An extra hop; the directory itself must be highly available
Geo / tenant	Split by region or by tenant (customer company)	Data stays local, helps compliance, one noisy tenant can't hurt others	Shards have uneven sizes → move big tenants to their own shards

Choosing a shard key. A good shard key: - has high cardinality (many different values), - spreads data evenly, and - matches the main access pattern, so most queries hit **one** shard.

For a multi-tenant SaaS, `tenant_id` is the natural choice. The largest tenants can be split further or given their own shards.

How does it work?

```
Router / client library: shard = hash(tenant_id) → shard 3
Shard 0 [tenants a..]  Shard 1 [...]  Shard 2 [...]  Shard 3 [tenant t-42] ← query goes only here
Cross-shard query (global top-10) → scatter to all shards, gather and merge (expensive; avoid on hot paths)
```

1. A router (or a client library) hashes the `tenant_id` to pick the shard — here, shard 3.
2. The query for tenant t-42 goes only to shard 3.
3. A query across all tenants (like a global top-10) must be sent to every shard. The results are then collected and merged. This is called *scatter-gather*. It is expensive, so avoid it on hot paths.

Rebalancing (moving data when you add shards): - With plain `% N`, adding one shard changes N. Almost every key moves to a new shard. - Consistent hashing, or many fixed **virtual partitions**, moves only a small fraction. For example, use 4,096 logical partitions and map them to physical nodes. Adding a node just moves some whole partitions to it. - Many systems (Citus, Vitess, Cassandra, DynamoDB) handle this for you.

Example

A loyalty ledger taking 50,000 writes/s: 1. Shard by `tenant_id`. All of a tenant's accounts sit on one shard, so per-tenant queries stay on one shard. 2. Inside each shard, partition the ledger table by month (Chapter 12.8). This makes it easy to drop old data (retention). 3. The largest FMCG tenant gets its own shard.

When would I use it?

Only after you have used up the simpler options: vertical scaling, read replicas, caching and table partitioning. Sharding adds a lot of complexity — cross-shard joins, transactions, rebalancing and harder operations.

What can go wrong?

- **A shard key that creates hot spots.** For example, IDs that always increase in range sharding (all new writes go to the last shard), or one giant tenant.
- **Cross-shard transactions.** Avoid them; use sagas instead (a chain of local steps, each with an undo step).
- **Global secondary indexes** (indexes across all shards) are hard to keep correct.
- **Uneven growth** between shards.
- **Downtime during resharding.**

Interview questions

Q1. How do you choose a shard key? It needs high cardinality and an even spread. It should match the main access path, so most queries hit one shard. For range sharding, avoid keys that always increase.

Q2. What problems does sharding introduce? Cross-shard queries and joins, distributed transactions, rebalancing, hot shards, and more operational complexity.

Q3. Partitioning vs sharding? Partitioning splits a table inside one database. It helps with pruning (skipping partitions a query doesn't need) and retention. Sharding spreads data across multiple database servers. It adds capacity and write scale.

11.10 CAP, PACELC and consistency models

Why?

> **In simple words:** When the network between your database copies breaks, you must choose: refuse some requests to stay correct, or keep answering with possibly old data. You make this choice per feature — money needs correct, likes can be a bit old.

A distributed data store cannot be perfectly consistent, always available, and tolerant of network breaks, all at the same time.

So you must know which guarantee each part of your system needs, and say it out loud. This is how you justify your database and replication choices.

What is happening internally?

- **CAP.** Sometimes the network splits nodes into groups that cannot talk to each other. This is a **partition**. During a partition, the system must choose:

- **Consistency** — refuse or delay requests it cannot answer correctly, or
- **Availability** — answer anyway, maybe with stale data.

Partitions will happen in real networks. So the real choice is C vs A *during* a partition. - **PACELC** adds the normal case: - If there is a **P**artition → choose **A** or **C**. - **E**lse (normal operation) → choose **L**atency or **C**onsistency. Waiting for every copy to agree is slower. - Dynamo/Cassandra are PA/EL (prefer availability and low latency). Spanner and etcd are PC/EC (prefer consistency). - **Consistency models**, from strong to weak:

Model	Guarantee	Example need
Linearizable (strong)	Every read sees the latest committed write	Balances, inventory counts, leader election
Sequential	All nodes see operations in the same order	
Causal	If one operation caused another, everyone sees them in that order	A reply appears after its message
Read-your-writes	A user always sees their own writes	Profile updates
Monotonic reads	Reads never go back in time (you never see older data after newer)	Feeds
Eventual	Copies become the same once writes stop	Likes, view counts, search indexes

Think of an ATM when the link to the bank is down. A strict bank refuses withdrawals (consistency). A relaxed one allows small withdrawals and settles later (availability).

How does it work?

Choose **per feature**, not for the whole system: - Redemptions need strong consistency. Otherwise the same points can be spent twice (double spend). - A leaderboard or an analytics dashboard can be a few seconds stale.

Example

The network between two regions breaks: - The payment service refuses writes in the smaller (minority) region. It chooses CP, because double charges are worse than a short outage. - The product catalogue keeps serving cached data in both regions. It chooses AP, because slightly old product details are acceptable.

When would I use it?

- When choosing stores and replication modes.
- When designing multi-region systems.
- When justifying caching and asynchronous updates.

What can go wrong?

- Claiming a system is "CA". In a real distributed system, partitions happen, so you cannot skip P.
- Assuming eventual consistency is fine for money.
- Not handling what users see under eventual consistency — for example, you need "payment processing..." states and read-your-writes.

Interview questions

Q1. Explain CAP with an example. During a network partition, you must choose consistency (reject or block some requests) or availability (serve data that might be stale). A payment ledger chooses C. A social feed chooses A.

Q2. What does PACELC add? It says that even with no partition, there is still a trade-off — between latency and consistency.

Q3. Where would you use strong vs eventual consistency in a loyalty platform? Strong consistency for balances, redemptions and idempotency checks (checks that stop the same request being processed twice). Eventual consistency for leaderboards, analytics, notifications and search.

11.11 Consistent hashing

Why?

> **In simple words:** With simple `% N` hashing, adding one server moves almost every key to a new place. Consistent hashing puts servers and keys on a circle, so adding a server moves only a small share of keys.

With `hash(key) % N`, adding or removing one cache or database node changes N. That moves almost every key to a different node. The result is a mass of cache misses, or a huge data migration.

Consistent hashing moves only about 1/N of the keys.

What is happening internally?

1. Picture a **ring** of numbers from 0 to $2^{32}-1$. Hash each node onto a point on this ring. Hash each key onto the ring too.
2. A key belongs to the first node you meet going clockwise from the key's position.
3. When you add a node, it takes only the keys between itself and the node before it.
4. When you remove a node, its keys go to the next node clockwise.
5. **Virtual nodes:** each physical node is placed at many points (100–200) on the ring. This spreads the load more evenly. A bigger machine can be given more points, so it takes more keys.

Think of a round dining table with plates (keys) and waiters (nodes) standing at spots. Each plate is served by the next waiter clockwise. A new waiter joining takes over only the plates just before him.

```
ring: 0 ──[A#1]──k1──[B#1]──k2──[A#2]──k3──[C#1]──… (wraps)
k1 → B, k2 → A, k3 → C.  Add node D between A#2 and C#1 → only keys in that arc move to D.
```

How does it work?

The ring is a `TreeMap` from hash position to node. `ceilingEntry` finds the first node at or after the key's hash (clockwise). If there is none, the search wraps around to the first entry.

```
class ConsistentHashRing<T> {
    private final TreeMap<Long, T> ring = new TreeMap<>();
    private final int vnodes;
    ConsistentHashRing(int vnodes) { this.vnodes = vnodes; }
    void add(T node)    { for (int i = 0; i < vnodes; i++) ring.put(hash(node + "#" + i), node); }
    void remove(T node) { for (int i = 0; i < vnodes; i++) ring.remove(hash(node + "#" + i)); }
    T nodeFor(String key) {
        Map.Entry<Long, T> e = ring.ceilingEntry(hash(key));
        return (e != null ? e : ring.firstEntry()).getValue();          // wrap around
```

```
    }
    private long hash(String s) {                                    // well-mixed hash: MD5's first 8
bytes (as in ketama)
        try {
            byte[] d =
java.security.MessageDigest.getInstance("MD5").digest(s.getBytes(java.nio.charset.StandardCharsets.UTF_8));
            long h = 0;
            for (int i = 0; i < 8; i++) h = (h << 8) | (d[i] & 0xff);
            return h;                                                // Murmur3/xxHash are faster
alternatives
        } catch (java.security.NoSuchAlgorithmException e) { throw new IllegalStateException(e); }
    }
}
```

Key points in the code: - `add` places each node at `vnodes` points, using the names `node#0`, `node#1` and so on. - `nodeFor` is O(log n), because `TreeMap` is a sorted tree. - The hash must be well mixed so the points spread evenly around the ring. Here it takes the first 8 bytes of MD5, as the ketama library does.

Example

A Redis cache tier grows from 4 nodes to 5: - With modulo hashing, about 80% of keys move. That causes a storm of cache misses on the DB. - With consistent hashing, only about 20% move.

When would I use it?

- Distributed caches.
- Cassandra/Dynamo partitioning.
- Load balancing with affinity (always routing the same key to the same server).
- Sharded rate limiters.
- Routing chat connections.

What can go wrong?

- **Too few virtual nodes** — the load is uneven.
- **A poor hash function** — points cluster together on the ring.
- **Hot keys still overload one node** — consistent hashing spreads keys, not traffic per key. Replicate hot keys.

Interview questions

Q1. What problem does consistent hashing solve? It keeps key movement small when nodes are added or removed. Only about 1/N of the keys move, instead of nearly all of them.

Q2. Why virtual nodes? They spread each physical node's ranges all around the ring, which evens out the load. They also let you give bigger nodes more points, so they take a bigger share.

11.12 Message queues and asynchronous processing (RabbitMQ and friends)

Why?

> **In simple words:** A queue is a waiting line between two services. One service drops a job in the line and moves on; another service picks it up when it is ready.

When services call each other directly and wait for the answer, they are tightly tied together. This is a *synchronous call chain*. The problems are easy to see:

- If the notification service is slow, checkout also becomes slow.
- If the loyalty service is down, orders fail.

A queue fixes this. The service that sends work is the *producer*. It puts the job in the queue and continues without waiting. The service that does the work is the *consumer*. This gives you four benefits:

- **Decoupling** — the producer does not care if the consumer is slow or down right now.
- **Absorbing traffic spikes** — a sudden burst waits in the queue; workers finish it at a steady speed. This is called *load levelling*.
- **Retries** — failed work can be tried again in the background. The user does not wait for it.
- **Fan-out** — one event can be sent to many consumers.

Think of a restaurant token system. You order at the counter, get a token and sit down. The kitchen cooks orders one by one. The cashier does not stand and wait for your food.

What is happening internally?

There are two families of messaging systems. A *message queue* deletes a message once it is handled. A *log* (or *stream*) keeps every message for days, and each reader remembers its own position.

	Message queue (RabbitMQ, Amazon SQS, Azure Service Bus)	Log / stream (Kafka, Kinesis, Event Hubs, Pulsar)
Model	A message is deleted once a consumer confirms it (acks)	Messages are only added to the end of a log and kept for days; each consumer remembers its position (offset)
Consumers	Several consumers share one queue and compete for messages	Consumers form groups; many separate groups can read the same data again
Ordering	Per queue (but parallel consumers and redelivery can break it)	Per partition
Replay	No (gone once acked)	Yes (move the offset back)
Routing	Rich (exchanges, topics, headers)	Topic + partition key
Per-message features	Ack/nack, TTL, delay, priority, DLQ	Mostly works in batches and offsets
Best for	Task/work queues, commands, RPC-style workflows	Event streaming, event sourcing, CDC, analytics pipelines, high throughput

A few terms from the table:

- **Ack** (acknowledge) — the consumer tells the broker "I am done, delete it". **Nack** means "I failed".
- **TTL** (time to live) — how long a message may wait before it expires.
- **DLQ** (dead-letter queue) — a side queue where failed or expired messages are parked for a human to check.
- **Offset** — the position number of a message in a log, like a page number.
- **CDC** (change data capture) — streaming every change made in a database as events.

RabbitMQ in particular: - The path of a message is: Producer → **exchange** → (binding with routing key) → **queue** → consumer. The exchange is a sorting desk. It reads the message's *routing key* (a label) and decides which queue gets it. Exchange types: - **direct** — the key must match exactly. - **topic** — the key can match wildcards, like `order.*.created` or `#`. - **fanout** — copy the message to every queue bound to it. This is pub/sub (publish/subscribe). - **headers** — match on message headers instead of the key. - **Acks**: the consumer sends an ack only after it finishes processing. If the consumer dies before acking, RabbitMQ gives the message to someone else. So each message is delivered at least once (*at-least-once*). A consumer can also `nack` or reject a message, with or without putting it back in the queue (*requeue*). - **Prefetch (QoS)**: this limits how many unacked messages one consumer can hold at a time. Because of this, a slow consumer does not hoard work, and load is shared fairly. - **Durability**: to survive a broker restart, you need three things together: durable queues, persistent messages and *publisher confirms* (the broker tells the producer "saved"). For high availability (HA), use **quorum queues**. These copy each queue to several nodes using Raft, a voting-based replication protocol. - **Dead-letter exchanges (DLX)**: some messages cannot be handled — they are rejected, expired (TTL) or the queue is full. These go to a DLX. You use a DLX for two things: 1. DLQs — a parking place for failed messages. 2. **Delayed retries** — send the message to a waiting queue with a TTL. When the TTL expires, it is dead-lettered back to the work queue for another try. - **Azure Service Bus** is closer to RabbitMQ than to Kafka. It offers queues and topics/subscriptions, *sessions* for ordering, a built-in DLQ, scheduled messages and duplicate detection.

Delivery semantics (how many times a message is delivered): - **At-most-once**: the consumer acks (or commits) first and processes later. If it crashes in between, the message is lost. So you may lose messages. - **At-least-once**: the consumer processes first and acks later. If it crashes after processing but before acking, the message comes again. So you may get duplicates. Because of this, **consumers must be idempotent** (11.14) — handling the same message twice must give the same result as handling it once. This is the practical default. - **Exactly-once**: possible only inside one closed system. For example, Kafka transactions for read-process-write inside Kafka. Across different systems you get "effectively once". That means at-least-once delivery + idempotency.

How does it work?

1. Checkout publishes a `SendReceipt` message to the notifications queue and returns to the user.
2. Three workers compete for messages. Each message goes to only one worker.
3. If a message fails three times, it goes to the DLX and then to `notifications.dlq`. Someone gets an alert, inspects it and replays it.
4. If 50k messages arrive at once, the queue holds them. Workers finish them at their own speed. KEDA (a Kubernetes autoscaler) adds more workers when the queue gets deep or lag grows.

```
Checkout ──publish "SendReceipt"──► [notifications queue] ──► worker 1 / worker 2 / worker 3 (competing)
                                          │ fails 3× → DLX → [notifications.dlq] → alert, inspect, replay
Spike of 50k messages → the queue absorbs it; workers drain at their own pace; autoscale on queue depth/lag
(KEDA)
```

Example

Perfect Store photo verification works like this:

1. The app uploads photos.
2. The API puts a `VerifyAudit{auditId}` message in the queue (*enqueues* it).
3. GPU/AI workers pick up messages at their own pace.
4. They write the results back and publish a notification event.

The user sees "verification in progress". They do not wait 30 seconds on an HTTP call.

When would I use it?

Use a queue when:

- The work is slow or unreliable, and the user does not need it done right now. Examples: notifications, image processing, report generation, third-party calls.
- Traffic comes in sudden spikes.
- One event must reach many consumers (fan-out).
- Services owned by different teams must talk to each other.

How to choose:

- **Work queue** (RabbitMQ / Service Bus / SQS) — for tasks and commands.
- **Kafka** — for event streams that several consumers need, or when you need replay.

What can go wrong?

- **Duplicates with non-idempotent consumers.** The same message is processed twice, and the effect happens twice.
- **Poison messages.** A message that always fails can block the queue. Use a DLQ with a retry limit.
- **Hidden slow consumers.** The queue keeps growing without limit, and nobody notices. Monitor queue depth and lag, and alert on them.
- **Lost messages with auto-ack.** The message is acked as soon as it is received. If the consumer then crashes, the message is gone.
- **Broken ordering.** Parallel consumers can process messages out of order. Don't assume order.
- **Wrong tool.** Don't use a queue when the user actually needs an answer right now (a synchronous response).

Interview questions

Q1. Why introduce a message queue? It decouples producers from consumers, so one being slow or down does not stop the other. It absorbs traffic spikes, retries failed work in the background, and fans out events to many consumers. Overall, the user path becomes more resilient and faster.

Q2. Queue (RabbitMQ) vs log (Kafka)? A queue hands out tasks and deletes each message once it is acknowledged; it has rich routing and per-message features like TTL, priority and DLQ. Kafka keeps an ordered log per partition that can be replayed. Many consumer groups read it independently, at very high throughput.

Q3. What delivery guarantee do you design for? At-least-once delivery with idempotent consumers, so duplicates are harmless. Exactly-once is possible only inside systems that support transactions end to end.

11.13 Kafka in depth

Why?

> **In simple words:** Kafka is a giant, shared notebook that you can only add lines to. Many readers read it at their own speed, each with their own bookmark, and can go back to old pages when needed.

Kafka has become the backbone of event-driven systems. It carries order events, CDC streams (database changes as events), analytics pipelines and audit logs.

Its design is a *partitioned, replicated, append-only log*. In plain words:

- **Append-only log** — new messages are only added at the end, never edited.
- **Partitioned** — the log is split into pieces, so many machines can work in parallel.
- **Replicated** — each piece is copied to several machines, so nothing is lost if one dies.

Because of this design you get very high throughput, ordering per key, durability and replay. Kafka questions come up in almost every SDE2 system-design round.

What is happening internally?

- **Storage**: a **topic** is split into **partitions**. Each partition is an ordered, append-only log. On disk, it is stored as several **segment files**. Every message has a number called its **offset** — its position in the partition.
- **Performance**: Kafka is fast for four reasons: 1. It writes to disk sequentially (always at the end), which is very fast. 2. It uses the OS page cache (the operating system keeps recent file data in memory). 3. It batches and compresses messages. 4. It uses **zero-copy** transfer (`sendfile`). Data goes from the page cache straight to the network, without passing through the application's memory.
- **Replication**: each partition has one **leader** and some followers. Producers and consumers talk to the leader. Followers copy from it. Followers that are fully caught up are called the **ISR** (in-sync replicas). A safe setup is:
- `replication.factor=3` — three copies of each partition.
- `min.insync.replicas=2` — at least two copies must be in sync.
- Producer `acks=all` — the write is confirmed only when all in-sync replicas have it.

With this, a write is acknowledged only after at least two replicas have it. So if one broker dies, no data is lost. - **Producer**: - It picks the partition using `hash(key) % partitions`. The same key always goes to the same partition, so messages with the same key stay in order. - It batches messages for speed, controlled by `linger.ms` (how long to wait to fill a batch) and `batch.size`. - The **idempotent producer** (on by default since 3.0) gives each message a sequence number. Because of this, a retry does not create a duplicate. - **Transactions** let a producer write to several partitions and commit consumer offsets as one atomic unit — all or nothing. This gives exactly-once read-process-write *within Kafka*. - **Consumer groups**: the partitions of a topic are divided among the members of a group. One partition goes to only one member of the group. So **max parallelism = number of partitions**. Different groups read independently, so each group gets every message (pub/sub). When members join or leave, Kafka reassigns partitions. This is **rebalancing**. *Cooperative-sticky assignment* moves as few partitions as possible, so there is less disruption. - **Offsets**: a consumer saves its position (commits its offset) in an internal topic called `__consumer_offsets`. If it commits *after* processing, you get at-least-once delivery. - **Retention**: Kafka deletes old data by time or by size. Or it uses **log compaction** — keep only the latest value for each key. This is ideal for changelogs and state. - **Coordination**: older Kafka used ZooKeeper to manage the cluster. Since 3.x, Kafka has **KRaft**, its own built-in Raft (voting-based) coordination. ZooKeeper was removed in Kafka 4.0. - **Ecosystem**: - Kafka Connect — ready-made source/sink connectors to move data in and out. - Debezium — CDC; it reads the database's log and turns changes into events. - Kafka Streams / Flink — stream processing. - Schema Registry — stores Avro/Protobuf/JSON schemas and enforces compatibility rules, so producers can't break consumers.

Think of a railway reservation register with several counters. Each counter (partition) has its own register, written line by line. Photocopies are kept at two other stations (replicas). Different departments (consumer groups) read the registers, each with its own bookmark (offset).

How does it work?

The picture below shows:

1. A producer sends events with `key = accountId` to the topic `points-events`.
2. The topic has three partitions, with leaders on different brokers.
3. The "notifications" group has two members. Member A reads p0 and p1; member B reads p2.
4. The "analytics" group has one member reading all three partitions.
5. Each group has its own offsets, so both groups see every event.

```
Producer (key = accountId) ──► topic points-events
   partition 0: [0][1][2][3]…   leader on broker 1, followers on 2 and 3
   partition 1: [0][1][2]…      leader on broker 2
   partition 2: …
Consumer group "notifications": member A ← p0, p1;  member B ← p2
Consumer group "analytics":     member X ← p0, p1, p2   (independent offsets — both see every event)
```

Choosing the partition count: divide your target throughput by what one partition can handle. Then add room for more consumers in future (e.g., 12–48 for a busy topic). Plan this well at the start. If you increase the count later, the key → partition mapping changes. During that change, per-key ordering breaks.

Example

Month-end spike: 500 accrual events/s for 2 hours. Here is how the setup handles it:

- The `order-completed` topic has 24 partitions, keyed by `outletId`.
- The loyalty consumer group autoscales from 4 to 24 pods based on **consumer lag** (how far the consumers are behind the latest message).
- Because the key is `outletId`, events for each outlet stay in order.
- Kafka keeps data for 7 days. So if you find a bug, you can fix the consumer and replay from an earlier offset.

When would I use it?

- Event-driven microservices.
- CDC pipelines (DB → Kafka → search/analytics).
- High-volume telemetry and logs.
- Event sourcing (storing every change as an event).
- Stream processing.

What can go wrong?

- **Too few partitions.** You can't add more consumers than partitions, so you can't scale.
- **Hot partitions.** If keys are skewed (one key gets most of the traffic), one partition gets overloaded.
- **Rebalance storms.** A consumer that takes longer than `max.poll.interval.ms` between polls is treated as dead. Its partitions move, then come back, again and again.
- **Data loss on broker failure.** This happens with `acks=1` or `min.insync.replicas=1`.
- **High storage cost.** Retention with no limit grows forever.

- **Broken consumers.** A schema change that is not compatible breaks the readers.
- **Wrong use.** Kafka is not a database. Don't use it for random lookups by key.

Interview questions

Q1. How does Kafka guarantee ordering? Only within a partition. So choose the message key so that events which must stay in order share the same key, and therefore the same partition.

Q2. What limits consumer parallelism? The number of partitions. Each partition is read by at most one member of a consumer group, so extra members sit idle.

Q3. How do you configure Kafka for no data loss? Replication factor 3, `min.insync.replicas=2`, and producer `acks=all` with idempotence on. On the consumer side, commit offsets only after processing.

Q4. What is log compaction? Kafka keeps only the latest record for each key, plus *tombstones* (a key with an empty value) to mark deletes. This turns the topic into a durable changelog of the current state.

11.14 Idempotency and delivery guarantees in practice

Why?

> **In simple words:** Idempotent means "doing it twice is the same as doing it once". If the user presses "Pay" two times, they should still pay only once.

Networks fail in the worst possible way: the request succeeds, but the response is lost. The caller does not know it succeeded. So:

- Clients retry.
- Queues redeliver.
- Users double-tap "Pay".

Without idempotency, every retry can double-charge, double-credit or double-ship. This is exactly the "duplicate reward postings under retry/failure" problem you solved at FieldAssist. Design for it on purpose, at every layer.

What is happening internally?

An operation is **idempotent** if doing it many times has the same effect as doing it once. By HTTP's own definition, GET, PUT and DELETE are idempotent. POST is not — unless you make it so.

Think of an ATM. If the network drops after cash is debited, the bank must not debit you again when the machine retries. It checks the transaction number first.

You handle this at four layers:

1. **API**:
2. The client creates an `Idempotency-Key` (a UUID) for each logical operation. On retry, it sends the same key again.
3. The server stores `(key, request hash, status, response)` with a **unique constraint** on the key. It does this in the **same transaction** as the business change.
4. On a replay (same key, same payload), the server returns the stored response.
5. Same key with a different payload → 422.
6. A second copy arrives while the first is still running → 409 (or wait for the first to finish).

7. **Consumers**: two options. - Keep a processed-event table keyed by event ID. Write to it in the same transaction as the effect. - Or make the effect naturally idempotent. For example, an upsert by business key, or a conditional update like `UPDATE … SET status='PAID' WHERE status='PENDING'` (the second run changes nothing).
8. **Downstream calls**: pass idempotency keys to payment gateways and partner APIs. Most of them support it.
9. **Retries**: use exponential backoff (wait longer after each failure) with jitter (a little randomness, so clients don't all retry together). Cap the number of attempts, and use retry budgets (a limit on what share of traffic can be retries).

How does it work?

1. Attempt 1: the server inserts the key with status `IN_PROGRESS`, debits points, writes the ledger and outbox, saves the response, and commits. The response is lost on the network.
2. Attempt 2 (the retry): the insert fails with a unique-key violation. The server reads the saved response and returns the same 201 body. Nothing is debited again.

```
POST /redemptions  Idempotency-Key: 7f3c…   (attempt 1: response lost on the network)
   BEGIN; INSERT idempotency(key=7f3c, status=IN_PROGRESS) → OK; debit; ledger; outbox;
          UPDATE idempotency SET status=DONE, response=…; COMMIT
POST /redemptions  Idempotency-Key: 7f3c…   (attempt 2, retry)
   INSERT idempotency(key=7f3c) → unique violation → SELECT response → return the same 201 body
```

Example

The table is keyed by tenant and key. It stores a hash of the request, so a different request with the same key can be caught.

```
CREATE TABLE idempotency_key (
  tenant_id     VARCHAR(40)  NOT NULL,
  key           VARCHAR(80)  NOT NULL,
  request_hash  CHAR(64)     NOT NULL,
  status        VARCHAR(16)  NOT NULL,            -- IN_PROGRESS, DONE
  response_body JSONB,
  created_at    TIMESTAMPTZ  NOT NULL DEFAULT now(),
  PRIMARY KEY (tenant_id, key)
);
-- expire rows after e.g. 24–72 h with a cleanup job (clients don't retry forever)
```

When would I use it?

Use it for every operation that:

- moves money, points or inventory,
- creates orders,
- sends messages, or
- is triggered by an event that may be retried.

What can go wrong?

- **Check and effect in separate transactions.** Two copies arrive at the same time. Both check, both find no key, and both do the effect.
- **Server-generated keys per attempt.** Each retry gets a new key, so the key is useless. The client must create it.
- **Not hashing the payload.** A different request that reuses a key silently gets the wrong (old) result.

- **Keys that never expire.** The table grows without limit.

Interview questions

Q1. How do you make a POST endpoint idempotent? The client sends an idempotency key. The server stores the key and the response under a unique constraint, in the same transaction as the business effect. On a duplicate, it returns the stored response instead of doing the work again.

Q2. How do you make a Kafka consumer idempotent? Save each processed event ID under a unique constraint, in the same transaction as the business effect. Or make the effect itself safe to repeat, such as an upsert or a conditional update.

Q3. Can you guarantee exactly-once delivery? Not across independent systems. Instead, you get exactly-once *effects*: deliver at least once, and make the processing idempotent.

11.15 The transactional outbox and sagas

Why?

> **In simple words:** The outbox makes sure "save to DB" and "send the event" never get out of step. A saga breaks one big multi-service job into small steps, each with an "undo" step if something later fails.

In microservices you face two related consistency problems:

1. **Dual write**: look at `save(order); kafka.send(OrderCreated)`. Two things can go wrong: - The DB commits, but the publish fails. - Or the publish succeeds, but the DB commit fails.

Either way, the systems disagree. And you cannot make a database and a message broker commit together as one atomic step. 2. **Distributed business transactions**: redeeming points for a partner voucher touches three systems — loyalty, the voucher partner and notifications. One option is *two-phase commit (2PC)*: a coordinator asks all parties to prepare, then tells all to commit. But 2PC is slow and blocking, and most brokers and SaaS APIs don't support it.

What is happening internally?

Transactional outbox:

1. In the **same local transaction** as the business change, also insert the event into an `outbox` table. Now both are saved together, or neither is.
2. A separate process, the **relay**, reads outbox rows, publishes them to Kafka and marks them as published. It finds new rows in one of two ways: - **Polling** — run a query like `SELECT … WHERE published_at IS NULL ORDER BY created_at LIMIT 100 FOR UPDATE SKIP LOCKED` again and again. `SKIP LOCKED` lets several relays run without picking the same rows. - **CDC** — Debezium reads the database's log and streams new outbox inserts.
3. Delivery is at-least-once (a row can be sent twice if the relay crashes after sending). So consumers must stay idempotent.

Think of a post office outbox tray. You write the letter and put it in the tray in the same moment. The postman collects from the tray later. The letter can't get "lost between desk and post box".

Saga: a long-running business transaction written as a chain of **local transactions** (each service commits its own step). Each step has a **compensating action**. If a later step fails, the compensation

undoes the earlier step in business terms (for example, credit points back). There are two styles:

- **Choreography**: services react to each other's events, with no central boss. For example: OrderCreated → PaymentService charges → PaymentCompleted → Inventory reserves → ... This is simple for 2–3 steps. But the overall flow is not written anywhere, so it is hard to follow.
- **Orchestration**: one coordinator tells each participant what to do. The coordinator is a state machine — Temporal, Camunda, AWS Step Functions, or your own table-driven orchestrator. It handles timeouts and retries. If a step fails, it runs the compensations in reverse order. The flow is explicit and easy to observe.

How does it work?

The outbox part:

1. In one transaction: debit 500 points (only if the balance is enough), add a ledger entry and insert an outbox row.
2. The relay runs every 200 ms (or uses CDC). It reads unpublished rows, sends them to Kafka with `acks=all`, and sets `published_at`.

The saga part (redeem points for a voucher):

1. Loyalty debits points. Compensation: credit them back.
2. The voucher partner issues a code, called with an idempotency key. Compensation: void the code.
3. Notification sends the code. No compensation needed.
4. If step 2 still fails after retries, the orchestrator runs compensation 1. The redemption becomes FAILED, and the user is told.

```
Outbox:
  BEGIN
    UPDATE account SET balance = balance - 500 WHERE id='A1' AND balance >= 500;
    INSERT INTO ledger_entry (...);
    INSERT INTO outbox (id, topic, key, payload) VALUES (uuid, 'points-events', 'A1',
'{"type":"PointsRedeemed",...}');
  COMMIT
  Relay (every 200 ms or via CDC): read unpublished → send to Kafka (acks=all) → mark published_at

Saga (orchestrated) — redeem points for a voucher:
  1. Loyalty: debit points (local tx)                compensation: credit points back
  2. Voucher partner: issue code (idempotency key)   compensation: void the code
  3. Notification: send the code                     (no compensation needed)
  Step 2 fails after retries → orchestrator runs compensation 1 → redemption = FAILED, user informed
```

Example

The outbox table below has a partial index on unpublished rows only. So the relay's query stays fast even when the table has many published rows.

```
CREATE TABLE outbox (
  id           UUID PRIMARY KEY,
  aggregate_id VARCHAR(40) NOT NULL,
  topic        VARCHAR(80) NOT NULL,
  payload      JSONB NOT NULL,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  published_at TIMESTAMPTZ
);
CREATE INDEX ix_outbox_unpublished ON outbox (created_at) WHERE published_at IS NULL;
```

For an orchestrator, keep a saga state table: `redemption_saga(id, step, status, attempts, last_error, updated_at)`. Because the state is saved, the saga can continue after a crash. Each step must also be

idempotent, because it may run again.

When would I use it?

- **Outbox**: whenever a DB change must reliably produce an event.
- **Sagas**: business processes that span several services or external partners. Examples: checkout, redemption with partner vouchers, payouts, onboarding.

What can go wrong?

- **Outbox problems:**
- The relay lags or fails. Monitor the age of the oldest unpublished row.
- The outbox table grows forever. Clean up published rows.
- Ordering breaks when several relay instances run. Partition the work by aggregate key.
- **Saga problems:**
- Some steps can't be fully undone — for example, an email already sent. Design *semantic* compensations instead (e.g., send a "please ignore" message).
- Users see in-between states. Show "processing".
- Steps that are not idempotent run twice after retries.
- Saga state is lost because it was not saved to storage.

Interview questions

Q1. What is the dual-write problem and how does the outbox solve it? Writing to a database and to a broker can't be done as one atomic step, so one can succeed while the other fails. The outbox writes the event into a table in the same DB transaction as the change. Then a relay or CDC publishes it reliably, at-least-once.

Q2. Saga vs 2PC? 2PC locks resources across all participants and gets stuck if the coordinator fails. Sagas use local transactions with compensating actions instead. You get eventual consistency without distributed locks, which is why sagas are preferred in microservices.

Q3. Choreography vs orchestration? In choreography, services react to each other's events with no central coordinator; it is simple, but the flow is hidden. In orchestration, a coordinator drives the steps and compensations; it is explicit and easier to monitor and change.

11.16 Rate limiting and API gateways

Why?

> **In simple words:** Rate limiting means "each caller gets only so many requests per second". The API gateway is the front gate where every request is checked before it goes in.

Without limits, one buggy client, scraper or tenant can flood your service. Then it becomes slow for everyone. Partners also give you quotas that you must respect when you call them.

Rate limiting keeps things fair and protects your capacity. The API gateway is the natural place to enforce it. It already handles authentication and routing for every request.

Think of a railway reservation counter that allows only a fixed number of forms per person per day. One person can't block the line for everyone else.

What is happening internally?

- **Algorithms** (explained with code in 10.5): fixed window, sliding window log, sliding window counter, token bucket, leaky bucket. A *token bucket*, for example, fills with tokens at a steady rate; each request spends one token; when the bucket is empty, requests are refused.
- **Distributed limiting**: when you have many servers, the counters must be shared. You keep them in Redis. The key is user, API key, IP or tenant + endpoint. You update the counters atomically (in one step, so two servers can't clash):
- `INCR` + `EXPIRE` for a fixed window.
- Sorted sets for a sliding log.
- A Lua script (runs inside Redis in one step) for a token bucket.

The full design is in 13.2. - **API gateway** jobs: routing, authentication (JWT validation), rate limiting, TLS termination (decrypting HTTPS at the edge), changing requests/responses, CORS, observability and canary routing (sending a small share of traffic to a new version). **No business logic.** - **BFF (backend for frontend)**: a separate gateway shaped for each client type — mobile, web or partner.

How does it work?

1. The client calls the API gateway.
2. The gateway validates the JWT and finds tenant = t-42, plan = GOLD.
3. It builds the rate-limit key `t-42:/redemptions` and checks a token bucket in Redis. A Lua script does this in one round trip.
4. If allowed, the request is routed to loyalty-service.
5. If denied, the gateway returns 429 with `Retry-After` and `X-RateLimit-Remaining` headers.

```
Client → API Gateway: validate JWT → tenant = t-42, plan = GOLD
         rate limit key "t-42:/redemptions" → Redis token bucket (Lua, 1 round trip)
         allowed → route to loyalty-service;  denied → 429 + Retry-After + X-RateLimit-Remaining
```

Example

- Plans: FREE gets 10 req/s and GOLD gets 100 req/s, per tenant.
- OTP endpoints: 5 requests/hour per phone number.
- Outbound calls to a partner API: 50 req/s across all pods. Use a shared Redis bucket, or give each of the N pods a share of 50/N.

When would I use it?

- Public and partner APIs.
- Login, OTP and password-reset endpoints, to stop abuse.
- Fairness between tenants in a multi-tenant system.
- Outbound calls to partners that rate-limit you.
- Load shedding during incidents (dropping extra work to stay alive).

What can go wrong?

- **Per-instance limits.** If each server has its own counter, the real limit becomes limit × number of instances.
- **Redis as a hard dependency.** If Redis is down, what happens? Decide per endpoint: fail-open (allow requests) or fail-closed (block them).

- **Hot keys.** A huge tenant puts all its load on one Redis key.
- **No `Retry-After`.** Clients don't know when to try again, so they retry immediately and hammer you.
- **Blocking real bursts.** Limits that are too strict block normal short bursts. A token bucket allows controlled bursts.

Interview questions

Q1. Where would you implement rate limiting? At the API gateway or edge for limits on clients, backed by a shared store like Redis so all instances see the same counts. Also inside your own clients, to respect outbound partner limits.

Q2. What should an API gateway do and not do? It should do routing, auth, rate limiting, TLS, observability and request shaping. It should not contain business logic.

11.17 API design: REST, gRPC, GraphQL and real-time

Why?

> **In simple words:** An API is a promise to other teams and apps about how to talk to your service. Pick the right style for each job, and design it carefully, because changing it later is painful.

APIs live for years. Clients build on them. A badly chosen style or careless design causes years of pain for those clients. Typical mistakes:

- no pagination,
- no versioning,
- write calls that are not idempotent (not safe to retry),
- a different error format on every endpoint.

Interviewers expect you to define clean APIs in your design. They also expect you to pick the right protocol for each interaction.

What is happening internally?

Style	Strengths	Typical use
REST/JSON over HTTP	Works everywhere, can be cached, easy to debug	Public, mobile and most internal APIs
gRPC (HTTP/2 + Protobuf)	Fast binary format, typed contracts, streaming, deadlines	Internal service-to-service calls, low latency
GraphQL	The client picks the fields it wants; one round trip can pull from many sources	Complex UIs that combine many entities
WebSockets	Both sides can send at any time (full-duplex); the connection stays open	Chat, live tracking, collaborative editing
Server-Sent Events	A simple one-way stream from server to client over HTTP	Notifications, LLM token streaming
Long polling	Works everywhere	Fallback for real-time
Webhooks	The server calls the client when an event happens	Payment callbacks, partner integrations

A few words from the table:

- **Protobuf** — Google's compact binary format with a fixed schema.
- **Deadline** — the caller says "I'll wait at most 300 ms", and the whole call chain respects it.
- **Long polling** — the client asks "anything new?", and the server holds the request open until there is something to send.

How does it work?

Use this design checklist:

- **Resources and verbs**: put nouns in paths (`/accounts/{id}/redemptions`). Use HTTP methods for actions. Return the correct status codes (7.2).
- **Idempotency**: add idempotency keys on POSTs that change data (11.14), so retries are safe.
- **Pagination**: for large or changing data, use **cursor/keyset** pagination (`?after=<lastId>&limit=50`). "Give me the next 50 after this ID" stays fast and does not skip or repeat rows. Use offset pagination only for small admin lists.
- **Filtering and sorting**: use query parameters. Put a maximum on the page size.
- **Versioning**: putting the version in the URI (`/v1/`) is explicit and common. Within one version, make only additive (backward-compatible) changes, like adding new optional fields.
- **Errors**: use one consistent format (`ProblemDetail`, 7.4).
- **Security**: auth scopes per endpoint, tenant checks and rate-limit headers.
- **Webhooks**:
- Sign the payloads with HMAC (a shared-secret signature), so receivers know the call is genuine.
- Retry with backoff.
- Make receivers idempotent.
- Allow replay of past events.

Example

- Create a redemption with an idempotency key; get back 201.
- List redemptions with a cursor (`after=R_812`); the response gives the `next` cursor.
- Read the balance.
- Receive a signed partner webhook, idempotent on event ID.
- Call an internal gRPC method with a 300 ms deadline.

```
POST   /v1/accounts/{id}/redemptions        Idempotency-Key    → 201 {id, status}
GET    /v1/accounts/{id}/redemptions?after=R_812&limit=20      → 200 {items:[…], next:"R_792"}
GET    /v1/accounts/{id}/balance                               → 200 {points, asOf}
POST   /webhooks/voucher-partner   X-Signature: sha256=…       → 200 (idempotent on event id)
gRPC   LoyaltyService.Accrue(AccrueRequest) returns (AccrueResponse)   // internal, deadline 300 ms
```

When would I use it?

Use this in the API step of every design:

- **REST** — the default.
- **gRPC** — between internal services with tight latency budgets.
- **WebSockets/SSE** — for live updates.
- **GraphQL** — when many clients need different shapes of the same data.

What can go wrong?

- **Breaking changes without versioning.** Old clients stop working.
- **Chatty APIs.** One screen needs N separate calls.
- **Offset pagination on huge tables.** The database must skip millions of rows, so it gets slow.
- **Leaking internal models.** Your DB structure becomes the public contract and can't change.
- **Webhooks without signatures or retries.** Anyone can fake them, and failed ones are lost.
- **GraphQL without query cost limits.** A deeply nested query can be very expensive.

Interview questions

Q1. REST vs gRPC? REST/JSON works everywhere, humans can read it, and it can be cached. gRPC is a typed, binary protocol on HTTP/2 with streaming and lower latency. So gRPC fits better for internal service-to-service calls.

Q2. How do you paginate a large, changing dataset? Use keyset/cursor pagination on an indexed, unique ordering column. Return an opaque "next" cursor that the client sends back for the next page.

Q3. How do you design reliable webhooks? Sign the payloads, retry with backoff, make receivers idempotent by keying on the event ID, and give a replay endpoint or dashboard.

11.18 Storage for files, search and analytics

Why?

> **In simple words:** Your main database is good at small, quick reads and writes. Big files, text search and huge reports each need their own special store, filled with copies from the main database.

Not all data belongs in your OLTP database — the one that handles day-to-day transactions. Here is why:

- Large files make it bloated.
- Full-text search is slow in SQL.
- Analytical queries that scan billions of rows slow down normal transactions.

So you use a specialised store for each job. Each one is fed from the *source of truth* — the main database that holds the correct data.

What is happening internally?

- **Object storage** (S3, Azure Blob):
- Cheap and very durable (11 nines — 99.999999999%).
- Holds unlimited files, each fetched by a key (like a file path).
- You can't edit a file in place; you replace it.
- Clients upload **directly** to storage using **pre-signed URLs**. A pre-signed URL is a temporary link that allows one specific upload and expires soon. Because of this, large files never pass through your app servers.
- The file's metadata (name, owner, key) lives in the DB.
- Files are delivered to users through a CDN.
- **Search engines** (Elasticsearch/OpenSearch):

- They use an **inverted index**: a map from each word to the list of documents that contain it — like the index at the back of a book.
- *Analysers* break text into words and clean them up: lowercasing, stemming (running → run), synonyms and transliteration.
- Results are ranked by relevance using BM25, a scoring formula.
- Indexing is near-real-time — new data is searchable within about a second.
- The search engine is fed by events or CDC. So it is **eventually consistent** (it catches up a little later) and is never the source of truth.
- **Analytics** (ClickHouse, BigQuery, Snowflake, Trino over a data lake):
- Data is stored by column, not by row (*columnar*), and compressed. It is processed in large chunks at once (*vectorised execution*). This makes big scans fast.
- It is fed by CDC/streams or by batch ETL (scheduled extract-transform-load jobs).
- Two common designs: **Lambda** architecture has a batch layer plus a fast "speed" layer; **Kappa** uses only streams.

Think of a library. The books (files) sit on shelves (object storage). The catalogue cards (DB rows) say which shelf. The subject index (search engine) helps you find books by word. The yearly statistics report (analytics) is made in a separate office so the front desk is not disturbed.

How does it work?

1. **Upload**: the app asks the API for an upload. The API returns a pre-signed PUT URL. The app uploads straight to Blob. An event triggers processing, and a metadata row is saved.
2. **Search**: Postgres changes go through Debezium CDC into Kafka. An indexer writes them to Elasticsearch. The search API reads from Elasticsearch.
3. **Analytics**: data flows from Postgres/Kafka into ClickHouse (MergeTree tables, materialised views). Dashboards read from ClickHouse.

```
Upload: app → POST /uploads → API returns pre-signed PUT URL → app uploads to Blob → event → processing → metadata row
Search: Postgres → Debezium CDC → Kafka → indexer → Elasticsearch ← search API
Analytics: Postgres/Kafka → ClickHouse (MergeTree, materialised views) ← dashboards
```

Example

Perfect Store audits:

1. Photos (2–5 MB each) go straight to Azure Blob through pre-signed URLs.
2. The audit row stores only the blob keys.
3. An AI verification worker processes the photos.
4. Compliance dashboards per region come from ClickHouse, not from the transactional database.

When would I use it?

- **Object storage** — for any user-uploaded media or documents.
- **Search engine** — for product, outlet or text search.
- **OLAP** (analytics store) — for dashboards and reports over a large history.

What can go wrong?

- **Uploads through app servers.** They use up memory and hit timeouts.
- **Public buckets.** Anyone can read the files, so data leaks.

- **Search index drift.** The index slowly differs from the source. You need reindexing and reconciliation (comparing and fixing).
- **Analytics on the OLTP primary.** Heavy reports slow down live transactions.

Interview questions

Q1. How would you handle large file uploads? Give the client a pre-signed URL so it uploads directly to object storage, using multipart/resumable upload for big files. Keep metadata in the DB, start processing from storage events, and deliver files through a CDN.

Q2. Why not use the database for full-text search? A search engine has inverted indexes, analysers and relevance scoring. It handles fuzzy, multilingual, ranked search at scale far better than SQL. Feed it via CDC, and keep the DB as the source of truth.

11.19 Reliability patterns

Why?

> **In simple words:** In a big system, something is always broken somewhere. Good design expects this, and makes sure one broken part can't take down everything else.

At scale, something is always failing — a pod, a zone, a dependency or a deploy. Reliable systems assume failure will happen. They limit its *blast radius* — how much of the system one failure can damage. Interviewers expect you to discuss how each component you draw can fail.

Think of a building with fuses on each floor. A short circuit on one floor trips that fuse only. The rest of the building keeps its lights on.

What is happening internally?

- **Timeouts**: put a timeout on every network call. Each timeout must be shorter than the caller's own timeout, so the whole chain fits in its *timeout budget*.
- **Retries**:
- Use exponential backoff (wait longer each time) with **jitter** (random spread, so clients don't retry together).
- Retry only idempotent operations and *transient* errors (temporary ones, like a timeout).
- Use retry budgets to avoid retry storms.
- **Circuit breakers**: if a dependency is unhealthy, stop calling it for a while and fail fast (8.7).
- **Bulkheads**: give each dependency its own thread or connection pool. Then one slow dependency can't use up everything. (The name comes from the walls inside a ship that stop water spreading.)
- **Graceful degradation**: serve stale or cached data, or a smaller set of features. For example, hide recommendations but keep checkout working.
- **Load shedding**: under overload, reject low-priority work early (429/503). This protects the core flows.
- **Redundancy**: run N+1 instances (one more than you need) across availability zones. For critical paths, use multi-region *active-passive* (one region on standby) or *active-active* (both serve traffic).
- **Backups and DR** (disaster recovery):
- **RPO** (recovery point objective) — how much data you can afford to lose.
- **RTO** (recovery time objective) — how fast you must be back up.

These two numbers decide your replication and backup design. Test restores regularly. - **Health checks**: liveness ("is it alive?") vs readiness ("can it take traffic?") (8.9). - **Safe deployments**: - Rolling, blue-green or canary releases, with automatic rollback if the SLO is breached. - Feature flags. - Backward-compatible schema changes (7.16). - **Coordination**: leader election and distributed locks use etcd, ZooKeeper or Kubernetes leases. Always add **fencing tokens**. A fencing token is a number that goes up with every new lock. If an old lock holder was paused and wakes up with a stale lock, its writes carry an old number and get rejected. - **Clocks**: machine wall clocks drift apart. So don't order distributed events by timestamps alone. Use sequence numbers, partitions or logical clocks (counters that only go up).

How does it work?

For each box in your diagram, ask three questions:

1. What if it is slow?
2. What if it is down?
3. What if it is wrong (bad data, duplicate messages)?

Then add the matching pattern and the alert for each answer.

Example

The voucher partner is down for 20 minutes during a festival campaign. Here is what happens:

1. The circuit breaker opens after 50% of calls fail.
2. Redemptions are still accepted and points are debited.
3. The saga parks the voucher step for a later retry. The user sees "voucher arriving shortly".
4. The bulkhead keeps the other APIs responsive.
5. The breaker goes *half-open* (lets a few test calls through). When they succeed, the backlog drains with rate limiting.
6. Alerts fire when the breaker opens and when the saga backlog gets too old.

When would I use it?

In the "failure handling" step of every design, and in production readiness reviews.

What can go wrong?

- **Retry storms.** Everyone retries at once and makes the outage worse.
- **Silent fallbacks.** Fallbacks hide failures, and no alert fires.
- **Untested backups.** You find out they don't restore only during a disaster.
- **Single-AZ deployments.** One zone outage takes everything down.
- **Distributed locks without fencing.** A stale lock holder can still write.
- **Deploys that can't roll back.** A schema change makes the old version incompatible.

Interview questions

Q1. How do you prevent cascading failures? Use timeouts, limited retries with backoff and jitter, circuit breakers, bulkheads, load shedding and graceful degradation. Together they stop one failing part from pulling down the services that call it.

Q2. What are RPO and RTO? RPO is the maximum data loss you can accept, measured in time. RTO is the maximum time you can take to restore service. Together they decide your replication, backup and

failover design.

Q3. Why do distributed locks need fencing tokens? A client can pause (because of GC or the network) and its lock can expire while another client takes the lock. A token that always increases lets the resource reject writes from the old, stale holder.

11.20 Observability, SLIs and SLOs

Why?

> **In simple words:** Observability means you can see what your system is doing from the outside. SLOs turn "is it healthy?" into a clear number that everyone agrees on.

You can't run what you can't see. And "is the system healthy?" needs an exact definition, based on what users experience.

An SLO (service level objective) turns reliability into a measurable target. That target then guides your alerts and your release decisions.

Think of a car dashboard. The speedometer and fuel gauge are your metrics. The rule "keep the speed under 80" is your objective.

What is happening internally?

- **Metrics** (Prometheus/Grafana, Azure Monitor): numbers measured over time.
- **RED** per service: rate (requests/sec), errors, duration.
- **USE** per resource: utilisation, saturation (how much work is waiting), errors.
- Business KPIs.
- **Logs** (ELK/OpenSearch/Loki): structured JSON, with trace, tenant and request IDs in every line.
- **Traces** (OpenTelemetry → Jaeger/Tempo/App Insights): one request's journey across services, queues and DB calls, split into timed steps called *spans*.
- **SLI** (service level indicator): what you measure. For example, "% of redemption requests completing successfully in < 500 ms".
- **SLO** (objective): the target. For example, 99.9% over 30 days.
- **Error budget**: 100% − SLO. Here that is 0.1%, which is about 43 minutes of "bad" per month. If the budget is burning fast, slow down releases and fix reliability.
- **SLA** (service level agreement): the promise in the customer contract, with penalties. It is looser than the internal SLO, so you get warned before you break it.
- **Alerting on burn rate**: page someone when the error budget is being used up fast. For example, 2% of the monthly budget in 1 hour. Don't page on every CPU spike.

How does it work?

1. Instrument the code once (Micrometer/OpenTelemetry).
2. Build a dashboard for each service.
3. Define 2–4 SLOs for each critical user journey.
4. Alert on symptoms (what users feel), not on causes.
5. Link every alert to a runbook (a step-by-step fix guide).

Example

Loyalty SLOs:

- Redemption availability: 99.9%.
- Redemption p99 latency: < 800 ms (99% of requests finish faster than this).
- Accrual freshness: 99% of order events appear in balances within 60 s. You measure this from consumer lag and event timestamps.

When would I use it?

From the first deploy of any service. Also in the "monitoring" step of every design interview.

What can go wrong?

- **Alert fatigue.** Alerts on causes (like CPU) fire all the time, so people start ignoring them.
- **Dashboards without SLOs.** Lots of charts, but no clear "good or bad" line.
- **High-cardinality metrics cost.** Labels with too many unique values (like user ID) make the metrics store huge and expensive.
- **Sampling away the traces you need.** Use *tail-based sampling* — decide what to keep after the request ends, so errors and slow requests are always kept.

Interview questions

Q1. SLI vs SLO vs SLA? The SLI is the measured indicator. The SLO is the internal target for it. The SLA is the external promise in the customer contract.

Q2. What is an error budget used for? It puts a number on how much unreliability is allowed. When it burns fast, alerts fire and feature releases slow down so the team can focus on reliability work.

11.21 Security in system design

Why?

> **In simple words:** Security means only the right person can do the right thing on the right data. In a multi-tenant system, the biggest job is making sure one company can never see another company's data.

Security failures are the most expensive kind. They cause data breaches, fraud, regulatory penalties and lost customer trust.

In a multi-tenant SaaS, many customer companies (tenants) share the same system. So one authorisation bug can show one brand's data to another brand. That is why designs are expected to cover security explicitly.

Think of a bank locker room. The guard checks your ID at the door (authentication). Your key opens only your own locker (authorisation). Every visit is written in the register (audit).

What is happening internally?

- **Authentication** (who are you?): OAuth2/OIDC with an identity provider. JWT access tokens are validated at the gateway and in the services (8.2).
- **Authorisation** (what can you do?): check roles and scopes. Also check **resource- and tenant-level** access on every query. The most common real-world vulnerability is broken access control —

OWASP #1.

- **Transport**: TLS everywhere. Use mTLS (both sides show certificates) between services, often through a service mesh.
- **Secrets**: keep them in a vault (Azure Key Vault, HashiCorp Vault) and rotate them. Never put them in code or images.
- **Data**:
- Encrypt data at rest.
- For sensitive data, use field-level encryption or tokenisation (replace the real value with a stand-in token). This covers PII (personal data) and payment data. For cards, use a PSP (payment service provider).
- Keep as little PII as possible, and mask it in logs.
- **Input**:
- Validate all input.
- Use parameterised queries (stops SQL injection).
- Encode output (stops XSS — cross-site scripting).
- Protect URL fetches against SSRF (tricking your server into calling internal addresses).
- **Edge**: WAF (web application firewall), DDoS protection and rate limits (11.16).
- **Audit**: keep immutable (unchangeable) audit logs for sensitive actions, like admin adjustments and payouts. Add maker-checker approvals: one person makes the change, a second person approves it.
- **Webhooks and partners**: signed requests, IP allow-lists and idempotency.

How does it work?

Threat-model each flow. That means asking:

1. Who can call it?
2. With which identity?
3. Which tenant's data does it touch?
4. What could an attacker replay or forge?
5. What gets logged?

Example

Admin points adjustment:

1. It needs `ROLE_ADMIN` and the same tenant claim as the account.
2. It creates a pending adjustment (the maker).
3. A second admin approves it (the checker).
4. The ledger entry records both identities.
5. The action is audited. If it is above a threshold, an alert fires.

When would I use it?

In the API and data steps of every design. Also when reviewing any feature that touches money, PII or cross-tenant data.

What can go wrong?

- **Trusting client-supplied tenant or user IDs.** The client can simply change them.

- **Secrets in Git.** Anyone with repo access, or a leak, gets them.
- **Verbose errors.** Error messages leak internal details.
- **Logging tokens or PII.** Logs become a security risk.
- **No rate limits on OTP/login.** Attackers can guess codes or passwords.
- **Overly broad service credentials.** One stolen credential opens too much.

Interview questions

Q1. How do you ensure tenant isolation in a multi-tenant system? Take the tenant from the authenticated token, never from the request body. Enforce it in authorisation checks, and apply mandatory tenant filters at the data layer — or use separate schemas/databases for strong isolation. Add tests that try cross-tenant access.

Q2. What security measures would you add to a payments or points system? Strong auth, idempotency, maker-checker for manual adjustments, audit logs, encryption, rate limits, anomaly alerts and reconciliation.

11.22 Generating unique IDs at scale

Why?

> **In simple words:** When many servers create records at the same time, each record still needs a unique ID. The trick is to make IDs unique without all servers asking one central place every time.

Orders, messages, events and short links all need unique IDs. There are two simple options, and both have problems:

- **One auto-increment database sequence.** With many shards and regions, everyone waits on this one counter. It becomes a bottleneck and a single point of failure (SPOF).
- **Random UUIDs.** They are unique, but large and unordered. That is bad for B-tree index locality (new rows land all over the index) and for sorting by time.

Think of restaurant token numbers. If each counter prints tokens starting with its own counter number, two counters can never give out the same token — and they don't need to talk to each other.

What is happening internally?

Approach	Properties
DB auto-increment / sequence	Simple and ordered; a bottleneck/SPOF across shards
UUIDv4 (random)	No coordination needed; 128 bits; random → poor index locality
UUIDv7	Starts with a timestamp, then randomness; no coordination; friendly to indexes
Snowflake (Twitter)	64-bit: 1 sign bit + 41-bit millisecond timestamp (~69 years) + 10-bit machine ID (1,024 nodes) + 12-bit sequence (4,096 IDs per ms per node) → roughly sorted by time, compact
Range allocation	Each service instance leases a block (e.g., 1M IDs) from a coordinator and hands them out locally

How does it work?

Each node makes Snowflake IDs like this:

1. On each request, read the current millisecond.
2. If it is the same millisecond as last time, add 1 to the sequence.
3. If the sequence goes past 4,095 (overflows), wait for the next millisecond.
4. If it is a new millisecond, reset the sequence to 0.
5. Combine the parts: `(timestamp - epoch) << 22 | machineId << 12 | sequence`.

The *epoch* is a fixed starting date. Subtracting it keeps the timestamp small enough to fit in 41 bits.

Example

```
final class Snowflake {
    private static final long EPOCH = 1_735_689_600_000L;   // 2025-01-01
    private final long machineId; private long lastMs = -1, seq = 0;
    Snowflake(long machineId) { if (machineId < 0 || machineId > 1023) throw new IllegalArgumentException();
this.machineId = machineId; }
    synchronized long next() {
        long now = System.currentTimeMillis();
        if (now < lastMs) throw new IllegalStateException("clock moved backwards");
        if (now == lastMs) {
            seq = (seq + 1) & 4095;
            if (seq == 0) while ((now = System.currentTimeMillis()) <= lastMs) { }   // sequence exhausted:
wait for the next ms
        } else seq = 0;
        lastMs = now;
        return ((now - EPOCH) << 22) | (machineId << 12) | seq;
    }
}
```

When would I use it?

- **Snowflake-style or UUIDv7 IDs** — for high-volume entities across shards: orders, events, messages, tweets (13.17).
- **Range allocation** — for short-link codes (13.1).
- **DB sequences** — when one database is enough.

What can go wrong?

- **Duplicate machine IDs.** Two nodes with the same ID can create the same IDs. Assign machine IDs through config, a coordinator or Kubernetes StatefulSet ordinals (each pod gets a fixed number).
- **Clock moving backwards.** NTP (time sync) can move the clock back. Refuse to create IDs, or wait until the clock catches up.
- **Exposing sequential IDs publicly.** Attackers can guess other IDs by counting (enumeration attacks). Use opaque public IDs instead.

Interview questions

Q1. How would you generate unique IDs across many servers? Use Snowflake-style IDs (timestamp + machine ID + sequence) or UUIDv7. Both are time-ordered and need no coordination. Use range leasing when IDs must be dense or short.

Q2. Why are random UUIDs bad as primary keys in B-tree indexes? Random inserts land all over the index. This causes page splits, poor cache locality and larger indexes. Time-ordered IDs are added near the end of the index instead.

12. Databases and SQL Internals

You already write SQL every day on SQL Server, PostgreSQL, ClickHouse and Trino. So this chapter skips the syntax basics. It focuses on what SDE2 interviews actually test:

- How the database runs a query.
- Why an index is used — or ignored.
- What happens when many transactions run at the same time.
- The SQL you are asked to write live: joins, window functions and CTEs (named sub-steps of a query).

Examples use PostgreSQL syntax. Where SQL Server is different, the difference is noted.

This sample schema is used in the whole chapter:

```
CREATE TABLE outlet (id BIGINT PRIMARY KEY, tenant_id INT NOT NULL, city TEXT, created_at TIMESTAMPTZ);
CREATE TABLE rep    (id BIGINT PRIMARY KEY, tenant_id INT NOT NULL, name TEXT, manager_id BIGINT REFERENCES
rep(id));
CREATE TABLE orders (id BIGINT PRIMARY KEY, tenant_id INT NOT NULL, outlet_id BIGINT REFERENCES outlet(id),
                     rep_id BIGINT REFERENCES rep(id), amount NUMERIC(12,2), status TEXT, created_at
TIMESTAMPTZ);
```

12.1 Joins and how the engine executes them

Why?

In simple words: A join stitches rows from two tables together using a matching column. Picking the right join gives the right answer. Knowing how the database runs the join tells you why the same query is fast today and slow tomorrow.

Real questions almost always need more than one table. For example:

- "Outlets that never placed an order."
- "Each rep with their manager."
- "Revenue per city."

If you choose the right join type, you get the correct answer. If you also know how the engine *executes* the join, you can explain performance changes. The same query can be fast one day and slow the next, because the database picked a different way to run it.

What is happening internally?

There are two layers. First, the **logical** join type — *what* rows you want back:

Join	Returns
`INNER JOIN`	Only rows that have a match on both sides
`LEFT JOIN`	All left rows; the right-side columns are NULL where there is no match
`FULL OUTER JOIN`	All rows from both sides, with NULLs where there is no match
`CROSS JOIN`	Every left row paired with every right row (Cartesian product)
Self join	A table joined to itself (rep → manager)

Join	Returns
Semi join (`EXISTS` , `IN`)	Left rows that have at least one match — each left row appears once, not duplicated
Anti join (`NOT EXISTS`)	Left rows that have no match at all

Second, the **physical** algorithm — *how* the database actually does the matching. A component called the **query planner** picks the algorithm. It decides using table **statistics**: stored facts like row counts and how values are spread.

Algorithm	How it works	Best when
Nested loop	For each row on the outer side, look up the matching inner rows (ideally using an index)	The outer input is small and the inner side has an index
Hash join	Build a hash table from the smaller input, then check each row of the larger input against it	Both inputs are large and unsorted, and the join uses equality (`=`)
Merge join	Both inputs are sorted on the join key; walk through them side by side	Both inputs are already sorted (for example, by indexes), or large range joins

Think of a nested loop like a shopkeeper checking each customer's name in a register. It is fast when there are only a few customers and the register has an alphabetical index. It is very slow for 10 lakh customers.

How does it work?

The planner works like this:

1. It estimates how many rows each step will produce.
2. It tries different join orders and different algorithms.
3. It gives each option a cost.
4. It picks the cheapest plan.

The plan is only as good as the estimates. Estimates go wrong when statistics are old (*stale*), or when two columns are related in a way the planner does not know (*correlated columns*). Then the planner makes a bad choice. The most common bad choice is a nested loop that runs over far more rows than it expected.

Example

```
-- Anti join: outlets that never ordered. Prefer NOT EXISTS to NOT IN (NOT IN returns nothing if the subquery
yields a NULL)
SELECT o.id, o.city
FROM outlet o
WHERE NOT EXISTS (SELECT 1 FROM orders x WHERE x.outlet_id = o.id);

-- Self join: each rep with their manager (LEFT keeps reps with no manager)
SELECT r.name AS rep, m.name AS manager
FROM rep r LEFT JOIN rep m ON m.id = r.manager_id;

-- The LEFT JOIN trap: a WHERE filter on the right table turns it into an INNER JOIN
SELECT o.id, COUNT(x.id)
FROM outlet o LEFT JOIN orders x ON x.outlet_id = o.id
WHERE x.status = 'DELIVERED'            -- outlets with no orders disappear (x.status is NULL)
GROUP BY o.id;
-- Fix: move the condition into ON
SELECT o.id, COUNT(x.id)
```

```
FROM outlet o LEFT JOIN orders x ON x.outlet_id = o.id AND x.status = 'DELIVERED'
GROUP BY o.id;
```

When would I use it?

In every reporting query and API query that needs more than one table. Interviews often ask you to write anti joins, self joins and LEFT JOIN aggregations live, on the spot.

What can go wrong?

- **NOT IN with NULLs.** If the subquery returns even one NULL, NOT IN returns no rows at all.
- **LEFT JOIN filter in WHERE.** A WHERE condition on the right table quietly turns the LEFT JOIN into an INNER JOIN.
- **Accidental Cartesian product.** You forget the join condition, so every row pairs with every other row.
- **Duplicate rows from a one-to-many join before aggregating.** One outlet with ten orders becomes ten rows, so sums get multiplied. Aggregate first, then join.
- **Stale statistics.** The planner thinks a table is small and picks a nested loop over millions of rows. Refresh statistics with `ANALYZE` in PostgreSQL or `UPDATE STATISTICS` in SQL Server.

Interview questions

Q1. Why NOT EXISTS instead of NOT IN? If the subquery returns even one NULL, NOT IN returns no rows at all. NOT EXISTS gives the correct result in that case, and the database usually runs it as an efficient anti join.

Q2. Nested loop vs hash vs merge join? Use a nested loop when the outer input is small and the inner side has an index. A hash join suits large, unsorted inputs joined on equality. A merge join is best when both inputs are already sorted on the join key.

Q3. Why does a LEFT JOIN sometimes behave like an INNER JOIN? For unmatched rows, the right table's columns are NULL. A WHERE condition on those columns removes exactly those rows. To keep them, move the condition into the ON clause.

12.2 Window functions

Why?

> **In simple words:** A window function lets each row look at its neighbours — rows in the same group — and calculate something like a rank or a running total. Unlike GROUP BY, it does not squash the rows together; every row stays in the result.

Many questions need a result on each row that depends on other rows. For example:

- The rank of a rep within their team.
- A running total of sales.
- The previous month's value.
- The top N items in each category.

`GROUP BY` does not help here, because it collapses many rows into one. Self joins and correlated subqueries (a subquery that runs once per outer row) work, but they are slow and messy.

Window functions solve this. They calculate over a "window" of related rows **while keeping every row**. They come up again and again in live SQL rounds.

What is happening internally?

The general shape is:

```
function(...) OVER (PARTITION BY <group> ORDER BY <order> [ROWS|RANGE BETWEEN ... AND ...])
```

What the engine does:

1. It sorts (or hashes) the rows by the partition key and the order key.
2. For each row, it works out the *frame* — the set of rows the function should look at.
3. It computes the function for that row over its frame.

When windows run: they are evaluated **after** WHERE, GROUP BY and HAVING, but **before** ORDER BY and LIMIT. Because of this, you cannot filter on a window result in the WHERE of the same query. Wrap the query in a subquery or CTE, and filter outside.

The main function families:

- **Ranking:** `ROW_NUMBER` (1,2,3,4 — always unique), `RANK` (1,2,2,4 — ties share a rank, then a gap), `DENSE_RANK` (1,2,2,3 — ties share a rank, no gap), `NTILE(n)` (splits rows into n equal buckets).
- **Offsets:** `LAG` (previous row's value), `LEAD` (next row's value), `FIRST_VALUE`, `LAST_VALUE`.
- **Aggregates over windows:** `SUM`, `AVG`, `COUNT`, `MIN`, `MAX`.
- **Frame default:** if you write ORDER BY but no frame, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. `RANGE` treats rows with equal ORDER BY values (ties) as one group of "peers". So in a running total, all tied rows get the same total — which surprises people. To avoid this, write `ROWS` explicitly.

Think of a class result sheet. Each student's row stays on the sheet, but next to it you write their rank within their own section. That is a window function with `PARTITION BY section ORDER BY marks DESC`.

How does it work?

Here, R2 and R3 tie at 700. `DENSE_RANK` gives both rank 2, and the next rep gets 3 (no gap):

```
monthly_sales (tenant 7, Oct):   rep  sales   DENSE_RANK() OVER (PARTITION BY tenant, month ORDER BY sales
DESC)
                                  R1   900     1
                                  R2   700     2
                                  R3   700     2
                                  R4   500     3
```

Example

```
-- 1. Top 3 reps per tenant per month (the classic)
SELECT * FROM (
  SELECT tenant_id, month, rep_id, sales,
         DENSE_RANK() OVER (PARTITION BY tenant_id, month ORDER BY sales DESC) AS r
  FROM monthly_sales) t
WHERE r <= 3;

-- 2. Running total and 7-day moving average per outlet
SELECT outlet_id, order_date, amount,
       SUM(amount) OVER (PARTITION BY outlet_id ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW) AS running_total,
       AVG(amount) OVER (PARTITION BY outlet_id ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
```

```
AS avg_7d
FROM daily_outlet_sales;

-- 3. Month-over-month growth
SELECT month, revenue,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
             / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 2) AS mom_pct
FROM monthly_revenue;

-- 4. Each outlet's first order (dedupe: keep one row per group)
SELECT * FROM (
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY outlet_id ORDER BY created_at) AS rn FROM orders o) t
WHERE rn = 1;

-- 5. Share of tenant revenue per outlet (window over an aggregate)
SELECT tenant_id, outlet_id, SUM(amount) AS rev,
       100.0 * SUM(amount) / SUM(SUM(amount)) OVER (PARTITION BY tenant_id) AS pct_of_tenant
FROM orders GROUP BY tenant_id, outlet_id;

-- 6. Gaps and islands: reps who visited on 5+ consecutive days
SELECT rep_id, MIN(visit_date) AS streak_start, MAX(visit_date) AS streak_end, COUNT(*) AS days
FROM (
  SELECT rep_id, visit_date,
         visit_date - (ROW_NUMBER() OVER (PARTITION BY rep_id ORDER BY visit_date))::int AS grp
  FROM (SELECT DISTINCT rep_id, visit_date FROM visits) v
) s
GROUP BY rep_id, grp
HAVING COUNT(*) >= 5;
-- consecutive dates minus their row number give the same value → that value identifies each streak
```

For example 6, the trick is simple. Consecutive dates go up by one each day. Row numbers also go up by one each row. So date minus row number stays the same along a streak. When a day is missed, the value jumps. That constant value labels each streak, and you group by it.

When would I use it?

- Leaderboards and rankings (incentive standings).
- Deduplication — keeping one row per group.
- Period-over-period comparisons (this month vs last month).
- Running balances and account statements.
- Streaks and sessionisation (grouping events into sessions).
- Percent-of-total reports.

What can go wrong?

- **Filtering on a window column in WHERE.** This is not allowed. Wrap the query in a subquery and filter outside.
- `LAST_VALUE` **returns the current row.** This happens because the default frame ends at the current row. Set the frame to `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.
- `RANGE` **vs** `ROWS` **surprises with ties.** Tied rows get the same running total under `RANGE`.
- **Huge sorts spill to disk on big partitions.** The sort no longer fits in memory and becomes slow. An index on the partition and order keys helps.

Interview questions

Q1. ROW_NUMBER vs RANK vs DENSE_RANK? ROW_NUMBER gives a unique sequence (1,2,3,4). RANK lets ties share a rank and then leaves a gap (1,2,2,4). DENSE_RANK lets ties share a rank with no gap (1,2,2,3).

Q2. How do you get the top N rows per group? In a subquery or CTE, rank rows with `PARTITION BY group ORDER BY metric DESC`. Then, in the outer query, keep only rows where the rank is N or less.

Q3. How do you compute a running total? Use `SUM(x) OVER (PARTITION BY key ORDER BY t ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`. Write `ROWS` explicitly so tied values don't get the same total.

12.3 CTEs and recursive queries

Why?

> **In simple words:** A CTE gives a name to one step of a big query, so you can read the query top to bottom like a recipe. A recursive CTE repeats a step again and again — perfect for walking down a manager → team tree.

Long queries with subqueries nested inside subqueries are hard to read and hard to debug. A **common table expression (CTE)** gives each step a name. Then the query reads from top to bottom, step by step.

A **recursive CTE** goes further. It solves hierarchy problems — manager chains, category trees, distributor networks. Without recursion, you cannot solve these in a single query, because you don't know in advance how many levels deep the tree goes.

What is happening internally?

- **A normal CTE** (`WITH name AS (…)`) defines a named subquery that lives only for that one statement.
- PostgreSQL 12 and later **inline** CTEs. This means the CTE is copied into the main query and optimised like a normal subquery. The exception is when you mark it `MATERIALIZED` — then it is computed once and stored.
- SQL Server always inlines CTEs.
- So a CTE is a readability tool, not a performance tool.
- **A recursive CTE** has two parts joined by `UNION ALL`:
- An **anchor** member — the starting rows.
- A **recursive** member — a query that joins the CTE back to the table to find the next level.

The engine runs it in a loop:

1. Run the anchor to get the first set of rows.
2. Feed those rows into the recursive member to get the next level.
3. Feed the new rows in again.
4. Stop when a step returns no new rows.

Think of a family tree. You start with one grandparent (the anchor). Then you find their children, then the children's children, and so on — until a generation has no children.

How does it work?

```
iteration 0 (anchor):    direct reports of 42          → {7, 9}
iteration 1:             reports of {7, 9}            → {15, 16, 21}
iteration 2:             reports of {15, 16, 21}      → {}   → stop
```

Example

```sql
-- Readable multi-step report
WITH delivered AS (
  SELECT rep_id, date_trunc('month', created_at) AS month, SUM(amount) AS sales
  FROM orders WHERE status = 'DELIVERED' GROUP BY 1, 2
),
ranked AS (
  SELECT *, RANK() OVER (PARTITION BY month ORDER BY sales DESC) AS r FROM delivered
)
SELECT * FROM ranked WHERE r <= 10;

-- Everyone under manager 42, with depth (sales hierarchy)
WITH RECURSIVE team AS (
  SELECT id, name, manager_id, 1 AS depth FROM rep WHERE manager_id = 42          -- anchor
  UNION ALL
  SELECT r.id, r.name, r.manager_id, t.depth + 1
  FROM rep r JOIN team t ON r.manager_id = t.id                                   -- recursive step
  WHERE t.depth < 20                                                              -- guard against cycles in
bad data
)
SELECT * FROM team ORDER BY depth;
-- SQL Server: identical without the RECURSIVE keyword; limit with OPTION (MAXRECURSION n)
```

When would I use it?

- Breaking complex reports into named steps.
- Organisation and sales hierarchies — for example, rolling incentives up to managers.
- Bill-of-materials (a product made of parts, which are made of smaller parts).
- Graph-like walks that don't go very deep.

What can go wrong?

- **Infinite recursion on cyclic data.** If bad data says A manages B and B manages A, the loop never ends. Add a depth limit, or keep an array of the path already visited and stop when you see a repeat.
- **Assuming CTEs are materialised (or not).** Engines behave differently. Don't assume a CTE is computed once.
- **Very deep or very wide recursion on big tables.** This gets slow. For large hierarchies, consider a *closure table* (a table storing every ancestor–descendant pair) or a *materialised path* (storing the full path like `/1/42/7/` on each row).

Interview questions

Q1. What is a recursive CTE? It is a CTE with two parts: an anchor query that gives the starting rows, and a recursive query that refers back to the CTE. The engine repeats the recursive part until no new rows come out. It is used for hierarchies.

Q2. Do CTEs improve performance? Usually no. Modern PostgreSQL and SQL Server inline them, so they run like normal subqueries. Their real benefit is readability. In PostgreSQL, use `MATERIALIZED` when you deliberately want the CTE computed once.

12.4 Classic SQL interview problems and evaluation order

Why?

> **In simple words:** Live SQL rounds keep asking the same handful of problems. Learn the templates, and learn the order in which SQL runs each clause — then you can write them fast and explain why

> they work.

Live SQL rounds reuse a small set of patterns:

- Nth highest value.
- Finding and deleting duplicates.
- "Did it happen in every period?" (this is called *relational division*).
- Pivots — turning row values into columns.
- Retention — who came back next month.

If you know these patterns, and the logical order in which SQL runs, you can write them quickly and explain them clearly.

What is happening internally?

Logical evaluation order — the order SQL conceptually runs each clause. It explains many alias and filter errors: `FROM/JOIN → WHERE → GROUP BY → HAVING → window functions → SELECT → DISTINCT → ORDER BY → LIMIT/OFFSET`

What this means in practice:

- WHERE runs before SELECT. So WHERE cannot see the aliases you define in SELECT.
- WHERE runs before GROUP BY. So WHERE cannot filter on aggregates like `SUM`. Use HAVING for that.
- Window functions run after WHERE. So you cannot filter on a window result at the same level.

How does it work?

First, recognise which pattern the question is. Then apply that pattern's template from the examples below.

Example

```sql
-- Nth highest salary (N = 2), ties handled
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) r FROM employee) t
WHERE r = 2;

-- Delete duplicate rows by business key, keeping the earliest
DELETE FROM orders o USING (
  SELECT id, ROW_NUMBER() OVER (PARTITION BY tenant_id, external_ref ORDER BY created_at) rn FROM orders) d
WHERE o.id = d.id AND d.rn > 1;

-- Outlets that ordered in every month of 2026 (relational division)
SELECT outlet_id FROM orders
WHERE created_at >= '2026-01-01' AND created_at < '2027-01-01'
GROUP BY outlet_id
HAVING COUNT(DISTINCT date_trunc('month', created_at)) = 12;

-- Pivot with conditional aggregation (works everywhere)
SELECT tenant_id,
       COUNT(*) FILTER (WHERE status = 'DELIVERED') AS delivered,      -- SQL Server: SUM(CASE WHEN
status='DELIVERED' THEN 1 ELSE 0 END)
       COUNT(*) FILTER (WHERE status = 'CANCELLED') AS cancelled
FROM orders GROUP BY tenant_id;

-- Month-on-month retention: active in month M and also in M+1
WITH m AS (SELECT DISTINCT outlet_id, date_trunc('month', created_at) AS mon FROM orders)
SELECT a.mon, COUNT(*) AS active, COUNT(b.outlet_id) AS retained
```

```
FROM m a LEFT JOIN m b ON b.outlet_id = a.outlet_id AND b.mon = a.mon + interval '1 month'
GROUP BY a.mon ORDER BY a.mon;
```

When would I use it?

In SQL rounds, in analytics questions during system-design interviews, and in your daily reporting work.

What can go wrong?

- **Using `LIMIT 1 OFFSET N-1` for Nth highest.** This is wrong when there are ties. Use `DENSE_RANK` instead.
- **`COUNT(col)` vs `COUNT(*)` with NULLs.** `COUNT(col)` skips NULLs; `COUNT(*)` counts every row.
- **Comparing dates using a function on the column.** For example `DATE(created_at) = …`. This stops the index from being used — it is *non-sargable* (see 12.5).
- **Integer division in percentages.** `5 / 10` gives 0 in integer maths. Multiply by `100.0` to force decimal maths.

Interview questions

Q1. WHERE vs HAVING? WHERE filters individual rows before grouping. HAVING filters whole groups after aggregation.

Q2. Why can't I use a SELECT alias in WHERE? In the logical order, WHERE runs before SELECT, so the alias doesn't exist yet. Either repeat the expression in WHERE, or wrap the query and filter outside.

12.5 How indexes work: B+ trees, clustered and covering indexes

Why?

> **In simple words:** An index is like the index at the back of a book. Instead of reading every page to find a word, you look it up and jump straight to the right page. Without it, the database reads the whole table.

Without an index, the database must scan the whole table to find rows. The work grows with table size — O(n) disk pages. With the right index, a lookup reads only 3–4 pages, no matter how big the table is.

Most "the API takes 30 seconds" problems are really index problems. And "why isn't my index being used?" is one of the top SDE2 database questions.

What is happening internally?

Relational database indexes are **B+ trees**. A B+ tree is:

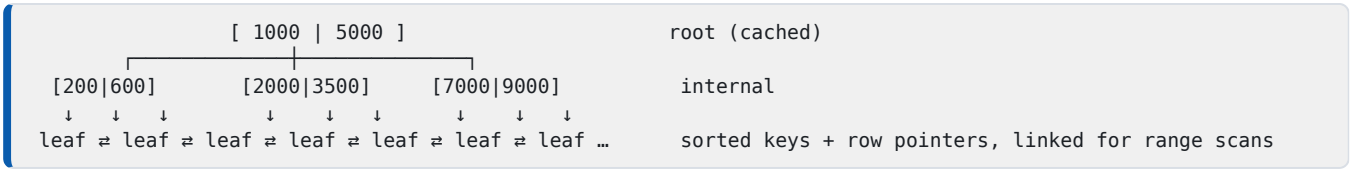
- **Balanced** — every path from the top to the bottom has the same length.
- **Very wide** — each 8 KB page holds hundreds of keys.

It has two kinds of pages:

- **Internal pages** hold *separator keys* (signposts like "keys below 1000 go left") and pointers to child pages.
- **Leaf pages** hold the actual keys in sorted order. **Each leaf is linked to its neighbours.** So a range scan finds the first leaf, then simply walks along the leaves in order.

Because each page fans out to about 500 children, a billion rows need only about 4 levels. The top levels are used so often that they stay in memory.

Think of a library. The signboard at the entrance tells you which floor. The floor sign tells you which shelf. The shelf has books in sorted order, and you can walk along it. Three or four steps, however big the library is.



The main kinds of index:

- **Clustered index** (SQL Server, MySQL InnoDB): the table rows themselves *are* the leaf level. Rows are stored in primary-key order. There can be only one per table.
- Secondary indexes store the primary key, not a direct row address. So a lookup through a secondary index goes: index → primary key → row. SQL Server calls this a *key lookup*.
- Random UUID primary keys land in random places. This causes *page splits* (a full page is split in two to make room) and fragmentation. Sequential or time-ordered keys just add to the end neatly (11.22).
- **Heap + secondary indexes** (PostgreSQL): rows live in an unordered pile called the *heap*. Every index points to a row's physical location, called the TID.
- **Covering index**: an index that contains every column the query needs. So the engine never has to visit the table at all. In PostgreSQL this is an *index-only scan*. In SQL Server you add extra columns with `INCLUDE`.
- **Other index types:**
- **Hash** — equality (`=`) only.
- **GIN** — PostgreSQL JSONB, arrays and full-text search.
- **GiST/BRIN** — GiST for geo data; BRIN for huge tables that are naturally in order (like append-only time data).
- **Partial** — indexes only some rows, e.g. `WHERE status = 'PENDING'`.
- **Unique** — also how you enforce idempotency keys (keys that make a repeated request safe to apply only once).

How does it work?

Composite index column order. A composite index covers several columns. It works like a phone book sorted by (last name, first name). Follow these rules:

1. Put equality-filtered columns first (`tenant_id`, `status`).
2. Then put the range or sort column (`created_at`).
3. **Leftmost-prefix rule:** an index on `(tenant_id, status, created_at)` helps `WHERE tenant_id = ?` and `WHERE tenant_id AND status`. It does **not** help `WHERE status = ?` alone. A phone book sorted by last name can't help you find everyone with first name "Rahul".

Why an index gets ignored:

- **A function or calculation on the column.** `WHERE DATE(created_at) = '2026-10-01'` can't use the index on `created_at`. Rewrite it as a range: `created_at >= '2026-10-01' AND created_at < '2026-10-02'`. A filter that can use an index like this is called "sargable". Or create an expression index on `DATE(created_at)`.

- **An implicit type conversion.** For example, a VARCHAR column compared to a number. In SQL Server, NVARCHAR vs VARCHAR mismatches do the same.
- **A leading wildcard.** `LIKE '%abc'` can't use a normal index, because you don't know the start of the value. Use trigram or full-text indexes.
- **Low selectivity.** If the filter matches roughly 10–20% or more of the rows, reading the whole table in order is cheaper than jumping around for many random lookups.
- **`OR` across different columns.** Rewrite as `UNION ALL`, or rely on the engine's bitmap OR (combining two index results).
- **Stale statistics, or a cached plan built for different parameter values.** In SQL Server, this second one is called *parameter sniffing*: the plan was built for the first parameter value it saw, and is bad for others.

Example

```
CREATE INDEX ix_orders_tenant_status_created
  ON orders (tenant_id, status, created_at DESC) INCLUDE (amount);
-- Serves: WHERE tenant_id = ? AND status = ? ORDER BY created_at DESC LIMIT 50   (no sort needed)
--         SELECT SUM(amount) … same filter                                        (index-only, no table
access)
-- Doesn't serve: WHERE status = 'OPEN'  (tenant_id not constrained → leftmost prefix broken)
```

When would I use it?

- On the filter, join and sort columns of every frequent query.
- Unique indexes for business keys and idempotency.
- Partial indexes for small "hot" subsets — for example, outbox rows not yet published, or pending payments.

What can go wrong?

- **Too many indexes.** Every write must update every index. This costs storage and multiplies write work (*write amplification*).
- **Duplicate or unused indexes.** Check the usage statistics and drop them.
- **Wrong column order** in a composite index.
- **Indexing a low-cardinality column alone**, like `status`. A column with only a few distinct values doesn't narrow things down much.
- **Random UUID keys in clustered indexes.** They cause page splits and fragmentation.

Interview questions

Q1. How does a B+ tree index speed up queries? It keeps keys sorted in a tree that is shallow and very wide. So a lookup reads only about 3–4 pages. Its leaves are linked together, so range scans read them one after another.

Q2. Clustered vs non-clustered index? A clustered index stores the table rows themselves in key order, so there can be only one per table. A non-clustered index is a separate structure that points to the rows.

Q3. Why might the database not use my index? Common reasons: a function or type conversion on the column, a leading wildcard, low selectivity, the wrong composite column order, stale statistics, or a bad cached plan.

Q4. How do you choose composite index column order? Put equality filters first, then range or sort columns. Respect the leftmost-prefix rule. Add INCLUDE columns so the index covers the whole query.

12.6 Query plans and query optimisation

Why?

> **In simple words:** A query plan is the database's step-by-step route for running your query. Reading it shows you exactly where the time went, so you fix the real problem instead of guessing.

Guessing why a query is slow can waste days. The query plan shows exactly what the database did:

- Which indexes it used.
- Which join algorithms it chose.
- How many rows came out of each step.
- Where the time went.

Reading plans is how you turn "it's slow" into a specific fix. It is also the heart of a good performance story in an interview — like your ~30 s → ~10 s API improvement.

What is happening internally?

- `EXPLAIN` shows the plan the optimiser chose. It shows **estimated** rows and costs. It does not run the query.
- `EXPLAIN ANALYZE` actually **runs** the query. It shows the **actual** rows and timings for each step.
- `BUFFERS` adds how many pages were read from memory cache vs from disk.
- In SQL Server, the equivalent is the *actual execution plan*, plus *Query Store* for history over time.

Think of Google Maps. `EXPLAIN` is the route it suggests before you leave. `EXPLAIN ANALYZE` is the record of the trip you actually took, with the time spent on each road.

How does it work?

Look for these clues in the plan:

Plan clue	Meaning	Usual fix
`Seq Scan` on a large table with a selective filter	There is no index the planner can use	Add a composite index that matches the filter + sort
Estimated 10 rows vs actual 500,000	Bad statistics, or columns related in a way the planner doesn't know	`ANALYZE`, extended statistics, or rewrite the query
Nested Loop with a huge outer side	Wrong join choice caused by a bad estimate	Fix the estimates; index the inner side
`Sort … external merge Disk`	The sort did not fit in memory and spilled to disk	An index that already gives the order, or more `work_mem`
`Index Scan` + many heap fetches / SQL Server Key Lookup in a loop	The index does not cover the query, so each row needs a trip to the table	Add `INCLUDE` columns
Large `Rows Removed by Filter`	The index finds rows, but most are thrown away afterwards	Add the filtered column to the index

Optimisation checklist — work through it in order:

1. **Measure first.** Find which query takes most of the time. Use the slow query log, `pg_stat_statements`, SQL Server Query Store, or APM traces.
2. **Read the plan.** Compare estimated rows with actual rows.
3. **Index for the access path.** Cover the filter first, then the sort, then the columns the query returns.
4. **Return less data.** Select only the columns you need. Use keyset pagination (continue from the last seen key, instead of a large OFFSET). Aggregate in the database, not in Java.
5. **Remove N+1 patterns** (7.10) — one query for a list, then one more query per item. Batch them with `IN (...)` or a join.
6. **Pre-aggregate.** Use materialised views, summary tables, or ClickHouse for analytics.
7. **Partition** big tables so queries skip irrelevant partitions (12.8).
8. **Cache** results that don't change often. Move heavy reports to read replicas or an OLAP system.
9. **Verify** the result using the same metrics after the change.

Example

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT outlet_id, SUM(amount)
FROM orders
WHERE tenant_id = 7 AND created_at >= now() - interval '30 days'
GROUP BY outlet_id;

-- Before: Seq Scan on orders (actual rows 9,800,000 scanned, 120,000 kept) … 8,400 ms
-- Add:    CREATE INDEX CONCURRENTLY ix_orders_tenant_created ON orders (tenant_id, created_at) INCLUDE
(outlet_id, amount);
-- After:  Index Only Scan using ix_orders_tenant_created (actual rows 120,000) … 95 ms
```

> **Say this:** "The incentive dashboard API took ~30 seconds. APM showed 90% of the time in one aggregation query. The plan was a sequential scan over the whole orders table because we filtered on `DATE(created_at)`. I rewrote the predicate as a range, added a composite covering index on (tenant, created_at) including the aggregated columns, and moved one heavy breakdown to a pre-aggregated table. The endpoint went to ~10 seconds and the DB CPU dropped." Adapt this to what you actually did — only claim the steps you took.

When would I use it?

- Every slow-endpoint investigation.
- Every new query on a big table — check its plan before you ship it.
- Design reviews of reporting features.

What can go wrong?

- **Testing plans on tiny dev data.** On production volumes, the planner chooses differently.
- **Adding indexes blindly.** Each index adds write cost.
- **Running `EXPLAIN ANALYZE` on destructive statements in production.** It really executes them — an UPDATE or DELETE will change data. Wrap it in a transaction and roll back.
- **Fixing one query while the real problem is the number of queries.** For example, an N+1 pattern firing hundreds of fast queries.

Interview questions

Q1. How would you find and fix a slow query? First find it, using the slow log, Query Store or APM. Run EXPLAIN ANALYZE and compare estimated rows with actual rows. Then fix indexes or rewrite the filters, return less data, and remove N+1 patterns. If needed, pre-aggregate or partition. Finally, confirm the improvement with metrics.

Q2. EXPLAIN vs EXPLAIN ANALYZE? EXPLAIN shows the planned strategy with estimates only. EXPLAIN ANALYZE actually runs the query and reports the real rows and timings for each step.

12.7 Transactions, MVCC, locks and database deadlocks

Why?

> **In simple words:** Many users change the same data at the same time. The database must keep them from stepping on each other — without making everyone wait in one long queue. MVCC and locks are how it does this.

Hundreds of transactions hit the same tables at the same moment. How the database keeps them apart (*isolation*) decides two things:

- **Correctness** — no money spent twice, no update silently lost.
- **Throughput** — who blocks whom, and how much work gets done per second.

Once you understand MVCC and locking, you can explain strange results (*anomalies*), deadlocks, and "why is this query just waiting?".

What is happening internally?

MVCC stands for multi-version concurrency control. PostgreSQL, Oracle and MySQL InnoDB use it, and so does SQL Server when snapshot isolation is on. Here is how it works:

- An UPDATE does not overwrite the row. It writes a **new version** of the row. The old version stays for transactions that started earlier.
- Each transaction reads the versions that were committed as of its **snapshot** — a fixed point in time. In READ COMMITTED, the snapshot is taken at the start of each statement. In REPEATABLE READ, it is taken at the start of the transaction.
- So **readers don't block writers, and writers don't block readers.** But writers still block **other writers** on the same row.
- PostgreSQL keeps the old versions inside the table. They are called *dead tuples*. `VACUUM` cleans them up. A **long-running transaction** stops this cleanup, so the table grows with junk. This is called *bloat*.
- **SQL Server's default READ COMMITTED uses locks, not versions.** Readers take shared locks and can block writers. If you turn on `READ_COMMITTED_SNAPSHOT`, it uses row versions stored in tempdb instead. This is a real difference from your SQL Server experience, and worth mentioning in an interview.

Think of a bank passbook. When a new entry is written, your earlier printout still shows the old balance. Someone reading the old printout doesn't stop the clerk from writing a new entry.

Locks (simplified):

- **Shared (S)** — taken for reads when the isolation level uses locking.

- **Exclusive (X)** — taken for writes.
- **Update (U)** — SQL Server only. Taken when you read a row you plan to update. It avoids *conversion deadlocks*, where two readers both try to upgrade to a write lock.
- **Intention locks** — placed at table or page level to signal "someone has a lock inside here".
- **Range/gap locks** — lock a range of keys, not just existing rows. Used in SERIALIZABLE in SQL Server and REPEATABLE READ in InnoDB. They prevent *phantoms* — new rows appearing in a range you already read.

Explicit row locks: `SELECT … FOR UPDATE` (PostgreSQL/MySQL) or `WITH (UPDLOCK, ROWLOCK)` (SQL Server). `FOR UPDATE SKIP LOCKED` lets many workers each take *different* rows from a queue table, without waiting on each other. This is used for outbox relays and job schedulers.

Lock escalation (SQL Server): if one statement holds thousands of row locks, SQL Server may swap them for a single table lock. That blocks everyone. So split big updates into smaller chunks (batches).

Deadlock: transaction A locks row 1 and waits for row 2. At the same time, transaction B locks row 2 and waits for row 1. Neither can move. The database's deadlock detector finds this cycle and aborts one of the transactions. PostgreSQL reports `deadlock detected`. SQL Server reports error 1205, "chosen as deadlock victim".

How does it work?

Step by step:

1. T1 updates account A, so it locks A.
2. T2 updates account B, so it locks B.
3. T1 now wants B. B is locked by T2, so T1 waits.
4. T2 now wants A. A is locked by T1, so T2 waits.
5. Each waits for the other — a cycle. The database rolls one back.

```
T1: UPDATE account SET balance=balance-10 WHERE id='A'   (locks A)
T2: UPDATE account SET balance=balance-10 WHERE id='B'   (locks B)
T1: UPDATE account SET balance=balance+10 WHERE id='B'   (waits for T2)
T2: UPDATE account SET balance=balance+10 WHERE id='A'   (waits for T1) → cycle → one is rolled back
Fix: always update accounts in ascending id order (both lock A first).
```

Isolation anomalies (full table in 7.14):

- **Dirty read** — reading data another transaction hasn't committed yet.
- **Non-repeatable read** — reading the same row twice and getting different values.
- **Phantom** — running the same range query twice and getting new rows.
- **Lost update** — two transactions update the same row, and one update overwrites the other.
- **Write skew** — two transactions read the same condition, and each writes a *different* row. Together they break a rule that each one alone would have kept. Example: two admins each approve "the last" budget slot, so two slots get used.

Write skew needs SERIALIZABLE isolation, or an explicit `FOR UPDATE` on the rows you read.

Example

```sql
-- Job queue table consumed by many workers without blocking each other
BEGIN;
SELECT id, payload FROM outbox
WHERE published_at IS NULL
```

```
ORDER BY created_at
LIMIT 100
FOR UPDATE SKIP LOCKED;          -- each worker gets a disjoint batch
-- publish to Kafka …
UPDATE outbox SET published_at = now() WHERE id = ANY(:ids);
COMMIT;
```

When would I use it?

- Designing concurrent updates — balances, inventory, seat holds.
- Job tables and outbox tables.
- Batch updates on busy ("hot") tables.
- Debugging blocking — "which session holds the lock?". In PostgreSQL, use `pg_locks` + `pg_stat_activity`. In SQL Server, use `sys.dm_tran_locks` and blocked process reports.

What can go wrong?

- **Long transactions.** They hold locks longer, cause bloat, and cause replication lag.
- **Network calls inside transactions.** A slow HTTP call keeps locks held the whole time.
- **Inconsistent lock ordering.** Different code paths lock rows in different orders, which causes deadlocks.
- **Missing indexes.** An UPDATE without a useful index scans and locks more rows than needed.
- **Not retrying deadlock victims and serialisation failures.** These are normal and expected. Retry them with a small backoff (a short wait), and make the retry idempotent — safe to run twice.

Interview questions

Q1. What is MVCC? Writers create new versions of a row instead of overwriting it. Readers see a consistent snapshot. So reads and writes don't block each other. Old versions are cleaned up later.

Q2. How do database deadlocks happen and how do you handle them? Transactions wait on each other's row locks in a cycle, and the database aborts one of them. Prevent it with a consistent lock order, short transactions, good indexes and single-statement updates. Retry the victim.

Q3. What is write skew and how do you prevent it? Two concurrent transactions read overlapping data. Based on it, each writes a different row, and together they break a rule (an invariant). Prevent it with SERIALIZABLE isolation, or by locking the rows you read with `FOR UPDATE`.

Q4. What does `FOR UPDATE SKIP LOCKED` do? It locks the selected rows and skips any rows already locked by others. So parallel workers each grab different work items without blocking each other.

12.8 Data modelling: normalisation, denormalisation and table partitioning

Why?

> **In simple words:** Normalising means storing each fact in one place, so it can't go out of sync. Denormalising means copying data on purpose to make reads faster. Partitioning means cutting one huge table into smaller pieces, usually by month.

The schema decides three things:

- **Correctness** — no *update anomalies* (the same fact stored twice, with one copy updated and the other not).

- **Performance** — how many joins each request needs.
- **Operability** — how you archive or delete old data.

A senior engineer knows when to normalise, when to denormalise on purpose, and how to keep huge tables manageable.

What is happening internally?

Normal forms, one line each:

- **1NF**: each column holds a single value, and there are no repeating groups. For example, no comma-separated SKU list in one column.
- **2NF**: no column depends on only *part* of a composite key.
- **3NF**: no transitive dependencies — a column should not depend on another non-key column. An outlet's city belongs in `outlet`, not repeated in every order.
- **BCNF**: every determinant (a column that decides another column's value) is a candidate key.

Denormalisation means copying data so you avoid joins or recalculation. Use it for:

- Read-heavy paths.
- Analytics tables.
- Cached aggregates — for example, a `balance` column kept next to the ledger and checked against it (reconciled) every night.
- **Historical snapshots** that must not change when master data changes. Store the price *at order time* on the order line. This is a correctness requirement, not just a performance trick.

Table partitioning splits a large table into child tables by a key — usually time. This is inside one database. It is different from sharding, which spreads data across servers (11.9). Benefits:

- Queries that filter on the key touch only the relevant partitions. This is called **partition pruning**.
- Dropping an old partition is instant. A huge `DELETE` is slow and heavy.
- Each partition's indexes are smaller.

Think of an office that keeps one file cabinet per month. To find last week's papers, you open only this month's cabinet. To throw away 2019, you remove the 2019 cabinets — no need to pull out papers one by one.

How does it work?

```
CREATE TABLE ledger_entry (
  id BIGINT NOT NULL, tenant_id TEXT NOT NULL, account_id TEXT NOT NULL,
  points BIGINT NOT NULL, created_at TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (id, created_at)                      -- the partition key must be part of the PK/unique keys
) PARTITION BY RANGE (created_at);

CREATE TABLE ledger_entry_2026_10 PARTITION OF ledger_entry
  FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
-- monthly job creates next month's partition; retention job detaches/drops partitions older than 7 years (or
archives them)
```

Example

The orders table grows by 60M rows a year. You use monthly range partitions. Now:

- Dashboard queries for "last 30 days" scan only one or two partitions.
- GDPR or retention deletes become a simple `DROP TABLE orders_2019_01`.

- Vacuum and index maintenance run per partition, which is smaller and faster.

When would I use it?

- **Normalise** OLTP (transactional) schemas by default.
- **Denormalise** on purpose, for read paths that are proven slow, and for history.
- **Partition** tables that keep growing forever (ledgers, events, orders, logs) by time. Choose a partition key that appears in almost every query's filter.

What can go wrong?

- **Denormalised copies drift out of sync.** Decide which copy is the source of truth. Decide how the copies get updated — events, triggers, or reconciliation jobs.
- **Partitioning on a key that queries don't filter by.** Then every query scans every partition.
- **Too many tiny partitions.** The planner spends too long just planning.
- **Unique constraints must include the partition key.** You can't enforce uniqueness on `id` alone across partitions.

Interview questions

Q1. When would you denormalise? For read-heavy paths where joins cost too much, for analytics, for cached aggregates, and for historical snapshots such as the price at order time.

Q2. Partitioning vs sharding? Partitioning splits a table inside one database, so queries can skip partitions and old data is easy to drop. Sharding spreads data across many database servers, to get more capacity and more write throughput.

12.9 OLTP vs OLAP: where ClickHouse and Trino fit

Why?

> **In simple words:** The database that takes orders (OLTP) is built for many tiny reads and writes. Dashboards need the opposite — a few huge scans over months of data. So analytics goes to a different kind of database (OLAP), like ClickHouse, which is built for scanning.

A dashboard that adds up months of orders across thousands of outlets will crush a transactional database. It will also slow down checkout for real users. Analytical work needs a different storage design.

Your experience with ClickHouse and Trino is a strength here. Be ready to explain *why* they are fast.

What is happening internally?

OLTP means online transaction processing. OLAP means online analytical processing.

	OLTP (PostgreSQL, SQL Server)	OLAP (ClickHouse, BigQuery, Snowflake; Trino over a lake)
Workload	Many small reads and writes by key	A few huge scans and aggregations
Storage	Row-oriented (all columns of one row stored together)	**Column-oriented** (each column stored separately, heavily compressed)

	OLTP (PostgreSQL, SQL Server)	OLAP (ClickHouse, BigQuery, Snowflake; Trino over a lake)
Indexes	B+ trees for point lookups	Sparse primary index + sort order; skip indexes; min/max per block
Updates	Cheap and transactional	Expensive or done later in the background (mutations, ReplacingMergeTree)
Joins	Strong	Possible but costly; prefer wide, denormalised tables

Why columnar storage is fast for analytics:

- A query that sums `amount` by `outlet_id` reads only those two columns. It skips the rest.
- Similar values are stored next to each other, so they compress 5–10×. Less data to read from disk.
- *Vectorised execution* processes values in batches, using SIMD (one CPU instruction working on many values at once).

Think of a school register. A row store is like one file per student, with all their marks. To find the class average in Maths, you open every file. A column store is like one sheet with only the Maths marks of all students — you read just that one sheet.

ClickHouse MergeTree:

1. Inserted data is sorted by the table's `ORDER BY` key.
2. It is written as immutable (never changed) **parts**.
3. Parts are merged together in the background.
4. The sparse primary index stores one mark per **granule** (a block of 8,192 rows). So a query reads only the granules that can match.
5. **Materialised views** keep rollups up to date at insert time.
6. TTLs (time-to-live rules) move or drop old data automatically.

Trino is a distributed SQL query engine with **no storage of its own**. It *federates* queries — one SQL query can read from many sources: Hive/Iceberg tables on object storage, PostgreSQL, Kafka, ClickHouse. It is great for ad-hoc analysis across systems. It is not meant for low-latency serving to users.

How does it work?

The data flow:

1. PostgreSQL holds the source of truth.
2. CDC (change data capture — Debezium reads every change from the database log) sends changes to Kafka.
3. ClickHouse reads from Kafka. A materialised view builds daily sales per outlet for dashboards and incentive metrics.
4. Batch exports go to a data lake (Iceberg on Blob storage). Trino queries the lake, Postgres and ClickHouse together for ad-hoc analysis.

```
PostgreSQL (truth) ──CDC (Debezium)──► Kafka ──► ClickHouse (MergeTree, ORDER BY (tenant_id, outlet_id, day))
                                                    └─ materialised view: daily sales per outlet →
dashboards, incentive metrics
Data lake (Iceberg on Blob) ◄── batch exports; Trino queries lake + Postgres + ClickHouse for ad-hoc analysis
```

Example

```
CREATE TABLE sales_events (
  tenant_id UInt32, outlet_id UInt64, sku LowCardinality(String), amount Decimal(12,2), event_time DateTime
) ENGINE = MergeTree
PARTITION BY toYYYYMM(event_time)
ORDER BY (tenant_id, outlet_id, event_time)
TTL event_time + INTERVAL 3 YEAR;

-- Scans only the granules for tenant 7 in the last 30 days, reading 3 columns
SELECT outlet_id, sum(amount) FROM sales_events
WHERE tenant_id = 7 AND event_time >= now() - INTERVAL 30 DAY
GROUP BY outlet_id ORDER BY sum(amount) DESC LIMIT 10;
```

> **Say this:** "Transactional truth stays in PostgreSQL or SQL Server; changes stream via CDC and Kafka into ClickHouse for dashboards and incentive aggregates; Trino handles ad-hoc analysis across sources. That split keeps heavy analytics from ever hurting checkout latency."

When would I use it?

- Dashboards and reports.
- Incentive metric aggregation (13.14).
- Metrics and log analytics (13.11).
- Any query over months or years of events.

What can go wrong?

- **Using ClickHouse like an OLTP database.** Frequent single-row updates and deletes, or point lookups by non-key columns, perform badly.
- **A poor `ORDER BY` key.** Queries can't skip granules, so they read everything.
- **Too many small inserts.** Each insert creates a new part. Batch your inserts.
- **Expecting strong consistency.** The analytics store is fed asynchronously, so it is always slightly behind the source.

Interview questions

Q1. Row store vs column store? A row store keeps each row together, which is ideal for transactional point reads and writes. A column store keeps each column together, which is ideal for scanning and aggregating a few columns over many rows, with high compression.

Q2. Why is ClickHouse fast for aggregations? It uses columnar storage, so it reads only the needed columns. Data is heavily compressed. Data is sorted by the primary key, and a sparse index lets it skip granules. It uses vectorised execution. And materialised views pre-aggregate data at insert time.

Q3. What is Trino used for? Distributed SQL across many data sources, without storing any data itself. It is used for ad-hoc analytics across a data lake and operational databases.

13. System Design Case Studies

This chapter has nineteen designs. Each one uses the same seven steps, and each step matches a part of the real interview:

- **Why?** = the requirements, and what the question is really testing.
- **What is happening internally?** = estimates, API, data model and architecture.
- **How does it work?** = the write path and the read path.
- **Example** = a concrete walkthrough with numbers.
- **When would I use it?** = other places where the same design shows up.
- **What can go wrong?** = bottlenecks, failure cases and deep dives.
- **Interview questions** = the follow-up questions you will get.

All numbers are assumptions. In the interview, say them out loud. Before reading each design, practise it on a blank page in 45 minutes.

13.1 URL Shortener (TinyURL / bit.ly)

Why?

> **In simple words:** You give a long link and get back a short code. When someone opens the short code, you look up the long link and send them there — very fast, millions of times a day.

Functional (what it must do): - Create a short link for a long URL. Optionally allow a custom alias and an expiry date. - Redirect from the short link to the long URL. - Basic click analytics.

Non-functional (how well it must do it): - Redirects must be fast (p99 < 50 ms — 99% of requests finish within 50 ms) and highly available. - Reads are about 100 times more than writes (read:write ≈ 100:1). - Two links must never get the same code (no collisions).

What it tests: ID generation, caching a path that is mostly reads, and keeping analytics off the hot path (the path every request takes).

What is happening internally?

Estimates: - 100M new links/month ≈ 40 writes/s (peak ~120). - Reads are ×100 → ~4,000/s (peak ~12,000). - Each record is ~500 B. So 100M × 12 × 5 years × 500 B ≈ 3 TB over 5 years. - Base62 (digits + lowercase + uppercase = 62 characters) with 7 characters gives 62^7 ≈ 3.5 trillion codes. That is plenty.

API:

```
POST /api/v1/urls   {longUrl, customAlias?, expiresAt?}  → 201 {shortUrl}
GET  /{code}         → 302 Location: longUrl   (301 is cached by browsers: less load, but clicks become
invisible)
GET  /api/v1/urls/{code}/stats
```

Data model: `url(code PK, long_url, owner_id, created_at, expires_at)`. You always look a link up by its code. So a key-value store (DynamoDB/Cassandra) fits well. Postgres split into shards by code also works.

Architecture:

- Redirects go through a CDN/edge, then a load balancer, then the redirect service. Redis holds the ~20% of codes that are hot. On a cache miss, you go to the URL store.
- The create service takes IDs from a range allocator and encodes them in base62.
- Clicks are sent to Kafka. Analytics consumers read them and write to ClickHouse.

```
Client → CDN/edge → LB → Redirect service → Redis (code→url, ~20% hot set) → miss → URL store
Create service → ID range allocator → base62 encode → URL store
Redirect service → Kafka "clicks" → analytics consumers → ClickHouse
```

How does it work?

Code generation (the core deep dive). There are four common ways:

Approach	How	Trade-off
Hash + truncate	base62(md5(longUrl))[0..7]	You must detect collisions and retry; the same URL always gets the same code (deduped)
Global counter + base62	Sequence → base62	No collisions; but one central counter for everyone; codes are easy to guess
Pre-allocated ID ranges	Each instance leases (borrows) a block of 1M IDs from a coordinator (a DB row or ZooKeeper), then encodes locally	No coordination per request, no collisions — preferred
Key generation service	Random unused keys are generated in advance and handed out in batches	Codes are hard to guess; but you run one more service

Write path: 1. Validate the URL. 2. Take the next ID from this instance's local range. 3. Encode it in base62. 4. Insert it. For a custom alias, use a conditional insert, so the insert fails if the alias is already taken. 5. Return the short URL.

Read path: 1. Look up the code at the edge or in Redis. 2. On a hit, return a 302 redirect. 3. On a miss, read from the store, put the result into Redis, then return 302. 4. Publish a click event in the background (asynchronously), so the user does not wait for it.

Example

1. Instance A leases IDs 5,000,000–5,999,999.
2. Request 1 gets ID 5,000,000. It is encoded in base62 and padded on the left to make a 7-character code. Then it is stored.
3. Later, a redirect for that code finds it in Redis in 0.8 ms.
4. The click event goes to Kafka. ClickHouse counts it a few seconds later.

When would I use it?

- Link shortening.
- Referral and campaign links.
- QR codes for retail promotions.
- In general, any service that maps "key → value", is mostly reads, and needs keys that are unique everywhere.

What can go wrong?

- **Collisions with hashing.** Two URLs can give the same truncated hash. You must detect this and retry.
- **Hot links.** One very popular link can overload one cache shard. Fix: copy hot keys to several places, and cache at the edge.
- **Slow analytics.** Writing analytics synchronously (while the user waits) slows down redirects.
- **Malicious URLs.** Scan URLs when they are created, and rate-limit each user.
- **Expired links still in cache.** Set the cache TTL (time to live) so it respects the link's expiry.

Interview questions

Q1. How do you generate short codes without collisions at scale? Give each instance its own pre-allocated range of IDs and encode them in base62. (Snowflake or UUIDv7 IDs, encoded, also work.) This way, no request needs to coordinate with other machines.

Q2. 301 or 302? A 301 is cached by browsers. That means less load on you, but you lose analytics because repeat clicks never reach you. A 302 lets you see every click. Choose based on whether analytics matter.

Q3. How do you keep redirects fast while counting clicks? Serve the redirect from the cache. Publish click events in the background to Kafka, and count them there.

13.2 Distributed Rate Limiter

Why?

> **In simple words:** A rate limiter counts how many requests each client sends and says "slow down" (HTTP 429) when they cross their limit. The hard part is keeping one shared, correct count when many servers handle the same client.

Functional (what it must do): - Limit requests per client — by user, API key, IP or tenant. - Limits are set per rule, for example 100/min per tenant per endpoint. - When the limit is crossed, return 429 with a `Retry-After` header. - Rules can be configured.

Non-functional (how well it must do it): - Add less than 2 ms of latency. - Stay correct across many gateway instances. - Have a clear, decided behaviour when the limiter itself fails: *fail-open* (allow requests) or *fail-closed* (block requests).

What it tests: rate-limiting algorithms (10.5), atomic updates in a shared store, and failure handling.

What is happening internally?

Placement: Put the limiter at the API gateway, or as a sidecar/middleware next to each service. Cache the rules locally and refresh them from a config service.

Store: Use a Redis cluster, with keys split across shards by client ID.

The check must be **atomic** — read, check and write must happen as one step. Otherwise two servers could both read "99 used" and both allow request 100. Redis **Lua scripts** solve this. The whole script runs in one round trip, and nothing else runs in between. So there is no race between the read and the write.

Here is a *token bucket* script. Think of a bucket that holds tokens. Each request takes a token. Tokens refill at a fixed rate. If the bucket is empty, the request is rejected.

```lua
-- Token bucket (KEYS[1] = bucket key; ARGV = capacity, refill_per_ms, now_ms, cost)
local b = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local cap, rate, now, cost = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3]), tonumber(ARGV[4])
local tokens = tonumber(b[1]) or cap
local ts = tonumber(b[2]) or now
tokens = math.min(cap, tokens + (now - ts) * rate)
local allowed = tokens >= cost
if allowed then tokens = tokens - cost end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', KEYS[1], math.ceil(cap / rate) * 2)
return allowed and 1 or 0
```

Alternatives: - **Fixed window** (`INCR key:minute` + `EXPIRE`): the simplest. But bursts can slip through at window edges (end of one minute + start of the next). - **Sliding log** (`ZADD` , `ZREMRANGEBYSCORE` , `ZCARD`): exact, but uses more memory because it stores every request time. - **Sliding window counter** (a weighted mix of two buckets): a good balance of accuracy and memory.

How does it work?

1. A request arrives and the gateway authenticates it.
2. The gateway builds the key, for example `t-42:/redemptions` .
3. It runs the script with `EVALSHA` , using Redis server time.
4. If the result is 1, it forwards the request.
5. If the result is 0, it returns 429 with `Retry-After` and `X-RateLimit-Remaining` .

Example

Tenant t-42 is on the GOLD plan: capacity 200, refill 100/s. It sends a burst of 300 requests in one second:

- 200 pass immediately (the bucket was full).
- The next ~100 pass as tokens refill.
- The rest get 429 with `Retry-After: 1` .

When would I use it?

- Public and partner APIs.
- Protecting OTP and login from abuse.
- Fairness between tenants.
- Protecting your outbound calls to partners that give you a quota.

What can go wrong?

- **Redis slow or down.** Keep local fallback buckets. Fail open for non-critical APIs; fail closed for OTP/login.
- **Hot keys for very large tenants.** One key gets too much traffic. Split it into N sub-keys, each with limit/N.
- **Multi-region limits.** Use a quota per region, or sync counts between regions in the background.
- **Clock skew.** App servers' clocks may differ. Use Redis `TIME` so everyone uses one clock.

Interview questions

Q1. How do you make the limiter atomic across instances? Keep the state in Redis and update it with a Lua script. Then read, check and write all happen in one atomic operation.

Q2. What happens if Redis is down? It is a deliberate policy decision. For normal APIs, fail open and use approximate local limits. For sensitive endpoints, fail closed. Either way, raise an alert.

Q3. Token bucket or sliding window? Use token bucket when you want to allow controlled bursts while keeping a smooth average rate. Use a sliding window counter for simple, accurate limits per time window.

13.3 Notification System

Why?

> **In simple words:** Services say "tell this user X", and the system picks the right channel (SMS, email, push, WhatsApp), respects their preferences, and delivers it — without sending duplicates and without bulk campaigns blocking urgent OTPs.

Functional (what it must do): - Send notifications by SMS, email, push and WhatsApp/in-app. - Triggered by services (order placed, points credited) and by campaigns (bulk sends). - Respect user preferences and opt-outs. - Templates with localisation (different languages). - Scheduling. - Delivery tracking.

Non-functional (how well it must do it): - Transactional messages (like OTPs) arrive within seconds. - Millions of campaign messages must not delay OTPs. - No duplicates. - Switch to a backup provider if one fails (*failover*).

What it tests: queue design, prioritisation, idempotency (doing the same request twice has the same effect as once) and third-party integration.

What is happening internally?

Estimates: 10M transactional messages/day (~120/s, peaks ~1,000/s). Plus 5M-message campaigns sent within an hour (~1,400/s).

The flow in the diagram, in plain words:

1. Producer services call `POST /notify`.
2. The Notification API validates the request and removes duplicates using the idempotency key. It checks preferences, quiet hours and DND. Then it renders the template in the user's language.
3. It puts the message on one of two Kafka topics. `notify.high` is for transactional messages; `notify.bulk` is for campaigns. Separate topics keep the priorities apart.
4. Channel workers (one consumer group per channel) send through provider adapters. Each adapter has a circuit breaker (stops calling a failing provider for a while), a per-provider rate limit, and failover to a second provider.
5. Providers call back through delivery webhooks, and a status updater records the result.

```
Producers (order, loyalty, campaign service)
   │  POST /notify {userId, templateId, params, channelHints, idempotencyKey, priority}
   ▼
Notification API: validate → dedupe (idempotency key) → preferences, quiet hours, DND → render template
(locale)
   ├─► Kafka "notify.high"  (transactional)        separate topics = priority isolation
```

```
   └─► Kafka "notify.bulk"  (campaigns, rate-shaped)
Channel workers (SMS / email / push / WhatsApp consumer groups)
   → provider adapters (Strategy) with circuit breakers, per-provider rate limits, failover to a secondary
provider
   → providers (Gupshup/Twilio, SES, FCM/APNs) ─► delivery webhooks ─► status updater
Stores: preferences + templates (Postgres + cache), notification log/status (Cassandra or ClickHouse),
scheduled sends (DB + scheduler)
```

How does it work?

1. Remove duplicates using an (event, user, channel) key. Render the template. Put the message on the right priority topic.
2. Channel workers read the message and apply the provider's rate limit. They call the provider with an idempotency/reference ID, and record SENT.
3. Provider webhooks update the status to DELIVERED/FAILED/READ. Each webhook's signature is verified, and the handling is idempotent, so a repeated webhook does no harm.
4. Failures are retried with backoff (waiting longer each time). After N attempts, the message goes to a DLQ (dead-letter queue — a parking place for failed messages), and a fallback channel is tried (SMS if push fails).

Example

1. A "points credited" event arrives. The template `points_credited_hi` (Hindi) is chosen.
2. Push is tried first. The device is offline for 10 minutes, so no delivery receipt comes back.
3. The system falls back to SMS, and the SMS is delivered.

Meanwhile, a 5M-message Diwali campaign drains through `notify.bulk`. It runs at the SMS provider's limit of 1,000 msg/s. It does not delay any OTPs, because they travel on `notify.high`.

When would I use it?

- Any messaging over several channels.
- Alerting.
- Marketing campaigns.
- In general: "spread work out to slow outside providers, with priorities".

What can go wrong?

- **Duplicate sends on retries.** Fix with idempotency keys.
- **Campaigns starving transactional messages.** Fix with separate topics and separate worker pools.
- **Provider throttling.** Shape the send rate and use more than one provider.
- **Regulatory issues.** Respect opt-outs, DND and quiet hours in each user's timezone.
- **Template bugs sent to millions.** Preview templates and roll out in stages.
- **Webhook storms.** Too many status callbacks at once. Batch the status updates.

Interview questions

Q1. How do you ensure OTPs aren't delayed by a marketing campaign? Give each priority its own topics, its own consumer pools and its own provider quotas. Also shape (throttle) the rate of bulk traffic.

Q2. How do you avoid sending duplicate notifications? Use an idempotency key per (event, user, channel). Store it with a TTL and a unique constraint. Also make the webhook handling idempotent.

Q3. How do you track delivery? Keep a status machine for each message, updated by provider callbacks. Store it in a store built for heavy writes, with dashboards showing delivery rates per provider.

13.4 News Feed (Twitter/LinkedIn/Instagram home feed)

Why?

> **In simple words:** When you open the app, you see recent posts from people you follow. The main choice is when to build that list: when someone posts (push), or when you open the app (pull) — and celebrities with millions of followers need special handling.

Functional (what it must do): - Users post. - Users follow others. - The home feed shows posts from the people you follow, newest first or ranked, page by page.

Non-functional (how well it must do it): - The feed loads in < 200 ms. - New posts may take a few seconds to appear. This is called *eventual consistency*. - 300M DAU (daily active users). - Far more reads than writes.

What it tests: fan-out strategies (how a post is spread to followers), caching at scale, and the celebrity problem.

What is happening internally?

Estimates: 300M DAU × 10 feed loads = 3B/day ≈ 35k QPS (peak ~100k). 150M posts/day ≈ 1.7k writes/s.

There are two ways to build feeds:

	Fan-out on write (push)	Fan-out on read (pull)
How	When someone posts, add the post ID to each follower's ready-made feed (a Redis list)	When someone opens the feed, fetch recent posts from everyone they follow and merge them
Read cost	O(1) — read one list	O(followees) — grows with how many people you follow
Write cost	O(followers) — huge for celebrities	O(1)
Best for	Normal users	Celebrities with millions of followers

The architecture in plain words:

1. The post service saves the post in the Posts DB, sends media to blob storage/CDN, and publishes a `PostCreated` event.
2. Fan-out workers ask the social graph service for the followers. They add the post ID to each follower's `feed:{userId}` list in Redis (capped at ~800 IDs).
3. The feed service reads the user's list, merges in recent celebrity posts, fills in the full post details (*hydrates* the IDs), ranks them, and returns one page using a cursor.

```
Post service → Posts DB (Cassandra/sharded, by post_id) + media to blob/CDN → PostCreated event
Fan-out workers → Social graph service (followers) → feed:{userId} lists in Redis (capped ~800 IDs)
Feed service: read feed:{userId} + merge recent posts of followed celebrities → hydrate IDs from post cache → rank → cursor page
```

How does it work?

Hybrid — use both methods: - **Normal authors:** push their posts to followers' feeds. - **Celebrities** (above a follower threshold): skip fan-out. Merge their recent posts in when a follower reads the feed. - **Inactive users:** do not precompute their feeds. Build the feed on their next login.

Example

- A user with 300 followers posts. Fan-out workers do 300 `LPUSH` + `LTRIM` operations within ~1 s.
- A celebrity with 20M followers posts. No fan-out happens. When their followers load the feed, it includes "recent posts from followed celebrities" — a small cached list. It is merged in by timestamp or score.

When would I use it?

- Activity feeds.
- Notification inboxes.
- "What's new" panels.
- B2B feeds, such as new schemes shown to an outlet.
- Any problem where one item must be shown to many people (one-to-many distribution).

What can go wrong?

- **Celebrity fan-out storms.** One post triggers millions of writes.
- **Feed caches too big.** Cache feeds only for active users, and cap list sizes.
- **Deleted or privacy-changed posts still showing.** Filter them out at hydration time.
- **Slow ranking.** Precompute the features used for ranking.
- **Unstable pagination.** New posts shift the pages. Use a cursor on (score + ID).

Interview questions

Q1. Push vs pull for feeds? Push makes reads cheap but makes writes expensive for popular authors. Pull is the opposite. Use a hybrid: push for most users, pull for celebrities.

Q2. How do you paginate a feed that changes constantly? Use a cursor based on (score or timestamp, postId), not page offsets. Offsets break when new posts are added at the top.

13.5 Chat System (WhatsApp / Slack)

Why?

> **In simple words:** Every phone keeps an open connection to a chat server. When you send a message, the server saves it, gives it a number in the conversation, and pushes it to the receiver's connection — or sends a push notification if they are offline.

Functional (what it must do): - 1:1 chat and group chat (≤ 500 members). - Presence (online/last seen). - Delivery and read receipts. - Offline delivery through push notifications. - History synced across multiple devices. - Media.

Non-functional (how well it must do it): - < 200 ms delivery when the receiver is online. - Messages are never lost. - Messages stay in order within each conversation. - 500M DAU.

What it tests: long-lived connections, routing messages to the right server, ordering, storage at huge write volume, and delivery guarantees.

What is happening internally?

Estimates: 500M DAU × 40 messages = 20B messages/day ≈ 230k messages/s. At ~100 B each, that is ~2 TB/day of text.

The architecture in plain words:

1. Clients connect over **WebSocket** (a connection that stays open both ways) to connection (gateway) servers. Each server holds ~100k–1M connections. These servers are *stateful* — they remember who is connected to them.
2. Each connection server records "userId → serverId" in a session registry in Redis.
3. The chat service stores messages in Cassandra. The partition key is `conversation_id` and the clustering key (sort order) is `message_seq`.
4. To deliver, the chat service finds the receiver's connection server in the registry and sends the message through internal pub/sub.
5. If the receiver is offline, it sends a push notification (FCM/APNs).
6. Large groups use fan-out workers on Kafka. Media is uploaded directly to blob storage using a pre-signed URL and served via CDN. Presence uses heartbeats stored in Redis with a TTL.

```
Clients ═ WebSocket ═▶ Connection (gateway) servers — stateful, ~100k–1M connections each
                           register userId → serverId in a session registry (Redis)
                           ▼
                       Chat service → Message store (Cassandra: partition = conversation_id, clustering =
message_seq)
                           ├─▶ route to the recipient's connection server (registry lookup + internal
pub/sub)
                           └─▶ recipient offline → push notification (FCM/APNs)
Group fan-out workers (Kafka) for large groups · Media → pre-signed upload to blob + CDN · Presence →
heartbeats in Redis with TTL
```

How does it work?

1. The sender's client sends `{clientMsgId, conversationId, body}` over its WebSocket.
2. The chat service gives the message a **per-conversation sequence number**. It saves the message, then confirms (acks) to the sender — "sent" ✓. The `clientMsgId` makes resends idempotent: if the client sends the same message twice, it is stored once.
3. The service finds the receivers' connection servers and pushes the message. When the receiver's device acks, "delivered" ✓✓ flows back to the sender. When the receiver opens the chat, it becomes "read".
4. Offline receivers get a push notification. When they reconnect, the client asks for all messages after its **last seen sequence** in each conversation.

Example

1. Asha sends "Order confirmed?" to the distributor group (40 members).
2. The message gets seq 1043 in the group conversation.
3. 32 online members receive it through their connection servers in ~100 ms.
4. 8 offline members get a push notification.
5. One of them reconnects an hour later. Its last seen seq was 1029, so it fetches seq > 1029 and receives 14 messages, in order.

When would I use it?

- Chat and live support.
- The conversational ordering channel of an assistant like NOVA.
- Collaborative apps.
- Any real-time messaging that goes both ways.

What can go wrong?

- **Connection server fails.** Clients reconnect to another server and resync using their sequence number.
- **Ordering across devices.** Fix with a server-assigned sequence per conversation.
- **Hot group conversations.** Use fan-out workers and rate limits.
- **Presence storms.** Too many online/offline updates. Fetch presence lazily, only when a chat is opened.
- **Message loss.** This happens if you ack before saving. Always save first, then ack.

Interview questions

Q1. How do you route a message to a user connected to a different server? Keep a session registry in Redis that maps each user to their connection server. Then send the message to that server through internal pub/sub or a direct RPC call.

Q2. How do you guarantee ordering and no loss? Save the message before acking. Assign a sequence number per conversation. Make sends idempotent using client message IDs. On reconnect, resync from the last seen sequence.

Q3. Why Cassandra for messages? The write volume is very high, and messages are read by conversation in time order. That fits Cassandra's partition key + clustering key model well, and Cassandra scales out by adding machines.

13.6 Distributed Cache (design Redis/Memcached)

Why?

> **In simple words:** A cache keeps hot data in memory so reads take well under a millisecond. To hold more data than one machine can, you split keys across many machines and keep copies so a crash does not lose everything.

Functional (what it must do): - `get`, `put` with a TTL (time to live), and `delete`. - Grow beyond one machine. - Survive node failures.

Non-functional (how well it must do it): - Sub-millisecond latency. - Millions of operations per second.

What it tests: in-memory data structures, eviction (removing old data when memory is full), partitioning, replication and consistency trade-offs.

What is happening internally?

- **Single node**: a hash map plus an eviction structure. This can be an LRU list (*least recently used* goes first). Or it can be approximate LRU/LFU (*least frequently used*) done by sampling a few keys, as Redis does. TTLs are enforced in two ways: lazily, when a key is accessed, and by regularly checking a random sample of keys that have expiry.

- **Partitioning** (splitting keys across nodes): consistent hashing with virtual nodes, done in the client or through a proxy. Redis Cluster instead uses 16,384 hash slots, and each node owns some of the slots.
- **Replication**: each shard has a primary and replicas, copied asynchronously (in the background). Failover is handled by Sentinel or by cluster consensus (the nodes vote).
- **Memory efficiency**: slab allocation in Memcached (memory pre-cut into fixed-size chunks), and compact encodings for small objects in Redis (listpacks).
- **Concurrency model**: Redis runs commands on a single thread, so it needs no locks (newer versions add I/O threads). Memcached is multi-threaded, with striped locks (many small locks, each covering part of the data).
- **Persistence (optional)**: snapshots (RDB) and append-only logs (AOF).

How does it work?

1. For `GET k`, the client hashes k to get a slot.
2. The slot tells it which node owns the key.
3. That node looks up the key in memory and returns the value.

If a node fails, its replica is promoted to primary. Keys written to the old primary just before the failure may be lost, because replication is asynchronous. For a cache, this is acceptable.

Example

- A 6-node Redis Cluster (3 primaries + 3 replicas, 32 GB each) holds 80 GB of hot data.
- You add 2 primaries. Some slots move to them while the cluster keeps running (online).
- One hot key — a festival scheme list read 50k times/s — is also copied into an in-process Caffeine cache on every app pod, with a 5 s TTL. So most reads never even reach Redis.

When would I use it?

- Caching.
- Sessions.
- Rate-limit counters.
- Leaderboards.
- Distributed locks (carefully).
- Short-lived coordination state.

What can go wrong?

- **Cache as the only copy.** Never keep critical data only in the cache.
- **Hot keys.** One very popular key overloads one node.
- **Cache and DB disagree.** Use cache-aside: delete the cache entry on every write, and use TTLs (11.5).
- **Large values.** A big value blocks Redis's single thread for everyone.
- **Eviction storms.** When memory is too small, keys are evicted constantly.

Interview questions

Q1. How would you scale a cache horizontally? Split keys across nodes with consistent hashing or hash slots. Add replicas for failover, and let the client route each key to the right node.

Q2. Why is Redis single-threaded yet fast? In-memory operations take microseconds. A single thread avoids lock contention and context switches. And I/O multiplexing (one thread watching many sockets) lets it serve many connections at once.

13.7 Loyalty and Rewards Platform (your system — rehearse it three levels deep)

Why?

> **In simple words:** Outlets earn points on orders and spend them on rewards. Every change is written as a permanent line in a ledger (like a bank passbook), so points are never credited twice, never spent twice, and every rupee can be traced.

Functional (what it must do): - Tenants (brands) set up their own programmes. - Outlets earn points on orders, scans and achievements. - They redeem points for rewards: catalogue items, vouchers, cashback. - Tiers, expiry, statements and admin adjustments. - Reverse the points when an order is cancelled.

Non-functional (how well it must do it): - Never double-credit, never double-spend. - Balance reads < 100 ms. - Accrual (adding points) may lag by a few seconds. - 700+ tenants and 8.9M outlets. - Strict tenant isolation — one tenant never sees another's data. - Full audit.

What it tests: idempotency, ledger design, sagas and multi-tenancy. And it is your strongest story.

What is happening internally?

Estimates: - 5M orders/day → 5M accrual events (~60/s; month-end peaks ~500/s). - 200k redemptions/day. - 20M balance reads/day (~230/s). - Ledger: ~6M entries/day × 200 B ≈ 1.2 GB/day ≈ 440 GB/year.

API and events:

```
POST /v1/accounts/{id}/redemptions   Idempotency-Key  {rewardId, points}
GET  /v1/accounts/{id}/balance
GET  /v1/accounts/{id}/statement?after=cursor
POST /v1/admin/adjustments           (maker-checker)
Consumes: OrderCompleted, OrderCancelled, AchievementUnlocked
Produces: PointsAccrued, PointsRedeemed, PointsExpired, TierChanged
```

(Maker-checker means one person makes the adjustment and a second person approves it.)

Data (PostgreSQL, monthly partitions for the ledger): - `account(id, tenant_id, outlet_id, balance, tier, version)` - `ledger_entry(id, tenant_id, account_id, type, points, idempotency_key UNIQUE(tenant_id, key), ref_id, lot_id, created_at)` — append-only: rows are only added, never changed - `lot(id, account_id, points_remaining, expires_at)` — each batch of earned points, used for FIFO expiry (oldest points expire first, 10.11) - `redemption(id, account_id, reward_id, points, status, partner_ref)` — saga state - `outbox`, `processed_event`

The architecture in plain words:

1. The order service publishes order events to Kafka. Accrual consumers read them (idempotently) and use a rules engine to compute points. In one local transaction, they write the ledger entry, the lot, the balance and an outbox row.

2. API requests come through the API gateway to the Loyalty API, backed by PostgreSQL (a primary for writes, replicas for statements).
3. Redemption is a *saga* — a series of steps, each with an undo. First debit points in a local transaction. Then call the voucher partner with an idempotency key. Then confirm, or compensate (undo).
4. An outbox relay (or CDC — change data capture, reading the DB's change log) publishes points events to Kafka. Notifications, analytics (ClickHouse) and the tier service consume them.
5. Balances are cached in Redis and evicted on change. An expiry job runs daily per tenant, in chunks, guarded by ShedLock (so only one instance runs it).

```
Order svc → Kafka order-events → Accrual consumers (idempotent; rules engine computes points)
                                  | local tx: ledger + lot + balance + outbox
API GW → Loyalty API → PostgreSQL (primary; replicas for statements)
           | redemption saga: debit (local tx) → voucher partner (idempotency key) → confirm / compensate
           ▼
        Outbox relay/CDC → Kafka points-events → notifications, analytics (ClickHouse), tier service
Balance cache: Redis (evicted on change) · Expiry job: daily per tenant in chunks (ShedLock)
```

How does it work?

Accrual (adding points): 1. Consume `OrderCompleted`. 2. If this event ID was already processed, skip it. 3. Compute the points. 4. In one transaction, insert the ledger entry, the lot, the balance update, the processed-event row and the outbox event.

Redemption (spending points): 1. Check the idempotency key. If it was seen before, return the stored response. 2. Debit atomically with a condition (`balance >= points`). If the balance is too low, nothing changes. 3. In the same transaction, write the ledger entry, the saga row and the outbox row. 4. The orchestrator calls the voucher partner, using the redemption ID as the idempotency key. 5. On success: confirm and notify. 6. On failure after retries: compensate (credit the points back) and mark the redemption FAILED.

Example

1. An outlet with 1,200 points redeems 1,000 for a ₹100 voucher. The user double-taps the button, so two requests arrive.
2. Request 1 debits the points (balance becomes 200) and stores the key.
3. Request 2 has the same key, so it just gets the stored response. No second debit.
4. The partner times out once. The retry uses the same key, so the partner returns the same voucher code.
5. The SMS goes out via the outbox → Kafka → notification service.

When would I use it?

- Points, wallets, credits, cashback, gift cards.
- Any balance system built on a ledger.

What can go wrong?

- **Double credit**: happens if the event-ID dedupe is not in the same transaction as the ledger write.
- **Double spend**: happens if you read the balance and then write it, without a conditional update or a lock.
- **Balance/ledger drift**: the stored balance stops matching the ledger. Run a nightly reconciliation that compares SUM(ledger) with the balance for each account, and alert on any difference.

- **Expiry at scale**: expire lot by lot, in chunks. The job must be resumable (can continue after a crash) and idempotent per lot. Notify users before their points expire.
- **Month-end spikes**: Kafka absorbs the burst. Scale consumers based on lag (how far behind they are). Partition by account so each account's events stay in order.
- **Tenant isolation**: put the tenant ID on every row and in every query. Apply rate limits per tenant.

> **Say this:** "The most important property is that every balance change is an immutable, idempotent ledger entry written in the same transaction as the balance update and the outbox event. Everything else — caching, notifications, analytics — hangs off that."

Interview questions

Q1. How do you prevent duplicate accruals when Kafka redelivers? Keep a unique constraint on the processed event ID (or idempotency key), and write it in the same transaction as the ledger entry. A redelivered event then fails the constraint and is skipped.

Q2. What if the voucher partner fails after points are debited? The saga retries with the same idempotency key, so the partner never issues two vouchers. If the failure is permanent, the saga compensates: it credits the points back and marks the redemption failed.

Q3. How do you keep balance reads fast? Keep a cached balance column that is updated in the same transaction as the ledger. Add Redis on top, evicted whenever the balance changes. Serve statements from read replicas.

13.8 Payment System

Why?

> **In simple words:** A payment system must never lose money and never charge twice. You track every payment through fixed states, make every step safe to repeat, never treat a timeout as a failure, and check your records against the bank's file every day.

Functional (what it must do):

- Merchants create *payment intents* — a record saying "this order needs ₹X".
- Customers pay by card, UPI or netbanking through external gateways. These gateways are called PSPs (payment service providers), like Razorpay or PayU.
- Handle the PSP's callbacks (webhooks) that tell you the result.
- Handle refunds.
- Keep a ledger — a record of every movement of money.
- Do reconciliation — match your records against the PSP's settlement files.

Non-functional (how well it must do it):

- Correctness matters more than speed.
- Exactly-once *effect* — a message may arrive twice, but money moves only once.
- A full audit trail.
- High availability.
- PCI-DSS compliance (the card industry's security rules). Never store card data yourself — the PSP gives you a *token* (a safe stand-in ID) to use instead.

This question tests state machines, idempotency at every hop (making each step safe to repeat), unknown outcomes and reconciliation.

What is happening internally?

```
Client → Payment API: POST /payments {orderId, amount, method}  Idempotency-Key
   → payment row (CREATED) + outbox, in one transaction
   → PSP "create order" (idempotency key = payment id) → status PENDING → return PSP intent / UPI deep link
Customer pays in the PSP/UPI app
PSP → webhook /psp/callback (verify signature) → idempotent transition PENDING → SUCCEEDED / FAILED
   → double-entry ledger rows (debit customer receivable, credit merchant payable) in the same transaction
   → outbox PaymentSucceeded → order service, notifications
Status poller: PENDING older than N minutes → query the PSP (webhooks get lost)
Reconciliation: daily settlement file vs ledger → mismatches to an ops queue
```

The flow in steps:

1. The client calls `POST /payments` with an `Idempotency-Key`, so a retry does not create a second payment.
2. In one DB transaction, you save the payment row (status CREATED) and an outbox row. An *outbox* row is a message saved in the same transaction, which a separate process later publishes — so the DB and the events never disagree.
3. You call the PSP's "create order" API. You use the payment ID as the idempotency key there too. The status becomes PENDING. You return the PSP intent or a UPI deep link to the client.
4. The customer pays inside the PSP page or UPI app.
5. The PSP calls your webhook `/psp/callback`. You first verify its signature, so nobody can fake it.
6. You move the status PENDING → SUCCEEDED or FAILED. This step is idempotent — safe if the webhook comes twice.
7. In the same transaction, you write double-entry ledger rows: debit the customer receivable, credit the merchant payable.
8. The outbox publishes `PaymentSucceeded` to the order service and notifications.
9. A **status poller** checks payments that have been PENDING for more than N minutes, by asking the PSP directly. This is needed because webhooks sometimes get lost.
10. **Reconciliation** runs daily. It compares the PSP's settlement file against your ledger and sends any mismatch to an ops queue.

Tables:

- `payment(id, order_id, amount_minor, currency, method, status, psp_ref, version)`
- `ledger_entry(id, account, direction, amount_minor, payment_id, created_at)` — the double-entry ledger
- `refund(...)`
- `idempotency_key(...)`

`amount_minor` means the amount in the smallest unit — paise, not rupees — stored as a whole number.

How does it work?

- **State machine.** A payment can only move along allowed paths: CREATED → PENDING → SUCCEEDED/FAILED, and SUCCEEDED → REFUND_PENDING → REFUNDED.
- Every move is a conditional update, like `WHERE status = 'PENDING'`.

- So if a duplicate callback arrives, the condition no longer matches and nothing changes. It is a *no-op*.
- **Double-entry ledger.** Every money movement is written as balanced debits and credits — the totals always match.
- Balances are calculated from these entries.
- Amounts are never edited in place; you only add new rows.
- Think of a shop's khata book: you never rub out a line, you only add a correcting line.
- **Unknown outcomes.** A timeout when calling the PSP is *not* a failure. The money may have moved. 1. Mark the payment UNKNOWN. 2. Ask the PSP's status API what really happened. 3. Then resolve it to SUCCEEDED or FAILED.
- Never retry a charge with a new idempotency key — that can charge the customer twice.

Example

A UPI payment of ₹1,250:

1. The API returns a deep link, and the user pays.
2. The webhook arrives twice, because the PSP retried.
3. The first webhook moves PENDING → SUCCEEDED and writes the ledger rows.
4. The second webhook sees the status is no longer PENDING. It returns 200 and changes nothing.

Another payment's webhook never arrives. After 10 minutes the poller asks the PSP, finds it SUCCEEDED, and completes it.

Next morning, reconciliation matches 99.98% of settlements. The two mismatches go to the ops team.

When would I use it?

Payments, wallets, payouts (such as incentive payouts to sales reps), refunds, and any integration where real money moves through an outside system.

What can go wrong?

- **Using floating-point for money.** `0.1 + 0.2` is not exactly `0.3` in floating point. Use minor units (paise) as whole numbers.
- **Treating a timeout as a failure.** The customer retries and gets charged twice.
- **Not checking webhook signatures.** Anyone can send a forged "payment succeeded" callback.
- **Non-idempotent refunds.** A retried refund sends money back twice.
- **No reconciliation.** Your records and the bank's slowly drift apart, and nobody notices.
- **Storing card data.** This puts your whole system in PCI scope — very costly audits and risk.

Interview questions

Q1. How do you avoid double charging? Use idempotency keys at every hop — from client to API, and from API to PSP. Make state changes conditional, so a repeat does nothing. And when the outcome is unknown, ask the PSP for the status instead of retrying blindly.

Q2. Why a double-entry ledger? Every movement balances debits and credits. So errors show up immediately, balances can be calculated from the entries, and the full history can be audited.

Q3. What if the PSP webhook never arrives? A poller asks the PSP for the status of payments that have been pending too long. Daily reconciliation against the settlement files catches anything still

missed.

13.9 Leaderboard (gaming / sales incentives)

Why?

> **In simple words:** A leaderboard needs a list that stays sorted by score while scores keep changing. Redis sorted sets do exactly this, so updating a score and asking "what is my rank?" are both very fast.

Functional (what it must do):

- Update a user's score.
- Show the top N users.
- Show a user's rank and the people just above and below them (neighbours).
- Keep separate boards per tenant, per region and per period (daily or monthly).

Non-functional (how well it must do it):

- Near real-time — scores show up within seconds.
- Top-N reads in under 50 ms.
- Millions of members on one board.

This question tests whether you choose the right data structure (sorted sets) and how you scale beyond one node.

What is happening internally?

Use Redis **sorted sets** (ZSETs). Inside, a sorted set is a *skip list* (a sorted linked list with "express lanes" for fast jumping) plus a hash map. The costs:

- `ZINCRBY` (add to a score) — O(log n).
- `ZREVRANGE 0 99` (top 100) — O(log n + k), where k is how many you fetch.
- `ZREVRANK` (a user's rank) — O(log n).

O(log n) means even with millions of users, it takes only around 20 steps.

```
Sales events → Kafka → score updater (idempotent per event) → ZINCRBY lb:{tenant}:{region}:{month} amount
repId
API: top 100 → ZREVRANGE … WITHSCORES ; my rank → ZREVRANK ; neighbours → ZREVRANGE rank-5 rank+5
Period close → snapshot to Postgres for payouts; key TTL after the period
```

The flow:

1. Sales events go to Kafka.
2. A score updater reads them. It is idempotent per event — the same event never counts twice.
3. It runs `ZINCRBY` on a key like `lb:{tenant}:{region}:{month}`.
4. The API serves the top 100 with `ZREVRANGE`, "my rank" with `ZREVRANK`, and neighbours with a range around your rank.
5. When the period closes, you snapshot the board to Postgres for payouts. Then the Redis key expires through a TTL (time to live).

The real source of truth stays in the database or ledger. The board in Redis is only a fast view, and you can rebuild it from the events at any time.

How does it work?

Ties. Two reps may have the same score. Put a tiebreaker inside the score itself. For example: `score × 10¹⁰ + (MAX_TS − achievedAt)`. The person who reached the score earlier gets a slightly higher number, so they rank higher.

Going beyond one sorted set (tens of millions of members):

1. Split users across shards by a hash of the user ID.
2. For the global top N, take each shard's top N and merge them.
3. For any user's global rank, you can estimate it using a *histogram* of score buckets — counts of how many users fall in each score range.

Example

A Gold-tier rep in the North region opens the app. It shows: "Rank 37 of 2,140 this month, ₹4,200 behind #36".

This is just `ZREVRANK` for the rank, plus `ZSCORE` for this user and the user one place above. It takes about 1 ms.

When would I use it?

Gamified incentives for field reps, gaming, contests, trending lists, and "top K" dashboards.

What can go wrong?

- **Double-counted events.** Remove duplicates by event ID before calling `ZINCRBY`.
- **Boards that grow forever.** Set a TTL on each period's key.
- **Calculating ranks from SQL on every request.** This is far too slow at scale.
- **Ties not handled in a fixed way.** Two users swap places randomly between page loads.

Interview questions

Q1. Which data structure would you use? A sorted set built on a skip list — Redis ZSETs. Score updates and rank lookups are O(log n), and fetching the top k is O(log n + k).

Q2. How do you scale beyond one Redis node? Shard the users. For the global top k, merge each shard's top k. For a global rank, estimate it using histograms of scores.

13.10 Typeahead / Search Autocomplete

Why?

> **In simple words:** As the user types each letter, you must show the best suggestions in a few milliseconds. So you work out the answers in advance, store them in a tree of letters (a trie), and just look them up.

Functional (what it must do):

- As the user types, return the top 5–10 suggestions, ranked by popularity. Optionally, personalise them for the user.

- Update popularity from search logs.

Non-functional (how well it must do it):

- Under 50 ms end to end, because it runs on every keystroke.
- Very high QPS (queries per second).
- Suggestions may be minutes or even hours old — that is acceptable.

This question tests tries, precomputation and caching.

What is happening internally?

Estimates: 10M daily active users × 10 searches each × about 6 keystrokes ≈ 600M requests per day. That is about 7k QPS on average, and around 20k QPS at peak.

```
Search logs → Kafka → aggregation job (hourly/daily: Spark/Flink) → top queries with counts
   → build a trie in which every node stores its precomputed top-K completions
   → serialise, shard by prefix range, load into the memory of Suggest servers (or Redis)
Client (debounce ~100 ms, cache prefixes) → CDN/edge cache for popular prefixes → Suggest service (O(prefix
length) lookup)
```

The flow:

1. Search logs go into Kafka.
2. An aggregation job (Spark or Flink, run hourly or daily) counts the top queries.
3. You build a *trie* — a tree where each step down is one letter. Every node stores its own precomputed top-K completions.
4. You serialise the trie, split it by prefix ranges (shards), and load it into the memory of the Suggest servers (or into Redis).
5. On the client side, the app waits about 100 ms after the last keystroke before sending a request. This is called *debounce*. The app also caches prefixes it has already seen.
6. Popular prefixes are served from a CDN/edge cache.
7. Anything else reaches the Suggest service, which looks it up in O(prefix length) — the time depends only on how many letters were typed.

How does it work?

To look up a prefix:

1. Walk down the trie, one character at a time.
2. At the last node, return the top-K list already stored there.

You never search the whole subtree at query time. Storing the top-K at every node uses more memory, but it makes lookups very fast. That is the trade-off: memory for speed.

Example

A retailer types "maz". You walk m → a → z. That node already holds ["maaza 600ml", "maaza 1.2l", "maaza 250ml"...]. It returns from memory in about 1 ms.

For SKU search in an ordering assistant like NOVA, a trie alone is not enough. Combine it with Elasticsearch for:

- *fuzzy matching* — handling spelling mistakes ("mazza" → "maaza"),
- *transliteration* — handling Hinglish queries typed in English letters.

When would I use it?

Search boxes, product and SKU lookup, address autocomplete, and command palettes.

What can go wrong?

- **Memory blows up.** Cap the size of top-K and remove rare prefixes.
- **Uneven shards** — one shard gets far more traffic. Shard by prefix ranges, and split a range when its load is high.
- **Offensive or old suggestions.** Use filter lists, and add a small real-time layer for trending queries.
- **Calling the server on every keystroke** without debounce and client caching. This multiplies the load.

Interview questions

Q1. How do you make suggestions fast? Keep a trie in memory with the top-K precomputed at each node. Add edge and client caching, and debounce requests so you don't call on every keystroke.

Q2. How do you keep suggestions fresh? Rebuild from the logs in regular batches, and merge in a small real-time trending layer at query time.

13.11 Metrics and Logging Pipeline

Why?

> **In simple words:** Thousands of services send a flood of numbers and logs. You catch the flood in Kafka, store it in a column-based database like ClickHouse, keep pre-summed totals for dashboards, and move old data to cheap storage.

Functional (what it must do):

- Collect events, metrics and logs from thousands of services and devices.
- Query totals and averages over time and by dimensions (tenant, service, region...).
- Show dashboards.
- Raise alerts.
- Apply retention policies — delete data after a set time.

Non-functional (how well it must do it):

- Take in 1M events per second.
- Query billions of rows in seconds.
- Keep the cost low.

This question tests streaming ingestion, columnar storage and tiering (hot vs cold storage). It is a strong question for you because of your ClickHouse and Trino experience.

What is happening internally?

```
Agents/SDKs (batch + compress) → ingestion gateway (auth, rate limits) → Kafka (partition by tenant/service)
  → stream processors (validation, enrichment, pre-aggregation, downsampling)
  → ClickHouse (MergeTree: PARTITION BY day, ORDER BY (tenant, metric, timestamp)); materialised views for
rollups
  → object storage (Parquet) for raw and cold data, queried with Trino
Query service → Grafana dashboards ; alerting service evaluates rules on rollups
```

The flow:

1. Agents and SDKs on each service collect data, batch it and compress it.
2. An ingestion gateway checks auth and applies rate limits.
3. Data goes into Kafka, partitioned by tenant or service.
4. Stream processors validate it, enrich it (add extra fields), pre-aggregate it (sum it up early) and downsample it (keep fewer points for old data).
5. Data lands in ClickHouse, in a MergeTree table with `PARTITION BY day` and `ORDER BY (tenant, metric, timestamp)`. Materialised views keep the rollups (summary tables).
6. Raw and cold data also goes to object storage as Parquet files, and you query it with Trino.
7. A query service feeds Grafana dashboards. An alerting service checks rules against the rollups.

How does it work?

- **Kafka absorbs bursts.** It sits between the producers and storage, so a sudden spike doesn't crash the database. Producers and storage don't depend on each other's speed.
- **Batch inserts** — thousands of rows per insert — keep ClickHouse efficient. It hates many tiny inserts.
- **Materialised views** keep per-minute and per-hour rollups up to date at the moment data is inserted. So dashboards never need to sum raw data.
- **TTL rules** move data from fast SSD (hot, 7 days) to cheap object storage (cold), and delete it when the retention period ends.

Example

Query: "p99 latency of the redemption API per tenant over the last 24 hours". (p99 = the time within which 99% of requests finished.)

It is served from a per-minute rollup table — about 1,440 rows per tenant. It takes tens of milliseconds. Without the rollup, you would scan a billion raw events.

When would I use it?

Observability platforms, product analytics, IoT telemetry, and audit-event analytics.

What can go wrong?

- **High-cardinality labels.** *Cardinality* means the number of distinct values. If you use user IDs as metric tags, you get millions of separate series, and storage explodes.
- **Too many small inserts.** ClickHouse slows down badly.
- **Late-arriving data.** Some events arrive after their time window has been summed. Use watermarks (a marker of "how late we still accept data") and re-aggregation windows.
- **One noisy tenant fills up ingestion** for everyone. Use per-tenant quotas.
- **Dashboards that query raw data** instead of rollups — slow and expensive.

Interview questions

Q1. Why a columnar store for metrics and logs? Queries read only a few columns, but across huge time ranges. A columnar store reads just those columns, compresses them well and scans them in vectorised batches (many values at once). So these aggregations are fast and cheap.

Q2. How do you handle 10× traffic bursts? Kafka buffers the burst. Consumers scale up based on lag (how far behind they are). Ingestion applies per-tenant rate limits and sampling.

13.12 Distributed Job Scheduler (cron at scale)

Why?

> **In simple words:** You have millions of jobs that must run at set times, and several scheduler machines. Each machine picks up a batch of due jobs inside a database lock, so no two machines take the same job. Each job run has a unique ID, so if it is delivered twice, the second copy does nothing.

Almost every product has work that must happen at a time, not on a user click:

- Expire loyalty points every night at 02:00 for each tenant.
- Send "your offer ends tomorrow" reminders.
- Run end-of-month incentive payouts.
- Retry a failed webhook in 10 minutes.

On one server, a Spring `@Scheduled` method or Linux `cron` is enough. At scale it breaks:

- If that one server is down at 02:00, the job never runs.
- If you run three copies of the service, the job runs three times.
- One server cannot run 10 million jobs per day on time.

So you need a **distributed job scheduler**: a service that stores jobs, decides when each one is due, and makes sure each due run happens — on time, once in effect, even when machines crash.

Functional requirements (what it must do):

1. Create a job: one-time ("run at 2026-10-05 10:00") or repeating (a cron expression like `0 2 * * *` plus a time zone).
2. Run each job at its time.
3. Retry failed runs with backoff (waiting longer after each failure).
4. Keep a history of runs: status, start, end, error.
5. Pause, resume, cancel and "run now".

Non-functional requirements (how well it must do it):

- **No missed runs and no double effects.** Delivery is *at-least-once* (every run happens, rarely twice); jobs are written so a repeat does no harm. Together this gives *effectively once*.
- **On time**: start within a few seconds of the due time.
- **Scale**: 10 million job definitions, peaks of many thousands of runs per second (everyone schedules at midnight).
- **No single point of failure**: losing any one machine must not stop scheduling.

Back-of-envelope estimate (say these numbers aloud in the interview):

- 10M jobs, average 3 runs/day → 30M runs/day ÷ ~10^5 s/day ≈ **300 runs/s average**.
- Peaks are the real problem. If 20% of jobs are due at exactly 00:00, that is 2M runs in one minute ≈ **33,000 runs/s for a minute**.

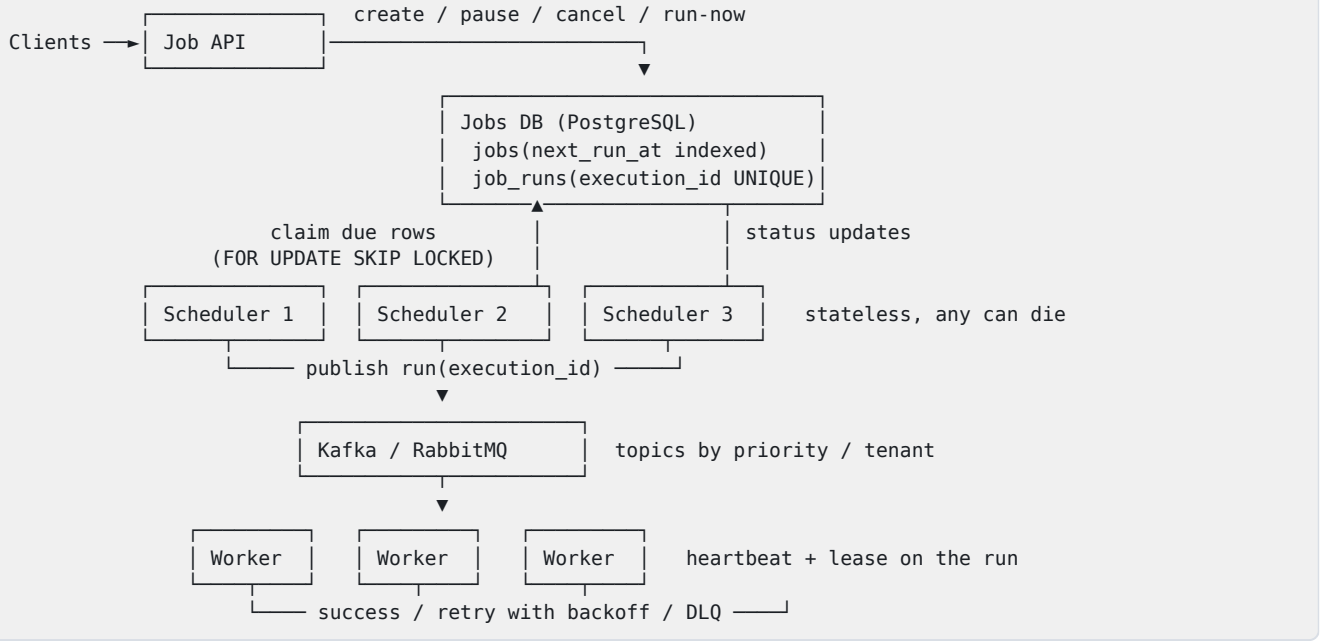
- Storage: a job row ≈ 500 bytes → 10M × 500 B = **5 GB**. A run-history row ≈ 200 bytes → 30M/day × 200 B = **6 GB/day**; keep 30 days hot (~180 GB), archive the rest.

So the database for definitions is small. The hard parts are **finding due jobs fast**, **spreading the peak**, and **never running a job twice in effect**.

This question tests coordination between machines, idempotency (safe to repeat), time handling and failure thinking.

What is happening internally?

The design separates **deciding when** (scheduler) from **doing the work** (workers). A queue sits between them.



The two main tables:

```
CREATE TABLE jobs (
  job_id        BIGINT PRIMARY KEY,
  tenant_id     BIGINT       NOT NULL,
  name          TEXT         NOT NULL,
  schedule_type TEXT         NOT NULL,          -- 'ONCE' or 'CRON'
  cron_expr     TEXT,                           -- e.g. '0 2 * * *'
  time_zone     TEXT         NOT NULL,          -- e.g. 'Asia/Kolkata' (never store a fixed offset)
  payload       JSONB        NOT NULL,          -- what the worker needs: handler name + arguments
  status        TEXT         NOT NULL,          -- ACTIVE, PAUSED, CANCELLED, DONE
  next_run_at   TIMESTAMPTZ,                    -- next fire time in UTC; NULL when nothing is due
  misfire_policy TEXT        NOT NULL DEFAULT 'RUN_ONCE',  -- RUN_ONCE, RUN_ALL, SKIP
  max_attempts  INT          NOT NULL DEFAULT 5,
  timeout_sec   INT          NOT NULL DEFAULT 900,
  version       BIGINT       NOT NULL DEFAULT 0  -- optimistic locking for API edits
);
-- partial index: only active jobs, ordered by time -> "what is due?" is a cheap index range scan
CREATE INDEX idx_jobs_due ON jobs (next_run_at) WHERE status = 'ACTIVE';

CREATE TABLE job_runs (
  execution_id  TEXT PRIMARY KEY,               -- job_id + '@' + scheduled_for  (idempotency key)
  job_id        BIGINT       NOT NULL,
  scheduled_for TIMESTAMPTZ  NOT NULL,
  attempt       INT          NOT NULL,
  status        TEXT         NOT NULL,          -- QUEUED, RUNNING, SUCCEEDED, FAILED, DEAD
  worker_id     TEXT,
  lease_until   TIMESTAMPTZ,                    -- worker must heartbeat before this
```

```
  started_at    TIMESTAMPTZ, finished_at TIMESTAMPTZ, error TEXT
);
```

Key ideas in this layout:

- **`next_run_at` is always stored in UTC.** The cron expression and time zone are used only to *calculate* it. So the "is it due?" check is a simple number comparison.
- **The partial index** contains only ACTIVE jobs, sorted by time. Finding due jobs is like opening a register at today's page — you never scan paused or finished jobs.
- **`execution_id` is UNIQUE.** The same scheduled slot can never create two run rows. This is the idempotency key the worker checks.

How does it work?

A. The scheduler loop (every scheduler node runs this, every ~1 second):

```
BEGIN;
SELECT job_id, cron_expr, time_zone, next_run_at
FROM   jobs
WHERE  status = 'ACTIVE' AND next_run_at <= now()
ORDER  BY next_run_at
LIMIT  500
FOR UPDATE SKIP LOCKED;            -- rows locked by another scheduler are skipped, not waited on

-- for each row:
INSERT INTO job_runs (execution_id, job_id, scheduled_for, attempt, status)
VALUES ('42@2026-10-03T20:30:00Z', 42, '2026-10-03T20:30:00Z', 1, 'QUEUED')
ON CONFLICT (execution_id) DO NOTHING;            -- already created? do nothing
UPDATE jobs SET next_run_at = <next fire time after the current one> WHERE job_id = 42;
COMMIT;
-- after commit: publish the run to the queue (or use an outbox table so publish can't be lost - see 11.15)
```

Step by step:

1. Open a transaction and lock up to 500 due rows. `SKIP LOCKED` means scheduler 2 does not wait for scheduler 1's rows — it simply takes the next free rows. Three schedulers get three different batches.
2. For each job, create a run row. The `execution_id` is `job_id@scheduled_time`.
3. Move `next_run_at` forward to the next fire time. Now the job is no longer "due", so no one else can pick this slot.
4. Commit. Steps 2 and 3 happen together or not at all. If the scheduler crashes before commit, the locks are released and another scheduler picks the same rows a second later. Nothing is lost.
5. Publish each run to the queue. To make "DB change + publish" safe, write the message to an **outbox table** in the same transaction and let a relay publish it (11.15).

Think of a railway parcel office with three clerks and one big register sorted by delivery time. Each clerk takes a pencil, marks the next free lines as "mine", writes a delivery slip for each and moves the line's date to its next delivery. A clerk never touches lines another clerk has marked.

B. Computing the next run time (time zones and DST).

- Parse the cron expression in the job's own time zone, then convert the result to UTC. In Spring: `CronExpression.parse("0 0 2 * * *").next(ZonedDateTime.now(zone))`.
- **DST gap** (clocks jump 02:00 → 03:00): a 02:30 job has no 02:30 that night. Java's `atZone` moves it forward to 03:30. Decide and document that policy.
- **DST overlap** (clocks go back, 01:30 happens twice): run once, at the first occurrence.

- India has no DST, but your global tenants do. Always store the zone name (`America/New_York`), never a fixed offset (`-05:00`).

C. Workers: leases, heartbeats and idempotency.

1. A worker takes a run from the queue.
2. It updates the run row: `status = RUNNING, worker_id = me, lease_until = now() + 60s`. This is a **lease** — "I own this run for 60 seconds".
3. While working, it renews the lease every ~20 seconds (a **heartbeat**).
4. If the worker dies, heartbeats stop and the lease expires. A **reaper** (a small background task) finds RUNNING runs with expired leases and puts them back on the queue.
5. Because a run can be delivered twice (queue redelivery, reaper, retry), the job's own work must be idempotent. Example: the points-expiry handler writes `INSERT ... ON CONFLICT (execution_id, account_id) DO NOTHING`, so a second run changes nothing. For long jobs, save a **checkpoint** (e.g. last processed account ID) so a retry continues instead of starting over.

D. Retries and the dead-letter queue.

- On failure: if `attempt < max_attempts`, re-queue with delay = `base × 2^attempt` plus random **jitter** (so 10,000 failed jobs don't all retry in the same second).
- After the last attempt: mark the run `DEAD`, send it to a **DLQ** (dead-letter queue) and alert the job owner. A human fixes the cause and clicks "replay".
- Retry only errors that can heal (timeouts, 503s). A validation error will fail every time — send it to DLQ immediately.

E. Missed runs after downtime (misfire policy).

Suppose the whole system was down from 01:00 to 05:00. An hourly job missed four slots. When it comes back, `next_run_at` is in the past. What should happen? This is the **misfire policy**, chosen per job:

Policy	Behaviour	Use it for
`RUN_ONCE` (default)	Run one catch-up now, then continue the normal schedule	Cleanup, expiry, sync jobs — one run catches up everything
`RUN_ALL`	Run every missed slot, in order	Billing or reports per period, where each slot is its own result
`SKIP`	Ignore missed slots, wait for the next one	Reminders and notifications — a 2-hour-late "good morning" is worse than none

F. Handling the midnight peak and scaling to millions of jobs.

1. **Jitter at create time.** For jobs that don't need an exact time ("daily cleanup"), store a random offset in `next_run_at` (say 0–10 minutes). The 00:00 spike becomes a gentle 10-minute hill.
2. **Pre-load the near future.** A loader moves jobs due in the next ~5 minutes from the DB into a fast in-memory structure — a **Redis sorted set** (score = due time, `ZRANGEBYSCORE` to pop due ones) or a **timing wheel** (a ring of time buckets, like a clock face with a bucket per second). The DB is touched once per job per window, not every second.
3. **Partition the work.** Split jobs into, say, 256 buckets by `hash(job_id) % 256`. Each scheduler node holds a **lease** on a set of buckets (stored in the DB, ZooKeeper or etcd). If a node dies, its bucket leases expire and other nodes take them over. Within a bucket, ordering by time is local and cheap.

4. **Separate queues by priority or tenant.** One tenant scheduling a million jobs must not delay everyone else. Give each priority (or large tenant) its own topic, and cap concurrency per tenant (**fairness**).
5. **Scale workers independently** of schedulers — workers do the heavy lifting; autoscale them on queue lag.

G. What to monitor.

- **Schedule lag**: `now − scheduled_for` when a run starts. This is the main health number; alert if p99 > 30 s.
- Queue lag (messages waiting), runs per second, failure rate per job type, DLQ size, number of expired leases.

Example

Daily points expiry for 1,000 tenants at 02:00 IST, with three scheduler nodes. The code below is a single-file simulation of the core ideas: DST-safe next-run calculation, atomic claiming by several nodes, at-least-once delivery with duplicates, idempotent execution, retries and a dead-letter queue. Run it with `java SchedulerDemo.java`.

```
// FILE: SchedulerDemo.java
import java.time.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

public class SchedulerDemo {

    // ---- 1. Job definition and execution record ----
    enum ExecStatus { CLAIMED, RUNNING, SUCCEEDED, FAILED, DEAD }

    static final class Job {
        final String id; final LocalTime runAt; final ZoneId zone; final int maxAttempts;
        Instant nextRunAt;                      // UTC instant of the next fire time
        Job(String id, LocalTime runAt, ZoneId zone, int maxAttempts, Instant firstRun) {
            this.id = id; this.runAt = runAt; this.zone = zone; this.maxAttempts = maxAttempts;
this.nextRunAt = firstRun;
        }
    }

    record Execution(String executionId, String jobId, Instant scheduledFor, int attempt) {}

    // ---- 2. "Daily at HH:mm in a time zone" -> next UTC instant (DST-safe) ----
    static Instant nextDailyRun(LocalTime at, ZoneId zone, Instant after) {
        LocalDate day = after.atZone(zone).toLocalDate();
        for (int i = 0; i < 3; i++) {
            // atZone() moves a time that falls in a DST gap forward, and picks the earlier offset in an
overlap
            Instant candidate = day.plusDays(i).atTime(at).atZone(zone).toInstant();
            if (candidate.isAfter(after)) return candidate;
        }
        throw new IllegalStateException("unreachable");
    }

    // ---- 3. Job store: claim due jobs atomically ----
    // `synchronized` stands in for the database transaction with SELECT ... FOR UPDATE SKIP LOCKED:
    // pick due rows, create one execution per row, move next_run_at forward, commit.
    static final class JobStore {
        private final Map<String, Job> jobs = new ConcurrentHashMap<>();
        void add(Job j) { jobs.put(j.id, j); }

        /** Atomically claim up to `limit` due jobs. A claimed job is no longer due, so no other node sees
it. */
        synchronized List<Execution> claimDue(String node, Instant now, int limit) {
            List<Execution> claimed = new ArrayList<>();
```

```
            jobs.values().stream()
                .filter(j -> !j.nextRunAt.isAfter(now))
                .sorted(Comparator.comparing(j -> j.nextRunAt))
                .limit(limit)
                .forEach(j -> {
                    String execId = j.id + "@" + j.nextRunAt;                    // idempotency key
                    claimed.add(new Execution(execId, j.id, j.nextRunAt, 1));
                    j.nextRunAt = nextDailyRun(j.runAt, j.zone, j.nextRunAt);       // advance in the same
"transaction"
                });
            return claimed;
        }
        Job get(String id) { return jobs.get(id); }
    }

    // ---- 4. Worker side: idempotent execution + retry with backoff + dead-letter queue ----
    static final Set<String> doneExecutions = ConcurrentHashMap.newKeySet(); // "processed executions" table
    static final List<String> deadLetters = Collections.synchronizedList(new ArrayList<>());
    static final AtomicInteger realRuns = new AtomicInteger(), retries = new AtomicInteger();

    static void execute(Execution e, JobStore store, BlockingQueue<Execution> queue, Set<String> flakyJobs) {
        if (doneExecutions.contains(e.executionId())) return;          // duplicate delivery -> no-op
        boolean ok = !(flakyJobs.contains(e.jobId()) && e.attempt() < 3); // flaky jobs succeed on attempt 3
        if (e.jobId().equals("broken")) ok = false;                     // always fails
        if (ok) {
            if (doneExecutions.add(e.executionId())) realRuns.incrementAndGet();
            return;
        }
        int max = store.get(e.jobId()).maxAttempts;
        if (e.attempt() >= max) { deadLetters.add(e.executionId()); return; }
        long backoffMs = (long) Math.pow(2, e.attempt()) * 100;         // 200, 400, 800 ms ... (simulated)
        queue.add(new Execution(e.executionId(), e.jobId(), e.scheduledFor(), e.attempt() + 1));
        retries.incrementAndGet();   // a real system waits backoffMs (plus jitter) before redelivery
    }

    public static void main(String[] args) throws Exception {
        ZoneId ist = ZoneId.of("Asia/Kolkata"), ny = ZoneId.of("America/New_York");

        // DST check: "daily at 02:30 New York" on the night clocks jump 02:00 -> 03:00 (8 March 2026)
        Instant beforeGap = ZonedDateTime.of(2026, 3, 7, 12, 0, 0, 0, ny).toInstant();
        Instant r1 = nextDailyRun(LocalTime.of(2, 30), ny, beforeGap);
        Instant r2 = nextDailyRun(LocalTime.of(2, 30), ny, r1);
        System.out.println("NY 02:30 job, run 1: " + r1.atZone(ny) + "  (02:30 does not exist that night)");
        System.out.println("NY 02:30 job, run 2: " + r2.atZone(ny));
        System.out.println("IST 02:00 job fires at UTC: " + nextDailyRun(LocalTime.of(2, 0), ist,
Instant.parse("2026-10-03T00:00:00Z")));

        // 1,000 tenant jobs due "now", 3 scheduler nodes racing to claim them
        JobStore store = new JobStore();
        Instant now = Instant.parse("2026-10-03T20:30:00Z");
        for (int i = 1; i <= 1000; i++) store.add(new Job("tenant-" + i, LocalTime.of(2, 0), ist, 4, now));
        store.add(new Job("broken", LocalTime.of(2, 0), ist, 3, now));
        Set<String> flaky = Set.of("tenant-7", "tenant-42");

        BlockingQueue<Execution> queue = new LinkedBlockingQueue<>();
        Map<String, Integer> claimedBy = new ConcurrentHashMap<>();
        ExecutorService schedulers = Executors.newFixedThreadPool(3);
        for (int n = 1; n <= 3; n++) {
            String node = "sched-" + n;
            schedulers.submit(() -> {
                List<Execution> batch;
                while (!(batch = store.claimDue(node, now, 50)).isEmpty()) {
                    for (Execution e : batch) { claimedBy.merge(e.executionId(), 1, Integer::sum);
queue.add(e); }
                }
            });
        }
        schedulers.shutdown(); schedulers.awaitTermination(10, TimeUnit.SECONDS);
        long doubleClaims = claimedBy.values().stream().filter(c -> c > 1).count();
        System.out.println("executions claimed: " + claimedBy.size() + ", claimed twice: " + doubleClaims);

        // Simulate the queue delivering 5 messages twice (at-least-once delivery)
```

```
        List<Execution> firstFive = new ArrayList<>(queue).subList(0, 5);
        queue.addAll(firstFive);

        ExecutorService workers = Executors.newFixedThreadPool(8);
        while (!queue.isEmpty()) {
            List<Execution> drained = new ArrayList<>(); queue.drainTo(drained);
            CountDownLatch latch = new CountDownLatch(drained.size());
            for (Execution e : drained) workers.submit(() -> { try { execute(e, store, queue, flaky); }
finally { latch.countDown(); } });
            latch.await();
        }
        workers.shutdown();

        System.out.println("real runs: " + realRuns.get() + " (1000 tenants, duplicates ignored)");
        System.out.println("retries: " + retries.get() + ", dead letters: " + deadLetters);
        System.out.println("tenant-1 next run: " + store.get("tenant-1").nextRunAt.atZone(ist));
    }
}
```

Output:

```
NY 02:30 job, run 1: 2026-03-08T03:30-04:00[America/New_York]  (02:30 does not exist that night)
NY 02:30 job, run 2: 2026-03-09T02:30-04:00[America/New_York]
IST 02:00 job fires at UTC: 2026-10-03T20:30:00Z
executions claimed: 1001, claimed twice: 0
real runs: 1000 (1000 tenants, duplicates ignored)
retries: 6, dead letters: [broken@2026-10-03T20:30:00Z]
tenant-1 next run: 2026-10-05T02:00+05:30[Asia/Kolkata]
```

What the output proves:

1. **DST**: on 8 March 2026 New York has no 02:30, so the run moves to 03:30. The next day it is back to 02:30.
2. **Claiming**: three nodes race over 1,001 due jobs, and not one execution is claimed twice — the same guarantee `FOR UPDATE SKIP LOCKED` gives you.
3. **Idempotency**: five messages were delivered twice, but there are exactly 1,000 real runs. The `execution_id` check turned the duplicates into no-ops.
4. **Retries**: the two flaky tenants succeed on attempt 3. The `broken` job fails 3 times and lands in the dead-letter queue instead of retrying forever.
5. **Next run**: after the 02:00 run on 4 October (IST), the next one is 5 October 02:00 IST.

In a real Spring Boot service the pieces map like this: the claim loop is a `@Scheduled(fixedDelay = 1000)` method calling a repository with a native `FOR UPDATE SKIP LOCKED` query inside `@Transactional`; publishing uses the outbox (8.5 / 11.15); workers are `@KafkaListener` consumers; `CronExpression` from Spring computes the next time.

When would I use it?

- **Use a distributed scheduler** when you have many jobs (thousands to millions), per-tenant schedules, user-created schedules ("remind me in 3 days"), or runs that must survive machine failures.
- **Simpler options when the need is small:**

Need	Simplest fit
A few fixed jobs in one Spring service, must run once across replicas	`@Scheduled` + **ShedLock** (a DB lock per job name)
Fixed jobs per service, on Kubernetes	**Kubernetes CronJob** (a new pod per run)

Need	Simplest fit
Many jobs with JDBC persistence and clustering, inside a Java app	**Quartz** in clustered mode
Long, multi-step workflows with timers and retries per step	**Temporal** (durable workflows)
Data pipelines with dependencies between tasks (DAGs)	**Airflow**
"Do X in N minutes" once, at moderate scale	Delayed messages (RabbitMQ delayed exchange, SQS delay) or a Redis sorted set

In the interview, mention one of these and then say why you are designing your own: scale, multi-tenancy, or custom policies.

What can go wrong?

Problem	What you see	Fix
Double run	Points expired twice, two emails sent	Atomic claim (`SKIP LOCKED` / leases) + UNIQUE `execution_id` + idempotent handlers
Lost run	Scheduler moved `next_run_at` but crashed before publishing	Outbox in the same transaction; reaper re-queues QUEUED runs older than N minutes
Stuck run	Worker hung or died; run RUNNING forever	Leases + heartbeats + reaper; per-job `timeout_sec`
Missed runs after outage	Burst of catch-up runs, or silent gaps	Per-job misfire policy (`RUN_ONCE` / `RUN_ALL` / `SKIP`)
Midnight thundering herd	33,000 runs/s spike, DB and downstream overload	Jitter, pre-loading into Redis / timing wheel, rate limits per tenant
Time zone / DST bugs	Job runs at 03:30, or twice, or not at all	Store zone name, compute next run in that zone, test gap and overlap dates
Clock skew between nodes	One node thinks a job is due early	Use the database's `now()` as the single clock, not each node's clock
Retry storm	Downstream is down; all jobs retry together and keep it down	Exponential backoff + jitter, a circuit breaker per downstream (8.7), DLQ
Noisy tenant	One tenant's million jobs delay everyone	Per-tenant queues or concurrency caps (fairness)
Run-history table grows forever	Slow queries, huge DB	Partition `job_runs` by month and drop old partitions (12.8)

Interview questions

Q1. How do you ensure a job runs once even with multiple scheduler nodes? Two layers. First, claiming is atomic: schedulers select due rows with `FOR UPDATE SKIP LOCKED` (or own partitions through leases), create the run and move `next_run_at` forward in the same transaction — so a slot can be claimed only once. Second, delivery is still at-least-once (queue redelivery, worker crashes), so each

run has an `execution_id` = job + scheduled time with a UNIQUE constraint, and handlers are idempotent. Together: effectively once.

Q2. How do you scale to millions of near-term jobs? Keep definitions in a DB with a partial index on `next_run_at`. Partition jobs into buckets by hash and give each scheduler a lease on some buckets. Pre-load jobs due in the next few minutes into a Redis sorted set or a timing wheel so the DB is not polled every second. Publish runs to a queue and scale workers separately based on queue lag. Spread fixed-time peaks with jitter.

Q3. A worker picks up a job and dies halfway. What happens? The worker holds a lease on the run and renews it with heartbeats. When it dies, the lease expires; a reaper finds the run and re-queues it as the next attempt. Because the handler is idempotent and saves checkpoints, the retry continues safely without double effects.

Q4. The whole system was down for 4 hours. What happens to jobs that should have run? Their `next_run_at` is in the past, so they are due immediately. The per-job misfire policy decides: run once to catch up (cleanup jobs), run every missed slot (per-period billing), or skip to the next slot (time-sensitive notifications). Also throttle the catch-up so the backlog doesn't overload downstream systems.

Q5. How do you handle time zones and daylight saving time? Store the IANA zone name with the job and always store `next_run_at` in UTC. Compute the next time with the cron expression in the job's zone, then convert to UTC. Define policy for the DST gap (move forward) and overlap (run at the first occurrence) and test both dates.

Q6. Why put a queue between schedulers and workers? It separates "when" from "how". Schedulers stay small and fast; workers can be slow, many and autoscaled. The queue absorbs peaks, gives retries and a DLQ, and lets you route jobs by priority or tenant. Without it, a slow job would block the scheduler loop and delay every other job.

Q7. When would you not build this, and use something off the shelf? For a few fixed jobs in one service, `@Scheduled` + ShedLock or a Kubernetes CronJob is enough. For long multi-step workflows, Temporal; for data pipelines with dependencies, Airflow; for clustered Java scheduling at moderate scale, Quartz. Build your own when you need millions of user-defined, multi-tenant schedules with custom misfire, fairness and retry policies.

13.13 Ride Matching and Nearby Search (Uber, delivery partner assignment)

Why?

> **In simple words:** Keep every driver's latest location in memory, grouped into map cells. When a rider asks, look in nearby cells, rank drivers by real travel time, and lock one driver before offering the ride so two riders never get the same driver.

Functional (what it must do):

- Drivers send their location every few seconds.
- Riders request rides.
- Match the nearest suitable driver who is free.
- Track the trip.

Non-functional (how well it must do it):

- Match within 1–2 seconds.
- Handle a huge rate of location writes.
- High availability for each city.

This question tests geo-indexing (searching by location), keeping hot state in memory, atomic assignment (all-or-nothing, so no clash) and partitioning by city. (The LLD version is in 10.17.)

What is happening internally?

Estimates: 1M active drivers × 1 update every 4 s = 250k writes per second. This is too many for a disk database. So keep current locations in memory.

A *geo index* is a structure that quickly answers "who is near this point?". The main options:

Geo index	Idea	Notes
Geohash	Turn lat/lng into a base32 string; a longer shared prefix ≈ closer together	Redis GEO (`GEOADD` , `GEOSEARCH`); at cell edges, also check the neighbouring cells
Quadtree	Keep splitting busy areas into four smaller boxes	Adjusts to how crowded an area is; in-memory
H3 / S2	Hexagon cells / cells on a sphere	Every neighbour is the same distance away; good for surge pricing per cell

```
Driver app → location service (WebSocket/UDP) → in-memory geo index per city shard (Redis GEO or custom)
                                                  └→ Kafka location stream → trip tracking, ETA, analytics
Rider request → matching service: candidates in nearby cells (expand rings until K found)
   → rank by ETA (routing engine), rating, acceptance probability → offer to driver (10–15 s timeout)
   → atomic reserve (AVAILABLE → OFFERED) to prevent double assignment → accept → trip ACCEPTED
Trip service (state machine, Postgres) · Pricing service (surge per H3 cell from supply/demand)
```

The flow:

1. The driver app sends locations (over WebSocket or UDP) to the location service.
2. The location service updates an in-memory geo index for that city's shard (Redis GEO or a custom index).
3. It also writes the location stream to Kafka, for trip tracking, ETA and analytics.
4. A rider request goes to the matching service. It finds candidates in nearby cells. If it doesn't find K drivers, it expands outward ring by ring.
5. It ranks candidates by ETA (from a routing engine), rating and how likely they are to accept.
6. It atomically reserves the top driver: AVAILABLE → OFFERED. This prevents double assignment.
7. It offers the ride, with a 10–15 s timeout.
8. If the driver accepts, the trip becomes ACCEPTED.
9. The trip service keeps the trip's state machine in Postgres. The pricing service sets surge per H3 cell, based on supply and demand.

How does it work?

1. A request comes in.
2. Search nearby — about 1 km first, then wider if needed.

3. Pick the top candidates by ETA.
4. Reserve the first one atomically.
5. Send the offer.
6. If the driver declines or the timer runs out, release them and try the next one.
7. When a driver accepts, create the trip and notify the rider.

Example

A rider in Gurugram Sector 29 requests a ride:

1. Search the rider's H3 cell plus 1 ring around it → 14 free drivers.
2. Rank them by ETA using the routing service.
3. Driver D7 (3 min away) is reserved and offered the ride. D7 declines.
4. Driver D2 (4 min away) is reserved next, and accepts within 6 s.
5. The trip is created.

When would I use it?

Ride-hailing, assigning food and grocery delivery partners (13.18), assigning field-force visits and planning routes (related to your Route Optimization product), and any "find what's near me" search.

What can go wrong?

- **Double assignment** if you don't reserve the driver atomically.
- **Using straight-line distance instead of road ETA.** A driver across a river may look close but be 20 minutes away.
- **GPS noise.** Snap locations to the nearest road.
- **Hot cells at big events** — too many people in one cell. Use smaller cells and surge pricing.
- **Queries that cross cities.** Partition by city to avoid them.

Interview questions

Q1. How do you find nearby drivers efficiently? Keep a geo index (geohash, H3 or quadtree) of current locations in memory. Search the rider's cell first, then the rings of neighbouring cells.

Q2. How do you prevent two riders getting the same driver? Change the driver's status atomically before sending the offer — with Redis SET NX, CAS (compare-and-set), or a conditional update. Release the driver if they decline or the offer times out.

Q3. How would you handle 250k location updates per second? Keep only the latest location per driver, in memory, sharded by city. Write the stream to Kafka for history. Avoid a synchronous DB write for each update.

13.14 Incentive Engine at Scale (your system)

Why?

> **In simple words:** Field reps earn bonuses for hitting targets. Show them their progress quickly using live streaming numbers, but calculate the actual money carefully, at month end, from the original data.

Functional requirements (what the system must do):

- Tenants (client companies) define incentive schemes for field reps and distributors, per period. A scheme sets targets on sales, visits, coverage or focus SKUs (products the company wants to push).
- Show live progress towards each target.
- Calculate payouts when the period closes.
- Explain how each result was calculated.
- Handle data that arrives late or gets corrected.

Non-functional requirements (how well it must do it):

- 700+ tenants and hundreds of thousands of users.
- Progress must be fresh within a few minutes.
- Payouts must be exactly correct and auditable (anyone can check how they were reached).
- Heavy load at month end.

This question tests streaming vs batch processing, rules engines (10.12), versioning, and *multi-tenant fairness* — making sure one big client cannot slow down everyone else.

What is happening internally?

The flow, top to bottom:

1. Source events (orders, visits, new outlets) are captured with CDC (Change Data Capture — reading every change from the database log) or app events, and sent to Kafka.
2. A stream job keeps running counters per (tenant, user, period, metric). It stores them as aggregates in Postgres or ClickHouse.
3. The scheme evaluation service (a rules engine) re-checks a user when a metric changes, or every N minutes. It reads the versioned scheme definitions (cached), walks the condition tree and saves a progress snapshot.
4. A cached progress API serves mobile dashboards and leaderboards (13.9).
5. At period close, the system freezes the data cut-off. Then a batch job recalculates every user from the source of truth.
6. This produces payout records in a ledger. They go through a maker-checker approval (one person prepares, another approves). Then they are exported to payroll or a wallet.

```
Sources (orders, visits, outlet onboarding) → CDC/events → Kafka
  → metric aggregation (stream): per (tenant, user, period, metric) counters → Postgres/ClickHouse aggregates
  → scheme evaluation service (rules engine): on metric change or every N minutes per user
        reads versioned scheme definitions (cached) → evaluates the condition tree → writes a progress
snapshot
  → progress API (cached) → mobile dashboards, leaderboards (13.9)
Period close: freeze the data cut-off → recompute every user from the source of truth (batch)
  → payout records (ledger) → approval workflow (maker-checker) → payroll/wallet export
```

How does it work?

- **Streaming for progress, batch for money.** Small, quick updates (incremental updates) keep the app responsive. But at period close, a full batch recalculation is the final, official answer. It correctly includes late and corrected data.
- **Rule versioning.** A period is always evaluated with the scheme version that was active when the period started. Editing a scheme creates a new version; it does not overwrite the old one.

- **Idempotent, re-runnable payouts.** *Idempotent* means running it twice gives the same result as running it once. Each payout is keyed by (scheme, user, period, version). A re-run replaces draft payouts, and it never creates a duplicate of an approved payout.
- **Explainability.** The evaluation trace (the step-by-step record of how the rules were checked) is saved with each payout.
- **Fairness.** Each tenant gets its own queue and quota. So one large tenant's recalculation cannot starve the others.

Think of a school exam. During the term, you see your running marks on a notice board — quick but rough. The final report card is made carefully at the end, from the actual answer sheets.

Example

1. By the 20th, a rep has sold ₹62,000 of focus SKUs.
2. The progress card shows 124% of target, and "4 more new outlets to reach the next slab".
3. On the 28th, an order is returned. An event reduces the rep's sales.
4. At period close, the batch job recalculates from the source data.
5. The payout uses the corrected figure. The trace is attached, so the rep's manager can see exactly how it was calculated.

When would I use it?

Any calculation driven by rules over summed-up activity. Examples: sales incentives and commissions, loyalty tier evaluation, SLA and bonus calculations, gamification.

What can go wrong?

- **Paying from streaming counters.** Counters drift because of duplicate events or late data. Fix: recalculate from the source at close.
- **Editing rules mid-period without versions.** Fix: version every scheme change.
- **Evaluating on raw transaction tables for every request.** This is too slow. Fix: pre-aggregate (keep summed-up totals ready).
- **Month-end load spikes.** Fix: autoscale and partition work by tenant.
- **Payouts nobody can explain.** This creates a big support burden. Fix: store the trace.

Interview questions

Q1. Why both streaming and batch? Streaming gives users timely progress. Batch over the source of truth guarantees payouts that are correct and can be reproduced, even when data arrives late or gets corrected.

Q2. How do you handle a scheme changed mid-month? I version the scheme definitions. Each period is evaluated with the version that was effective for it. I also show users which version applies to them.

13.15 Instagram (photo sharing + feed)

Why?

> **In simple words:** The phone uploads the photo straight to cloud storage, background workers make smaller copies, and a CDN serves those copies from servers near the user. The feed is the same feed design as 13.4.

Functional requirements:

- Upload photos and short videos with captions.
- Follow users.
- Home feed.
- Likes and comments.
- Profile grid.
- Stories that expire after 24 hours.

Non-functional requirements:

- 500M DAU (daily active users).
- Feed loads in under 200 ms.
- Uploads are visible within seconds.
- Read-heavy: far more viewing than posting.
- Media delivered fast all over the world.

It tests the media pipeline, plus the feed design (13.4).

What is happening internally?

Estimates:

- 100M uploads a day ≈ 1,200 uploads per second.
- About 3 MB per upload, including resized versions → about 300 TB of media a day.
- Feed reads: about 115k QPS (queries per second).
- About 100 GB of metadata a day.

```
Client → POST /media/upload-url → media service returns a PRE-SIGNED URL
Client → PUT bytes directly to object storage            (app servers never proxy large files)
Storage event → Kafka media.uploaded → processing workers: virus scan, EXIF strip, resize
(thumb/320/640/1080), WebP/AVIF, video transcode
Client → POST /posts {mediaIds, caption} → post service → posts DB (sharded by post_id) → PostCreated
PostCreated → feed fan-out (hybrid, 13.4) · hashtag/search indexer · notifications
Reads: CDN serves all media; feed returns post IDs → hydrate from post cache → media URLs point to the CDN
```

In plain steps:

1. The client asks the media service for an upload URL. It gets a *pre-signed URL* — a temporary link that allows uploading one file straight to storage.
2. The client uploads the bytes directly to object storage. App servers never pass large files through themselves.
3. Storage fires an event to Kafka. Processing workers scan for viruses, strip EXIF data (hidden photo info like GPS location), resize to several sizes, convert to WebP/AVIF, and transcode videos.
4. The client creates the post with the media IDs and caption. The post service saves it (sharded by `post_id`) and publishes `PostCreated`.
5. `PostCreated` triggers feed fan-out (hybrid, 13.4), the hashtag/search indexer and notifications.
6. On reads, the CDN serves all media. The feed returns post IDs, which are filled in with details from the post cache (*hydration*). Media URLs point to the CDN.

Data:

- Follows: `follower_id, followee_id`, indexed in both directions.

- Posts: keyed by `post_id`, plus a `user_id → post_ids` timeline for profile grids.
- Likes: sharded counters (one count split across several keys), plus a `likes(post_id, user_id)` table.
- Comments: partitioned by `post_id`.
- Stories: in a TTL store (a store where each item has an expiry time and is deleted automatically).

How does it work?

Upload:

1. The client gets a pre-signed URL.
2. It uploads directly to storage.
3. Processing creates the size variants in the background (asynchronously).
4. Post creation refers to the media IDs.
5. Fan-out delivers the post to followers' feeds.

Read:

1. Get the feed post IDs.
2. Hydrate them (fill in details).
3. Return CDN URLs. The client picks the image size that fits its screen.

Example

1. A user posts a 4 MB photo.
2. The direct upload to Blob storage takes 1.5 s on 4G.
3. Processing creates 4 sizes in 2 s.
4. The post appears in followers' feeds within about 3 s.
5. 95% of image bytes are served from the CDN edge (servers close to the user).

When would I use it?

Any platform where users upload media. Also product catalogues with images, and Perfect Store photo audits. The pattern is the same: direct upload + background processing + CDN.

What can go wrong?

- **Passing uploads through app servers.** This chokes them.
- **Processing synchronously.** The post is blocked until processing ends.
- **Hot posts overwhelming counters.** Fix: shard the counters and cache them.
- **Privacy.** Private accounts must be checked during hydration.
- **Stories not expiring.** Fix: use a TTL.

Interview questions

Q1. How do you handle uploads at scale? Clients upload directly to object storage using pre-signed URLs. Storage events trigger background processing pipelines. Metadata goes into the database, and the media is delivered through a CDN.

Q2. How do you count likes on a viral post? I use sharded counters in Redis and add them up into the database from time to time. A separate likes table answers "did I like it?".

13.16 YouTube / Netflix (video upload and streaming)

Why?

In simple words: Chop each video into small pieces, convert every piece into many qualities in parallel, and let the player pick a quality for each piece based on how fast the internet is right now.

Functional requirements:

- Upload videos.
- Transcode them (convert them) into several resolutions.
- Stream with adaptive bitrate — the quality changes automatically with network speed.
- Search.
- View counts.

Non-functional requirements:

- Videos start fast and rarely pause to buffer (rebuffer), all over the world.
- Processing is allowed to take minutes.
- Huge storage, and huge egress (data sent out to viewers).

It tests large-file pipelines, parallel transcoding and CDN strategy.

What is happening internally?

Estimates:

- 500 hours of video uploaded every minute.
- 1 hour of 1080p ≈ 2–3 GB.
- All the versions (*renditions*) together ≈ 2× the original. So petabytes of new storage are needed every day.
- Playback egress is the biggest cost. So the CDN carries about 95%+ of the bytes.

```
Upload: resumable chunked upload (pre-signed URLs, e.g. 8 MB parts) → object storage (originals)
   → video.uploaded → transcoding DAG: split into segments → parallel encode per segment × resolution (240p…
4K) × codec (H.264/VP9/AV1)
   → package HLS/DASH (2–6 s segments + manifests) → thumbnails, captions → store renditions → metadata READY
→ notify
Playback: Video API (metadata, manifest URL, signed CDN token) → CDN edge serves segments
   player measures bandwidth and switches rendition per segment (adaptive bitrate)
Views: client beacons → Kafka → stream aggregation → approximate counts
```

In plain steps:

1. **Upload.** The video is uploaded in chunks (for example 8 MB parts) using pre-signed URLs. The upload is *resumable* — if the network drops, it continues from where it stopped. The original goes into object storage.
2. **Transcode.** A `video.uploaded` event starts a transcoding DAG (a directed acyclic graph — a set of tasks with a fixed order and no loops). The video is split into segments. Each segment is encoded in parallel, for each resolution (240p...4K) and each codec (H.264/VP9/AV1).
3. **Package.** The output is packaged as HLS/DASH (streaming formats): 2–6 s segments plus *manifests* (index files that list the segments and qualities). Thumbnails and captions are made. Renditions are stored, the metadata is marked READY, and the uploader is notified.

4. **Playback.** The Video API returns metadata, the manifest URL and a signed CDN token. The CDN edge serves the segments. The player measures bandwidth and switches quality per segment. This is *adaptive bitrate*.
5. **Views.** The client sends small "I watched" pings (*beacons*) to Kafka. A stream job adds them up into approximate counts.

How does it work?

Segment-level parallelism. One worker encoding a 2-hour video takes a long time. Split into segments, many workers share the work, so it finishes in minutes. If one segment fails, only that segment is retried.

Adaptive bitrate.

1. The manifest lists all available qualities (renditions).
2. Before each segment, the player checks the measured download speed and how much video it has buffered.
3. It picks the best quality it can safely play for that segment.

Think of a car's gears. On an open highway you shift up; in traffic you shift down. The player does the same with video quality, segment by segment.

Example

1. Someone uploads a 10-minute 1080p video (~400 MB).
2. The network drops at 60%. Because the upload is resumable, it continues from there.
3. The video is split into 100 six-second segments × 5 renditions = 500 encode tasks.
4. 200 workers process them, so it is ready in about 2 minutes.
5. A viewer on a weak network starts at 360p. After 10 seconds, the player climbs to 720p.

When would I use it?

Video platforms, training videos for field forces, and any pipeline for large media (audio, documents).

What can go wrong?

- **Non-resumable uploads.** These fail often on mobile networks.
- **Serial transcoding.** Encoding one piece after another is far too slow.
- **Origin overload on viral content.** Fix: pre-warm the CDN, and add an *origin shield* (a middle cache layer that protects the origin servers).
- **Exact view counts at huge scale.** These are too costly. Fix: count approximately, and deduplicate per session.
- **Storage cost.** Fix: move old videos to cold (cheaper) storage tiers, and keep fewer renditions for videos that are rarely watched.

Interview questions

Q1. How does adaptive bitrate streaming work? The video is encoded into several renditions (qualities). Each is split into short segments, listed in a manifest. The player chooses a rendition for each segment based on bandwidth and how full its buffer is.

Q2. How do you transcode quickly? Split the video into segments. Encode them in parallel across a fleet of workers, organised as a DAG. Then package and publish.

13.17 Twitter / X — what's specific beyond the generic feed

Why?

> **In simple words:** "Design Twitter" is the normal feed design from 13.4 plus a few Twitter-only parts — special tweet IDs, fast search, and trending topics. Show you can adapt a known design to them.

When interviewers say "design Twitter", they usually expect the feed (13.4) plus Twitter-specific parts: timelines, IDs, search and trending.

It tests whether you can adapt a design you know to the details of a specific product.

What is happening internally?

- **Timelines.** There are two kinds:
- The *home timeline* — what you see from people you follow. It is a fan-out cache of tweet IDs per user in Redis, about 800 entries.
- The *user timeline* — one user's own tweets, read from the tweet store.
- Celebrities' tweets are not fanned out. They are merged in when you read your timeline.
- **Tweet IDs.** Snowflake IDs (11.22). They are unique, roughly sorted by time, and generated without servers needing to coordinate.
- **Search.** A near-real-time *inverted index* (a map from each word to the tweets containing it), built on Lucene, in the Earlybird style. It is fed from the tweet stream. Recent data stays in memory; older data is in larger indexes.
- **Trending.** A stream job counts hashtags and terms over *sliding windows* (for example, the last 5 minutes, moving forward continuously), per region. It compares current counts with a baseline. What matters is *velocity* — how fast mentions are rising — not raw volume. A Count-Min Sketch (a small, approximate counting structure) finds the most frequent items (*heavy hitters*).
- **Counters.** Likes and retweets use sharded counters, saved to the database in the background.

Trending is like the noise level in a stadium. A crowd that is always loud is not news. A sudden roar is.

How does it work?

1. A tweet gets a Snowflake ID.
2. It is saved in the tweet store.
3. It is fanned out to followers' timelines (or skipped, for celebrities).
4. The search indexer adds it to the index.
5. The trending stream counts its terms.
6. Mentioned users get notifications.

Example

1. It is a cricket final. Mentions of "#INDvAUS" in Mumbai jump from 200/min to 40,000/min.
2. Within 30 seconds, the trending job's velocity score spikes.
3. The topic shows up in regional trends.
4. Celebrity tweets reach followers through the read-time merge, so there are no fan-out storms.

When would I use it?

Social products, real-time topic detection, and any messaging system with many writes and large fan-out.

What can go wrong?

- **Fan-out storms.** One tweet triggers millions of timeline writes.
- **Spam taking over trends.** Fix: rate limits and quality signals.
- **Search index lag.** New tweets take too long to become searchable.
- **Inconsistent counters.** Fix: accept eventual consistency (counts become correct after a short delay), and reconcile them later.

Interview questions

Q1. How would you detect trending topics? A stream processor keeps sliding-window counts per term and per region. Each term is scored by how fast it is rising compared with its normal baseline. Approximate structures like Count-Min Sketch keep this cheap.

13.18 Food Delivery (Swiggy / Zomato)

Why?

> **In simple words:** Fast search for restaurants, a careful and correct order flow, and smart timing so the delivery partner reaches the restaurant just as the food is ready — with each city handled separately.

Functional requirements:

- Browse nearby restaurants and menus.
- Place an order and pay.
- The restaurant accepts and prepares the food.
- Assign a delivery partner.
- Live tracking and ETA (estimated time of arrival).
- Ratings.

Non-functional requirements:

- Discovery (browsing) in under 300 ms.
- Ordering is strongly consistent — no lost orders and no duplicate orders.
- A partner is assigned within seconds.
- Lunch and dinner peaks of 5–10× normal traffic.
- City-level isolation — a problem in one city must not affect others.

It combines search, ordering, payments, dispatch and real-time tracking.

What is happening internally?

Estimates:

- 3M orders a day, with a dinner peak of about 300 orders per second.
- 300k active partners, each sending location every 5 s → about 60k location writes per second.

```
Discovery: app → listing/search service → Elasticsearch (geo + cuisine + availability) + Redis (menus,
open/closed)
Ordering:  app → cart → order service (Postgres, idempotency key) → payment (13.8) → OrderPlaced
           order state machine: PLACED → ACCEPTED → PREPARING → READY → PICKED_UP → DELIVERED (or CANCELLED +
refund)
Restaurant: tablet app ◄ order push (WebSocket/FCM) → accept/reject, prep-time estimate
Dispatch:  OrderAccepted → assignment service (per city): candidates from the geo index near the restaurant
(13.13)
           → score by ETA-to-restaurant vs food-ready time, partner load, batching opportunities → offer →
atomic reserve
Tracking:  partner app → location service → geo index + Kafka → customer app via WebSocket/SSE (throttled
updates)
ETA:       ML model (prep time + travel time + handover), re-estimated on each state change
```

In plain words:

- **Discovery.** The app calls the listing/search service. Elasticsearch handles location, cuisine and availability. Redis holds menus and open/closed status.
- **Ordering.** Cart → order service (Postgres, with an *idempotency key* — a unique request ID, so a repeated request does not create a second order) → payment (13.8) → `OrderPlaced` event.
- **Order states.** PLACED → ACCEPTED → PREPARING → READY → PICKED_UP → DELIVERED. Or CANCELLED, with a refund.
- **Restaurant.** Its tablet app receives the order by push (WebSocket or FCM). The restaurant accepts or rejects and gives a prep-time estimate.
- **Dispatch.** `OrderAccepted` goes to the city's assignment service. It finds nearby partners from the geo index (13.13). It scores them by: time to reach the restaurant vs food-ready time, partner's current load, and chances to batch orders. Then it makes an offer and reserves the partner atomically (all-or-nothing, so two orders can't grab the same partner).
- **Tracking.** The partner app sends locations to the location service. These go into the geo index and Kafka, and on to the customer app over WebSocket/SSE, with updates throttled (limited in rate).
- **ETA.** An ML model adds prep time + travel time + handover time. It re-estimates on every state change.

How does it work?

1. The order is placed.
2. The restaurant accepts, with a prep time of 15 minutes.
3. The dispatcher waits, then assigns a partner who will arrive just as the food is ready. - Assign too early, and the partner wastes time waiting. - Assign too late, and the food goes cold.
4. The partner gets the offer and accepts.
5. Pickup → live tracking → delivery.
6. The partner and the restaurant are paid.

Example

1. It is dinner peak in Gurugram: 4,000 orders in 30 minutes.
2. The city's dispatch shard sees two orders from the same restaurant going in the same direction. It batches both onto one partner.
3. A restaurant has not accepted an order for 5 minutes. The order is auto-cancelled, with an immediate refund.
4. A partner goes offline mid-route. The system reassigns the order.

When would I use it?

Hyperlocal delivery, quick commerce, field-service dispatch, and B2B order-to-delivery flows for distributors.

What can go wrong?

- **Duplicate orders from a double tap.** Fix: idempotency keys.
- **Stale menu stock.** Fix: invalidate the cache within seconds.
- **One partner assigned twice.** Fix: atomic reserve.
- **Peak overload.** Fix: buffer in Kafka, autoscale, and switch off non-essential features (graceful degradation).
- **No auto-cancel or reassignment paths.** Orders get stuck.

Interview questions

Q1. How do you decide when to assign a delivery partner? I use the predicted food-ready time and the partner's travel time, so the partner arrives just as the food is ready. I re-check this as the estimates change.

Q2. Why partition by city? Almost every operation stays inside one city. Partitioning by city limits the *blast radius* — how far a failure spreads — and lets each city scale on its own.

13.19 E-commerce Checkout and Flash Sales (Flipkart Big Billion Days)

Why?

> **In simple words:** Never sell more than you have. For a flash sale, stop the crowd before it reaches the database — use a waiting room and a fast Redis counter, so only about as many buyers as there are items ever get through.

Functional requirements:

- Browse and search.
- Cart.
- Checkout with inventory reservation (holding stock for a buyer for a while).
- Payment.
- Order creation.
- Flash sales: limited stock that goes on sale at a fixed time.

Non-functional requirements:

- Never oversell.
- Checkout must stay up at 10–50× normal traffic when a sale starts.
- Browsing may be eventually consistent (slightly out of date for a short time).
- Payments must be correct.

It tests inventory concurrency (many buyers fighting for the same stock), traffic shaping (controlling how fast traffic reaches the backend) and sagas (multi-step flows where each step can be undone if a later step fails).

What is happening internally?

Normal checkout:

```
Cart (Redis/DB) → checkout: recompute price and promotions server-side (cart prices are only hints)
  → inventory reservation:  UPDATE inventory SET reserved = reserved + q
                            WHERE sku = ? AND available - reserved >= q        (atomic; 1 row updated =
success)
  → order PENDING_PAYMENT + reservation with TTL (10–15 min)
  → payment (13.8) → success: commit reservation → stock decrement → order CONFIRMED → fulfilment events
                   → failure/timeout: release the reservation (a sweeper releases expired ones)
```

In plain steps:

1. The cart lives in Redis or the database.
2. At checkout, the server recalculates price and promotions itself. Prices in the cart are only hints — never trust them.
3. Inventory is reserved with one conditional `UPDATE`. It only succeeds if enough stock is free. If exactly 1 row is updated, the reservation worked. This is *atomic* — it either fully happens or doesn't happen at all.
4. The order is created as PENDING_PAYMENT, with a reservation that expires after 10–15 minutes (a TTL).
5. Payment runs (13.8). - **Success:** the reservation is committed, stock is reduced, the order becomes CONFIRMED, and fulfilment events go out. - **Failure or timeout:** the reservation is released. A background *sweeper* job also releases any reservations that have expired.

Flash sale (1M users hit "Buy" for 10,000 units at 12:00:00): 1. **Absorb the spike before the database.** Product pages are cached on the CDN. A *waiting room* (virtual queue) lets users in at a controlled rate. Per-user and per-device rate limits apply, and bots are detected and blocked. 2. **Pre-load stock into Redis.** An atomic `DECR` (or a Lua script that checks and decrements in one step) decides admission fast. If the stock is still ≥ 0 after your decrement, you get a time-limited purchase token. Everyone else sees "sold out" immediately. 3. **Create orders asynchronously.** Purchases from token holders are put in a queue. Workers create orders and reservations at a rate the database can handle. The database's conditional update is still the final guard against overselling. 4. **Reconcile.** Unpaid tokens expire, and their stock is returned (`INCR`). After the sale, Redis and database counts are compared and fixed. 5. **Avoid hot rows.** A *hot row* is one database row that everyone tries to lock at once. Split one SKU's stock into N buckets (rows or keys), and pick one at random. This spreads out the lock contention.

Think of the Tatkal ticket rush. If all the crowd ran onto the platform at once, nobody would get anything. Instead, a token counter hands out exactly as many tokens as there are seats. Only token holders go on to book.

How does it work?

- **Normal traffic** goes straight to the reservation step, guarded by the database.
- **Flash-sale traffic** is filtered first by the waiting room and then by Redis admission. So only about as many requests as there are units ever reach the order pipeline.

Example

1. 10,000 phones are on sale, and 1M users want them.
2. The waiting room admits 50k users a minute.
3. Redis hands out exactly 10,000 tokens in about 40 seconds.

4. 9,200 buyers pay within 10 minutes.
5. 800 tokens expire. Their units go back to the pool for the next users the waiting room lets in.
6. The database never sees more than about 2,000 order writes per second.

When would I use it?

Flash sales, ticket drops, limited-edition launches, coupon redemptions with a cap, and booking systems with scarce inventory.

What can go wrong?

- **Locking a single inventory row for a million requests.** Requests pile up behind the lock (*lock convoys*) and time out.
- **Trusting prices sent by the client.**
- **Charging the customer without creating an order.** Fix: the payment-success event triggers an idempotent retry of order creation. If the order still can't be created, refund.
- **Bots buying everything.**
- **Cache and database stock drifting apart.**

Interview questions

Q1. How do you prevent overselling? With an atomic conditional decrement or reservation at the source of truth — a database conditional update, or a Redis Lua script. I add reservation TTLs and reconciliation on top.

Q2. How do you survive the flash-sale spike? I shape the traffic before it reaches the backend, using a CDN, a waiting room and rate limits. Buyers are admitted through fast atomic counters in Redis. Orders are then created asynchronously, at a rate the system can sustain.

Q3. What if payment succeeds but order creation fails? The payment-success event drives an idempotent retry of order creation — retrying is safe and never creates two orders. If it still can't complete, the system refunds automatically.

13.20 More systems to practise

> **In simple words:** Use the same seven steps to practise these other common designs. For each one, the brackets show the key ideas the interviewer will look for.

- **Google Drive/Dropbox** — chunked upload, dedupe (store each identical chunk only once), sync, conflict resolution.
- **Web crawler** — the frontier (the queue of URLs still to visit), politeness (don't overload any one website), dedupe with Bloom filters (a small structure that quickly says "definitely not seen" or "maybe seen").
- **Distributed key-value store** — the Dynamo paper.
- **Proximity service (Yelp)** — find places near a location.
- **Stock exchange order matching** — an in-memory order book per symbol, and a sequencer (puts all orders in one fixed order).
- **Google Docs collaborative editing** — OT/CRDTs (techniques that merge edits from many people at the same time).
- **Hotel and airline booking.**

- **An API gateway.**
- **An LLM-powered assistant like NOVA** — RAG pipeline (fetch relevant documents and give them to the model), tool calling, guardrails, latency budget, caching, evaluation.

Part 5 — Interviews: Stories, Scripts and Revision

This last part turns what you know into how you perform on the day. It covers three things:

- the behavioral round,
- ready scripts so your mind doesn't go blank,
- a revision bank built for flashcards.

14. Behavioral Rounds and Your Resume

At SDE2 level, the behavioral round often decides your level. Different companies give it different names:

- Amazon: the "Bar Raiser" round.
- Google: "Googleyness and leadership".
- Everywhere: hiring-manager rounds.

If your technical rounds are strong but your stories are vague, you can be *down-levelled* — offered a lower level than you applied for.

14.1 STAR, done properly

STAR is a simple order for telling a work story: **S**ituation, **T**ask, **A**ction, **R**esult. The table shows how much time to spend on each part, and what each part must contain.

Part	Share of answer	What it must contain
Situation	10%	The background in 1–2 sentences: the product, the scale, and why it mattered
Task	10%	*Your* responsibility — the goal or the problem you had to solve
Action	60%	What **you** did — your decisions, trade-offs, options you rejected, and how you convinced others. Say "I", not "we".
Result	20%	The outcome in numbers, what you learned, and what you'd do differently next time

Notice that Action gets 60%. This is where the interviewer judges *you*. Most people spend too long on the Situation instead.

Keep each story to about 2 minutes. Then stop and let the interviewer ask questions.

Every story must survive three rounds of "why?" and "what exactly did you do?". If you can't answer the third follow-up, the story is not ready.

> **Rule:** Tell only real stories, with real numbers you can defend. Interviewers are trained to dig into details. Made-up or inflated details fall apart by the third follow-up question. It is the same lesson as the "500+ problems" line on the resume.

14.2 Your story bank — build it from your own work

During Week 14, write each story in a digital doc using the template below.

The prompts in the table point at work already on your resume. Fill in the true details from your memory and from your team's records — tickets, dashboards and PRs.

Template

```
Title:
Leadership principles it shows:
S (1-2 lines):
T (1 line):
```

```
A (5-7 bullets: decision, alternatives, trade-off, who you convinced, how you verified):
R (numbers + learning):
Likely follow-ups and my answers:
```

#	Story prompt (from your resume)	Questions it answers
1	**API latency ~30 s → ~10 s** — how you found the slow part (Grafana), what you changed, how you checked it worked	Deep dive / ownership / bias for action / "hardest technical problem"
2	**Duplicate reward postings under retries (FlickPoints)** — the root cause, the idempotent design (safe to repeat: running it twice has the same effect as once), and how you rolled it out without downtime	Dive deep / insist on high standards / a bug you fixed
3	**NOVA order pipeline reliability** — owning production troubleshooting for 190K+ retailer users	Ownership / customer obsession / handling pressure
4	**Incentive rules and reward-persistence layer** — turning business rules that kept changing into a design that is easy to maintain	Ambiguity / invent and simplify / working with product
5	**Perfect Store validation** — combining photo evidence, GPS logs and checklists	Complex requirements / trade-offs
6	**Zoho invoice automation (~40% less manual finance effort)** — spotting a pain point outside engineering	Customer obsession / frugality / initiative beyond your role
7	**Role-based ticketing system (~30% support efficiency)**	Delivering results / internal customers
8	**A production incident you owned** (pick a real one) — how it was detected, how you limited the damage, the root cause, and how you prevented it from happening again	Ownership / learning from failure
9	**A design disagreement** with a peer, lead or PM — how you used data, what was decided, and how you disagreed but then committed	Have backbone / earn trust / conflict
10	**Mentoring or influence through code reviews since becoming Senior**	Hire and develop / leadership without authority
11	**A failure or missed deadline** — what you'd do differently	Self-awareness / learn and be curious
12	**A decision made with incomplete data under time pressure**	Bias for action / judgement

Link every story to 2–3 principles. Then 10–12 stories are enough to answer any question they ask. One good story can be reused for many questions.

14.3 Amazon Leadership Principles (used, in spirit, by most product companies)

Amazon publishes these principles. Most product companies ask about the same qualities, even if they use different words. The right column tells you what the interviewer is listening for in your answer.

Principle	What they listen for
Customer Obsession	You started from the user's problem, and you measured the impact on users

Principle	What they listen for
Ownership	You acted beyond your own area; you thought long-term; you never say "that's not my job"
Invent and Simplify	You made a complex system or process simpler
Are Right, A Lot	Good judgement; you asked for different views; you changed your mind when data showed you were wrong
Learn and Be Curious	You learned something new fast (your move to Java is a great example)
Hire and Develop the Best	Mentoring others; raising the bar in code reviews
Insist on the Highest Standards	You refused to ship something broken; you raised the quality bar
Think Big	You proposed a bolder direction
Bias for Action	You took a calculated risk, using decisions that could be undone
Frugality	You did more with less
Earn Trust	You admitted mistakes, listened, and gave honest feedback
Dive Deep	You knew the details; you used metrics and logs; you found the root cause
Have Backbone; Disagree and Commit	You disagreed politely, with data; once the decision was made, you committed fully
Deliver Results	You shipped even when things went wrong
Strive to be Earth's Best Employer / Success and Scale Bring Broad Responsibility	Team health and wider impact (rarely the focus at SDE2)

14.4 The questions you will definitely get

"Tell me about yourself." (90 seconds. Practise it many times, but don't memorise it word for word — it should sound natural.)

> **Say this:** "I'm a backend engineer with three-plus years at FieldAssist. It's a multi-tenant retail-execution SaaS used by 700+ consumer brands across 32 countries. I own the incentives, loyalty, Perfect Store and NOVA backends. NOVA is our agentic ordering assistant, used by about 190,000 retailers. The work I'm proudest of is making our loyalty workflows idempotent, so retries never double-post rewards. I also cut a critical API from around 30 seconds to 10. Our production stack is .NET, but I've used Java as my main language for problem-solving and design. I recently built a Spring Boot ledger service to go deep on the Java ecosystem. Now I'm looking for an SDE2 role where Java is the core stack and the scale problems are bigger."

"Why are you switching from .NET to Java?"

> **Say this:** "The engineering problems I enjoy — distributed workflows, consistency, scale — are the same in both. The two runtimes are close cousins. Most product companies solving those problems at large scale run on the JVM. I already think and solve problems in Java, so I want my day job to match. I didn't want that to be just a claim, so I built [project] in Spring Boot with Kafka, Postgres and Testcontainers."

"Why do you want to leave?" Talk about what you are moving *towards*: bigger scale, deeper engineering, the Java ecosystem, and learning from strong peers. Never complain about your current employer.

"Why this company?" Give three reasons, all specific to them:

- one product reason,
- one engineering reason,
- one personal connection to their domain.

Read their engineering blog before the interview to find these.

Other common questions — prepare a story for each:

- your hardest bug;
- a time you disagreed with your manager;
- a time you missed a deadline;
- a time you went above and beyond;
- the most innovative thing you built;
- a time you received critical feedback;
- how you handle multiple priorities;
- a time you influenced people without having authority over them;
- a time you had to learn something quickly;
- a time you simplified something.

Questions to ask them. Always have 3 ready. For example:

- "What does a great first six months look like in this role?"
- "What's the hardest technical problem the team is facing this year?"
- "How do design decisions get made and reviewed here?"
- "How is on-call structured?"

14.5 Resume rules for SDE2 product-company applications

1. **One page.** Recruiters spend less than 10 seconds on the first look.
2. **Every bullet = action verb + what + how (the tech) + a measurable result.** For example: "Built queue-driven idempotent accrual workflows (, unique keys) eliminating duplicate reward postings across ~ events/day." Fill the brackets only with facts you can verify.
3. **Only claims you can defend for 10 minutes.** Remove "Solved 500+ DSA problems" now. Add a true count later — for example "Solved 250+ problems on LeetCode" once it is true. Your profile link will show the real number.
4. **Skills section: honest and in order.** Put first the language you would code the interview in (Java). Then the rest. Add Spring Boot only after the project exists.

5. **Projects.** Put the Java ledger project first under Projects. Include the stack, the concurrency guarantee, a load-test number, and a GitHub link with a good README.
6. **Fix inconsistencies.** Dates must not overlap (right now, Senior starts Apr 2026 while the previous role "ends" Jun 2026). Remove duplicated bullets (the FlickPoints bullet appears twice).
7. **Keywords recruiters search for:** Java, Spring Boot, microservices, Kafka, distributed systems, PostgreSQL, Redis, AWS/Azure, Kubernetes, system design.
8. Export as a PDF with the file name `Mayank_Shrivastava_SDE2_Java.pdf`.

14.6 Getting interviews: referrals and recruiters

A referral gets you an interview several times more often than a cold application.

Referral message (LinkedIn, to an engineer at the company)

```
Hi <Name>, I'm a backend engineer at FieldAssist (multi-tenant retail SaaS, 700+ brands) working on
loyalty/incentive platforms and an agentic ordering assistant used by ~190K retailers. I'm applying for
<Role, job ID> at <Company>. I've built idempotent event-driven workflows and recently a Spring Boot +
Kafka ledger service (<GitHub link>). Would you be open to referring me? Happy to share my resume, and
no worries if not. Thanks!
```

Send 3 messages a day, starting from Week 13. Track them in a sheet with these columns: company, role, contact, date, status, next step.

Talking to recruiters:

- Share your expected salary range only after you understand the level and the role.
- Ask about the interview process — how many rounds, and what each one focuses on. Then prepare for exactly that.
- If you need time to prepare, ask for 2–3 weeks before scheduling.

Offers and negotiation:

- Try to get your offers at the same time. Plan your interviews so the final rounds fall close together.
- Negotiate on total compensation, not just base pay. Use competing offers as your data.
- Be polite and specific.
- Notice periods in India are often 60–90 days, and this matters. Ask whether the company will buy out your notice period (pay your current employer so you can join sooner).

15. The Interview Playbook: Never Go Blank

Going blank in an interview is not a knowledge problem. It happens for two reasons:

- You have only *recognised* the material — you watched it or read it. You never practised *pulling it out of memory* under pressure.
- You don't have a ready next step to say out loud.

This chapter fixes both. It gives you training habits, and a script for every round. So there is always something useful to say.

15.1 Why people blank, and the fixes

Cause	What it feels like	Fix
Passive learning (videos, re-reading)	"I've seen this, but nothing comes"	Close the source. Solve or explain it again from memory. Use spaced repetition (review at growing gaps: 1 day, 3 days, 1 week)
Practising in silence	Your mind goes empty when someone watches	Say every practice solution out loud. Record yourself once a week
No fallback move	You panic, and the panic feeds itself	Memorise the scripts below. When stuck, just run the next step
Perfectionism	You wait for the best idea before you speak	Always start with brute force. Say it first, then improve it
Stress reaction in the body	Racing heart, tunnel vision	Breathe slowly twice (4 s in, 6 s out). Write the problem down. Say it back in your own words
Unfamiliar setting	Shared editor, no IDE autocomplete	Practise in plain editors (LeetCode, CoderPad-style, Google Docs)

15.2 Coding round script (45 minutes)

1. **Restate** the problem in your own words. Confirm the input types, input sizes and output format (2 min).
2. **Examples**: write one normal case and one edge case — empty, single item, duplicates, negatives, very large (2 min).
3. **Brute force**: say the simplest solution out loud with its time and space complexity, even if it is clearly slow (2 min).
4. **Pattern search**: go through your pattern triggers out loud (15.3). For example: "Sorted? → two pointers or binary search. Subarray? → sliding window or prefix sum..." Pick one, and explain why it fits (3–5 min).
5. **Agree** on the approach and its complexity with the interviewer before you start coding.
6. **Code** cleanly in Java. Write the method signature first, then helper methods. Use meaningful names. Explain your decisions as you go (15 min).
7. **Dry-run** your example line by line. Then dry-run the edge case (5 min).
8. **Complexity**: state the final time and space complexity, and suggest one possible improvement (2 min).

When stuck (say these out loud)

- "Let me try a smaller input by hand and look for a pattern."
- "What if the array were sorted? What if I process it from the right?"
- "What would I need to store so I don't recompute this?" (→ hashing or DP)
- "Can I binary-search on the answer? Is feasibility monotonic — once an answer works, do all bigger answers also work?"
- After about 5 minutes of being truly stuck, ask a focused question: "I'm considering X and Y. X fails because... Would it be reasonable to think about Y?" A targeted question like this earns a hint, and you don't look lost.

15.3 Coding round: what this guide adds to your DSA prep

Your DSA resource already covers patterns and problems, so this guide does not repeat them.

What this guide adds for the coding round is Java fluency. The goal is simple: the language should never slow you down.

Need in a coding round	Section
Pick the right collection and know its cost	2.1 (choosing), 2.4 HashMap, 2.6 TreeMap, 2.8 ArrayDeque, 2.9 PriorityQueue
Sort objects by several keys without bugs	1.13 Comparable/Comparator (never `a - b`)
Avoid overflow and boxing traps	1.2 Integer cache, 1.3 overflow
Build strings efficiently	1.5 StringBuilder
Design a data structure where every operation is O(1)	10.3 LRU cache (HashMap + doubly linked list)
Coding questions with a concurrency angle	4.14 bounded blocking queue, producer–consumer, odd–even printing

Keep a short personal list of the pattern triggers from your DSA resource. For example: "sorted → two pointers / binary search; subarray → window / prefix sum...". Say this list out loud in step 4 of the script above.

15.4 LLD round script (60–90 minutes)

1. **Clarify.** List 4–6 core use cases and write them down. Confirm what is out of scope. Ask about concurrency and persistence (saving data).
2. **Entities.** Nouns become classes, enums or records. Verbs become methods.
3. **Relationships.** Draw a quick class diagram.
4. **Interfaces first.** Name the design patterns at the places where behaviour can vary.
5. **Code** in this order: models → repositories (in-memory) → services → strategies → a `main` demo. Make the happy path work end to end before adding extras.
6. **Walk through** one scenario using the code.
7. **Close** with three points: - Thread safety — what is shared, and how it is protected. - Extensibility — where a new feature would plug in. - Testing approach.

Justify every pattern in one sentence. That is what interview-level LLD sounds like. For example: *"I used Strategy for pricing because pricing varies independently of the Ride entity, and I want to add new pricing schemes without modifying ride logic."* Practise saying a line like this for each pattern, in every problem you solve.

When stuck, say: "Let me write the main use case as a sequence of calls first." Write `main` using the API you *wish* you had. This often unlocks the design.

15.5 HLD round script (45–60 minutes)

1. **Functional requirements** — list 3–5. Also say clearly what is out of scope.
2. **Non-functional requirements** — users/DAU (daily active users), QPS (queries per second), read/write ratio, latency, availability, consistency and data size.
3. **Estimates** — calculate out loud: QPS, storage per year, cache size.
4. **API** — 3–6 endpoints.
5. **Data model and store choice** — give a one-line reason for the choice.
6. **High-level diagram** — then walk one read request and one write request through it.
7. **Offer deep dives**: "The interesting parts are A and B — which would you like to explore?"
8. **Close** with bottlenecks, failure modes, a plan for 10× scale, and monitoring.

When stuck, go back to the requirements. Say: "Given we need X at Y QPS, the bottleneck is Z, so...". Naming the bottleneck is always a valid next sentence.

15.6 Java/Spring theory round

Answer in three layers:

1. **Definition** — 1 sentence.
2. **How it works** — 2–3 sentences.
3. **When or why it matters in production** — 1 example from your own work.

Example: "`@Transactional` is declarative transaction management. Spring wraps the bean in a proxy that begins a transaction before the method and commits or rolls back after — rolling back on unchecked exceptions by default. In practice the classic bug is self-invocation, where an internal call skips the proxy; I'd move the method to another bean."

If you don't know the answer, say: "I haven't used that directly. My understanding is ... and I'd verify by ...". Never bluff with a confident wrong answer.

15.7 Mock interviews: how to run them and score them

From Week 13, do one mock every Sunday. From Week 15, do two per week.

Trade mocks with a peer — you interview them too. You learn as much from the interviewer's side.

Scoring rubric (1–4 each)

Dimension	1	4
Problem understanding	Jumped into code; solved the wrong problem	Clarified, gave examples, confirmed constraints

Dimension	1	4
Approach	No brute force, no pattern	Brute force → optimal, with reasons
Communication	Long silences	Continuous, structured narration
Code quality	Buggy, messy	Clean, modular, correct on the first run
Testing	None	Dry run + edge cases; found own bugs
Complexity	Wrong or missing	Correct for both time and space

Weekly mock mix from Week 15 (when you can find partners):

- 2 DSA
- 1 LLD
- 1 HLD
- 1 Java/backend theory round. A peer asks 15–20 questions from Chapter 16 and asks two follow-ups on each.

Don't wait until you feel "100% ready". Mocks are how you *get* ready.

Timed DSA simulation (45 min, Java only, no autocomplete):

1. 5 min — understand the problem.
2. 5 min — brute force.
3. 10 min — optimise.
4. 20 min — code.
5. 5 min — test and state the complexity.

Practise in a plain editor: LeetCode's editor with autocomplete off, or a blank doc.

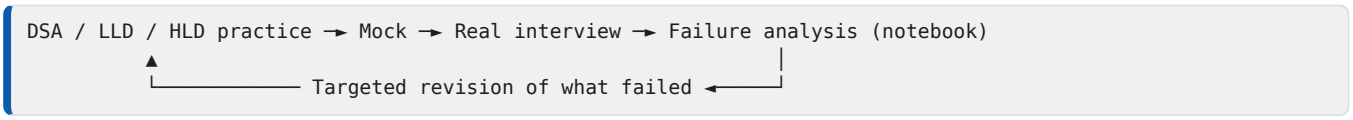
The failure notebook

After every mock **and every real interview**, write one entry for each mistake. Do it within the hour, while it is fresh.

```
Date / company / round:
Question:
Mistake:                    (what went wrong — be specific: "used HashMap, missed that order mattered")
Why I made it:              (didn't clarify / didn't recognise pattern / panicked / didn't know concept)
Correct approach:           (the right solution or answer, written fully)
Pattern or concept missed:  (e.g. "monotonic stack", "@Transactional self-invocation", "fan-out on read")
What I'll revise:           (chapter/section + a re-solve date 3 days out)
```

Review the notebook every Sunday. Look at the "why I made it" column. The causes that keep coming back are your real weak spots. Fix those causes — not just the individual questions.

Interview mode (Weeks 13–16): the loop



Don't follow the path *course → course → course → "now I'll start interviewing"*. That approach wastes months.

Instead, start applying from Week 13. Apply to practice companies first. Then your early real interviews feed this loop.

Re-attempt every failed mock problem 3 days later.

15.8 Online assessments (OA)

An OA usually has 2–3 problems in 60–90 minutes. It runs on HackerRank, Codility or HackerEarth, with hidden tests.

- Read all the problems first. Solve the easiest one first.
- When input comes through stdin, use fast I/O: `BufferedReader` + `StringTokenizer`, not `Scanner`.
- Watch for `int` overflow and edge cases. Hidden tests target exactly these.
- Partial credit counts. Submit a brute force that passes some tests first, then optimise.
- Keep a personal Java template file with imports, a fast reader and common helpers.

15.9 The day before and the day of

Day before:

- No new topics.
- Review your flashcards, your story bank and your pattern triggers.
- Prepare your setup: camera, mic, charged laptop, quiet room, water, paper and pen.
- Sleep 7–8 hours.

Day of:

- 1 hour before: solve 2 easy problems out loud to warm up.
- 10 minutes before: read your "tell me about yourself" answer and your 3 questions to ask.
- During the round: restate the problem, write things down, think out loud.
- Between rounds: don't replay the last round in your head. Reset with a short walk and some water.

After each round: within the hour, write down the questions and how you did. Patterns across companies tell you what to fix.

15.10 The weekly review (Sunday, 1 hour)

1. Tick off the week's tasks in the plan (Chapter 0).
2. Pick 3 random sections from the week. Re-explain each one without notes — all seven steps, out loud.
3. Answer 10 random Q&A cards out loud.
4. Update the failure notebook and schedule your re-attempts.
5. Check your gates (the pass/fail checkpoints in the plan). If a gate fails, repeat the weak part next week instead of moving on.
6. Plan next week's sections from the plan in Chapter 0.

16. Rapid Revision Bank

Short, flashcard-sized answers for your last two weeks of revision. Every row is also in the companion Anki CSV, so you can review on your phone. If an answer here feels unfamiliar, go back to the chapter section shown in brackets.

16.1 Core Java

Question	Answer
JDK vs JRE vs JVM?	JVM (Java Virtual Machine) runs bytecode; JRE (runtime) = JVM + libraries; JDK (dev kit) = JRE + dev tools (javac, jcmd...). [1.1]
Is Java pass-by-value?	Always. For objects, the reference is copied, not the object. [1.6]
Why is String immutable?	So it can be pooled and shared, kept secure, cache its hashCode, and be thread-safe. [1.4]
`==` vs `equals`?	`==` compares references (or primitive values); `equals` compares logical content. [1.12]
equals/hashCode contract?	Equal objects must have equal hash codes; equal hash codes do not mean equal objects. [1.12]
Integer cache range?	−128 to 127, when using `Integer.valueOf` (and autoboxing). [1.2]
StringBuilder vs StringBuffer?	Same methods; StringBuffer is synchronized, so slower. Use StringBuilder. [1.5]
Interface vs abstract class?	Interface: a contract for many unrelated classes, no state. Abstract class: shared state and code for related classes. [1.10]
Overloading vs overriding?	Overloading: chosen at compile time by parameters. Overriding: chosen at runtime by the actual object type. [1.9]
Can static methods be overridden?	No, they are hidden; the call depends on the reference type, not the object. [1.9]
`final` on variable/method/class?	Variable: assign once / method: can't override / class: can't subclass. [1.7]
Checked vs unchecked exceptions?	Checked: must declare or handle. Unchecked: RuntimeException subclasses, no need. [1.15]
Does `finally` always run?	Yes, except on System.exit or a JVM crash. Never return from finally. [1.15]
try-with-resources?	Closes AutoCloseable resources automatically, in reverse order; extra exceptions are kept as "suppressed". [1.15]
Type erasure consequences?	Generic types vanish at runtime, so: no primitives as type args, no `new T()`, no generic arrays, no instanceof on parameterised types. [1.16]
PECS?	Producer `extends`, consumer `super`: read from `? extends T`, write to `? super T`. [1.16]
How to make a class immutable?	Final class, private final fields, no setters, defensive copies of mutable fields. [1.14]

Question	Answer
Comparable vs Comparator?	Comparable: natural order inside the class. Comparator: outside order you can combine. [1.13]
Why not `a - b` in comparators?	It can overflow and give the wrong sign; use `Integer.compare`. [1.13]
Shallow vs deep copy?	Shallow shares nested objects; deep copies them too. Prefer copy constructors over clone. [1.14]

16.2 Collections

Question	Answer
HashMap default capacity and load factor?	16 and 0.75, so it resizes at 12 entries. [2.4]
When does a HashMap bucket treeify?	When a chain reaches 8 and the table has ≥ 64 slots; turns back into a list at 6. [2.4]
Why power-of-two capacity?	Index is a fast `(n-1) & hash`, and on resize each entry stays or moves by exactly n (one bit decides). [2.4]
HashMap worst-case lookup in Java 8+?	O(log n), because big buckets become trees. [2.4]
Null keys in HashMap / ConcurrentHashMap / TreeMap?	One allowed / none / none (it needs to compare keys). [2.4, 4.9]
ArrayList growth?	Grows 1.5× by copying into a bigger array (Arrays.copyOf). [2.2]
ArrayList vs LinkedList?	ArrayList: fast index access and CPU-cache friendly. LinkedList: O(1) insert only at a node you already hold. Pick ArrayList almost always. [2.2, 2.3]
What backs HashSet?	A HashMap whose values are a dummy object. [2.7]
TreeMap complexity and structure?	A red-black (self-balancing) tree, O(log n), with NavigableMap methods like floorKey/ceilingKey. [2.6]
LinkedHashMap access order?	`new LinkedHashMap<>(cap, 0.75f, true)` + override removeEldestEntry → an LRU (least recently used) cache. [2.5]
Stack/queue class to use?	ArrayDeque. [2.8]
Is PriorityQueue iteration sorted?	No; only the head is guaranteed to be the minimum. [2.9]
Fail-fast vs fail-safe iterators?	Fail-fast throws CME (ConcurrentModificationException) if the collection changes; fail-safe iterates a snapshot or weakly consistent view. [2.10]
Remove while iterating?	Use `iterator.remove()` or `removeIf`. [2.10]
`Arrays.asList` gotcha?	Fixed size (no add/remove); with an `int[]` you get `List<int[]>`. [2.11]
unmodifiableList vs List.copyOf?	A read-only view of the original vs an independent immutable copy. [2.11]
Sort algorithms used?	Primitives: dual-pivot quicksort; objects: TimSort (stable — equal items keep their order). [2.11]

16.3 Java 8–21

Question	Answer
Functional interface?	Exactly one abstract method; a lambda can be used for it. [3.1]
Function vs Supplier vs Consumer vs Predicate?	T→R, ()→T, T→void, T→boolean. [3.1]
Intermediate vs terminal ops?	Intermediate: lazy, return a stream. Terminal: actually run the pipeline. [3.2]
map vs flatMap?	One-to-one vs one-to-many, flattened into one stream. [3.2]
Reuse a stream?	No — you get IllegalStateException. [3.2]
Parallel streams risk?	They use the shared common ForkJoinPool, so blocking calls starve it; overhead is not worth it for small data. [3.4]
toMap duplicate keys?	IllegalStateException, unless you pass a merge function. [3.3]
groupingBy with counting?	`groupingBy(f, Collectors.counting())` → Map. [3.3]
orElse vs orElseGet?	orElse always builds the default (eager); orElseGet builds it only if needed (lazy). [3.5]
Where not to use Optional?	Fields, method parameters, collections. [3.5]
Record features?	Final class, immutable fields, auto-generated accessors/equals/hashCode/toString, compact constructor for checks. [3.7]
Sealed classes?	Limit which classes can extend it with `permits`; switches can then cover all cases. [3.8]
Switch expression `yield`?	Returns a value from a block `{ }` arm of a switch expression. [3.9]
Virtual threads?	Cheap threads scheduled by the JVM that step off the OS thread during blocking I/O; for I/O-bound concurrency. [4.12]
Pinning?	A virtual thread can't step off its OS thread (inside synchronized or native code) in Java 21. [4.12]
Instant vs LocalDateTime?	A point in time in UTC vs a date-time with no time zone. [3.6]

16.4 Concurrency

Question	Answer
start() vs run()?	start() runs the code on a new thread; run() runs it on the current thread. [4.1]
sleep vs wait?	sleep keeps its locks; wait releases the monitor and needs notify to wake. [4.5]
Why wait in a loop?	Threads can wake up for no reason (spurious wakeups), and the condition may have changed. [4.5]
volatile guarantees?	Visibility + ordering, but not atomicity (`count++` is still unsafe). [4.6]
happens-before examples?	Unlock→next lock, volatile write→read, Thread.start, Thread.join, submitting to an executor. [4.6]

Question	Answer
Why volatile in DCL singleton?	In DCL (double-checked locking), it stops other threads seeing a half-built object. [4.6]
ReentrantLock over synchronized?	tryLock with timeout, can be interrupted, fairness option, multiple Conditions. [4.7]
CAS and ABA?	CAS = atomic compare-and-swap. ABA (value changes A→B→A unnoticed) is fixed with stamped references. [4.8]
LongAdder vs AtomicLong?	LongAdder spreads updates over many cells → faster when many threads update; reading the sum is slower. [4.8]
ThreadPoolExecutor admission order?	Core threads first → then queue → then extra threads up to max → then reject. [4.3]
Problem with newFixedThreadPool?	Its queue has no limit → memory blows up under load. [4.3]
Rejection policies?	Abort, CallerRuns, Discard, DiscardOldest. [4.3]
Pool size I/O-bound?	cores × (1 + wait time / compute time). [4.3]
CHM in Java 8?	ConcurrentHashMap: CAS on empty bins, synchronized on the bin's head node, lock-free reads, threads help with resizing. [4.9]
containsKey-then-put race fix?	Use one atomic call: putIfAbsent / computeIfAbsent / merge / compute. [4.9]
CountDownLatch vs CyclicBarrier vs Semaphore?	Wait for N events (one-shot) / N threads wait for each other (reusable) / limit how many run at once. [4.10]
thenApply vs thenCompose vs thenCombine?	Map the value / flatMap another async call / zip two futures together. [4.11]
CompletableFuture default executor problem?	It uses the shared common pool — pass your own executor for I/O. [4.11]
Deadlock conditions and prevention?	Mutual exclusion, hold-and-wait, no preemption, circular wait; prevent with fixed lock order or tryLock. [4.13]
How to detect deadlock in prod?	Take a thread dump; look for "Found one Java-level deadlock". [4.13]
ThreadLocal risk?	Values leak into the next request on a pooled thread; always call remove(). [4.15]

16.5 JVM

Question	Answer
Heap generations?	Young (Eden, S0, S1 survivor spaces) and Old. [5.3]
PermGen replaced by?	Metaspace (in native memory) in Java 8. [5.2]
GC roots?	Starting points for GC: thread stacks, static fields, JNI (native code) references, active threads. [5.4]
Default GC since Java 9?	G1. [5.5]

Question	Answer
G1 vs ZGC?	G1: heap split into regions, aims for a pause-time target. ZGC: compacts while the app runs, pauses under 1 ms. [5.5]
StackOverflowError cause?	Recursion that goes too deep. [5.2]
Memory leak causes in Java?	Static caches, listeners never removed, ThreadLocals, inner classes holding the outer object, unclosed resources, keys changed after insert. [5.7]
Tools for high CPU?	`top -H` → convert thread ID to hex → find that nid in a thread dump; or JFR (Java Flight Recorder) / async-profiler. [5.8]
Tools for OOM?	For OutOfMemoryError: `-XX:+HeapDumpOnOutOfMemoryError` + MAT (Memory Analyzer) dominator tree; compare heap histograms over time. [5.9]
Class loader delegation?	A loader asks its parent first; order is bootstrap → platform → app. [5.1]
JIT tiers?	Interpreter → C1 (quick compile) → C2 (optimised compile), with deoptimisation if assumptions break. [5.10]
Container heap sizing?	`-XX:MaxRAMPercentage≈75` (heap = about 75% of the container's memory). [5.11]

16.6 Spring Boot and JPA

Question	Answer
@SpringBootApplication = ?	@Configuration + @EnableAutoConfiguration + @ComponentScan. [6.9]
Why constructor injection?	Final fields, clear dependencies, easy to test, circular dependencies fail at start-up. [6.2]
@Component vs @Bean?	@Component: a class found by scanning. @Bean: a factory method inside a @Configuration class. [6.3]
Bean scopes?	singleton, prototype, request, session, application. [6.5]
Prototype in singleton fix?	Get a fresh one each time via ObjectProvider / @Lookup / a scoped proxy. [6.5]
Bean lifecycle order?	Construct → inject → Aware callbacks → BPP (BeanPostProcessor) before → @PostConstruct → BPP after (proxy made here) → @PreDestroy. [6.6]
How does auto-configuration work?	Starters bring conditional config classes; they step back if you define the bean yourself. [6.9]
Self-invocation problem?	Calling a method from the same class skips the proxy → no @Transactional/@Async/@Cacheable. [6.7]
JDK proxy vs CGLIB?	JDK proxy: based on an interface. CGLIB: makes a subclass (Boot's default). [6.7]
Property precedence top?	Command-line args > environment variables > profile files > application.yml. [6.8]
Filter vs Interceptor?	Filter: Servlet level, before Spring. Interceptor: Spring MVC level, knows which handler runs. [7.5]
Global error handling?	@RestControllerAdvice + @ExceptionHandler → return a ProblemDetail. [7.4]

Question	Answer
N+1 fixes?	Fetch join, @EntityGraph, batch fetch size, DTO (plain data object) projection. [7.10]
LazyInitializationException?	A lazy field is read after the session closed; load it inside the transaction; turn off OSIV (Open Session In View). [7.11]
Dirty checking?	Changes to managed entities are UPDATEd automatically at flush. [7.9]
Default fetch types?	ToOne is EAGER, ToMany is LAZY — make ToOne LAZY too. [7.8]
@Transactional default rollback?	Only on unchecked exceptions and Errors, not checked exceptions. [7.12]
REQUIRED vs REQUIRES_NEW vs NESTED?	Join existing or create one / always a separate new one / a savepoint inside the current one. [7.13]
Isolation levels?	Read Uncommitted, Read Committed (Postgres default), Repeatable Read (MySQL default), Serializable. [7.14]
Optimistic vs pessimistic locking?	Optimistic: @Version column + retry on conflict. Pessimistic: SELECT FOR UPDATE locks the row. [7.15]
Prevent overdraw atomically?	`UPDATE ... SET bal = bal - p WHERE id=? AND bal >= p` and check the updated row count. [7.15]
IDENTITY vs SEQUENCE ids?	IDENTITY turns off Hibernate batch inserts; SEQUENCE allows them. [7.16]
ddl-auto in prod?	Set to validate, and change schema with Flyway/Liquibase migrations. [7.16]
JWT validation checks?	Signature, exp (expiry), iss (issuer), aud (audience). [8.2]
When CSRF needed?	For cookie-based browser sessions; not for APIs using bearer tokens. [8.1]
@WebMvcTest vs @SpringBootTest?	Only the web layer vs the full application context. [8.4]
Why Testcontainers?	Tests run against a real DB/Kafka in Docker, so behaviour matches production. [8.4]
Kafka consumer idempotency?	Safe to process twice: store processed event IDs (unique key) in the same transaction as the effect. [8.5]
Cache-aside write rule?	Update the DB first, then evict (delete) the cache key. [8.6]
Circuit breaker states?	Closed → Open → Half-open. [8.7]
Scheduled job once per cluster?	ShedLock / Quartz in cluster mode / a Kubernetes CronJob. [8.8]

16.7 LLD

Question	Answer
SOLID?	Single responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion. [9.3–9.7]
Composition over inheritance — why?	Less tight coupling, can change behaviour at runtime, no fragile base class. [9.1]

Question	Answer
Strategy vs State?	Strategy: the client picks the algorithm. State: the object moves itself between states. [9.17, 9.19]
Decorator vs Proxy vs Adapter?	Add behaviour / control access / convert one interface into another. [9.12–9.14]
Factory vs Builder?	Factory decides which class to create; Builder builds one complex object step by step. [9.9, 9.10]
Best singleton in Java?	An enum, or the holder-class idiom. [9.8]
Pattern for undo?	Command (+ Memento to save state). [9.20]
Pattern for rule trees?	Composite + Specification (+ Strategy for payouts). [10.12]
Parking lot concurrency?	Atomically take a spot from a thread-safe free-spot structure, one per spot type. [10.2]
LRU in O(1)?	HashMap + doubly linked list with dummy head/tail (sentinel) nodes. [10.3]
Seat double-booking prevention?	Lock per show / conditional DB update / Redis SET NX (set only if absent) with TTL. [10.10]
Splitwise simplify?	Work out net balances, then greedily match the biggest debtor and creditor using two heaps. [10.9]
LLD interview order?	Requirements → entities → relationships → interfaces/patterns → code → walk through a scenario → extensions. [9.25]

16.8 HLD

Question	Answer
1M requests/day ≈ ? QPS	About 12 QPS (queries per second), since a day ≈ 10^5 seconds. [11.2]
99.9% vs 99.99% downtime/year?	About 8.8 hours vs about 53 minutes. [11.3]
L4 vs L7 load balancer?	L4 routes on TCP/IP info only; L7 understands HTTP (paths, headers). [11.4]
Cache stampede fixes?	Let only one request reload (single-flight or lock), add random jitter to TTLs, serve stale data while refreshing. [11.5]
Cache penetration fix?	Cache "not found" results too; or use a Bloom filter to reject unknown keys. [11.5]
Composite index rule?	Works from the leftmost column; put equality columns before range columns. [12.5]
Quorum condition?	R + W > N (read replicas + write replicas > total replicas). [11.8]
Replication lag anomaly fix?	Read-your-writes: read from the leader right after a user writes. [11.8]
Good shard key?	Many distinct values, spreads load evenly, matches the main way you query. [11.9]
CAP in one line?	When the network splits, you must choose consistency or availability. [11.10]
PACELC adds?	Else (no split): choose low latency or consistency. [11.10]

Question	Answer
Consistent hashing benefit?	Adding/removing a node moves only ~1/N of keys; virtual nodes even out the load. [11.11]
Queue vs log?	Queue: message deleted once acknowledged, spreads work. Log: messages kept, can be replayed, ordered per partition. [11.12]
Kafka ordering?	Only within one partition; pick the message key with that in mind. [11.13]
Kafka max consumer parallelism?	The number of partitions. [11.13]
No-loss Kafka config?	acks=all, replication factor 3, min.insync.replicas=2, idempotent producer. [11.13]
Exactly-once across systems?	Deliver at-least-once + make consumers idempotent (safe to process twice). [11.12]
Dual-write fix?	Transactional outbox table + a relay or CDC (change data capture) to publish. [11.15]
Saga types?	Choreography (services react to events) vs orchestration (one coordinator directs). [11.15]
URL shortener ID generation?	Hand out pre-allocated ID ranges to servers + encode in base62. [13.1]
Feed fan-out choice?	Push for normal users, pull for celebrities (hybrid). [13.4]
Chat delivery transport?	WebSockets + a registry of who is connected where + push notification if offline. [13.5]
Leaderboard store?	Redis sorted sets (ZINCRBY, ZREVRANGE, ZREVRANK). [13.9]
Typeahead structure?	A trie with the top-K suggestions precomputed at each node; sharded and cached. [13.10]
Nearby drivers index?	Geohash/Redis GEO, a quadtree, or H3 hexagon cells. [13.13]
Payments correctness keys?	State machine, idempotency at every step, double-entry ledger, reconciliation. [13.8]
SLI vs SLO vs SLA?	What you measure / your internal target / the promise in the contract. [11.20]

16.9 Databases and SQL

Question	Answer
LEFT JOIN filter trap?	A condition on the right table in WHERE turns it into an INNER JOIN; put it in ON. [12.1]
NOT IN vs NOT EXISTS?	NOT IN returns nothing if the subquery has a NULL; use NOT EXISTS. [12.1]
Nested loop vs hash vs merge join?	Small outer + indexed inner / large unsorted with equality / both inputs already sorted. [12.1]
ROW_NUMBER vs RANK vs DENSE_RANK?	1,2,3,4 / 1,2,2,4 / 1,2,2,3. [12.2]
Top N per group?	DENSE_RANK() OVER (PARTITION BY g ORDER BY x DESC) in a subquery, then filter <= N. [12.2]

Question	Answer
Running total?	SUM(x) OVER (PARTITION BY k ORDER BY t ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW). [12.2]
Previous row value?	LAG(x) OVER (ORDER BY t). [12.2]
Gaps and islands trick?	date − ROW_NUMBER() stays the same for every row in a consecutive streak. [12.2]
Recursive CTE use?	Hierarchies: manager chains, category trees. [12.3]
SQL logical evaluation order?	FROM → WHERE → GROUP BY → HAVING → window → SELECT → DISTINCT → ORDER BY → LIMIT. [12.4]
B+ tree lookup cost?	About 3–4 page reads even for a billion rows; leaves are linked, so ranges are fast. [12.5]
Clustered vs non-clustered index?	Clustered: the rows themselves are stored in key order (only one per table). Non-clustered: a separate structure pointing to rows. [12.5]
Covering index?	Has every column the query needs (via INCLUDE) → no trip to the table. [12.5]
Why is my index not used?	Function/cast on the column, leading wildcard (`LIKE '%x'`), filter matches too many rows, wrong column order, outdated statistics. [12.5]
Reading EXPLAIN — first check?	Estimated vs actual row counts; full scans where a filter is selective; sorts spilling to disk. [12.6]
MVCC?	Multi-version concurrency control: writers create new row versions, readers see a snapshot, so reads and writes don't block each other. [12.7]
SQL Server READ COMMITTED difference?	It uses locks, unless READ_COMMITTED_SNAPSHOT is turned on. [12.7]
Avoid DB deadlocks?	Touch rows in the same order, keep transactions short, good indexes, single atomic statements, retry the victim. [12.7]
Write skew?	Two transactions read the same condition, then write different rows, breaking a rule together; needs SERIALIZABLE or FOR UPDATE. [12.7]
FOR UPDATE SKIP LOCKED use?	Many workers each pick different rows from a job or outbox table, without waiting on each other. [12.7]
Partitioning vs sharding?	Partitioning: split a table inside one DB (skip partitions, drop old data). Sharding: split across servers (scale writes). [12.8]
Row store vs column store?	Row store for OLTP (single-row reads/writes); column store for OLAP (big scans and aggregates). [12.9]
RabbitMQ vs Kafka?	RabbitMQ: routing, ack and TTL per message, work queues. Kafka: very high throughput, messages kept and replayable. [11.12]